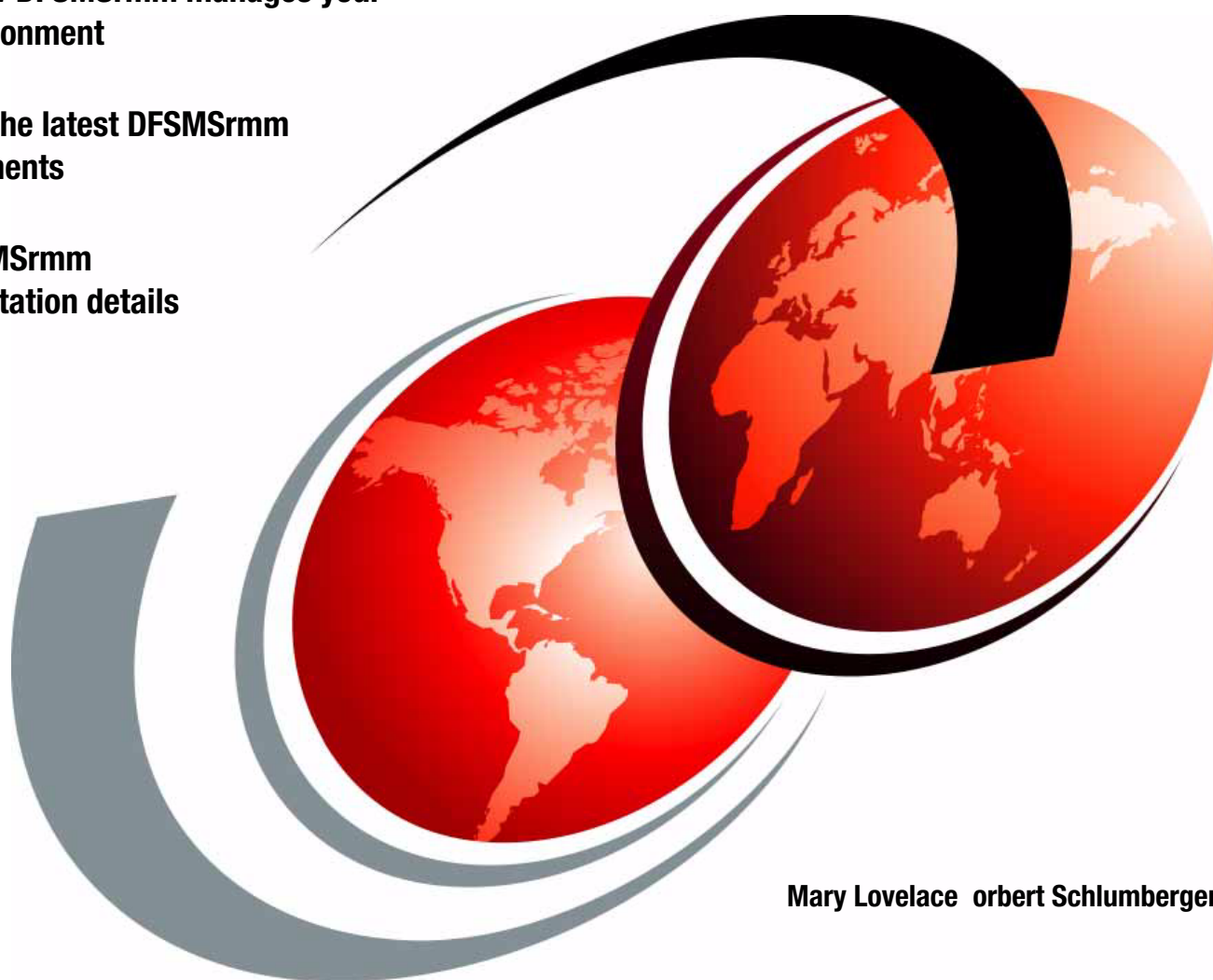


DFSMSrmm Primer

Learn how DFSMSrmm manages your tape environment

Discover the latest DFSMSrmm enhancements

Find DFSMSrmm implementation details



Mary Lovelace Robert Schlumberger

Redbooks



International Technical Support Organization

DFSMSrmm Primer

November 2012

Note: Before using this information and the product it supports, read the information in “Notices” on page xiii.

Fourth Edition (November 2012)

This edition applies to Version 1, Release 13 of z/OS DFSMSrmm (product number 5694-A01).

Contents

Notices	xiii
Trademarks	xiv
Preface	xv
The team who wrote that redbook	xv
Become a published author	xvi
Comments welcome	xvi
Chapter 1. DFSMSrmm enhancements	1
1.1 Changes introduced with z/OS V1R13 DFSMSrmm	2
1.1.1 Customize point-and-shoot fields in the DFSMSrmm dialog.	2
1.1.2 Specifying the retention method to be used for new tape data sets	2
1.1.3 IBM TS1140 support	2
1.1.4 Copying data set attributes in a tape copy application	2
1.1.5 Excluding specific data sets from VRSEL processing	3
1.1.6 Inventory management considerations for EXPROC and VRSEL	3
1.1.7 Volume Usage, Capacity, and Compression considerations	3
1.2 Changes introduced with z/OS V1R12 DFSMSrmm	4
1.2.1 Ease of Use and Flexibility Enhanced	4
1.2.2 Simplification	5
1.2.3 Storage Reporting	5
1.2.4 Availability	5
1.2.5 Scalability & Performance	5
1.3 Changes introduced with z/OS V1R11 DFSMSrmm	5
1.3.1 EDGINERS Tape Label Scan	5
1.3.2 Dynamic Installation Exits	6
1.3.3 Returning Volumes to the System-Managed Library	6
1.3.4 VRSEL GDG option	7
1.3.5 DFSMSrmm Report Generator	7
1.3.6 DFSMSrmm usability items	8
1.3.7 VRSEL(OLD) Parmlib Option	8
1.4 Changes introduced with z/OS V1R10 DFSMSrmm	8
1.4.1 Enable use of DFSMSdss COPY Service	8
1.4.2 SMF record changes	9
1.4.3 Audit Controls for Release Processing	9
1.4.4 XREPTTEXT tailoring via SYSIN	9
1.4.5 Volume Replacement Policies	10
1.4.6 Report Generator enhancements	10
1.4.7 System Managed Library partitioning	11
1.4.8 Common Information Model (CIM) Provider	11
1.5 Changes introduced with z/OS V1R9 DFSMSrmm	12
1.5.1 Task management support	12
1.5.2 Multitasking of utilities	12
1.5.3 Control data set (CDS) serialization	12
1.5.4 JCL data set names	12
1.5.5 Shared parmlib support	13
1.5.6 TSO subcommands	13
1.5.7 3592 Model E05 software support	13
1.5.8 REPORT17 of EDGRRPTE REXX Exec	14

1.5.9 Common Information Model (CIM) provider	14
1.6 Changes introduced with z/OS V1R8 DFSMSrmm	14
1.6.1 Support true e-mail address for the RMM NOTIFY function	14
1.6.2 Setting up DFSMSrmm Common Time Support	15
1.6.3 DFSMSrmm VRS policy management simplification	15
1.6.4 DFSMSrmm usability items	15
1.6.5 Tape data set authorization	15
1.7 Changes introduced with z/OS V1R7 DFSMSrmm	16
1.7.1 Issue DFSMSrmm TSO commands from the console	16
1.7.2 Improved security control over DFSMSrmm functions	16
1.7.3 Use of Large Format data sets	16
1.7.4 Enterprise enablement	17
1.8 Changes introduced with z/OS V1R6 DFSMSrmm	17
1.8.1 ISPF dialog enhancements	17
1.8.2 DFSMSrmm client/server support	17
1.8.3 Enhanced IBM 3592 support	17
1.8.4 Command reference summary in Guide and Reference	18
1.8.5 C++ API with XML Output Option	18
1.9 Changes introduced with z/OS V1R5 DFSMSrmm	18
1.9.1 Enhanced multilevel security	18
1.10 Changes introduced with z/OS V1R4 DFSMSrmm	18
1.10.1 Back up the DFSMSrmm control data set and journal at any time	18
1.10.2 Checking Authorization to Ignore Volumes	19
1.10.3 Duplicate Volume support	19
1.10.4 Enhanced multi-level security	19
1.11 Changes introduced with z/OS V1R3 DFSMSrmm	19
1.11.1 CDS utilization displays now available	19
1.11.2 Journal utilization and status displays now available	19
1.11.3 Movement date maintained for ON LOAN volumes	19
1.11.4 Enhanced EJECT support	20
1.11.5 Enhanced foreign tape authorization	20
1.11.6 3590-H support	20
1.11.7 Report Generator enhancements	20
1.11.8 Expiration Date field added to DFSMSrmm Data Set record	20
1.11.9 Multivolume dialog support	20
1.11.10 Special character support	21
1.11.11 Changed messages for improved diagnostics	21
1.11.12 HELP moved from SYS1.SEDGHLP1 to SYS1.HELP	22
Chapter 2. Introduction to DFSMSrmm	23
2.1 Environment	24
2.1.1 CDS backup and recovery	25
2.2 Reports	25
2.2.1 Security	27
2.3 How is DFSMSrmm structured?	27
2.4 What are the DFSMSrmm interfaces?	29
2.4.1 Software interfaces	29
2.4.2 User interfaces	33
2.4.3 Application programming interface	35
2.4.4 TCP/IP interface	35
2.5 Library and storage location management	36
2.5.1 Removable media library	37
2.5.2 Storage location	42

2.6	HOME location	43
2.7	Media types	44
2.7.1	Tape Cartridges used in the 3592 Tape Drives	45
2.7.2	IBM 3592 Tape Cartridge read and write formats	47
2.7.3	DFSMSrmm IBM Media Information	48
2.7.4	IBM 3592 Cartridge memory	48
2.8	-Volume management	49
2.8.1	Types of volumes	49
2.8.2	Volume status	50
2.8.3	Validation	52
2.8.4	Support for duplicate volumes	54
2.9	Policies for retention and movement	57
2.9.1	Data set retention	57
2.9.2	Volume retention	60
2.9.3	Movement	60
2.10	SMS ACS support	61
2.11	DFSMSrmm automated tape library support	62
2.11.1	Cartridge entry processing	63
2.11.2	Ejecting volumes from system-managed libraries	64
2.11.3	Volume-Not-In-Library processing	64
2.11.4	Verifying the databases	64
2.11.5	VTs import/export processing	65
2.11.6	Defining stacked volumes to DFSMSrmm	65
2.12	Catalog synchronization	66
2.13	Implementing the RMMplex	67
2.13.1	RMMplex	67
Chapter 3.	Preparing the environment	69
3.1	DFSMSrmm implementation	70
3.1.1	z/OS DFSMSrmm Customization Wizard	70
3.1.2	Updating the installation-wide exits	71
3.1.3	Updating and validating the PARMLIB members	72
3.1.4	Enabling ISPF Data Set List (DSLST) Support	116
3.1.5	Rexx Variable Constraint Relief	117
3.1.6	Creating a starting procedure	119
3.1.7	Defining an alias for high-level qualifier RMM	122
3.1.8	Protecting DFSMSrmm resources	122
3.1.9	Defining RACF resources and groups for DFSMSrmm	123
3.1.10	Creating the DFSMSrmm CDS	124
3.1.11	Creating the DFSMSrmm CDS as extended format	127
3.1.12	Creating the DFSMSrmm journal	129
3.1.13	Restarting MVS with DFSMSrmm implemented	131
3.1.14	Starting and restarting DFSMSrmm	131
3.1.15	Stopping DFSMSrmm	134
3.1.16	Setting up DFSMSrmm utilities	136
3.1.17	Running the installation verification program	136
3.2	Tailoring the DFSMSrmm ISPF dialog	136
3.2.1	Adding DFSMSrmm to ISPF panels	137
3.2.2	Modifying the DFSMSrmm panel	138
3.2.3	Other changes	139
3.3	Making TSO HELP information available to users	139
3.4	Verifying your DFSMSrmm implementation	140
3.4.1	Displaying the PARMLIB options and control information	140

3.4.2	Adding owner information	143
3.4.3	Adding racks to DFSMSrmm.	144
3.4.4	Adding volumes to DFSMSrmm	145
3.4.5	Adding bins to DFSMSrmm.	147
3.4.6	Adding a DSNAME VRS to DFSMSrmm	147
3.4.7	Creating tape data sets.	148
3.4.8	Allocating inventory management data sets	149
3.4.9	Running inventory management	150
3.4.10	Confirming movements	151
3.4.11	Creating reports	152
3.4.12	Running EDGINERS to initialize volumes automatically.	153
3.4.13	Restoring the CDS	154
3.4.14	Verifying the CDS	157
3.4.15	Testing implemented exits	157
3.4.16	Testing application use of tape	160
3.5	Education	161
3.6	New operator procedures	162
Chapter 4.	DFSMSrmm retention methods	163
4.1	Overview	164
4.2	Specifying a retention method.	164
4.2.1	Expiration date	164
4.2.2	Retention date.	165
4.2.3	Using the EDGRMMnn parmlib option RETENTIONMETHOD.	165
4.2.4	EDG_EXIT100 retention method support.	166
4.2.5	Subcommands for RETENTIONMETHOD parameters	170
4.2.6	Syntax format of the RETENTIONMETHOD operand	170
4.2.7	Using the SEARCHVOLUME subcommand.	171
Chapter 5.	Exclude Data Sets from VRSEL processing	179
5.1	Overview	180
5.2	Subcommands for VRSELEXCLUDE parameters	180
5.2.1	Syntax format of the VRSELEXCLUDE operand	180
5.2.2	Using EDG_EXIT100 to excluding Data Sets from VRSEL support.	181
5.2.3	Using the SEARCHDATASET subcommand	184
5.2.4	Inventory Management VRSEL/EXPROC Processing	189
Chapter 6.	Setting up an RMMplex	195
6.1	RMMplex.	196
6.1.1	Maintenance levels	197
6.2	Client/server terminology	197
6.2.1	DFSMSrmm server	198
6.2.2	SERVERNAME.	198
6.2.3	PORT	198
6.2.4	SERVERTASKS	198
6.2.5	DFSMSrmm client.	198
6.2.6	LOCALTASKS	198
6.3	Implementing client and server systems.	199
6.3.1	Checking your TCP/IP configuration.	199
6.3.2	EDGRMMxx DFSMSrmm options.	204
6.3.3	Updating EDGRMMxx for a client.	206
6.3.4	Updating EDGRMMxx for a server	207
6.4	Updating the DFSMSrmm CDSID (optional).	207
6.4.1	Sample EDGUTIL job control	207

6.5 Updating your firewall (optional)	208
6.6 Starting DFSMSrmm in an RMMplex	208
6.6.1 Starting and restarting DFSMSrmm on your client and server	208
6.7 Managing volumes in an RMMplex	210
6.7.1 Catalogs in an RMMplex	210
6.7.2 RACF considerations in an RMMplex	211
6.7.3 System-managed libraries in an RMMplex	212
6.7.4 HOUSEKEEP processing in an RMMplex	212
6.8 Operator commands	213
Chapter 7. Tailoring your DFSMSrmm environment	215
7.1 IBM Tivoli Workload Scheduler	216
7.1.1 IBM Tivoli Workload Scheduler overview	216
7.1.2 Scheduling your inventory management tasks	217
7.1.3 Preparation	219
7.1.4 CDS verify	219
7.1.5 CDS backup	219
7.1.6 Inventory management	219
7.1.7 Erasing and labeling volumes	221
7.1.8 Scratch processing	221
7.1.9 Scratch reporting	221
7.1.10 Ejecting volumes from a system-managed tape library	222
7.1.11 Producing reports	222
7.1.12 Move confirmation	223
7.1.13 Additional considerations	223
7.2 Setting up DFSMSrmm Common Time Support	224
7.2.1 How DFSMSrmm using the Date and Time	225
7.2.2 Date and Time in a DFSMSrmm Client Server Environment	225
7.2.3 Enable Common Time Support	226
7.2.4 Potential Problems Using Local Time	229
7.3 Dynamic Installation Exits	229
7.3.1 DFSMSrmm Installation Exit calls	230
7.3.2 Managing Exit Modules	231
7.3.3 Setting Up the EDG_EXIT100 Routine Environment	234
7.3.4 Controlling the Exit Routine Through the z/OS Dynamic Exits Facility	236
7.3.5 Deleting the EDG_EXIT100 Exit Routines	237
7.3.6 Writing an Exit Routine for the EDG_EXIT100 Exit	238
7.4 Activating RACF TAPEVOL and TAPEDSN	238
7.4.1 Before you can start	239
7.4.2 DFSMSrmm RACF tape security support	239
7.4.3 DFSMSrmm automatic tape security support	239
7.4.4 DFSMSrmm OPTION command operand TPRACF	240
7.4.5 DFSMSrmm VLPOOL command operand RACF	240
7.4.6 Running DFSMSHsm and DFSMSrmm	241
7.4.7 Defining RACF profiles	241
7.4.8 Final RACF activities	242
7.5 Maintaining your control data set	243
7.5.1 Using EDGHSKP	243
7.5.2 Using EDGBKUP	246
7.5.3 EDGUTIL	253
7.6 Initializing and erasing volumes	254
7.7 Disposition processing	259
7.8 Software products and foreign tapes	260

7.8.1	Defining software products	261
7.8.2	Foreign tapes	261
7.8.3	Bypass DFSMSrmm and SMS processing	263
7.9	DFSMSrmm volume pools	263
7.9.1	Defining pools	264
7.9.2	Scratch pooling	266
7.10	Deleting non-existent volumes	266
7.11	Repairing actions	267
7.11.1	Using the VERIFY parameter	268
7.11.2	Using the MEND parameter	270
7.11.3	Using EDGSPLCS	275
7.11.4	DFSMSrmm CDS and DFSMSHsm consistency checking	283
7.11.5	Using DFSMSrmm with BTLS.	284
7.11.6	Using DFSMSrmm with non-IBM libraries.	286
Chapter 8.	System Managed Library partitioning with z/OS V1.10.	291
8.1	System Managed Library partitioning	292
8.2	The sequence of processing	292
8.3	Reasons to partitioning a SMS managed library.	293
8.3.1	Partitioned library with shared CDS and TCDB	293
8.3.2	Partitioned library with shared TCDB and different CDSs.	293
8.3.3	Partitioned library with unshared CDS and TCDB	294
8.3.4	Partitioned library with shared CDS and unshared TCDB	295
8.4	Partitioning Support.	296
8.5	Using PRTITION and OPENRULE commands.	296
8.5.1	Defining PRTITION commands.	297
8.5.2	Defining OPENRULE commands	303
8.5.3	Testing various OPENRULE settings	308
8.6	Converting REJECT Commands	323
8.6.1	Examples of Converting REJECT Commands	323
8.7	Conversion from REJECT to PRTITION and OPENRULE	325
8.7.1	Partitioned library with shared CDS and TCDB	327
8.7.2	Partitioned library with shared TCDB and different CDSs.	328
8.7.3	Partitioned library with unshared CDS and TCDB	329
8.7.4	Partitioned library with shared CDS and unshared TCDB	330
Chapter 9.	Catalog synchronization	333
9.1	Overview of catalog synchronization.	334
9.1.1	DFSMSrmm catalog processing	335
9.1.2	Reasons to re-synchronize catalog synchronization.	335
9.2	Preparing DFSMSrmm catalog synchronization	336
9.2.1	Message data set considerations	337
9.2.2	Activity log data set considerations (optional).	337
9.2.3	Journal considerations	338
9.2.4	Consideration for create system ID.	340
9.2.5	Disabling catalog synchronization.	342
9.3	Synchronizing with a shared catalog environment	343
9.3.1	Setting a create system ID for all data sets have not one.	345
9.3.2	Clearing the catalog synchronization	346
9.3.3	Updating the EDGRMMxx DFSMSrmm options on all systems	346
9.3.4	Synchronizing your catalogs	347
9.3.5	Checking catalog synchronization.	348
9.3.6	Enabling the journal data set (optional)	349

9.3.7	Catalog re-synchronization with a shared catalog environment	350
9.4	Synchronizing when catalogs are not shared	350
9.4.1	Setting the create system ID if catalogs are not shared	353
9.4.2	Clearing the catalog synchronization	355
9.4.3	Updating the EDGRMMxx DFSMSrmm options	355
9.4.4	Synchronizing your catalogs	357
9.4.5	Enabling the journal data set (optional)	360
9.4.6	Housekeeping considerations	360
9.4.7	Catalog re-synchronization with a shared catalog environment	361
9.5	IPV6 Supports ne	362
Chapter 10. Enabling ISPF Data Set List (DSLST) Support.		363
10.1	Implementation steps	364
10.2	Use the ISPF Configuration Utility	364
10.3	Using the ISPF Data Set List Utility Support	370
10.4	Move the ISPCFIGU module to the SISPLPA library	374
Chapter 11. Policy management.		375
11.1	VRS related PARMLIB options	376
11.1.1	CATRETPD	377
11.1.2	GDG option	378
11.1.3	MOVEBY	380
11.1.4	RETAINBY	381
11.1.5	REUSEBIN	383
11.1.6	VRSCCHANGE	383
11.1.7	VRJOBNAME	384
11.1.8	VRSMIN	385
11.1.9	VRSEL	385
11.1.10	VRSDROP	386
11.1.11	VRRETAIN	387
11.2	VRS types	389
11.2.1	Data set VRS	389
11.2.2	Volume VRS	394
11.2.3	Name VRS	394
11.3	Specifying the VRS parameters	395
11.3.1	DSNAME and NAME retention types	395
11.3.2	Special NAME retention type	397
11.3.3	Volume retention types	397
11.3.4	Retention limits	397
11.3.5	Movement policies	398
11.3.6	Release options	398
11.4	Chaining VRSs	399
11.4.1	VRS chain and subchain	400
11.5	VRSEL GDG option	401
11.5.1	Generation data set and DSN vital record specification creation	406
11.5.2	CYCLEBY GENERATION and DUPLICATE BUMP	408
11.5.3	CYCLEBY CRDATE and DUPLICATE BUMP	409
11.5.4	CYCLEBY GENERATION and DUPLICATE BUMP with generation wrap	411
11.5.5	CYCLEBY CRDATE and DUPLICATE BUMP with generation wrap	413
11.5.6	CYCLEBY GENERATION and DUPLICATE DROP	414
11.5.7	CYCLEBY CRDATE and DUPLICATE DROP	415
11.5.8	CYCLEBY GENERATION and DUPLICATE KEEP	416
11.5.9	CYCLEBY CREATION and DUPLICATE KEEP	417

11.5.10	CYCLEBY GENERATION and DUPLICATE COUNT	418
11.5.11	CYCLEBY CREATION and DUPLICATE COUNT	419
11.6	Managing DFSMSHsm tapes	420
11.7	DFSMS ACS support	421
11.8	Modifying VRS	421
11.9	Deleting VRS	422
11.10	DFSMSrmm VRS policy management simplification	422
11.10.1	Separation of Data Set Name Mask from the Policy	423
11.10.2	Release Options Applied if VRS Matched	424
11.10.3	Special ABEND, DELETED and OPEN via DSNAME match	426
11.10.4	Find unused VRSs	427
11.10.5	VRS last reference date	428
11.10.6	Incomplete VRS chains - dummy VRS '*broken*'	431
11.10.7	Toleration and removal of really old functions	432
11.10.8	Conversion to DFSMSrmm from other tape management systems	434
11.11	Trial run	436
11.12	VRS examples	437
11.12.1	ADDVRS operand defaults	437
11.12.2	Adding a single VRS	437
11.12.3	VRS chain with one NEXTVRS operand	438
11.12.4	VRS chain with two ANDVRS operands	441
11.12.5	VRS chain with two subchains	442
11.13	Complex VRS chain with three subchains	443
11.14	VRS hints and tips	444
Chapter 12.	DFSMSrmm ACS support	447
12.1	DFSMSrmm SMS ACS support	448
12.1.1	How ACS support works	449
12.1.2	ACS support for non-system managed volumes	449
12.1.3	Non-system managed tape libraries	451
12.1.4	SMS read-only variables	452
12.1.5	Implementing SMS ACS processing	456
12.1.6	Migration considerations	472
12.2	Partitioning a SMS library	475
12.3	Using DFSMSrmm with virtual tape solutions	479
12.3.1	VTs support for stacked volumes	480
12.3.2	Stacked volume support for non-VTS	484
Chapter 13.	Tape security with z/OS V1.8	485
13.1	Tape data set authorization	486
13.1.1	Recommendations for tape security	486
13.1.2	Overview of the TAPEVOL and TAPEDSN processing	487
13.1.3	How the z/OS DFSMS V1.8 tape data set authority checking works	489
13.2	Implementation	493
13.2.1	Check all high level qualifiers on tape	494
13.2.2	Update DEVSUPnn parmlib member	494
13.3	Removing TAPEVOL and TAPEDSN processing	497
13.3.1	Check and modify your RACF Settings	497
13.3.2	Check and modify your DFSMSHsm settings	501
13.3.3	Cleanup your TAPEVOL profiles using DFSMSrmm settings	502
13.3.4	Cleanup your TAPEVOL profiles using commands	504
13.4	Error messages	506
13.5	Testing various security settings	509

13.5.1	Test case 1	510
13.5.2	Test case 2	513
13.5.3	Test case 3	516
13.5.4	Test case 4	518
13.5.5	Test case 5	521
13.5.6	Test case 6	524
13.5.7	Test case 7	527
13.5.8	Test case 8	530
13.5.9	Test case 9	533
13.5.10	Test case 10	535
13.5.11	Test case 11	538
13.5.12	Test case 12	541
13.5.13	Test case 13	544
13.5.14	Test case 14	546
13.5.15	Test case 15	549
13.5.16	Test case 16	552
13.5.17	Test case 17	556
13.5.18	Test case 18	559
13.5.19	Test case 19	563
13.5.20	Test case 20	567
13.5.21	Test case 21	568
13.5.22	Test case 22	570
13.5.23	Test case 23	574
13.5.24	Test case 24	577
13.5.25	Test case 25	580
13.5.26	Test case 26	582
13.5.27	Test case 27	585
13.5.28	Test case 28	588
13.5.29	Test case 29	591
13.5.30	Test case 30	593
13.5.31	Test case 31	596
13.5.32	Test case 32	599
13.5.33	Test case 33	602
13.5.34	Test case 34	604
13.5.35	Test case 35	606
13.5.36	Test case 36	609
Chapter 14. Report Generator		613
14.1	Report Generator Overview	614
14.2	Setting up the Report Generator for Your Installation	614
14.3	Create a Report	620
Chapter 15. Splitting and merging your DFSMSrmm CDS		633
15.1	When to merge or split	634
15.1.1	During conversion	634
15.1.2	Changing the number of systems	634
15.1.3	Considerations	634
15.1.4	Control record	635
15.1.5	Action record	635
15.1.6	Data set record	635
15.1.7	VRS record	636
15.1.8	Owner record	636
15.1.9	Product record	637

15.1.10 Rack numbers record	638
15.1.11 Bin numbers record.	639
15.1.12 Volume record.	640
15.2 PARMLIB options	641
15.2.1 LOCDEF	641
15.2.2 VLPOOL	641
15.2.3 REJECT	641
15.2.4 OPENRULE	642
15.2.5 PRITITION	642
15.3 Running IDCAMS REPRO	642
15.4 Merging the CDS.	643
15.5 Splitting the CDS.	646
15.5.1 Converting to an existing CDS	652
15.6 Merging DFSMSRmm environments	652
15.6.1 One or more RMMplexes	653
15.6.2 Considerations for merging RMMplexes.	653
15.6.3 CDS record types	655
15.6.4 Step 3: Review the DFSMSRmm PARMLIB options	661
15.6.5 Step 4: Review the DFSMSRmm started task JCL	663
15.6.6 Methodology for merging RMMplexes	664
15.6.7 Tools and documentation	667
Appendix A. Security topics	673
RACF implementation	674
Assigning a RACF user ID for DFSMSRmm	674
Identifying DFSMSRmm to RACF	674
Defining DFSMSRmm resources to RACF.	675
Defining RACF groups for DFSMSRmm users	677
CA-Top Secret implementation	693
Defining user groups.	693
Protecting DFSMSRmm data sets	694
Defining DFSMSRmm resources	695
ACF2 implementation.	696
Defining data set rules	696
Defining DFSMSRmm resources	698
Appendix B. The DFSMSRmm competitive matrix and migration tools	701
Tape management systems competitive matrix.	702
Migration tools	707
Glossary	709
Related publications	727
IBM Redbooks	727
Other publications	727
Online resources	727
How to get Redbooks.	728
Help from IBM	728
Index	729

Notices

This information was developed for products and services offered in the U.S.A.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not give you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing, IBM Corporation, North Castle Drive, Armonk, NY 10504-1785 U.S.A.

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs.

Trademarks

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corporation in the United States, other countries, or both. These and other IBM trademarked terms are marked on their first occurrence in this information with the appropriate symbol (® or ™), indicating US registered or common law trademarks owned by IBM at the time this information was published. Such trademarks may also be registered or common law trademarks in other countries. A current list of IBM trademarks is available on the Web at <http://www.ibm.com/legal/copytrade.shtml>

The following terms are trademarks of the International Business Machines Corporation in the United States, other countries, or both:

Redbooks (logo) ®

The following terms are trademarks of other companies:

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.

Microsoft, Windows, Windows NT, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

Intel, Intel logo, Intel Inside, Intel Inside logo, Intel Centrino, Intel Centrino logo, Celeron, Intel Xeon, Intel SpeedStep, Itanium, and Pentium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

UNIX is a registered trademark of The Open Group in the United States and other countries.

Linux is a trademark of Linus Torvalds in the United States, other countries, or both.

Other company, product, or service names may be trademarks or service marks of others.

Preface

DFSMSrmm™ from IBM® is the full function tape management system available in OS/390® and z/OS®. With DFSMSrmm, you can manage all types of tape media at the shelf, volume, and data set level, simplifying the tasks of your tape librarian.

Are you a new DFSMSrmm user? Then this IBM Redbook introduces you to the DFSMSrmm basic concepts and functions. You learn how to manage your tape environment by implementing the DFSMSrmm management policies.

Are you already using DFSMSrmm? In that case, this redbook provides the most up-to-date information about the new functions and enhancements introduced with the latest release of DFSMSrmm. You will find useful information for implementing these new functions and getting more benefits from DFSMSrmm.

Do you want to test DFSMSrmm functions? If you are using another tape management system and are thinking about converting to DFSMSrmm, you may start DFSMSrmm and run it in parallel to your current system for testing purposes.

This book is intended to be a starting point for new professionals and a handbook for using the basic DFSMSrmm functions.

The team who wrote that redbook

This redbook was produced by a team of specialists from around the world working at the International Technical Support Organization (ITSO), San Jose Center.

Mary Lovelace is a Consulting IT Specialist at the International Technical Support Organization. She has more than 20 years of experience with IBM in large systems, storage, and storage networking product education, system engineering and consultancy, and systems support. She has written many Redbooks publications about z/OS storage products, IBM Tivoli® Storage Productivity Center, Tivoli Storage Manager, and Scale Out NAS.

Norbert Schlumberger is an IT Architect with IBM Germany. He has 35 years of experience in storage software and storage management for IBM and customer systems, including 24 years of experience with DFSMSrmm. He has developed, delivered, and taught DFSMSrmm education to customers around the world. He has many tools available, such as a Tape Copy Tool to support the DFSMSrmm business. His areas of expertise include performing conversions from vendor tape management products to DFSMSrmm and new DFSMSrmm implementations. Norbert provides marketing support for DFSMSrmm, including IBM 3494 / IBM 3495 ATLS, VTSs and TS7700 Virtual Engines. He has worked at IBM for 39 years.

The original authors of this book are:

Yolanda Cascajo Jiménez
Sue Hamner
Andreas Henicke
Begoña Fernández Vila
Dong Rui Ling

Thanks to the following people for their contributions to this project:

Robert Haimowitz
ITSO, Raleigh Center

Become a published author

Join us for a two- to six-week residency program! Help write a book dealing with specific products or solutions, while getting hands-on experience with leading-edge technologies. You will have the opportunity to team with IBM technical professionals, Business Partners, and Clients.

Your efforts will help increase product acceptance and customer satisfaction. As a bonus, you will develop a network of contacts in IBM development labs, and increase your productivity and marketability.

Find out more about the residency program, browse the residency index, and apply online at:

ibm.com/redbooks/residencies.html

Comments welcome

Your comments are important to us!

We want our Redbooks to be as helpful as possible. Send us your comments about this or other Redbooks in one of the following ways:

- Use the online **Contact us** review redbook form found at:

ibm.com/redbooks

- Send your comments in an e-mail to:

redbook@us.ibm.com

- Mail your comments to:

IBM Corporation, International Technical Support Organization
Dept. QXXE Building 80-E2
650 Harry Road
San Jose, California 95120-6099



DFSMSrmm enhancements

You may be an experienced DFSMSrmm user and are thinking of migrating from your current DFSMSrmm release up to z/OS V1R13. If so, then you will find the information in this chapter useful. This chapter documents the new functions included in z/OS V1R3 up to z/OS V1R13, what users will find different as compared to the previous release, and what is needed to use each new function. We refer to related chapters for details about each function.

The new function of following z/OS DFSMSrmm releases are described:

- ▶ “Changes introduced with z/OS V1R13 DFSMSrmm” on page 2
- ▶ “Changes introduced with z/OS V1R12 DFSMSrmm” on page 4
- ▶ “Changes introduced with z/OS V1R11 DFSMSrmm” on page 5
- ▶ “Changes introduced with z/OS V1R10 DFSMSrmm” on page 8
- ▶ “Changes introduced with z/OS V1R9 DFSMSrmm” on page 12
- ▶ “Changes introduced with z/OS V1R8 DFSMSrmm” on page 14
- ▶ “Changes introduced with z/OS V1R7 DFSMSrmm” on page 16
- ▶ “Changes introduced with z/OS V1R6 DFSMSrmm” on page 17
- ▶ “Changes introduced with z/OS V1R5 DFSMSrmm” on page 18
- ▶ “Changes introduced with z/OS V1R4 DFSMSrmm” on page 18
- ▶ “Changes introduced with z/OS V1R3 DFSMSrmm” on page 19

1.1 Changes introduced with z/OS V1R13 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R13.

1.1.1 Customize point-and-shoot fields in the DFSMSrmm dialog

To easily see the fields that are enabled for point-and-shoot, you must customize the color, intensity and highlighting of the point-and-shoot fields. Issue the ISPF system command PSCOLOR from any ISPF command line and adjust the Point-and-Shoot Panel Element as desired.

1.1.2 Specifying the retention method to be used for new tape data sets

Among the important decisions to be made when using DFSMSrmm are how to retain tape data sets and for how long. You might want to retain a given data set for a specific period of time after it is created, or retain it based on some event (for example, while the data set is catalogued), or retain it permanently.

DFSMSrmm provides two retention methods for retaining tape data sets:

- ▶ The EXPDT retention method, which allows expiration to be on a specific date - the expiration date.
- ▶ The VRSEL retention method, which uses vital record specifications to implement retention and movement policies through which a retention date is calculated each time that VRSEL inventory management processing is run. DFSMSrmm retains a volume based on this retention date and on the volume expiration date.

You can use the EDG_EXIT100 installation exit to set the retention method to be used for new tape data sets. When you create a new tape volume set, or rewrite an existing set from the first file you can override the system default retention method.

Note: Use the RETENTIONMETHOD parmlib option if you want to set a default system-wide retention method. If you do not specify a default retention method, the system uses the VRSEL retention method as default.

1.1.3 IBM TS1140 support

DFSMSrmm now supports the new media types MEDIA11, MEDIA12 and MEDIA13 as well as the new recording formats EFMT4 and EEFMT4. Following the correspondence between the DFSMSrmm media types and the media types used in the system-managed tape data class construct.

MEDIA11/EATC	IBM Enterprise Advanced Tape Cartridge
MEDIA12/EAWTC	IBM Enterprise Advanced WORM Tape Cartridge
MEDIA13/EAETC	IBM Enterprise Advanced Economy Tape Cartridge

1.1.4 Copying data set attributes in a tape copy application

A tape copy application can use the EDG_EXIT100 installation exit to copy the data set attributes from the original to the copy during OPEN processing. In the EDG_EXIT100

installation exit you can specify the data set details for the source data set from which attributes should be copied.

1.1.5 Excluding specific data sets from VRSEL processing

You can use the EDG_EXIT100 installation exit to exclude specific data sets from DFSMSrmm VRSEL processing as they are created or rewritten. You can specify this for any data set, but DFSMSrmm ignores the request unless the data set is on a volume that is managed by the VRSEL retention method. The data set VRSELEXCLUDE attribute is set for all data sets on volumes managed by the EXPDT retention method and is not affected by this support.

When DFSMSrmm excludes a data set from VRSEL processing, it ensures that the data set vital record attribute is reset and the retention date is set to current date. The matching VRS information is left unchanged.

1.1.6 Inventory management considerations for EXPROC and VRSEL

The inventory processing for processing vital records and performing expiration processing has been changed as following:

- ▶ You do not need to run VRSEL processing unless any volumes are defined with the VRSEL retention method. Only EXPROC processing is required to handle expiration of all volumes managed by the EXPDT retention method.
- ▶ EXPROC processing provides a summary of volumes by retention method.
- ▶ The expiration date of volumes is set during OPEN processing, so for volumes managed by the EXPDT retention method, no special considerations exist for open data sets; they are managed based on the volume EXPDT.
- ▶ For volumes managed by the EXPDT retention method, no special considerations exist for data sets closed by ABEND processing or that are DELETED; they are managed based on the volume EXPDT.
- ▶ Volumes managed by the EXPDT retention method are included only in the EXPDTPROP limit. VRSRETAIN and VRSDROP limits apply only to volumes managed by VRSEL retention method.

1.1.7 Volume Usage, Capacity, and Compression considerations

DFSMSrmm tracks information about what you write onto tape volumes, including record length, block size and number of blocks. DFSMSrmm also tracks information about the data written to the tape volume, such as the compressed size and physical size of the file and can calculate the compression ratio. In addition, information is recorded about the media capacity, percentage used, physical space used, and the overall compression ratio.

Basic information about the media can be defined by command, but, when a volume is used for output, DFSMSrmm records the media type, recording format, capacity and percentage used. Normally this information is provided by the tape drive when the drive is an IBM 3590, or IBM TS11x0 or later drive. For older tape devices such as 3490, and also for virtual tape emulating 3490, this information is not available and is derived from media information hard-coded in DFSMSrmm. For non-IBM media, and to override these details for IBM media, you can define media information to DFSMSrmm using the MEDINF command in the DFSMSrmm parmlib.

1.2 Changes introduced with z/OS V1R12 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R12.

DFSMSrmm™ provides multiple enhancements in z/OS V1.12. The ACTIVITY file is updated to include the reason why a DFSMSrmm retention limit was reached. This function is also available now for z/OS V1.10 and z/OS V1.11 with the PTF for APAR OA30881. New reports created from the ACTIVITY and extract files are intended to help you see why retention limits were triggered. Also, OPENRULE ignore processing is available for duplicate tape volumes and support is intended to allow you to set a volume hold attribute to prevent expiration and to search and report on volumes which have the hold attribute. It is also intended that the DFSMSrmm ISPF dialog search results can be bypassed when using the CLIST option.

- ▶ Additional z/OS V1.12 scalability and performance improvements:
DFSMS now supports additional data set types, including basic and large format sequential data sets, partitioned (PDS/PDSE) data sets, direct (BDAM) data sets and catalogs in the extended addressing space (EAS) on extended address volumes (EAVs). Support is also included for generation data groups (GDGs) and VSAM volume data sets (VVDSSs). Overall, EAV helps you relieve storage constraints as well as simplify storage management by providing the ability to manage fewer, large volumes as opposed to many small volumes. Other new EAV related functions include:
 - Support is added to make all data sets used by DFSMSrmm eligible for allocation in the extended addressing space of an EAV. This includes the DFSMSrmm journal and dynamically allocated temporary files.
- ▶ With z/OS V1.12 DFSORT:
Information about DFSMSrmm active and queued tasks is available via the DFSMSrmm API and via a TSO/E subcommand, enabling storage applications to monitor and act on the available information. In addition you can use the DFSMSrmm dialog to manage these tasks.
- ▶ Details on the optimization improvements for z/OS V1.12 include:
DFSMSrmm helps with reporting of data sets and logical volumes which are copy exported from a TS7700 virtualization engine.
- ▶ The Integrated Storage Management Facility (ISMF) includes a Data Collection application, DCOLLECT, which provides storage-related measurement data that can be used as input to the DFSMSrmm Report Generator to create customized reports or to feed other applications such as billing applications. In z/OS V1.12, DCOLLECT data class (DC) records are updated to include information about all data class attributes. Also, data set (D) records now include job names, and storage group (SG) records now include information about OAM Protect Retention and Protect Deletion settings.
- ▶ In z/OS 1.12, DFSMSrmm supports IPv6.

1.2.1 Ease of Use and Flexibility Enhanced

- ▶ Retention Limit Reporting
- ▶ Ignore for duplicate volumes
- ▶ Automation for WTORs in Production and Parallel Running
- ▶ Expiration Override for Volumes
- ▶ ISPF Dialog CLIST option to avoid search results list

1.2.2 Simplification

Now you have the availability to request information about DFSMSrmm subsystem address space status, tasks, and queued requests.

In addition to the MODIFY operator command you can now use the RMM LC STATUS subcommand to retrieve information about the DFSMSrmm subsystem, subsystem requests, and task status.

1.2.3 Storage Reporting

EDGJCEXP provides a report on copies of logical volumes that have been exported from TS7700 Virtualization Engine. The report consolidates point in time information from the copy export status file, the library and DFSMSrmm to help you identify tape data that has been copy exported.

1.2.4 Availability

DFRMM shutdown now issues an additional message to list the job names of the address spaces preventing shutdown. DFSMSrmm subsystem interface processing now correctly detects that DFRMM is or has been stopped.

With DFSMSrmm on z/OS V1R12 and later releases the ASID/JOB value is automatically added to all trace records created outside of the DFRMM subsystem address space; this enables the ASID and jobname to be stored to the trace record.

1.2.5 Scalability & Performance

- ▶ IPV6 Supports networking strategy
- ▶ EAV supported for all data sets
- ▶ EAS exploited for temporary data sets
- ▶ XTIO, DSAB above, and uncaptured UCB support

1.3 Changes introduced with z/OS V1R11 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R11.

1.3.1 EDGINERS Tape Label Scan

The DFSMSrmm tape utility EDGINERS performs initialization and erasure of tapes. For tape label read and display another utility like DITTO / File Manager has to be used. Now EDGINERS is updated to support the reading and cross-verification of tape label information with the records defined in the DFSMSrmm control data set. The new function SCAN helps identifying and managing tapes, that are coming from other systems or are in a problem state. The EDGINERS SCAN function reads the VOL1 and header labels for the first file on the specify volume.

You can SCAN volumes that are:

- ▶ SMS managed or not SMS managed

- ▶ known to RMM or not known to RMM
- ▶ of all status values except volumes damaged
- ▶ of all types, including WORM and encrypted tapes

and if you have READ authorization.

1.3.2 Dynamic Installation Exits

DFSMSrmm now uses the system Dynamic Exit Services to manage calls to the installation exits, determine if exit modules exist, and to provide error handling and recovery. The installation exits DFSMSrmm provides are shown in Table 1-1.

Table 1-1 Sample installation exits IBM provides

Exit Name	Default Exit Routine	SAMPLIB Samples Provided
EDG_EXIT100	EDGUX100	EDGUX100, EDGCVRSX
EDG_EXIT200	EDGUX200	EDGUX200
EDG_EXIT300	EDGUX300	EDGUX300

DFSMSrmm installation exit modules can be loaded from any APF authorized library, the default is the LINKLST

Dynamic Exit Services is used to load and activate the default (EDGUXn00) exit module at initialization time. The default exit is activated in first position. All other exit related processing is handled by Dynamic Exit Services. If you do not use the default exit module (EDGUXn00) you must use PROGxx in z/OS parmlib or the SETPROG operator command, or CSVDYNEX macro.

1.3.3 Returning Volumes to the System-Managed Library

The sample volume not in library installation exit, CBRUXVNL, is enhanced to enable fewer installations to require customization. It is intended that volumes will be requested to be entered into the tape library whenever possible.

Additional checks and customization are added to the sample volume not in library installation exit so that there are more cases where a decision can be taken. **EDG8121D** is now issued if a Tape Configuration Database (TCDB) volume entry exists, or if the volume is known to be in a different library, but only if the volume is defined to DFSMSrmm, and as long as the volume is not on loan, and would not be rejected during OPEN (status is pending release, init action pending, volume is scratch, or not for use on MVS).

When you regularly store system managed volumes outside of a system managed library, and also have the home location set to SHELF (or some other LOCDEF-defined storage location with a type of HOME) you can ensure that the volumes will be called for by CBRUXVNL by using the SMSTAPE(PURGE(NO)) option. This results in the TCDB volume entry being retained while the volume is ejected. The DFSMSrmm supplied sample exit checks if the TCDB entry exists and, for DFSMSrmm managed volumes will prompt the operator to re-enter the volume into a library.

An alternative to the SMSTAPE(PURGE(NO)) option is to use SMSTAPE(PURGE(ASIS)) and ensure that the library eject default is set to KEEP. This enables you to be selective about the libraries to which the purge/keep option applies. SMSTAPE(PURGE(ASIS)) is the default.

1.3.4 VRSEL GDG option

New DFSMSrmm parmlib options provide flexibility in how tape generation data sets are managed for cyclic retention.

Use the GDG option to specify how generation data groups are handled for cycle retention by VRSEL processing. Cycle retention includes both the CYCLES and the BYDAYSCYCLE retention types. The correct sequence for determining the retention can be either by using the generation number or the creation order. You can also specify how duplicate generations are handled and have the flexibility to include or exclude duplicates from the cycles count as required by your application processing.

1.3.5 DFSMSrmm Report Generator

Extensive changes are made for the DFSMSrmm Report Generator, improving usability, enabling more customization of reports, and simplifying the way that selection information can be specified. This is accomplished through exploitation of recent changes to DFSORT and ICETOOL, data typing, and report type inheritance. The changes further improve the reporting available for DFSMSrmm, DFSMSHsm and other DFSMS components. Following you can see all the changes where made:

- ▶ You can now override a data type within the dialog. The updated data type is saved in the report type and report definition.
The Report Generator remembers the original data type and your override. You can use report type inheritance to benefit from any new data type values in report types.
- ▶ You can now inherit changes in report types into existing report definitions. This enables changes in data types, comments and other criteria to be merged into pre-existing report definitions – improving the usability and enabling your reports to benefit from improved report types shipped with DFSMSrmm.
- ▶ You can now select which fields are not to be included in total and break totals. When you specify that a field used for a report column is not to be subject to totaling the Report Generator uses the NOST option with the ICETOOL reporting tool. When used with ICETOOL all numeric fields are automatically totaled unless you request they are to be excluded.
- ▶ You can now specify a list of field values and the text to be used for them in the report. The Report Generator uses existing field equate values to construct an initial list of possible values; before you can use any of these you must provide a new value to which the field will be changed for the report. Only those which have a change value are used for report generation.
- ▶ When you specify the column width to be used the generated ICETOOL statements now override the default ICETOOL processing. If no override is provided, the default is to set the width at a maximum of the column header text, and the data size; which is:
 - 10 characters for 4 byte fields, 5 for 2 byte fields, and 3 for 1 byte fields of binary data.
 - For numeric fields declared with Z (zoned) or P (packed) the default is set to the number of possible digits.
- ▶ You can now use the equated assembler symbols instead of the absolute value. An option in the dialog displays the available equates ready for you to use.
When equates are available the Report Generator uses these as a basic set to enable support for the specification of an alternate value for use in the report – this is called a 'change value'. For example 'I' can be changed to 'INFO' and so on.
- ▶ We can now ship guidance within report types and report definitions and this information is presented to the user on request and when generating JCL. You can edit and add to this

information, which is stored in the definitions. Existing report types and definitions are updated with help and guidance information to make clearer things like what variables to set in JCL, etc. How to run HSM preprocessor to convert data etc. The help information is split into 3 parts; Type, Report, and JCL help. It is browsed or edited as a single set of help information.

- ▶ You can now request that the reports created from the DFSMSrmm report extract include the date and time when the extract was created. The report extract type and samples are updated to exploit this. In fact you can do this for any type of report where the input records include one or more values that you want to include in the report title. This exploits new DFSORT ICETOOL capability to specify multiple report TITLE strings.
- ▶ A new reporting tool is shipped which enables records to be manipulated with DFSORT. The output of this tool is not a report, but rather reformatted records. When using the new reporting tool the Report Generator can be considered as a partial replacement for the now withdrawn DFSORT ISPF panels. The JCL generated by the reporting tool includes comments which contain DFSORT symbol definitions so that you can easily process the record further using DFSORT or ICETOOL.
- ▶ All report types are updated to include the relevant data types and help information. In addition all report samples are updated to inherit all new information from the report type, including the data type and help information.
- ▶ You can now use substring for record selection criteria regardless of the data type of the field.

1.3.6 DFSMSrmm usability items

- ▶ Use of MATCHVRS in the ISPF data set dialog
- ▶ New ADDVOLUME Subcommand Fields
- ▶ Extended SEARCHVOLUME Subcommand
- ▶ VRS Location definition handling
- ▶ REXX Variables
- ▶ DFSMSrmm Journaling
- ▶ Using the EDGUPDT Utility
- ▶ API Multi-Entry Return

1.3.7 VRSEL(OLD) Parmlib Option

You must no longer be using VRSEL(OLD) parmliib option. Prior z/OS V1.11 releases EDGHSKP issued a warning message when VRSEL(OLD) was in use.

1.4 Changes introduced with z/OS V1R10 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R10.

1.4.1 Enable use of DFSMSdss COPY Service

With DFSMSrmm release z/OS 1.10 you can request that DFSMSrmm back up the control data set and the journal, using the DFSMSdss COPY command.

DFSMSrmm no longer relies on concurrent copy and virtual concurrent copy for fast and non-intrusive creation of copies/backups of the DFSMSrmm control data set. DFSMSrmm supports the use of DSS copy services and exploitation of Fast Replication services provided by DASD subsystems. This enables almost instantaneous copies of the control data set to be created enabling reduced recovery time objective and eliminates the impact that use of concurrent copy can have on HyperSwap.

DFSMSrmm control data set backup can now additionally be performed using copy services. ADDRDSU is used with Fast Replication to enable a control data set 'backup' to be created. New parameter options are provided to request this method for DFSMSrmm control data set backup.

1.4.2 SMF record changes

RMM creates SMF records when requested, but can now optionally use the IBM assigned record type 42 instead of using SMF record types from the user-range. SMFAUD and SMFSEC options each use a record type unless the IBM record type is used, in which case subtypes are used within record type 42. The subtypes used by DFSMSrmm are 22 for audit records and 23 for security records.

1.4.3 Audit Controls for Release Processing

The existing parmlib options for ensuring that VRS managed retention is as expected include the VRSMIN and VRSCCHANGE operands. If you want to control which volumes are released you have to exploit the NOTIFY release option of VLPOOL so that all volumes can be kept in pending release status until the list of volumes is validated against some installation based criteria. The latter is a manual action which is required, but which could be easily automated.

New parmlib options are added which can be used to ensure that data is being retained as expected and only released by DFSMSrmm processing if the numbers or percentages of volumes are within policy limits. EDGHSKP VRSEL and EXPROC processing counts the numbers of volumes which are:

- ▶ VRS retained; newly and initial count
- ▶ Newly assigned since the last VRSEL run
- ▶ Planned to be set pending release; Dropped from VRS and dropped by EXPDT
- ▶ EXPDT retained initial count

and applies the new parmlib options to determine the action to be taken. New messages are added to the MESSAGE file, and are issued if VRSEL or EXPROC are run and the corresponding parmlib option has not disabled the function. The new messages list the numbers and percentages of volumes. Regardless of disablement some existing messages have a percentage added for completeness. In addition based on the set limit and action, an existing message EDG2310I is issued if action is FAIL, and the EDGHSKP return code set if the limit threshold is exceeded. This processing for the new options is identical to the existing processing for VRSMIN.

1.4.4 XREPTTEXT tailoring via SYSIN

You can create an extract data set to use as input for creating reports. You can select the records to be extracted from the DFSMSrmm control data set. When you specify the RPTTEXT command in the EDGHSKP SYSIN file, you can be selective SYSIN, you can use the DD name to determine if extended records are created. When you specify the XREPTTEXT DD statement, the extract contains only extended records that are a combination of data set

information and volume information that can be useful as input to DFSMSrmm reporting tools. When you code only the REPTXT DD, all records other than the extended records are created.

Report extract processing now checks for the existence of the RPTXT command in the SYSIN file, and if RPTXT command is found writes the selected extract records to either REPTXT or XREPTXT DD. XREPTXT DD is checked for and used in preference.

When there is no RPTXT command in SYSIN or the RPTXT command specifies no RECORDS operand:

REPTXT	RMM creates all records except extended records in the extract (unchanged)
XREPTXT	RMM creates only extended records (changed).

EDG6013I message is used to list the RPTXT command options from SYSIN. This is done in the same way that EXPROC is already listed.

1.4.5 Volume Replacement Policies

New limits and policies are used to determine if the REPLACE actions should be set. MEDINF commands in parmlib can now include REPLACE policy settings;

- ▶ one for each media type and recording format combination.

The place and timing for the checks for the limits is unchanged. In addition,

- ▶ DFSMSrmm now checks at OPEN time to see if a volume to be used for output processing has the REPLACE action pending and rejects the volume. Note that if the REPLACE action is pending the volume must be pending release and already subject to reject by DFSMSrmm O/C/EOV processing.
- ▶ REPLACE release action is set, an owner set for the volume and the volume set pending release, thus setting the release action pending.
- ▶ DFSMSrmm intercepts WTO messages that contain SARS MIM information and set the REPLACE release action for any volume where the recommended recovery action is to copy off the data or replace the volume. See WTO IEA486E. In addition for scratch volumes, an owner is set for the volume and the volume set pending release, thus setting the release action pending.

1.4.6 Report Generator enhancements

The DFSMSrmm Report Generator is an Interactive System Productivity Facility (ISPF) application that you can use to create reports. The Report Generator:

- ▶ Provides reports that you can run as-is or that you can modify as you wish. You can use samples to create reports for volumes, data sets, racks, owners, and the retention and movement policies that are established for your installation. You can modify these samples to create tailored reports. DFSMSrmm ships samples in SYS1.SAMPLIB. See "Running a Report Generator Report" on page 7 to run one of these reports.
- ▶ Generates job control language (JCL) that is based on specifications that you use to submit the report jobs. The generation of JCL depends on the report type and therefore the macros that map the data records. The generation knows, based on the macro name and keyword options used, whether to generate a DCOLLECT jobstep, a DFSMSshm FSR reformat, a DFSMSrmm extract, or a copy of SMF records.
- ▶ Includes samples for reporting from DCOLLECT and DFSMSshm data.

The DFSMSrmm Report Generator is updated to support keywords for assembler macros from which report types are derived, and to add new built-in data extract steps. This is in support of new SMF record types from DFSMSrmm, and for DFSMSHsm and DCOLLECT reporting.

Any new report type or report definitions created on z/OS V1R10 which include macro keyword information can be used by a lower level release, however the macro keywords are ignored and are removed if the report type or report definition is changed or updated. The original input report type or report definition is unaffected and can be reused later on a supporting release to exploit the macro keywords.

1.4.7 System Managed Library partitioning

In z/OS 1.10 new parmlib controls are provided to enable simplified and more powerful partitioning of system managed libraries and controlling of application use of tape volumes. Now You can use PRTITION and OPENRULE commands in the EDGRMM parmlib member to control library partitioning and the use of volumes by applications.

Note: If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands.

You can partition a system-managed library including a VTS by performing these tasks:

- ▶ Specify the USE operand value on the RMM ADDVOLUME or RMM CHANGEVOLUME subcommands. You can set this value to MVS or VM or both. If you do not specify MVS for a volume, DFSMSrmm prevents the volume from being defined in the volume catalog on this system.
- ▶ Through the use of the PRTITION and OPENRULE parmlib commands, you can simplify the maintenance of the parmlib members as your libraries and volume ranges change. Operands on the OPENRULE and PRTITION commands allow global actions to be set. You can use one or more specific overrides based on volume sets that have different requirements. Typically, you could add a new range of volumes for use by a single partition and only that one system would need to be updated. The OPENRULE and PRTITION commands allow you to define whether they apply to volumes defined to DFSMSrmm or not. You can use operands on the OPENRULE command to automatically ignore volumes, rather than using EXPDT=98000, ACCODE=xCANORES or a customized EDGUX100.
- ▶ Define parmlib member EDGRMMxx REJECT prefixes. You can use REJECT to prevent a volume not defined to DFSMSrmm from being defined in a system-managed tape library. The REJECT ANYUSE(prefix) operand prevents a volume from being defined in the system-managed tape library on the current system. The REJECT OUTPUT(prefix) operand allows you to define the volume to the system-managed tape library but only use the volume for input processing.

1.4.8 Common Information Model (CIM) Provider

The DFSMSrmm CIM agent now has an option to register itself using the SMI-S Storage Library profile so that storage management clients (and TPC in particular) can use Service Location Protocol (SLP) to detect the CIM agent and determine which registered profiles it can support.

DFSMSrmm now ships the web service for installation either as an Enterprise Archive (EAR) under WebSphere or as a Web Archive under Tomcat (WAR) or other web service

middleware. For details of externals, handling, installation and customization refer to 'DFSMSrmm Tomcat Web Service User Guide' manual.

1.5 Changes introduced with z/OS V1R9 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R9.

1.5.1 Task management support

Any task in the system that requests DFSMSrmm subsystem services and fails, or is interrupted because a TSO-user used Attention (ATTN), or is cancelled by the operator, results in any corresponding long-running subsystem request failing. In addition, there are checkpoints built into long-running requests so that when the requestor ends (such as a job being cancelled), DFSMSrmm processing is interrupted at a safe and convenient point. Long running local tasks are DFSMSrmm subsystem requests that last long enough to be included in the results of a QUERY ACTIVE command, and the task token can be obtained and used.

1.5.2 Multitasking of utilities

DFSMSrmm utilities, EDGUTIL and EDGHSKP, interaction with system managed volumes in an IBM system managed library is improved through multiple changes that should, especially in larger VTS installations, result in shorter elapsed time and more flexibility. The following are discussed:

- ▶ EDGHSKP EXPROC
- ▶ EDGUTIL
- ▶ Using EDGSPLCS

1.5.3 Control data set (CDS) serialization

DFSMSrmm now requires you to supply a control data set identifier in the CSDID parameter of SYS1.PARMLIB member EDGRMMxx, specifying the identifier of the CDS to be used on that system. DFSMSrmm uses the CSDID as part of the ENQ name used to serialize updates to the RMM CDS. The DFSMSrmm CIM Provider also uses the CSDID to distinguish between multiple control data sets. If the CSDID is not yet set in the CDS you can optionally use EDGUTIL UPDATE to set it, otherwise DFSMSrmm will set the CSDID when first started. We recommend that you use a unique CSDID for each CDS.

At DFSMSrmm startup time, DFSMSrmm matches the CDSID in the control data set control record value with the CDSID operand in parmlib member EDGRMMxx.

1.5.4 JCL data set names

The data set naming requirements for tape data sets supported by the TSO subcommands and API are relaxed so that any 44 character string can be used. This enables use of unqualified data set names such as those that are accepted by the MVS JCL DSNAME keyword. Quoted data set name strings allow any character string to be specified for a data set name except for leading blank and leading hex zero which are not supported.

Important: Quotes make the difference!

1.5.5 Shared parmlib support

Some information in the EDGRMMxx parmlib may need to be specific to a subset of your systems. For example, the REJECT or VLPOOL entries may need to be different across systems. To enable this information to be handled on a system by system basis you can specify a second parmlib member to be used.

Use the MEMBER operand in the primary DFSMSrmm parmlib to identify a second parmlib member that contains overriding or additional parmlib options. Some information in the EDGRMMxx parmlib member may need to be specific to a subset of your systems.

1.5.6 TSO subcommands

TSO subcommand parsing rules are further relaxed to support different product versions and declassification of data sets and volumes. The following topics are discussed.

- ▶ Changes in subcommand parsing.
 - The rules for the Product Level have changed.
 - A new NOSECLEVEL parameter was introduced to reset the SECLEVEL for a volume or data set.
- ▶ CLIST enhancements.
 - New parameters START and ADD were introduced to enable the user to either overwrite or add to an existing CLIST data set.
 - Any existing or user specified DCB record format parameters are now supported.
- ▶ Search command enhancements.
 - A new CONTINUE parameter was introduced for all search commands as a way to break down search results into manageable quantities.
 - There is a new STORAGEGROUP parameter for the Search Volume command.
- ▶ Report 17
 - A new report, REPORT 17, is added to the EDGRRPTE exec. It summarizes information for logical and stacked volumes to support stacked volume management.

1.5.7 3592 Model E05 software support

Using the existing media types (MEDIA5, MEDIA6, MEDIA7 and MEDIA8) and the two extended length future media types (MEDIA9 and MEDIA10), an encryption enabled 3592 Model E05, reads and writes using the new Enterprise Encrypted Format 2 (EEFMT2) recording technology. It can also read and write using the Enterprise Format 1 (EFMT1) and Enterprise Format 2 (EFMT2) non-encrypted recording technologies.

In order to request EEFMT2 in the stand-alone and in the system-managed IBM tape library environment, a DFSMS dataclass must be used which specifies EEFMT2 as its recording technology, otherwise EFMT2 is the default recording technology that is used. Dataclass can also be used to request the EFMT1 recording format and to explicitly request the EFMT2 format. A mix of recording formats is not supported on the same tape cartridge. An enhanced 3592 Model E05 that does not have the encryption feature enabled, can only read and write in the non-encryption formats (EFMT1 and EFMT2) which is the same as the base 3592 Model E05. In a mixed 3592 environment, new microcode is also needed for the 3592 Model J and the base 3592 Model E05 so that it recognizes a volume with the new EEFMT2 recording technology.

1.5.8 REPORT17 of EDGRRPTE REXX Exec

When a stacked volume, containing exported logical volumes, is ejected from the library, as the logical volumes expire, RMM places the volumes in a "pending release" state and then when the logical volumes are imported into the library, RMM completes the return to scratch process enabling the volumes to be reused. As the exported logical volumes expire, you need the ability to do off-site stacked volume management so you can determine when to bring a stacked volume back on-site for possible reuse. RMM has enough information in its database for you to create and run reports, however, a specific stacked volume management report did not exist.

With z/OS V1R9, RMM provides a new stacked volume management report for customers to customize and run, that includes the ability to report on the percentage of active data on a stacked volume and to also report on the percentage of active logical volumes on a stacked volume. A new report, REPORT17, is added to EDGRRPTE reporting exec which summarizes information for logical and stacked volumes. The stacked volumes are presented in order of increasing percentage of active number of volumes and percentage used. The least used stacked volumes are listed first.

1.5.9 Common Information Model (CIM) provider

The RMM CIM provider is the link for a customer to obtain real-time RMM data within a Common Information Model (CIM) environment. This functionality will enable a CIM client such as a PC to obtain RMM data.

In z/OS V1R8 DFSMSrmm shipped a Common Information Model (CIM) Provider for use with the OpenPegasus CIMOM. The provider supported a subset of the resources managed by DFSMSrmm. The CIM model supported by DFSMSrmm is further extended in this release to cover all of the resources managed by DFSMSrmm. The CIM provider is updated to support out-of-process mode functionality added to OpenPegasus CIM Server with 2.5.1. This has an impact on the way that the DFSMSrmm provider code is installed and run.

In addition, the RMM extensions to the CIM object model are based on the latest CIM V2.11 specification. To further enable selection of resources via the API the SEARCH subcommands are all now capable of specifying continuation, enabling "chunking" of returned entries.

1.6 Changes introduced with z/OS V1R8 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R8.

1.6.1 Support true e-mail address for the RMM NOTIFY function

The established DFSMSrmm notification feature is extended to optionally send mails to internet addresses. E-mail notification is provoked in the same way as the known notifications. There is no difference to the current processing, only the result (e-mail instead of the established messages) is different.

Prerequisites for e-mail notification:

- ▶ SMTP server available in the customer environment and known to DFSMSrmm
- ▶ owner attribute ,e-mail address' is set

1.6.2 Setting up DFSMSrmm Common Time Support

Before DFSMSrmm common time support (UTC), also known as GMT, is enabled, all dates and times are stored in the DFSMSrmm control data set in local time. When the control data set is shared, and the sharing systems are set to run in different time zones, the local dates and times in the control data set may be from any of your systems. When you display information or extract records, you need to be aware of how the records were created, on which system, and where they may have been updated in order to interpret the dates and times shown. The same consideration also applies for records created or updated prior to enabling common time support because DFSMSrmm assumes they are times local to the system running the DFSMSrmm subsystem and converts the values based on that assumption.

When you enable common time support, DFSMSrmm maintains the records in the control data set in common time. Most date and time fields are paired together to enable an accurate conversion to and from common time and between different time zones. In some cases, DFSMSrmm has date fields in control data set records, and there is no associated time field. For these date fields, DFSMSrmm uses an internal algorithm that approximates conversion between time zones based on the time zone offsets involved.

1.6.3 DFSMSrmm VRS policy management simplification

In this new release the VRS processing changed and the RMM TSO subcommand and the reporting are enhanced as following:

- ▶ Separation of Data Set Name Mask from the Policy
- ▶ Release Options Applied if VRS Matched
- ▶ Special ABEND, DELETED and OPEN via DSNAME match
- ▶ Find unused VRSs
- ▶ Incomplete VRS chains - dummy VRS `**broken**`
- ▶ Toleration and removal of really old functions.
- ▶ Conversion to DFSMSrmm from other tape management systems

1.6.4 DFSMSrmm usability items

- ▶ Updates to RMM TSO SEARCHVOLUME subcommand
- ▶ ISPF Lists Show Retention Information
- ▶ SELECT primary Command in RMM Dialog Search Results
- ▶ RMM TSO CHANGEVRS subcommand
- ▶ RMM TSO SEARCHOWNER subcommand
- ▶ REXX Variable Constraint Relief
- ▶ Enabling ISPF Data Set List (DSLIST) Support

1.6.5 Tape data set authorization

DFSMS V1.8 provides new options for securing tape data sets using the system authorization facility (SAF), to allow you to protect data sets on tape using the RACF DATASET class without the need to activate the TAPEDSN option or the TAPEVOL class. This new tape data set authorization checking allows you to specify that all data sets on a tape volume should

have common authorization. You can specify whether users are authorized to overwrite existing files on a tape volume. In addition, you can specify that a user must have access to the first file on a tape volume to add additional files on that tape volume.

1.7 Changes introduced with z/OS V1R7 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R7.

1.7.1 Issue DFSMSrmm TSO commands from the console

The MVS modify command can now be used to issue an RMM TSO command from the console. The command output is returned to the console and system log.

Figure 1-1 shows you a sample command.

```
F DFRMM,CMD=LISTCONTROL ALL
```

Figure 1-1 RMM TSO command from the console

Note: There may be some truncation and/or line output wrapping when using this function. The effective line length of SYSLOG is less than the 79 column standard on TSO. Whether this results in loss of significant information depends on the command. Lines that end with “=” have typically been truncated, and they may also have information wrapped to the next line.

1.7.2 Improved security control over DFSMSrmm functions

New resource profiles are implemented in z/OS v1R7 DFSMSrmm to better control access to DFSMSrmm resources. It is no longer necessary to have CONTROL access to STGADMIN.EDG.MASTER to run daily DFSMSrmm tasks.

When checking authorization to use RMM subcommands and operands, DFSMSrmm checks in the following sequence:

1. CONTROL access to STGADMIN.EDG.MASTER. If the user is authorized, no further checking is performed.
2. Next, DFSMSrmm checks for specific subcommand operands and for each operand that requires specific authorization checks for the required access. If the resource is not protected, authorization continues with the next step. If the resource is protected, but the user is not authorized, the subcommand fails.
3. Finally, DFSMSrmm continues with ownership checks and RELEASE and FORCE checking, if required.

1.7.3 Use of Large Format data sets

Large format data sets can have greater than 65,535 tracks per volume. Before z/OS V1R7, most sequential data sets were limited to 65,535 tracks on each volume, although most hardware storage devices supported far more tracks per volume. To support this hardware capability, z/OS V1R7 allows users to create new large format data sets, which are physical sequential data sets with the ability to grow beyond the previous size limit. Large format data

sets are not required to have greater than 65,535 tracks at initial allocation, but once allocated in large format they can expand to greater than 65,535 tracks.

1.7.4 Enterprise enablement

In DFSMS V1.7, the direction of extending DFSMSrmm to the enterprise continues. Extending DFSMSrmm to the enterprise is being accomplished through enabling the object oriented interface to the RMM API to be used via Web Services and creating a plug-in adapter for use with SNIA CIMOM SMI-S.

With z/OS V1R7 and the DFSMSrmm enterprise enablement enhancement, you can use the high-level language application programming interface as a Web service. This enables the API to be used from any system or platform that can run Java, C++, or any language that supports the Web services standards. Now, it is as if the high-level language application programming interface is available as a locally callable program. A single call to the application programming interface to run a subcommand and receive all the output is all that is needed.

1.8 Changes introduced with z/OS V1R6 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R6.

1.8.1 ISPF dialog enhancements

The DFSMSrmm ISPF dialog has been enhanced for better usability. You can:

- ▶ Request that DFSMSrmm prime panels with saved variables
- ▶ Create a list of DFSMSrmm TSO subcommands using the dialog
- ▶ Use new line operators in dialog lists to better manage DFSMSrmm resources
- ▶ Specify a new user option to define the type of output station where you want your system-managed cartridges ejected

1.8.2 DFSMSrmm client/server support

DFSMSrmm can be used in a client/server environment. Part of this function includes new operator commands to stop and start the subsystem or IP tasks. This function also provides new REXX variables that you can use to write REXX execs to obtain information about your client/server environment.

1.8.3 Enhanced IBM 3592 support

DFSMSrmm provides support for you to use the IBM Total Storage Enterprise Tape System 3592 WORM tapes. Information has been added about the following topics:

- ▶ Added new media types MEDIA6, MEDIA7, and MEDIA8
- ▶ Added operand values for ADDVOLUME, CHANGEVOLUME, and SEARCHVOLUME
- ▶ Added new return codes and reason codes
- ▶ Added new REXX variables and updated several existing variables

- Updated command syntax diagrams for ADDVOLUME, CHANGEVOLUME, and SEARCHVOLUME

1.8.4 Command reference summary in Guide and Reference

The *z/OS DFSMSrmm Command Reference Summary* is no longer available as a separate document. The command summary information has been added to the *DFSMSrmm Guide and Reference*, SC26-7404, as an appendix.

1.8.5 C++ API with XML Output Option

You can use C++ and other high-level programming languages to write programs to obtain information about DFSMSrmm resources. You use the same DFSMSrmm subcommand strings that you can use with the EDGXCI application programming interface (API).

You can receive output as structured field introducers or in Extensible Markup Language (XML). The XML output contains data and tags to define the data. DFSMSrmm provides a schema called rmmxml.xsd that contains the definitions for the XML. For XML output, DFSMSrmm converts the data to character in Unicode format as defined in the XML Schema file for the DFSMSrmm resources.

1.9 Changes introduced with z/OS V1R5 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R5.

1.9.1 Enhanced multilevel security

Enhanced multilevel security to support the RACF name hiding function is now provided.

- DFSMSrmm provides added protection for data set and volumes by supporting the RACF name hiding function.
- Recognition of RACF SETROPTS MLNAMES support has been added since introducing this feature in z/OS 1.3.

1.10 Changes introduced with z/OS V1R4 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R4.

1.10.1 Back up the DFSMSrmm control data set and journal at any time

You can back up the DFSMSrmm control data set and journal at any time. New JCL examples have been added that show how to request a back up of the DFSMSrmm control data set and how to clear the journal.

Backup has been enhanced to run with other Inventory Management (housekeeping) functions such as the VRSEL, RPTEXT, or XRPTXT processes.

1.10.2 Checking Authorization to Ignore Volumes

You can control whether DFSMSrmm manages or ignores volumes based on whether they are defined to DFSMSrmm or not. You can use RACF profiles to separate authorization checking for volumes that are defined DFSMSrmm and that are not defined to DFSMSrmm. To support this new processing two new resources in RACF class FACILITY are added:

- STGADMIN.EDG.IGNORE.TAPE.RMM.volser
- STGADMIN.EDG.IGNORE.TAPE.NORMM.volser

1.10.3 Duplicate Volume support

You can use volumes with duplicate volume serial numbers. Information and examples about how to define volumes with duplicate volume serial numbers have been added. You can also use the DFSMSrmm EDGINERS utility to initialize duplicate volumes.

Duplicate volume support allows you to:

- ▶ Manage duplicate volume serials without using EDGUX100
- ▶ Use any number of duplicate volumes with the same VOL1 label
- ▶ Enable support for Tape Open input and output processing
- ▶ Return to scratch automated to relabel duplicate volsers to match VOL1

1.10.4 Enhanced multi-level security

DFSMSrmm provides added protection for data sets and volumes by providing command authorization support using the DFSMSrmm EDGRMMxx parmlib OPTION COMMANDAUTH operand.

Multilevel security support allows:

- ▶ COMMANDAUTH PARMLIB member to control calls to RACF DATASET class, RACF TAPEVOL class, or both
- ▶ COMMANDAUTH defaults to maintain access by OWNER for previous release compatibility

1.11 Changes introduced with z/OS V1R3 DFSMSrmm

This section describes the enhancements to DFSMSrmm introduced with z/OS V1R3.

1.11.1 CDS utilization displays now available

CDS utilization is available via the LISTCONTROL command in the control record.

1.11.2 Journal utilization and status displays now available

Journal utilization is now available via the LISTCONTROL command in the control record.

1.11.3 Movement date maintained for ON LOAN volumes

Volumes Movement Tracking Date now maintains date tape that was placed On Loan.

1.11.4 Enhanced EJECT support

Enhanced EJECT support allows for:

- ▶ Eject of volumes synchronization with the Tape Configuration Data Base (TCDB)
- ▶ Eject failed if CDS/TCDB status mismatch
- ▶ DFSMSrmm checking of whether a volume is in an EJECT queue

1.11.5 Enhanced foreign tape authorization

Foreign tape authorization has been enhanced to offer:

- ▶ New RACF facility classes:
STGADMIN.EDG.IGNORE.TAPE.**RMM**.*
STGADMIN.EDG.IGNORE.TAPE.**NORMM**.*
- ▶ Control of how foreign tapes should be processed, either with volumes defined in DFSMSrmm or volumes out of DFSMSrmm control

1.11.6 3590-H support

3590-H includes support of the following items:

- ▶ D/T3590-H support for 384TRACK recording format
- ▶ EDGINERS updates to the mediatype and recording format

1.11.7 Report Generator enhancements

The Report Generator provides:

- ▶ Field-to-field compare including position override
- ▶ &TODAY with relative values (+/-); Days, Months, and Years
- ▶ Break TOTALS when using Grouping
- ▶ JCL skeleton now named in Report Tool Table
- ▶ Optional Extract step in generated JCL
- ▶ Report Definition Table shows Selection fields

1.11.8 Expiration Date field added to DFSMSrmm Data Set record

The addition of the Expiration Date field to the DFSMSrmm Data Set record allows for:

- ▶ Expiration Date and Original Expiration Date now supplied in Data Set record
- ▶ Report extract and extended report extract file changed to add Expiration Date and Original Expiration Date
- ▶ Enhanced DFSMSrmm TSO command support to allow for entry of a date or a retention period for add and changing of data sets and volumes

1.11.9 Multivolume dialog support

The multivolume dialog support provides:

- ▶ A dialog option to process entire multivolume set for Change and Release updates
- ▶ Dialog informational messages that are issued for multiframe, multivolume, or multiframe/multivolume

- ▶ Dialog 'L' line command, which has been enhanced to list all data sets on the set in sequence

1.11.10 Special character support

Special characters are now supported for the following DFSMSrmm entry types:

- ▶ VOLSER
- ▶ RACK
- ▶ POOL

These characters are also supported on the REJECT parameter. There is no support for special characters for the BIN entry type. The special characters now supported are: , . ' / () * & + - = and blank.

Special considerations

Note the following considerations for special character support.

- ▶ For volumes that reside in IBM automated tape libraries, only alphanumeric characters are supported (with barcode labels) unless the unlabeled tape facility is used. When the unlabeled tape facility is used, then the alphanumeric characters plus - (hyphen), # (number or pound sign), & (ampersand), \$ (dollar sign) and @ (at sign) are supported.
- ▶ Leading blanks are not allowed. DFSMSrmm does not support a leading blank on the volser.
- ▶ Asterisks are allowed anywhere in a volser.
AA001*, *A0001 and A*0001 are all valid volsers
Using an asterisk in a volser could lead to unexpected results when using the RMM dialog. The RMM dialog treats the asterisk as a wildcard character and therefore returns the actual volser requested and any others that match the mask specified.
- ▶ The ADDVOLUME volser COUNT(n) supports fewer than six characters in the VOLSER field. The following ADDVOLUME subcommand adds volumes A0001 through A0015:
ADDVOLUME A0001 COUNT(15)
- ▶ RACKs can now be less than six characters.
- ▶ Use the TPRACF(N) option for special character volsers. You should protect them outside RMM with a generic RACF TAPEVOL profile, if required. This is because RACF does not support special characters in tape volume volsers for the TAPEVOL class.

1.11.11 Changed messages for improved diagnostics

There are two changed messages to improve error diagnostics.

- ▶ The catalog name is included in CATSYNCH VERIFY messages:
EDG2236I DATA SET DQQ.SAMMEL.ESA.TREND.G0001V00 VOLUME B51987 DSSEQ 1 IN
CATALOG UCAT.X IS NOT DEFINED TO DFSMSrmm
- ▶ Pool information is added to the tape reject message:
EDG4021I VOLUME volser REJECTED, IT IS NOT IN AN ACCEPTABLE SCRATCH POOL. rtype
rvalue REQUESTED. mtype mvalue MOUNTED

type = SG, PREFIX. value = SG name, pool prefix value

1.11.12 HELP moved from SYS1.SEDGHLP1 to SYS1.HELP

You no longer need to copy DFSMSrmm TSO/E help into SYS1.HELP because DFSMSrmm TSO/E help is now included in SYS1.HELP along with the rest of DFSMS. When all systems in your sysplex are on z/OS V1R3 DFSMS, then SYS1.SEDGHLP1 can be removed from your concatenations and deleted from the systems.



Introduction to DFSMSrmm

Data Facility System Managed Storage removable media manager (DFSMSrmm) is a full function tape management system that is available as a functional component of z/OS. It enables you to manage your removable media as an enterprise library across systems that can share disk.

DFSMSrmm can manage all of your tape volumes and the data sets on those volumes. It protects tape data sets from being accidentally overwritten, manages the movement of tape volumes between libraries and vaults over the life of the tape data sets, and handles expired and scratch tapes, all according to policies that you define. Plus DFSMSrmm manages other removable media that you define to it. For example, it can record the shelf location for optical disks and track their vital record status. DFSMSrmm does not yet automatically record information for optical volumes.

This chapter provides a brief introduction to DFSMSrmm basic concepts and functions. By the end of this chapter, you will have a basic understanding of DFSMSrmm.

2.1 Environment

All data set, volume, and policy information is kept in the DFSMSrmm control data set (CDS). The CDS is a VSAM key-sequenced data set (KSDS) that contains all inventory information, and can be either an extended format (EF), or a non-extended format VSAM data set. The only other dedicated data set is the DFSMSrmm journal. This is a sequential data set that is used to record all updates to the CDS. When not shared with a z/OS release below z/OS V1R12, the DFSMSrmm journal can be placed in an extended addressing space (EAS) on an extended address volume (EAV). To tailor your processing options, create the EDGRMMxx PARMLIB member. Figure 2-1 shows the DFSMSrmm environment.

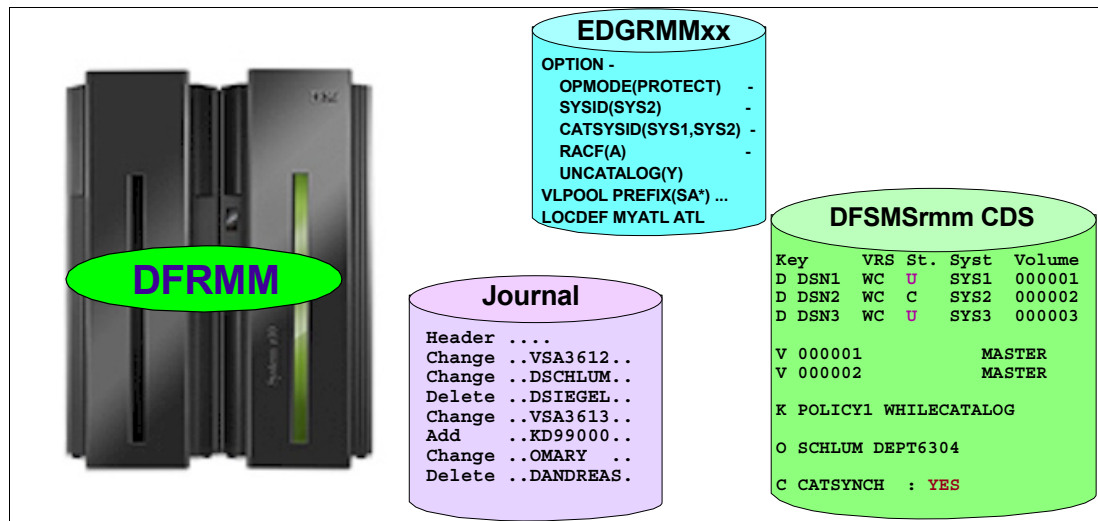


Figure 2-1 DFSMSrmm environment

Note: The benefits available for extended format (EF) VSAM data sets include:

- ▶ Data striping
- ▶ Data compression
- ▶ VSAM extended addressability
- ▶ Partial space release
- ▶ System-managed buffering
- ▶ CDS size greater than 4 GB

All data sets used by DFSMSrmm can be eligible for allocation in the extended addressing space of an extended address volume (EAV). This includes the DFSMSrmm journal and any dynamically allocated temporary files.

All DFSMSrmm interactions take place under control of the subsystem. Utilities are provided to start the HOUSEKEEP function, so that you can schedule DFSMSrmm work through your production scheduling systems. One benefit of running as a subsystem is evident in the support for system-managed tape: DFSMSrmm uses a subsystem-to-subsystem interface to provide as much support as possible without involving the operator.

DFSMSrmm supplies ISPF panels as well as TSO commands. Thus, almost anything you can do online, you can also do in batch through TSO command execution.

DFSMSrmm provides utilities to manage your inventory, create reports, maintain the DFSMSrmm CDS, and erase and initialize volumes.

An application program interface (API) is provided to enable user-written programs to access the DFSMSrmm subsystem, and allows interrogation and update of the subsystem.

Use the EDGHSKP utility, with the DFSMSrmm subsystem active to run inventory management activities, which include:

- ▶ Vital record processing, to determine which data sets to retain and which volume moves are required, based on VRSs
- ▶ Expiration processing, to identify volumes ready to be released and returned to scratch
- ▶ Storage location management processing, to set a destination for a volume. Optionally run storage location management to assign shelf locations in storage locations for volumes that are being sent out or returned to the removable media library
- ▶ Backing up the CDS and the journal, and clearing the journal
- ▶ Creating an extract data set for report generation

EDGHSKP also has some trial run and predictive features that allow a trial-run of inventory management vital record processing without making any changes to the control data set or journal.

Use the EDGUTIL utility to create, update, and verify the CDS. Use the EDGBKUP utility to back up and recover the CDS and journal. Both EDGUTIL and EDGBKUP can be executed independently of the subsystem.

2.1.1 CDS backup and recovery

When backup is performed using the DFSMSdss™ DUMP command, DFSMSrmm backs up the CDS and then the journal. DFSMSrmm resets the journal data set and discards journal records if the back up completes successfully. To restore the CDS using the latest CDS backup, both the latest journal backup and the current journal must be used for forward recovery.

DFSMSrmm clears the journal when the CDS and journal are backed up. Clearing the journal data set prevents the journal from becoming full, and reduces the risk of losing updates to the CDS. Backing up the CDS and the journal using DFSMSdss allows you to write the data directly to tape. Using DFSMSdss concurrent copy allows simultaneous updates to the DFSMSrmm CDS so that other tape activity need not be interrupted.

Note: With DFSMSrmm release z/OS 1.10 or higher you can request that DFSMSrmm back up the control data set and the journal, using the DFSMSdss COPY command.

2.2 Reports

You can obtain information and create reports, using:

- ▶ DFSMSrmm ISPF dialog, TSO RMM commands, or the API
- ▶ EDGAUD, EDGJRPT JCL, and EDGRPTD report utilities
- ▶ EDGRRPTE sample REXX exec to create 17 standard reports
- ▶ DFSORT™'s ICETOOL utility
- ▶ The DFSMSrmm Report Generator

Searches and lists

You can do an online search using either the DFSMSrmm ISPF dialog or TSO RMM subcommands to create lists of resources and display information recorded in the DFSMSrmm CDS. Here are some examples:

- ▶ Operators can create lists of scratch volumes to be pulled for use.
- ▶ Tape librarians and system programmers can create lists of software products and the volumes on which they reside.
- ▶ General users can create lists of volumes they own, shown in the example in Figure 2-2.

Volume	Owner	Rack	Assigned date	Expiration date	Location	Dsets	St	Act	Dest.
DV0001	MHLRES1		2003/182	2003/182	SHELF	0	MR	0	
DV0002	MHLRES1	DV0002	2003/182	2003/182	SHELF	1	MV		
DV0003	MHLRES1	DV0003	2003/182	2003/182	SHELF	1	MV		
DV0004	MHLRES1	DV0004	2003/182	2003/182	SHELF	1	MV		
DV0005	MHLRES1	DV0005	2003/182	2003/182	SHELF	1	MV		
DV0006	MHLRES1	DV0006	2003/182	2003/182	SHELF	1	MV		
DV0007	MHLRES1	DV0007	2003/182	2003/182	SHELF	1	MV		
DV0008	MHLRES1	DV0008	2003/182	2003/182	SHELF	1	MV		
DV0009	MHLRES1	DV0009	2003/182	2003/182	SHELF	1	MV		
DV0010	MHLRES1	DV0010	2003/182	2003/182	SHELF	1	MV		
SEARCH COMPLETE - MORE ENTRIES MAY EXIST									
10	ENTRIES LISTED								

Figure 2-2 List of volumes owned by a single user

With DFSMSrmm, you can use the TSO RMM SEARCH subcommands with the CLIST operand to create a data set of executable subcommands. For example, you can create subcommands to confirm volume movement for volumes identified during a SEARCHVOLUME request.

Report utilities

You can create several types of reports using DFSMSrmm reporting utilities. Use EDGRPTD, EDGJRPT, or the DFSMSrmm Report Generator to create movement and inventory reports, and EDGAUD to create security and audit reports. EDGRPTD uses the DFSMSrmm extract data set as input. EDGJRPT and the Report Generator use the extended report extract file. EDGAUD uses System Management Facility (SMF) records as input.

You can use the reports to:

- ▶ Identify volumes that should be moved between the removable media library and storage locations
- ▶ Determine your volume inventory in the removable media library and storage locations
- ▶ Identify volumes that are in transit or should be marked as moved
- ▶ Identify all accesses to volumes and changes to information recorded in the DFSMSrmm CDS
- ▶ Separate volumes waiting to return to scratch from those that are private or have other release actions pending

You can use DFSORT or a similar program to generate a formatted report using the information in the EXTRACT data set created by the EDGHSKP utility. Also, the ACTIVITY file and the EDGJACTP sample JCL can be used to create a report from it. For example, you can produce an extract data set listing all volumes to be used on VM with information about volume owners. Then use DFSORT's ICETOOL utility to sort the information by volume and produce a report, complete with title and header information.

2.2.1 Security

You can choose the authorization levels of users for all DFSMSRmm functions. DFSMSRmm uses the MVS™ system authorization facility (SAF) for its authorization checking. You define DFSMSRmm resources to z/OS Security Server Resource Access Control Facility (RACF®) for use during authorization checking. DFSMSRmm can create volume profiles, change them, and delete them on registration, expiration, or release of volumes. DFSMSRmm optionally supports security profiles in the RACF DATASET and TAPEVOL classes as a way to authorize the use of certain RMM TSO subcommands against data set and volume information defined in the CDS.

You can use the access list that DFSMSRmm provides to set the access list in RACF, as well as for authorization checking on non-RACF systems. To display the DFSMSRmm access list, use the TSO RMM LISTVOLUME subcommand or the DFSMSRmm ISPF dialog volume list function. You can also find the access list in the volume records in the report extract data set.

DFSMSRmm provides automatic security classification through installation specified criteria based on data set names. It provides the following control of classified volumes:

- ▶ Audit trail of access and change of status through SMF
- ▶ This audit trail produces information about RACF user ID, group, and job name.
- ▶ Operator confirmation of use
- ▶ Erasure of data when a volume is released before it is returned to scratch status

DFSMSRmm provides the following ways of optionally keeping an audit trail for volumes defined to it:

- ▶ Control data set information
- ▶ SMF audit records
- ▶ RACF audit information

DFSMS V1.8 provides new options for securing tape data sets using System Authorization Facility (SAF). These are designed to allow you to define profiles to protect data sets on tape using the DATASET class without the need to activate the TAPEDSN option or the TAPEVOL class. DFSMS also provides options you can use to specify that all data sets on a tape volume should have common authorization and that users are authorized to overwrite existing files on a tape volume.

Note: For optimum tape security use the combined capabilities of DFSMSRmm, DFSMSdftp, and RACF.

2.3 How is DFSMSRmm structured?

In Figure 2-3 on page 28, you can see the basic DFSMSRmm structure. DFSMSRmm runs in its own subsystem address space inside the z/OS operating system. It communicates directly

with the Subsystem Interface (SSI), allowing DFSMSrmm to fully control the tape usage in the system.

A user address space is any task requesting information from DFSMSrmm or any user of tape. For example, this could be a batch job creating a tape data set, or an end user listing owned volumes.

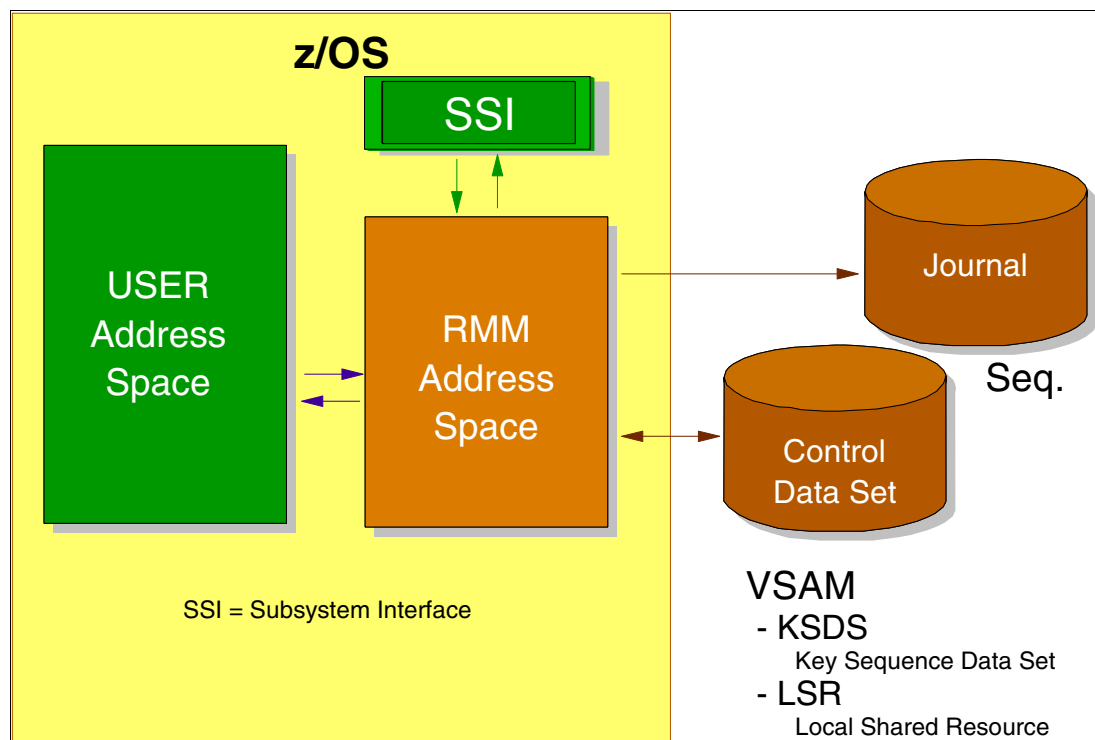


Figure 2-3 DFSMSrmm structure

DFSMSrmm uses the following data sets for its activity:

► **Control data set**

The control data set is a VSAM key-sequenced data set that contains the complete inventory of the removable media library and storage locations. DFSMSrmm records all changes made to the inventory, such as adding or deleting volumes in the control data set. The control data set should be RACF protected to prevent unauthorized access to the data. DFSMSrmm requires CONTROL access to the control data set.

► **Journal**

The journal is a sequential data set, which contains a record of all changes made to the control data set since the last backup.

For information about these data sets, such as structures, size, and definition, refer to 3.1, "DFSMSrmm implementation" on page 70.

When multiple systems exist, then they can share the same control data set. The control data set cannot be a SYS1 data set if the control data set is to be shared. Figure 2-4 on page 29 shows the structure in this environment.

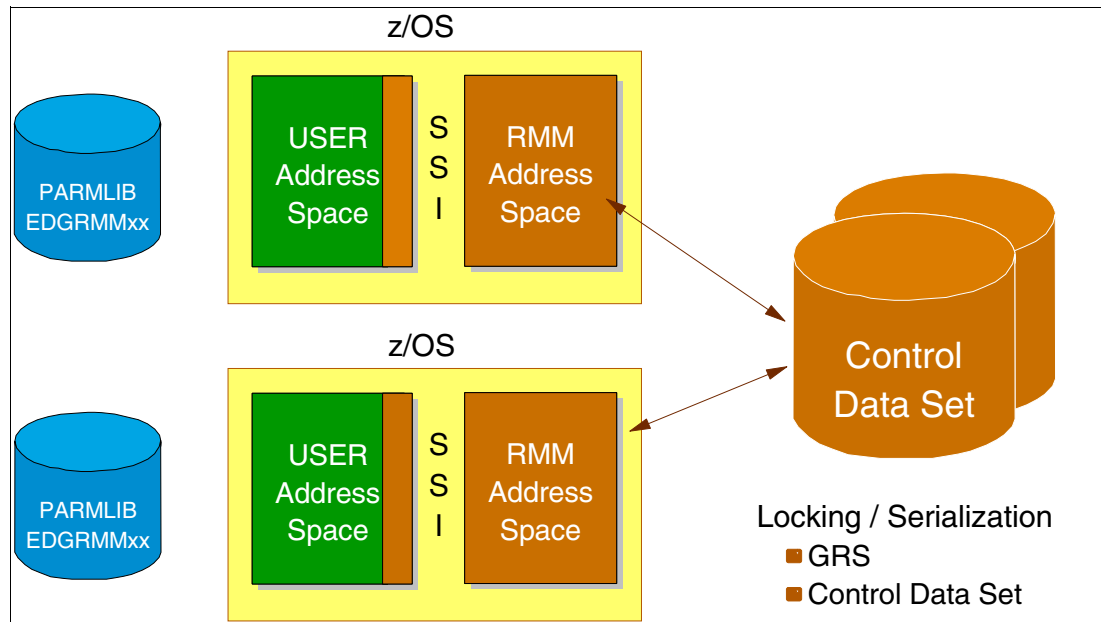


Figure 2-4 DFSMSrmm in a multisystem environment

When multiple systems share one CDS, you can use Global Resource Serialization (GRS) resource definition to solve the possible contention problems. Refer to “GRSRNLxx (optional): GRS resourcename list” on page 102 for details about how to customize GRS reserve and release actions for DFSMSrmm.

2.4 What are the DFSMSrmm interfaces?

DFSMSrmm interacts with different system and non-system components to perform the tape management functions. We introduce three different types of interfaces: software interfaces, user interfaces, and programming interfaces.

Software interfaces are those interfaces between DFSMSrmm and some system components or activities, such as allocation, open, close, and end of volume events (OCE), and between DFSMSrmm and other software components, such as DFSMSdftp™, DFSMSshsm™, DFSMSdss, RACF, catalog, and Object Access Method (OAM).

User interfaces are those that allow the communication between DFSMSrmm and the end user. These user interfaces are the Interactive System Productivity Facility (ISPF) dialogs and TSO commands.

Finally, the programming interfaces allow communication between DFSMSrmm and user applications, or programs, to obtain information from DFSMSrmm or to provide information to DFSMSrmm.

2.4.1 Software interfaces

Figure 2-5 on page 30 shows a general view of what we consider the main DFSMSrmm software interfaces. Using these interfaces, DFSMSrmm either provides support to other users or request services from them. For detailed information about this topic, refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

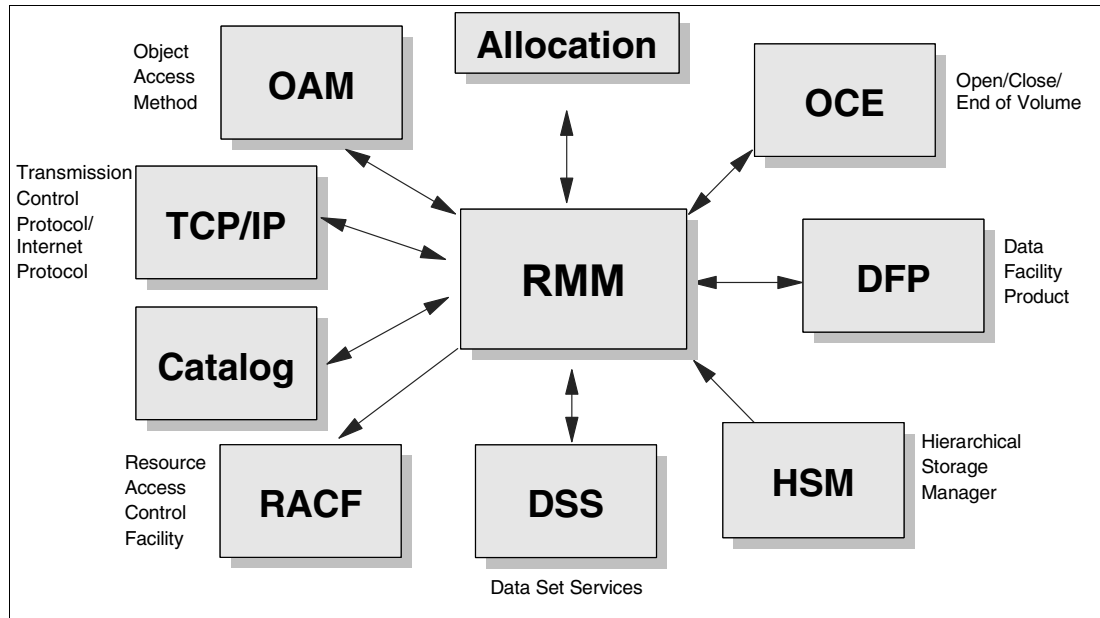


Figure 2-5 Interfaces between DFSMSrmm and other software components

The kind of information that is exchanged between DFSMSrmm and each of the software interfaces, as well as the activities performed, include:

► **Allocation**

At allocation time, DFSMSrmm intercepts the mount message, assigns a scratch pool, and finds the rack where the volume resides. After that, it modifies the mount message to finish the response to this request.

► **Open/close/end of volume interface (OCE)**

DFSMSdfp provides tape exits for vendor or customer use. DFSMSrmm has the capabilities to work with these exits to manage the open, close, and end of volume event of tape. When such events happen, DFSMSrmm performs these actions:

- Validate the mounted tapes
- Store information in the CDS for volume and data sets if authorized
- Return block-ID for high-speed locate
- Reject the volume if:
 - A wrong volume is mounted for a specific volume request.
 - A private volume is mounted in response to scratch request.
 - An attempt is made to read a scratch volume.
 - An attempt is made to overwrite a data set and the data set name does not match.
 - An attempt is made to use a specific scratch volume.
 - The first file of the volume does not match what DFSMSrmm has recorded.
 - An attempt is made to read or write a volume using nonstandard labels.
 - Unauthorized functions are used, such as bypass label processing, write label, and so on.

► **DFSMSdfp interface**

DFSMSrmm provides EDGMSGEX exit as an interface to DFSMSdfp. DFSMSrmm uses the IGXMSGEX exit, the MSGDISP installation exit of DFSMSdfp, to call EDGMSGEX to

retrieve DFSMSrmm information to update tape drive displays and control the use of cartridge loaders.

For system-managed tapes, when the volume is open or closed or an end-of-volume event occurs, the Storage Management Subsystem (SMS) ACS routines determine the volume information.

For non-system-managed tapes, DFSMSrmm calls SMS ACS routines to get information about the storage group for volumes and the management class for data sets.

Figure 2-6 shows the differences between these allocations.

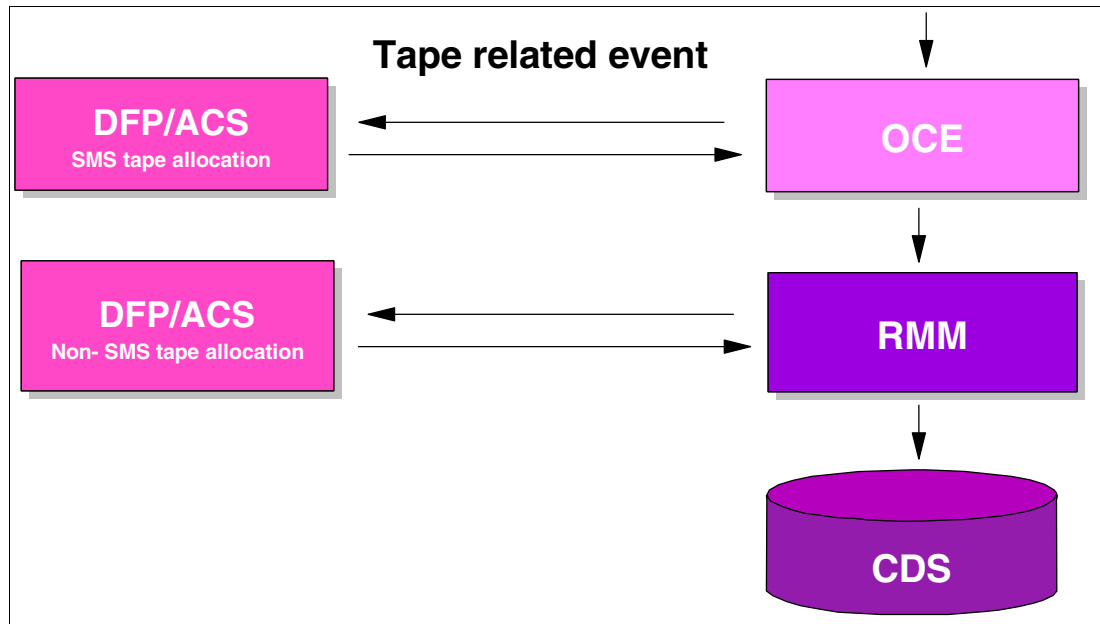


Figure 2-6 SMS tape allocation

► DFSMSHsm and Tivoli® Storage Manager interface

DFSMSrmm provides EDGTVEXT and EDGDFHSM programming interfaces for use by products like DFSMSHsm and Tivoli Storage Manager when using DFSMSrmm tape management.

DFSMSrmm treats DFSMSHsm like any other user, and retains volumes based on retention period and vital record specifications. DFSMSrmm recognizes when DFSMSHsm is opening tape data sets, and for input processing, tolerates the data set names that DFSMSHsm uses as long as the last 17 characters of the data set name match.

For Tivoli Storage Manager, you should define policies (vital record specifications) to cover the Tivoli Storage Manager tape data sets. A single policy specifying the Tivoli Storage Manager high level qualifier is sufficient to include all Tivoli Storage Manager tape data sets.

► DFSMSdss interface

DFSMSrmm makes use of DFSMSdss user interaction module (UIM) interface to enable the usage of DFSMSdss to backup the CDS. When you use DFSMSdss *concurrent copy* for backup, you minimize the time when DFSMSrmm must prevent updates to the CDS.

► RACF interface

DFSMSrmm does not provide any security functions itself but relies on installed security products, such as RACF or equivalent, to process requests. DFSMSrmm uses the MVS

Security Access Facility (SAF) interface to perform authorization checks and other security processing.

You can use RACF to protect DFSMSrmm resources and to authorize DFSMSrmm functions.

DFSMSrmm also provides support for using RACF standard tape volume security protection. You can use any combination of RACF TAPEVOL and TAPEDSN options to protect tape volumes as well as tape data sets.

► **Catalog interface**

DFSMSrmm uses catalog search interface (CSI) to synchronize the control data set with the catalog for tape data sets. It can also exploit catalog exits to mirror catalog updates in the CDS and uncatalog tape data sets according to the definition in PARMLIB.

► **BTLS interface**

If your installation uses BTLS to drive IBM 3494 or IBM 3495 Automated Tape Library (ATL) Dataserver or an IBM TS7700 Virtualization Engine operations, you need to notify BTLS when a volume has to return to SCRATCH status. In your tape management system environment, this is done with a batch job that gets information from your tape management system database and then updates the BTLS catalog. In DFSMSrmm, you can use a simple REXX CLIST that synchronizes the DFSMSrmm database with the BTLS catalog.

► **SMS interface**

If you have an IBM 3494 or an IBM 3495 Automated Tape Library Dataserver or an IBM TS7700 Virtualization Engine, you must ensure that it works correctly in the DFSMSrmm environment. You have to change the OAM exits, using the DFSMSrmm version, and provide the correct input to the extract program for those volumes that belong to the tape library.

DFSMSrmm provides programming interface EDGLCSUX to use the OAM. The OAM installation exits used by DFSMSrmm are CBRUXCUA, CBRUXEJC, CBRUXENT and CBRUXVNL.

When one of the following events happens:

- Change volume use attribute
- Cartridge eject
- Cartridge entry
- Volume-not-in-library

OAM address space uses the corresponding installation exit to send requests to DFSMSrmm. When DFSMSrmm queries information about library or volumes and does some volume related actions such as eject, delete volume and change volume status, DFSMSrmm uses such OAM executable macros as CBRXLCS to send requests to the OAM address space.

- Short-on-scratch processing interface

DFSMSrmm offers an integrated SMS interface that updates the TCDB automatically.

In addition, without manual intervention, you can free volumes that are in PENDING RELEASE status when a short-on-scratch condition is detected inside the ATL. Use the PARMLIB option, SCRATCHPROC(RMMSCR), where RMMSCR is the name of the procedure DFSMSrmm starts to replenish scratch volumes in the ATL.

► **High-speed locate function**

DFSMSrmm now records in the control data set the starting and ending tape block IDs for each data set when it is created. With this new technique, DFSMSrmm enables the use of tape block IDs for applications that do not support them (see Figure 2-7 on page 33).

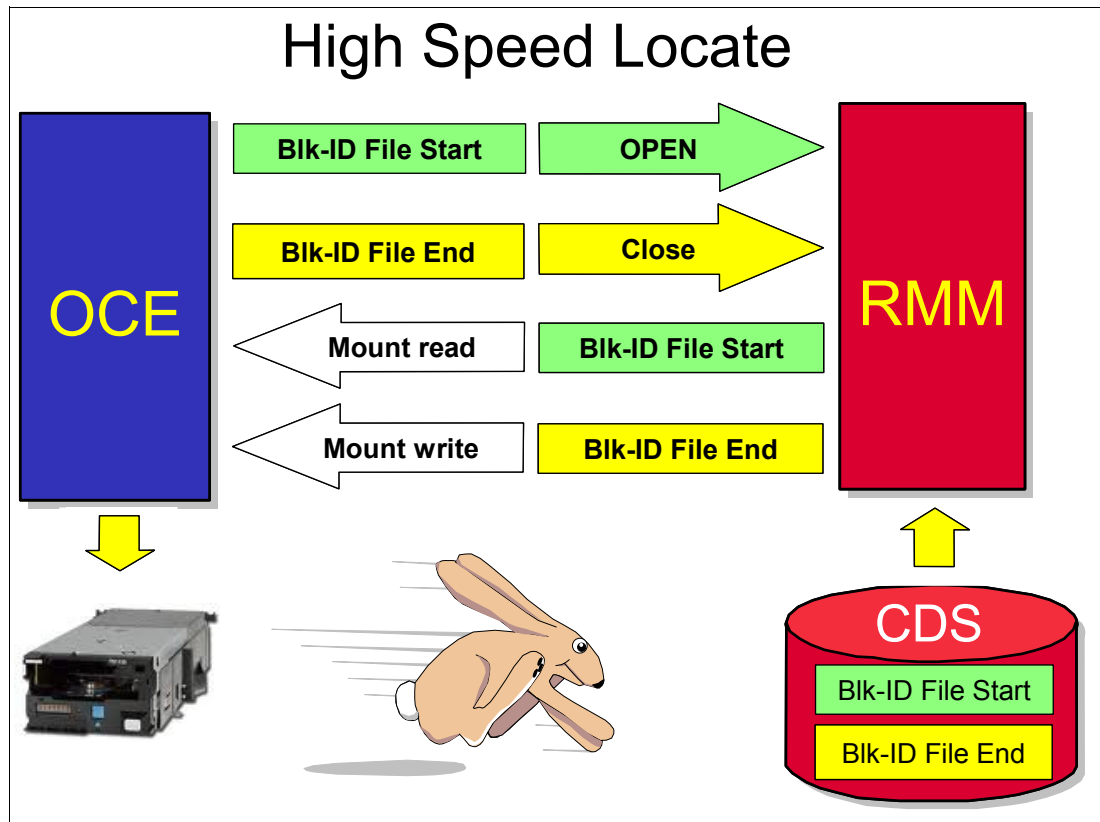


Figure 2-7 High-speed locate overview

The benefit of using this function is that it enables high-speed positioning when there is a large number of data sets in a high capacity tape cartridge.

To gain benefit from this function, you need OS/390 Version 2 Release 10 or higher and DFSMSrmm. The system and DFSMSrmm automatically record and retrieve block IDs, and then use these for high-speed data set positioning whenever it is possible.

2.4.2 User interfaces

DFSMSrmm user interfaces allows the user to interact with DFSMSrmm for performing any of the actions related to the tape management. The user can interact with DFSMSrmm in two ways: ISPF panels and TSO commands.

► ISPF dialogs

The DFSMSrmm ISPF dialog is provided in the ISPF environment, and you can also tailor it to meet your requirements, such as making DFSMSrmm available on any selection panel you want, controlling the access for different type of users. From the ISPF panels, the user can perform most commonly used DFSMSrmm functions, such as define, delete, display volumes, policies, data sets. Figure 2-8 on page 34 shows you the result of a SEARCHVOLUME request to list all volume4s beginning with VT*, have a status of MASTER and stored in the location VTFM001.

DFSMSrmm Volumes (Page 1 of 2)						Row 1 to 21 of 300		
Command ==>						Scroll ==> CSR		
Enter HELP or PF1 for the list of available line commands								
Use the RIGHT command to view other data columns								
S	Volume serial	Owner	Assigned date	Expir./ Retn. date	S R Status	Location	Dest- ination	Tr- ans Data sets
--	-----		-----	-----	-----	-----	-----	-----
	VT0000		2011/324		SCRATCH	VTFM001		N 1
	VT0001		2011/324		SCRATCH	VTFM001		N 1
	VT0002	STC	2010/293	PERMANENT	VRS	VTFM001		N 1
	VT0003		2011/324		SCRATCH	VTFM001		N 1
	VT0004		2011/324		SCRATCH	VTFM001		N 1
	VT0005	STC	2010/313	PERMANENT	VRS	VTFM001		N 1
	VT0006	STC	2010/313	PERMANENT	VRS	VTFM001		N 1
	VT0007	STC	2010/313	PERMANENT	VRS	VTFM001		N 1
	VT0008	MHLRES7	2011/324	2011/333	MASTER	VTFM001		N 1
	VT0009	STC	2010/313	PERMANENT	VRS	VTFM001		N 1
	VT0010	STC	2010/286	PERMANENT	VRS	VTFM001		N 1
	VT0011	STC	2010/286	PERMANENT	VRS	VTFM001		N 1
	VT0012	STC	2010/288	PERMANENT	VRS	VTFM001		N 1
	VT0013	MHLRES7	2011/324	2011/333	MASTER	VTFM001		N 1
	VT0014	STC	2010/288	PERMANENT	VRS	VTFM001		N 1
	VT0015	MHLRES7	2011/324	PERMANENT	VRS	VTFM001		N 1
	VT0016	MHLRES7	2011/324	PERMANENT	VRS	VTFM001		N 1
	VT0017	MHLRES7	2011/325	2011/334	MASTER	VTFM001		N 1
	VT0018	STC	2010/293	PERMANENT	VRS	VTFM001		N 1
	VT0019	STC	2010/293	PERMANENT	VRS	VTFM001		N 1
	VT0020	STC	2010/293	PERMANENT	VRS	VTFM001		N 1

Figure 2-8 ISPF SEARCHVOLUME result

► TSO commands

DFSMSrmm allows you to use a TSO command to define resources, manage resources and create reports. You use the command shown in Figure 2-9 to add a new VRS.

```
RMM ADDVRS DSN('YJCRES4.TEST1.**') DAYS COUNT(1)
```

Figure 2-9 RMM ADDVRS subcommand

► TSO commands from the console

The MVS modify command can now be used to issue an RMM TSO command from the console. The command output is returned to the console and system log.

Figure 2-10 shows you a sample command.

```
F DFRMM,CMD=LISTCONTROL ALL
```

Figure 2-10 RMM TSO command from the console

2.4.3 Application programming interface

DFSMSrmm provides an API that allows customers to write programs to obtain services from DFSMSrmm. Figure 2-11 on page 35 shows the relationship between DFSMSrmm API and other software components. Using the API, you can issue any of the RMM TSO subcommands from an assembler program.

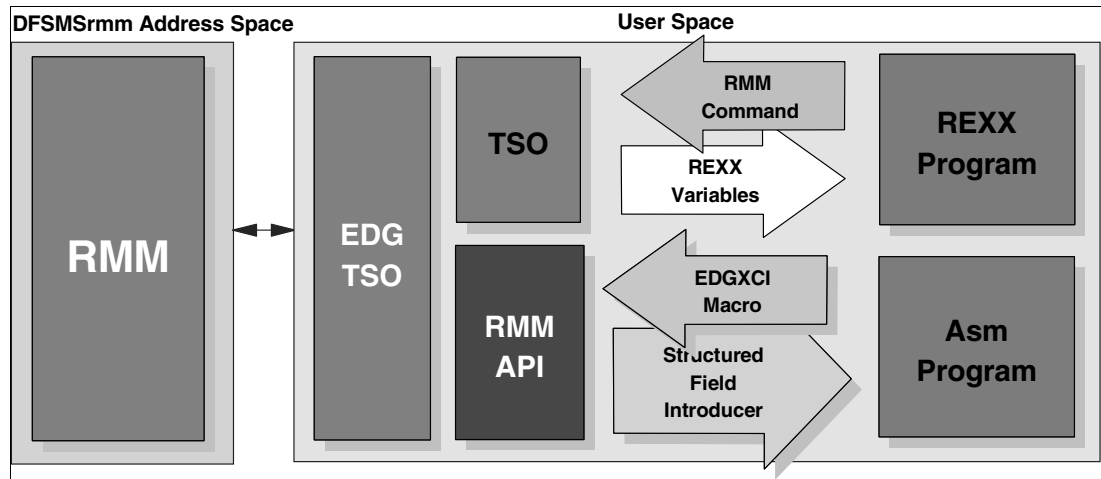


Figure 2-11 DFSMSrmm API

You can use the API to obtain information about DFSMSrmm resources and use the data to create reports or to implement automation. To use the DFSMSrmm API, you must have High Level Assembler installed on your system, and code assembler language programs. For more information about the API, refer to *DFSMSrmm Application Programming Interface*, SC26-7403.

With DFSMSrmm release z/OS 1.6 or later, you can use C++ and other high-level programming languages to write programs to obtain information about DFSMSrmm resources. You use the same DFSMSrmm subcommand strings that you can use with the EDGXCI application programming interface.

2.4.4 TCP/IP interface

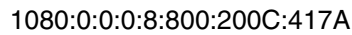
DFSMSrmm uses the TCP/IP interface when implementing the RMMplex (see 2.13, “Implementing the RMMplex” on page 67, for more information). This enables DFSMSrmm to share a control data set between multiple systems without having a shared DASD environment.

IPv6 Supports networking strategy

DFSMSrmm on z/OS V1R12 and later releases is an IPv6 enabled application that supports both IPv4 and IPv6 sockets. You can continue to use IPv4 on all systems or you can run a mixed environment with one or more V1R12 systems using IPv6 and other systems using IPv4. Once all systems are V1R12, you have the option of moving all systems to IPv6. In a mixed environment, dual-mode IP stacks are required. You can use IP addresses that are compliant to either IPV4 or IPV6.

Note: DFSMSrmm client/server processing is dependent on Internet Protocol V4.

Figure 2-12 shows you an example of a IPV6 address.



```
1080:0:0:0:8:800:200C:417A
```

Figure 2-12 *IPv6 address*

2.5 Library and storage location management

You decide where to store your removable media according to how often the media is accessed and for what purpose it is retained. For example, you might keep volumes that are frequently accessed in an ATL, and you probably use at least one *storage location* or vault to retain volumes for disaster recovery and audit purposes. You might also have locations where volumes are sent for further processing. These locations could be data centers within your company or customer and vendor sites.

DFSMSrmm manages:

- ▶ **Removable media library**, which contains all tape and optical volumes that are available for immediate use and includes the shelves where they reside. A removable media library usually includes other libraries, such as:
 - System-managed tape libraries; for example, the IBM 3494 and IBM Virtual Tape Server (VTS) Automated Tape Library Dataserver models, the IBM TS7700 Virtualization Engine and manual tape libraries.
 - Non-system-managed tape libraries, IBM System Storage VTF Mainframe, vendor robotic systems or traditional tape libraries.
- ▶ **Storage locations** that are not part of the removable media library because their volumes are not generally available for immediate use. Storage locations are typically used to store removable media that are kept for disaster recovery or vital records.
- ▶ **Loan locations**, which are used to track volumes sent to third parties.

Figure 2-13 shows the relationship of the libraries and storage locations.

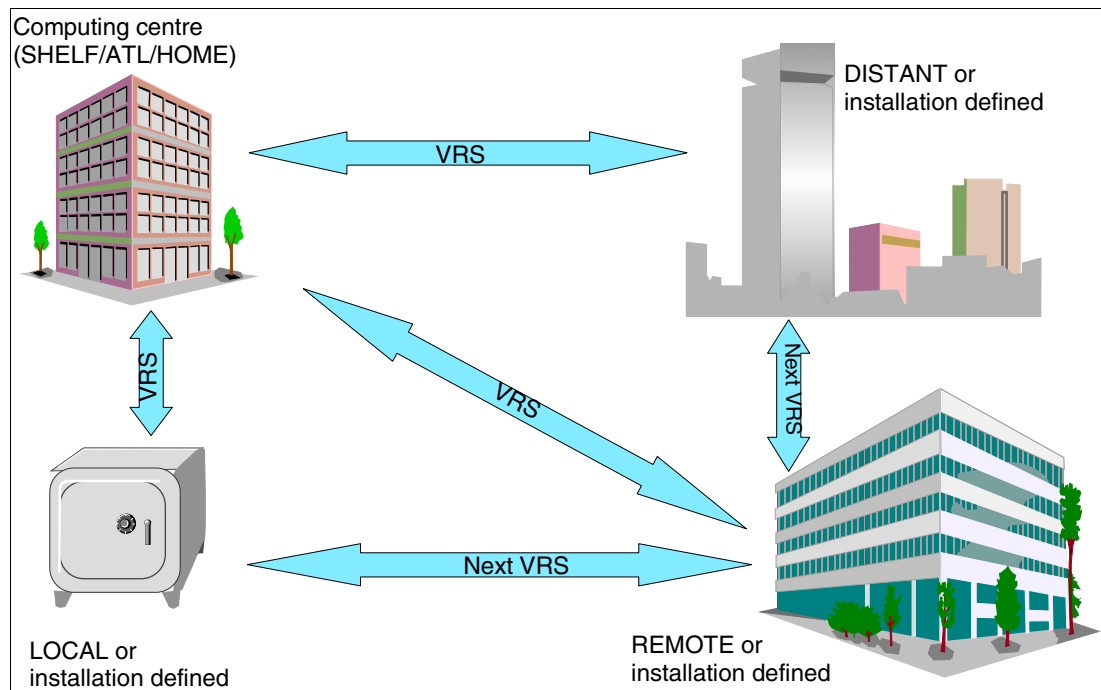


Figure 2-13 Removable media libraries and storage locations

2.5.1 Removable media library

A *removable media library* contains all the tape and optical volumes that are available for immediate use, including the shelves where they reside.

A removable media library usually includes other libraries: *system-managed libraries* such as *automated or manual tape libraries*; and *non-system-managed libraries*, containing the volumes, shelves, and drives not in an automated or a manual tape library.

In the removable media library, you can store your volumes in shelves, where each volume occupies a single *shelf location*. This shelf location is referred to as a *rack number* in the TSO RMM subcommands and in the DFSMSrmm ISPF dialog.

A rack number matches the volume's external label. DFSMSrmm uses the volume serial number to assign a rack number when adding a volume, unless you specify otherwise. The format of the volume serial you define to DFSMSrmm must be one to six alphanumeric characters. The rack number must be six alphanumeric or national characters. This information is illustrated in Figure 2-14.

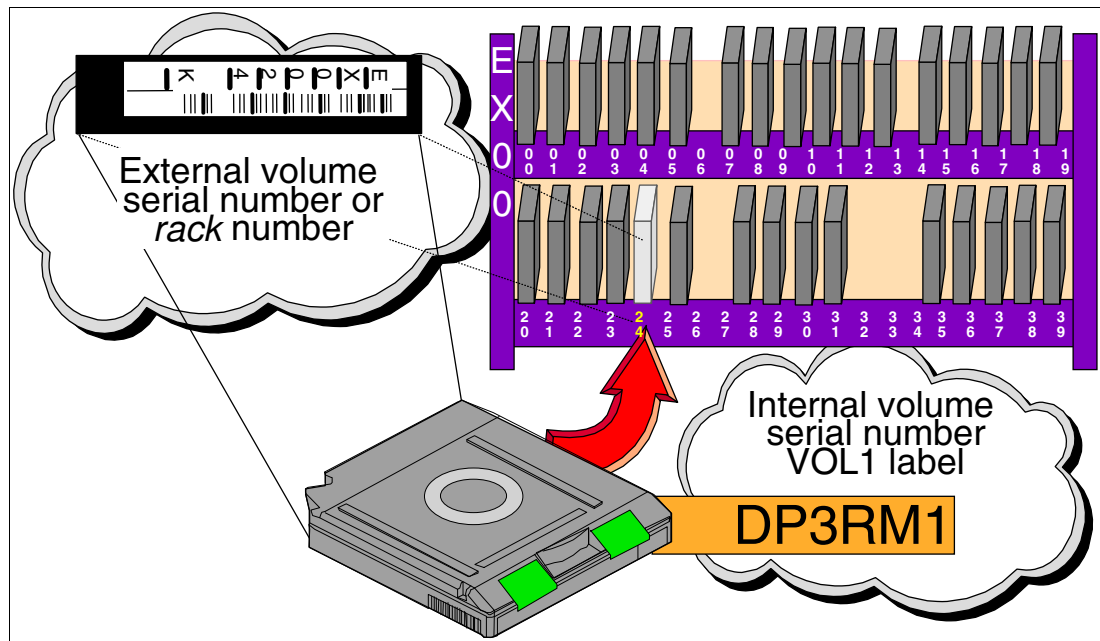


Figure 2-14 Internal labels, external labels, and rack numbers

System-managed tape library

A system-managed tape library is a collection of tape volumes and tape devices, defined in the tape configuration database (TCDB). The TCDB is an ICF catalog of type VOLCAT. It is a new SMS control data set containing tape library and tape volume records.

A system-managed tape library can be either automated or manual. You can have several automated or manual tape libraries. Use an installation-defined library name to define each ATL or MTL to the system. DFSMSrmm treats each system-managed tape library as a separate location or destination.

Automated tape library

An *automated tape library* consists of robotics components, cartridge storage areas (or shelves), tape subsystems, and controlling hardware and software, together with the set of tape volumes that reside in the library and can be mounted on the library tape drives. DFSMSrmm provides support for the automated IBM 3494 Tape Library Dataserver, the IBM 3495 Tape Library Dataserver, and also the IBM TS7700 Virtualization Engine.

The IBM 3494 Tape Library Dataserver supports 3490E, 3590 and 3592 tape subsystems. The IBM 3495 Tape Library Dataserver supports 3490, 3490E, and 3590 tape subsystems. The IBM 3494 Tape Library Dataserver can also include the 3494 VTS. DFSMSrmm supports the IBM TS7700 grid and the IBM Peer-to-Peer VTS by ensuring that you cannot use the names of distributed VTS libraries with DFSMSrmm. You must only use the names of consolidated (composite) libraries with DFSMSrmm.

Figure 2-15 shows an overview of the system managed tape environment.

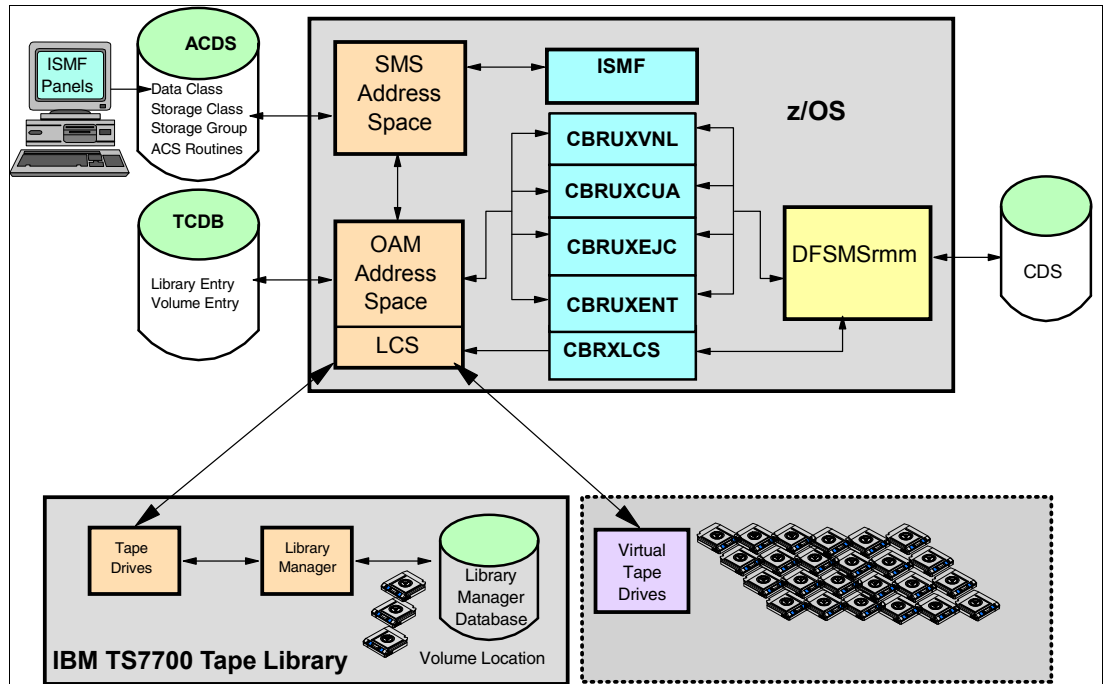


Figure 2-15 System managed tape overview

Multi-cluster grid terms

This section covers terms that are specific to a multi-cluster grid configuration.

Composite library

The composite library is the logical image of the grid that is presented to the host. It is distinctly different from the IBM Virtual Tape Server. A stand-alone cluster TS7700 Virtualization Engine has a five-character hexadecimal Composite Library ID defined. A composite library is presented to the host with both TS7700 Virtualization Engine stand-alone and multi-cluster grid configurations. In the case of a stand-alone TS7700 Virtualization Engine, the host sees a logical tape library with sixteen 3490E tape control units. These units each have sixteen IBM 3490E tape drives, and are attached through two or four FICON channel attachments.

Figure 2-16, "TS7700 Virtualization Engine three-cluster grid configuration" illustrates the host view of a three-cluster grid configuration.

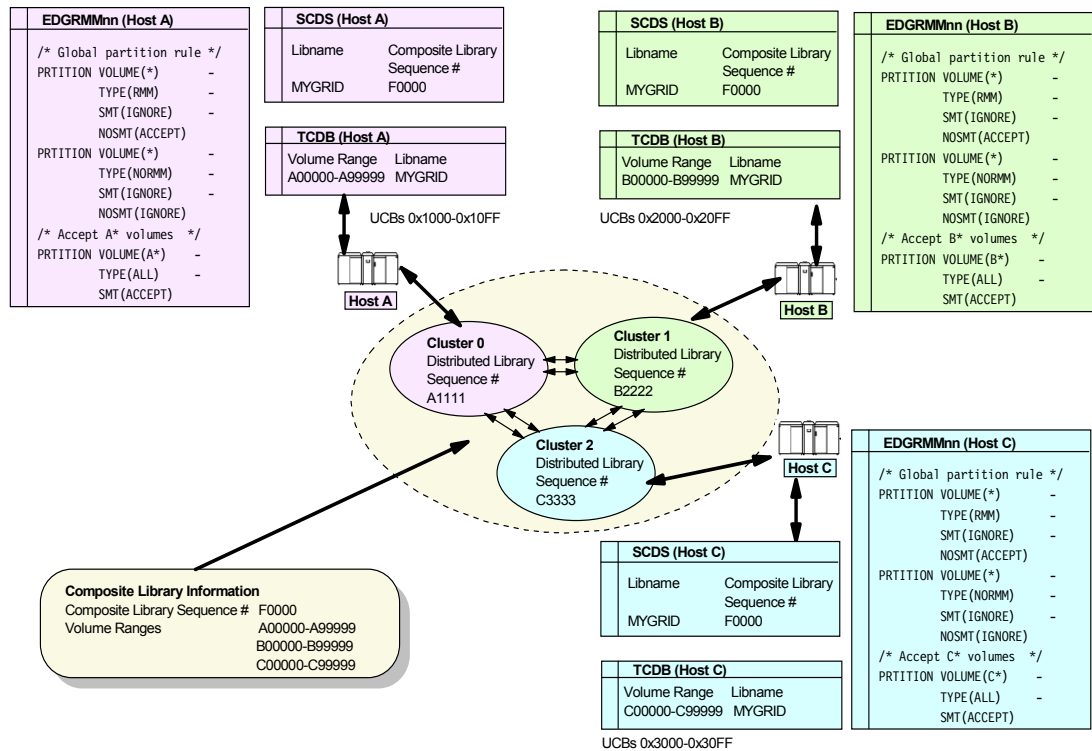


Figure 2-16 TS7700 Virtualization Engine three-cluster grid configuration

Important: A Composite Library ID must be defined both for a multi-cluster grid and a stand-alone cluster. For a stand-alone cluster, the Composite Library ID must not be the same as the Distributed Library ID. For a multiple grid configuration, the Composite Library ID must differ from any of the unique Distributed Library IDs. Both the Composite Library ID and Distributed Library ID are five-digit hexadecimal strings.

Distributed library

Each cluster in a grid is a distributed library, which consists of a TS7700 Virtualization Engine. In the case of a TS7740 Virtualization Engine, it is also attached to a physical TS3500 tape library.

A distributed library consists of the following cluster hardware components:

- A Virtualization Engine
- A TS7700 tape volume cache
- A 3952-F05 frame
- Attachment to a physical library (TS7740 Virtualization Engine only)
- A number of physical tape drives (TS7740 Virtualization Engine only)

A grid configuration with multiple TS7700 Virtualization Engines must have a distributed library defined at the host for each cluster. Each of the engines must have the Grid Enablement feature installed. The host has sufficient knowledge about the distributed libraries to allow appropriate console message handling from the corresponding cluster. At the host, the distributed library is only

defined to SMS. It is defined using the existing ISMF windows and has no tape devices defined. The virtual tape devices are defined for the composite library only.

Copy consistency point

In a grid environment, you can specify where and when you want to have a copy of your data. Currently, the three settings consist of two copy consistency points and an option to have no copy at all:

RUN	The copy occurs as part of the rewind/unload (RUN) operation and completes before the rewind/unload operation at the host finishes. This mode is comparable to the immediate copy mode of the Peer-to-Peer VTS.
Deferred	The copy occurs after the rewind/unload operation at the host. This mode is comparable to the deferred copy mode of the Peer-to-Peer VTS.
No Copy	No copy is made.

On each cluster in a multi-cluster grid, a copy consistency point setting is specified for the local cluster and one for each of the other clusters. The settings can be different on each cluster in the grid. When a volume is mounted on a virtual tape device, the copy consistency point policy of the cluster to which the virtual device belongs is honored.

Dynamic Grid Load Balancing

Dynamic Grid Load Balancing is an algorithm that is used within the TS7700 Virtualization Engine. It continually monitors and records the rate at which data is processed by each network link. Whenever a new task starts, the algorithm uses the stored information to identify the link that will most quickly complete the data transfer. The algorithm also identifies degraded link performance and sends a warning message to the host.

Dynamic Grid Load Balancing

Dynamic Grid Load Balancing is an algorithm that is used within the TS7700 Virtualization Engine. It continually monitors and records the rate at which data is processed by each network link. Whenever a new task starts, the algorithm uses the stored information to identify the link that will most quickly complete the data transfer. The algorithm also identifies degraded link performance and sends a warning message to the host.

Manual tape library

A manual tape library (MTL) is an installation-defined set of tape drives and a user-defined set of tape volumes with mount capability on those tape drives. The volumes can be physically stored in shelf storage located near the MTL, but since these volumes are specifically defined as residing in the MTL, they are known as library-resident volumes. When the volumes are logically ejected from the MTL, they become shelf-resident volumes. In an MTL environment, the operator or tape librarian responds to commands at the MVS console, manually loading and unloading the tape cartridges.

Before a tape cartridge can be used, the tape cartridge must first be logically entered into an MTL. Cartridges can be entered into an MTL through invocation of the CBRXLCS manual cartridge entry (MCE) general use programming interface, or through invocation of the LIBRARY ENTER command.

DFSMSrmm can initiate the MCE process. DFSMSrmm uses MCE processing to add information about a volume residing in an MTL into the TCDB. The RMM ADDVOLUME or CHANGEVOLUME subcommands with the MEDIATYPE operand can be used to define the volume to the MTL. You must ensure that DFSMSrmm has the correct media type for the volume to avoid allocation and mount problems when the volumes are used (refer to Figure 2-19 on page 44 and Table 2-1 on page 46).

Note: You can setup an IBM System Storage VTF Mainframe as a MTL.

Non-system-managed tape library

A non-system-managed tape library encompasses all the volumes, shelves, and drives not in an automated or manual tape library. You might know this library as the *traditional* tape library. DFSMSrmm provides complete tape management functions for the volumes and shelves in this traditional tape library. DFSMSrmm defines volumes in a non-system-managed library as being *shelf* resident.

All tape media and drives that the operating system supports are supported in a non-system-managed tape library environment. Using DFSMSrmm, you can fully manage all types of tapes in a non-system-managed tape library, including 3420 reels and 3480, 3490, 3590 and 3592 (TS1120, TS1130 and TS1140) cartridge system tapes.

2.5.2 Storage location

A storage location comprises shelf locations that you define to DFSMSrmm. **A shelf location in a storage location is identified by a *bin* number.** DFSMSrmm manages two types of storage locations: installation-defined storage locations and DFSMSrmm built-in storage locations. For more information about built-in storage locations, refer to Chapter 6, “Managing Storage Locations”, in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

You can define an unlimited number of installation-defined storage locations. You can use up to eight-character alphanumeric names for a storage location and define the type or shape of the media in the location, and the bin numbers that DFSMSrmm assigns to the shelf locations in the location. You can request DFSMSrmm shelf management when you want DFSMSrmm to assign a specific shelf location to a volume in the location.

You can now define that a storage location is a home location, allowing volumes to be scratched while in that location. You can use a storage location defined as a home location for all volumes stored in a non-IBM robot tape library.

You can also use the DFSMSrmm built-in storage locations, LOCAL, DISTANT, and REMOTE. Although the names of these locations imply their purpose, they do not mandate their actual location. All volumes can be in the same or separate physical locations.

For example, an installation can have the LOCAL storage location onsite, as a vault in the computer room, the DISTANT storage location can be a vault in an adjacent building, and the REMOTE storage location can be a secure facility across town. An installation defined storage location of SHELF can be onsite, and used for volume movement within a library to make better use of the library. Because DFSMSrmm provides shelf management for storage locations, storage locations can be managed at the shelf location level. Optionally, you can define storage locations that are not shelf-managed.

Although DFSMSrmm automatically shelf-manages built-in storage locations, you must first define the bins you want to use to DFSMSrmm. For bin numbers in built-in storage locations, the numbers are fixed in range, starting at bin number 000001. For installation-defined storage locations, a bin number is any six alphanumeric or national characters in any combination.

2.6 HOME location

Starting with z/OS V1.3 DFSMSrmm you can define any location as a home location for your volumes. To do this, specify each location in your EDGRMMxx PARMLIB member as shown in Figure 2-17 on page 43.

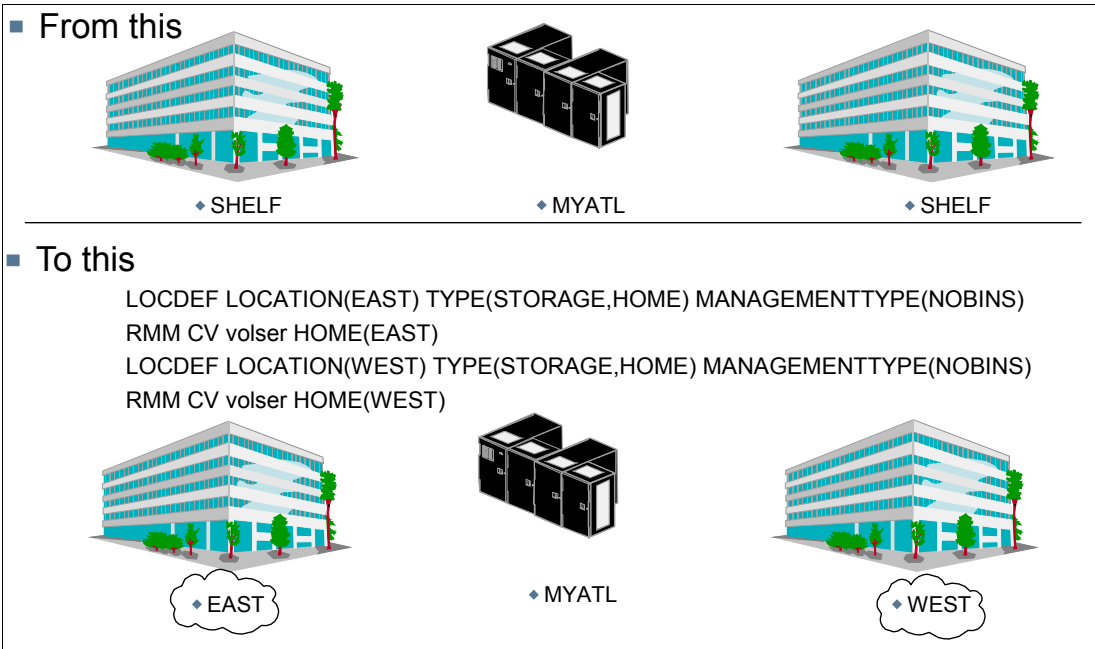
LOCDEF LOCATION(EAST) TYPE(STORAGE,HOME) AUTOMOVE(YES)
--

Figure 2-17 Define a storage location as a home location

When you identify a storage location as a home location, you can manage volumes in a storage location, such as volumes, that reside in a LIBRARY location. You can:

- ▶ Schedule release actions for volumes when they return to their storage home location
- ▶ Return volumes to scratch status while they reside in their storage home location
- ▶ allow automated movement when the **required location** does not match the current location of the volume

Figure 2-18 shows the choices made between z/OS V1.3 DFSMSrmm and previous DFSMSrmm releases. In previous DFSMSrmm releases, the home location was always called SHELF if it was not an automated or manual tape library.



2.7 Media types

The media type specifies the type of physical media of the volumes defined to DFSMSrmm.

DFSMSrmm uses the volume's media type as defined to DFSMSrmm when requesting manual cartridge entry; you must ensure that DFSMSrmm has the correct media type to avoid allocation and mount problems when the volumes are later used. You use the RMM ADDVOLUME or CHANGEVOLUME subcommands with the MEDIATYPE operand to set the media type.

Figure 2-19 on page 44 shows the different media types and the terms used in SMS and DFSMSrmm.

Cartridge Type	CST 3480/3490	CST 3490	ECCST 3490	HPCT 3590	HPCT 3590	HPCT 3590	EHPCT 3590	EHPCT 3590	EHPCT 3590
Tracks	18	36	36	128	25	384	128	256	384
Native Compacted	200 MB 600 MB	400 MB 1.2 GB	800 MB 2.4 GB	10 GB 30 GB	20 GB 60 GB	30 GB 90 GB	20 GB 60 GB	40 GB 120 GB	60 GB 180 GB
ATL Media Label	1 or blank	1 or blank	E	J	J	J	K	K	K
OAM Media Type	01	01	02	03	03	03	04	04	04
SMS Tape Category	MEDIA1	MEDIA1	MEDIA2	MEDIA3	MEDIA3	MEDIA3	MEDIA4	MEDIA4	MEDIA4

The diagram below the table illustrates the physical media types corresponding to the rows in the table. It includes illustrations of various tape and diskette formats, such as IBM 3494, Virtual Tape Server, CST, ECCST, and IBM 3590 Cartridge.

Figure 2-19 Different cartridge media types

Figure 2-20 shows how you can use the different cartridges on all the available tape drives.

Cartridge Type	CST	CST	ECCST	HPCT	HPCT	HPCT	EHPCT	EHPCT	EHPCT
Tracks	18	36	36	128	256	384	128	256	384
3480 without IDRC	read & write	rewrite*	-	-	-	-	-	-	-
3480/3490 with IDRC	read & write	rewrite*	-	-	-	-	-	-	-
3490E writes 36 tracks	read or rewrite*	read & write	read & write	-	-	-	-	-	-
3590 B1x writes 128 tracks	-	-	-	read & write	rewrite*	rewrite*	read & write	rewrite*	rewrite*
3590 E1x writes 256 tracks	-	-	-	read or rewrite*	read & write	rewrite*	read or rewrite*	read & write	rewrite*
3590 H1x writes 384 tracks	-	-	-	read or rewrite*	read or rewrite*	read & write	read or rewrite*	read or rewrite*	read & write

Figure 2-20 IBM 3480 and IBM 3490 tape drive matrix

Note: To reformat 384-track format cartridges to 128-/256-track format, you need the following EC levels when rewriting a tape in a format different from the current format:

- ▶ 3590B = D19328
- ▶ 3590E = F23079

2.7.1 Tape Cartridges used in the 3592 Tape Drives

The capacity of the 3592 tape cartridge depends on the format used when writing from beginning of tape (BOT). Each tape drive model has different formatting capabilities. The J1A can only read or write in Enterprise Format 1 (EFMT1). The E05 can only read or write in EFMT1 or EFMT2. The E06/EU6 can read tapes formatted in EFMT1, EFMT2, or EFMT3. However, the E06/EU6 can only write using EFMT2 or EFMT3. The E07 can read tapes formatted in EFMT2, EFMT3, or EFMT4. However, the E07 can only write in EFMT3, or EFMT4. Furthermore, the E07 can neither read nor write using type JA, JJ, JR, or JW cartridges.

All 3592 tape drives support cartridge reuse. The 3592 tape cartridges can be reformatted to any tape format supported by the tape drive when writing from BOT. When reformatting, all existing data on the cartridge is erased. All compatible 3592 tape cartridge types can be used with the encryption function of the 3592 Models E05, E06, and E07. Encrypted cartridges use a unique format, Enterprise Encrypted Format 2 (EEFMT2), Enterprise Encrypted Format 3 (EEFMT3), or Enterprise Encrypted Format 4 (EEFMT4). Table 2-1 shows you the new IBM 3592 tape matrix.

Table 2-1 ITypes of IBM 3592 tape cartridges

Product, Type, SMS Category	Native capacity				Part number
	E07	E06/EU6	E05	JA1	
Data, JA, MEDIA5, (610 m)	640 GB (596.04 GiB) E06 format ⁴	640 GB (596.04 GiB) E06 format	500 GB (465.66 GiB) E05 format	300 GB (279.39 GiB) (J1A format)	18P7534
	500 GB (465.66 GiB) E05 format	500 GB (465.66 GiB) E05 format			
	300 GB (279.39 GiB) J1A format. ⁵	300 GB (279.39 GiB) J1A format.	300 GB (279.39 GiB) J1A format.		
Extended Data, JB, MEDIA9, (825 m)	1600 GB (1490.12 GiB) E07 format	1000 GB (931.32 GiB)	700 GB (651.93 GiB)	Not supported	23R9830
	1000 GB (931.32 GiB) E06 format				
Advanced Data, JC, MEDIA11, (825 m)	4000 GB (3725.29 GiB) E07 format	Not supported	Not supported	Not supported	46X7452
Economy, JJ, MEDIA7, (246 m)	128 GB (119.21 GiB) E06 format ⁵	128 GB (119.21 GiB) E06 forma	100 GB (93.13 GiB) E05 format	60 GB (58.88 GiB) J1A format	24R0317
	100 GB (93.13 GiB) E05 format	100 GB (93.13 GiB) E05 format			
	60 GB (58.88 GiB) J1A format ⁵	60 GB (58.88 GiB) J1A format	60 GB (58.88 GiB) J1A format		
Advanced Economy, JK, MEDIA13, (146 m)	500 GB (465.66 GiB) E07 format	Not supported	Not supported	Not supported	46X7453
Economy WORM, JR, MEDIA8, (246 m)	128 GB (119.21 GiB) E06 format ⁵	128 GB (119.21 GiB) E06 forma	100 GB (93.13 GiB) E05 format	60 GB (58.88 GiB) J1A format	24R0317
	100 GB (93.13 GiB) E05 format	100 GB (93.13 GiB) E05 format			
	60 GB (58.88 GiB) J1A format	60 GB (58.88 GiB) J1A format	60 GB (58.88 GiB) J1A format		

Product, Type, SMS Category	Native capacity				Part number
	E07	E06/EU6	E05	JA1	
WORM, JW, MEDIA6, (610 m)	640 GB (596.04 GiB) E06 format ⁴	640 GB (596.04 GiB) E06 format ⁴	500 GB (465.66 GiB) E05 format	300 GB (279.39 GiB) J1A format.	18P7538
	500 GB (465.66 GiB) E05 format	500 GB (465.66 GiB) E05 format			
	300 GB (279.39 GiB) J1A format. ⁵	300 GB (279.39 GiB) J1A format.	300 GB (279.39 GiB) J1A format.		
Extended WORM, JX, MEDIA10, (825 m)	1600 GB (1490.12 GiB) E07 forma	1000 GB (931.32 GiB) E06 forma	700 GB (651.93 GiB)	Not supported	23R9831
	1000 GB (931.32 GiB) E06 forma				
Advanced WORM, JY, MEDIA12, (880 m)	4000 GB (3725.29 GiB) E07 format	Not supported	Not supported	Not supported	46X7454
Cleaning, CLNxxxJA3	N/A ⁴	N/A ⁴	N/A ⁴	N/A ⁴	18P7535
Notes: ¹ This designation appears as the last two characters on standard bar code labels. In addition, for cleaning cartridges, the first three characters of the volume serial number (VOLSER) are CLN. ² The Data type cartridge can also be ordered in a 260 GB (242.14 GiB) segmented, capacity scaled format providing 64.42 GB (60 GiB) of high-performance random access, and an additional 200 GB (186.26 GiB) of capacity. ³ Where xxx equals three numerals. ⁴ N/A = not applicable. ⁵ The 3592-E07 supports reading JA, JJ, JR, and JW cartridges only with code level D3I3_5CD or later					

2.7.2 IBM 3592 Tape Cartridge read and write formats

This topic provides general information on the IBM 3592 tape cartridge.

The 3592 tape drive has a bidirectional read/write head capable of operating at four different recording densities, depending on the tape drive model. The 3592 E07 reads data in EFMT2 (896 tracks on 16 channels), EFMT3 (1152 tracks on 16 channels), and EFMT4 (2176 tracks on 32 channels) but only writes EFMT3 and EFMT4. The 3592 E06 reads data in EFMT1 (512 tracks on 8 channels), EFMT2, and EFMT3, but only writes EFMT2 and EFMT3. The 3592 E05 drive reads and writes EFMT1 and EFMT2. The 3592 J1A reads and writes only EFMT1. Table 2-2 shows you the new IBM 3592 recording format.

Table 2-2 Supported 3592 read (R) and write (W) formats

IBM 3592 Tape Drive	EFMT1 512 Tracks / 8 Channels	EFMT2 896 Tracks / 16 Channels	EFMT3 1152 Tracks / 16 Channels	EFMT4 2176 Tracks / 32 Channels
Model J1A	R/W	Not supported	Not supported	Not supported
Model E05	R/W	R/W ¹	Not supported	Not supported
Model E06 ²	R	R/W	R/W	Not supported
Model E07 ²	R ⁴	R ⁴	R/W ³	R/W
¹ Model E05 can read and write EFMT1 operating in native or J1A emulation mode. ² Model E06 and E07 do not support emulation. ³ Cartridge types JB and JX only. ⁴ Model E07 can read JA, JJ, JR, and JW cartridge types only with code level D3I3_5CD or higher.				

2.7.3 DFSMSrmm IBM Media Information

The Media type field specifies the type of physical media of the volumes defined to DFSMSrmm. For media information other than IBM the media type is based on the installation-defined media information.

Possible values (Examples for media information IBM):

*	The volume is not a cartridge.
CST	Cartridge System Tape
ECCST	Enhanced Capacity Cartridge System Tape
EHPCT	Extended High Performance Cartridge Tape
HPCT	High Performance Cartridge Tape
MEDIA5/ETC	IBM Enterprise Tape Cartridge
MEDIA6/EWTC	IBM Enterprise WORM Tape Cartridge 3592
MEDIA7/EETC	IBM Enterprise Economy Tape Cartridge 3592
MEDIA8/EEWTC	IBM Enterprise Economy WORM Tape Cartridge 3592
MEDIA9/EXTC	IBM Enterprise Extended Tape Cartridge 3592
MEDIA10/EXWTC	IBM Enterprise Extended WORM Tape Cartridge 3592
MEDIA11/EATC	IBM Enterprise Advanced Tape Cartridge
MEDIA12/EAWTC	IBM Enterprise Advanced WORM Tape Cartridge
MEDIA13/EAETC	IBM Enterprise Advanced Economy Tape Cartridge

2.7.4 IBM 3592 Cartridge memory

This topic describes the cartridge memory (CM) of the 3592 tape cartridge.

Each 3592 data cartridge contains a passive, contactless, silicon storage device called CM. This CM holds information about the cartridge and the media in the cartridge, and holds statistics about the media in the cartridge. The cartridge and media information is stored in a protected, read-only area of the CM. This information is read by the CM reader in the drive, by using a contactless, radio-frequency interface, when the cartridge is loaded into the drive. The media performance statistics are stored in an unprotected, read/write area of the CM. These statistics are updated by the CM reader just before the cartridge is unloaded. The media performance statistics are maintained by the Statistical Analysis and Reporting System

(SARS) portion of the drive microcode. Each cleaning cartridge also contains a CM, which tracks the number of cleaning uses.

2.8 -Volume management

DFSMSrmm manages volumes based on the volume type, volume status, and the volume media.

2.8.1 Types of volumes

DFSMSrmm provides support for physical, logical, and stacked volumes (see Figure 2-21 on page 49).

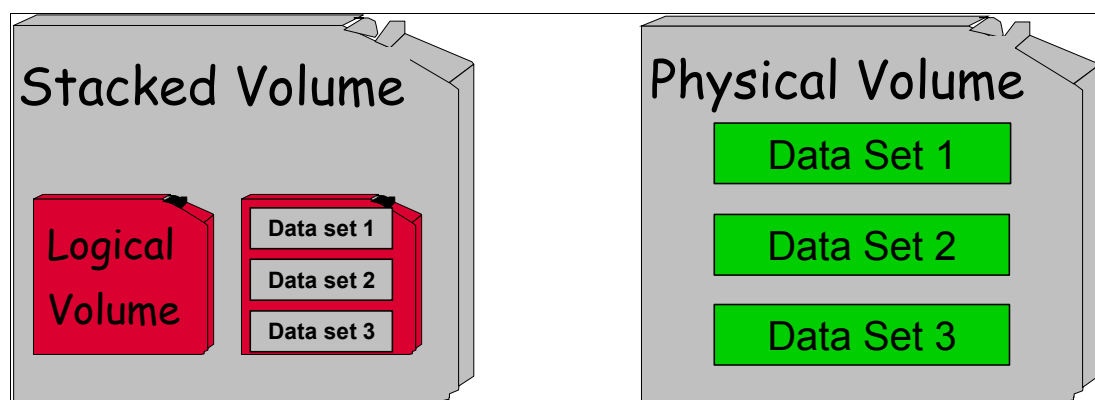


Figure 2-21 Stacked, logical, and physical volumes

Physical volumes

DFSMSrmm provides support for all types of physical volumes, tracking their use and the data they contain, and providing vaulting and retention services according to policies assigned to the data sets on the volumes.

Logical volumes

These reside in a VTS or on exported stacked volumes. DFSMSrmm tracks the use of logical volumes and the data that resides on the volumes. Retention services are provided according to policies assigned to the data sets on the volumes. DFSMSrmm supports exporting of logical volumes on stacked volumes and provides vaulting services for these stacked volumes based on the policies assigned to the data sets on the contained logical volumes. Logical volumes can be removed from a VTS using export processing.

Stacked volumes

These have a one-to-one association with physical tape media and are used in a VTS to store logical volumes. Stacked volumes are not used by MVS applications, they are used by the VTS and its associated utilities. Stacked volumes can be removed from a VTS to allow transportation of logical volumes to a vault or to another VTS.

DFSMSrmm has added support for stacked volumes to allow identification of the stacked volumes in a VTS and direct management of these volumes when exported. These stacked volumes are assigned to specific slots when moved to storage locations that require shelf management.

DFSMSrmm provides support for importing logical volumes on stacked volumes either into the VTS from which it was exported, or into a different VTS.

If stacked volume support is enabled, you can define stacked volumes and the logical volumes associated with them using the RMM ADDVOLUME subcommand as shown Figure 2-22.

```
RMM AV convol TYPE(STACKED) LOCATION(SHELF)
RMM AV volser TYPE(LOGICAL/PHYSICAL) STATUS(MASTER) CONTAINER(convol)
```

Figure 2-22 Adding a stacked volume as a container volume

You also can use stacked volume support as a way to manage the boxes in which you transport and store tapes. You can associate volumes to a box, by defining the box as a stacked volume, and using RMM TSO subcommands to add the volumes to the box as the CONTAINER.

2.8.2 Volume status

Within DFSMSrmm, a volume can be in one of six different statuses:

Scratch:	This status indicates that the volume is available for use. That is a volume that does not contain valid data.
Master:	The status assigned to a volume at open time after a non specific mount.
User:	Status assigned to a volume after a user get volume request or assigned this status through an DFSMSrmm command.
Pending release:	This is a transient status that occurs after a master or user volume expires and prior to is becoming scratch.
Init:	The volume is a scratch volume waiting to be initialized. Once initialized, the status changes to SCRATCH.
Entry:	The volume is a scratch volume coming into an automatic tape library (ATL). When entered, the status changes to SCRATCH or INIT.

Figure 2-23 shows the life cycle of a volume.

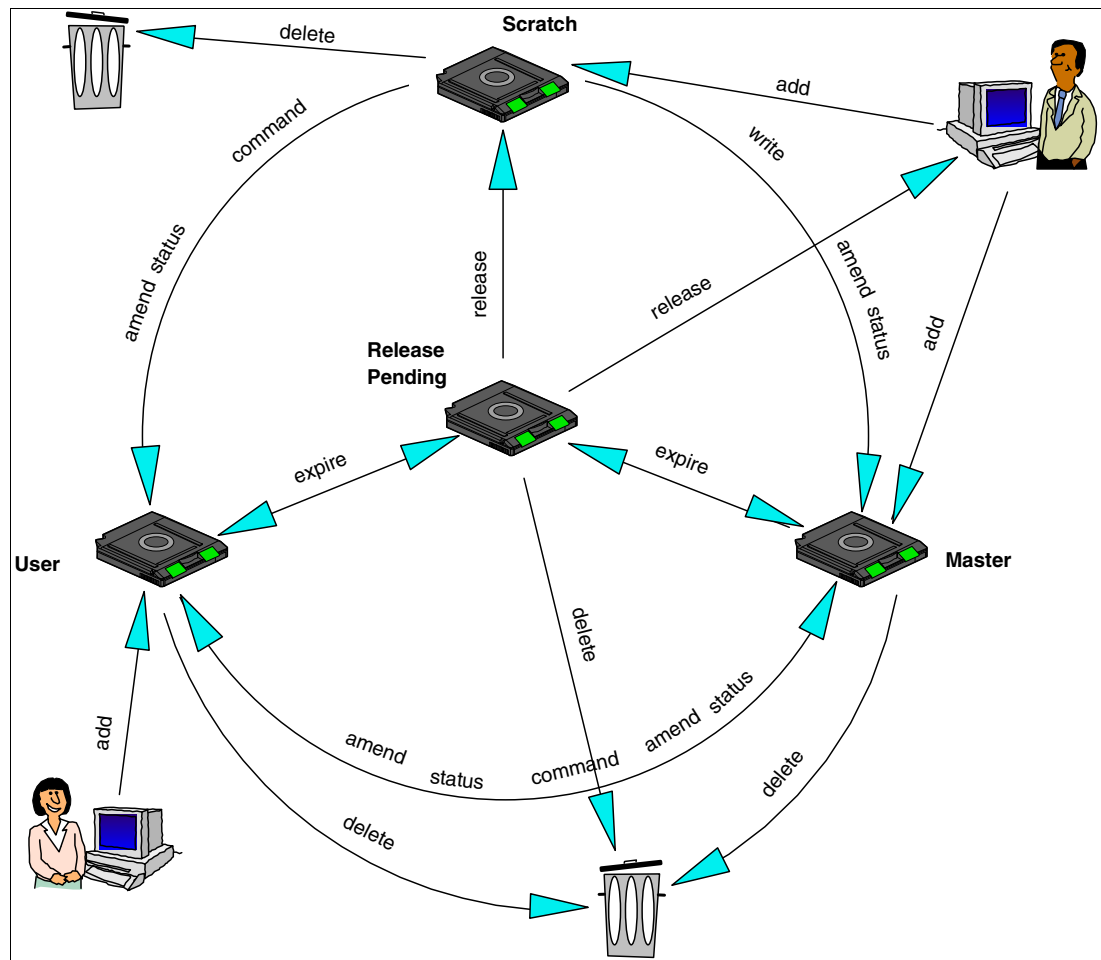


Figure 2-23 Life cycle of a volume

Most of the volumes that DFSMSrmm manages in your installation are probably *scratch* volumes, that is, volumes that are used again and again by different users. Each time a volume is used, it is retained and managed by policies that you define to DFSMSrmm. When the data is no longer required, the volume is returned to scratch status, and is ready for use by another user.

With DFSMSrmm, you can add or delete volumes without stopping your normal tape activities.

In addition to managing scratch volumes, DFSMSrmm can manage any volume you define to it, including those that you treat as *foreign volumes* under your existing system. DFSMSrmm supports any volume serial (VOLSER). You can still process volumes as foreign to DFSMSrmm by either not defining them to DFSMSrmm or using the DFSMSrmm *ignore* support through the EDGUX100 exit.

DFSMSrmm records information for data sets on all files of a tape volume. It also can manage the RACF TAPEVOL and/or TAPEDSN profiles for tape volumes and uncataloged data sets. Figure 2-24 shows how DFSMSrmm processes records related to tape data sets.

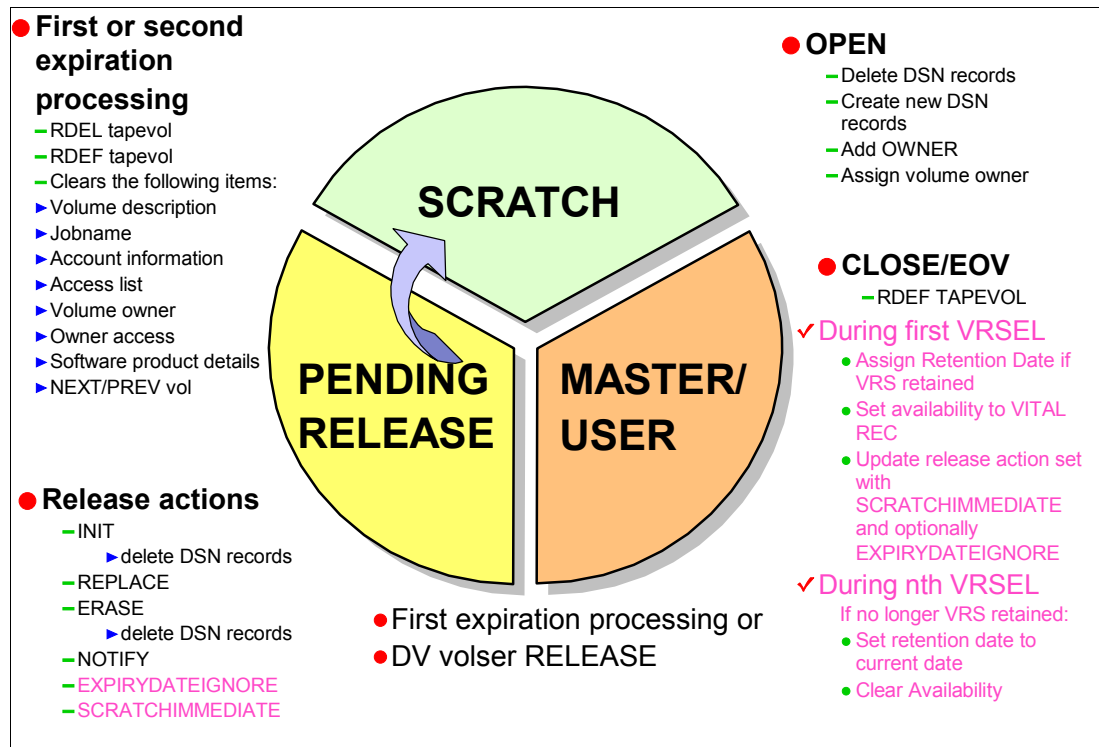


Figure 2-24 Timing of related events for a volume

Note: If you have specified the “SCRATCHIMMEDIATE” keyword in your VRS definitions the status is directly changed from MASTER/USER to SCRATCH without to run a second expiration processing.

If you have installed DFSMSrmm z/OS V1R13 or higher and you have specified the retention method RETENTIONMETHOD(EXPDT) in EDGRMMnn parmlib member the DFSMSrmm inventory management EXPROC processing always attempts to return volumes to scratch on the same run as the volume is released (this is as if the SCRATCHIMMEDIATE attribute is set for the volume). DFSMSrmm maintains a consistent expiration date and time for all data set records of a multi-volume data set.

2.8.3 Validation

DFSMSrmm automatically validates volumes, ensuring that only valid scratch volumes are mounted for nonspecific mount requests and that the right volume is mounted for a specific mount request. This validation eliminates the unintentional overwriting of a valid master volume, or a volume retained for disaster recovery or vital record management.

When a data set on a volume is opened and closed, DFSMSrmm automatically:

- ▶ Changes the volume status from scratch to master for nonspecific mount requests
- ▶ Sets an expiration date for the volume and ensures that the maximum expiration date is not exceeded (repeated at close time) as defined in the PARMLIB member
- ▶ Records information about data sets on the volume (the data set name is recorded at open time; all other information is recorded at close time)
- ▶ Counts the number of times a volume is used

- ▶ Counts the number of temporary and permanent errors encountered
- ▶ Sets a security classification (as defined in the DFSMSrmm PARMLIB) based on the data sets that reside on the volume
- ▶ Prevents reading of data on a volume in scratch status when DFSMSrmm is running in protect mode

All of these actions occur at open time, except where noted.

Using EDGUX100 to ignore volume serial numbers

DFSMSrmm can ignore volumes regardless of whether the volumes were defined to DFSMSrmm.

To request that DFSMSrmm ignore a volume, tailor the EDGUX100 DFSMSrmm installation exit to use undefined volumes or duplicate volumes. When you use the EDGUX100 installation exit, DFSMSrmm calls the exit each time a volume is opened. The sample installation exit checks the JCL-specified EXPDT value for the special date 98000 or for the ACCODE value xCANORES. If the 98000 special date or the ACCODE value xCANORES is found, the EDGUX100 exit uses the installation exit parameter list to request that DFSMSrmm ignore the volume. Figure 2-27 on page 54 shows how the EDGUX100 user exit works. This support was introduced with APAR OW53763.

To request that DFSMSrmm ignore a volume, you also have to define a RACF FACILITY class entity as shown in Figure 2-25.

```
STGADMIN.EDG.IGNORE.TAPE.volser
```

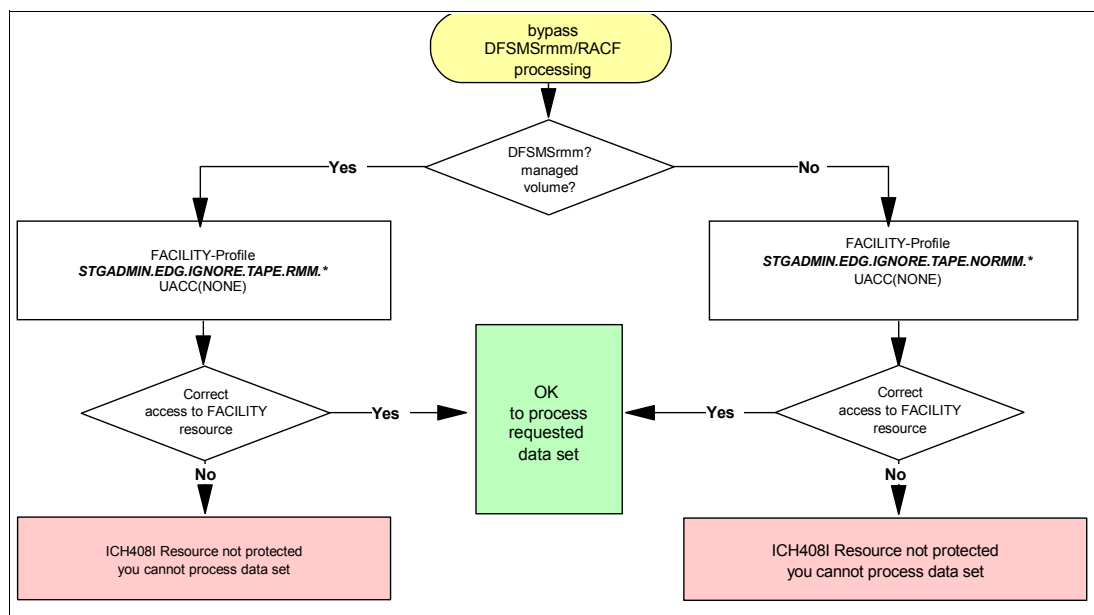
Figure 2-25 Global resource to ignore tape volumes

If you want to distinguish between volumes managed by DFSMSrmm and volumes not managed by DFSMSrmm, define the RACF profiles as shown Figure 2-26 to check if the user is authorized to request that DFSMSrmm ignore the volume.

```
STGADMIN.EDG.IGNORE.TAPE.RMM.volser
STGADMIN.EDG.IGNORE.TAPE.NORMM.volser
```

Figure 2-26 More specific resources to ignore tape volumes

In the RACF profile *volser* is the volume serial number of the mounted volume or requested tape volume. Figure 2-27 shows how the BYPASS DFSMSrmm function works.



Note: When the profiles
STGADMIN.EDG.IGNORE.TAPE.RMM.volser
and/or
STGADMIN.EDG.IGNORE.TAPE.RMM.volser
are not defined DFSMSrmm looks for
STGADMIN.EDG.IGNORE.TAPE.volser.

If you define the *RMM* and *NORMM* ignore profiles, DFSMSrmm makes the authorization checks as shown in Table 2-3.

Table 2-3 *DFSMSrmm ignore profile checking*

Authorization check	Profile used
Volser not found in CDS	STGADMIN.EDG.IGNORE.TAPE. NORMM .volser
Volser found in CDS and match on HDR1 17 character data set name	STGADMIN.EDG.IGNORE.TAPE. RMM .volser
Volser found in CDS and no match on HDR1 17 character data set name	STGADMIN.EDG.IGNORE.TAPE. NORMM .volser

2.8.4 Support for duplicate volumes

In DFSSrmm, volume serial numbers are used to identify volumes and to identify the volume label. DFSSrmm allows you to define a volume using a serial number that is different from the one that is recorded in the volume label. This way you can define volumes with duplicate volume serial numbers.

DFSMSrmm open processing has been updated to detect duplicate volumes and to use the correct CDS volume information to manage the volume. You can continue to have duplicate volumes ignored, but the recommendation is to add them to the DFSMSrmm CDS and manage them. Figure 2-28 shows how DFSMSrmm supports duplicate volume serial

numbers by specifying VOLSER, RACK, and the new VOL1 operand of the ADDVOLUME function.

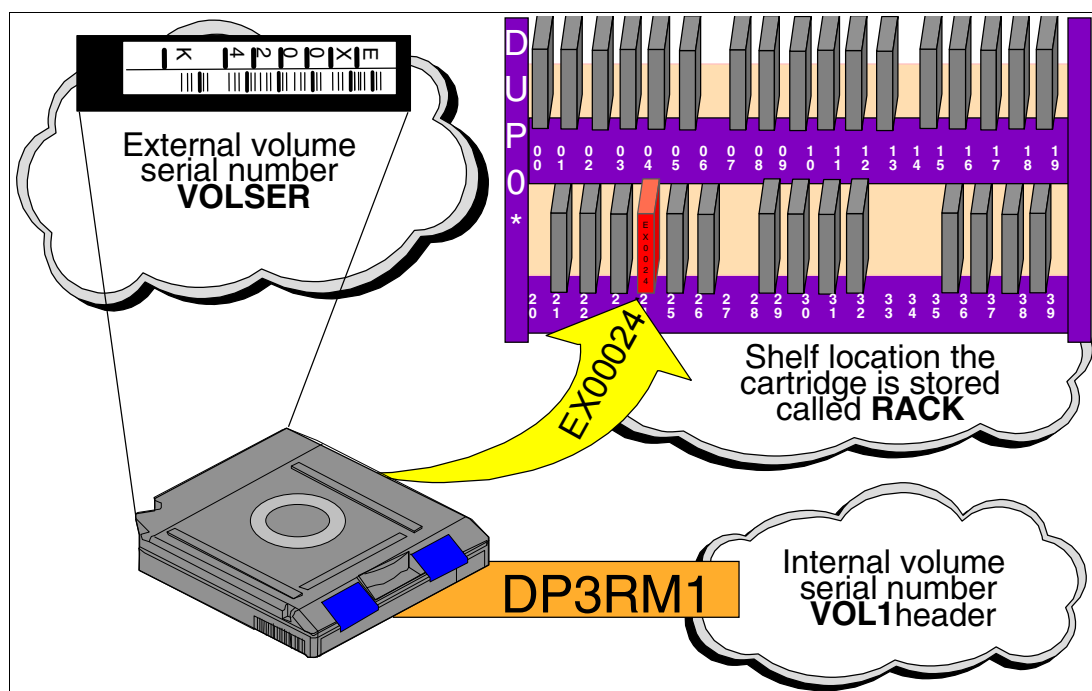


Figure 2-28 DFSMSrmm duplicate volume support

Each volume defined to DFSMSrmm must be defined using a unique volume serial number, but for private physical volumes you can also define the VOL1 label volser. You can have any number of duplicate volumes with the same VOL1 label volser. You can use duplicate volumes for both input and output processing, and do not need to use BLP or DFSMSrmm ignore processing. You do not need to enable duplicate volume support. You simply start to define and use duplicate volumes when you need the support. You use the DFSMSrmm ADDVOLUME or CHANGEVOLUME subcommand with VOL1(volser) to define a duplicate volume. See Figure 2-29 on page 56.

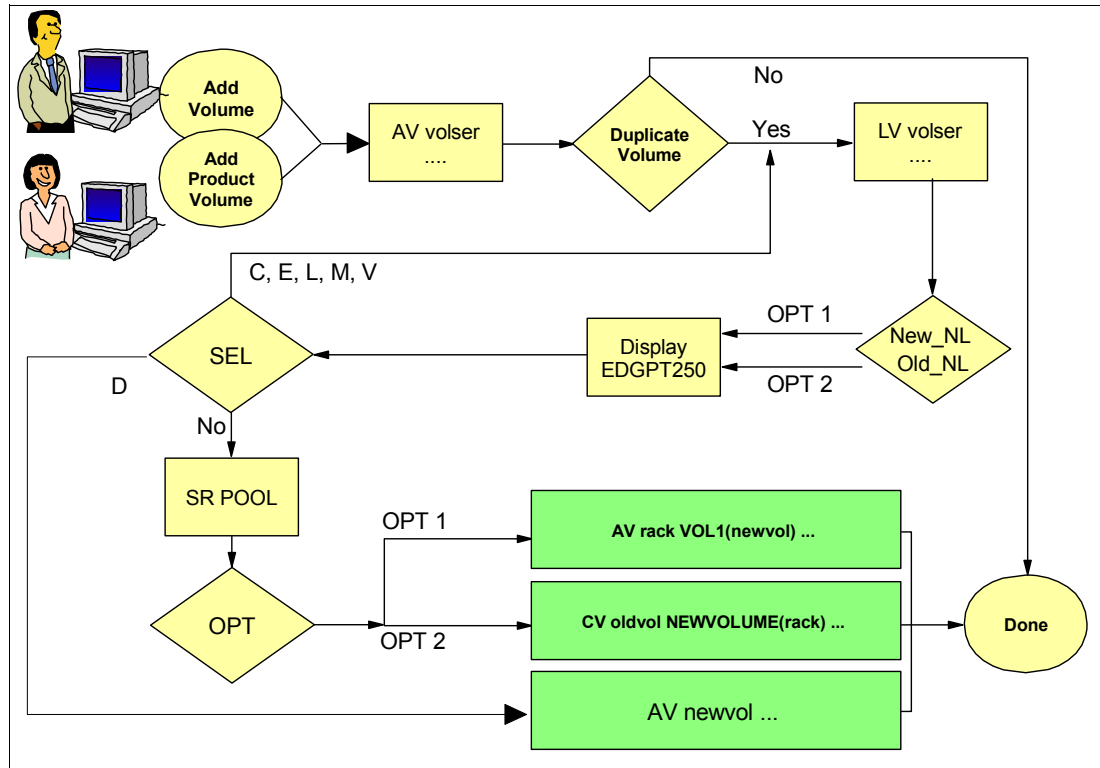


Figure 2-29 Adding a duplicate volume serial number

A no-label volume cannot be a duplicate volume, because you can define such a volume to DFSMSrmm using any unique value for the volser. Also, no special processing is required by DFSMSrmm to distinguish the volume from another identical volume.

DFSMSrmm is the only component in the system that knows a volume is a duplicate. DFSMSrmm hides this fact from others by substituting the VOL1 label volser read from the volume with the requested volume when DFSMSrmm has validated that the correct volume is mounted. The requested volume is the unique volume serial number that you specify in your JCL or allocation request that identifies the volume to DFSMSrmm.

DFSMSrmm exploits the capabilities of the Tape Mount exit to check the VOL1 label volser. DFSMSrmm uses the requested volser to find a volume defined to RMM, and validates that the VOL1 label volser matches the VOL1 label volser defined to DFSMSrmm. The user specified volser identifies the volume record to be read from the DFSMSrmm CDS. The VOL1 label volser must match the DFSMSrmm recorded VOL1 label volser, or the volume record has no recorded VOL1 label volser, and the requested volume matches the VOL1 label volser.

Note: In DFSMSrmm, there are three different values to identify a volume:

VOLSER: To specify the volume serial number

VOL1: To define a duplicate volume when the VOL1 label volume serial number does not match the volume you are defining to DFSMSrmm

RACK: Specifies a shelf location in the removable media library where DFSMSrmm stores the volume

If you add a new volume with a volume serial number of a volume previously defined in the CDS, DFSMSrmm displays a new panel as shown in Figure 2-30.

DFSMSrmm Add Duplicate Volume

Command ==>

Scroll ==> CSR

Volume **VT0002** duplicates existing volume **VT0002**
Enter selected option for volume **VT0002** or use line command below
Any recommended option is pre-selected for you

Option ==>

- 1 Add the volume as a duplicate
- 2 Change the existing volume to be a duplicate

Specify a pool for the duplicate volume
Pool prefix ==>

Enter HELP or PF1 for the list of available line commands

Volume	VOL1	Rack	Data	Product	Feat
S	serial	volser	number	Status	Location
---	-----	-----	-----	-----	-----
VT0002		MASTER	VTFM001	1	SL

Figure 2-30 Adding a duplicate volume panel

2.9 Policies for retention and movement

DFSMSrmm provides policy management for movement and retention at the data set, volume, or volume set level. Every tape data set can have a policy, and each policy can specify movement as well as retention. The retention and movement policies you define to DFSMSrmm are known as *vital record specifications* (VRSs). You use them to indicate how long and where you want to keep data sets or volumes. You also use them to define how volumes are to be moved among the libraries that DFSMSrmm supports and the storage locations defined for vital records and disaster recovery purposes.

You can create a vital record specification *chain* to cause a sequence of volume moves. The first vital record specification in the chain is a data set name, SMS management class, DFSMSrmm management value or volume vital record specification. These specify one location in which the data set or volume being retained is to be stored. You would add one name vital record specification for each additional location or retention value that applies to the volume or data set.

DFSMSrmm records the starting location for a volume when the volume is initially defined to DFSMSrmm, or when the volume information is changed. This starting location is known to DFSMSrmm as a *home location*. Home is the place from which volumes start, and to which they are returned, when the identified retention and movement actions have been completed.

2.9.1 Data set retention

You use *data set names* and *data set name masks* to define retention policies for data sets, and you use *job names* and *job name masks* to define retention policies to further qualify the criteria for applying retention and movement policies. For data sets, you can request the following types of retention.

► **Retention by PARMLIB option - VRSEL(NEW)**

You can control the extent to which data set name and management value VRSs are used. You can define different retention criteria in a next VRS only if you specify VRSEL(NEW).

Note: The VRSEL option OLD is no longer supported.

► **Retention by cycles**

You can retain a minimum number of cycles or copies of a data set. This type of retention applies to generation data groups (GDGs) or like-named data sets identified by pseudo-GDG data set names. For non-GDG data sets, DFSMSrmm considers each occurrence of a data set to be a cycle.

► **Retention by BYDAYSCYCLE**

You can specify to retain all instances of a data set created on a single day as a single cycle.

► **Retention until expired**

You can retain a data set until volume expiration date has been reached or the retention amount has elapsed.

► **Retention by number of elapsed days**

You can retain each copy of a data set a given number of days since it was created.

► **Retention by days since last referenced**

You can retain each copy of a data set for a set number of days since the data set was read or written.

► **Retention while data set is cataloged**

You can retain any data set as long as it remains cataloged. Catalog status can also be used in combination with management by cycles, elapsed days, days since last referenced, and volume expiration date.

► **Retention by extra days**

On a NEXTVRS, you can specify the number of days since a name vital record specification started to retain the data set. The number of extra days is specified with the COUNT operand. EXTRADAYS is the number of days since the NAME vital record specification started to retain the data set. The number of days depends on when the previous vital record specification stopped retaining the data set and the time when vital record processing is run. This option can be used only on a NEXTVRS and not a primary VRS.

► **Retention to a specific date**

You can set a deletion date for a VRS. When that date is reached, the VRS is deleted. All data sets and volumes that would match the VRS become eligible for release processing, or they might match a less specific VRS that specifies different retention and movement information.

► **Retention by expiration date**

You can retain the data set on a volume as long as the volume expiration date has not yet been reached. You can also use a combination of catalog status, volume expiration date, and other retention options to retain data sets.

► **Retention of open data sets**

You can specify a separate policy to apply to all data sets that are currently open.

► **Retention of data set closed by abend processing**

You can specify a separate policy to apply to all data sets that were open at the time of an application or system abend.

- **Retention of data set data sets closed with normal disposition DELETE**
You can specify a separate policy to apply to all data sets that were created with normal disposition of DELETE.
- **Retention of set**
This allows you to retain all volumes of a set until all of the data sets on those volumes have expired. Use the EDGRMMxx PARMLIB option RETAINBY(SET).

Figure 2-31 shows an example of a multifile/multivolume set. The first and second data set are not expired, but the third one has since yesterday. The third volume of the set is only retained if PARMLIB option RETAINBY(SET) is specified.

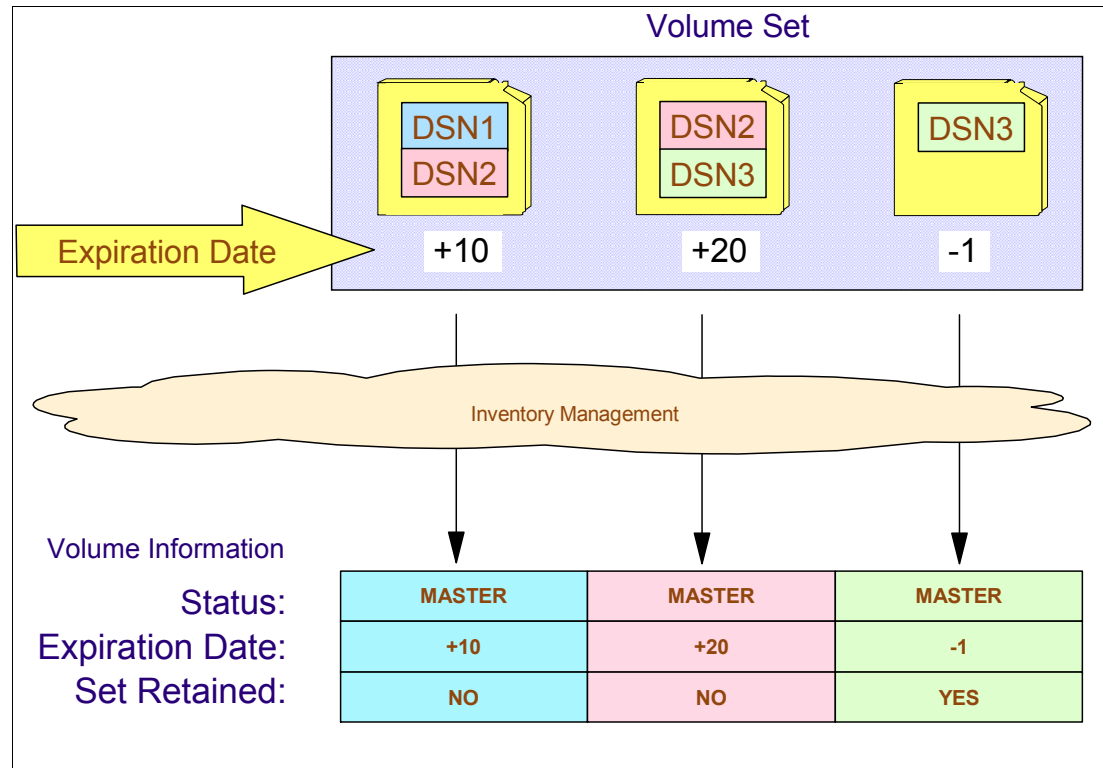


Figure 2-31 *RETAINBY(SET): Expiration date*

Your installation can also define a VRS management value or a SMS management class to define retention and movement policies. If you have specified VRSEL(NEW) in the EDGRMMxx PARMLIB member, you can have up to two policies per data set: one policy using a data set name mask, and one using VRS management class or VRS management value.

A VRS management value assigns management and retention values to tape data sets. You can define data set VRSs for VRS management values to provide support for special JCL-specified expiration dates, and to allow DFSMSrmm to manage those data sets with special dates. You can use the sample installation exit EDGUX100, in SAMPLIB, to assign VRS management values.

2.9.2 Volume retention

To define retention and movement policies for volumes, you can use a specific or generic VOLSER. If you use a specific VOLSER, the volume matching that number is retained by days since creation. If you use a generic VOLSER, any volumes matching the generic VOLSER are retained. These volumes must be of user or master status.

For a volume, you can request the following types of retention using a specific or generic VOLSER:

- ▶ **Retention by number of elapsed days since creation**
Indicates that a volume be retained until the specified number of days since creation have elapsed
- ▶ **Retention by number of volumes**
Indicates how many volumes should be retained

2.9.3 Movement

DFSMSrmm records the starting location for a volume when the volume is initially defined to DFSMSrmm, or through a change to the volume information. This starting location is known to DFSMSrmm as a *home location*. Volumes are returned to the home location when the identified retention and movement actions have been completed.

You can give any system-managed library or storage location as a target destination for a volume move. Obviously this location must be defined to DFSMSrmm. The location could be an ATL, MTL, storage location or shelf.

You can also define policies to provide retention information for data sets and volumes that must be moved through multiple locations before they expire. You define such policies by creating a VRS chain.

When multiple data sets on the same volume are retained by VRSs, and each VRS contains a different destination, DFSMSrmm decides where to move the volume according to priority number.

Unlike some other systems, DFSMSrmm does not do storage location management on the basis of the first file on a volume. Storage location management is based on the priority of the storage location management requirements for all data sets on a volume. Thus, DFSMSrmm must sometimes do conflict resolution, and only retains and moves volumes with a data set retained by a VRS.

A recently added function of DFSMSrmm allows you to move a SET of volumes, thus keeping related volumes together. Use the PARMLIB option MOVEBY(SET).

Storage location management with DFSMSrmm is essentially similar to vault management in other tape management systems. What seems to be unique to DFSMSrmm is the concept of volume-in-transit. Volumes have been scheduled for a move or are on their way, but they have not yet been received at the storage location. Operators can tell which volumes are still in transit. A special command moves volumes from in-transit status to storage-location-resident status. Installations that have established special storage location names to cover transport time need fewer storage locations. Installations that do not want librarians issuing this command can automate the confirmation of movement by issuing TSO commands or through REXX procedures.

If you specified VRSEL(NEW) in the EDGRMMxx member of PARMLIB, you can now have separate movement and retention policies. Use a VRS based on management value or management class name to define retention criteria; use a VRS based on data set name mask for defining movement.

2.10 SMS ACS support

You can use the ACS routines to process the special calls that DFSMSrmm makes to the SMS subsystem for ACS processing to set a SMS management class. DFSMSrmm requests that the management class and storage group routines are run. The environment variable is set to RMMPOOL, so that you can differentiate allocation requests for system-managed data sets from requests by DFSMSrmm for a storage group.

You can use SMS ACS to decide if data sets are to be system-managed or non-system-managed. Using the SMS pre-ACS calls to DFSMSrmm help you to use your SMS ACS logic. Use the pre-ACS processing for decisions on the MSPOL and MSPOLICY values that come from DFSMSrmm scratch pool and EDGUX100 exit processing. You can plan to move the EDGUX100 exit decisions into your SMS ACS routines to enable the EDGUX100 exit processing to be ignored or removed some time in the future.

DFSMSrmm attempts to pass values for the MSPOL and MSPOLICY ACS read-only variables when the PARMLIB option PREACS(YES) is set. If you do not use the EDGUX100 exit, the MSPOL variable is set to the DFSMSrmm system based scratch pool decision. If you do not use the EDGUX100 exit, there is never a value set for MSPOLICY. DFSMSrmm sets a pre-ACS variable only if the variable has not yet been set using the pre-ACS exit. Since DFSMSrmm gets control after the installation exit, any vendor or customer decisions take precedence (see Figure 2-32).

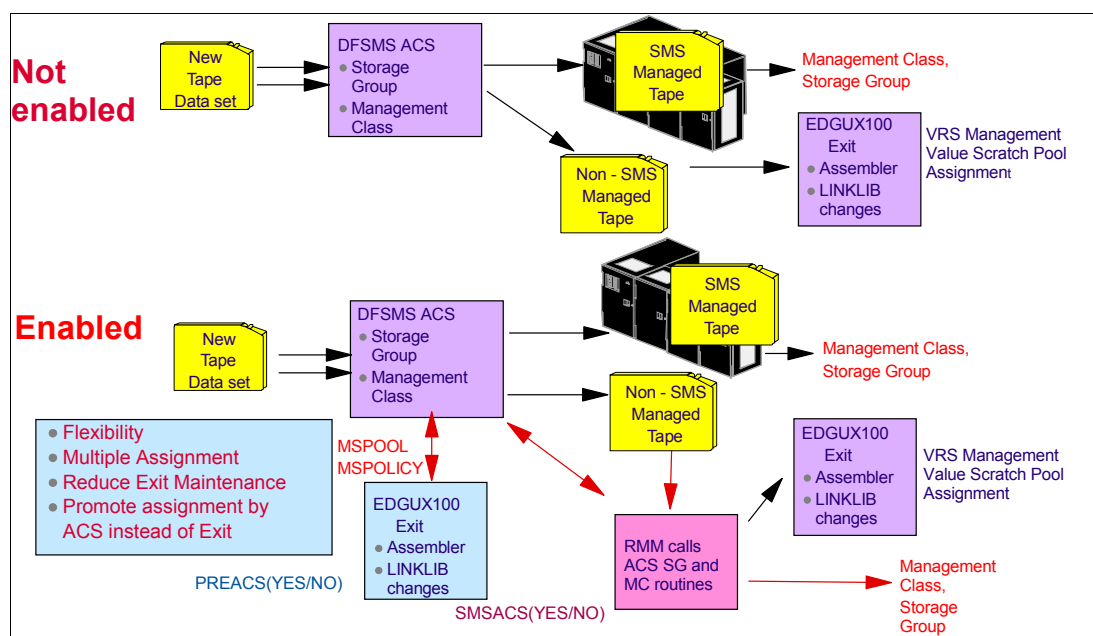


Figure 2-32 DFSMSrmm and SMS ACS support

2.11 DFSMSrmm automated tape library support

DFSMSrmm excels at ATL management. In an ATL environment, you have different databases to store volume and location information. In the DFSMSrmm CDS, you have the volume serial number, the volume status, and the volume location information. In the SMS tape volume database (TCDB), you have the volume serial number, the volume status, and the location information. In the library manager database, you have the volume serial number and the volume status information.

DFSMSrmm keeps all three databases in sync by using all four available OAM installation exits:

CBRUXCUA	Change use attribute exit
CBRUXEJC	Eject cartridge exit
CBRUXENT	Entry cartridge exit
CBRUXVNL	Volume not in library exit

Refer to the “Running DFSMSrmm with System-Managed Tape Libraries” chapter in *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for more details. Figure 2-33 shows the entire ATL environment.

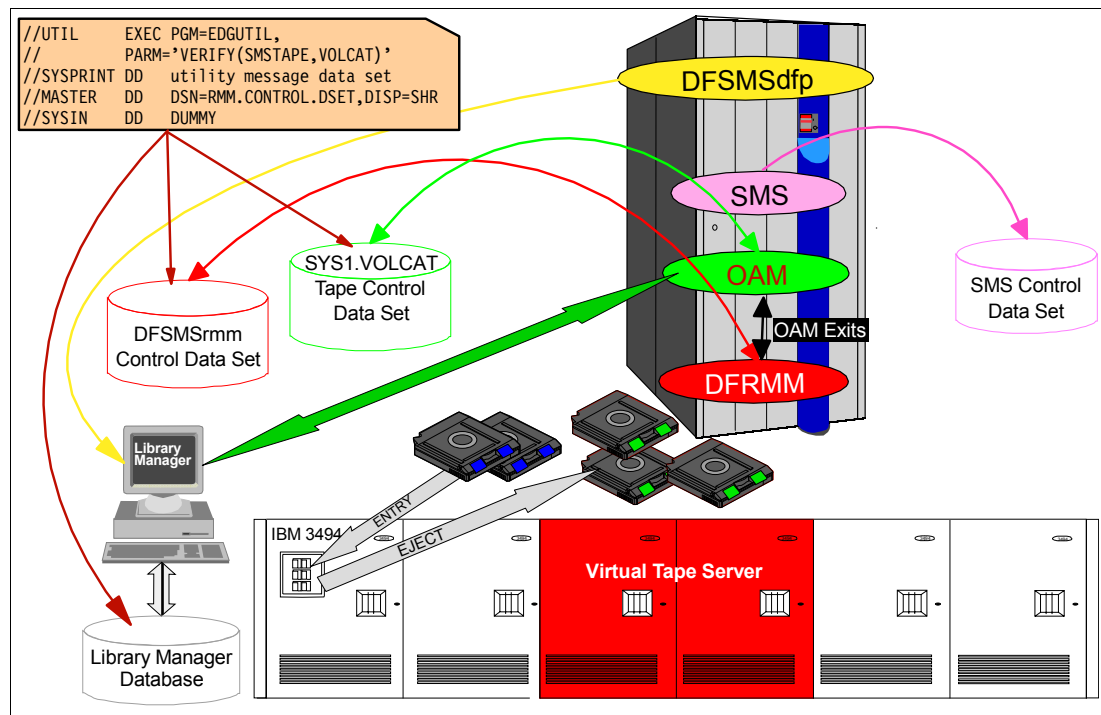


Figure 2-33 ATL environment

The components of system-managed tape are:

- ▶ OAM and its Library Control System (LCS) component
- ▶ Storage Management Subsystem (SMS)
- ▶ Integrated Storage Management Facility (ISMF)
- ▶ Installation wide exits
- ▶ Tape management system
- ▶ Library Manager

DFSMSrmm provides the following support for system-managed tape libraries:

- ▶ Cartridge entry processing
- ▶ Cartridge eject processing
- ▶ Expiration management
- ▶ Volume-not-in-library support
- ▶ Movement between libraries and storage locations
- ▶ Partitioning the libraries with other systems
- ▶ VTS import/export processing
- ▶ Tracking and managing logical, physical, and stacked volumes

The following sections review some of the items in the list.

2.11.1 Cartridge entry processing

DFSMSrmm automatically adds volumes to the DFSMSrmm control data set during entry processing and when you use volumes. If a volume defined to DFSMSrmm is entered into a system-managed tape library without using the RMM CHANGEVOLUME command to set the volume location to the library name, DFSMSrmm updates the DFSMSrmm control data set volume record with the library name, library type, and volume entry status. DFSMSrmm does not update the home location name for the volume.

DFSMSrmm allows you to partition system-managed tape libraries using the DFSMSrmm REJECT PARMLIB option to identify volumes that you want to partition. Volumes can be identified based on volume prefix and as individual volumes defined to DFSMSrmm on z/OS.

The REJECT parameter of the EDGRMMxx PARMLIB can be used to control cartridge entry processing into the library and whether the cartridge is to be used for input processing only. In Figure 2-34 are some examples of using the REJECT command in the EDGRMMxx member.

```
REJECT OUTPUT(CC12*)
REJECT ANYUSE(VM1*),OUTPUT(VM*)
REJECT ANYUSE(DD0*)
REJECT ANYUSE(*)
```

Figure 2-34 Old REJECT definitions

Important: Define all volumes to DFSMSrmm before entering them into a library. Define private volumes prior to entry into the automated tape library, and set the ISMF default cartridge entry status to scratch.

Use the new PRTITION and OPENRULE parameters to replace the REJECT parameter of the EDGRMMxx PARMLIB definitions. Use the OPENRULE command to identify special processing for DFSMSrmm to perform during OPEN processing. It applies equally to both system-managed and non-system managed volumes. Use the PRTITION command to identify special processing for DFSMSrmm to perform during library entry, export/import, eject, and CUA processing, during OPEN processing, and EXPROC processing. The PRTITION commands apply to all volumes, not only system-managed volumes. However, only system-managed volumes benefit for the functions such as library insert, CUA, and eject.

Note: While the REJECT command is still supported by DFSMSrmm, you should use the OPENRULE and PRTITION commands to prevent a range of tapes from being used on a particular system. If you are already using REJECT commands, you should consider converting them to OPENRULE and PRTITION commands.

In Figure 2-35 on page 64 are some examples of using the OPENRULE and PRTITION commands in the EDGRMMxx member.

```
/* Allow read of undefined selected volumes */
OPENRULE VOLUME(CC12*) TYPE(RMM) OUTPUT(REJECT) INPUT(ACCEPT)
OPENRULE VOLUME(VM*) TYPE(RMM) OUTPUT(REJECT) INPUT(ACCEPT)

/* Ignore for input selected system managed volumes if authorized */
OPENRULE VOLUME(DD0*) TYPE(NORMM)
      INPUT(IGNORE) OUTPUT(REJECT)
OPENRULE VOLUME(VM1*) TYPE(NORMM)
      INPUT(IGNORE) OUTPUT(REJECT)

/* Global partition rule - ignore all smt and nosmt volumes */
PRTITION VOLUME(*) TYPE(RMM) -
      SMT(IGNORE) NOSMT(IGNORE)

/* This partition owns JT* volumes and the volume is added to the
/*      DFSMSrmm CDS if not yet defined
PRTITION VOLUME(JT*) TYPE(ALL) SMT(ACCEPT)
```

Figure 2-35 New OPENRULE and PRTITION definitions

Important: Define all volumes to DFSMSrmm before entering them into a library. Define private volumes prior to entry into the automated tape library, and set the ISMF default cartridge entry status to scratch.

2.11.2 Ejecting volumes from system-managed libraries

You can eject physical volumes from a system-managed library by using various methods:

- ▶ Using the library manager console commands
- ▶ Using ISMF commands
- ▶ Using DFSMSrmm commands through ISPF or batch

2.11.3 Volume-Not-In-Library processing

When a volume is requested for a job, and is not in a system-managed tape library, the DFSMSrmm supplied CBRUXVNL exit is called so the volume can be inserted into a library to prevent job failures from occurring.

2.11.4 Verifying the databases

You can use the DFSMSrmm utility EDGUTIL with the parameters VERIFY(VOLCAT,SMSTAPE) to check the consistency of information in the DFSMSrmm control data set, the tape configuration database (TCDB), and the library manager database.

If you have differences, you can use the same utility with a PARM of MEND to correct the errors. Figure 2-36 on page 65 shows the entire environment if you have an IBM ATL installed.

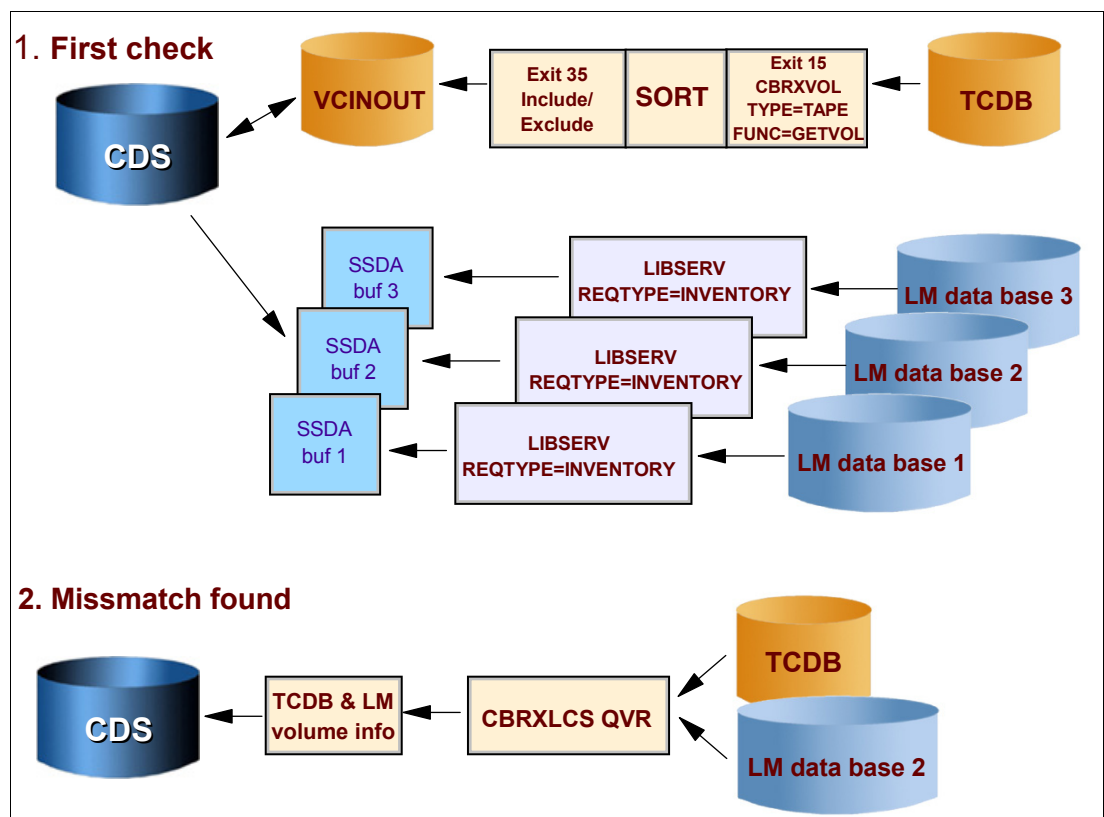


Figure 2-36 EDGUTIL utility to verify the VOLCAT and LM data base

Important: To correct discrepancies you can use the MEND function, but customer knowledge is necessary

2.11.5 VTS import/export processing

DFSMSrmm supports VTS libraries such as automated system-managed libraries with extensions to handle the special requirements for different volume types and functional capability. DFSMSrmm supports VTS import/export in different ways depending on whether DFSMSrmm stacked volume support is enabled. When you enable stacked volume support, DFSMSrmm tracks logical volumes and stacked volumes.

When you do not enable stacked volume support, DFSMSrmm does not track the stacked volumes. DFSMSrmm supports the IBM Peer-to-Peer VTS by ensuring that you cannot use the names of distributed VTS libraries with DFSMSrmm. You must only use the names of consolidated libraries with DFSMSrmm.

2.11.6 Defining stacked volumes to DFSMSrmm

You can define stacked volumes whether stacked volume support is enabled. DFSMSrmm defines the stacked volumes automatically at export time when stacked volume support is enabled. There is no automatic host notification when stacked volumes enter the VTS, leave

the VTS, or change category, so you must use DFSMSrmm commands if you want to have all stacked volumes defined to DFSMSrmm.

Note: Always define stacked volumes to DFSMSrmm. This ensures that the volumes cannot be used outside of the VTS. It also ensures that DFSMSrmm checks at cartridge entry processing time that a physical volume or logical volume does not duplicate a stacked volume.

2.12 Catalog synchronization

When you run DFSMSrmm with user catalogs and the DFSMSrmm control data set unsynchronized, DFSMSrmm issues catalog locates as required to check if data sets are cataloged. Catalog locates use the standard catalog search to find if a data set is cataloged. Any data sets cataloged using JOBCAT or STEPCAT might not be found to be cataloged, because the standard catalog search might not find them.

When you run DFSMSrmm with user catalogs and the DFSMSrmm control data set synchronized, DFSMSrmm does not need to issue catalog locates to find if a data set is cataloged. The catalog status tracked by DFSMSrmm in the control data set is used to determine if a data set is cataloged. When DFSMSrmm tracks catalog status, it does so regardless of whether JOBCAT or STEPCAT is used. During catalog synchronization, DFSMSrmm uses the Catalog Search Interface (CSI) to retrieve data set catalog information. CSI returns catalog information for all data sets in all catalogs that are in or connected to the master catalog. Because of this, DFSMSrmm can detect a data set is cataloged even if it cannot be found using the standard catalog search.

If you have switched on catalog synchronization, you do not need shared ICF-user catalogs to retain tape data sets as long as they are cataloged. If you have unshared ICF-user catalogs you have to split your daily inventory management. This is because you have to run the expiration processing on each system that has unshared ICF-user catalogs to return expired volumes back to scratch status. In Figure 2-37 on page 67, the ICF-user catalogs are shared between systems SYS1 and SYS2, but not with SYS3, so you have to run the expiration processing on either system SYS1 or SYS2, and on system SYS3. All other functions can run on any system sharing the DFSMSrmm control data set.

Refer to the “Maintaining the Control Data Set” chapter in *DFSMSrmm Implementation and Customization Guide*, SC26-7405 or to chapter Chapter 9, “Catalog synchronization” on page 333, for more details.

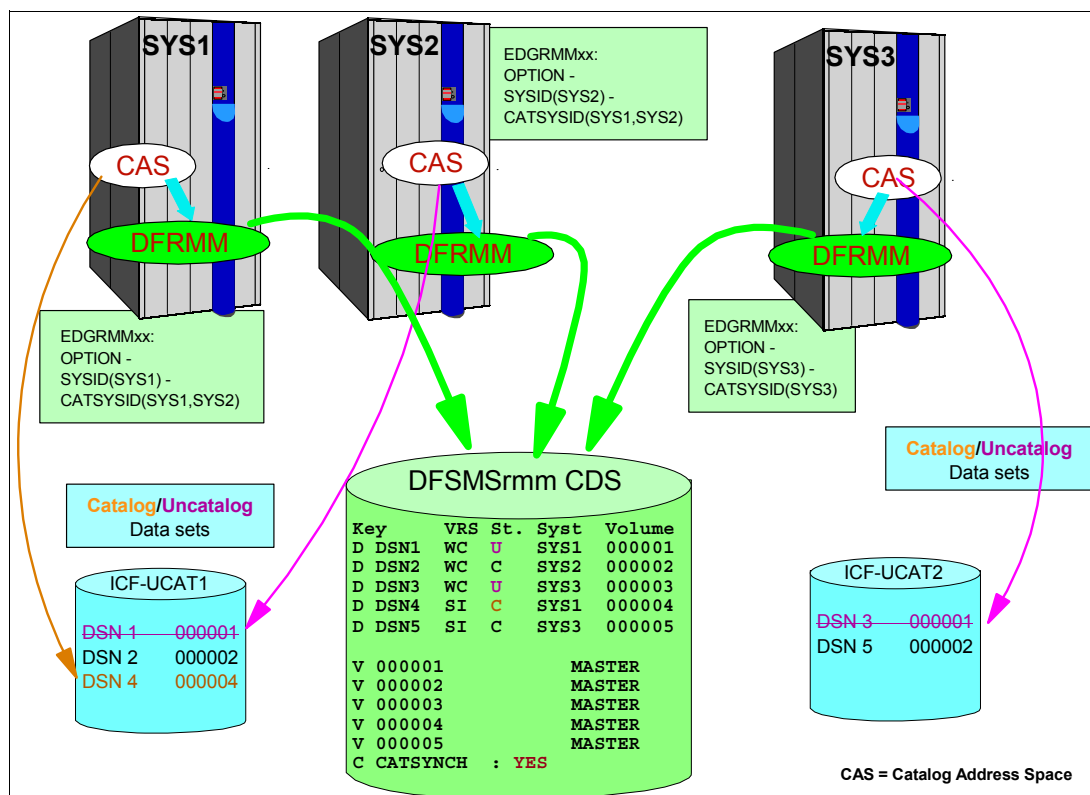


Figure 2-37 Catalog status tracking

2.13 Implementing the RMMplex

With DFSMSrmm z/OS V1.6 or later, you can share a single DFSMSrmm control data set between multiple single systems or multiple sysplexes without needing a shared DASD environment. This new function announced with DFSMSrmm z/OS V1.6 is called *client/server support*.

To use this new function, you must have TCP/IP installed on all systems that share a single DFSMSrmm control data set. Also all systems must be on z/OS V1.6 or later. Refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for more information.

Restriction: All systems (client and server) must be running with z/OS V1.6 or later. Other platforms than z/OS are not supported.

2.13.1 RMMplex

The RMMplex is one or more MVS systems each running a DFSMSrmm subsystem that shares a control data set. The RMMplex can optionally include one or more DFSMSrmm subsystems as servers, one or more client subsystems, in addition to standard DFSMSrmm subsystems. The server subsystems and standard subsystems have direct access to and share the DFSMSrmm control data set. The client systems have no direct access to the DFSMSrmm control data set but share the control data set via the server. All systems that share a control data set in this way are part of the same RMMplex.

Figure 2-38 shows a complex RMMplex implementation with two sysplexes and one non-plex sharing one single DFSMSrmm control data set.

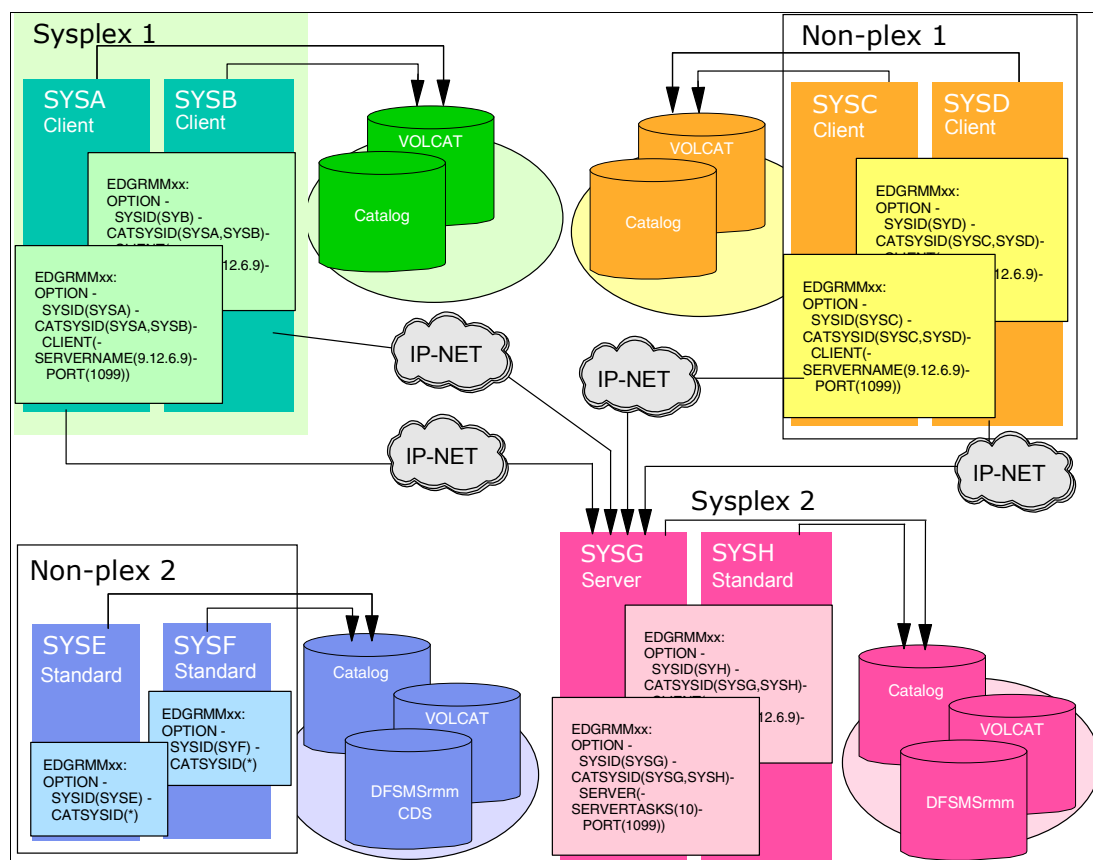


Figure 2-38 RMMplex with six systems sharing one DFSMSrmm CDS

The second non-plex shows a normal shared DFSMSrmm control data set. Each plex has a unique DASD environment that is not shared between the other plexes. Each plex also has its own MVS catalog environment.

DFSMSrmm server

A *server* in DFSMSrmm terminology is an MVS system where DFSMSrmm is up and active. This DFSMSrmm has direct access to the DFSMSrmm control data set and supports requests coming from other systems via TCP/IP.

DFSMSrmm client

A *client* in DFSMSrmm terminology is an MVS system where DFSMSrmm is up and active. This DFSMSrmm has no direct access to the DFSMSrmm control data set and sends all requests to the DFSMSrmm control data set via TCP/IP to the server specified in a EDGRMMnn PARMLIB member.



Preparing the environment

This chapter explains how to start DFSMSrmm on your system and test DFSMSrmm functions without affecting your existing environment.

The objectives of this chapter are to:

- ▶ Implement and start DFSMSrmm on a test system or in a test environment
- ▶ Provide a base for learning about DFSMSrmm

By the end of this chapter, you will have DFSMSrmm started in a test system and have set it up for basic operations in your environment. If DFSMSrmm is running without any errors you have repeat all this steps on you production system(s).

3.1 DFSMSrmm implementation

This section guides you through the process of getting DFSMSrmm started on your system with minimal effort and without impacting any existing tape management system you might have. The intention is to do only essential customization and setup in order to get DFSMSrmm started quickly.

It is essential that you introduce DFSMSrmm in your installation in a way that avoids any impact to your current processing. It is possible to do that, but you must select the correct DFSMSrmm PARMLIB options and ensure that the correct installation-wide exits are in use. When you start DFSMSrmm, your existing tape management system can continue to run and be unaffected.

Although we recommend that you use test data and test volumes initially with DFSMSrmm, you can start DFSMSrmm by using a control data set (CDS) that you create during the extract and conversion steps. If you start DFSMSrmm with a converted CDS, you should consider the environment within which you plan to run or test. Consider whether the test environment shares any data, catalogs, or volumes with your production environment. Any changes that DFSMSrmm makes as a result of processing can possibly update shared data. Use the correct DFSMSrmm PARMLIB options to prevent updates to RACF, the TCDB, and catalogs.

Regardless of how you plan to use DFSMSrmm, when you introduce it into any of your systems, it becomes visible to operators and perhaps some users. Those who will come into contact with DFSMSrmm should be provided with at least basic information about the product and how it affects them.

Follow the steps to implement DFSMSrmm as documented in this redbook, and refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for more details.

3.1.1 z/OS DFSMSrmm Customization Wizard

This wizard is designed to help you set up DFSMSrmm by selecting a standard setup that uses DFSMSrmm default values or a customized setup that you tailor for your installation. It builds the EDGRMMxx parmlib member, provides JCL jobs to setup RACF for DFSMSrmm, and jobs to allocate and initialize the datasets needed by DFSMSrmm. Editions of this wizard are available for:

- ▶ z/OS V1R12
- ▶ z/OS V1R11
- ▶ z/OS V1R10
- ▶ z/OS V1R9
- ▶ z/OS V1R8
- ▶ z/OS V1R7

Having downloaded the needed version of the wizard, unpack the zip archive for local exploiting.

You can find the DFSMSrmm migration wizard on the following Internet Web site, according to your installed operating system:

<http://www-03.ibm.com/systems/z/os/zos/downloads/#rmm1>

These interactive versions of the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, provide information that you can use to plan your implementation of the latest DFSMSrmm functions and features.

3.1.2 Updating the installation-wide exits

The installation-wide exits that you install with DFSMSrmm replace any previously installed exits. The DFSMSrmm supplied programming interface and exits are:

- ▶ DFSMSrmm installation exits
 - **EDGCVRSX** Sample to Use the Installation Exit EDG_EXIT100
 - **EDGUX100** Sample to Use the Installation Exit EDG_EXIT100
 - **EDGUX200** Sample to Use the Installation Exit EDG_EXIT200
 - **EDGUX300** Sample to Use the Installation Exit EDG_EXIT300
- ▶ DFSMSShsm tape volume programming interface
 - **EDGTVEXT**
- ▶ DFSMSdfp MSGDISP exit for updating tape drive displays
 - **IGXMSGEX**
- ▶ OAM exits
 - **CBRUXENT** (cartridge entry exit)
 - **CBRUXEJC** (cartridge eject exit)
 - **CBRUXCUA** (cartridge change use attributes exit)
 - **CBRUXVNL** (volume not in library exit)

The EDGDFHSM programming interface is used by DFSMSShsm and Tivoli Storage Manager to tell DFSMSrmm which tapes are no longer in use by these products. The equivalent interface between DFSMSShsm and your tape management system is provided by the ARCTVEXT exit shipped with this product .

Note: DFSMSrmm samples provided in AEDGSRM1 or SMPSTS

Before you install DFSMSrmm, you must determine if any of the DFSMSrmm supplied exits are used on your system. If they are used, and you still require the functions after conversion or during parallel running to DFSMSrmm, you must merge your existing exits with the DFSMSrmm supplied exits.

If you share one or more exits between your tape management and DFSMSrmm, you must check if updates are needed in any of the exits to achieve coexistence when running in parallel.

There are two ways to do this. You can merge your existing exits or use a router exit which invokes your existing exit and the DFSMSrmm exit or you modifying the sample CBRUXEJC, CBRUCENT and IGXMSGEX the this exit will call your existing exit. Figure 3-1 shows you the switch you can replace. Samples of the ARCTVEXT, the OAM CBRUXENT and CBRUXEJC, and the message IGXMSGEX exits are provided with DFSMSrmm in AEDGSRM1 or SMPSTS.

```
CBRUXEJC TITLE 'DFSMSRMM CBRUXEJC SAMPLE USER EXIT'  
&API      SETC 'NO'      Replace 'NO' with 'YES' if API call req'd$09A  
&ANOEJC   SETC 'ANOEJC'   Replace ANOEJC with required name  $07A  
&PARALLEL SETC '&SYSPARM' Replace &SYSPARM with YES if req'd  $07A
```

Figure 3-1 Switch for running sample DFSMSrmm exits in parallel

If you implemented DFSMSShsm and Tivoli Storage Manager, do not use the ARCTVEXT router exit use the ADMTVEXT instead. If you are using the ARCTVEXT router exit and you

set the call of the ARCTVEXT in your DFSMSHsm settings, you receive many DFSMSrmm error messages. In this case, DFSMSHsm releases each volume twice in DFSMSrmm, because DFSMSHsm calls the EDGDFHSM programming interface first and then calls the ARCTVEXT router exit.

Note: Create one front-end exit for each pair of exits, if you want both DFSMSrmm and your tape management system to be called and there is no other solution available.

3.1.3 Updating and validating the PARMLIB members

Update the following SYS1.PARMLIB members:

IEFSSNxx	Subsystem definition
IKJTSOxx	TSO/E commands and programs
IFAPRDxx	Product enablement policy
EDGRMMxx	DFSMSrmm options
COMMNDnn (optional)	Commands automatically issued at initialization
SMFPRMxx (optional)	System Management Facilities (SMF) parameters
GRSRNLxx (optional)	Global resource serialization resourcename lists
IFGPSEDI (optional)	Enhanced Data Integrity
DEVSUPxx (optional)	Device Support Options
IGDSMSxx (optional)	Storage Management Subsystem definition
ALLOCnn (optional)	Allocation System defaults
AUTORnn (optional)	Auto-reply policy specifications

Perform this step once for each MVS image. Details about these members are described in the *MVS Initialization and Tuning Reference*, SA22-7592. For information about using the MVS commands, refer to the *MVS System Commands Summary*, SA22-7628.

IEFSSNxx: Subsystem definition

There are considerations when defining the DFSMSrmm subsystem name and started procedure name. You should review the information provided in this book before updating the IEFSSNxx PARMLIB member.

You need to make two changes in IEFSSNxx:

- ▶ Define a DFSMSrmm subsystem name to MVS.
- ▶ Add the name of the subsystem interface initialization program, EDGSSSI, to fully enable DFSMSrmm.

We recommend that you implement the changes in two steps. Define the DFSMSrmm system at this time (see Figure 3-2) and then you can add the name of the subsystem interface later. If you make only the first change at this time, you can choose whether to start the DFSMSrmm procedure and what time best suits your particular situation. If you implement both changes at once, DFSMSrmm rejects all tape mounts until the subsystem procedure is active, or you use the EDGRESET utility to disable the interface.

SUBSYS SUBNAME(SMS)	/* SMS	*/
INITRTN(IGDSSIIN)		
SUBSYS SUBNAME(JES2)	/* JES2 PRIMARY SUBSYSTEM START	*/
PRIMARY(YES) START(YES)		
SUBSYS SUBNAME(DFRM)	/* Name of the DFSMSrmm subsystem	*/
SUBSYS SUBNAME(AOPA)	/* NetView	*/

Figure 3-2 Defining the DFSMSrmm subsystem name

If you plan to run DFSMSrmm in parallel with any other tape management system, we recommend that you schedule an IPL to set the DFSMSrmm subsystem enabled in IEFSSNxx (see Figure 3-3) prior to the migration and parallel running.

SUBSYS	SUBNAME(SMS)	/* SMS	*/
	INITRTN(IGDSSIIN)		
SUBSYS	SUBNAME(JES2)	/* JES2 PRIMARY SUBSYSTEM START	*/
	PRIMARY(YES) START(YES)		
SUBSYS	SUBNAME(DFRM)	/* Name of the DFSMSrmm subsystem	*/
	INITRTN(EDGSSSI)	/* activate subsystem	*/
SUBSYS	SUBNAME(AOPA)	/* NetView	*/

Figure 3-3 Defining the DFSMSrmm subsystem name with interface enabled

You can use the MVS system command SETSSI to define the DFRM subsystem dynamically. You can issue the SETSSI command from one of the following consoles:

- ▶ A console that has master authority
- ▶ A console to which an operator with sufficient RACF authority has logged on

To define the DFSMSrmm subsystem, use the MVS system command shown in Figure 3-4 on page 73

SETSSI ADD,SUBNAME=DFRM

Figure 3-4 SETSSI ADD command

Where:

ADD	specifies that a subsystem is to be dynamically added.
SUBNAME	specifies the subsystem name to be dynamically added.

You can define the DFSMSrmm subsystem and activate the subsystem interface dynamically using the MVS system commands shown in Figure 3-5:

SETSSI ADD,SUBNAME=DFRM,INITRTN=EDGSSSI
SETSSI ACTIVATE,SUBNAME=DFRM

Figure 3-5 SETSSI ADD command and ACTIVATE

Where:

ADD	specifies that a subsystem is to be dynamically added.
SUBNAME	specifies the subsystem name to be dynamically added.
INITRTN	specifies the name of the subsystem initialization routine.

Figure 3-6 shows you how you can check the successful add of the subsystem by using the MVS system command.

DISPLAY SSI

Figure 3-6 Check the success of the subsystem adding

Example 3-1 shows the output of the command.

Example 3-1 *DISPLAY SSI command output*

```

IEFJ100I 19.31.54 SSI DISPLAY 121
SUBSYS=JES2 (PRIMARY)
    DYNAMIC=YES      STATUS=ACTIVE      COMMANDS=REJECT
SUBSYS=MSTR
    DYNAMIC=NO       STATUS=ACTIVE      COMMANDS=N/A
SUBSYS=SMS
    DYNAMIC=YES      STATUS=ACTIVE      COMMANDS=REJECT
SUBSYS=J21S
    DYNAMIC=YES      STATUS=INACTIVE    COMMANDS=REJECT
SUBSYS=J21A
    DYNAMIC=YES      STATUS=INACTIVE    COMMANDS=REJECT
SUBSYS=RACF
    DYNAMIC=YES      STATUS=ACTIVE      COMMANDS=REJECT
SUBSYS=OAM1
    DYNAMIC=YES      STATUS=INACTIVE    COMMANDS=REJECT
SUBSYS=BP01
    DYNAMIC=YES      STATUS=INACTIVE    COMMANDS=REJECT
SUBSYS=BP02
    DYNAMIC=YES      STATUS=INACTIVE    COMMANDS=REJECT
SUBSYS=DFRM
    DYNAMIC=YES      STATUS=ACTIVE      COMMANDS=REJECT
SUBSYS=OPCC
    DYNAMIC=YES      STATUS=ACTIVE      COMMANDS=REJECT
SUBSYS=OPCT
    DYNAMIC=YES      STATUS=ACTIVE      COMMANDS=REJECT

```

To deactivate the DFSMSrmm subsystem DFSMSrmm interface dynamically, use the MVS system command shown in Figure 3-7.

```
SETSSI DEACTIVATE,SUBNAME=DFRM
```

Figure 3-7 *IMVS command to deactivate the DFRMM subsystem interface*

IKJTSOxx: TSO/E commands and programs

Update IKJTSOxx to authorize DFSMSrmm commands and utilities. If you use the TSO CSECT facility rather than IKJTSOxx, refer to *TSO/E Customization*, SA22-7783, for more information.

To authorize the TSO RMM command, add the statement in Figure 3-8.

```

AUTHCMD NAMES(...
              RMM              /* DFSMSrmm TSO commands */
              ... )

```

Figure 3-8 *IKJTSOxx operand AUTHCMD*

To authorize the DFSMSrmm utilities that you can use within TSO, or call through the TSO Service Facility, add the statements shown in Figure 3-9 and Figure 3-10.

```

AUTHTSF NAMES(...
            EDGAUD          /* DFSMSrmm utility          */
            EDGBKUP         /* DFSMSrmm utility    2010/09/07 */
            EDGHSKP         /* DFSMSrmm utility          */
            EDGRPDT         /* DFSMSrmm utility          */
            EDGUPDT         /* DFSMSrmm utility    2010/09/07 */
            EDGUTIL         /* DFSMSrmm utility          */
            ... )

```

Figure 3-9 IKJTSOxx operand AUTHTSF

```

AUTHPGM NAMES(...
            EDGAUD          /* DFSMSrmm utility          */
            EDGBKUP         /* DFSMSrmm utility    2010/09/07 */
            EDGHSKP         /* DFSMSrmm utility          */
            EDGRPDT         /* DFSMSrmm utility          */
            EDGUPDT         /* DFSMSrmm utility    2010/09/07 */
            EDGUTIL         /* DFSMSrmm utility          */
            ... )

```

Figure 3-10 IKJTSOxx operand AUTHPGM

To use the updated version of the IKJEFTxx member dynamically, use one of the commands in your TSO session or on the MVS console shown in Figure 3-11.

```

TSO PARMLIB UPDATE(xx)
SET IKJTSO=xx

```

Figure 3-11 Update IKJEFT dynamically

See Figure 3-12 how you can check the successful addition of the RMM command and utilities using the TSO or MVS command.

```

TSO PARMLIB LIST
DISPLAY IKJTSO,ALL

```

Figure 3-12 Display the TSO PARMLIB settings

Example 3-2 shows the output of the TSO PARMLIB LIST command.

Example 3-2 *Output of the TSO PARMLIB LIST command*

```

TSO/E PARMLIB SETTINGS :

SYS1.SYSPROG.PARMLIB(IKJTS000) on volume SBOX01
Activated by **IPL** on 2012-10-13 at 15:51:13 from system SC70
Applies to :    SC70
...
CURRENT PARMLIB SETTINGS FOR AUTHCMD:

ADYOPCMD  ADA      EXITEDIT  RACTRACE  RACTR      RECEIVE
TRANSMIT  XMIT     LISTB     LISTBC    SE         SEND
CONSOLE   CONSPROF PA        PERMALOC  CONCATPA   PARMLIB

```

IKJPRMLB	TESTAUTH	TESTA	RMM	MVPXDISP	NETSTAT
TRACERTE	PING	LIBLIST	Q	QCBXA	QCBESA
DITTO	DITTOA	DITTOU	TL	RACONVRT	SYNC
LISTD	LISTDS	RACONVRT	CONSPROF	AD	ADDSD
AG	ADDGROUP	AU	ADDUSER	ALG	ALTGROUP
ALD	ALTDSD	ALU	ALTUSER	BLKUPD	CO
CONNECT	DD	DELDSD	DG	DELGROUP	DU
DELUSER	LD	LISTDSD	LG	LISTGRP	LISTDATA
LDATA	LU	LISTUSER	RALT	RALTER	RDEF
RDEFINE	RDEL	RDELETE	RE	REMOVE	RL
RLIST	RACLINK	RACDCERT	RACMAP	RVARY	PASSWORD
PW	PE	PERMIT	SHCDS	SETCACHE	SETR
SETROPTS	SINFO	SR	SEARCH	BACKUP	OPINFO
OPSTAT	SRSTAT	WSSTAT	DIAGNOSE	EXAMINE	FREMIGID
HCEMCS	ARMDREG	CQUERY	CESTPATH	CESTPAIR	CSUSPEND
CDELPAIR	CDELPATH	CRECOVER	XQUERY	XSTART	XEND
XSET	XADDPAIR	XDELPAIR	XSUSPEND	XRECOVER	XQML
SAPITEST	IHSBSNFR	IRRDPIO0	CSFTTKE	VLFNOTE	FCESTABL
FCQUERY	FCWITHDR	MSDPX	LPR		

CURRENT PARMLIB SETTINGS FOR AUTHPGM:

ARY\$TSOC	AMASPZAP	AOPBATCH	CC	EXITEDIT	HESEVMI
HESEVMX	ICKDSF	IEBCOPY	IEBGENER	IFCEREPI	Q
QCBXA	QCBESA	DRMAUTH	EDGAUD	EDGBKUP	EDGHSKP
EDGRPTD	EDGUPDT	EDGUTIL	FEC\$TSOC	HAA\$EXCP	ICHDSMOO
ICHUT100	ICHUT200	ICHUT400	IDCAMS	IRRDSCO0	IRRIRA00
IRRUT100	IRRUT200	IRRUT400	ITPENTER	CSFDAUTH	CSFDPKDS
IXCMIAPU	SINFO	DITTOA	IGWPIT	IEARELCN	ARMDREG
BDGJV1	SAPITEST	IRRDPIO0	IOEGRWAG	IOENEWAG	IOESALVG
IXCMIAPU	IXCM2APU	ADB2ATH	ADB2UTIL		

CURRENT PARMLIB SETTINGS FOR AUTHTSF:

ARY\$TSOC	IEBCOPY	ICQASLIO	ICQASLCO	DRMAUTH	IHVUUSD
ITPENTER	CSFDAUTH	CSFDPKDS	EQQMINOR	EQQMINOH	EDGAUD
EDGBKUP	EDGHSKP	EDGRPTD	EDGUPDT	EDGUTIL	FEC\$TSOC
IAMRECVR	IAMXMONA	FMNSMF	HAA\$EXCP	ADB2ATH	ADB2UTIL
CAT01IKB	IKJEFF76				

....

IFAPRDxx: Product enablement policy

When an installation order requires an IBM product, such as z/OS, that provides product enablement, IBM supplies a tailored IFAPRD00 member of SYS1.PARMLIB. This tailored member enables the product and any optional features ordered with the product. IFAPRDxx is the PARMLIB member that ensures that you are licensed to use the DFSMS components you want. IFAPRDxx replaced IGDDFPKG in OS/390 Version 1, Release 1.

Review your IFAPRDxx PARMLIB member to verify that IFAPRDxx was updated during installation to reflect that DFSMSrmm is licensed for your use (see Figure 3-13). For more information about IFAPRDxx, refer to *z/OS MVS Initialization and Tuning Reference*, SA22-7592.

```

PRODUCT OWNER('IBM CORP')
      NAME(OS/390)
      ID(5647-A01)
      VERSION(*) RELEASE(*) MOD(*)
      FEATURENAME(DFSMSRMM)
      STATE(ENABLED)

```

Figure 3-13 Enabling DFSMSRmm license into IFAPRDxx

There is also the possibility to define a global product enablement policy setting for all products as you can see in Figure 3-14

```

WHEN (SYSNAME(*))
      PRODUCT NAME(*) STATE(ENABLED)

```

Figure 3-14 Global product enablement policy

You can use the MVS SET PROD operator command (see Figure 3-15) to modify the enablement policy dynamically by specifying which IFAPRDxx member the system is to use. Statements in the IFAPRDxx member modify, not replace, an existing policy. The change to the policy takes place immediately but does not affect any product instances that are already running. To update the IFAPRDxx member dynamically, use the following MVS system command:

```

SET PROD=xx

```

Figure 3-15 SET PROD command

See Figure 3-16 how can use the MVS DISPLAY command to display the active enablement policy.

```

DISPLAY PROD,REGISTERED
or
DISPLAY PROD,STATE

```

Figure 3-16 DISPLAY PROD command

Example 3-3 on page 77 shows the output of a DISPLAY PROD command.

Example 3-3 *Output of a DISPLAY PROD command*

```

RESPONSE=SC70
IFA111I 19.53.51 PROD DISPLAY 033
S OWNER      NAME      FEATURE      VERSION  ID
E IBM CORP    z/OS        z/OS        01.12.00 5694-A01
E IBM CORP    z/OS        DFSMSHSM    **.**.**. 5694-A01
E IBM CORP    z/OS        DFSMSRMM    **.**.**. 5694-A01
E IBM CORP    z/OS        DFSMSTVS    **.**.**. 5694-A01
N IBM CORP    z/OS        JES2        01.12.00 5694-A01
N IBM CORP    z/OS        RACF        **.**.**. 5694-A01
E IBM CORP    z/OS        RMF         **.**.**. 5694-A01
E IBM CORP    z/OS        Security Server **.**.**. 5694-A01
E IBM CORP    z/OS        SDSF        **.**.**. 5694-A01

```

Example 3-4 shows you the result if you have a global product enablement policy setting.

Example 3-4 *Output of a DISPLAY PROD command with global product enablement policy*

```
RESPONSE=SC70
IFA111I 18.24.13 PROD DISPLAY 639
S OWNER          NAME          FEATURE          VERSION  ID
E *              *              *              *.*.*.*
```

IFGPSEDI: Enhanced Data Integrity

The purpose of enhanced data integrity in the IFGPSEDI member of SYS1.PARMLIB is to detect programs that cause data loss due to multiple programs updating a sequential data set simultaneously. This is a useful function. However, if your system programmer chooses to activate it, DFSMSrmm receives spurious warning messages or ABENDs.

To activate the Enhanced Data Integrity function, the system programmer performs these actions:

1. Creates SYS1.PARMLIB member IFGPSEDI with MODE(WARN) or MODE(ENFORCE)
2. RelPLs the system or enters the command shown in Figure 3-17

```
S IFGEDI
```

Figure 3-17 Set the enhanced data integrity

You can specify one of three different mode options:

DISABLE	If the member has MODE(DISABLE) or MODE is omitted, then the member has no effect.
WARN	If MODE(WARN) is in effect, then during the normal running of DFSMSrmm programs, the system issues message IEC984I or message IEC985I for the DFSMSrmm journal, ACTIVITY, MESSAGE, REPORT, REPTXT, and XREPTXT data sets. You can ignore these messages. We recommend that you code those names in the DSN parameter in the IFGPSEDI member to suppress these warning messages.
ENFORCE	If MODE(ENFORCE) is in effect, then normal running of DFSMSrmm programs causes ABEND 213-FD for the DFSMSrmm journal, ACTIVITY, MESSAGE, REPORT, REPTXT, and XREPTXT data sets. You must code the names of these data sets in the DSN parameter in the IFGPSEDI member to suppress these OPEN failures. Alternately you can change MODE(ENFORCE).

EDGRMMxx: DFSMSrmm options

You need to create an EDGRMMxx PARMLIB member for DFSMSrmm (see Example 3-5 on page 97). The member name is in the form EDGRMMxx, where xx is a two-character alphanumeric suffix of your choice. The default is EDGRMM00. A sample EDGRMMxx member is provided in SAMPLIB member EDGIVPPM. You can copy it to PARMLIB, and modify it according to your installation requirements.

In the sections that follow, we describe the most important options to specify in the EDGRMMxx PARMLIB member when DFSMSrmm is running in parallel to your tape

management system. Use the following information together with the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, which describes in detail the options you can use with DFSMSrmm.

OPTION command

This section details the various OPTION commands used within EDGRMMxx. We recommend that you specify the following OPTION subparameters in EDGRMMxx:

► OPMODE(R)

Specifies the mode of DFSMSrmm.

R / Record-only mode You can use DFSMSrmm TSO commands, ISPF dialog, and inventory management functions. In addition, DFSMSrmm records information about magnetic tape volumes used on the system, including details about volume owners and data set names.

For volumes in an automated tape library dataset server, DFSMSrmm records information about entry and eject activities. During entry processing, DFSMSrmm provides volume status information to OAM. If a volume is not already defined to DFSMSrmm, DFSMSrmm automatically defines it as long as there are no conflicts with either the VOLSER or corresponding rack number.

DFSMSrmm does not validate or reject volumes when in record-only mode. When DFSMSrmm is running in record-only mode, the volume status in the TCDB is not updated. When you issue DFSMSrmm commands, the TCDB is updated if appropriate and the volume is system-managed. When volumes are returned to scratch status, no TCDB updates are made.

Table 3-1 and Table 3-2 on page 79 provide information about the options which are affected by the OPMODE value.

Table 3-1 How OPMODE Honors the Settings of Various Options

Option	Manual	Record only	Warning	Protect
EXPDTCHECK	N	N	Y	Y
UNCATALOG	Y	Y	Y	Y
TPRACF	Y	Y	Y	Y
SMSTAPE(UPDATE)	Y	Y	Y	N
SMSTAPE(PURGE)	N	Y	Y	Y

Table 3-2 How OPMODE Value Affects System-Managed Tape Library Support

Main Area of Activity	Manual	Record only	Warning	Protect
Command Processing	OPT	OPT	OPT	YES
Expiration Processing	OPT	OPT	OPT	YES

Main Area of Activity	Manual	Record only	Warning	Protect
Support for CBRUXxxx Exits	N/A	OPT	OPT	YES
Purging TCDB Records During Eject Processing	N/A	OPT	OPT	YES
Legend: ▶ N/A Function not available ▶ OPT Function can be controlled by SMSTAPE option ▶ YES Function is always provided				

▶ **BACKUPPROC(EDGBKUP)**

Specifies the name of the procedure that you want to start automatically when the journal percentage full threshold is reached.

Specify a valid alphanumeric procedure name from 1 to 8 characters. If no name is specified, then no automatic start command is issued.

▶ **CDSID(PROD)**

Specifies the identifier of the control data set that must be used on this system. Specify a value one to eight characters long. This ID is used by DFSMSrmm Web services to distinguish retrieved data between multiple control data sets. Ensure that each DFSMSrmm control data set has a unique CDSID ID.

▶ **COMMANDAUTH(OWNER)**

Specifies the type of authorization that DFSMSrmm is to check. Specify OWNER when you expect the owners of volume information and data set information to be able to update their own data sets and volumes using RMM TSO subcommands. Specify DSN when you expect changes to volume and data set information to be authorized using the RACF DATASET class and TAPEVOL class.

▶ **DATEFORM(J)**

Specifies the date format for messages and reports. Use the date format preferred in your computing center as shown in Table 3-3.

Table 3-3 *DATEFORM options*

Value	Language	Format	Example
A	American	mm/dd/yyyy	12/15/2010
E	European	dd/mm/yyyy	15/12/2010
I	ISO	yyyy/mm/dd	2010/12/15
J	Julian	yyyy/ddd	2010/349

▶ **DISPDDNAME(LOANDD)**

Specifies the name of the DD card that identifies the data set, which contains disposition control statements that DFSMSrmm processes during CLOSE or EOVS processing. For detailed information, refer to 6.5, "Disposition processing" on page 251.

▶ **DISPMSGID(EDG4054I)**

Specifies the message number that DFSMSrmm uses for write-to-operator messages specified in the disposition control file.

▶ **DSNAME(RMM.PROD.CDS)**

Specifies the name of the DFSMSrmm CDS. Specify a name up to 44 characters long.

Note: If you do not specify DSNAME, you must specify the data set name in the MASTER DD statement in the DFSMSrmm started procedure. If you specify a name for both DSNAME and MASTER DD, DFSMSrmm ignores the MASTER DD statement.

► **EXPDTDROP(PERCENT(10,WARN))**

Use EXPDTDROP to specify a maximum number or percentage of existing expiration date retained volumes that can be dropped from retention and the action to be taken by DFSMSrmm. DFSMSrmm counts the number of EXPDT-retained physical and logical volumes at the start of inventory management expiration processing and the number of these to be set to pending release. An EXPDT-retained volume is one that is not VRS-retained and is not newly assigned. When you specify count, this is an absolute maximum number of volumes that can be released by a single run of EDGHSKP EXPROC processing. When you specify a percentage, this is a maximum percentage of the existing EXPDT-retained volumes that can be released by a single run of EDGHSKP EXPROC processing. This processing occurs each time that you run inventory management EXPROC processing.

When VRSEL and EXPROC are run together in a single EDGHSKP run, volumes that are dropped by VRSEL processing are counted only towards the VRSDROP limit, not to the EXPDTDROP limit.

► **GDG(CYCLEBY(GENERATION) DUPLICATE(BUMP))**

Use the GDG option to specify how generation data groups are handled for cycle retention by VRSEL processing. Cycle retention includes both the CYCLES and the BYDAYSCYCLE retention types. The correct sequence for determining the retention can be either by using the generation number or the creation order. You can also specify how duplicate generations (generation data sets) are handled and have the flexibility to include or exclude duplicate generations from the cycles count as required by your application processing.

Where:

CYCLEBY(GENERATION) Specifies that retention is to be based on the generation number. DFSMSrmm determines the generation number by applying a similar algorithm to that used by Catalog processing. Both the creation order, and the generation number from the data set name are considered, allowing wraps in generation number to be correctly handled.

CYCLEBY(GENERATION) is the default.

DUPLICATE(BUMP) Specifies that a duplicate generation is to be bumped by the current VRS sub chain and considered for retention by a subsequent VRS sub chain.

DUPLICATE(BUMP) is the default.

► **JRNLCNAME(RMM.PROD.JRNLC)**

Specifies the name of the journal. Specify a name up to 44 characters long.

Note: If you do not specify JRNLMAME, you can specify a name in the JOURNAL DD statement in the DFSMSrmm started procedure. If you do not specify a journal name in either EDGRMMxx or the started procedure, DFSMSrmm does not provide journaling. If you specify a name in both, DFSMSrmm uses the JRNLMAME value and ignores the JOURNAL DD statement.

► JRNLMTRAN(NO)

Use the JRNLMTRAN operand to specify whether the unchanged copy of a CDS record is journaled, as well as the updated copy.

Where:

NO	Specifies that only the updated copy of the CDS record is to be journaled.
YES	Specifies that the pre-update copy of the CDS record being updated is to be journaled, in addition to the updated copy of the CDS record. Set this option only on a test or recovery system when you plan to use the EDGUPDT utility to duplicate CDS record updates back in the production CDS. As a result of using this option you should plan on providing up to 33% more journal data set space to accommodate the additional records.

► MASTEROVERWRITE(LAST)

Specify to control how DFSMSrmm allows the overwriting of a volume.

Where:

ADD	Specify this value so new data can be created and no existing data can be destroyed. No existing file on a volume can be re-created, but the last file can have new data added to it. When adding data to the last file, DFSMSrmm checks that the data set name used must match the existing data set name. Select this option when you want the last file on the volume to be extended or a new file added to the volume.
------------	--

Note: DFSMSrmm enforces the MASTEROVERWRITE(ADD) option on a WORM tape that is in master status. This is done to ensure that you see a message from DFSMSrmm rather than one of a number of symptoms as a result of the tape drive preventing overwrites.

LAST	Specify this value to ensure that when an existing file on a master volume is being written to that only the last file on the volume can be used. The data set name used must match the existing data set name. Select this option when you want the last file on the volume to be used for output.
MATCH	Specify this value to ensure that when an existing file on a master volume is being used for output that exactly the same data set name must be used. Select this option when you want any existing file on the volume to be re-created regardless of whether it is the last file on the volume as long as the same data set name is used. When you use an existing tape file for output all the files that are higher in sequence are destroyed.
USER	Specify this value to allow any existing file on a master volume to be used for output regardless of the data set names being

used and its relative file position on the volume. Select this option when you want validation of master volumes to be just the same as for user status volumes. When you use an existing tape file for output all the files that are higher in sequence are destroyed.

Default: MASTEROVERWRITE(LAST)

► **MAXHOLD(100)**

Specifies the maximum number of activities DFSMSrmm performs to the CDS before the reserve is released and reacquired. Specify MAXHOLD to minimize the impact that long operations, such as RMM TSO ADD subcommands with large count values, or large searches, have on other users.

► **MAXRETPD(NOLIMIT)**

Specifies the maximum retention period that a user can request for data sets on volumes. Specify NOLIMIT or a value between 0 and 93000 days. When a value between 0 and 93000 days is specified, the value is added to the current date to determine the maximum allowed expiration date.

Where:

NOLIMIT

Specify NOLIMIT to use the dates 99365 or 99366, which means the data sets never expire. If the calculated date is 31 December 1999, the expiration date 1 January 2000 is used.

If the DFSMSrmm ISPF dialog or RMM TSO subcommands are used to specify a retention period or expiration date that exceeds the MAXRETPD value, DFSMSrmm fails the subcommand or panel request.

► **MCATTR(VRSELXDI)**

Specifies whether DFSMSrmm should be enabled to use of DFSMS management class (MC) expiration attributes that apply to tape management. The management class expiration attributes processed by DFSMSrmm are:

- Expire after days Non-usage, which is equivalent to LASTREF(extra days)
- Expire after Date/Days, which is equivalent to expiration date / retention period

Where:

–

ALL

The use of all applicable management class attributes is enabled. The MC attributes are exploited as they are appropriate, depending on the retention method. The MC attribute Retention Limit is not applied.

NONE

No management class attributes are used. NONE is the default.

VRSELXDI

The use of management class attributes is enabled. The MC attributes are exploited as they are appropriate, depending on the retention method, with the exception of the MC attribute Expire after Date/Days, which is ignored if the data set is on a volume managed by the VRSEL retention method. The MC attribute Retention Limit is not applied. VRSELXDI is recommended if it is desired that the processing of VRSEL managed volumes is the same as in prior DFSMSrmm releases.

► **MEDIANAME(3490)**

Specify to set a default medianame value that DFSMSrmm uses when you do not specify a media name for a volume. The medianame is used when you add volumes, define pools in your installation, define a default pool for your installation, and when the EDGINERS utility selects volumes for automatic processing.

► **MOVEBY(VOLUME)**

Specify the MOVEBY operand to move volumes that are retained by DFSMSrmm vital record specifications as a set of volumes or as individual volumes.

Where:

SET

Specify this value when you want volumes moved as a set. DFSMSrmm moves all volumes in the same multivolume set that are retained by vital record specifications. All the volumes are retained in the same location selected by the vital record specification priority or location priority for the volume.

VOLUME

Specify this value when you want volumes moved as individual volumes. DFSMSrmm moves the volumes without considering the location of the other volumes in the multivolume set.

► **PREACS(NO)**

Specify the PREACS operand to control whether DFSMSrmm-supplied and EDGUX100 installation exit-supplied values are input to SMS pre-ACS processing:

Where:

NO

Specify NO to avoid DFSMSrmm Pre-ACS processing using the DFSMSrmm EDGUX100 installation exit.

► **RETAINBY(VOLUME)**

Use the RETAINBY option for volumes managed by the VRSEL retention method to specify whether DFSMSrmm retains multivolume sets as a set or as individual volumes.

Note: For volumes managed by the EXPDT retention method, use the RETAI BY suboption of the RETENTIONMETHOD(EXPDT) option to obtain a similar function.

Where:

SET

When you retain by set, if any volume in a set is retained by a vital record specification, all volumes in the set are retained as vital records. DFSMSrmm uses highest retention date of all volumes in the set as the retention date for all volumes retained as vital records in a set. If no volume in a set is retained by a vital record specification, DFSMSrmm performs expiration processing by set. DFSMSrmm does not expire volumes in a set if at least one volume in a set is still not ready to expire because it has not reached its expiration date and you have not specified that you want the expiration date ignored.

VOLUME

When you retain by volume, DFSMSrmm retains a volume based on vital record specifications and on the volume expiration date. DFSMSrmm does not consider other volumes in the set.

► **RETENTIONMETHOD(VRSEL)**

Use this operand to set the system-wide retention method for new tape volume sets created during Open/Close/End-of-Volume (O/C/EOV) processing, and for tape volumes added to the DFSMSrmm CDS. RETENTIONMETHOD can be abbreviated as RM.

Where:

VRSEL

Specify VRSEL to set the default retention method for new tape volume sets to be VRSEL. This option enables DFSMSrmm inventory management to attempt to match data sets and volumes to vital record specifications, and if a match is found, to determine if the data set or volume is to be retained by VRS. The VRSEL retention method is controlled by all the other VRS related options in parmlib including OPTION RETAINBY and MOVEBY.

► **RETPD(5)**

Specifies the default retention period for all new data sets on volumes. Specify a value between 0 and 93000 days. The specified value is added to the current date to determine the expiration date. Select a default retention for parmlib RETPD that is a small value to ensure that all tape data created outside the service levels is released as soon as possible. The MAXRETPD value you specify in the parmlib limits the calculated expiration date.

Note: The value you specify for the RETPD operand should be the same as the value you use in your tape management system.

► **RETENTIONMETHOD(VRSEL)**

Use this operand to set the system-wide retention method default for new tape volume sets. New tape volume sets may be created during Open/Close/End-of-Volume (O/C/EOV) processing, or through DFSMSrmm commands. A tape volume set may be a multivolume set, or a single tape volume.

Where:

EXPDT

Specify EXPDT to set the default retention method for new tape volume sets to be based on EXPDT. Data sets and volumes managed by this retention method are never processed by VRSEL inventory management. When you specify the EXPDT retention method the DFSMSrmm inventory management EXPROC processing always attempts to return volumes to scratch on the same run as the volume is released (this is as if the SCRATCHIMMEDIATE attribute is set for the volume). DFSMSrmm maintains a consistent expiration date and time for all data set records of a multivolume data set, unless the volume set is retained by first file. EXPDT can be specified either as EXPDT or EXPDT(options). The available options are:

LASTREF(extra_days) LASTREF specifies the default for the data set record LASTREF attribute. The LASTREF attribute specifies the number of days that the data set will be retained after the data set was last referenced by a read or write operation . LASTREF applies only to data sets on volumes managed by the EXPDT retention method.

extra_days

is a decimal number between 0 and 93000. The value must not exceed the maximum retention period (MAXRETPD) specified in the EDGRMMxx parmlib member. An extra_days value of 0 has the same effect as the NOLASTREF operand.

When a data set is added to DFSMSrmm on a volume managed by the EXPDT retention method and neither LASTREF nor NOLASTREF are specified for the data set, then DFSMSrmm uses the default LASTREF value.

DFSMSrmm uses the data set LASTREF value to determine the data set expiration date. The extra days are added to the date of last reference. The data set expiration date is set to the maximum of the date calculated using data set LASTREF value and the date resulting from applying the EXPDT, RETPD, or default RETPD. Any reference to the data set by a read or write operation will change the expiration date.

If neither LASTREF nor NOLASTREF are specified in parmlib, NOLASTREF is used by default.

NOLASTREF

NOLASTREF is the default setting for the data set record LASTREF attribute. NOLASTREF specifies that DFSMSrmm does not consider the data set last reference date when determining the data set expiration date. NOLASTREF applies only to data sets on volumes managed by the EXPDT retention method.

RETAINBY(FIRSTFILE | SET | VOLUME)

RETAINBY specifies how DFSMSrmm is to retain volumes or multivolume sets that are managed by the EXPDT retention method:

FIRSTFILE

The expiration date of the first file is used to set the expiration date of the volume or multivolume set. All volumes in a set will have the exact same expiration date and will be released to scratch in the same run of DFSMSrmm inventory management.

SET

The expiration date is the maximum of all data set expiration dates of all the volumes in the set. All volumes in the set will have the exact same expiration date. Any file on any volume of the set can increment the volume expiration date. All files of a multivolume data set have the same expiration date.

VOLUME

The expiration date is determined separately for each volume in the set. Unless defined differently, the expiration date is the maximum of all data set expiration dates on the volume. Each file on a volume can increment the volume expiration date. All files of a multivolume data set have the same expiration date.

Note:

1. In a multivolume set, RETAINBY is assigned only to the first volume in a multivolume sequence. All other volumes added to the set assume the same RETAINBY.
2. For volumes managed by the VRSEL retention method, use the RETAINBY option to obtain a similar function.

VRSEL

Specify VRSEL to set the default retention method for new tape volume sets to be VRSEL. This option enables DFSMSrmm inventory management to attempt to match data sets and volumes to vital record specifications, and if a match is found, to determine if the data set or volume is to be retained by VRS. The VRSEL retention method is controlled by all the other VRS related options in parmlib including OPTION RETAINBY and MOVEBY.

► **RETPD(5)**

Specifies the default retention period for all new data sets on volumes. Specify a value between 0 and 93000 days. The specified value is added to the current date to determine the expiration date. Select a default retention for parmlib RETPD that is a small value to ensure that all tape data created outside the service levels is released as soon as possible. The MAXRETPD value you specify in the parmlib limits the calculated expiration date.

DFSMSrmm sets a default retention period as follows:

- If you specify RETPD or EXPDT, the value is used as the data set's new expiration date.
- If you do not specify RETPD or EXPDT, then:
 1. DFSMSrmm uses the EXPDT or RETPD allocation attribute of a data class, if all these are true:
 - The data set is associated with a data class, either explicitly by the DATACLAS keyword on the JCL or implicitly by an automatic class selection routine
 - The data class has an EXPDT or RETPD allocation attribute
 - The Storage Management Subsystem is active.
 2. DFSMSrmm uses the management class Expire after Date/Days as the data set's new expiration date, if all these are true:
 - You do not specify RETPD or EXPDT in the JCL or data class
 - The data set is assigned to a management class
 - The management class Expire after Date/Days is set
 - The EDGRMMxx parmlib option MCATTR allows the exploitation of the management class for this type of volume
 - The Storage Management Subsystem is active.
 3. DFSMSrmm uses the default retention period set in EDGRMMxx, if RETPD or EXPDT are not otherwise specified.

Whenever a new data set is written to tape, DFSMSrmm checks whether the volume's expiration date should be updated, based on whether the new data set has a longer expiration date than the volume on which it is written. DFSMSrmm gets the expiration date for a data set from the job file control block (JFCB) or from the management class at open time. If there is a date in the JFCB or management class, DFSMSrmm compares this date to the current expiration date for the volume. If the date in the JFCB allows the volume to be retained longer, DFSMSrmm uses that date to update the volume's expiration date.

If there is no expiration date in the JFCB or management class, DFSMSrmm uses the EDGRMMxx RETPD value to calculate the new expiration date. If the RETPD value allows the volume to be retained longer, DFSMSrmm uses that date to update the volume's expiration date.

You can set the date in the JFCB in several ways, including:

- RETPD and EXPDT keywords in the JCL

- Data class when the Storage Management Subsystem is active and the volume is system-managed
- A user program, using the RDJFCB macro and the OPEN TYPE=J after modifying the JFCB
- Installation exits in use on your particular system

If the use of the management class attributes is enabled by the EDGRMMxx parmlib OPTION MCATTR, you can use the management class Expiration Attributes ('Expire after Date/Days' and 'Expire after Days Non-usage') to set the data set EXPDT and LASTREF attributes, where appropriate, to define more specific default retention periods.

When you use the VRSEL retention method, you can also use vital record specifications to define more specific default retention periods for users by using a data set name prefix.

Default: RETPD(5)

► REUSEBIN(STARTMOVE)

Specify to control how DFSMSrmm reuses bins when a volume is moving from a bin.

Where:

STARTMOVE

A bin can be reused as soon as a volume starts moving out of a bin. Extended bin support must be enabled before you can use this operand. For more information, see the *DFSMSrmm Implementation and Customization Guide*, SG26-7405, and the chapter in that book titled “Enabling Extended Bin Support” to enable extended bin support.

Note: The use of STARTMOVE is recommended because most vendor tape management systems reuse bins directly.

► SMFAUD(YES)

Specifies the SMF record type to be used for audit records. Specify YES or a number between 128 and 255 that is different from the value for SMFSEC. The value must conform to standard SMF conventions.

Where:

YES

IBM recommends that you do not use an SMF record type number nnn, but instead use the IBM assigned record number by specifying YES. The IBM assigned record number is type 42, and the subtype is 22. You cannot mix SMF record types. For example, you cannot specify YES for SMFAUD and a record type for SMFSEC.

► SMFSEC(YES)

Specifies the SMF record type to be used for security records. Specify YES or a number between 128 and 255 that is different from the value for SMFAUD. The value must conform to standard SMF conventions.

Where:

YES

IBM recommends that you do not use an SMF record type number nnn, but instead use the IBM assigned record number by specifying YES. The IBM assigned record number is type 42, and the subtype is 23. You cannot mix SMF record types. For example, you cannot specify YES for SMFAUD and a record type for SMFSEC.

► SMSACS(YES)

Specify this operand to control whether DFSMSrmm calls SMS ACS processing to enable use of storage group and management class values with DFSMSrmm.

Where:

YES

Specify YES to enable DFSMSrmm calls to the SMS ACS processing to obtain management class and storage group names. If values are returned by the SMS ACS routines, the values are used instead of the DFSMSrmm and EDGUX100 decisions.

► **SMSTAPE(UPDATE(EXITS, SCRATCH, COMMAND), PURGE(YES))**

Use SMSTAPE to specify how DFSMSrmm updates the TCDB and controls system-managed tape processing.

Where:

UPDATE

Use UPDATE to select the system-managed tape functions DFSMSrmm provides. The UPDATE operand has three subparameters: EXITS, SCRATCH, and COMMAND. You can specify one or more of the subparameters. When DFSMSrmm is running in PROTECT mode, DFSMSrmm ignores the UPDATE operand and performs processing as though you specified EXITS, SCRATCH, and COMMAND. When DFSMSrmm is running in WARNING or RECORD mode, DFSMSrmm does not update TCDB information unless you request the update. You can specify one or more of the values. When you specify a value, DFSMSrmm performs the updates to the TCDB.

- **EXITS:** Specify when you want DFSMSrmm volume status information to override the OAM volume status during entry processing and you want to use the DFSMSrmm VNL exit.
- **SCRATCH:** Specify when you want DFSMSrmm to update the volume status in the TCDB during expiration processing when volumes are returned to scratch status.
- **COMMAND:** Specify when you want to use the RMM TSO subcommands or the DFSMSrmm API to update the TCDB. This controls change of status, TDSI and owner information, eject processing and manual cartridge entry processing.

PURGE

Use PURGE to control how DFSMSrmm affects the TCDB volume records during EJECT processing. The default is PURGE(ASIS) in all operating modes except MANUAL mode. In manual mode, DFSMSrmm provides no support for eject processing.

- **YES:** Specify when you want DFSMSrmm to force the TCDB volume records to be purged at eject time.

► **TPRACF(N)**

Do not create RACF tape profiles. Refer to chapter *Chapter 13, "Tape security with z/OS V1.8"* on page 485

Attention: DFSMSrmm manipulates your RACF TAPEVOL and TAPEDSN profiles in each operating mode if the TPRACF function is set to A or P.

► **TVEXITPURGE**

Specifies how DFSMSrmm processes volumes to be purged by callers of EDGTVEXT or EDGDFHSM.

Where:

EXPIRE(days)	Use the EXPIRE(days) option to set the volume expiration date to the current date plus days for volumes to be purged. Use the EXPIRE(days) parameter when you use the EXPDT retention method, using days as a way to delay expiration of the volume. When using the VRSEL retention method, you can optionally use this operand in combination with vital record specifications that use the UNTILEXPIRED retention type. First, the EXPIRE(days) is applied to set a new volume EXPDT. Next, by running VRSEL you can then optionally extend retention using the extra days retention type. For example, specify EXPIRE(0) when using a VRS with extra days retention or you can use a non-zero EXPIRE(days) value and avoid using an extra days retention VRS.
days	is the number of days for which DFSMSrmm retains the volume before considering it for release. The value is a one to four digit decimal number and is added to today's date to compute the new expiration date. If the value exceeds the maximum retention period (MAXRETPD) it is reduced to the MAXRETPD value. The default value for days is 0. That is, TVEXTPURGE(EXPIRE) is the same as TVEXTPURGE(EXPIRE(0)).
NONE	DFSMSrmm take no action for volumes to be purged.
RELEASE	DFSMSrmm releases volume to be purged according to the release actions set for the volume. You must run expiration processing to return a volume to scratch status. This is the default.

► **UNCATALOG(N)**

Do not use DFSMSrmm uncatalog processing if DFSMSrmm is not your primary tape management system. If UNCATALOG(N) is set, DFSMSrmm does not uncatalog data sets under any circumstances.

Attention: DFSMSrmm uncatalogs data sets in each operating mode if the UNCATALOG function is set to Y or S.

► **VRSDROP(PERCENT(10,WARN))**

VRS-retained volumes that can be dropped from vital records retention and the action to be taken by DFSMSrmm. DFSMSrmm counts the number of VRS-retained physical and logical volumes at the start of vital record selection processing and the number of these dropped by vital record processing. When you specify COUNT, this is an absolute maximum number of volumes that can be dropped by a single run of EDGHSKP VRSEL processing. When you specify PERCENT, this is the maximum percentage of the existing VRS-retained volumes that can be dropped by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing.

VRSDROP processing is intended to provide limited checking for volumes that are already VRS-retained. It considers how many of these existing VRS-retained volumes are

removed from VRS retention. Those volumes dropped from VRS-retention can be set pending release, depending on VRS release options and the volume expiration date. They can also become EXPDT retained and on the next run of EXPROC, they will be considered by EXPDTRDOP limit processing.

► **VRSJOBNAME(2)**

Specify this operand to select whether DFSMSrmm uses the job name from VRS as the primary or secondary match.

Where:

VRSJOBNAME(2)	The data set name takes precedence over the job name during VRS match processing. This is the option that most closely matches the use of job names in your tape management system.
----------------------	---

► **VRSRETAIN(PERCENT(80,WARN))**

Use VRSRETAIN to specify a minimum number or percentage of newly assigned volumes that are to be retained by vital records retention and the action to be taken by DFSMSrmm. A newly assigned volume is one that has a volume assignment date and time that is higher than the run date and time of the previous VRSEL processing and that is not VRS-retained. DFSMSrmm counts the number of newly assigned physical and logical volumes at the start of vital record selection processing and the number of these that become VRS retained by vital record processing. When you specify count, this is an absolute minimum number of volumes that are to be retained by a single run of EDGHSKP VRSEL processing. When you specify a percentage, this is the minimum percentage of the newly assigned volumes that are to be retained by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing.

VRSRETAIN processing is intended to provide limited checking for volumes containing newly created data sets that have not yet been processed by inventory management VRSEL. The data sets on the volumes have been created since the start of the last completed VRSEL run. It considers how many of these volumes become VRS-retained during the new VRSEL run. Those volumes that become VRS-retained will be considered by the VRSDROP limit checking in future VRSEL runs. Those volumes that are not retained by VRS can be set pending release, depending on VRS release options and the volume expiration date. They can also become EXPDT retained and on the next run of EXPROC, they will be considered by EXPDTRDOP limit processing.

► **VRSEL(NEW)**

Use VRSEL to control how DFSMSrmm inventory management vital record processing uses retention and movement information defined in VRSs.

Where:

NEW

Specify whether you want:

- DFSMSrmm to process retention information in NAME VRSs, release options defined in VRSs, and VRSs chained using the RMM ADDVRS operand.
- To convert using the conversion tools and require functions close to that given by your tape management system.

Note: The VRSEL option OLD is no longer supported.

REJECT command

Attention: The new command to controlling the use of tape volumes is OPENRULE and for partitioning your tape libraries is PRTITION. When you use either PRTITION or OPENRULE commands, the REJECT commands are no longer used so you must start using both PRTITION and OPENRULE at the same time to avoid loss of function.

DFSMSrmm can reject a range of tapes only if you first define them to DFSMSrmm. It can reject all tapes not defined to it, which you specify with a single asterisk for the prefix, but it cannot reject a range of tapes not defined to it.

You can specify multiple REJECT commands to define different ranges of tapes. However, use only one ANYUSE operand and one OUTPUT operand for each REJECT command.

Where:

ANYUSE(*)	Specifies the shelf locations of volumes not available for read or write processing. You can specify a generic shelf location consisting of a one to five character prefix followed by an *. Specify a single asterisk if, at OPEN time, you want to reject all volumes not defined to DFSMSrmm. You can use the installation exit, EDGUX100, to request that DFSMSrmm ignore such tapes.
------------------	---

Note: If you specify REJECT ANYUSE(*), all tapes not defined to the DFSMSrmm CDS cause an error message to be issued by DFSMSrmm when the tapes are used.

MEDIINF commands

Use the MEDINF command to define media characteristics for OEM media products. The MEDINF values convert external values for media type and recording technology to internal values when supplied in the DFSMSrmm TSO subcommand input, as well as externalizes recorded values in reports and DFSMSrmm TSO subcommand output. To connect a volume to a specific media information entry, assign the media information name with the RMM TSO subcommands ADDVOLUME and CHANGEVOLUME using the MEDINF operand.

- **NAME(medinf_name)**
Specifies a name for the media information. Specify a value between 1 and 8 alphanumeric characters. This value is used to match the recorded volume information to the MEDINF installation defined media information. The values defined for MEDIATYPE, RECORDINGFORMAT, and CAPACITY are taken if the assigned media information for a volume matches medinf_name.
Default: None.
- **MEDIATYPE(mediatype_id,mediatype_name)**
Use this operand to identify the internally-used mediatype_id to be converted to the externally-used mediatype_name in reports and in the output of the DFSMSrmm TSO subcommands. Also, use this operand to identify the externally-used mediatype_name to be converted to the internally-used mediatype_id that is recorded in the control data set when used as input to RMM TSO subcommands.

When you specify multiple MEDINF entries that use the same mediatype_id, but with different mediatype_name values, the first such MEDINF entry is used for the internal-to-external conversion. The remaining values are synonyms used for external-to-internal conversions.

mediatype_id	is a number in the range from 1 to 255.
mediatype_name	is 1 to 8 alphanumeric or national characters.

Default: None. When you do not specify the MEDIATYPE operand, DFSMSrmm displays a mediatype_id of 0 and mediatype_name of *.

► **RECORDINGFORMAT(recordingformat_id,recordingformat_name)**

Use this operand to identify the internally-used recordingformat_id to be converted to the externally-used recordingformat_name in reports and in the output of the DFSMSrmm TSO subcommands. Also, use this operand to identify the externally-used recordingformat_name to be converted to the internally-used recordingformat_id that is recorded in the control data set when used as input to RMM TSO subcommands.

When you specify multiple MEDINF entries that use the same recordingformat_id, but with different recordingformat_name values, the first such MEDINF entry is used for the internal-to-external conversion. The remaining values are synonyms used for external-to-internal conversions.

recordingformat_id is a number in the range from 1 to 255.

recordingformat_name is 1 to 8 alphanumeric or national characters.

Default: None. When you do not specify the RECORDINGFORMAT operand, DFSMSrmm displays a recordingformat_id of 0 and recordingformat_name of *.

► **CAPACITY(capacity)**

Specifies the media capacity in MB available on the non-IBM media that has a media type mediatype_id and a recording format recordingformat_id. Specify a value from 0 to 4294967295.

When you specify a capacity value, DFSMSrmm uses that value to set the capacity of a volume when the volume is added or when the recording format or media type of the volume is manually changed. When new data is written on a volume, DFSMSrmm uses your specified capacity only if the tape drive does not return the volume capacity.

Do not specify the CAPACITY operand on a MEDINF command that specifies the same MEDINF NAME and MEDIATYPE mediatype_id and RECORDINGFORMAT recordingformat_id as an earlier MEDINF command. If you do, an information message EDG0243I is issued at start-up time, the CAPACITY operand is ignored, and the capacity from the earlier entry is used instead. When you use synonyms, use the CAPACITY operand only when the combination of MEDINF NAME, MEDIATYPE mediatype_id, and RECORDINGFORMAT recordingformat_id are unique.

Default: When you do not specify the CAPACITY operand, DFSMSrmm displays a capacity of 0.

► **REPLACE(PERM | TEMP | AGE | WMC)**

Use this operand to identify the policies for replacement of volumes based on such values as read and write errors, age, and numbers of times written. Volumes are identified for replacement when one or more of the policy values are met or exceeded.

Do not specify the REPLACE operand on a MEDINF command that specifies the same MEDINF NAME and MEDIATYPE mediatype_id and the RECORDINGFORMAT recordingformat_id as an earlier MEDINF command. If you do, an information message EDG0243I is issued at start-up time, the REPLACE operand is ignored, and the replace policy from the earlier entry is used instead. When you use synonyms, only use the REPLACE operand if the combination of MEDINF NAME, MEDIATYPE mediatype_id and RECORDINGFORMAT recordingformat_id are unique.

When you add or modify a volume replacement policy, DFSMSrmm does not implement it retrospectively. Volumes that are already set with the REPLACE release action are not reprocessed and the action reset. In order to have DFSMSrmm reconsider the policy for

all non-pending release volumes, you first have to manually change the REPLACE release action to SCRATCH, and then run EXPROC processing.

► **PERM(count)**

Use this operand when your policy is to be based on permanent I/O errors. DFSMSRmm compares your value with the sum of permanent read and write errors over the life of the volume.

The value range is 0 to 99999. Specify 0 to disable replacement based on permanent errors.

There is no default value.

► **TEMP(count)**

Use this operand when your policy is to be based on temporary I/O errors. DFSMSRmm compares your value with the sum of temporary read and write errors over the life of the volume.

The value range is 0 to 99999. Specify 0 to disable replacement based on temporary errors.

There is no default value.

► **AGE(years)**

Use this operand when your policy is to be based on the age of the volume. DFSMSRmm compares your value with the volume creation, and current date and time.

The value range is 0 to 99999. Specify 0 to disable replacement based on age.

There is no default value.

► **WMC(count)**

Use this operand when your policy is to be based on how many times the volume is mounted for output and written to. DFSMSRmm compares your value with the write mount count for the volume.

The valid value range is 0 to 65535. Specify 0 to disable replacement based on volume write mount count. Values in the range 65536 to 99999 are accepted, but have the same effect as specifying 0 (replacement based on volume write mount count is disabled).

There is no default value.

OPENRULE commands

Use the OPENRULE command to identify special processing for DFSMSRmm to perform during OPEN processing. It applies equally to both system-managed and non-system managed volumes.

In a shared CDS RMMplex, these commands allow you to control sharing of volumes between systems while still maintaining a single CDS. OPENRULE allows you to specify what cannot be used as well as what can be used.

– **OPENRULE VOLUME(*) TYPE(NORMM) ANYUSE(IGNORE BY(ANY))**

All volumes “VOLUME(*)” not defined in the DFSMSRmm control data set “TYPE(NORMM)” can be used with the DFSMSRmm bypass processing keyword in your JCL (EXPDT=98000) if you have the needed access to the
STGADMIN.EDG.IGNORE.TAPE.NORMM.* or
STGADMIN.EDG.IGNORE.TAPE.*
resource defined in RACF class FACILITY

– **OPENRULE VOLUME(*) TYPE(RMM) ANYUSE(ACCEPT)**

This rule will allow “ANYUSE(ACCEPT)” the normal use of all volumes “VOLUME(*)” defined to DFSMSrmm “TYPE(RMM)”

PRTITION commands

Use the PRTITION command to identify special processing for DFSMSrmm to perform during library entry, export/import, eject, and CUA processing, during OPEN processing, and EXPROC processing. The PRTITION commands apply to all volumes, not only system-managed volumes. However, only system-managed volumes benefit for the functions such as library insert, CUA, and eject.

– **PRTITION VOLUME(*) TYPE(NORMM) SMT(IGNORE) NOSMT(IGNORE)**

This command will reject all volume “VOLUME(*)” requests not defined in the DFSMSrmm CDS “TYPE(RMM)”. There is no differentiation between system-managed tapes “SMT(IGNORE)” and non-system-managed tapes “NOSMT(IGNORE)”

SECCLS command

Use the SECCLS command to define security classes for data sets and volumes. These security classes appear in reports and RMM TSO subcommand output. DFSMSrmm records these security classes in the DFSMSrmm CDS only, it does not make RACF aware of them. There is no connection between these definitions and any similar definitions or function provided in RACF, but you can use similar values for overall consistency.

The examples of the SECCLS commands provided in Example 3-5 on page 97 shows the use of the following functions:

- ▶ Volumes must be erased on release.
- ▶ Issue operator confirmation messages.
- ▶ Write SMF security records each time a tape data set in this class is opened.

MNTMSG command

Use the MNTMSG command to tailor mount and fetch messages so they display the VOLSER and pool name or rack number. This information makes it easier for the operator to pull and mount volumes.

Important: Do not change the mount and fetch messages if you are using a vendor robotics system like an Oracle VSM.

VLPOOL command

Use the VLPOOL command to define pools. When you add a new volume to the library, DFSMSrmm assigns it a shelf location from the specified pool. The most important parameters are identified in the following list.

▶ **AUTOSCRATCH**

Use this operand to override the scratch processing release action for volumes in this pool. By default, AUTOSCRATCH(YES) return to scratch is performed automatically when expiration processing is running on a system that has access to the catalogs, TCDB, and library for the volume. If you need to take any special actions not performed by DFSMSrmm, perhaps on another system, specify AUTOSCRATCH(NO).

When you specify AUTOSCRATCH(YES) and do not manually confirm the scratch release action, DFSMSrmm performs return to scratch processing, including:

- UNCATALOG parmlib option.
- TPRACF parmlib option.

– SMSTAPE(UPDATE(SCRATCH)).

When you specify AUTOSCRATCH(NO) or you manually confirm the scratch release action for any volume, DFSMSrmm does not perform this part of the return to scratch processing. DFSMSrmm assumes that you have manually performed the cleanup already.

When you specify AUTOSCRATCH(NO), DFSMSrmm does not return the volume to scratch status until you have manually confirmed the volume release action and run expiration processing.

DFSMSrmm checks the AUTOSCRATCH setting and the scratch release action for the volume when the volume is returning to scratch. If the scratch release action is set and the volume is in a pool with AUTOSCRATCH(NO), DFSMSrmm leaves the volume in pending release status. You must perform whatever actions or cleanup activity you require and confirm that the scratch release action has been performed using the subcommand: RMM CV volser CRLSE(SCRATCH). Run expiration processing on any DFSMSrmm system to return the volume to scratch.

Default: AUTOSCRATCH(YES)

► **MASTEROVERWRITE**

Specify the VLPOOL MASTEROVERWRITE operand to control how DFSMSrmm allows the overwriting of a volume. You can specify if you want to allow data to be appended to the end of a volume or overwritten, or to allow new files to be added to a volume. The MASTEROVERWRITE value applies to all volumes that reside in a pool. If you do not specify a MASTEROVERWRITE value, DFSMSrmm uses the MASTEROVERWRITE value that you set using the EDGRMMxx parmlib member OPTION MASTEROVERWRITE operand, as described in “OPTION command” on page 79.

► **TYPE**

In DFSMSrmm, you can specify one of two categories of pools:

Where:

- | | |
|----------|--|
| R | A rack pool is shelf space assigned to hold volumes that are generally read-only, and that enter and leave your installation on an ad-hoc basis. These volumes are typically software product volumes and customer volumes and do not adhere to your installation's naming conventions. |
| S | A scratch pool is assigned to shelf space to hold all other volumes in the removable media library. The volumes assigned to this shelf space can be used to satisfy scratch requests as long as the volumes are in scratch status. Once the volume has been written to, it becomes a master volume until the data is no longer required by the installation. The volume remains in the same DFSMSrmm scratch pool in that it occupies the same shelf space regardless of status. |

We recommend that you define all the pools used in your tape management system when running in parallel.

► **RACF(N)**

Specifies the type of RACF tape support that DFSMSrmm should provide for volumes in the pool you are defining.

Where:

- | | |
|----------|---|
| N | Specify if you do not want DFSMSrmm to create RACF tape profiles for the volumes in the pool. |
|----------|---|

When you are defining RACF tape support for volumes in a pool, you must look at the RACF tape support you have defined for your installation with the OPTION TPRACF command.

Example 3-5 shows a sample EDGRMMxx PARMLIB member that you can use as a model when preparing for DFSMSrmm recording mode.

Example 3-5 Sample EDGRMMxx PARMLIB member

OPTION	OPMODE(R)	/* Record-Only Mode	*/-
	ACCOUNTING(J)	/* Accounting from JOB	*/-
	BACKUPPROC(EDGBKUP)	/* Name of BACKUP-proc	*/-
	BLP(RMM)	/* DFSMSrmm controls BLP	*/-
	CATRETPD(0012)	/* catalog retention	*/-
	CATSYSID(*)	/* all catalogs shared	*/-
	CDSID(PROD)	/* control data set id	*/-
	COMMANDAUTH(OWNER)	/* type of authorization	*/-
	DATEFORM(J)	/* Date format	*/-
	DISPDDNAME(LOANDD)	/* DISP ctrl DD card	*/-
	DISPMSGID(EDG4054I)	/* DISP message number	*/-
	DSNAME(RMM.PROD.CDS)	/* CDS data set name	*/-
	EXPDTDROP(PERCENT(10,WARN))	/* Write warning msg only	*/-
	GDG(CYCLEBY(GENERATION) DUPLICATE(BUMP))		-
	IPLDATE(N)	/* IPL date checking	*/-
	JRNNAME(RMM.PROD.JRNL)	/* JRNL data set name	*/-
	JRNLTRAN(NO)	/* Normal journaling	*/-
	JOURNALFULL(75)	/* Percentage JRNL full	*/-
	LINECOUNT(054)	/* Lines per page	*/-
	MASTEROVERWRITE(LAST)	/* Overwriting of a vol	*/-
	MAXHOLD(100)	/* Number of I/O oper.	*/-
	MAXRETPD(NOLIMIT)	/* Maximum retention	*/-
	MCATTR(VRSELXDI)	/* SMS MC usage	*/-
	MEDIANAME(3490)	/* Default medianame	*/-
	MOVEBY(VOLUME)	/* spec. how to move vols	*/-
	MSG(M)	/* case for message txt	*/-
	NOTIFY(Y)	/* Notify volume owners	*/-
	PDA(OFF)	/* PDA is disabled	*/-
	PDABLKCT(255)	/* number of blocks	*/-
	PDABLKSZ(31)	/* blocksize in K	*/-
	PDALOG(OFF)	/* PDA output disabled	*/-
	PREACS(NO)	/* Disable EDGUX100 ACS pr.	*/-
	RETAINBY(VOLUME)	/* spec. how to retain vols	*/-
	RETENTIONMETHOD(VRSEL)	/* for new tape data sets	*/-
	RETPD(0005)	/* Default retention	*/-
	REUSEBIN(STARTMOVE)	/* reuse BIN as soon as pos.	*/-
	SCRATCHPROC(EDGXPROC)	/* ATL/MTL procedure	*/-
	SMFAUD(YES)	/* SMF type 42-S22 audit rec	*/-
	SMFSEC(YES)	/* SMF type 42-S23 sec. rec	*/-
	SMSACS(YES)	/* enable MV ACS processing	*/-
/*	SMSTAPE(UPDATE(EXITS,SCRATCH,COMMAND),PURGE(YES))	ATL	*/-
	SYSID(EGZB)	/* Name of the system	*/-
	TPRACF(N)	/* RACF tape support	*/-
	TVEXTPURGE(EXPIRE(2))	/* set an expiration date	*/-
	UNCATALOG(N)	/* Catalog support	*/-
	VRSCCHANGE(INFO)	/* No additional action	*/-
	VRSDROP(PERCENT(10,WARN))	/* Write warning msg only	*/-

```

VRSEL(NEW) /* New VRS processing */-
VRSJOBNAME(2) /* DATASETNAME/JOBNAME */-
VRSMIN(0000000100,WARN) /* Warn if < 100 VRSS */-
VRSRETAIN(PERCENT(80,WARN)) /* Write warning msg only */

/* ***** */
MNTMSG MSGID(IEF233A) ID(001) VOLUME(016) RACK(016)
MNTMSG MSGID(IEF233D) ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID('IEF234E K') ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID('IEF234E R') ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID('IEF234E D') ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID(IEF455D) ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID(IEC501A) ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID('IEC502E K') ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID('IEC502E D') ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID('IEC502E R') ID(001) VOLUME(016) RACK(999)
MNTMSG MSGID('IEC502E RD') ID(001) VOLUME(017) RACK(999)
MNTMSG MSGID('IEC502E RK') ID(001) VOLUME(017) RACK(999)
MNTMSG MSGID(IAT5110) ID(001) VOLUME(044) RACK(999)
MNTMSG MSGID(IAT5210) ID(001) VOLUME(050) RACK(999)
MNTMSG MSGID(IAT5410) ID(001) VOLUME(020) RACK(999)

/* ***** */
/* MEDINF command added at 2008/03/21 Norbert */-
/* ***** */
MEDINF NAME(VTFM) /* VTFM global replacement */-
REPLACE(PERM(0) WMC(0) TEMP(0) AGE(0))
MEDINF NAME(VTFM) /* define the VTFM volumes */-
MEDIATYPE(5,ETC) /* standard definition */-
CAPACITY(1000)
MEDINF NAME(VTFM) /* define the VTFM volumes */-
MEDIATYPE(5,ETC) -
RECORDINGFORMAT(5,EFMT1) -
CAPACITY(1000)

/* ***** */
/* REJECT command removed at 2008/03/21 Norbert */-
/* ***** */
/* REJECT ANYUSE(*) Replaced by the following OPENRULE and */-
/* PRTITION commands */-
/* ***** */

/* ***** */
/* PRTITION command added at 2008/03/21 Norbert */-
/* ***** */
PRTITION VOLUME(*) /* added 2008/03/21 NS */-
TYPE(NORMM) /* volumes not defined in CDS */-
SMT(IGNORE) /* system-managed */-
NOSMT(IGNORE) /* non-system-managed */-
/* ***** */
/* OPENRULE commands are added at 2008/03/21 Norbert */-
/* ***** */
OPENRULE VOLUME(*) /* added 2008/03/21 NS */-
TYPE(NORMM) /* volumes not defined in CDS */-
ANYUSE(IGNORE BY(ANY)) /* bypass RMM processing */-

```

```

/* ignore all types of request */
OPENRULE VOLUME(*) /* added 2008/03/21 NS */-
          TYPE(RMM) /* all volumes defined in CDS */-
          ANYUSE(ACCEPT) /* allow OPEN processing */

/* ***** */
VLPPOOL PREFIX(NS*) /* volume prefix */ -
        AUTOSCRATCH(NO) /* do not return to scratch */ -
        TYPE(R) /* do not satisfy scratch req */ -
        RACF(N) /* SAF tape security is inuse */ -
        MASTEROVERWRITE(USER) /* allow to overwrite DSNs */ -
        MEDIANAME(3590) /* media name used */ -
        EXPDTCHECK(Y) /* check VOL1 expdt */ -
        DESCRIPTION('Norberts scratch pool')
VLPPOOL PREFIX(THS*) /* volume prefix */ -
        AUTOSCRATCH(NO) /* do not return to scratch */ -
        TYPE(S) /* satisfy scratch requests */ -
        RACF(N) /* SAF tape security is inuse */ -
        MEDIANAME(3592) /* media name used */ -
        EXPDTCHECK(N) /* do not check VOL1 expdt */ -
        DESCRIPTION('3592 POOL')
VLPPOOL PREFIX(VT*) /* volume prefix */ -
        AUTOSCRATCH(YES) /* return to scratch auto */ -
        DESCRIPTION('VTFM 3592 POOL')
        EXPDTCHECK(N) /* do not check VOL1 expdt */ -
        MASTEROVERWRITE(ADD) /* ctrl DSN overwriting */ -
        MEDIANAME(3592) /* media name used */ -
        NAME($$DFLT) /* pool name */ -
        RACF(N) /* Type of RACF tape support */ -
        TYPE(S) /* satisfy scratch requests */

/* ***** */
LOCDEF LOCATION(SANJOSE) -
        TYPE(STORAGE) -
        PRIORITY(2000) -
        MEDIANAME(*) -
        MANAGEMENTTYPE(BINS)
LOCDEF LOCATION(IBMATL1) -
        TYPE(LIBRARY) -
        PRIORITY(4800)
LOCDEF LOCATION(VTFM001) -
        AUTOMOVE(NO) -
        MANAGEMENTTYPE(NO BINS) -
        MEDIANAME(*) -
        TYPE(STORAGE, HOME)

/* ***** */
SECCLS NUMBER(010) -
        NAME(UCL) -
        SMF(N) -
        MESSAGE(N) -
        ERASE(N) -
        DESCRIPTION('UNCLASSIFIED') -
        MASK('***')
SECCLS NUMBER(050) -

```

	NAME(IUO)	-
	SMF(Y)	-
	MESSAGE(N)	-
	ERASE(N)	-
	DESCRIPTION('INTERNAL USE ONLY')	-
	MASK('**.IUO.**',	-
	'**.INTERNAL.**',	-
	'**.INT.**')	-
SECCLS	NUMBER(100)	-
	NAME(IC)	-
	SMF(Y)	-
	MESSAGE(Y)	-
	ERASE(Y)	-
	DESCRIPTION('EXTERNAL USE ONLY')	-
	MASK(**.IC.**)	-

COMMNDxx optional): Commands automatically issued at initialization

Update COMMNDxx to automatic commands the system internally issues as part of system initialization. COMMNDxx is useful for automatic entry of commands that are frequently issued at system initialization.

To start the DFRMM started task at system initialization add the highlighted statement in Figure 3-18.

```

...
COM='S VTAM,,,(LIST=&NODE;)'
COM='S NETVIEW,SUB=MSTR,CAT=&CATALOG;,SYS=&SYSNAME;'
COM='S APPC,SUB=MSTR,APPC=02'
COM='S ICR&SYSNAME;.SYSLOG'
COM='SET DAE=01'
COM='SET DAE=&DAE;'
COM='SETXCF START,POLICY,TYPE=CFRM,POLNAME=&POLNAME;'
COM='S DFRMM,M=70'
...

```

Figure 3-18 COMMNDxx to start DFRMM started task at initialization

SMFPRMxx (optional): SMF parameters

If you want security or audit records, update SMFPRMxx to define the SMF record numbers that DFSMSrmm generates. In DFSMSrmm, you define the SMF record numbers in the EDGRMMxx PARMLIB member using two operands: OPTION SMFAUD for auditing records and OPTION SMFSEC for security records. For more information about these commands, see the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

Select two SMF record numbers in the 128 to 255 range that your installation is not using. Add your SMFAUD and SMFSEC record numbers to SMFPRMxx as specified in *z/OS MVS System Management Facilities (SMF)*, SA22-7630. By default, there is no SMF recording.

You can use the MVS command DISPLAY SMF as shown in Figure 3-19 to display SMF data:

```
DISPLAY SMF,0
```

Figure 3-19 Display SMF information

Note: The 0 directs the system to display the current SMF options.

Example 3-6 shows the output of the display SMF command.

Example 3-6 *Output of a DISPLAY SMF command*

```
IEE967I 18.39.10 SMF PARAMETERS 982
      MEMBER = SMFPRM00
      SMFDLEXIT(USER3(IRRADU86)) -- DEFAULT
      SMFDLEXIT(USER2(IRRADU00)) -- DEFAULT
      SMFDPEXIT(USER3(IRRADU86)) -- DEFAULT
      SMFDPEXIT(USER2(IRRADU00)) -- DEFAULT
      EMPTYEXCPSEC(NOSUPPRESS) -- DEFAULT
      MULCFUNC -- DEFAULT
      BUFUSEWARN(25) -- DEFAULT
      BUFSIZMAX(0128M) -- DEFAULT
      MAXEVENTINTRECS(00) -- DEFAULT
      SYNCVAL(00) -- DEFAULT
      DUMPABND(RETRY) -- DEFAULT
      SUBSYS(STC,NOINTERVAL) -- SYS
      SUBSYS(STC,NODETAIL) -- SYS
      SUBSYS(STC,EXITS(IEFUS0)) -- PARMLIB
      SUBSYS(STC,EXITS(IEFUJP)) -- PARMLIB
      SUBSYS(STC,EXITS(IEFUJI)) -- PARMLIB
      SUBSYS(STC,EXITS(IEFACTRT)) -- PARMLIB
      SUBSYS(STC,EXITS(IEFU85)) -- PARMLIB
      SUBSYS(STC,EXITS(IEFU84)) -- PARMLIB
      SUBSYS(STC,EXITS(IEFU83)) -- PARMLIB
      SUBSYS(STC,EXITS(IEFU29)) -- PARMLIB
      SUBSYS(STC,TYPE(0:18,20:98,100:255)) -- PARMLIB
      SYS(NODETAIL) -- PARMLIB
      SYS(NOINTERVAL) -- PARMLIB
      SYS(EXITS(IEFU29)) -- PARMLIB
      SYS(EXITS(IEFUTL)) -- PARMLIB
      SYS(EXITS(IEFUJI)) -- PARMLIB
      SYS(EXITS(IEFUS0)) -- PARMLIB
      SYS(EXITS(IEFUJP)) -- PARMLIB
      SYS(EXITS(IEFUS1)) -- PARMLIB
      SYS(EXITS(IEFUJV)) -- PARMLIB
      SYS(EXITS(IEFACTRT)) -- PARMLIB
      SYS(EXITS(IEFU85)) -- PARMLIB
      SYS(EXITS(IEFU84)) -- PARMLIB
      SYS(EXITS(IEFU83)) -- PARMLIB
      SYS(TYPE(0:18,20:98,100:255)) -- PARMLIB
      NOBUFFS(MSG) -- PARMLIB
      LASTDS(MSG) -- PARMLIB
      LISTDSN -- PARMLIB
      SID(SC70) -- PARMLIB
      DDCONS(NO) -- PARMLIB
      JWT(2400) -- PARMLIB
      MEMLIMIT(NOLIMIT) -- PARMLIB
      STATUS(010000) -- PARMLIB
      MAXDORM(3000) -- PARMLIB
      INTVAL(10) -- PARMLIB
      REC(PERM) -- PARMLIB
      NOPROMPT -- PARMLIB
      DSNAME(SYS1.SC70.MAN3) -- PARMLIB
      DSNAME(SYS1.SC70.MAN2) -- PARMLIB
      DSNAME(SYS1.SC70.MAN1) -- PARMLIB
```

Figure 3-20 Enabling DFSMSrmm license into IFAPRDxx

You can use the MVS SET SMF operator command (see Figure 3-21) to modify the SMF recording options dynamically by specifying which SMFPRMxx parmlib member is to be used.

Using the SET command, the installation can replace all the existing SMF options. For example, an installation can activate SMF recording after an IPL in which NOACTIVE is specified by using the SET command and choosing the parmlib member that contains the ACTIVE option. The SET command, however, cannot change the SID parameter.

```
SET SMF=xx
```

Figure 3-21 SET SMF command

The two alphanumeric characters indicating the SMFPRMxx member of the logical parmlib containing the parameters the system is to use when restarting SMF.

GRSRNLxx (optional): GRS resourcename list

If you are not using global resource serialization (GRS) to control enqueues among sharing systems, you do not have to update GRSRNLxx. If you have multiple systems connected in a GRSplex, decide how you want GRS to handle the reserve on the CDS:

Restriction: The length of RNAME must be exactly 23 characters. If the length of your CDSID is less than 8 characters, we recommend you use a generic entry as described below. Otherwise, you must specify RNAME as a hexadecimal constant:

RNAME(X'D4C1E2E3C5D94BD9C5E2C5D9E5C54Bxx..xx4040')

where:

X'D4C1E2E3C5D94BD9C5E2C5D9E5C54B' represents MASTER.RESERVE, and X'xx..xx4040', your CDSID appended with blanks to 8 characters in length.

- If the volume where the CDS resides does not contain other critical data, and real reserves will not cause any shared disk contention problems, add the reserve name to the GRS exclusion list. This leaves the real hardware reserve in effect and cause the associated SYSTEMS enqueue to be converted to a local SYSTEM enqueue. The delay associated with sending the additional unnecessary enqueue around the GRSplex is then avoided.

The required statements in GRSRNLxx to convert the SYSTEMS enqueue to a local SYSTEM enqueue are shown in Figure 3-22.

```
RNLDEF RNL(EXCL) TYPE(SPECIFIC)
        QNAME(SYSZRMM)
        RNAME(MASTER.RESERVE)
RNLDEF RNL(EXCL) TYPE(SPECIFIC)
        QNAME(SYSZRMM)
        RNAME(X'D4C1E2E3C5D94BD9C5E2C5D9E5C54BD7D9D6C440404040')
```

Figure 3-22 GRS EXCL definition TYPE(SPECIFIC) with a hexadecimal constant

Important: You must define both resources if you are using DFSMSrmm release z/OS V1.9 or higher and you have specified TYPE(SPECIFIC), depend the first time DFSMSrmm starts it needs to reserve on MASTER.RESERVE and initialize RMM CDSID support.

- If the volume where the CDS resides contains other critical data, and real reserves can impact other systems or cause disk lockouts, add the reserve name to the GRS reserve conversion list. This leaves the global SYSTEMS ENQUEUE in effect and remove the real hardware reserve.

Figure 3-23 shows the required statements in GRSRNLxx to convert the RESERVE to a SYSTEMS enqueue.

```
RNLDEF RNL(CON) TYPE(GENERIC)
      QNAME (SYSZRMM)
      RNAME (MASTER.RESERVE)
```

Figure 3-23 GRS CON definition with TYPE(GENERIC)

Note: If you are not using GRS, and use an equivalent product instead, use the ENQ resource name information provided in the DFSMSrmm publications to define the resource correctly to the other product. Do not use generic or global options for SYSZRMM. You must ensure that each resource with the SYSZRMM major name is processed correctly.

Figure 3-24 shows the difference between a system enqueue and a hardware reserve.

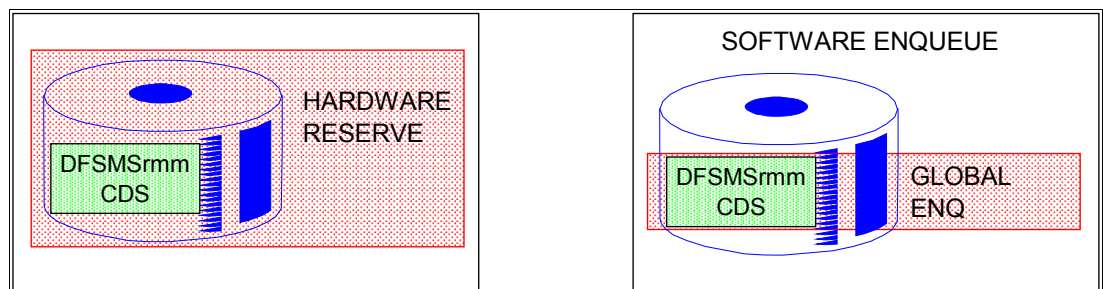


Figure 3-24 Use of GRS enqueue and hardware reserve

You can use the MVS SET GRSRNL command shown in Figure 3-25 to change the current GRSRNL definitions:

```
SET GRSRNL=(xx[,xx]...)
```

Figure 3-25 MVS SET GRSRNL command

Attention: Use care when you enter the SET GRSRNL command to change heavily used or highly critical resources. Work that requires resources for a critical application, or resources used by the operating system may become suspended or delayed, which can impair the performance of that critical application or the operating system itself.

The two alphanumeric characters indicating the GRSRNLxx member(s) of the logical PARMLIB that contains the specified GRS resource name lists. GRSRNL allows you to change the current RNLs specified in one or more GRSRNLxx PARMLIB member. Do not use parentheses when only one PARMLIB member is specified.

You cannot specify the GRSRNL= parameter if GRSRNL has already been set to EXCLUDE in the logical PARMLIB member IEASYSxx.

The GRSRNL= parameter has a sysplex scope only when all systems share the same PARMLIB data set.

Figure 3-26 shows you how you can use the MVS DISPLAY GRS command to display your current GRS information.

```
DISPLAY GRS,ALL
```

Figure 3-26 MVS DISPLAY GRS command

Example 3-6 contains the output of the DISPLAY GRS command.

Example 3-7 DISPLAY GRS messages

```
RESPONSE=SC70
ISG343I 18.58.51 GRS STATUS 997
LIST TYPE QNAME RNAME
INCL GEN SPFEDIT
INCL GEN SYSDSN
INCL GEN SYSIKJBC
INCL GEN SYSIKJUA SYS1
INCL SPEC SYSIKJUA VAIN
INCL GEN SYSZVOLS
EXCL SPEC SPFEDIT PASSWORD
EXCL SPEC SPFEDIT SYS1.DCMLIB
EXCL SPEC SPFEDIT SYS1.NUCLEUS
EXCL SPEC SPFEDIT SYS1.SVCLIB
EXCL GEN SYSCTLG
EXCL SPEC SYSDSN PASSWORD
EXCL SPEC SYSDSN SYS1.DCMLIB
EXCL GEN SYSDSN SYS1.JES3
EXCL SPEC SYSDSN SYS1.NUCLEUS
EXCL SPEC SYSDSN SYS1.SVCLIB
EXCL SPEC SYSIGGV2 CATALOG.SHRICF1.VIODFBK
EXCL SPEC SYSIGGV2 CATALOG.SHRICF1.VIODFPK
EXCL SPEC SYSIGGV2 CATALOG.TOTICF1.V033SMP
EXCL SPEC SYSIGGV2 CATALOG.TOTICF1.V033SM1
EXCL SPEC SYSIGGV2 UCAT.VBOOK01
EXCL SPEC SYSIGGV2 UCAT.VOS3R2B
EXCL GEN SYSZJES2
CON GEN ARCGPA
CON GEN DSPURIO1
CON SPEC HWRESERV BOOK01
CON SPEC HWRESERV BOOK02
CON SPEC HWRESERV BOOK03
CON SPEC HWRESERV BOOK04
CON SPEC HWRESERV IODFBK
CON SPEC HWRESERV IODFPK
CON SPEC HWRESERV OS3R2B
CON SPEC HWRESERV O33SMP
CON SPEC HWRESERV O33SM1
```

```

CON  SPEC HWRESERV 033SM2
CON  SPEC HWRESERV 033SM3
CON  SPEC HWRESERV 033SM4
CON  GEN  IGDSDXS
CON  GEN  SPFEDIT
CON  GEN  SPZAPLIB
CON  GEN  SYSIEWLP
CON  GEN  SYSIGGV2
CON  GEN  SYSVTOC
CON  GEN  SYSZRACF
CON  SPEC SYSZRMM  MASTER.RESERVE
CON  GEN  SYSZVDS
NO ENQ RESOURCE CONTENTION EXISTS
NO REQUESTS PENDING FOR ISGLOCK STRUCTURE
NO LATCH CONTENTION EXISTS
SYSTEM      STATE                SYSTEM      STATE
SC63        CONNECTED            SC65        CONNECTED
SC70        CONNECTED            SC64        CONNECTED
THERE IS NO RNL CHANGE IN PROGRESS.
GRS STAR MODE INFORMATION
LOCK STRUCTURE (ISGLOCK) CONTAINS 1048576 LOCKS.
THE CONTENTION NOTIFYING SYSTEM IS SC63
SYNCHRES:    YES
ENQMAXU:     16384
ENQMAXA:     250000
GRSQ:        CONTENTION

```

Table 3-4 shows you the DFSMSrmm resource symbolic names. Do not use GRS to change or alter any of these ENQs. The resource symbolic names are provided for information only because DFSMSrmm processing manages serialization for you.

Table 3-4 DFSMSrmm resource symbolic names

Major Name	Minor Name	Resource	Scope
SYSZRMM	HSKP.dsn.volser	Inventory management data set serialization	SYSTEMS
SYSZRMM	MASTER.RESERVE	DFSMSrmm control data set serialization at startup and when the CDSID is not yet known	SYSTEMS
SYSZRMM	MASTER.RESERVE.cdsid	DFSMSrmm control data set serialization	SYSTEMS
SYSZRMM	MHKP.ACTIVE	Serialize inventory management functions on the same DFSMSrmm subsystem	SYSTEMS
SYSZRMM	MHKP.dsn.volser	Inventory management data set serialization	SYSTEMS
SYSZRMM	RMM.ACTIVE	Ensure only one system run per z/OS image	SYSTEMS
SYSZRMM	BUFFER.CONTROL	Buffer management	STEP
SYSZRMM	EDGINERS.volser	Serialize volume labeling	SYSTEMS
SYSZRMM	SHUTDOWN	Serialize DFSMSrmm shutdown and refresh processing	SYSTEMS
SYSZRMM	INACTIVE	Serialize DFSMSrmm activation enabling only a single WTOR to be issued to the operator	SYSTEMS
SYSZRMM	EXIT_IS_ACTIVE	Exit recovery serialization	SYSTEMS

Major Name	Minor Name	Resource	Scope
SYSZRMM	WTOR_ENQ	Exit recovery serialization	SYSTEMS
SYSZRMM	WTORIPC	TCP/IP error recovery serialization on CLIENT systems	SYSTEMS
SYSZRMM	EXIT_id_UNAVAIL	Exit recovery serialization where id can be 100, 200, or 300 representing the last three characters of the DFSMSrmm installation exits EDG_EXIT100, EDG_EXIT200, or EDG_EXIT300.	SYSTEMS

Setting up MIM multiimage integrity for DFSMSrmm

If you installed MIM as opposed to GRS, you must update your MIM definitions. DFSMSrmm uses the RESERVE macro and ENQ macro to serialize use of its own resources. The SCOPE specified by DFSMSrmm ensures that serialization is across systems, within a system, or within a task, as required. All serialized resources use the SYSZRMM QNAME, and the RNAME is determined based on the resource required. If you do nothing to define the SYSZRMM QNAME, the serialization required by DFSMSrmm should be provided.

For a complete list of the resources serialized for DFSMSrmm, and the RNAMEs, and SCOPE of each request refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

To serialize the DFSMSrmm CDS, the RESERVE macro with a RNAME of MASTER.RESERVE is used, which results in:

- A hardware reserve
- A SYSTEMS enqueue

In a multisystem environment, the SYSTEMS enqueue is not required nor is a hardware reserve. There may also be times when a hardware reserve is not required because of other data on the same volume. You can optionally use MIMQNAME member entries to optimize the DFSMSrmm serialization for your environment.

If you want to use hardware reserves, we recommend the MIMQNAME member of the CNTL data set shown in Figure 3-27 on page 106.

SYSZRMM	GDIF=YES,	/* GDIF SHOULD PROCESS THIS QNAME
	SCOPE=SYSTEMS,	/* Identify which ENQUEUES to manage
	RESERVES=KEEP,	/* Use Hardware reserves if requested
	EXEMPT=YES,	/* EXEMPT LIST is required for SCOPE=SYSTEMS
	ECMF=NO,	/* ECMF not required
	TRACE=NONE	/* Do not trace requests

Figure 3-27 MIMQNAME member using hardware reserves

The SCOPE=SYSTEMS ENQs issued by DFSMSrmm should be propagated to ensure correct serialization, except for the ENQ related to the hardware reserve. The EXEMPT=YES identifies that one ENQ should not be propagated. You specify these in the GDIEXMPT member (see Figure 3-28).

```

* * * * *
* DFSMSrmm LOCAL EXCLUSION STATEMENTS. DO NOT PROPAGATE ENQS *
* FOR CDS SERIALIZATION. *
* * * * *
LOCAL QNAME=SYSZRMM,RNAME=MASTER.RESERVE
LOCAL QNAME=SYSZRMM,RNAME=MASTER.RESERVE.PROD

```

Figure 3-28 GDIEXMPT member

Important: You must define both resources if you are using DFSMSrmm release z/OS V1.9 or higher, depend the first time DFSMSrmm starts it needs to reserve on MASTER.RESERVE and initialize RMM CDSID support.

Figure 3-29

If you do not want to use hardware reserves, and rely on propagation of the ENQs that serialize the DFSMSrmm CDS, we recommend the MIMQNAME member of the CNTL data set shown in Figure 3-30

```

SYSZRMM GDIF=YES, /* GDIF SHOULD PROCESS THIS QNAME
          SCOPE=SYSTEMS, /* Identify which ENQUEUES to manage
          RESERVES=CONVERT, /* Use Hardware reserves if requested
          EXEMPT=YES, /* EXEMPT LIST is required for SCOPE=SYSTEMS
          ECMF=NO, /* ECMF not required
          TRACE=NONE /* Do not trace requests

```

Figure 3-30 MIMQNAME member using propagation of ENQs

You can optionally specify other keyword values, such as ECMF=YES or TRACE=CONFLICT or TRACE=ALL depending on your local requirements and whether you are testing with DFSMSrmm or running in parallel.

DEVSUPxx (optional): Device Support Options

DFSMS V1.8 and higher provides new options for securing tape data sets using System Authorization Facility (SAF). These are designed to allow you to define profiles to protect data sets on tape using the DATASET class without the need to activate the TAPEDSN option or the TAPEVOL class. DFSMS also provides options you can use to specify that all data sets on a tape volume should have common authorization and that users are authorized to overwrite existing files on a tape volume.

For optimum tape security use the combined capabilities of DFSMSrmm, DFSMSdfp, and RACF. It is recommended you specify the following parameters in your DEVSUPxx parmlib member shown in Figure 3-31

```

TAPEAUTHDSN=YES
TAPEAUTHF1=YES
TAPEAUTHRC4=FAIL
TAPEAUTHRC8=FAIL

```

Figure 3-31 DEVSUPnn sample

Where:

TAPEAUTHDSN

Enables tape authorization checks in the DATASET class but without DSTYPE=T.

DSTYPE=T indicates to RACF that the check is for data set on a tape volume and that special RACF tape data set and a tape volume processing is to be performed. Without DSTYPE=T RACF authorization checking considers only profiles in the DATASET class.

The system uses the data set name specified in the allocation or JCL to check your authorization to read or write the specified file. In addition, the system determines the RACF erase-on-scratch setting from the RACF profile and passes it to your tape management system.

Use this option only when you have a tape management system, such as DFSMSrmm, installed and actively checking that the 44 character data set name specified by the user matches the data set name on tape. Without a tape management system, tape data set open processing can only validate the last 17 characters of the data set name against the tape volume labels.

When you request bypass label processing (BLP) and the mounted volume uses standard labels, OPEN issues the authorization check that the user is authorized to use BLP. This processing uses the existing ICHBLP resource in the RACF FACILITY class. When you specify TAPEAUTHDSN=YES only, it replaces the check that RACF makes as part of tape volume authorization checking.

TAPEAUTHF1

Enables additional tape authorization checks in the DATASET class for existing files on the same tape volume when any other file on the tape volume is opened.

This function depends on the tape management system returning the 44 character data set name and data set sequence number to OPEN/EOV through the IFGTEP during the Volume Mount exit Volume Security function; if no data set name is returned by the tape management system, processing is as if this keyword had not been specified.

Although intended to enable an additional authorization check for the first data set when any other data set on the tape volume is opened, the implementation allows your tape management system to request one or more additional authorization checks when any data set on a tape volume is opened. Each additional data set name and data set sequence number returned results in an additional RACROUTE. Do not use this function unless you have a tape management system and it can return a data set name and data set sequence number. A data set sequence number is the label number normally specified in the JCL LABEL keyword and stored in the catalog.

When TAPEAUTHDSN=YES is in use, any additional RACROUTE matches that issued for TAPEAUTHDSN except for the data set name and data set sequence number. Otherwise TAPEAUTHF1 uses a RACROUTE that matches that used for SETROPTS TAPEDSN. When neither TAPEAUTHDSN nor SETROPTS TAPEDSN is in use, TAPEAUTHF1 support is not provided.

TAPEAUTHRC4	Denies accessing of data sets that are not protected by a security profile.
TAPEAUTHRC8	Denies accessing of data sets that typically cannot be accessed.

The combination of DFSMSrmm, DFSMSdfp, and RACF ensures:

- ▶ Full 44 character data set name validation.
- ▶ Validation that the correct volume is mounted.
- ▶ Control the overwriting of existing tape data sets.
- ▶ Management of tape data set retention.
- ▶ Control over the creation and destruction of tape volume labels.
- ▶ No limitations caused by RACF TAPEVOL profile sizes and TVTOC limitations.
- ▶ All tape data sets on a volume have a common authorization.
- ▶ Use of generic DATASET profiles, enabling common authorization with DASD data sets.
- ▶ Authorization for all tape data sets regardless of the tape label type.
- ▶ Authorization for the use of bypass label processing (BLP).
- ▶ Exploitation of RACF 'erase on scratch' support.
- ▶ Use of DFSMSrmm FACILITY class profiles for data sets unprotected by RACF.
- ▶ Your authorization to use a volume outside of DFSMSrmm control through 'ignore' processing also enables authorization to the data sets on that volume.

Use the MVS SET DEVSUP command shown in Figure 3-32 to implementing the new tape data set security settings.

```
/SET DEVSUB=70
or
/T DEVSUB=70
```

Figure 3-32 Update the tape security settings

Example 3-8 shows the successful result of the command.

Example 3-8 SET DEVSUB messages

```
T DEVSUP=00
IEE252I MEMBER DEVSUP00 FOUND IN SYS1.PARMLIB
IEE536I DEVSUP  VALUE 00 NOW IN EFFECT
IEA253I DEVSUP  3480X RECORDING MODE DEFAULT IS COMPACTION.
IEA253I DEVSUP  ISO/ANSI TAPE LABEL VERSION DEFAULT IS V3
IEA253I DEVSUP  TAPE OUTPUT DEFAULT BLOCK SIZE LIMIT IS 32760
IEA253I DEVSUP  COPYSDB DEFAULT IS INPUT
IEA253I DEVSUP  STORAGE LIMIT FOR TAPE DDR SWAP DEFAULTED TO 1000M
IEA253I DEVSUP  TAPEAUTHDSN: YES
IEA253I DEVSUP  TAPEAUTHF1: YES
IEA253I DEVSUP  TAPEAUTHRC4: ALLOW
IEA253I DEVSUP  TAPEAUTHRC8: WARN
IEA253I DEVSUP  PERFORM NORMAL EXPIRATION DATE PROCESSING
```

Note: Only the tape security setting is updated all other values like the media categories are not updated.

For more information, refer to chapter *Chapter 13, "Tape security with z/OS V1.8"* on page 485.

IGDSMSxx (optional): Storage Management Subsystem definition

Update the storage management subsystem definition member to set the CA_RECLAIM DATACLAS value. This parameter specifies whether or not to use CA reclaim for KSDSs according to the CA Reclaim attribute in their data classes.

DATACLAS indicates that the SMS-managed KSDSs and non-SMS-managed KSDSs will go by the data class specification of CA Reclaim=YIN at define time or subsequent ALTER setting. Only the resultant catalog setting of CA Reclaim=Y will do CA reclaim; CA Reclaim=N will not do CA reclaim. It is the way to specify some KSDSs not to use CA reclaim.

Note: If CA_RECLAIM is in effect there is no longer a need to re-organize your VSAM KSDS DFSMSrmm control data set.

```
CA_RECLAIM(DATACLAS)
```

Figure 3-33 CA_RECLAIM command syntax

Use the MVS DISPLAY SMS command to check the current SMS settings as shown in Figure 3-34.

```
DISPLAY SMS,OPTIONS
```

Figure 3-34 MVS DISPLAY SMS command

Example 3-9 contains the output of the DISPLAY SMS command.

Example 3-9 DISPLAY SMS messages

```
RESPONSE=SC70
IGD002I 17:55:33 DISPLAY SMS 201
ACDS      = SYS1.SMS.ACDS.NEW
COMMDS    = SYS1.SMS.COMMDS
ACDS LEVEL = z/OS V1.13
INTERVAL = 15          DINTERVAL = 150
SMF_TIME = YES         CACHETIME = 3600
CF_TIME = 1800         PDSE_RESTARTABLE_AS = YES
PDSE_BMFTIME = 3600    PDSE1_BMFTIME = 3600
PDSE_LRUTIME = 60      PDSE1_LRUTIME = 50
PDSE_LRUCYCLES = 15    PDSE1_LRUCYCLES = 200
LOCAL_DEADLOCK = 15    GLOBAL_DEADLOCK = 4
REVERIFY = NO          DSNTYPE = PDS
ACSDFAULTS = NO        PDSESHARING = EXTENDED
OVRD_EXPDT = NO        SYSTEMS = 8
PDSE_HSP_SIZE = OMB     PDSE1_HSP_SIZE = 256MB
USE_RESOWNER = YES      RLS_MAX_POOL_SIZE = 100MB
RLSINIT = YES           RLSTMOUT = 0
COMPRESS = GENERIC      LOG_OF_LOGS = IGWTVS.LOG.OF.LOGS
QTIMEOUT = 300          TVSNAME = 070
AKP = 1000              TV_START_TYPE = WARM
MAXLOCKS = (0,0)
CICSVR_INIT = NO        CICSVR_DSNAME_PREFIX = DWWUSER.V3R1M0
CICSVR_RCDS_PREFIX = DWW
CICSVR_GRPNAME_SUFFIX = PROD
CICSVR_ZZVALUE_PARM =
```



```

CICSVR_UNDOLOG_CONTROL =
CICSVR_UNDOLOG_PREFIX = DWW
CICSVR_RCDS_PREFIX = DWW
CICSVR_GRPNAME_SUFFIX = PROD
CICSVR_ZZVALUE_PARM =
CICSVR_UNDOLOG_CONTROL =
CICSVR_UNDOLOG_PREFIX = DWW
CICSVR_BACKOUT_CONTROL =
CICSVR_GENERAL_CONTROL =
Rls_MaxCfFeatureLevel = Z
RlsAboveThebarMaxPoolSize = 0 RlsFixedPoolSize = 0
PDSE_MONITOR = (YES,0,0) PDSE1_MONITOR = (YES,0,0)
PDSE_DIRECTORY_STORAGE = 2000M PDSE1_DIRECTORY_STORAGE = 2000M
PDSE_BUFFER_BEYOND_CLOSE = NO PDSE1_BUFFER_BEYOND_CLOSE = NO
GDS_RECLAIM = YES DSSTIMEOUT = 0
BLOCKTOKENSIZE = NOREQUIRE FAST_VOLSEL = ON
USEEAV = YES BREAKPOINTVALUE = 21
OAMPROC = SUPPRESS_DRMSGS = NO
OAMTASK = PDSE_SYSEVENT_DONTSWAP = NO
DB2SSID = D9DG SAM_USE_HPF = YES
CA_RECLAIM = DATACLAS
IGD002I 17:55:33 DISPLAY SMS
TRACE = ON SIZE = 128K TYPE = ALL
JOBNAME = * ASID = *
TRACING EVENTS:
MODULE = ON SMSSJF = ON SMSSSI = ON ACSINT = ON
OPCMD = ON CONF C = ON CDSC = ON CONF S = ON
MSG = ON ERR = ON CONFR = ON CONFA = ON
ACSPRO = ON IDAX = ON DISP = ON CATG = ON
VOLREF = ON SCHEDP = ON SCHEDS = ON VTOCL = ON
VTOCD = ON VTOCR = ON VTOCC = ON VTOCA = ON
RCD = ON DCF = ON DPN = ON TVR = ON
DSTACK = ON UAFF = ON DEBUG = ON
VOLSELMSG = (OFF,0) TYPE = ALL JOBNAME = *
ASID = * STEPNAME = *
DSNAME = *

```

When CA_RECLAIM is not in use you can use the MVS SETSMS command to change the setting as shown in Figure 3-35.

```
SETSMS CA_RECLAIM(DATACLAS)
```

Figure 3-35 MVS SETMVS command syntax

If the CA_RECLAIM value is changed you should get an additional message as shown in Figure 3-36 on page 111.

```

IGW467I DFSMS CA_RECLAIM PARMLIB VALUE CHANGED ON SYSTEM: SC64 854
OLD VALUE: NONE
NEW VALUE: DATACLAS
IEE712I SETSMS PROCESSING COMPLETE

```

Figure 3-36 SETSMS messages

If you have specified the CA-RECLAIM in the PARMLIB and activated it as described above you should reorganize your DFSMSrmm control data set final time. To check you DFSMSrmm control data set run an IDCAMS EXAMINE as shown in Figure 3-37 to see if there is a need.

```
//MHLRES7R JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM S=SC64
//STEP1 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
    EXAMINE -
        NAME(RMM.CONTROL.DSET) -
        INDEXTEST -
        DATATEST
/*
```

Figure 3-37 Sample JCL to run IDCAMS EXAMINE

Check message IDC01728I as you can see in Figure 3-38 and when there are empty control areas left you should reorganize your DFSMSrmm control data set.

```
    EXAMINE -
        NAME(RMM.CONTROL.DSET) -
        INDEXTEST -
        DATATEST
IDC01700I INDEXTEST BEGINS
IDC11773I          133 KEYS PROCESSED ON INDEX LEVEL  1, AVERAGE KEY LENGTH:   42.4
IDC11773I          117 KEYS PROCESSED ON INDEX LEVEL  2, AVERAGE KEY LENGTH:   22.7
IDC11773I           2 KEYS PROCESSED ON INDEX LEVEL  3, AVERAGE KEY LENGTH:   25.5
IDC11774I CURRENT INDEX CFSIZE IS 2048, RECOMMENDED MINIMUM INDEX CFSIZE IS 1536
IDC01724I INDEXTEST COMPLETE - NO ERRORS DETECTED
IDC01701I DATATEST BEGINS
IDC01728I FOUND 113 EMPTY CONTROL AREAS THAT HAVE NOT BEEN RECLAIMED
IDC01709I DATATEST COMPLETE - NO ERRORS DETECTED
IDC01708I 133 CONTROL INTERVALS ENCOUNTERED
IDC01710I DATA COMPONENT CONTAINS 524 RECORDS
IDC01711I DATA COMPONENT CONTAINS 113 DELETED CONTROL INTERVALS
IDC01712I MAXIMUM LENGTH DATA RECORD CONTAINS 1036 BYTES
IDC01722I 99 PERCENT FREE SPACE
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

Figure 3-38 Sample EXAMINE output

ALLOCxx (optional): Allocation System default

Update the storage management subsystem definition member to set specifies the policy on whether to recall a migrated data set when you use an IEFBR14 JCL program with DD DISP=(x,DELETE) to delete the data set. Figure 3-39 shows you the correct use of the IFBR14_DELMIGDS command.

Specifies the policy on whether to recall a migrated data set when you use an IEFBR14 JCL program with DD DISP=(x,DELETE) to delete the data set. The recall is, in most cases, unnecessary, as the data set is being deleted anyway.

```
SYSTEM IEFBR14_DELMIGDS(NORECALL)
```

Figure 3-39 Sample IFBR14_DELMIGDS command

Where:

LEGACY	Indicates that the system is to recall HSM-migrated data sets before deletion.
NORECALL	Indicates that the system can delete (through HSM HDELETE processing) the data set without first recalling the data set to the primary storage.

Default: LEGACY

Note: This new function is available in z/OS V1.12 or higher.

AUTORxx (optional): Auto-reply policy specifications

Use the parmlib member AUTORxx to activate auto-reply processing on a system. The member AUTOR00 contains the auto-reply policy suggested by IBM. You can modify the member (which is not recommended), or define another AUTORxx member to customize the auto-reply policy. The member AUTOR00 also contains comments that provide the message text for each WTOR and the rule used to select the WTOR.

Syntax rules for AUTORxx

The following syntax rules apply to the AUTORxx parmlib member:

- ▶ Data is specified in columns 1-71.
- ▶ Comments can start in any column and can span lines, and must start with /* and end with */. Comments may appear in any place where a blank is accepted, except within quoted strings.
- ▶ Generally, syntax errors cause the auto-reply changes to be rejected, while in the following cases, syntax errors are ignored and allow the changes to become active:
 - The maximum number of message IDs is reached.
 - Duplicate message IDs are specified.
 - NOTIFYMSGs is specified in different members.

Syntax format of AUTORxx

Use the syntax as shown in Figure 3-40 to add your DFSMSrmm related messages.

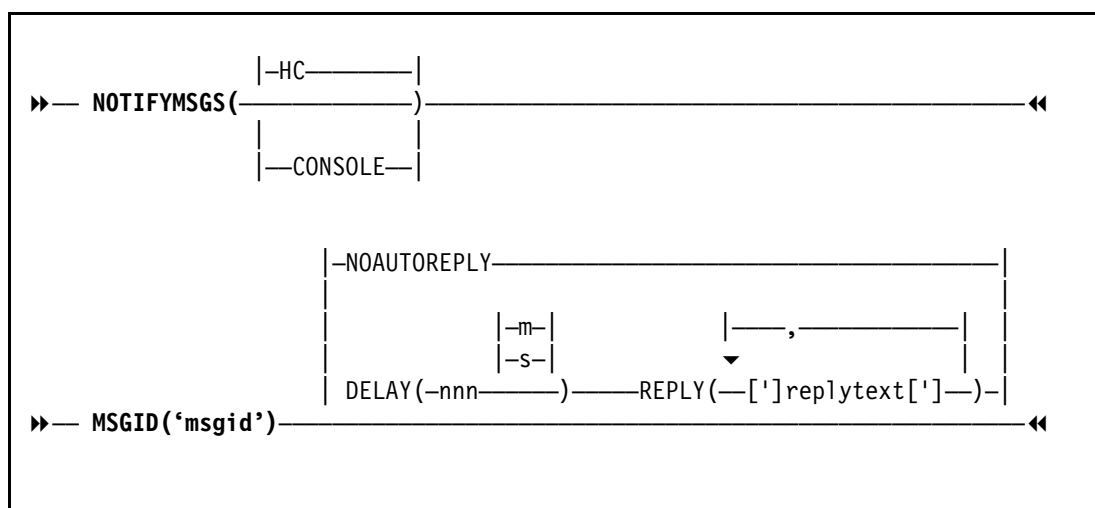


Figure 3-40 AUTORxx command syntax

Note: The maximum number of message IDs that do not contain wildcards is limited to 10,413. The maximum number of message IDs that contain wildcards is limited to 1,500

Updating Auto-reply Policy and WTORs Information

You can use the MVS SET AUTOR command shown in Figure 3-41 to change the current Auto-reply Policy information.

```
SET AUTOR=(RM,00)
or
SET AUTOR=00
```

Figure 3-41 Update AUTO-replay and WTORs information

The two alphanumeric characters indicating the AUTORxx member(s) of the logical PARMLIB that contain the specified Auto-reply Policy information. AUTOR allows you to change the current Auto-reply Policy specified in one or more AUTORxx PARMLIB member. Do not use parentheses when only one PARMLIB member is specified.

After the new setting is in effect you should get the information shown in Figure 3-42.

```
SET AUTOR=(RM,00)
IEE252I MEMBER  AUTORM FOUND IN SYS1.IBM.PARMLIB
IEE252I MEMBER  AUTR00 FOUND IN SYS1.PARMLIB
CNZ2600I AUTO-REPLY POLICY MODIFIED.
```

Figure 3-42 Display AUTOR command output

Note: If you specify only one parmlib member with APPC=, ASCH=, AUTOR=, CEE=, CNGRP=, GRSRNL=, MPF=, OMVS=, PROG=, or SCH=, you do not need to enter the parentheses.

Displaying Auto-reply Policy and WTORs Information

You can use the DISPLAY AUTOR command shown in Figure 3-43 to display the current auto-reply policy.

```
DISPLAY AUTOR,POLICY
or
DISPLAY AUTOR,P
```

Figure 3-43 Display your current AUTOR settings

Example 3-10 contains the result of a DISPLAY AUTOR,POLICY command.

Example 3-10 DISPLAY AUTOR,POLICY command output

```
RESPONSE=SC70
CNZ2603I 16.41.49 AUTOR POLICY 217
POLICY ACTIVATED AT 16.28.35 ON 10/01/2010 NOTIFYMSG(S(HC)
FROM PARMLIB MEMBERS RM,00
--MSG ID-- DELAY MEM ----REPLY TEXT----
...
```

EDG0103D	1M	RM RETRY
EDG0107A	1M	RM 00
EDG0115D	1M	RM CANCEL
EDG0117D	1M	RM CANCEL
EDG0123D	5M	RM N
EDG0127D	5M	RM CANCEL
EDG0215D	2M	RM N
EDG0228D	2M	RM CANCEL
EDG0358D	10M	RM CANCEL
EDG0361D	10M	RM RETRY
EDG1107D	2M	RM RESTART
EDG1200D	30S	RM CANCEL
EDG1203D	30S	RM CANCEL
EDG2103D	30S	RM L
EDG2106D	30S	RM L
EDG3213D	30S	RM RETRY
EDG4000D	2M	RM RETRY
EDG4001D	2M	RM RETRY
EDG4010D	1M	RM RETRY
EDG4012D	1M	RM RETRY
EDG6605D	10M	RM RETRY
EDG6607D	10M	RM RETRY
EDG6609D	10M	RM RETRY
EDG6626A	10M	RM END
EDG6627A	10M	RM S
EDG6629D	1M	RM CANCEL
EDG6663D	10M	RM F
EDG6671D	10M	RM F
EDG8008D	1M	RM RETRY
EDG8010D	1M	RM RETRY
EDG8011D	1M	RM RETRY
EDG8013D	1M	RM RETRY
EDG8102D	1M	RM RETRY
EDG8108D	1M	RM RETRY
EDG8110D	1M	RM RETRY
EDG8113D	1M	RM RETRY
EDG8121D	1M	RM RETRY
EDG8122D	1M	RM RETRY
EDG8123D	1M	RM RETRY

....

The messages shown in the example above are the default defined in the AUTORRM parmlib member and can be used if DFSMSrmm is running in protect mode.

Where:

column 1	MessageID in the range of 1 to 10 characters
column 2	The amount of time, in minutes (M) or seconds (S), in the range between 0 to 999, to wait, after the WTOR is issued and before auto-reply processing replies to the WTOR.
column 3	The AUTORxx parmlib member suffixes that specified the policy
column 4	The reply that is used if auto-reply processing replies to the WTOR.

You can use the DISPLAY AUTOR command shown in Figure 3-44 to display the current WTORS being monitored by auto-reply processing.

```
DISPLAY AUTOR,WTOR
or
DISPLAY AUTOR,W
```

Figure 3-44 Display your current AUTOR processing

Example 3-11 contains the result of a DISPLAY AUTOR,POLICY command.

Example 3-11 *DISPLAY AUTOR,WTOR command output with pending actions*

```
D AUTOR,W
CNZ2604I 06.47.02 AUTOR WTORS
0009 STATUS=06.47.58 SYS=SY1
MSG=IEE800D CONFIRM VARY FORCE FOR 3D0 - REPLY NO OR YES
REPLY=NO
0008 STATUS=06.47.46 SYS=SY1
MSG=IXC371D CONFIRM REQUEST TO VARY SYSTEM SY1 OFFLINE. REPLY
SYSNAME=SY1 TO REMOVE SY1 OR C TO CANCEL.
REPLY=C
```

Example 3-12 contains the result of a DISPLAY AUTOR,WTOR command if there is no action pending.

Example 3-12 *DISPLAY AUTOR,WTOR command output without pending actions*

```
RESPONSE=SC70
CNZ2604I 16.50.20 AUTOR WTORS 220
NO PENDING AUTO-REPLIES TO WTORS
```

Note: This new function is available in z/OS V1.12 or higher.

3.1.4 Enabling ISPF Data Set List (DSLISL) Support

To enable direct entry into the DFSMSrmm ISPF dialog from the ISPF Data Set List Utility, use the ISPF Configuration Utility to update the ISPF Configuration Table. To enable this function, select the Enable RM/Tape Commands option. For details of how to use the ISPF Configuration Utility, refer to *z/OS ISPF Planning and Customizing*. Figure 16 on page 50 shows that the data set list support is enabled, as well as showing the default values for the RM/Tape Command EDGRPD34 and Command APPLID EDG. You do not need to change these values.

Implementation steps

The ISPF Data Set List support can be implemented for a single user or it can be a installation-wide setting. Following the steps below to implementing ISPF Data Set List support for a single user:

1. Create a new PDS with VB 255 to store the ISPF default settings
2. Under ISPF, call 'TSO ISPCCONF' in any command line
3. Select option 1 "Create/Modify Settings and Regenerate Keyword File" in the *ISPF Configuration Utility* primary panel, and added the name to the keyword file dsname field

you have allocated before, and added a member name.
You should get message *Keyword file loaded*

4. Select option 3 "*PDF Exits and Other PDF Settings*" in the *ISPF Configuration Utility* primary panel
5. Scroll to the correct place and add '/' to *Enable RM/Tape Commands*
6. Exit the dialog using two times the PF3 key
you should get message *Keyword file saved*
7. Select option 4 "*Build Configuration Table Load Module*"
8. Specify any library in your ISPLLIB concatenation as the "*Output Configuration Table Load Module Data Set*"
9. ENTER to load module built
10. Exit ISPF, ensure the specified load library is ISPLLIB concatenated.
11. Start ISPF and it works. (that is just for a single user, not for all users)

To implementing the ISPF Data Set List support for all user you can following the steps as described before, but the configuration table load module should be stored in the SISPLPA library. To load the module to the LPA you must IPL your system. For detailed information defer to "Move the ISPCFIGU module to the SISPLPA library" on page 374.

For more information, refer to chapter *Chapter 10, "Enabling ISPF Data Set List (DSLIST) Support"* on page 363

3.1.5 Rexx Variable Constraint Relief

A new operand, VARSTORAGE, was added to the PROFILE command to specify whether variables in the CLIST or authorized REXX variable pools can use storage above the 16MB line.

If you have a large environment and you have a need to list a big number of volumes normally you always get a return code 4 reason code 10 *Insufficient storage for search processing. More records might exist* is avoided.

If your TSO LOGON session parameter size is 30000 than approximate 2400 volumes retrieved with setting VARSTORAGE(LOW). If you change your settings to VARSTORAGE(HIGH) you can retrieve approximate 20.000 volumes.

VARSTORAGE specifies the storage location to be used for CLIST variables or REXX OUTTRAP variables containing output from authorized commands. A CLIST or REXX exec uses the VARSTORAGE setting of the PROFILE command when the it starts. This setting then remains unchanged for the life of the CLIST or REXX exec, even if the CLIST or REXX exec issues a new PROFILE command with a different VARSTORAGE setting. The new setting will only apply when a new CLIST or REXX exec begins.

VARSTORAGE (HIGH) indicates that CLIST variables and REXX OUTTRAP variables containing output from authorized commands invoked by REXX can be kept in storage above the 16M line.

VARSTORAGE (LOW) indicates that CLIST variables and REXX OUTTRAP variables containing output from authorized commands invoked by REXX can only be kept in storage below the 16M line. If you specify VARSTORAGE with no operands, VARSTORAGE(LOW) is the default.

This is the default value when your user profile is created.

To show your current setting use the TSP PROFILE command as shown in Figure 3-45 without any additional operands.

```
TSO PROF
```

Figure 3-45 Display your current profile settings

Figure 3-46 shows you your current profile settings include the new VARSTORAGE information. If you have not set the VARSTORAGE you can see that the default setting LOW is set.

```
CHAR(0) LINE(0) PROMPT INTERCOM NOPAUSE NOMSGID MODE WTPMSG
NORECOER PREFIX(MHLRES5) PLANGUAGE(ENU) SLANGUAGE(ENU) VARSTORAGE(LOW)
DEFAULT LINE/CHARACTER DELETE CHARACTERS IN EFFECT FOR THIS TERMINAL
```

Figure 3-46 Your current profile settings

If you have requested to search for all volumes in your DFSMSrmm data base normally you the message "more volumes exists" on the screen as shown in Figure 3-47 and the search ended before all volumes are listed.

Panel Help Scroll

DFSMSrmm Volumes (Page 1 of 2)More volumes may exist

Command ==>Scroll ==> CSR

There is not enough storage available to list all the volumes

Enter HELP or PF1 for the list of available line commands

Use the RIGHT command to view other data columns

Volume	Assigned	Expiration		Dest-	Tra-	Data
S serial Owner	date	date	Status	Location ination	nsit	sets
AFS201 VGRMLIB	2004/303	1999/365	USER	SHELF	N	0
AFS202 VGRMLIB	2004/303	1999/365	USER	SHELF	N	0
BTS001 VGRMLIB	2005/005	1999/365	USER	SHELF	N	0
BTS005 VGRMLIB	2005/005	1999/365	USER	SHELF	N	0
BTS006 VGRMLIB	2005/005	1999/365	USER	SHELF	N	0
BTS007 VGRMLIB	2005/005	1999/365	USER	SHELF	N	0
BTS008 VGRMLIB	2005/005	1999/365	USER	SHELF	N	0
BTS010 VGRMLIB	2005/005	1999/365	USER	SHELF	N	0
BTS011 VGRMLIB	2005/005	1999/365	USER	SHELF	N	0
BTS012 VGRMLIB	2005/005	1999/365	USER	SHELF	N	0

Figure 3-47 Display your incomplete search result

To update VARSTORAGE value use the TSO PROFILE command as shown in Figure 3-48

```
TSO PROFILE VARSTORAGE(HIGH)
```

Figure 3-48 Updapte your TSO profile settings

Figure 3-49 shows you your new profile settings include the new VARSTORAGE information.


```
CHAR(0) LINE(0) PROMPT INTERCOM NOPAUSE NOMSGID MODE WTPMSG
NORECOER PREFIX(MHLRES5) PLANGUAGE(ENU) SLANGUAGE(ENU) VARSTORAGE(HIGH)
DEFAULT LINE/CHARACTER DELETE CHARACTERS IN EFFECT FOR THIS TERMINAL
```

Figure 3-49 TSO profile settings

3.1.6 Creating a starting procedure

At the begin of this section you get a description about your system symbols and the DFSMSrmm problem determination aid.

Display your system symbols

System symbols are elements that allow systems to share PARMLIB definitions while retaining unique values in those definitions. You can code the statements in the IEASYMxx member of PARMLIB, so only one IEASYMxx member is needed to define static system symbols and system parameters for all systems in a multisystem environment. If you wish, you can code additional IEASYMxx members to organize the definitions more clearly.

If you have to define DFSMSrmm on multiple systems, you can check the symbols defined in your system before you create the procedure. To check all the defined symbols in an IEASYMxx member, you can use the following MVS DISPLAY SYMBOLS command shown in Figure 3-50 to get a list of all symbols and the values of it.

```
DISPLAY SYMBOLS
```

Figure 3-50 MVS DISPLAY SYMBOLS command syntax

Example 3-7 shows all the available system-wide symbols and the values on your system.

Example 3-13 Display SYMBOLS output

```
DISPLAY SYMBOLS
IEA007I STATIC SYSTEM SYMBOL VALUES 715
      &SYSALVL. = "2"
      &SYSCclone. = "70"
      &SYSNAME. = "SC70"
      &SYSPLEX. = "SANDBOX"
      &SYSR1. = "Z1DRB1"
      &ALLCLST1. = "CANCEL"
      &CMDLIST1. = "70,00"
      &COMMDSN1. = "COMMON"
      &CPACMLQ1. = ".ZOSR1D."
      &CPENABLE. = "(10,30)"
      &DB2. = "V8"
      &DB2LEVEL. = "DB2V8"
      &DB2VER. = "DB2V8"
      &DCEPROC1. = "."
      &DFHSMCMD. = "00"
      &DFHSMHST. = "6"
      &DFHSMPRI. = "NO"
      &DFSPROC1. = "."
      &DIAGXX. = "03"
      &DLIB1. = "Z1DDL1"
```

```

&DLIB2.    = "Z1DDL2"
&DLIB3.    = "Z1DDL3"
&DLIB4.    = "Z1DDL4"
&IEFSSNXX. = "R7"
&IFAPRDXX. = "01"
&IGDSMSXX. = "05"
&J2PRM.    = "G"
&LPALIST1. = "3A"
&MLPALST1. = "0E"
&MQ600SET. = "V080703"
&MQ600VOL. = "SB0X83"
&MQ701SET. = "V101020"
&MQ701VOL. = "SB0X83"
&MSTJCLXX. = "3A"
&OMVSPARM. = "3A"
&PLPADSN1. = "PLPA"
&PRISUBSY. = "JES2"
&PROCLIB1. = "TEST1"
&PROGXXA1. = "A0"
&PROGXXE1. = "E5"
&PROGXXL1. = "D0"
&PROGXXL2. = "3A"
&PROGXXL3. = "L0"
&PROGXXP1. = "P0"
&PROGXXS1. = "S0"
&RMFMON01. = "XA"
&RSUVALUE. = "0"
&SDSFARM.  = "2A"
&SECSUBSY. = "JES3"
&SMFPARMS. = "00"
&SRES2.    = "Z1DRB2"
&SYSLEVEL. = "Z0SR1D"
&SYSR2.    = "Z1DRB2"
&SYSR3.    = "Z1DRB3"
&VOL01.    = "MLDC65"

```

Using the Problem Determination Aid facility

To analyze DFSMSrmm running problems you can use the Problem Determination Aid (PDA) facility to gather related information. The PDA facility is required for IBM service, because it traces module and resource flow. The PDA facility consists of in-storage trace, optional DASD log data sets, EDGRMMxx PARMLIB member options, and operator commands to control tracing.

If you need an external DASD record of trace data, perform this step once for each MVS image.

Two separate log data sets are used by the PDA facility. DFSMSrmm recognizes these log data sets by using EDGPDOX and EDGPDOY DD statements. The EDGPDOX and EDGPDOY data sets must be pre-allocated and cataloged to be used by DFSMSrmm. Both data sets must be allocated on the same volume. The DFSMSrmm RACF user ID requires ALTER access to the EDGPDOX and EDGPDOY data sets.

Before you can use the PDA facility, you need to:

- Determine how long you want to keep trace information
- Optionally allocate storage on DASD for the PDA log data sets, EDGPDOX and EDGPDOY
- Implement the PDA facility based on how long you want to keep trace data

Refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for detailed information about the PDA facility.

Updating the procedure library

Create a procedure in SYS1.PROCLIB to start the DFSMSrmm subsystem address space. Example 3-8 shows the sample JCL.

```
//DFRMM PROC M=00,OPT=MAIN
//IEFPROC EXEC PGM=EDG&OPT.,PARM='&M',TIME=1440,REGION=60M
//PARMLIB DD DDNAME=IEFRDER
//IEFRDER DD DISP=SHR,DSN=SYS1.PARMLIB
//EDGPDOX DD DISP=SHR,DSN=RMM.&SYSNAME..RMMPDOX
//EDGPDOY DD DISP=SHR,DSN=RMM.&SYSNAME..RMMPDOY
```

Figure 3-51 Sample DFSMSrmm starting procedure

In this example, we use REGION=0M to get the maximum region size. If your default region size is small, we recommend that you specify REGION=40M.

Where:

DFRMM	Specifies the procedure name. You can use any procedure name from 1 to 8 characters long.
M	Allows you to specify a PARMLIB member suffix. M=00 tells DFSMSrmm to use EDGRMM00.
OPT	Allows you to specify: – OPT=RESET to disable the subsystem interface – OPT=MAIN to execute the DFSMSrmm subsystem.

Restriction: Before DFSMSrmm disables the subsystem interface, it ensures that the user who made the request is authorized. In this case, the RACF user of the STC needs ALTER access the STGADMIN.EDG.RESET.SSI resource defined in RACF FACILITY class.

EDGPDOX	An optional statement required only for an external trace output recording. If not specified, no logging of the PDA trace output is performed. The data set specified must be pre-allocated and cataloged. DFSMSrmm RACF user ID requires ALTER authority to this data set. This must be allocated on the same volume as EDGPDOY.
EDGPDOY	An optional statement required only for external trace output recording. There are the same considerations as for EDGPDOX.

Important: You must define a pair of trace output data sets for each MVS image. Do not share the trace data sets with multiple DFSMSrmm systems or with other system components.

&SYSNAME	This specifies a system symbol to use the same procedure on different systems.
REGION	As you determine the REGION size for the DFSMSrmm started procedure, the amount of virtual storage that DFSMSrmm uses depends on the resources you have defined. DFSMSrmm virtual storage usage can be affected by any REGION size controls or restrictions that your systems might have in place such as in IEFUSI. The sample DFRMM procedure specifies REGION=60M, which normally provides all the private region below 16MB and 60MB above 16MB. To enable DFSMSrmm to use all available virtual storage, specify REGION=0M. If you want to set a specific region size, consider these tips along with the current region size of your DFRMM started procedure, to determine if you need to make any changes to the REGION size.

For special considerations or restrictions when coding the starting procedure, refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

3.1.7 Defining an alias for high-level qualifier RMM

We recommend that you create an alias in the ICF user catalog for RMM as shown in Figure 3-52.

```
//DEFALIAS JOB ...
//STEP1 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=A
//SYSIN DD *
      DEFINE ALIAS -
              (NAME(RMM) -
              RELATE('SYSCAT1.SYST01'))
```

Figure 3-52 Defining an alias in an ICF-user catalog

The DEFINE ALIAS command defines an alias, RMM, for the user catalog SYSCAT1.SYST01. All catalog entries for data sets defiled with a high-level qualifier of RMM are defined in user catalog SYSCAT1.SYST01.

3.1.8 Protecting DFSMSrmm resources

To protect DFSMSrmm functions, you need to use an external security product, such as RACF. Invocation to this product should be through the security authorization facility (SAF). If RACF is not installed, you must provide equivalent function through the SAF interface.

The security implementation tasks are:

1. Assign a RACF user ID to DFSMSrmm.
2. Identify DFSMSrmm to RACF.
3. Define RACF groups for DFSMSrmm.
4. Define DFSMSrmm resources to RACF.
5. Check the STGADMIN.ADR.DUMP.CNCURRNT resource.

Before you start DFSMSrmm, you need to create a RACF user ID for DFSMSrmm and define the DFSMSrmm started task to RACF. You can find more detailed information about how to implement RACF security for DFSMSrmm, and sample definitions, in Appendix A, “Security topics” on page 455.

Assigning DFSMSrmm a RACF user ID

You can assign a RACF user ID that matches the name of the DFSMSrmm procedure you created in 3.1.4, “Creating a starting procedure” on page 81, but any installation-selected RACF user ID is acceptable. As data sets are created for use by the DFSMSrmm procedure, add the DFSMSrmm RACF user ID to the access list for the data sets.

We used the RACF command shown in Figure 3-53 to define the DFSMSrmm user ID.

```
ADDUSER DFRMM DFLTGRP(SYS1) NAME('DFSMSrmm Userid')
```

Figure 3-53 Define a new RACF user

Note: If you do not specify the DFLTGRP parameter on the ADDUSER command, the current connect group of the user issuing the ADDUSER command is used as the default.

Identifying DFSMSrmm procedures to RACF

To assign RACF identities to a started procedures, you can use the RACF STARTED class to add or modify security definitions for new and existing started procedures.

Enter the RACF commands described in Figure 3-54 to assign RACF identities to the DFSMSrmm started procedure.

```
RDEFINE STARTED (DFRMM.*) UACC(NONE) STDATA(USER(DFRMM) GROUP(SYS1))  
SETROPTS RACLIST(STARTED) REFRESH  
SETROPTS GENERIC(STARTED) REFRESH
```

Figure 3-54 RACF commands to identifying DFSMSrmm procedures

Note: Remember to refresh the in-storage profiles, using the SETROPTS REFRESH command, after you add profiles to the STARTED class.

If you plan to use the EDGLABEL, EDGXPROC, or the EDGBKUP procedures, you must define the procedures in the RACF STARTED class.

Before RACF 2.1 no STARTED class was available for use, the only way to associate a started procedure with a RACF user ID was by coding the RACF started procedures table, ICHRIN03.

3.1.9 Defining RACF resources and groups for DFSMSrmm

To implement RACF security for DFSMSrmm resources, you should define a set of user groups to handle different kinds of resources at different access levels. For more information, refer to Appendix A, “Security topics” on page 455.

STGADMIN.EDG.RESET.SSI FACILITY class profile

At this stage there is one security profile in particular that you should be aware of and create. When you first install and start to use DFSMSrmm, it is likely that you need to stop or even remove DFSMSrmm from your system. Although you can easily stop DFSMSrmm, if you have EDGSSSI in the subsystem name table DFRM entry, or have been using a running mode that

supports tape recording, DFSMSrmm prevents all tape usage until it is restarted. To allow tapes to be used, you must restart DFSMSrmm, or remove it from the system.

The RACF FACILITY class profile STGADMIN.EDG.RESET.SSI controls the use of the RESET facility for removing DFSMSrmm from the system. Be sure to define this security profile and authorize the DFSMSrmm started task to reset the subsystem interface. Grant the DFSMSrmm started task procedure RACF user ID ALTER access authority to STGADMIN.EDG.RESET.SSI. Refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for specific details about removing DFSMSrmm from the system.

Note: You can use the RESET facility without defining this profile when you do not have a security product installed.

3.1.10 Creating the DFSMSrmm CDS

The DFSMSrmm CDS is a VSAM KSDS that contains the complete inventory of the removable media library. DFSMSrmm records all changes made to the inventory, such as adding or deleting volumes in the CDS.

You can define the DFSMSrmm control data set as either an extended format (EF) or a non-extended format VSAM data set. Using a non-extended format data set for the DFSMSrmm control data set, limits the control data set size to a maximum of 4 GB. Using an EF data set enables you to use VSAM functions such as multivolume allocation, compression, or striping. EF also enables you to define a control data set that uses VSAM extended addressability (EA) to enable the control data set to grow above 4 GB.

To define an EF control data set, you must include the DATACLASS keyword on the AMS DEFINE command and reference the correct data class. Refer to *z/OS DFSMS: Using Data Sets*, SC26-7410, for more information about EF data sets. Also refer to *DFSMSdfp Storage Administration Reference*, SC26-7402, for information about defining data classes with DSNTYPE=EXT and EXTENDED ADDRESSABILITY=Y.

DFSMSrmm requires CONTROL access to the control data set. The control data set cannot be a SYS1.xx data set if the control data set is to be shared. Additionally, batch LSR cannot be used with the DFSMSrmm control data set. In our example, we use the non-extended format.

Note: All data sets used by DFSMSrmm can be eligible for allocation in the extended addressing space of an EAV. This includes the DFSMSrmm journal and any dynamically allocated temporary files.

To create the CDS on the system, you use the following steps:

1. Calculate the space for the DFSMSrmm CDS.
2. Protect the CDS.
3. Allocate the DFSMSrmm CDS.
4. Initialize the CDS using the EDGJUTIL utility.

Calculating the space for the DFSMSrmm CDS

There is sample JCL in SAMPLIB member EDGJMFAL to allocate the CDS. You can modify it to meet your installation standards.

The control data set is one of the most important components of DFSMSrmm. Therefore, to avoid space problems while running DFSMSrmm, you must calculate the space required before its allocation.

Table 3-5 shows the space requirement for each kind of record in the CDS.

Table 3-5 *DFSMSrmm CDS space requirement*

Control data set content	DASD space
Control record	1 MB (MB equals approximately 1 000 000 bytes)
Data sets	584 KB for every 1000 data sets
Shelf locations in the library that do not contain volumes	164 KB for every 1000 shelf locations
Shelf locations in storage locations	140 KB for every 1000 shelf locations
Owners	38 KB per 1000 volumes
Software products, average five volumes per product	420 KB for every 1000 software products
Volumes	1 MB for every 1000 volumes
Vital record specifications	212 KB for every 1000 vital record specifications

Protecting the CDS

To protect the DFSMSrmm CDS, define a discrete profile, and then permit the appropriate RACF groups to the RACF data set profile. Figure 3-55 contains sample JCL. For detailed information about implementing RACF security for DFSMSrmm, refer to Appendix A, “Security topics” on page 455.

```

/* ***** *
/*   To execute this job you need the RACF SPECIAL attribute
/* ***** *
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/*   DEFINE CONTROL DATA SET DATA SET PROFILE */
/* ***** */
  ADDSD 'RMM.PROD.CDS' UACC(NONE) GENERIC
    PERMIT 'RMM.PROD.CDS' GENERIC-
      ID(EDGADMIN) ACC(READ)
    PERMIT 'RMM.PROD.CDS' GENERIC-
      ID(EDGLIB) ACC(READ)
    PERMIT 'RMM.PROD.CDS' GENERIC-
      ID(EDGSPG) ACC(ALTER)

```

Figure 3-55 Sample JCL to protect the DFSMSrmm CDS

Allocating the DFSMSrmm CDS

The JCL to create the DFSMSrmm CDS in Figure 3-56 has been modified to use special attributes.

```

//STEP01 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
DELETE RMM.PROD.CDS CL
SET MAXCC = 0
/*
//STEP02 EXEC PGM=IDCAMS ALLOCATION MASTER FILE
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,UNIT=3390,VOL=SER=DFRMM4
//SYSIN DD *
DEFINE CLUSTER(NAME(RMM.PROD.CDS) -
FILE(MASTER) -
FREESPACE(15 0) -
KEYS(56 0) -
REUSE -
RECSZ(512 9216) -
SHR(3 3) -
KILOBYTES(4500 1500) -
STORAGECLASS(gspace) -
VOLUMES(DFRMM4)) -
DATA(NAME(RMM.PROD.CDS.DATA) -
BUFFERSPACE(829440) -
CISZ(26624) -
INDEX(NAME(RMM.PROD.CDS.INDEX) -
CISZ(2048)
/*

```

Figure 3-56 Sample JCL to allocate the DFSMSrmm CDS

Note: For initial testing with DFSMSrmm, a small CDS is sufficient. When you convert your existing data to DFSMSrmm, you need a CDS sized to accommodate your existing and planned requirements.

Initializing the CDS using the EDGJUTIL utility

Before using the DFSMSrmm CDS, a control record must be written in the CDS using the EDGUTIL utility. The control record contains information about the number of shelf locations in the library and storage locations.

Sample JCL is provided in SYS1.SAMPLIB member EDGJUTIL, which you can modify to meet your installation standards. Figure 3-57 shows a sample JCL that can be used to initialize the CDS.

```

//EDGUTIL EXEC PGM=EDGUTIL,PARM='CREATE'
//SYSPRINT DD SYSOUT=*
//MESSAGE DD SYSOUT=*
//MASTER DD DISP=OLD,DSN=RMM.PROD.CDS
//SYSIN DD *
CONTROL CDSID(PROD) EXTENDEDBIN(YES) CATSYNCH(NO)
/*

```

Figure 3-57 Sample JCL to initialize the DFSMSrmm CDS

Where:

CDSID(PROD)	Specifies one-to-eight characters that identify the control data set by name.
CATSYNCH(NO)	Specify CATSYNCH(NO) to force synchronization of the DFSMSrmm control data set and user catalogs the next time inventory management is run. See 1.9, “Catalog synchronization” on page 42, for more information.
EXTENDED BIN(YES)	Enables DFSMSrmm extended bin support, which allows the reuse of bins at the start of a move. Extended bin support must be enabled if you want to use the DFSMSrmm PARMLIB OPTION command REUSEBIN(STARTMOVE) operand as shown in Figure 3-4 on page 72 to reuse bins when a volume moves from a bin.

3.1.11 Creating the DFSMSrmm CDS as extended format

You can define the DFSMSrmm CDS as an extended format (EF) VSAM data set. This function is enabled with APAR OW47639. The benefits available for extended format data sets include:

- ▶ Data striping
- ▶ Data compression
- ▶ VSAM extended addressability
- ▶ Partial space release
- ▶ System-managed buffering
- ▶ CDS above 4 GB

Extended format enables you to define a CDS that uses VSAM Extended Addressability (EA) to enable the CDS to grow above 4 GB.

Extended format data sets must be system-managed. The mechanism to request extended format is through the SMS data class attribute **Data Set Name Type**, specifying a value of **EXT**. For an extended format data set to be allocated using extended addressability, the attribute “**Extended Addressability ... Y**” (see Figure 3-58 on page 128) must be set.

	DATA CLASS DEFINE	Page 2 of 4
Command ==>		
SCDS Name . . . : SYS1.SMS.SCDS		
Data Class Name : EFKSDS		
To DEFINE Data Class, Specify:		
Data Set Name Type	EXT	(EXT, HFS, LIB, PDS or blank)
If Ext	R	(P=Preferred, R=Required or blank)
Extended Addressability . . .	Y	(Y or N)
Record Access Bias		(S=System, U=User or blank)
Space Constraint Relief	N	(Y or N)
Reduce Space Up To (%) . . .		(0 to 99 or blank)
Dynamic Volume Count		(1 to 59 or blank)
Compaction		(Y, N, T, G or blank)
Spanned / Nonspanned		(S=Spanned, N=Nonspanned or blank)

Figure 3-58 Data class specifying extended format

To define an extended format or extended addressability DFSMSrmm CDS, you must include the DATACLAS keyword as shown in Figure 3-59, and reference the correct data class. For more information about how to reference a data class, refer to *DFSMS Access Method Services for Catalogs*, SC26-7394.

```
//STEP01 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,UNIT=3390,VOL=SER=DFRMM4
//SYSIN DD *
DELETE RMM.PROD.CDS CL
SET MAXCC = 0
/*
  DEFINE CLUSTER(NAME(RMM.PROD.CDS) -
    FILE(MASTER) -
    FREESPACE(15 0) -
    KEYS(56 0) -
    REUSE -
    RECSZ(512 9216) -
    SHR(3 3) -
    KILOBYTES(4500 1500) -
    STORAGECLASS(gspace) -
    DATACLASS(EFKSDS) -
    VOLUMES(DFRMM4)) -
  DATA(NAME(RMM.PROD.CDS.DATA) -
    BUFFERSPACE(829440) -
    CISZ(26624)) -
  INDEX(NAME(RMM.PROD.CDS.INDEX) -
    CISZ(2048))
/*
```

Figure 3-59 Sample JCL to allocate an extended format DFSMSrmm CDS

3.1.12 Creating the DFSMSrmm journal

The DFSMSrmm journal contains a record of all changes made to the CDS since the last backup. We recommend that you create the journal and use it to forward recover changes made since the last backup. Each time you successfully back up the CDS by using the EDGHSKP utility, the journal data set is cleared.

We recommend that you allocate the journal data set on a different volume than the CDS, back up the journal data set, and maintain multiple generations of it.

To create the DFSMSrmm journal on the system, perform the following steps:

1. Calculate the space of the DFSMSrmm journal.
2. Protect the DFSMSrmm journal.
3. Allocate the DFSMSrmm journal.

Calculating the space of the DFSMSrmm journal

Table 3-6 shows the space requirement for each kind of record in the journal data set.

Table 3-6 *DFSMSrmm journal space requirement*

Journal content	DASD space
Changes by users	1.5 KB for each change made
Data sets	1.5 KB for each data set retained by a vital record specification
Data sets no longer retained by a vital record specification	1.5 KB for each data set no longer retained by a vital record specification
Expiring volumes 1.5 KB for each expiring volume	1.5 KB for each expiring volume
Non-scratch mounts	6.7 KB for each mount
Scratch mounts	8.3 KB for each mount
Volumes	1.5 KB for each volume retained by a vital record specification
Volume checked in/out	2.6 KB for each volume in or out of the library
Volumes returned to scratch	3.0 KB for each volume returned to scratch
Volumes to and from storage locations	3.5 KB for each volume moved to or from a storage location
Volumes no longer retained by a vital record specification	1.5 KB for each volume that has not reached its expiration date and is no longer retained by a vital record specification
Volumes that are exported or imported	1.5 KB for each logical volume exported or imported
Vital record specifications	1.3 KB for each vital record specification created

Divide the total KB of space by 4, because allocation is by average record size using a 4 K value. Use this number to in the space allocation (SPACE=) shown in Figure 3-61 on page 130.

Important: If you have specified JRNLTRAN(YES) you should plan on providing up to 33% more journal data set space to accommodate the additional records.

Protecting the DFSMSrmm journal

To protect the DFSMSrmm journal, define a discrete profile, and then permit the appropriate RACF groups to the RACF data set profile. Figure 3-60 on page 130 contains sample JCL. For detailed information about implementing RACF security for DFSMSrmm, refer to Appendix A, “Security topics” on page 455.

```
//RACFDSN JOB ,RMM,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
//* *****
//*      To execute this job you need the RACF SPECIAL attribute
//* *****
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/*  DEFINE JOURNAL DATA SET DATA SET PROFILE                               */
/*  ***** */
  ADDSD 'RMM.PROD.JRNL' UACC(NONE) GENERIC
    PERMIT 'RMM.PROD.JRNL' GENERIC-
      ID(EDGADMIN) ACC(READ)
    PERMIT 'RMM.PROD.JRNL' GENERIC-
      ID(EDGLIB) ACC(READ)
    PERMIT 'RMM.PROD.JRNL' GENERIC-
      ID(EDGSYSPG) ACC(ALTER)
```

Figure 3-60 Sample JCL to protect the DFSMSrmm journal

Allocating the DFSMSrmm journal

Sample JCL to create the DFSMSrmm journal is provided in SYS1.SAMPLIB member EDGJNLAL. You can modify the sample JCL to meet your installation standards. Example 3-14 shows sample JCL to create the DFSMSrmm journal.

```
//STEP01 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
  DEL RMM.PROD.JRNL
  SET MAXCC=0
/*
//STEP02 EXEC PGM=IEFBR14
//SYSPRINT DD SYSOUT=*
//JOURNAL DD DISP=(,CATLG,DELETE),
//          DSN=RMM.PROD.JRNL,
//          UNIT=3390,VOL=SER=DFRMM3,
//          AVGREC=U,SPACE=(4096,(6957),,CONTIG)
```

Figure 3-61 Sample JCL to allocate the DFSMSrmm journal

For initial testing with DFSMSrmm, either no journal or a small journal is sufficient. Later, when you use converted data, you need to use a journal that is large enough for your daily needs.

3.1.13 Restarting MVS with DFSMSrmm implemented

Attention: You can schedule the IPL to implement the DFRM subsystem any time ahead of migration and parallel running, but you need to issue the following command as part of that IPL before tapes are used.

S DFRMM,OPT=RESET

You can use this command only if you defined the started task DFRMM as shown in Figure 3-51 on page 121, and you defined the resource STGADMIN.EDG.RESET.SSI in RACF class FACILITY as explained in 3.1.7, “Defining RACF groups for DFSMSrmm” on page 85. The RACF user ID of the started task needs ALTER access to this profile.

You are ready to start the system with DFSMSrmm implemented. You may be able to dynamically implement changes to the MVS system PARMLIB members or modified installation exits depending on your MVS level. You must:

- ▶ IPL with CLPA to include new levels of the DFSMSrmm code that have LPALIB as the target library
- ▶ IPL when you make changes to IFAPRDxx to change the licensing options.
- ▶ IPL to implement the DFRM subsystem if you plan to run DFSMSrmm in parallel with any other tape management system

You can implement DFSMSrmm without the need of an IPL if you use the TSO command (see Figure 3-5 on page 61) and MVS commands (see Figure 3-2 on page 59) as described previously.

Important: If you installed a z/OS release prior to z/OS V1.3 and your tape management system is running, you cannot update the subsystem name table dynamically to implement DFSMSrmm. In this case, you have to re-IPL your MVS system with CLPA.

If you installed DFSMSrmm Release 2.10, you can update the subsystem name table dynamically to implement DFSMSrmm, but there are error messages such as **EDG0003E** or **EDG0004E**, which you can ignore. When you are prompted by the **EDG0103D** message, reply **RETRY**.

With z/OS V1.3 and following releases, there is no longer a requirement to have DFSMSrmm started before other tape management systems. DFSMSrmm does not update LPA or use dynamic hooks into DFSMSdfp. Therefore, the only need for an IPL with CLPA beginning with z/OS V1.3 is if any DFSMSrmm maintenance goes into LPALIB.

3.1.14 Starting and restarting DFSMSrmm

Usually, the DFSMSrmm subsystem starts automatically through standard initial program load (IPL). Under some conditions, you might need to start or restart the subsystem.

To start DFSMSrmm using the default procedure name and PARMLIB member, enter the f MVS command shown in Figure 3-62

```
S DFRMM
```

Figure 3-62 Start the DFRMM task

If you want, start DFSMSrmm with different parameters than the defaults. You can use the MVS command as you can see in Figure 3-63 to specify another PARMLIB data set and member name.

```
S DFRMM,M=xx,DSN=parmlib_dataset
```

Figure 3-63 Start the DFRMM task with different parameters

Where:

- M** specify the suffix of the EDGRMMxx member in PARMLIB.
- DSN** specify a different PARMLIB data set name|

After this command, you should see the message shown in Figure 3-64

```
EDG0105I DFSMSrmm SUBSYSTEM INITIALIZATION COMPLETE
```

Figure 3-64 EDG0105I initialization message

Figure 3-65 shows you the message that tells you that the subsystem interface was not activated before.

```
EDG0103D DFSMSrmm SUBSYSTEM INTERFACE IS INACTIVE - ENTER "IGNORE",  
"CANCEL" OR "RETRY"
```

Figure 3-65 EDG0103D initialization message

Enter **RETRY** to continue with the initialization. After that, you receive the EDG0105I message.

You can restart DFSMSrmm using the MVS MODIFY command, for example:

- ▶ To restart DFSMSrmm and implement new PARMLIB options, specify the EDGRMM parmlib member suffix in the command as shown in Figure 3-66

```
F DFRMM,M=xx
```

Figure 3-66 Restart DFRMM specifying a different member suffix

- ▶ To turn Problem Determination Aid (PDA) tracing on or off, specify one of you can use the commands shown in Figure 3-67

```
F DFRMM,PDA=ON  
F DFRMM,PDA=OFF
```

Figure 3-67 Turn problem determination on or off

- ▶ To manually quiesce DFSMSrmm to stop all tape related activities and de-allocate the control data set and journal use the command you can see in Figure 3-68

```
F DFRMM,QUIESCE
```

Figure 3-68 Quiesce DFSMSrmm

- To refresh DFSMSrmm installation exits and enable any exits in an z/OS environment before V1R11 that are currently disabled, specify the command shown in Figure 3-69.

```
F DFRMM,REFRESH EXITS
```

Figure 3-69 Refresh installation exits prior z/OS V1R11

- With z/OS V1.11, the MODIFY DFRMM,REFRESH EXIT command is no longer supported. You can use the SETPROG EXIT command instead, which allows to manage the exit modules independently. Remember, that also the EXIT statement of the PROGxx parmlib member can be used to add an exit module. In Figure 3-70 you can see how you can add, modify or delete a single DFSMSrmm installation exit using the SETPROG operator command.

```
SETPROG EXIT,ADD,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300  
SETPROG EXIT,MODIFY,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300,STATE=INACTIVE  
SETPROG EXIT,DELETE,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300
```

Figure 3-70 SETPROG command syntax to manage exit modules

To refresh an exit you have to delete the exit first and then you must add the exit to reload the module from the library as shown in Figure 3-71

```
SETPROG EXIT,ADD,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100  
SETPROG EXIT,ADD,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100
```

Figure 3-71 Refresh installation exits with z/OS V1R11 or higher

Note: Beside deleting and adding an exit module, you can also temporarily inactivate it, but keep in mind, that setting the state to ACTIVE again does not imply a reload of the module from the library.

You can use the SETPROG EXIT command to control exits that have been defined to the dynamic exits facility. To activate or change the current used EDGUX100 user exit you have to use the command shown in Figure 3-72 this will replace the old sequence you have to use before z/OS V1.12.

```
SETPROG EXIT,REFRESH,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100
```

Figure 3-72 SETPROG command syntax to refresh an dynamic exit

For more information about these commands, see *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

3.1.15 Stopping DFSMSrmm

The DFSMSrmm subsystem must always be activated in your system. In some recovery situations, you might need to remove DFSMSrmm from your system. In that case, you must follow this procedure:

1. All jobs that are processing and using tapes should be completed. Before you stop DFSMSrmm, stop batch initiators to avoid failing a job that opens a tape data set. Also, obtain a list of the number of requests waiting to be processed and the number of active requests. Figure 3-73 shows you the correct use the query active command.

`F DFRMM,QUERY ACTIVE`

Figure 3-73 Query active command syntax

As a result of this command, you see these messages shown in

`EDG1113I F=function requestor_type=requestor_name time TKN=token_value
EDG1114I COMMAND COMPLETE, TOTAL TASKS total_count, ACTIVE active_count
EDG1118I queued_count QUEUED REQUESTS, INCLUDING nowait_count NOWAIT
catalog_count CATALOG`

Figure 3-74 DFSMSrmm query active messages

Where:

F	Identifies the requested DFSMSrmm function; the values are internal to DFSMSrmm
Requestor_type:	Identifies the requestor. It can be JOB, STC, or TSU
Time:	Lists the time that the request was started
TKN:	Identifies the request
Total_count:	The number of tasks available to process DFSMSrmm requests
Active_count:	The number of tasks currently processing
Queued_count:	The number of requests waiting to be processed by DFSMSrmm
Nowait_count:	The number of requests waiting to be processed by DFSMSrmm for which the request does not wait for the results; includes the count of catalog requests
Catalog_count:	The number of requests waiting to be processed by DFSMSrmm to reflect catalog update activity in the DFSMSrmm control data set

Figure 3-75 shows sample output from this command. In this sample, the only request to DFSMSrmm is the HOUSEKEEP processing executed by the YCJRES1B job.


```

EDG1119I DFSMSrmm STATUS IS ACTIVE. JOURNAL ENABLED.
EDG1120I Function System Task Name Started Token S IP Status
EDG1113I HSKP JOB=MHLRES7H 17:58:08 00300028 : :
EDG1114I LOCAL TASKS 10, ACTIVE 1, SERVER TASKS 0, ACTIVE 0
EDG1122I HELD 0 HELD 0
EDG1118I 0 QUEUED REQUESTS, INCLUDING 0 NOWAIT 0 CATALOG
EDG1121I DEBUG: DISABLED, PDA TRACE LEVEL: 1-2-3-4- RESERVE: 17:58:10
EDG1101I DFSMSrmm MODIFY COMMAND ACCEPTED

```

Figure 3-75 DFSMSrmm MODIFY command output

2. Optionally, remove the DFSMSrmm subsystem. Usually, you do not need to remove it. If you want to allow tape use without DFSMSrmm recording, then you can use the reset command shown in Figure 3-76.

```
S DFRMM,OPT=RESET
```

Figure 3-76 Remove the DFSMSrmm subsystem

Important: To use this command, the user ID of the started task needs ALTER access to the resource **STGADMIN.EDG.RESET.SSI** defined in the RACF FACILITY class.

Then, wait for the response shown in Figure 3-77.

```

IEF403I DFRMM - STARTED - TIME=18.48.27 - ASID=0063.
EDG0181I DFSMSRMM SUBSYSTEM INTERFACE SUCCESSFULLY INACTIVATED

```

Figure 3-77 DFSMSrmm subsystem reset command output

The use of OPT=RESET must be RACF protected. Refer to “Defining DFSMSrmm resources to RACF” on page 675, for additional information about the RACF profiles.

Important: The OPT=RESET operand should only be used during parallel running or testing, when DFSMSrmm must be completely removed from the system due to an error, or because you need to reconvert.

3. Shut down the DFSMSrmm subsystem, using the following command you can see in Figure 3-78

```
P DFRMM
```

Figure 3-78 Stop the DFRMM task

When you see the messages in Figure 3-79, DFSMSrmm is completely removed from your system.

```

IEF352I ADDRESS SPACE UNAVAILABLE
$HASP395 DFRMM ENDED

```

Figure 3-79 Stop DFRMM messages

3.1.16 Setting up DFSMSrmm utilities

There are several DFSMSrmm utilities that you should set up now:

- ▶ EDGHSKP is used to perform the following inventory management activities:
 - Vital record processing, to determine which volumes to retain and which volume moves are required, based on VRSs
 - Expiration processing to identify volumes ready to be released and returned to scratch
 - Storage location management processing to assign shelf locations to volumes being moved to storage locations
 - Backup and recovery of the CDS and journal

Use the DFSMSrmm backup utilities rather than other backup utilities, such as the IDCAMS EXPORT command, because DFSMSrmm provides the necessary serialization and forward recovery functions. DFSMSrmm backup utilities check whether the CDS is in use, tell the DFSMSrmm subsystem that backup or recovery is in process, and provide a way to forward recover.

- Creation of an extract data set

You can produce movement and inventory reports by producing an extract data set from the CDS, and creating a report from it with the report utility.

- ▶ EDGBKUP is used to back up and recover the control data set and back up the journal.
- ▶ EDGAUD and EDGRPTD can help you in getting information about your removable media library and storage locations. You can also get security related information about volumes and data sets defined to DFSMSrmm, and audit trail information about volumes, shelf assignments, and user activity.
- ▶ EDGUTIL is the utility used to create, update, and verify the CDS.
- ▶ EDGINERS is the DFSMSrmm utility that helps you erase and initialize tape volumes either automatically or manually. You can use EDGINERS to replace the DFSMSdftp utility IEHINITT.

You should create a sample schedule of how frequently to run DFSMSrmm utilities. For more detailed information about the DFSMSrmm utilities, refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

3.1.17 Running the installation verification program

The quickest and simplest way of getting DFSMSrmm started and verifying that the key pieces are installed successfully is to run the installation verification procedure (IVP). You can run the IVP at any time, for example, after installing maintenance on your system.

See the *z/OS Program Directory* for the IVP procedures. The information includes considerations for running in an environment where another tape management system is in use.

3.2 Tailoring the DFSMSrmm ISPF dialog

In this section, we describe how to make the DFSMSrmm ISPF panels available to the user, and how to modify the panels layout only for end users and librarians.

To make the DFSMSrmm ISPF dialog available in the ISPF Master Application Menu, use one of the following techniques:

- ▶ Concatenate the DFSMSrmm target ISPF libraries with your existing ISPF libraries and use one of these methods:
 - Add DFSMSrmm to the ISPF dialog as explained in 3.2.1, “Adding DFSMSrmm to ISPF panels” on page 100.
 - Use the default method supplied by DFSMS. Select option **R** from the Interactive Storage Management Facility (ISMF) primary option menu.
 - Use the RMMISPF EXEC to invoke the dialog.
- ▶ Use the ISPF LIBDEF facility to make the DFSMSrmm target ISPF libraries available to your users, and use the EDGRMLIB EXEC to enter the dialog.
- ▶ When using the LIBDEF facility, if you are using different target library names as listed in the *OS/390 Program Directory* and *ServerPac: Installing Your Order*, you must modify the EDGRMLIB EXEC or produce your own similar REXX exec or CLIST.

3.2.1 Adding DFSMSrmm to ISPF panels

You can add DFSMSrmm selection in ISPF Main Application Menu, so that users can choose it easily. To add the selection, follow these steps:

1. Add the DFSMSrmm selection to the body of the chosen ISPF selection panel. For example, add the following text:


```
RM RMM DFSMSrmm dialog
```
2. Add one of the following statements to the ZSEL processing list in the)PROC section of the chosen ISPF panel:
 - If you are not using LIBDEF, add the selection shown in Figure 3-80.

```
R, 'CMD(%RMMISPF) NEWAPPL(EDG) '
```

Figure 3-80 Start RMM dialog using a new application

- If you are using LIBDEF, add the following selection shown in Figure 3-81

```
R, 'CMD(%EDGRMLIB) '
```

Figure 3-81 Start RMM dialog without a new application

Figure 3-82 on page 137 shows an example definition using LIBDEF.

```
...
$AO @AOC      @- Automated Operations Control/MVS Dialogs      +
$RM @RMM      @- DFSMSrmm dialog                                +
$CN @CONS     @- Console Display and Search Facility            +
...
)PROC
&ZSEL =TRANS(
    ..
    AO, 'CMD(%$AOC140) '
    RM, 'CMD(%EDGRMLIB) '
    ..
....
```

Figure 3-82 Adding DFSMSrmm to an ISPF selection panel

3.2.2 Modifying the DFSMSrmm panel

When you enter the DFSMSrmm ISPF dialog, usually the first panel you see is the Primary Option Menu panel. You can change the panel navigation for the different type of users to go directly to a lower level panel, so that when they enter DFSMSrmm, tape librarians always see the Librarian Menu and end users see the DFSMSrmm User Menu:

- ▶ You can modify the selection as follows, so that only the USER option of the DFSMSrmm dialog is available to the majority of end users. Choose one of the following ways to change:
 - When using LIBDEF, see Figure 3-83 how to add the selection to the)PROC section.

```
R, 'CMD(%RMMISPF USER) NEWAPPL(EDG) '
```

Figure 3-83 ISPF PROC section

- When using LIBDEF, modify the supplied EDGRMLIB EXEC to include the USER parameter on the RMMISPF EXEC call. For example, enter the REXX statement to replace the one supplied in EDGRMLIB as shown in Figure 3-84

```
address "ISPEXEC""SELECT CMD(%RMMISPF USER) NEWAPPL(EDG)PASSLIB"
```

Figure 3-84 ISPEXEC call using a new application

If you are not using LIBDEF, Table 3-7 shows the names of the default libraries that you concatenate to the DD statements in the TSO logon procedure or a user-supplied start-up CLIST. If you are using LIBDEF, the EDGRMLIB EXEC allocates these default libraries to the user. You need to modify the EDGRMLIB exec to contain the new library names, after you changed the names of the target libraries on your system.

Table 3-7 *DFSMSrmm libraries and their default DD names*

DFSMSrmm data set name	DD statement	Content
SYS1.SEDGEXE1	SYSPROC (or SYSEXEC)	REXX execs
SYS1.SEDGMENU	ISPMLIB	English messages
SYS1.SEDGPENU	ISPPLIB	English panel
SYS1.DGTSLIB	ISPSLIB	Skeletons
SYS1.DGTTLIB	ISPTLIB	Tables
SYS1.HELP	SYSHELP	TSO HELP

Note: The DFSMSrmm online help or the RMM TSO command is moved from SYS1.SEDGHLP1 to SYS1.HELP in z/OS V1.3.

- ▶ You can modify the selection as follows so that the LIBRARIAN option of the DFSMSrmm dialog is available to the librarians. Choose one of the following ways to change:
 - When using LIBDEF, add the following selection to the)PROC section as shown in Figure 3-85.

```
R, 'CMD(%RMMISPF LIBRARIAN) NEWAPPL(EDG) '
```

Figure 3-85 ISPF PROC section to call the library ISPF menu directly.

- When using LIBDEF, modify the supplied EDGRMLIB EXEC to include the USER parameter on the RMMISPF EXEC call. For example, enter the following REXX statement to replace the one supplied in EDGRMLIB as shown in Figure 3-86.

```
address "ISPEXEC" "SELECT CMD(%RMMISPF LIBRARIAN) NEWAPPL(EDG)PASSLIB
```

Figure 3-86 ISPEXEC call for the librarian using a new application

3.2.3 Other changes

There are other changes that you may want to implement in the DFSMSrmm ISPF dialog.

- ▶ Add local dialog

DFSMSrmm has a dummy panel named EDGP@LCL, which provides easy access to local dialog extensions. You can use these extensions without having to modify DFSMSrmm.

- ▶ Modify add product volume defaults

You can modify the dialog EXEC EDGRPADV to change the values to ones that suit your installation.

Recommendation: Install your changes using an SMP/E USERMOD after editing a copy of the EDGRPADV EXEC.

- ▶ Modify DFSMSrmm messages

DFSMSrmm provides messages for report titles and user notification. You can customize them by:

- Updating the text of a message in the DFSMSrmm message table EDGMTAB
- Applying changes to EDGMTAB by creating an SMP/E-installable USERMOD

For more information, refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

3.3 Making TSO HELP information available to users

DFSMSrmm provides online help for the RMM TSO command. The help information is, by default, installed into the SYS1.HELP data set, although this can be changed by you during the SMP/E installation process.

You can update the TSO logon procedure or use the TSO ALLOCATE command. Example 3-16 shows how to update the TSO logon procedure.

Example 3-14 *Update the TSO logon procedure*

```
//IKJACCNT PROC
//IKJACCT EXEC PGM=IKJEFT01, PARM='ISPPDF',
// DYNAMNBR=150,TIME=1440
//SYSPROC DD DSN=SYS1.OS390.CLIST,DISP=SHR
```

```
//SYSHELP DD DSN=SYS1.HELP,DISP=SHR
//*      DD DSN=SYS1.SEDGHLP1,DISP=SHR
//ADMPD  DD DSN=GDDM.SADMPD,DISP=SHR
//ADMCGM DD DSN=GDDM.SADMSAM,DISP=SHR
//ADMGGMAP DD DSN=GDDM.SADMMAP,DISP=SHR
//ADMPROJ DD DSN=GDDM.SADMMAP,DISP=SHR
//ADMIMG  DD DSN=GDDM.SADMMAP,DISP=SHR
//ADMGIMP DD DSN=GDDM.SADMMAP,DISP=SHR
//ADMGDF  DD DSN=GDDM.SADMMAP,DISP=SHR
//ADMSYMBL DD DSN=GDDM.SADMSYM,DISP=SHR
//SYSPRINT DD TERM=TS,SYSOUT=*
//SDFSDUMP DD SYSOUT=*
//SYSTEM  DD TERM=TS,SYSOUT=*
//SYSIN   DD TERM=TS
//*
```

Note: The DFSMSrmm online help or the RMM TSO command is moved from SYS1.SEDGHLP1 to SYS1.HELP in z/OS V1.3.

3.4 Verifying your DFSMSrmm implementation

To test DFSMSrmm, you can use a small, test CDS or a converted CDS. We recommend that you use simple DFSMSrmm function tests for your first testing. Later, after you have analyzed the data and customized the exits and parameters, you can test those functions that are used in production.

Verifying DFSMSrmm functions by submitting batch jobs is quite simple. This section shows sample batch jobs that we used to verify DFSMSrmm functions in our system. Run the sample jobs in the sequence shown to test basic DFSMSrmm functions in your system.

3.4.1 Displaying the PARMLIB options and control information

Use the RMM LISTCONTROL subcommand to display information from the DFSMSrmm CDS control record and options defined in the EDGRMMxx PARMLIB member.

For example, display your installation's options and rules, restricting the information displayed to the control record information and system options only (see Figure 3-87). Specify the ALL operand to display all options and rules defined to your installation.

```
//LISTCNTL JOB ,140.SCHLUMBERGER,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//* *****
//*          STEP 1  DISPLAY INFORMATION          *
//* *****
//STEP01   EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN  DD *
RMM LISTCONTROL ALL
/*
```

Figure 3-87 Sample JCL to display PARMLIB options and control information

Example 3-15 shows the output from the RMM LISTCONTROL command.

Example 3-15 Sample RMM LISTCONTROL subcommand output

```

READY
RMM LC ALL
Control record:
Type = MASTER      Create date = 2009/309      Create time = 16:38:11
                  Update date = 2012/287      Update time = 15:51:46
Journal: Utilization = 4% (75% threshold)      STATUS: = ENABLED
CDS:   Utilization = 19%
Exit status:
    EDG_EXIT100 = ENABLED
    EDG_EXIT200 = NONE
    EDG_EXIT300 = ENABLED
Options:
    Stacked Volumes      = NONE
    Extended Bin         = DISABLED
    Common Time          = DISABLED
    CDSID ENQ name       = ENABLED
Last backup:
    Date = 2012/130      Time = 22:07:52
Last journal backup:
    Date = 2012/130      Time = 22:07:52
Last report extract:
    Date = 2012/130      Time = 22:07:49
Last scratch procedure:
    Date =                Time =
Rack numbers           = 0
LOCAL store bins       = 0
DISTANT store bins     = 0
REMOTE store bins      = 0
Control functions in progress:
Backup      = N  Restore      = N
Verify      = N  Expiration   = N
Report Extract = N  Disaster Store = N
VRS         = N  Synchronize  = N
Client/Server:
    host name =
    IP address =

System options:
PARMLIB Suffix = 02
Operating mode = P      Retention period: Default = 0      Maximum = NOLIMIT
                        Catalog = 6      hours
                        Retention method: Method = VRSEL
Control data set name   = RMM.CONTROL.DSET
Journal file data set name = RMM.JOURNAL.DSET
Journal threshold       = 75%      Journal transaction = NO
Catalog SYSID          = Notset
Scratch procedure name  = EDGXPROC
Backup procedure name   = EDGCDSBK
IPL date check = N      Date format = J      RACF support = N
SMF audit = 0      SMF security = 42      CDS id = SC70
MAXHOLD value = 100    Lines per page = 54      System ID = SC70
BLP = RMM      TVEXT purge = RELEASE Notify = N
                        days = 0
Uncatalog = Y      VRS job name = 2      Message case = M
MASTER overwrite= USER Accounting = J      VRS selection = NEW
VRS change = INFO  GDG duplicate = BUMP      GDG cycle by = GENERATION
VRSMIN action = INFO VRSMIN count = 1
VRSDROP action = INFO VRSDROP count = 0      percent = 10
VRSRETAIN action= INFO VRSRETAIN count= 0      percent = 80
EXPDTDROP action= INFO EXPDTDROP count= 0      percent = 10
Disp DD name = DISPDD Disp msg ID = EDG4054I

```

```

Retain by      = SET      Move by      = SET      CMDAUTH Owner = NO
PREACS        = NO       SMSACS        = NO       CMDAUTH Dsn  = YES
Reuse bin     = CONFIRMMOVE                                     Media name   = 3480
                                                         Local tasks  = 10

```

PDA: ON

```

Block count   = 255      Block size   = 27      Log           = ON

```

SMSTAPE:

```

Update scratch = YES      Update command = YES      Update exits  = YES
Purge          = ASIS

```

Client/Server:

```

Subsystem type = STANDARD Port           = 0
Server         Server tasks             = 0
host name      =
IP address     =

```

Security classes:

Number	Name	SMF	MSG	Erase	Description
10	UCL	Y	N	N	UNCLASSIFIED
75	IUO	Y	N	N	IBM INTERNAL USE ONLY
100	IC	Y	N	Y	IBM CONFIDENTIAL
125	ICR	Y	N	Y	IBM CONFIDENTIAL RESTRICTED
150	RIC	Y	Y	Y	REGISTERED IBM CONFIDENTIAL

Volume Pools:

Pool	System	RA CF	Ty pe	Expdt check	Pool name	Media name	Description

Action Scratch Overwrite							

NS*		N	R	Y		3590	NORBERTS SCRATCH POOL
					MANUAL	USER	
THS*		N	S	N		3590	3592 POOL
					AUTO	USER	
VT*		N	S	N	\$\$DFLT	3592	VTFM 3592 POOL
					AUTO	LAST	
*		N	S	O		3480	DEFAULT VOLUME POOL
					AUTO	USER	

Mount/Fetch messages:

Message ID	ID	Volume	Rack
IEF233A	1	15	999

NO INSTALLATION DEFINED REJECT PREFIXES. PARTITION AND OPENRULE COMMANDS ARE IN
CONT:- USE

Location definitions:

Location	Def	Mgtype	Ltype	Priority	AM	Medianames
	N		AUTO	4800	Y	
	N		MANUAL	4900	Y	
DISTANT	N		STORE	200	Y	
LOCAL	N		STORE	300	Y	
REMOTE	N		STORE	100	Y	
SHELF	N			5000	Y	
VTFM001	Y	NOBINS	HSTORE	2000	N	*

Media Information :

Name	Media type	Recording format	Capacity (MB)	Replace Policy			
				Perm	Temp	WMC	Age
VTFM	5 VT3590G2	5 VT3590G2	1000	0	0	0	0
VTFM	5 VT3590G2	0 *	1000	0	0	0	0
VTFM	0 *	0 *	0	0	0	0	0

Partition Entries:

Volume or Range	Type	SMT	NOSMT	Location
		Action	Action	
*	RMM	ACCEPT	ACCEPT	SHELF
*	NORMM	IGNORE	IGNORE	

Openrule Entries:

Volume or Range	Type	Input		Output	
		Action	Condition	Action	Condition
*	RMM	ACCEPT		ACCEPT	
*	NORMM	REJECT		REJECT	

Action Status

REPLACE	Pending
SCRATCH	Pending
INIT	Unknown
ERASE	Unknown
RETURN	Unknown
NOTIFY	Unknown

Source	Target	Move Type	Status
SHELF	ATVIGA	NOTRTS	Pending
END			

3.4.2 Adding owner information

Use the ADDOWNER subcommand to define an owner to DFSMSrmm. An owner can be an individual or group defined by a RACF group name, or any value you choose. The owner value contains one to eight alphanumeric characters.

DFSMSrmm automatically creates an owner record if a user who is not defined to DFSMSrmm requests a job that writes to a volume managed by DFSMSrmm. DFSMSrmm uses the user ID that requests the job as an DFSMSrmm owner ID. In all other cases, the owner must be defined manually, and must be defined before you can use a RMM TSO subcommand that includes owner information.

Use the LISTOWNER subcommand to display information about a single owner defined to DFSMSrmm. For example, add details of a new owner and request information recorded by DFSMSrmm about the owner whose owner ID is SCHLUM (see Figure 3-88).

```

//STEP01 EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
RMM ADDOWNER SCHLUM
                FNAME('NORBERT')
                SNAME('SCHLUMBERGER')
                DEPARTEMENT('International Projects (A047)')
                ADD1('IBM Deutschland GmbH')
                ADD2('IBM Allee 1')
                ADD3('D-71139 Ehningen')
                INTEL('919-3579')
                EXTEL('++49-(0)-171-2222663')
                USER(SCHLUM)
                NODE(IBMDE)
                EMAIL(SCHLUM$DE.IBM.COM

RMM LISTOWNER SCHLUM
/*

```

Figure 3-88 Sample JCL to add owner information

Example 3-16 shows the output of the TSO RMM LISTOWNER subcommand.

Example 3-16 Sample RMM LISTOWNER subcommand output

```

Last name   = SCHLUMBERGER           First names = NORBERT
Department  = International Projects (A047)
Address     = IBM Deutschland GmbH
              IBM Allee 1
              D-71139 EHNINGEN

Telephone:
Internal    = 919-3579                External   = 49-(0)-171-2222663
Electronic mail:
Userid      = SCHLUM                  Node        = IBMDE
Email       = SCHLUM@DE.IBM.COM

Volumes     = 0

Last Change information:
Date        = 11/10/2012   Time = 07:00:20   System  = EGZB
User change date = 16/09/2003   Time = 17:34:49   User ID = *HKP

```

3.4.3 Adding racks to DFSMSrmm

The ADDRACK subcommand defines shelf locations in the removable media library and storage locations. DFSMSrmm defines shelf spaces in the removable media library as rack numbers and bin numbers in storage locations. Use the ADDRACK subcommand to define empty, or available, rack and bin numbers to DFSMSrmm. You must supply an initial rack number.

Note: There is no longer a need to add RACKs except if you would like to define RACKs to manage volumes with volume numbers not in sequence very easy.

The SEARCHRACK subcommand creates a list of shelf locations defined in the removable media library. Shelf locations in the removable media library are called rack numbers.

For example, add 20 racks to the removable media library, starting with rack number **SC0000**. Second create a list of the created rack numbers (see Figure 3-89).

```
//STEP01 EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
RMM ADDRACK NS0000 -
        LOCATION(SHELF) -
        MEDIANAME(3592) -
        COUNT(15)

RMM SEARCHRACK RACK(NS*) LIMIT(*)
/*
```

Figure 3-89 Sample JCL to add RACK information

Example 3-17 shows the output of the TSO RMM SEARCHRACK subcommand.

Example 3-17 Sample RMM SEARCHRACK subcommand output

Rack/Bin number	Location	Media name	Status	Current volume	Moving-in volume	Moving-out volume	Old volume
NS0000	3592	NS0000	EMPTY	SHELF			
NS0001	3592	NS0001	EMPTY	SHELF			
NS0002	3592	NS0002	EMPTY	SHELF			
NS0003	3592	NS0003	EMPTY	SHELF			
NS0004	3592	NS0004	EMPTY	SHELF			
NS0005	3592	NS0005	EMPTY	SHELF			
NS0006	3592	NS0006	EMPTY	SHELF			
NS0007	3592	NS0007	EMPTY	SHELF			
NS0008	3592	NS0008	EMPTY	SHELF			
NS0009	3592	NS0009	EMPTY	SHELF			
NS0010	3592	NS0010	EMPTY	SHELF			
NS0011	3592	NS0011	EMPTY	SHELF			
NS0012	3592	NS0012	EMPTY	SHELF			
NS0013	3592	NS0013	EMPTY	SHELF			
NS0014	3592	NS0014	EMPTY	SHELF			
15	ENTRIES LISTED						

3.4.4 Adding volumes to DFSMSrmm

The ADDVOLUME subcommand adds one or more volumes to DFSMSrmm. You have to define all volumes that you use in the DFSMSrmm CDS. DFSMSrmm monitors all tape volume mounts and automatically updates information about your predefined tape volumes when they are used.

The SEARCHVOLUME subcommand creates a list of volumes matching the search criteria you specify. You can display lists of volumes according to ownership, assigned date, status, movement, action, pool, or media name.

For example, add 10 new scratch volumes in STEP01 to the removable media library using a rack number that is the same as the VOLSER. The volumes are not initialized. Then, add 10 new scratch volumes to the removable media manager into the racks, but in this case the volumes must be labeled before they can be used, as indicated by INITIALIZE(Y). Finally, create a list of all previous added volumes (see Figure 3-90 on page 146.

```
//STEP01 EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
RMM ADDVOLUME NS0000 -
                STATUS(SCRATCH) -
                INITIALIZE(N) -
                LOCATION(SHELF) -
                MEDIANAME(3592) -
                MEDIATYPE(*) -
                COUNT(10)

RMM ADDVOLUME NS0010 -
                STATUS(SCRATCH) -
                INITIALIZE(Y) -
                LOCATION(SHELF) -
                MEDIANAME(3592) -
                MEDIATYPE(*) -
                POOL(SC*) -
                COUNT(5)

RMM SEARCHVOLUME VOLUME(NS*) OWNER(*) LIMIT(*)
/*
```

Figure 3-90 Sample JCL to add volume information

Note: In the second example we will add the volumes using a different pool prefix. In this case you have to define rack numbers first.

Example 3-18 shows the output of the TSO RMM SEARCHVOLUME subcommand.

Example 3-18 Sample RMM SEARCHVOLUME subcommand output

Volume	Owner	Rack	Assigned date	Expiration date	Location	Dsets	St	Act	Dest.
NS0000		NS0000	2010/289		SHELF	0	S		
NS0001		NS0001	2010/289		SHELF	0	S		
NS0002		NS0002	2010/289		SHELF	0	S		
NS0003		NS0003	2010/289		SHELF	0	S		
NS0004		NS0004	2010/289		SHELF	0	S		
NS0005		NS0005	2010/289		SHELF	0	S		
NS0006		NS0006	2010/289		SHELF	0	S		
NS0007		NS0007	2010/289		SHELF	0	S		
NS0008		NS0008	2010/289		SHELF	0	S		
NS0009		NS0009	2010/289		SHELF	0	S		
NS0010		SC0000	2010/289		SHELF	0	I	I	
NS0011		SC0001	2010/289		SHELF	0	I	I	
NS0012		SC0002	2010/289		SHELF	0	I	I	

NS0013	SC0003	2010/289	SHELF	0	I	I
NS0014	SC0004	2010/289	SHELF	0	I	I

3.4.5 Adding bins to DFSMSrmm

The ADDBIN subcommand defines shelf locations in the storage locations. DFSMSrmm defines shelf spaces in storage locations as bin numbers. Use the ADDBIN subcommand to define empty bin numbers to DFSMSrmm.

The SEARCHBIN subcommand creates a list of shelf locations defined in the storage locations. Shelf locations in the storage locations are called bin numbers.

For example, add 5 empty shelf locations to the built-in storage location SANJOSE, starting with bin number 000001 (see Figure 3-91). The example assumes that you have defined storage location SANJOSE with the LOCDEF command in PARMLIB member EDGRMMxx. The second command creates a list of the new bin numbers.

```
//STEP01 EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
RMM ADDBIN 000001 -
            LOCATION(SANJOSE) -
            MEDIANAME(3592) -
            COUNT(5)

RMM SEARCHBIN BIN(*) LIMIT(*) LOCATION(SANJOSE)
/*
```

Figure 3-91 Sample JCL to add bins

Example 3-19 shows the results of the TSO RMM SEARCHBIN subcommand.

Example 3-19 Sample RMM SEARCHBIN subcommand output

Rack/Bin number	Location	Media name	Status	Current volume	Moving-in volume	Moving-out volume	Old volume
-----	-----	-----	-----	-----	-----	-----	-----
000001	SANJOSE	3590	EMPTY				
000002	SANJOSE	3590	EMPTY				
000003	SANJOSE	3590	EMPTY				
000004	SANJOSE	3590	EMPTY				
000005	SANJOSE	3590	EMPTY				
SEARCH COMPLETE - MORE ENTRIES MAY EXIST							
5	ENTRIES LISTED						

3.4.6 Adding a DSNAME VRS to DFSMSrmm

The RMM ADDVRS command adds a new VRS to DFSMSrmm. A VRS is used to define retention and movement policies for data set and volumes.

The LISTVRS subcommand displays details about a single VRS. Specify a data set name when requesting information about a data set VRS.

For example, add a data set VRS to retain all data sets matching the DSNAME mask SCHLUM.RMMTEST.MOVE.** in location REMOTE until the data set is no longer cataloged (see Example 3-27). The Second command requests information about the data set VRS defined for SCHLUM.RMMTEST.MOVE.** data sets.

```
//STEP01 EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
RMM ADDVRS DSNAME('SCHLUM.RMMTEST.MOVE.**') -
          CYCLES -
          COUNT(99999) -
          DESCRIPTION('RETAIN AND MOVE DATA SETS') -
          LOCATION(REMOTE) -
          OWNER(SCHLUM) -
          STORENUMBER(1) -
          WHILECATALOG

RMM LISTVRS DSNAME('SCHLUM.RMMTEST.MOVE.**')
/*
```

Figure 3-92 Sample JCL to add VRS

Example 3-20 shows the result of the TSO RMM LISTVRS subcommand.

Example 3-20 Sample RMM LISTVRS subcommand output

```
RMM LISTVRS DSNAME('SCHLUM.RMMTEST.MOVE.**')
Data set mask = SCHLUM.RMMTEST.MOVE.**                      Type = DSNAME
Job name mask =                                           Retain until expired = NO
Count          = 99999 CYCLES                             Retain while cataloged = YES
Delay          = 0      Days in the HOME location
Store number = 1      CYCLES in the REMOTE location
Priority       = 0

                                Release Options:
                                Expiry date ignore = NO
                                Scratch immediate = NO

Next VRS in chain   =          using          VRS

VRS Owner          = SCHLUM
Description         = RETAIN AND MOVE DATA SETS
Last Reference: Date =          Time =
Vital Record Specification to be deleted on 1999/365

Last Change information:
Date               = 2010/289      Time = 20:45:17      System = SC70
User change date = 2010/289      Time = 20:45:17      User ID = MHLRES7
```

3.4.7 Creating tape data sets

Use the IEBGENER utility to create a multifile, multivolume chain to understand the differences between DFSMSrmm and your tape management system. Figure 3-93 on page 149 shows sample JCL for creating different data sets on tape.

```

//CREATE1 JOB DE0A047,140.SCHLUMBERGER,...
//*      *****
//*      * 1. STEP  ALLOC 1. FILE  DSN=SCHLUM.RMMTEST.FILE1.VOL1  *
//*      *          VOLUME 1 / FILE 1 / RETPD 5480 (APPROX. 15 Y.) *
//*      *****
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DSN=DFRMM.TEST.DATA.SMALL,DISP=SHR
//SYSUT2 DD  DSN=SCHLUM.RMMTEST.FILE1.VOL1,
//          DISP=(,CATLG,DELETE),
//          UNIT=VT3590,VOL=(,RETAIN),
//          DCB=*.SYSUT1,LABEL=RETPD=5480
//SYSIN DD  DUMMY
...
//*      *****
//*      * 5. STEP  ALLOC 5. FILE  DSN=SCHLUM.RMMTEST.FILE5.VOL3  *
//*      *          VOLUME 3 / FILE 2 / RETPD 10                  *
//*      *****
//STEP05 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DSN=DFRMM.TEST.DATA.SMALL,DISP=SHR
//SYSUT2 DD  DSN=SCHLUM.RMMTEST.FILE5.VOL3,DISP=(,CATLG,DELETE),
//          DISP=(,CATLG,DELETE),
//          DCB=*.SYSUT1,LABEL=(5,SL,,RETPD=10),
//          VOL=(,RETAIN,,REF=*.STEP04.SYSUT2)
//SYSIN DD  DUMMY

```

Figure 3-93 Sample JCL to create data sets on tape

3.4.8 Allocating inventory management data sets

Before you can use the EDGHSKP utility to run inventory management activities, you must define several data sets, but not all data sets are required for all functions. Some data sets used during inventory management must be pre-allocated and cataloged because these data sets are used by both the EDGHSKP utility and the DFSMSrmm subsystem. Refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for additional information and sample jobs.

Figure 3-94 shows the sample JCL we used to allocate these data sets.

```

//STEP01 EXEC PGM=IEFBR14
//MESSAGE DD  DSN=RMM.PROD.MSGS,DISP=(NEW,CATLG),
//          UNIT=SYSDA,AVGREC=U,SPACE=(4096,(10,10))
//REPORT DD  DSN=RMM.HSKP.REPORT.FILE.NAME,DISP=(NEW,CATLG),
//          LRECL=137,RECFM=VBA,
//          UNIT=SYSDA,AVGREC=U,SPACE=(4096,(50,30))
//ACTIVITY DD  DSN=RMM.HSKP.ACTIVITY.FILE.NAME,DISP=(NEW,CATLG),
//          UNIT=SYSDA,AVGREC=U,SPACE=(4096,(50,30))
//REPTXT DD  DSN=RMM.HSKP.REPORT.EXTRACT.FILE.NAME,DISP=(NEW,CATLG),
//          UNIT=SYSDA,AVGREC=U,SPACE=(4096,(1000,500))
//XREPTXT DD  DSN=RMM.HSKP.REPORT.XEXTRACT.FILE.NAME,DISP=(NEW,CATLG),
//          UNIT=SYSDA,AVGREC=U,SPACE=(4096,(1000,500))

```

Figure 3-94 Sample JCL to pre-allocate EDGHSKP data sets

The REPTXT and XREPTXT DD statements allocate data sets that contain the extract copy of the DFSMSrmm control data set. The extract copy is called the extract data set. You must specify either the REPTXT DD statement or the XREPTXT DD statement when you use the EDGHSKP RPTXT parameter. You do not need both data sets created. A single REPTXT data set can be used for either normal extract or extended extract.

You can use also the sample JCL in SAMPLIB member EDGJHKPA to allocate ACTIVITY, MESSAGE, REPORT, and RPTXT files required to run inventory management functions.

3.4.9 Running inventory management

After you create racks, volumes, VRSSs, and data sets, you must run inventory management as follows:

1. Run vital record processing (VRSEL) before expiration and storage location management processing to identify which volumes to retain and where volumes should be moved according to their VRSSs.
2. Run storage location management (DSTORE) to set a destination location, and optionally assign shelf locations in storage locations to volumes that are being sent out of the removable media library.
3. Run expiration processing (EXPROC) to identify those volumes not required for vital records that are ready to expire. During expiration processing, release actions for volumes are noted.
4. Run extract data set processing (RPTXT) to a sequential data set that contains information from DFSMSrmm and use it as input to EDGRPTD, the DFSMSrmm reporting utility.
5. Back up the CDS and optionally the journal (BACKUP), and reset the journal data set when the CDS and journal are backed up using the EDGHSKP utility.

Figure 3-95 on page 151 shows sample JCL to perform inventory management and to copy the message data set to your job log.


```

//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
RMM SEARCHVOLUME VOLUME(*) ACTION(SCRATCH) LIMIT(*) OWNER(*)
/*
//STEP02 EXEC PGM=EDGHSKP,
//      PARM='VRSEL,DSTORE,EXPROC,RPTTEXT,BACKUP,DATEFORM(J)'
//SYSPRINT DD SYSOUT=*
//MESSAGE DD DSN=RMM.PROD.MSGS,DISP=SHR
//REPORT DD DSN=RMM.HSKP.VRSEL,DISP=SHR
//ACTIVITY DD DSN=RMM.HSKP.ACTIVITY,DISP=SHR
//REPTTEXT DD DSN=RMM.HSKP.EXTRACT,DISP=SHR
//BACKUP DD DSN=RMM.HSKP.BACKUP,DISP=(,CATLG,DELETE),
//      SPACE=(TRK,(150,10),RLSE),
//      DCB=(GDG,LRECL=9216,BLKSIZE=0,RECFM=VB)
//JRNLBKUP DD DSN=RMM.HSKP.JRNLBKUP,DISP=(,CATLG,DELETE),
//      SPACE=(TRK,(10,10),RLSE),
//      DCB=(GDG,LRECL=9248,BLKSIZE=0,RECFM=VB)
//*      *****
//STEP03 EXEC PGM=IEBGENER
//SYSPRINT DD DUMMY
//SYSUT1 DD DSN=RMM.PROD.MSGS,DISP=SHR
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY
//*      *****

```

Figure 3-95 Sample JCL to run inventory management

3.4.10 Confirming movements

Confirm your outstanding volume moves by using the RMM CHANGEVOLUME command. In DFSMSrmm, confirm each volume move, except for volumes moved to an IBM tape library, to confirm that you moved the volume.

You can confirm a move in two ways:

- For a single volume, use the RMM TSO CHANGEVOLUME subcommand as shown in Figure 3-96 with a VOLSER and the CONFIRMMOVE operand.

No further actions are required.

```
RMM CHANGEVOLUME vvvvvv CMOVE
```

Figure 3-96 RMM confirm move for a single volume

- Figure 3-97 on page 152 shows you a global confirmation of volume movements, specify an asterisk for the VOLSER and ALL for the from and to location.

In this case, you must re-run the storage location management processing to update the DFSMSrmm CDS.

```
RMM CHANGEVOLUME * CMOVE(ALL,ALL)
or
RMM CHANGEVOLUME * CMOVE(ALL,SANJOSE)
RMM CHANGEVOLUME * CMOVE(SANJOSE,ALL)
```

Figure 3-97 RMM global confirm move

3.4.11 Creating reports

You can create several types of reports using the DFSMSrmm report utilities, EDGRPTD and EDGAUD, and also using the DFSMSrmm report generator. DFSMSrmm provides standard reports and samples shipped in SAMPLIB. You can use the sample DFSMSrmm EDGJRPT JCL that invokes the EDGRRPTE exec to create the reports. The input to the reporting execs is the extended extract data set. Documentation about the reports is shipped in SYS1.SAMPLIB in the EDGDOC file.

Use the reports created to help you keep track of your DFSMSrmm environment, and compare these reports with those of your tape management system running in parallel. You can use DFSORTS's ICETOOL to write customized reports, or you can use the DFSMSrmm TSO subcommands to create lists of information defined to DFSMSrmm.

Creating reports with EDGRPTD

Use EDGRPTD to create inventory reports for auditing the physical contents of the installation media library and storage locations. You can also use EDGRPTD to create movement reports that list volumes to be moved from one location to another. Use these reports to make an inventory of your volumes and to identify volumes that need to be pulled and moved to other locations.

DFSMSrmm-supplied reports

DFSMSrmm provides REXX execs and JCL in SYS1.SAMPLIB member EDGJRPT that you can use as-is to create the following reports:

- ▶ Pull list for scratch tapes sorted by volume serial number
- ▶ Pull list for scratch tapes sorted by data set name
- ▶ List of all volumes in scratch status
- ▶ List of volumes that are ready to scratch
- ▶ Ready to scratch
- ▶ Inventory list by volume serial number
- ▶ Inventory list by data set name
- ▶ Inventory of data set including number of KB used
- ▶ Inventory of volume serial number by location
- ▶ Inventory of data set names by location
- ▶ Inventory of BIN numbers by location
- ▶ List of all data set names at loan location
- ▶ List of all volume serial numbers at loan location
- ▶ List of all multiple volume, multiple data set
- ▶ Movement report including the first data set name on the volume
- ▶ Movement report sorted by storage location bin number
- ▶ Movement report sorted by volume serial number
- ▶ Inventory list sorted by volume serial number including volume count
- ▶ Inventory of duplicate volume
- ▶ Inventory of Stacked Volumes by Percent Active
- ▶ Inventory of data sets by volume retention method

You can copy these reports and use them to create reports that are tailored for your installation.

Creating reports with EDGAUD

Use EDGAUD to create security and audit reports using either previously selected and sorted SMF records or raw SMF data. DFSMSrmm produces SMF records when you specify the SMFAUD or SMFSEC installation options in the PARMLIB member EDGRMMxx. DFSMSrmm uses the current subsystem startup option for the SMF record types and default report option unless you override them with the EDGAUD EXEC parameters.

Creating reports with the report generator

The report generator guides you through ISPF panels to define your reports and then it produces JCL. The generated JCL includes all the commands needed for producing the report as you specified. To obtain the report, you must submit the JCL. You can use this JCL for subsequent reports.

For detailed information about how to create a report, refer to chapter Chapter 14, “Report Generator” on page 613.

3.4.12 Running EDGINERS to initialize volumes automatically

With EDGINERS, volumes can be initialized and erased, based on the volume status and actions recorded in the CDS, or you can let your users or operator perform the processing. Refer to *DFSMSrmm Guide and Reference*, SC26-7404, for more information about operator tasks such as responding to initialization messages, tape mount messages, and using the LABEL procedure to request EDGINERS processing. Refer also to *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for a description of the EDGLABEL procedure provided by DFSMSrmm.

During automatic processing, the process requests each volume to be mounted, as with any batch job, because an operator reply is not required. When the volume is mounted, processing continues. If the operator cannot mount a volume, DFSMSrmm allows them to skip processing the current requested volume by replying to a WTOR.

As part of volume expiration processing, DFSMSrmm records when volumes need to be initialized or erased as they are released for return to scratch. If you use the EXEC parameters in your JCL to set up automatic processing, DFSMSrmm initializes or erases those volumes without librarian intervention. EDGINERS issues WTOR messages and MSGDISP requests to the operator and drive to get a volume mounted and demounted.

You can run EDGINERS automatic processing to initialize the volumes you are adding to DFSMSrmm. When you add scratch volumes to DFSMSrmm, use the RMM TSO subcommand operand, INIT(Y), on the ADDVOLUME command to ensure that volumes are initialized before they are available as scratch. Specify the initialization parameters on the PARM statement in your EDGINERS JCL. DFSMSrmm selects all volumes marked as requiring initialization and performs automatic processing. Figure 3-98 shows sample JCL for automatic processing.

```
//STEP01 EXEC PGM=EDGINERS,PARM='MEDIANAME(3592),VERIFY'  
//SYSPRINT DD SYSOUT=*  
//TAPE DD UNIT=(VT3590,,DEFER)
```

Figure 3-98 Sample JCL for EDGINERS automatic processing

This job initializes the volumes NS0010 through NS0014 added in Example 3-23 on page 111. The VERIFY parameter requests that DFSMSrmm ask the operator to remount each volume that has been successfully erased or labeled. The volumes are requested in reverse order, and the volume labels read to ensure that operator errors have not occurred (for example, a mismatch between the internal label and the external label).

3.4.13 Restoring the CDS

To restore the CDS, you first delete the existing CDS and then redefine it using Access Method Services (AMS) commands. The EDGBKUP utility is used to forward recover the CDS.

During the recovery you must stop all DFSMSrmm activities using the MVS command shown in Figure 3-99

```
MODIFY DFRMM,QUIESCE
```

Figure 3-99 Stop all tape activities

Important: In a multihost environment issuing a DFSMSrmm quiesce affects only the host on which you issue the command. If you want all hosts quiesced, you must issue the command on each host that is sharing the control data set.

Using the backups of the CDS and journal taken by the EDGHSKP utility during inventory management, you can recover the DFSMSrmm CDS at any time. Figure 3-100 shows how you can use the latest backup of the CDS and the current journal file to restore and forward recover to the point of failure.

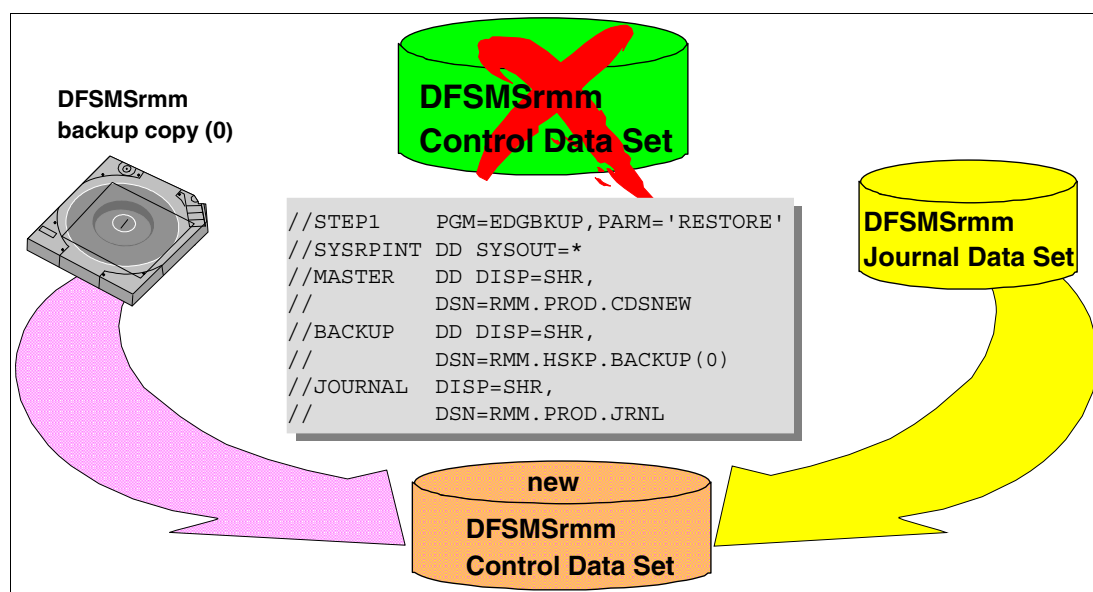


Figure 3-100 CDS recovering from latest backup

If the latest backup of the CDS is not available, you can use an older backup and the backups of the journal to recover. Figure 3-101 shows how you can use the previous backup generation of the CDS, the latest journal backup, and the current journal file to restore and forward recover to the point of failure.

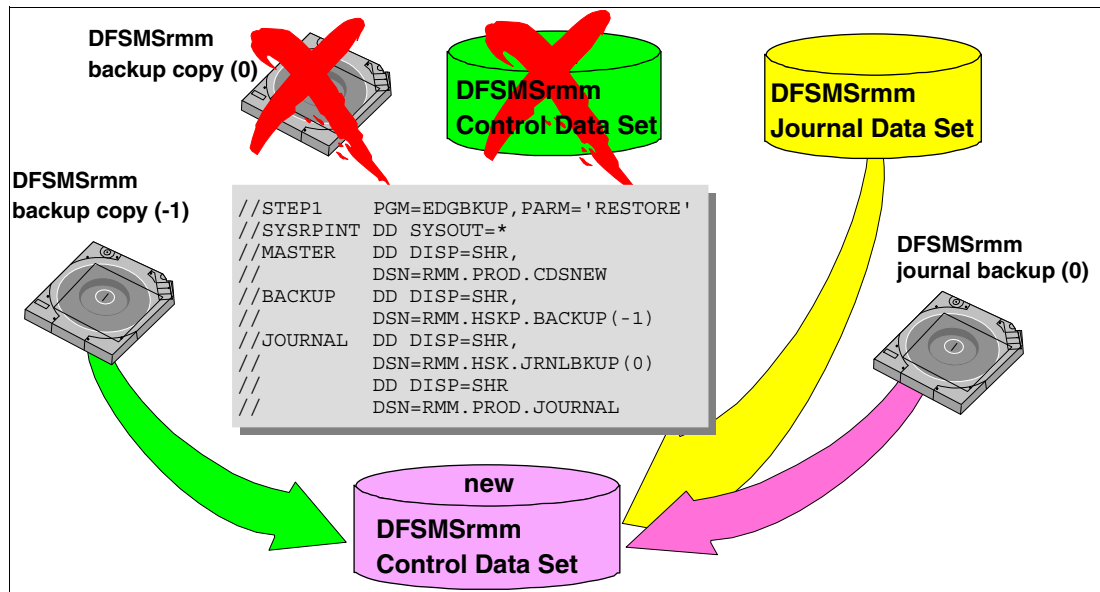


Figure 3-101 CDS recovering from backup-1

Using this process, you can recover from any CDS backup as long as you have the corresponding journal backups.

You can restore the DFSMSrmm CDS and forward recover the restored CDS with the journal. Use the JOURNAL DD statement to concatenate multiple journal data sets. If you forward recover using multiple journal data sets, ensure that the journal data sets are concatenated in the correct order in which changes were originally made.

When a backup is taken using DFSMSdss, by specifying BACKUP(DSS) for any backup being restored, a journal backup is also required for forward recovery. For the latest CDS backup, both the latest journal backup and the active journal must be used for forward recovery. In a remote recovery site, only the journal backup is required for successful forward recovery of the restored CDS.

In our example, we use a different data set name for the DFSMSrmm control data set than the original one. If you continue with the new restored CDS, you have to rename it or you must update the EDGRMMxx PARMLIB member to specify the new name in the DSNAME option.

Figure 3-102 on page 156 shows sample JCL to delete and define the CDS before the forward recovery.

```

//STEP01 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,UNIT=3390,VOL=SER=DFRMM4
//SYSIN DD *
  DELETE RMM.PROD.CDSNEW CL
  SET MAXCC = 0
/*
  DEFINE CLUSTER(NAME(RMM.PROD.CDSNEW) -
    FILE(MASTER) -
    FREESPACE(15 0) -
    KEYS(56 0) -
    REUSE -
    RECSZ(512 9216) -
    SHR(3 3) -
    KILOBYTES(4500 1500) -
    STORAGECLASS(gspace) -
    DATACLASS(EFKSDS) -
    VOLUMES(DFRMM4)) -
  DATA(NAME(RMM.PROD.CDSNEW.DATA) -
    BUFFERSPACE(829440) -
    CISZ(26624)) -
  INDEX(NAME(RMM.PROD.CDSNEW.INDEX) -
    CISZ(2048))
/*

```

Figure 3-102 Sample JCL to delete and define the CDS

Figure 3-103 shows sample JCL to forward recover the CDS for a backup with BACKUP(AMS). In this case you have to specify a JOURNAL DD statement with the actual journal.

```

//STEP03 EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=RMM.PROD.CDSNEW,DISP=SHR
//BACKUP DD DSN=RMM.HSKP.BACKUP(0),DISP=SHR
//JOURNAL DD DSN=RMM.PROD.JRNL,DISP=SHR

```

Figure 3-103 Sample JCL to forward recover the CDS (backup with AMS)

Figure 3-104 shows sample JCL to forward recover the CDS for a backup with BACKUP(DSS).

```

//STEP03 EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=RMM.PROD.CDSNEW,DISP=SHR
//BACKUP DD DSN=RMM.HSKP.BACKUP(0),DISP=SHR
//JOURNAL DD DSN=RMM.HSKP.JRNLBKUP(0),DISP=SHR
// DD DSN=RMM.PROD.JRNL,DISP=SHR

```

Figure 3-104 Sample JCL to forward recover the CDS (backup with DSS)

Note: If you are using DSS you must always specify the last backup copy of your journal and the actual journal.

You can check that restore and forward recovery were successful by displaying volume information. All volumes should have been initialized if the job shown in Figure 3-98 on page 153 ended without any errors.

When the CDS recovery is complete, restart DFSMSrmm using the MVS operator command shown in Figure 3-105.

```
MODIFY DFSRMM,M=xx
```

Figure 3-105 Restart DFSMSrmm

Important: In a multihost environment issuing a DFSMSrmm modify affects only the host on which you issue the command. If you want all hosts re-started, you must issue the command on each host that is sharing the control data set.

3.4.14 Verifying the CDS

Use the EDGUTIL utility to verify the contents of the CDS. You should perform the verification each time you recover your DFSMSrmm CDS. Figure 3-106 shows a sample JCL to verify the CDS.

```
//STEP01 EXEC PGM=EDGUTIL,PARM='VERIFY(ALL)'  
//SYSPRINT DD SYSOUT=*  
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDSNEW  
//SYSIN DD DUMMY
```

Figure 3-106 Sample JCL to verify the CDS contents

Attention: If you specify VERIFY(ALL) DFSMSrmm performs extra processing with the TCDB and library manager. That can be a very long running process. DFSMSrmm scans both the DFSMSrmm control data set and the TCDB sequentially to find DFSMSrmm volumes that are not in the TCDB, and TCDB volumes that are not in the DFSMSrmm control data set. DFSMSrmm also checks any volume that is found to be system-managed, either by definition to DFSMSrmm or retrieved from the TCDB, against the library manager database for an IBM automated tape library.

3.4.15 Testing implemented exits

If you use the exits presented in the following sections, you must test them.

EDGUX100

Use the DFSMSrmm EDGUX100 installation exit to:

- Ignore foreign or duplicate volumes and optionally provide an external volume serial number for use in messages intercepted and updated by.

- ▶ Use JCL special expiration dates to manage volumes by setting vital record specification management values to retain data sets
- ▶ Set any expiration date, including a zero value, for a data set
- ▶ Perform pooling management:
 - Manage scratch pools based on job name and data set name
 - Select a pool to be used for a non-specific tape volume request
 - Select a specific pool to be used for a non-specific tape volume request and request that the tape drive cartridge loader is disabled
- ▶ Pass vital record specification management value and pooling decisions to pre-ACS processing that can be used in ACS routines to assign storage group and management class
- ▶ Obtain information to modify input from DFSMSrmm disposition processing
- ▶ Request the recording of only the first file on a multifile volume
- ▶ Change the location for a volume
- ▶ Support the use of the storage group name for pooling for volumes that reside in manual tape libraries

You can use the supplied sample EDGUX100 exit to support the special expiration dates 98000 and 99000 without modifications to the sample code. If you plan to support other special dates using the exit and have modified it, you also need to test them. The results and the testing of the additional dates are the same as for the 99000 date.

To test use of the 99000 special date, create a new tape data set by specifying **EXPDT=99000**. Use the DFSMSrmm LISTDATASET command or the list data set dialog to display details of the data set that should include D99000 as the VRS management value.

To test the use of **EXPDT=98000** or **ACCODE=XCANORES** bypass tape management, first define a RACF FACILITY class resource (see Appendix A, “Security topics” on page 455) to allow the use of the ignore function to bypass DFSMSrmm recording. Whether you test to check that ignore is allowed or that ignore is not allowed, DFSMSrmm issues a message of explanation. To test the ignore function, DFSMSrmm must be running in either warning or protect mode.

EDGUX200

Use the DFSMSrmm EDGUX200 installation exit to:

- ▶ Return a volume to scratch status in an external inventory
- ▶ Prevent a volume from returning to scratch status
- ▶ Request that DFSMSrmm ignores data set information recorded for a volume

The sample EDGUX200 provides no specific function. Test the exit only if you plan to use it and have modified the supplied source code. The exit is intended for communication with software that has an inventory of scratch tapes. To test the exit, you need a tape volume that is in pending release status and is to return to scratch. Run inventory management EXPROC processing and then check that the external inventory has been correctly updated.

System-managed tape support by using the OAM exits

Test the following exits only if you have an IBM system managed tape library.

CBRUXENT	cartridge entry exit
CBRUXEJC	cartridge eject exit
CBRUXCUA	cartridge change use attributes exit

CBRUXVNL

volume not in library exit

DFSMSrmm provides sample exits that are ready to use, installed and active on your system, and do not require modification. However, you can customize the exits to meet your needs. CBRUXVNL is the exit you may want to consider customizing to meet your needs.

A simple set of tests can be used to test these exits:

1. Define a volume to DFSMSrmm for entry to the library using the command shown in Figure 3-107.

```
RMM ADDVOLUME volser LOCATION(ATL) STATUS(MASTER)
```

Figure 3-107 RMM ADDVOLUME subcommand

If you list or display the volume, it should be in MASTER status with a destination of ATL.

2. Enter the volume into the library.
 - a. To test CBRUXENT, list or display the volume in DFSMSrmm. It should now show a location of ATL and no destination. If you display the volume using ISMF, the volume should display as private.
 - b. To test CBRUXCUA, use ISMF to change the volume status to scratch. If DFSMSrmm is in protect mode, the change is rejected. If DFSMSrmm is in warning mode, the change is allowed, but a warning message is issued that DFSMSrmm only allows the change in warning mode. Messages are not issued in record or manual mode.
 - c. To test CBRUXEJC, eject the volume from the library. You can use any method for the ejection. When the eject is completed and the volume displays in DFSMSrmm, the volume should show as being in transit.
 - d. To test CBRUXVNL, ensure that the volume is in private or master status, and run a job to create or read a file on the tape.

You should see message EDG8124I on the console that lists DFSMSrmm information or message EDG8123D. After receiving EDG8124I, depending on the circumstances, either WTOR EDG8121D or EDG8122D appear on the console. Reply RETRY after the volume is entered into the library, and the job should run successfully.

Checking that all OAM exits are enabled

Use the MVS command in Figure 3-108 to verify that all OAM exits are enabled.

```

D SMS,OAM
F OAM,D,OAM,L=MHLRES7-Z
CBR1100I OAM status: 167
TAPE TOT  ONL  TOT  TOT  TOT  TOT  TOT  ONL  AVL  TOTAL
      LIB  LIB  AL  VL  VCL  ML  DRV  DRV  DRV  SCRTCH
        3   1   1   0   0   0   16   0   0     9
There are also 0 VTS distributed libraries defined.
CBRUXCUA processing ENABLED.
CBRUXEJC processing ENABLED.
CBRUXENT processing ENABLED.
CBRUXVNL processing ENABLED.
CBRUXSAE processing ENABLED.
CBRUXSAE processing ENABLED for STORE.
CBRUXSAE processing ENABLED for RETRIEVE.
CBRUXSAE processing ENABLED for QUERY.
CBRUXSAE processing ENABLED for CHANGE.
CBRUXSAE processing ENABLED for DELETE.
EDGTVEXT processing ENABLED.
Access Backup processing ACTIVE for UNREADABLE VOLUMES, using 1st
backup copy.
Access Backup processing ACTIVE for OFFLINE LIBRARIES, using 1st
backup copy.
Access Backup processing ACTIVE for NOT OPERATIONAL LIBRARIES, using
1st backup copy.
Access Backup processing ACTIVE for DB2 OBJECT TABLE ERRORS, using
1st backup copy.
Access Backup processing ACTIVE for LOST VOLUMES, using 1st backup
copy.
Access Backup processing ACTIVE for FILE SYSTEM ERRORS, using 1st
backup copy.
Diagnostic messages ACTIVE for OSREQFS. Limit= 10.
DB2 SSID: D9DG
XCF GROUP NAME: OAMTEST
XCF MEMBER NAME: OAM70
CBROAM: 70
OAM1 Parms: TIME=LOC MSG=EM UPD=Y QB=Y
             MOS=2000 OTIS=N LOB=A DP=A

```

Figure 3-108 *Displaying the OAM status*

3.4.16 Testing application use of tape

You should be familiar with the ways in which tape is used in your installation. Although we assume that everyone's use is the same, it is surprising how often we find new and devious uses of tape. To avoid discovering problems in production with DFSMSrmm, we suggest that you test the ways in which you use tapes today to ensure that DFSMSrmm supports them as expected. For example, you may use BLP (perhaps to scratch tapes) or non-labeled tapes, or you may have key business functions that depend on foreign tape processing.

Use tests that you have already in place for your production tape usage under DFSMSrmm to validate that they are supported and that you know which actions may be required to support them. For example, RACF profiles are required to use NL output to scratch and to process tapes that are to be ignored by DFSMSrmm.

NL scratch tapes

DFSMSrmm normally supports only standard label scratch tapes for non-specific NL tapes, and relabels them at time of use to NL and standard label at return to scratch. To use NL

scratch tapes you can use the system tape exit IFG019VM. DFSMSrmm now ships a sample IFG019VM exit as member EDG019VM in SAMPLIB. The sample exit checks for a supported request and issued a WTOR to the operator to obtain the volume serial number. Testing should ensure that it does support the installation needs for non-specific NL tape.

Interface between DFSMSShsm and DFSMSrmm

You should test the built-in interface between DFSMSShsm and DFSMSrmm. Before you start this kind of test, ensure that DFSMSShsm has the correct authority for the STGADMIN resources in RACF CLASS FACILITY. If you have a release of DFSMSrmm earlier than OS/390 Version 1 Release 4.0 you must define the use of the ARCTVEXT installation exit in the DFSMSShsm PARMLIB member. You can test the function of the EDGTVEXT interface, the use of the ARCTVEXT exit EDGTVEXT interface, or the use of the ARCTVEXT exit by deleting a volume from the DFSMSShsm tape volume pool using the HSM DELVOL command.

The TVEXTPURGE PARMLIB operand specifies how DFSMSrmm processes volumes to be purged by callers of EDGTVEXT or EDGDFHSM. It is the interface between DFSMSShsm and DFSMSrmm.

There are three options that can be specified for TVEXTPURGE.

Where:

NONE	DFSMSrmm takes no action for volumes to be purged.
RELEASE	DFSMSrmm releases volumes to be purged according to the release actions set for the volume. You must run expiration processing to return a volume to scratch status.
EXPIRE	Use the EXPIRE option to set the volume expiration date to the current date for volumes to be purged.

The recommended option is TVEXTPURGE(RELEASE).

Suggested DFSMSShsm system option parameters.

Use the DFSMSShsm system option parameters shown in Figure 3-109.

SETSYS EXITOFF(ARCTVEXT) -	
SELECTVOLUME(SCRATCH) -	-
TAPEDELETION(SCRATCH) -	-
TAPESECURITY(EXPIRATION) -	-
PARTIALTAPE(MARKFULL)	

Figure 3-109 Suggested DFSMSShsm system option parameters

Note: Specify EXITOFF(ARCTVEXT) if DFSMSrmm is your only tape management product because DFSMSShsm always calls the DFSMSrmm programming interface EDGTVEXT.

3.5 Education

Consider which groups of users may be affected by the initial introduction of DFSMSrmm for testing. Your colleagues, the operators, and possibly the tape librarian needs to learn about DFSMSrmm at some stage. At this stage, provide a minimum level of education so that any

potential impact on your production system can be minimized by ensuring that affected users are aware of changes.

| 3.6 New operator procedures

When you start to run DFSMSrmm, system operators are most likely affected. DFSMSrmm can cause the following changes:

- ▶ Mount and fetch messages are updated.
- ▶ Scratch pool names may be different and are included in the MVS mount messages.
- ▶ Tape drive displays are updated with rack number and pool information.
- ▶ DFSMSrmm issues WTORs that require operator action and decisions.

In addition, you may want the operator to be involved in starting and stopping DFSMSrmm.

Refer to *DFSMSrmm Guide and Reference*, SC26-7404, for details about messages and activities that affect the operator.

DFSMSrmm retention methods

This chapter explains how can use the different retention methods DFSMSrmm provides to retain your DFSMSrmm managed tape volumes..

The objectives of this chapter are to:

- ▶ Overview
- ▶ Specifying a retention method
 - Expiration date
 - Retention date
- ▶ Using the EDGRMMnn parmlib option RETENTIONMETHOD
- ▶ EDG_EXIT100 retention method support
- ▶ Subcommands for RETENTIONMETHOD parameters
- ▶ Syntax format of the RETENTIONMETHOD operand
- ▶ Using the SEARCHVOLUME subcommand

By the end of this chapter, you will understand the different retention methods you can use, how you can select different retention methods and the use of the new subcommand operands.

4.1 Overview

Retention methods for tapes Among the important decisions to be made when using DFSMSrmm is how to retain tape data sets and for how long. You might want to retain a given data set for a specific period of time after it is created, or retain it based on some event (for example, while the data set is catalogued), or retain it permanently. With z/OS V1R13, DFSMSrmm offers two retention methods, EXPDT and VRSEL, for retention of your volumes and data sets on your volumes. For each volume set, you can specify whether the volumes and data sets in that volume set should be managed by the expiration date or by VRSEL retention vital record specification (VRS) policies.

Every volume has a retention method, either EXPDT or VRSEL. The default retention method specified in parmlib is used, unless one is assigned by the EDG_EXIT100 installation exit, by the RMM TSO ADDVOLUME or CHANGEVOLUME subcommand.

When data is created the storage administrator can select an appropriate retention method for a volume set. The basic choice is either:

VRSEL

The VRSEL retention method can retain a volume beyond its expiration date by using a vital record specification to define a retention policy. The VRSEL retention method can also be used for setting a movement policy. Volumes and data sets managed by this retention method are subject to frequent VRSEL processing. In every run of VRSEL processing, the matching of your data sets and volumes to the vital record policies is done again, so the retention and movement management can change from one inventory management run to another.

EXPDT

The EXPDT retention method is based on the expiration date and avoids VRSEL processing. As a result, the retention information can be known when a tape data set is created. You can also manually override an expiration date to release a volume before the original expiration date is reached. The movement of the volumes must be managed manually.

4.2 Specifying a retention method

The EDGRMMxx parmlib member has a new operand to set the system-wide retention method for new tape volumes or tape volume sets created during Open/Close/End-of-Volume (O/C/EOV) processing, and for tape volumes added to the DFSMSrmm Control Data Set (CDS).

4.2.1 Expiration date

One important parameter for both retention methods is the expiration date of the data set and the volume. The data set expiration date or retention period is determined when a new tape data set is created. The expiration date or retention period can be specified at multiple levels:

1. The default retention period RETPD specified in the DFSMSrmm parmlib member
2. The EXPDT or RETPD in the DFSMS data class if the data set is associated with a DFSMS data class
3. The JCL DD statement, using the EXPDT or RETPD keywords
4. The DFSMSrmm installation exit EDG_EXIT100.

Note: When you change the expiration date for a data set record representing one part of a multivolume data set on volumes managed by the EXPDT retention method, DFSM-Srmm updates the expiration date and time for all the data set records for the data set.

4.2.2 Retention date

z/OS V1R13 DFSMSrmm provides an enhancements to make tape management easier and to improve administrator productivity by providing the following:

- A retention date in the volume
With this enhancement, the retention date is displayed instead of the expiration date in the search results list, if the volume or data set is VRS retained.
- A data set search result
When a resource is retained by VRS, the search results list for volumes or data sets might show retained resources with an expiration date that is already passed. The storage administrator can more easily determine from the search results list why the volume is retained, without viewing the volume and data set details.

4.2.3 Using the EDGRMMnn parmlib option RETENTIONMETHOD

Use this operand to set the system-wide retention method default for new tape volume sets. New tape volume sets may be created during Open/Close/End-of-Volume (O/C/EOV) processing, or through DFSMSrmm commands. A tape volume set may be a multi-volume set, or a single tape volume. RETENTIONMETHOD can be abbreviated as RM as shown in Figure 4-1.

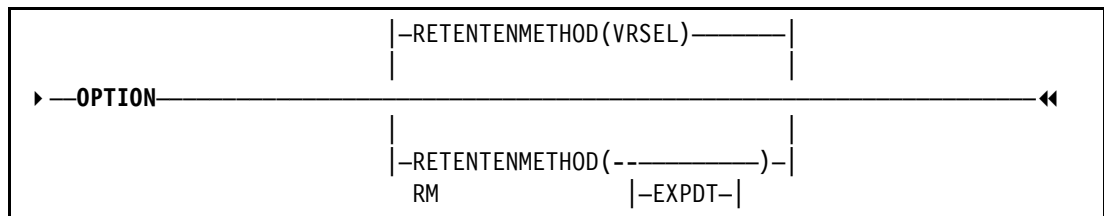


Figure 4-1 EDGRMMnn RETENTIONMETHOD operand

Where:

VRSEL

The VRSEL retention method can retain a volume beyond its expiration date by using a vital record specification to define a retention policy. The VRSEL retention method can also be used for setting a movement policy. Volumes and data sets managed by this retention method are subject to frequent VRSEL processing. In every run of VRSEL processing, the matching of your data sets and volumes to the vital record policies is done again, so the retention and movement management can change from one inventory management run to another.

This is the default.

EXPDT

The EXPDT retention method is based on the expiration date and avoids VRSEL processing. As a result, the retention information can be known when a tape data set is created. You can also manually override an expiration date to release a volume before the

original expiration date is reached. The movement of the volumes must be managed manually.

Use the RMM TSO LISTCONTROL subcommand with the OPTION operand to list the current setting. Figure 4-2 shows you the result of the LISTCONTROL subcommand that includes the new Retention method information.

```

System options:
PARMLIB Suffix = 02
Operating mode = P      Retention period: Default = 0      Maximum = NOLIMIT
                          Catalog = 6      hours
                          Retention method: Method = VRSEL
Control data set name   = RMM.CONTROL.DSET
Journal file data set name = RMM.JOURNAL.DSET
Journal threshold       = 75%      Journal transaction = NO
Catalog SYSID          = Notset
Scratch procedure name  = EDGXPROC
Backup procedure name   = EDGCDSBK
IPL date check = N      Date format = J      RACF support = N
SMF audit = 42          SMF security = 42      CDS id = SC70
MAXHOLD value = 100     Lines per page = 54      System ID = SC70
BLP = RMM              TVEXT purge = RELEASE Notify = N
                          days = 0
Uncatalog = Y          VRS job name = 2      Message case = M
MASTER overwrite= USER Accounting = J      VRS selection = NEW
VRS change = INFO      GDG duplicate = BUMP   GDG cycle by = GENERATION
VRSMIN action = INFO    VRSMIN count = 1
VRSDROP action = INFO    VRSDROP count = 0      percent = 10
VRSRETAIN action= INFO   VRSRETAIN count= 0      percent = 80
EXPDTDROP action= INFO   EXPDTDROP count= 0      percent = 10
Disp DD name = DISPDD   Disp msg ID = EDG4054I
Retain by = SET          Move by = SET      CMDAUTH Owner = NO
PREACS = NO             SMSACS = NO      CMDAUTH Dsn = YES
Reuse bin = CONFIRMMOVE Media name = 3480
                          Local tasks = 10

PDA: ON
Block count = 0      Block size = 0      Log = ON
SMSTAPE:
Update scratch = YES      Update command = YES      Update exits = YES
Purge = ASIS

Client/Server:
Subsystem type = STANDARD Port = 0
Server          Server tasks = 0
host name =
IP address =

```

Figure 4-2 LISTCONTROL OPTION output

4.2.4 EDG_EXIT100 retention method support

You can use the EDG_EXIT100 installation exit to set the retention method to be used for a new tape data. When you create a new tape volume or tape volume set, or rewrite an existing set from the first file, you can override the system default retention method.

With z/OS V1R13, every volume has a retention method, either EXPDT or VRSEL. The default retention method specified in parmlib is used, unless one is assigned by the EDG_EXIT100 installation exit or by the ADDVOLUME or CHANGEVOLUME command. The tape copy application can use the EDGPL100 macro to invoke the installation exit EDG_EXIT100 to copy the data set attributes from the source data set to the copy data set during OPEN processing. EDG_EXIT100 can notify DFSMSrmm that the data set being created is being copied from another. During OPEN processing the exit can identify the source data set from which DFSMSrmm will obtain all existing data set attributes, which will be used for the target data set. DFSMSrmm EOV processing ensures that the attributes are copied to all target data set records when the output data set becomes a multivolume data set.

Tailor the Sample EDGUX100 Exit Module

Update the sample EDGUX100 exit module based on your retention requirements. You must perform these tasks:

1. Define the system default retention method. Refer to Parmlib Member EDGRMMxx OPTION Command
2. Define the DFSMSrmm default and maximum retention periods. OPTION RETPD and MAXRETPD
3. Update the sample EDGUX100 exit module based on your retention requirements:
 - e. Copy the sample EDGUX100 exit module and use the copy as a base for your exit module.
 - f. Update the exit module. Perform your processing only when the PL100_CAN_RETMET bit is set to B'1'. Set PL100_RETENTIONMETHOD to the value required, and ensure the PL100_SET_RETMET bit is set to B'1'. If you do not set a retention method, the system default retention method is used.
 - g. Make any other changes required, such as setting or clearing the EXPDT. The sample EDGUX100 exit module includes an example of setting the retention method. To use the sample EDGUX100 exit module for this function, modify the table as shown in Figure 99. The order in which the table entries are listed is important because the exit scans the table until it finds the first entry where the job name, data set name and program name masks match the current request. You can change the priority of matching by changing the order of the table entries.

The sample EDGUX100 exit module includes an example of setting the retention method. To use the sample EDGUX100 exit module for this function, modify the table as shown in Figure 4-3 on page 168. The order in which the table entries are listed is important because the exit scans the table until it finds the first entry where the job name, data set name and program name masks match the current request. You can change the priority of matching by changing the order of the table entries.

RMTAB	DS OF	START OF RM TABLE
	SPACE 1	
DC	CL8'*	JOBNAME
DC	CL44'RMMUSER.RMVRSEL.*'	DATA SET NAME
DC	CL8'*	PROGRAM NAME
DC	AL1(PL100_RM_VRSEL)	RETENTION METHOD VRSEL
DC	XL3'00'	RESERVED
	SPACE 1	
DC	CL8'*	JOBNAME
DC	CL44'RMMUSER.RMEXPDT.*'	DATA SET NAME
DC	CL8'*	PROGRAM NAME
DC	AL1(PL100_RM_EXPDT)	RETENTION METHOD EXPDT
DC	XL3'00'	RESERVED
	SPACE 1	
DC	CL8'RM END'	END OF RM TABLE MARKER

Figure 4-3 Sample RETENTIONMETHOD selection table

Where:

JOBNAME

One-to-eight alphanumeric or national characters including % and *.

% can be used to ignore a positional character in the job name.

* can be used to ignore all remaining characters in the job name. A jobname of * means that the entry applies to all jobs.

DATA SET NAME

Can be up to forty-four characters, following z/OS data set naming conventions, including % and *.

% can be used to ignore a positional character in the data set name.

* can be used to ignore all remaining characters in the data set name. A data set name of * means that the entry applies to all data sets.

The use of the character * is not the same as in the generic data set names supported by DFSMSrmm for vital records specifications and search data set masks. Here the * works like the characters *.*might in a generic data set name mask.

PROGRAM NAME

A value up to eight alphanumeric character including % and *.

% can be used to ignore a positional character in the program name.

* can be used to ignore all remaining characters in the program name. A program name of * means that the entry applies to all programs.

retention method One of:

PL100_RM_VRSEL to assign the retention method VRSEL

PL100_RM_EXPDT to assign the retention method EXPDT

To compile and link the updated EDGUX100 user exit use the JCL shown in Figure 4-4 on page 169.

```
//ASMCL  PROC SM=,LM=
//C      EXEC PGM=ASMA90,PARM='NODECK,XREF(SHORT),LINECOUNT(58)'
//SYSLIN DD DSN=RMM.ADDONS.OBJ(&LM),DISP=SHR
//SYSLIB DD DSN=SYS1.MACLIB,DISP=SHR
//       DD DSN=SYS1.MODGEN,DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD UNIT=SYSDA,SPACE=(7040,400)
//SYSUT2 DD UNIT=SYSDA,SPACE=(3520,400)
//SYSUT3 DD UNIT=SYSDA,SPACE=(3520,400)
//SYSIN  DD DSN=RMM.ADDONS.SOURCE(&SM),DISP=SHR
//L      EXEC PGM=HEWL,PARM='LIST,MAP,XREF,RENT,REUS,',COND=(4,LT,C)
//SYSLMOD DD DSN=SYS1.SANDBOX.MIGLIB(&LM),DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD UNIT=SYSDA,SPACE=(3520,400)
//SYSLIN DD DSN=RMM.ADDONS.OBJ(&LM),DISP=SHR
//       PEND
//COMPLNKO EXEC ASMCL,LM=EDGUX100,SM=EDGUX100
```

Figure 4-4 Compile and link updated EDGUX100 user exit

You can use the SETPROG EXIT command to control exits that have been defined to the dynamic exits facility. To activate or change the current used EDGUX100 user exit you have to use the command shown in Figure 4-5 this will replace the old sequence you have to use before z/OS V1.12.

```
SETPROG EXIT,REFRESH,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100
```

Figure 4-5 SETPROG command syntax to refresh an dynamic exit

Before z/OS V1.12 the correct sequence to replace an existing exit was as following you can see in Figure 4-6.

```
MODIFY DFRMM,QUIESCE

SETPROG EXIT,DELETE,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100

SETPROG EXIT,ADD,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100

MODIFY DFRMM,M=nn
```

Figure 4-6 SETPROG command syntax sequence before z/OS V1.12

Where:

ADD	Adds an exit routine to an exit.
REPLACE	Replaces an exit routine for an exit.
DELETE	Deletes an exit routine from an exit.
EXITNAME	The 1-16 character name of the exit.
MODNAME	The 1-8 character name of the exit routine. If DSNAME is not specified, the system tries to locate the exit routine using the LPA, the LNKLIST concatenation, and the nucleus.

Tip: DFSMSrmm provides a second sample for EDGUX100. This sample is called EDGCVRSX. It is different from the EDGUX100 sample because the special date, retention method, VRSELEXCLUDE, and pooling function is table driven and you can change the table dynamically. Refer to SAMPLIB member EDGCMM01 and the IBM Red book DFSMSrmm Primer for documentation on using EDGCVRSX for EDGUX100.

4.2.5 Subcommands for RETENTIONMETHOD parameters

You can use the TSO RMM ADDVOLUME or CHANGEVOLUME subcommands with the RETENTIONMETHOD operand to set the retention method for a tape volume or tape volume set. Specify this operand for the first volume in a multivolume sequence. All other volumes added to the set assume the same retention method.

Once a retention method is defined for a non-scratch volume, it is not overridden to the system-wide default during OPEN output processing, but can be changed by installation exit EDG_EXIT100. Volumes in a set always assume the retention method of the first volume in the set.

Specify EXPDT to set the retention method for a tape volume set to be based on EXPDT. Data sets and volumes managed by this retention method are never processed by VRSEL inventory management.

Specify VRSEL to set the retention method for a tape volume set to be VRSEL. This option enables DFSMSrmm inventory management to attempt to match data sets and volumes to vital record specifications, and if a match is found, to determine if the data set or volumes are to be retained by VRS.

4.2.6 Syntax format of the RETENTIONMETHOD operand

Use this operand to set the retention method for a tape volume set. Specify this operand for the first volume in a multivolume sequence. All other volumes added to the set assume the same retention method. Once a retention method is defined for a non-scratch volume, it is not overridden to the system-wide default during OPEN output processing, but can be changed by installation exit EDG_EXIT100. Volumes in a set always assume the retention method of the first volume in the set. You can see the correct syntax in Figure 4-7 on page 171 that can be used to set the retention method to EXPDT or VRSEL.

Important:

Authorization requires either:

- Based on STGADMIN.EDG.MASTER access, if the resource below is not defined
- UPDATE access to STGADMIN.EDG.CV.RM to allow any volume to be updated.

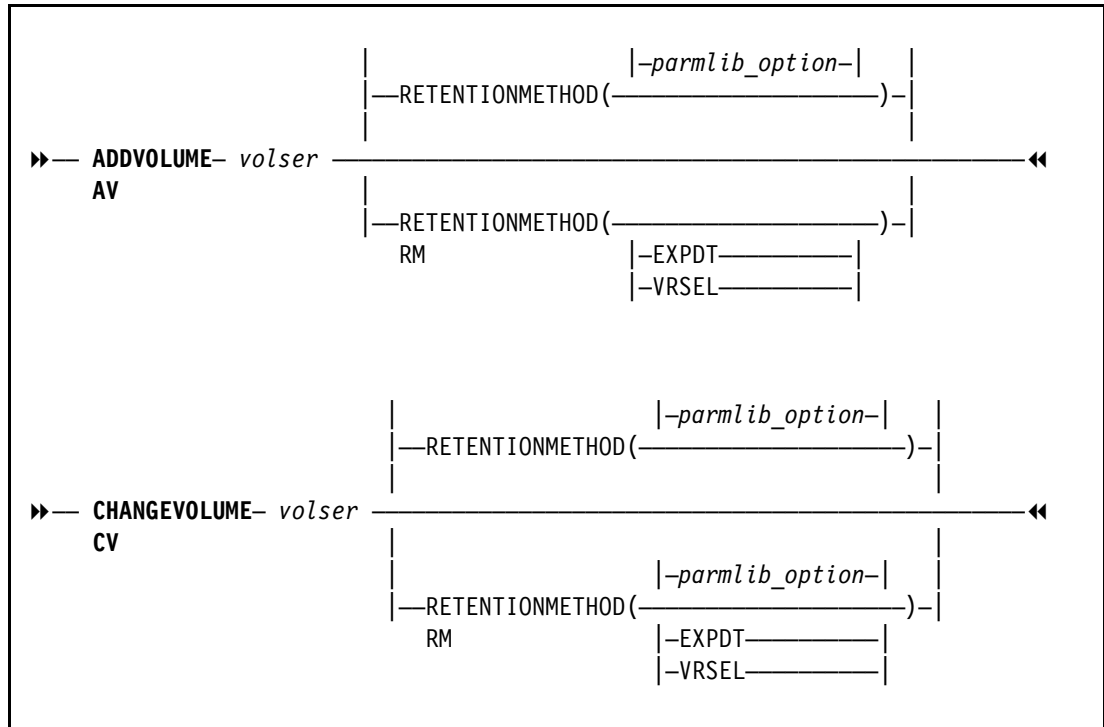


Figure 4-7 RETENTIONMETHOD operand command syntax

Where:

EXPDT

to set the retention method for a tape volume set to be based on EXPDT. Data sets and volumes managed by this retention method are never processed by VRSEL inventory management.

VRSEL

to set the retention method for a tape volume set to be VRSEL. This option enables DFSMSrmm inventory management to attempt to match data sets and volumes to vital record specifications, and if a match is found, to determine if the data set or volumes are to be retained by VRS.

Note: When you manually define or change a volume that is a start of a volume set, you can specify the retention method for the volume set. All other volumes added to the set assume the same retention method. If you do not specify a retention method, DFSMSrmm will use the default retention method specified by the RETENTIONMETHOD option in EDGRMMxx.

4.2.7 Using the SEARCHVOLUME subcommand

Using the SEARCHVOLUME subcommand with the RETENTIONMETHOD operand to specifies a list limited to volumes that have a specified retention method. Specify EXPDT to select volumes with the EXPDT retention method. Specify VRSEL to select volumes with the VRSEL retention method. The new RETENTIONMETHOD operand syntax is shown in Figure 4-8 on page 172.

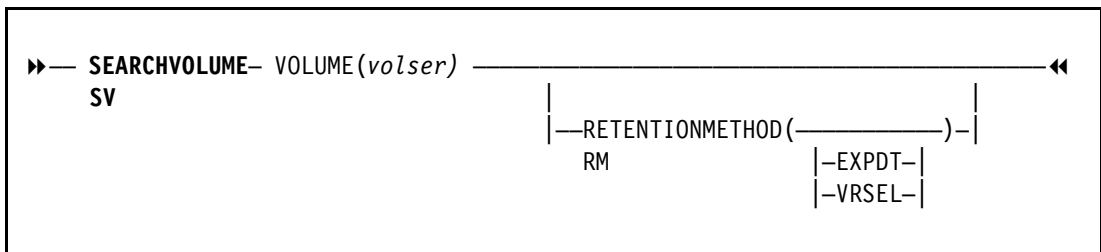


Figure 4-8 RETENTIONMETHOD operand command syntax

Where:

- EXPDT** to set the retention method for a tape volume set to be based on EXPDT. Data sets and volumes managed by this retention method are never processed by VRSEL inventory management.
- VRSEL** to set the retention method for a tape volume set to be VRSEL. This option enables DFSMSrmm inventory management to attempt to match data sets and volumes to vital record specifications, and if a match is found, to determine if the data set or volumes are to be retained by VRS.

Example 1:

Figure 4-9 shows you how you can use the RMM SEARCHVOLUME subcommand to get a list of all volumes owned by user SCHLUM.

Note: If you do not specify the RETENTIONMETHOD operand DFSMSrmm searches all volumes, regardless of the retention method assigned to a volume.

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(SCHLUM)
```

Figure 4-9 SEARCHVOLUME without the RETENTIONMETHOD operand

You get the list of all 6 volumes the owner SCHLUM are currently owned as you can see in Figure 4-10.

Volume	Owner	Rack	Assigned date	Expiration date	Location	Dsets	St	Act	Dest.
VT0008	SCHLUM		2011/324	2011/333	VTFM001	1	M		
VT0013	SCHLUM		2011/324	2011/333	VTFM001	1	M		
VT0015	SCHLUM		2011/324	2011/333	VTFM001	1	M		
VT0016	SCHLUM		2011/324	2011/333	VTFM001	1	M		
VT0017	SCHLUM		2011/325	2011/334	VTFM001	1	M		
VT0021	SCHLUM		2011/325	2011/334	VTFM001	1	M		
EDG3012I	6		ENTRIES LISTED						

Figure 4-10 SEARCHVOLUME result without using the RETENTIONMETHOD operand

Example 2:

Figure 4-11 shows you how you can use the RMM SEARCHVOLUME subcommand with the RETENTENMETHOD(EXPDT) operand to get only a list of all volumes owned by user SCHLUM but have a retention method of EXPDT.

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(SCHLUM) RETENTIONMETHOD(EXPDT)
```

Figure 4-11 SEARCHVOLUME using the RETENTIONMETHOD(EXPDT) operand

You get the list of volumes assigned to owner SCHLUM and have a retention method of EXPDT set in the volume record as you can see in Figure 4-12.

Volume	Owner	Rack	Assigned date	Expiration date	Location	Dsets	St	Act	Dest.
VT0008	SCHLUM		2011/324	2011/333	VTFM001	1	M		
VT0013	SCHLUM		2011/324	2011/333	VTFM001	1	M		
EDG3012I	2		ENTRIES LISTED						

Figure 4-12 SEARCHVOLUME result using the RETENTIONMETHOD(EXPDT) operand

Now you can use the LISTVOLUME subcommand to get all the details of one of the listed volumes as shown in Figure 4-13.

```
RMM LISTVOLUME VT0008
```

Figure 4-13 LISTVOLUME subcommand

Figure 4-14 on page 174 shows you the result of the LISTVOLUME subcommand.

```

Volume information:
Volume = VT0008  VOL1 =          Rack =          Owner = SCHLUM
Type = LOGICAL      Stacked count = 0      Jobname = TEST9999
Worldwide ID =          WORM = N
Creation: Date = 2009/222      Time = 12:49:45      System ID = SC70
Assign:   Date = 2011/324      Time = 16:22:36      System ID = SC70
                                           User ID = SCHLUM
Expiration date = 2011/333      Original = 2011/333
      set by = OCE_JFCB
Retention date =          Set retained = NO
Retention method= EXPDT
      set by = OCE_DEF
Data set name = RMM.F13002.ON.TWO.VOLUMES.AND.CATLG
Volume status:          Hold = N      File 1 Data set seq = 1
Status = MASTER      Availability =          Label = SL
Current label version =          Required label version =
Media information: VTFM
Density = IDRC Type = VT3590G2      Format = VT3590G2      Compaction = YES
Special attributes = NONE      Vendor =
Encryption Key Labels:          Method:
1=
2=
Action on release:
Scratch immediate = N      Expiry date ignore = N
Scratch = Y      Replace = N      Return = N      Init = N      Erase = N      Notify = N
Actions pending:
Scratch = N      Replace = N      Return = N      Init = N      Erase = N      Notify = N
Storage group =
Loan location =          Account = 999,P0K
Old loan loc =
Description =
Security class =          Description =

```

Figure 4-14 LISTVOLUME VT008 result

Where:

Retention method: The Retention method field specifies which retention processing is to be used. Retention method can only be changed for the first volume in a multi-volume sequence and will be assumed for all volumes of the set. Retention method cannot be changed for a SCRATCH volume, unless you specify the STATUS field as well.

EXPDT

The retention method for a tape volume set is only based on the expiration date. Data sets and volumes managed by this retention method are never processed by VRSEL inventory management.

Set by

This field shows you where the retention method is coming from.

UNDEFINED

Not set.

CMD

Retention method set by TSO subcommand.

CMD_DEF

Default Retention method applied during subcommand processing.

OCE_DEF

Default Retention method applied during tape recording.

OCE_EXIT	EDG_EXIT100 set Retention method during tape recording.
LCS_DEF	Default Retention method applied for system managed tapes when called from OAM installation exits.
CNVT	Retention method set during conversion.
EXPORT_DEF	Default Retention method applied during export processing.
INERS_DE	Default Retention method applied during tape initialization.

Example 3:

Figure 4-15 shows you how you can use the RMM SEARCHVOLUME subcommand with the RETENTENMETHOD(VRSEL) operand to get only a list of all volumes owned by user SCHLUM but have a retention method of VRSEL.

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(SCHLUM) RETENTIONMETHOD(VRSEL)
```

Figure 4-15 SEARCHVOLUME using the RETENTIONMETHOD(VRSEL) operand

You get the list of volumes assigned to owner SCHLUM and have a retention method of VRSEL set in the volume record as you can see in Figure 4-16.

Volume	Owner	Rack	Assigned date	Expiration date	Location	Dsets	St	Act	Dest.
VT0015	SCHLUM		2011/324	2011/333	VTFM001	1	M		
VT0016	SCHLUM		2011/324	2011/333	VTFM001	1	M		
VT0017	SCHLUM		2011/325	2011/334	VTFM001	1	M		
VT0021	SCHLUM		2011/325	2011/334	VTFM001	1	M		
SCHLUM									
EDG3012I	4		ENTRIES LISTED						

Figure 4-16 SEARCHVOLUME result using the RETENTIONMETHOD(VRSEL) operand

Now you can use the LISTVOLUME subcommand to get all the details of one of the listed volumes as shown in Figure 4-18 on page 176.

```
RMM LISTVOLUME VT0021
```

Figure 4-17 LISTVOLUME subcommand

Figure 4-14 on page 174 shows you the result of the LISTVOLUME subcommand.

```

Volume information:
Volume = VT0021  VOL1 =          Rack =          Owner = SCHLUM
Type = LOGICAL          Stacked count = 0          Jobname = TEST9999
Worldwide ID =          WORM = N
Creation: Date = 2009/222    Time = 12:49:45    System ID = SC70
Assign:   Date = 2011/325    Time = 10:59:17    System ID = SC70
                                   User ID = SCHLUM
Expiration date = 2011/334    Original = 2011/334
      set by = OCE_JFCB
Retention date =          Set retained = NO
Retention method= VRSEL
      set by = OCE_DEF
Data set name = RMM.F13002.ON.TWO.VOLUMES.AND.CATLG
Volume status:          Hold = N    File 1 Data set seq = 1
Status = MASTER    Availability =          Label = SL
Current label version =          Required label version =
Media information: VTFM
Density = IDRC Type = VT3590G2    Format = VT3590G2    Compaction = YES
Special attributes = NONE    Vendor =
Encryption Key Labels:          Method:
1=
2=
Action on release:
Scratch immediate = N    Expiry date ignore = N
Scratch = Y    Replace = N    Return = N    Init = N    Erase = N    Notify = N
Actions pending:
Scratch = N    Replace = N    Return = N    Init = N    Erase = N    Notify = N
Storage group =
Loan location =          Account = 999,P0K
Old loan loc =
Description =
Security class =          Description =

```

Figure 4-18 LISTVOLUME VT008 result

Where:

Retention method: The Retention method field specifies which retention processing is to be used. Retention method can only be changed for the first volume in a multi-volume sequence and will be assumed for all volumes of the set. Retention method cannot be changed for a SCRATCH volume, unless you specify the STATUS field as well.

VRSEL

DFSMSrmm inventory management attempts to match data sets and volumes to VRSEs, and if a match is found, to determine if the data sets or volumes are to be retained by VRS.

Set by

This field shows you where the retention method is coming from.

UNDEFINED

Not set.

CMD

Retention method set by TSO subcommand.

CMD_DEF

Default Retention method applied during subcommand processing.

OCE_DEF

Default Retention method applied during tape recording.

OCE_EXIT

EDG_EXIT100 set Retention method during tape recording.

LCS_DEF	Default Retention method applied for system managed tapes when called from OAM installation exits.
CNVT	Retention method set during conversion.
EXPORT_DEF	Default Retention method applied during export processing.
INERS_DE	Default Retention method applied during tape initialization.



Exclude Data Sets from VRSEL processing

This chapter explains how you can exclude specific data sets from DFSMSrmm VRSEL processing as they are created or rewritten.

The objectives of this chapter are to:

- ▶ Overview
- ▶ Subcommands for VRSELEXCLUDE parameters
- ▶ Syntax format of the VRSELEXCLUDE operand Overview
- ▶ Using EDG_EXIT100 to excluding Data Sets from VRSEL support
- ▶ Using the SEARCHDATASET subcommand
- ▶ Inventory Management VRSEL/EXPROC Processing

By the end of this chapter, you will understand how you can exclude data sets from vital record selection processing by using different exclusion methods.

5.1 Overview

You can use the EDG_EXIT100 installation exit to exclude specific data sets from DFSMSrmm VRSEL processing as they are created or rewritten. You can specify this for any data set, but DFSMSrmm ignores the request unless the data set is on a volume that is managed by the VRSEL retention method. The data set VRSELEXCLUDE attribute is set for all data sets on volumes managed by the EXPDT retention method and is not affected by this support.

When DFSMSrmm excludes a data set from VRSEL processing, it ensures that the data set vital record attribute is reset and the retention date is set to current date. The matching VRS information is left unchanged.

5.2 Subcommands for VRSELEXCLUDE parameters

All data in the DFSMSrmm inventory is managed by dynamic VRS policies. With z/OS V1R13 DFSMSrmm provides a new operand VRSELEXCLUDE for the RMM CHANGEDATASET and SEARCHDATASET subcommands.

5.2.1 Syntax format of the VRSELEXCLUDE operand

Use this operand to override DFSMSrmm VRSEL processing. You can specify this for any data set on a volume managed by the VRSEL retention method. If VRSELEXCLUDE(YES) is specified for a data set already retained as a vital record, its vital record attribute is reset and the retention date set to the current date. The data set VRSELEXCLUDE attribute is set to YES for all data sets on volumes managed by the EXPDT retention method. For more information about the EXPDT retention method refer to chapter Chapter 4, “DFSMSrmm retention methods” on page 163.

When a data set spans volumes, you should set the VRSELEXCLUDE attribute for each data set record – one data set record for each of the volumes on which the data set resides.

You can see the correct syntax in Figure 5-1 that can be used to set the vital record selection processing to YES or NO.

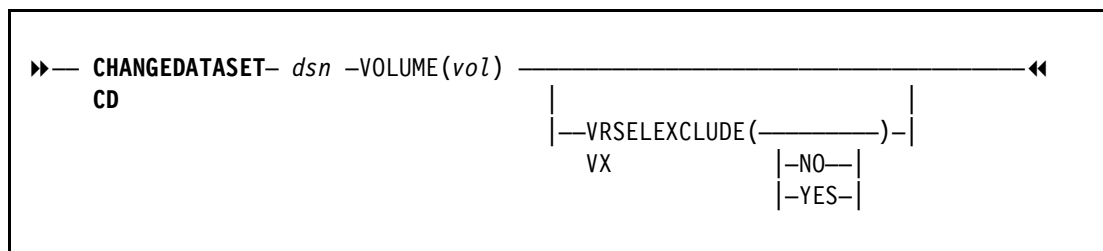


Figure 5-1 RETENTIONMETHOD operand command syntax

Where:

- | | |
|------------|---|
| NO | to ensure a data set is included in VRSEL processing. |
| YES | to exclude a data set from VRSEL processing. |

Important:

Authorization requires either:

- Based on STGADMIN.EDG.MASTER access, if the resource below is not defined
- UPDATE access to STGADMIN.EDG.CD.VX to allow any data set to be updated.

5.2.2 Using EDG_EXIT100 to excluding Data Sets from VRSEL support

A new option is provided via EDG_EXIT100 to request to override DFSMSrmm VRSEL processing for specific data sets as they are created or re-written. You can specify this for any data set but DFSMSrmm ignores your request unless the data set is on a volume which is managed by VRSEL retention method. The data set VRSELEXCLUDE attribute is set for all data sets on volumes managed by the EXPDT retention method, and is not affected by this support. If a data set is already retained as a vital record, the vital record attribute is reset and the retention date set to the current date.

When DFSMSrmm honours your request to exclude a data set from VRSEL, processing ensures that the data set vital record attribute is reset and the retention date is set to current date. The matching VRS info is left unchanged.

You can use the EDG_EXIT100 installation exit to set the VRSELEXCLUDE attribute for VRSEL managed data sets at OPEN time when you create a new tape data set or rewrite existing data sets.

When DFSMSrmm honours your request to exclude a data set from VRSEL, processing ensures that the data set vital record attribute is reset and the retention date is set to current date. The matching VRS info is left unchanged.

You can use the EDG_EXIT100 installation exit to set the VRSELEXCLUDE attribute for VRSEL managed data sets at OPEN time when you create a new tape data set or rewrite existing data sets.

Update the sample EDGUX100 exit module based on your retention requirements. You must perform these tasks:

1. Copy the sample EDGUX100 exit module and use the copy as a base for your exit module.
 - a. Update the exit module. Only perform your processing when the PL100_CAN_VRSELEXCLUDE bit is set to B'1'.
 - b. Set PL100_SET_VRSELEXCLUDE bit to B'1' for data sets. If you do not request VRSELEXCLUDE the default for the retention method will be used. If the installation exit sets PL100_SET_VRSELEXCLUDE any VRS management value set in PL100_VRS is ignored.
 - c. You do not need to set the PL100_SET_VRSELEXCLUDE bit when you also request to set the retention method to EXPDT: DFSMSrmm always sets the VRSELEXCLUDE attribute for data sets managed by the EXPDT retention method.
2. Make any other changes required such as setting the retention method when creating the first file on the tape, or clearing the EXPDT.

The sample EDGUX100 exit module includes an example of setting the VRSELEXCLUDE attribute. To use the sample EDGUX100 exit module for this function, modify the table as shown in Figure 5-2. The order in which the table entries are listed is important because the

exit scans the table until it finds the first entry where the job name, data set name and program name masks match the current request. You can change the priority of matching by changing the order of the table entries.

VXTAB DS OF		START OF VRSELEXCLUDE TABLE
SPACE	1	
DC	CL8'*'	JOBNAME
DC	CL44'RMMUSER.VX.*'	DATA SET NAME
DC	CL8'*'	PROGRAM NAME
DC	CL8'SCHLUM*'	JOBNAME
DC	CL44'RMM.ADDONS.*'	DATA SET NAME
DC	CL8'*'	PROGRAM NAME
DC	CL8'BACK%%%'	JOBNAME
DC	CL44'SSC.BACK*'	DATA SET NAME
DC	CL8'ADRSSU'	PROGRAM NAME
SPACE	1	
DC	CL8'VX END'	END OF VX TABLE MARKER

Figure 5-2 Sample VRSELEXCLUDE selection table

Where:

JOBNAME	One-to-eight alphanumeric or national characters including % and *. % can be used to ignore a positional character in the job name. * can be used to ignore all remaining characters in the job name. A jobname of * means that the entry applies to all jobs.
DATA SET NAME	Can be up to forty-four characters, following z/OS data set naming conventions, including % and *. % can be used to ignore a positional character in the data set name. * can be used to ignore all remaining characters in the data set name. A data set name of * means that the entry applies to all data sets. The use of the character * is not the same as in the generic data set names supported by DFSMSrmm for vital records specifications and search data set masks. Here the * works like the characters *.*might in a generic data set name mask.
PROGRAM NAME	A value up to eight alphanumeric character including % and *. % can be used to ignore a positional character in the program name. * can be used to ignore all remaining characters in the program name. A program name of * means that the entry applies to all programs.

To compile and link the updated EDGUX100 user exit use the JCL shown in Figure 5-3 on page 183.


```

//ASMCL   PROC SM=,LM=
//C        EXEC PGM=ASMA90,PARM='NODECK,XREF(SHORT),LINECOUNT(58)'
//SYSLIN   DD DSN=RMM.ADDONS.OBJ(&LM),DISP=SHR
//SYSLIB   DD DSN=SYS1.MACLIB,DISP=SHR
//         DD DSN=SYS1.MODGEN,DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUT1   DD UNIT=SYSDA,SPACE=(7040,400)
//SYSUT2   DD UNIT=SYSDA,SPACE=(3520,400)
//SYSUT3   DD UNIT=SYSDA,SPACE=(3520,400)
//SYSIN    DD DSN=RMM.ADDONS.SOURCE(&SM),DISP=SHR
//L        EXEC PGM=HEWL,PARM='LIST,MAP,XREF,RENT,REUS,',COND=(4,LT,C)
//SYSLMOD  DD DSN=SYS1.SANDBOX.MIGLIB(&LM),DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUT1   DD UNIT=SYSDA,SPACE=(3520,400)
//SYSLIN   DD DSN=RMM.ADDONS.OBJ(&LM),DISP=SHR
//         PEND
//COMPLNKO EXEC ASMCL,LM=EDGUX100,SM=EDGUX100

```

Figure 5-3 Compile and link updated EDGUX100 user exit

To activate or change the current used EDGUX100 user exit you have to use the commands shown in Figure 5-4. If the exit was loaded previously you must first delete them and then you have to use the add function.

```

SETPROG EXIT,DELETE,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100

SETPROG EXIT,ADD,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100

```

Figure 5-4 SETPROG command syntax

Before z/OS V1.12 the correct sequence to replace an existing exit was as following you can see in Figure 5-5.

```

MODIFY DFRMM,QUIESCE

SETPROG EXIT,DELETE,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100

SETPROG EXIT,ADD,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100

MODIFY DFRMM,M=nn

```

Figure 5-5 SETPROG command syntax sequence before z/OS V1.12

Where:

ADD	Adds an exit routine to an exit.
REPLACE	Replaces an exit routine for an exit.
DELETE	Deletes an exit routine from an exit.
EXITNAME	The 1-16 character name of the exit.
MODNAME	The 1-8 character name of the exit routine. If DSNAME is not specified, the system tries to locate the exit routine using the LPA, the LNKST concatenation, and the nucleus.

Tip: DFSMSrmm provides a second sample for EDGUX100. This sample is called EDGCVRSX. It is different from the EDGUX100 sample because the special date, retention method, VRSELEXCLUDE, and pooling function is table driven and you can change the table dynamically. Refer to SAMPLIB member EDGCMM01 and the IBM Red book DFSMSrmm Primer for documentation on using EDGCVRSX for EDGUX100.

5.2.3 Using the SEARCHDATASET subcommand

Using the SEARCHVOLUME subcommand with the VRSELEXCLUDE operand to specifies a list limited to data sets that have a specified vital record exclude method. Specify YES to exclude a data set from VRSEL processing. Specify NO to ensure a data set is included in VRSEL processing. The new VRSELEXCLUDE operand syntax is shown in Figure 5-6.

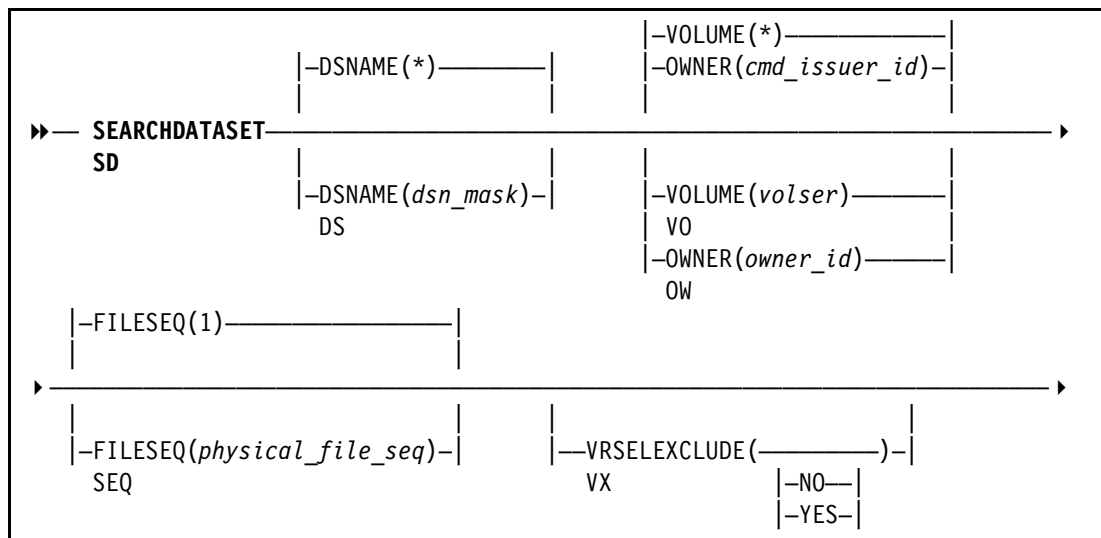


Figure 5-6 VRSELEXCLUDE operand command syntax

Where:

- NO** to ensure a data set is included in VRSEL processing.
- YES** to exclude a data set from VRSEL processing.

Example 1:

Figure 5-7 shows you how you can use the RMM SEARCHDATASET subcommand to get a list of all data sets owned by user SCHLUM.

Note: If you do not specify the VRSELEXCLUDE operand DFSMSrmm searches all data sets, regardless of the vital record selection criteria set for a data set.

```
RMM SEARCHDATASET LIMIT(*) OWNER(SCHLUM)
```

Figure 5-7 SEARCHDATASET without the VRSELEXCLUDE operand

You get the list of all 10 data sets the owner SCHLUM are currently owned as you can see in Figure 5-8 on page 185.

Data set name	Volume	Owner	Create date	Seq
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0008	SCHLUM	2011/324	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0013	SCHLUM	2011/324	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0015	SCHLUM	2011/324	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0016	SCHLUM	2011/324	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0017	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0021	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0022	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0023	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0024	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0025	SCHLUM	2011/325	1
EDG3012I 10	ENTRIES LISTED			

Figure 5-8 SEARCHDATASET result without using the VRSELEXCLUDE operand

Example 2:

Figure 5-7 on page 184 shows you how you can use the RMM SEARCHDATASET subcommand to get a list of all data sets owned by user SCHLUM and have the VRSELEXCLUDE set to YES.

```
RMM SEARCHDATASET LIMIT(*) OWNER(SCHLUM) VRSELEXCLUDE(YES)
```

Figure 5-9 SEARCHDATASET using VRSELEXCLUDE(YES) operand

Figure 5-8 shows only this 8 data sets have the VRSELEXCLUDE set to YES.

Data set name	Volume	Owner	Create date	Seq
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0008	SCHLUM	2011/324	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0013	SCHLUM	2011/324	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0017	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0021	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0022	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0023	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0024	SCHLUM	2011/325	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0025	SCHLUM	2011/325	1
EDG3012I 8	ENTRIES LISTED			

Figure 5-10 SEARCHDATASET using VRSELEXCLUDE(YES) operand

Now you can use the LISTDATASET subcommand to get all the details of one of the listed data sets as shown in Figure 5-11.

```
RMM LD 'RMM.F13002.ON.TWO.VOLUMES.AND.CATLG' VOL(VT0025)
```

Figure 5-11 LISTDATASET subcommand

Figure 5-12 on page 186 shows you the result of the LISTDATASET subcommand.

```

Data set name = RMM.F13002.ON.TWO.VOLUMES.AND.CATLG
Volume       = VT0025           Physical file sequence number = 1
Owner        = SCHLUM           Data set sequence = 1
Create date  = 2011/325   Create time = 19:35:33 System ID       = SC70
Expiration date = 2011/325   Original expir. date =
      set by    = OCE_DEF
Block size    = 80            Block count      = 1
Data set size(KB) = 1
Physical size(KB) = 0          Compression      = 0.00
Percent of volume = 0          Total block count = 2
Logical Record Length = 0      Record Format    = F
Date last written = 2011/325   Date last read  = 2011/325
Job name       = TEST9999     Last job name   = TEST9999
Step name      = STEP03       Last step name  = STEP03
Program name   = EOVTST       Last program name = EOVTST
DD name        = OUT          Last DD name    = OUT
Device number  = 0406         Last Device number = 0406
Management class =            VRS management value =
Storage group  =              VRS retention date =
Storage class  =              VRS retained      = NO
Data class     =              Closed by Abend    = NO
                                   Deleted        = NO
VRSEL exclude = YES          Catalog status   = UNKNOWN

Primary VRS details:
  Name         =
  Job name     =              Type               =
  Subchain NAME =              Subchain start date =

Secondary VRS details:
  Value or class =
  Job name     =
  Subchain NAME =              Subchain start date =

Security Class =              Description        =
BES key index  = 0

Last Change information:
Date           = 2011/325   Time = 20:25:03   System = SC70
User change date = 2011/325   Time = 20:25:03   User ID = SCHLUM

```

Figure 5-12 LISTDATASET result

Where:

Retention Date this field displays the date a data set is no longer retained by the current VRS. For data sets not retained by a VRS the retention date is either the date no longer retained by VRS, or null.
Possible values:

Calendar date the date calculated by VRSEL processing in the currently selected date format.

WHILECATLG data set is retained by a VRS with WHILECATALOG.

CYCL/cccc data set is retained by a VRS defined with CYCLES, where ccccc is the COUNT of cycles.

CATRETPD data set is retained by a VRS defined with WHILECATALOG and the data set is not cataloged, but the CATRETPD parmlib option applies.

PERMANENT data set is retained by a VRS defined with DAYS COUNT(99999).

VRSEL exclude this field specifies whether the data set is excluded from vital record processing.
The VRSELEXCLUDE attribute is set to YES for all data sets on volumes managed by the EXPDT retention method. It can optionally be set to YES for any data set on a volume which is managed by the VRSEL retention method.
When a data set spans volumes you should set the VRSELEXCLUDE attribute for each data set record - one data set record for each of the volumes on which the data set resides.
Possible values:

YES to exclude a data set from VRSEL processing

NO to ensure a data set is included in VRSEL processing

Note: If a data set is not vital record selected the retention date field is always empty.

Example 3:

Figure 5-13 shows you how you can use the RMM SEARCHDATASET subcommand to get a list of all data sets owned by user SCHLUM and have the VRSELEXCLUDE set to NO.

```
RMM SEARCHDATASET LIMIT(*) OWNER(SCHLUM) VRSELEXCLUDE(NO)
```

Figure 5-13 SEARCHDATASET using VRSELEXCLUDE(NO) operand

Figure 5-14 shows only this 2 data sets have the VRSELEXCLUDE set to NO.

Data set name	Volume	Owner	Create date	Seq
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0015	SCHLUM	2011/324	1
RMM.F13002.ON.TWO.VOLUMES.AND.CATLG	VT0016	SCHLUM	2011/324	1
EDG3012I 2 ENTRIES LISTED				

Figure 5-14 SEARCHDATASET using VRSELEXCLUDE(NO) operand

Now you can use the LISTDATASET subcommand to get all the details of one of the listed data sets as shown in Figure 5-15.

```
RMM LD 'RMM.F13002.ON.TWO.VOLUMES.AND.CATLG' VOL(VT0015)
```

Figure 5-15 LISTDATASET subcommand

Figure 5-16 on page 188 shows you the result of the LISTDATASET subcommand.

```

Data set name = RMM.F13002.ON.TWO.VOLUMES.AND.CATLG
Volume       = VT0015           Physical file sequence number = 1
Owner        = SCHLUM           Data set sequence = 1
Create date  = 2011/324   Create time = 19:41:58 System ID       = SC70
Expiration date = 2011/333   Original expir. date = 2011/333
                set by      = OCE_JFCB
Block size   = 80             Block count           = 1
Data set size(KB) = 1
Physical size(KB) = 0         Compression           = 0.00
Percent of volume = 0         Total block count      = 1
Logical Record Length = 0     Record Format         = F
Date last written = 2011/324   Date last read      = 2011/324
Job name       = TEST9999     Last job name       = TEST9999
Step name      = STEP03       Last step name      = STEP03
Program name   = EOVTST       Last program name   = EOVTST
DD name        = OUT          Last DD name        = OUT
Device number  = 044C         Last Device number  = 044C
Management class =           VRS management value =
Storage group  =              VRS retention date = PERMANENT
Storage class  =              VRS retained      = YES
Data class     =              Closed by Abend      = NO
                                   Deleted           = NO
                                   Catalog status      = UNKNOWN

VRSEL exclude = NO
Primary VRS details:
  Name          = RMM.F13002.**
  Job name      =              Type                 = DATASET
  Subchain NAME =              Subchain start date =
Secondary VRS details:
  Value or class =
  Job name      =
  Subchain NAME =              Subchain start date =
Security Class  =              Description          =
BES key index  = 0

Last Change information:
Date           = 2011/325   Time = 20:25:03   System = SC70
User change date = 2011/325   Time = 20:25:03   User ID = SCHLUM

```

Figure 5-16 LISTDATASET result

Where:

Retention Date this field displays the date a data set is no longer retained by the current VRS. For data sets not retained by a VRS the retention date is either the date no longer retained by VRS, or null.
Possible values:

Calendar date the date calculated by VRSEL processing in the currently selected date format.

WHILECATLG data set is retained by a VRS with WHILECATALOG.

CYCL/cccc data set is retained by a VRS defined with CYCLES, where ccccc is the COUNT of cycles.

CATRETPD	data set is retained by a VRS defined with WHILECATALOG and the data set is not cataloged, but the CATRETPD parmlib option applies.
PERMANENT	data set is retained by a VRS defined with DAYS COUNT(99999).
VRSEL exclude	this field specifies whether the data set is excluded from vital record processing. The VRSELEXCLUDE attribute is set to YES for all data sets on volumes managed by the EXPDT retention method. It can optionally be set to YES for any data set on a volume which is managed by the VRSEL retention method. When a data set spans volumes you should set the VRSELEXCLUDE attribute for each data set record - one data set record for each of the volumes on which the data set resides. Possible values:
YES	to exclude a data set from VRSEL processing
NO	to ensure a data set is included in VRSEL processing

5.2.4 Inventory Management VRSEL/EXPROC Processing

With release z/OS V1.13 the inventory management processing is changed:

- ▶ You do not need to run VRSEL processing unless any volumes are defined with the VRSEL retention method. Only EXPROC processing is required to handle expiration of all volumes managed by the EXPDT retention method.
- ▶ EXPROC processing provides a summary of volumes by retention method.
- ▶ The expiration date of volumes is set during OPEN processing, so for volumes managed by the EXPDT retention method, no special considerations exist for open data sets; they are managed based on the volume EXPDT.
- ▶ For volumes managed by the EXPDT retention method, no special considerations exist for data sets closed by ABEND processing or that are DELETED; they are managed based on the volume EXPDT.
- ▶ Volumes managed by the EXPDT retention method are included only in the EXPDTDROP limit. VRSRETAIN and VRSDROP limits apply only to volumes managed by VRSEL retention method.

Updated EDGHSKP messages

For inventory management (VRSEL) there is one new message EDG2246I that is added and for expiration processing (EXPROC) no new messages are created, but existing messages are expanded and adapted with new/additional inserts, as you can see in Figure 5-17 on page 190

```

EDG6001I INVENTORY MANAGEMENT STARTING ON 2011/326 AT 13:26:39 -
          PARAMETERS IN USE ARE
          DATEFORM(J),VRSEL,EXPROC,BACKUP(AMS)
EDG2309I THE PARMLIB OPTIONS CURRENTLY IN USE ARE
          VRSEL(NEW)
          VRSJOBNAME(2)
          VRSMIN(1,INFO)
          VRSCCHANGE(INFO)
          VRSDROP(PERCENT(10),INFO) VRSRETAIN(PERCENT(80),INFO)
          EXPDTPDROP(PERCENT(10),INFO)
          SMSTAPE(PURGE(ASIS) UPDATE(EXITS,SCRATCH,COMMAND))
          CATRETPD(6)
          UNCATALOG(Y)
          TPRACF(N)
          NOTIFY(N)
          SYSID(SC64)
          CATSYSID()
          RETAINBY(SET)
          MOVEBY(SET)
          GDG(CYCLEBY(GENERATION),DUPLICATE(BUMP))
EDG2229I NUMBER OF VRS RECORDS READ IS 3
EDG2238I NUMBER OF UNUSED VRS RECORDS IS 1
EDG2246I NUMBER OF DATA SET RECORDS EXCLUDED FROM VRSEL =      8   7%
EDG2242I INITIAL NUMBER OF VRS RETAINED VOLUMES =      107  33%
EDG2244I NUMBER OF VRS RETAINED VOLUMES TO BE DROPPED =      0   0%
EDG2243I INITIAL NUMBER OF NEWLY ASSIGNED VOLUMES =      2   1%
EDG2245I NUMBER OF NEWLY ASSIGNED VOLUMES TO BE RETAINED=      0   0%
EDG2444I EXIT PROCESSING DISABLED FOR THIS EXPROC RUN -
          NO ACTIVE EXIT MODULE FOR EXIT EDG_EXIT200
EDG2427I INITIAL NUMBER OF EXPDT RETAINED VOLUMES =      6   2%
EDG2428I NUMBER OF EXPDT RETAINED VOLUMES TO BE DROPPED =      2  33%
EDG2420I PHYSICAL VOLUMES READ =      30   9%
EDG2420I LOGICAL VOLUMES READ =      300  91%
EDG2420I RM_VRSEL VOLUMES READ =      322  98%
EDG2420I RM_EXPDT VOLUMES READ =      8   2%
EDG2420I TOTAL VOLUMES READ =      330 100%
EDG2421I PHYSICAL VOLUMES UPDATED =      4  13%
EDG2421I LOGICAL VOLUMES UPDATED =      4   1%
EDG2421I RM_VRSEL VOLUMES UPDATED =      6   2%
EDG2421I RM_EXPDT VOLUMES UPDATED =      2  25%
EDG2421I TOTAL VOLUMES UPDATED =      8   2%
EDG2435I PHYSICAL VOLUMES SELECTED FOR EXPROC =      30 100%
EDG2435I LOGICAL VOLUMES SELECTED FOR EXPROC =      300 100%
EDG2435I RM_VRSEL VOLUMES SELECTED FOR EXPROC =      322 100%
EDG2435I RM_EXPDT VOLUMES SELECTED FOR EXPROC =      8 100%
EDG2435I TOTAL VOLUMES SELECTED FOR EXPROC =      330 100%
EDG2424I LOGICAL VOLUMES SET PENDING RELEASE =      4   1%
EDG2424I RM_VRSEL VOLUMES SET PENDING RELEASE =      2   1%
EDG2424I RM_EXPDT VOLUMES SET PENDING RELEASE =      2  25%
EDG2424I TOTAL VOLUMES SET PENDING RELEASE =      4   1%
EDG2425I PHYSICAL VOLUMES RETURNED TO SCRATCH =      4  13%
EDG2425I LOGICAL VOLUMES RETURNED TO SCRATCH =      2   1%
EDG2425I RM_VRSEL VOLUMES RETURNED TO SCRATCH =      4   1%
EDG2425I RM_EXPDT VOLUMES RETURNED TO SCRATCH =      2  25%
EDG2425I TOTAL VOLUMES RETURNED TO SCRATCH =      6   2%
EDG2426I PHYSICAL VOLUMES - SCRATCH RECORDS WRITTEN =      2  10%
EDG2426I RM_EXPDT VOLUMES - SCRATCH RECORDS WRITTEN =      2  14%
EDG2426I TOTAL VOLUMES - SCRATCH RECORDS WRITTEN =      2   6%
EDG2429I MAIN INVENTORY MANAGEMENT UPDATES HAVE COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK VRSEL COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK EXPROC COMPLETED SUCCESSFULLY
EDG6426I CONTROL DATA SET AND JOURNAL BACKUP SUCCESSFUL
EDG6901I UTILITY EDGHSKP COMPLETED WITH RETURN CODE 0

```

Figure 5-17 EDGHSKP messages

Where:

EDG2420I *volume_type* VOLUMES READ = *number percent*%

Explanation	DFSMSrmm issues this message when the number of volumes read is greater than zero. In the message text: <i>volume_type</i> This is the volume type and can be: PHYSICAL LOGICAL STACKED RM_VRSEL RM_EXPDT TOTAL <i>number</i> This is the number of <i>volume_type</i> volumes read during inventory management. <i>percent</i> This is the percentage of total volumes read by DFSMSrmm inventory management processing.
System action	Processing continues.
Operator response	None.
Source	DFSMSrmm
Detecting Module	EDGMUPD
Routing Code	11
Descriptor Code	7

EDG2421I *volume_type* VOLUMES UPDATED = *number percent*%

Explanation	DFSMSrmm issues this message when the number of volumes updated is greater than zero. In the message text: <i>volume_type</i> This is the volume type and can be: PHYSICAL LOGICAL STACKED RM_VRSEL RM_EXPDT TOTAL <i>number</i> This is the number of <i>volume_type</i> volumes that were updated in the DFSMSrmm control data set. <i>percent</i> The percentage of the <i>volume_type</i> volumes read by DFSMSrmm inventory management processing.
System action	Processing continues.
Operator response	None.
Source	DFSMSrmm
Detecting Module	EDGMUPD
Routing Code	11
Descriptor Code	7

**EDG2424I *volume_type* TOTAL VOLUMES, THIS RUN, SET PENDING RELEASE =
number percent%**

Explanation	This message is issued for information only. In the message text: <i>volume_type</i> volume type is one of the following: PHYSICAL LOGICAL RM_VRSEL RM_EXPDT TOTAL <i>number</i> The number of <i>volume_type</i> volumes set pending release in this run of inventory management. <i>percent</i> This is the percentage of <i>volume_type</i> volumes selected for EXPROC processing (see message EDG2435I).
System action	Processing continues.
Operator response	None.
Source	DFSMSrmm
Detecting Module	EDGMUPD
Routing Code	11
Descriptor Code	7

**EDG2425I *volume_type* TOTAL VOLUMES RETURNED TO SCRATCH =
number percent%**

Explanation	This message is issued for information only. In the message text: <i>volume_type</i> volume type is one of the following: PHYSICAL LOGICAL RM_VRSEL RM_EXPDT TOTAL <i>number</i> the number of <i>volume_type</i> volumes returned to scratch status after all release actions have been completed. <i>percent</i> This is the percentage of <i>volume_type</i> volumes selected for EXPROC processing (see message EDG2435I).
System action	Processing continues.
Operator response	None.
Source	DFSMSrmm
Detecting Module	EDGMUPD
Routing Code	11
Descriptor Code	7

EDG2426I *volume_type* TOTAL NUMBER OF SCRATCH RECORDS WRITTEN = *number percent%*

Explanation	DFSMSrmm issues this message to the MESSAGE file when EXPROC processing for system managed volumes requests deferred processing of volumes to scratch. In the message text: <i>volume_type</i> volume type is one of the following: PHYSICAL LOGICAL RM_VRSEL RM_EXPDT TOTAL <i>number</i> The number of system managed <i>volume_type</i> volumes that are ready for returning to scratch on this system, for which a record has been written to the EDGSPLCS output file. <i>percent</i> The percentage of <i>volume_type</i> volumes selected for EXPROC processing (see message EDG2435I).
System action	DFSMSrmm inventory management processing continues.
Operator response	None.
System programmer response	Run the EDGSPLCS utility to process the scratch statements produced by EDGHSKP EXPROC processing for system managed volumes.
Source	DFSMSrmm
Detecting Module	EDGMUPD

EDG2435I *volume_type* VOLUMES SELECTED FOR EXPROC = *number percent%*

Explanation	DFSMSrmm issues this message to the MESSAGE file during inventory management EXPROC processing. This message is issued for information only. In the message text: <i>volume_type</i> This is the volume type and can be: PHYSICAL LOGICAL STACKED RM_VRSEL RM_EXPDT TOTAL <i>number</i> the number of <i>volume_type</i> volumes read during inventory management. <i>percent</i> This is the percentage of total volumes read by DFSMSrmm inventory management processing.
System action	DFSMSrmm inventory management processing continues.
Operator response	None.

System programmer response

None.

Source DFSMSrmm

Detecting Module EDGMUPD



Setting up an RMMplex

This chapter explains how to set up your systems so that you can use a single DFSMSrmm control data set (CDS) on multiple systems without having a shared direct access storage device (DASD) environment. It provides detailed information about the implementation and use of an RMMplex.

The objectives of this chapter are to:

- ▶ Client/server terminology
- ▶ Implementing client and server systems
- ▶ Starting DFSMSrmm in an RMMplex
- ▶ Managing volumes in an RMMplex
- ▶ Operator commands

By the end of this chapter, you will understand the different VRS types and how to create them. Sample VRS definitions are provided for a better understanding.

6.1 RMMplex

An RMMplex can optionally include one or more RMM subsystems as servers, one or more client subsystems in addition to standard subsystems. The server subsystems and standard subsystems have direct access to and share the DFSMSrmm control data set. The client systems have no direct access to the DFSMSrmm control data set, but share the CDS via the server. All systems which share a DFSMSrmm control data set in this way are part of the same RMMplex.

We use the configuration shown in Figure 6-1 on page 197, which shows a complex RMMplex implementation with two sysplexes and one non-plex sharing one single DFSMSrmm CDS. The second non-plex shows a normal shared DFSMSrmm CDS. Each plex has its own DASD environment that is not shared between the other plexes and each plex has a unique MVS catalog environment.

An RMMplex can optionally include one or more RMM subsystems as servers, one or more client subsystems in addition to standard subsystems. The server subsystems and standard subsystems have direct access to and share the CDS. The client systems have no direct access to the CDS, but share the CDS via the server. All systems which share a CDS in this way are part of the same RMMplex.

Note: All dates and times used on the client system are local time. All dates and times used on the server are in local time. When data is retrieved from the RMM CDS and displayed by subcommand processing, the dates and times used are exactly as stored in the DFSMSrmm control data set. No conversion from server time zone to client time zone is performed.

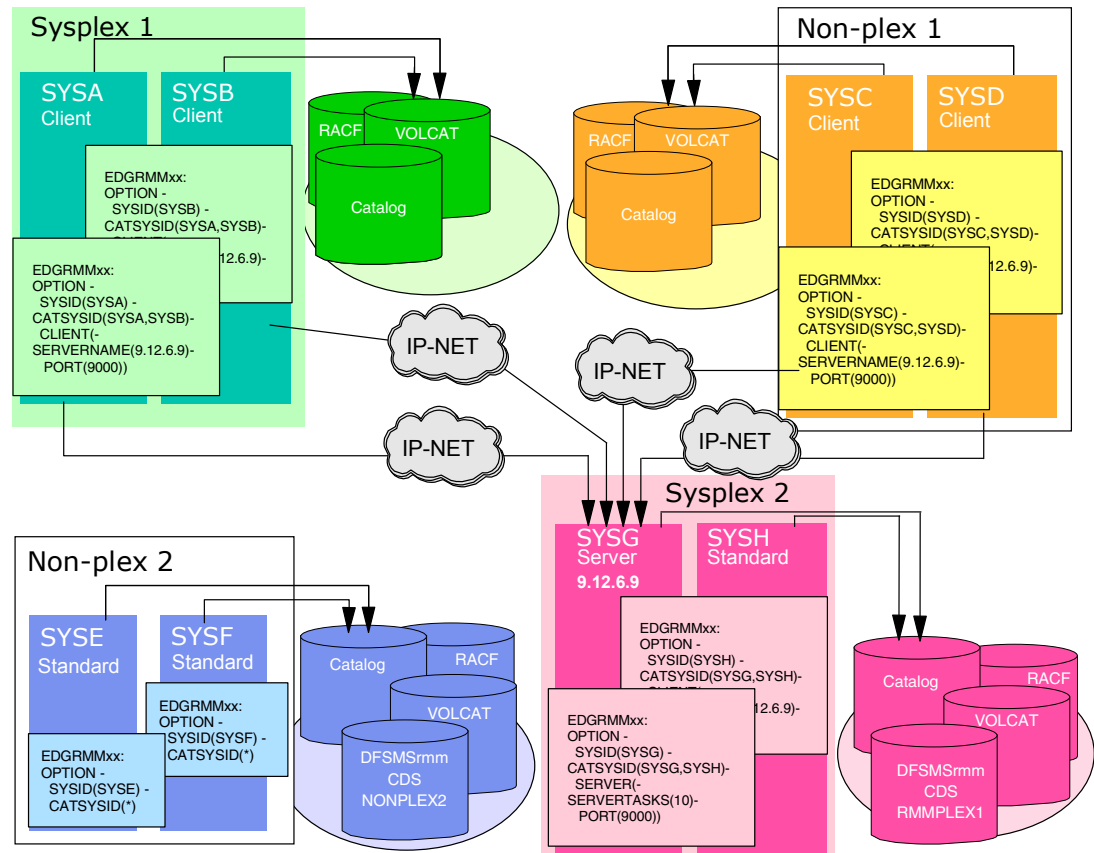


Figure 6-1 RMMplex with six systems sharing one DFSMSrmm CDS

6.1.1 Maintenance levels

The DFSMSrmm clients and servers can be on different releases. The minimum level is z/OS V1R6. As with CDS record toleration, DFSMSrmm provides toleration for future changes in the requests which are passed between systems. The record mappings for the requests remain restricted materials of IBM.

The server checks the version of the request data area and accepts it for processing if the version number matches that known to the server. If the level is higher on the client, the server fails the request with a new, specific return code and reason code combination. When a new level of data area is required, the level is set only into the data area when the new fields are actually used, so that an up-level client can at least perform non-new functions via a lower-level server.

6.2 Client/server terminology

In a DFSMSrmm environment, we use the term client and server as explained in the following sections.

6.2.1 DFSMSrmm server

A “server” in DFSMSrmm terminology is a system that runs when MVS and DFSMSrmm are up and active. This DFSMSrmm has direct access to the DFSMSrmm control data set and supports requests coming from other systems via TCP/IP.

6.2.2 SERVERNAME

The server name can be specified either be an IP address, a fully qualified domain name, or the host name of a server. DFSMSrmm uses the Domain Name System (DNS) to resolve a domain name or host name into an IP address. The values for the host name must conform to the following requirements:

- ▶ They must have a maximum of 63 characters.
- ▶ They must contain one or more tokens separated by a period.
- ▶ Each token must be larger than one character.
- ▶ The first character in each token must start with a letter.
- ▶ The remaining characters in each token must be a letter, number, or hyphen.

6.2.3 PORT

The port indicates the port number used for IP communication. You can specify a value from 1 to 65535. The port number must be the same for the client system and the server system to establish a network connection.

Note: Port numbers 1 to 1023 are reserved and should not be used.

6.2.4 SERVERTASKS

Use the SERVERTASKS operand to set the number of DFSMSrmm tasks that should be available on the server to handle socket connections from client systems. You can specify a value from 1 to 999. DFSMSrmm uses this number to determine the number of tasks to be started for processing all client requests on this server.

Recommendation: Depending on the number of client systems, increase this value to allow three tasks per client. You do not need more server tasks than the sum of local tasks across your client systems.

6.2.5 DFSMSrmm client

A “client” in DFSMSrmm terminology is a system that runs when MVS and DFSMSrmm are up and active. This DFSMSrmm has no direct access to the DFSMSrmm control data set and sends all requests to the DFSMSrmm control data set via TCP/IP to the “server” specified in EDGRMMxx PARMLIB member.

6.2.6 LOCALTASKS

Use the LOCALTASKS operand to set the number of tasks available on each system for processing locally initiated requests. On a client system, LOCALTASKS is also the maximum number of tasks that can make a socket connection to the server. You can specify a value from 1 to 999.

6.3 Implementing client and server systems

Before you start, make sure that all systems are z/OS V1.6 or later. You must also have installed Internet Protocol V4.

Note: You can share the CDS with other systems that run any supported level of DFSMSrmm with any supported level of DFSMSrmm that has appropriate toleration maintenance installed.

You can convert existing DFSMSrmm systems to be either client or server systems, or add new DFSMSrmm systems to an RMMplex. In addition, you can merge existing DFSMSrmm systems into an RMMplex by merging the CDSs as explained in 15.6, “Merging DFSMSrmm environments” on page 652.

You can get the IP address of your DFSMSrmm server using different commands.

6.3.1 Checking your TCP/IP configuration

Before you can update the EDGRMMxx PARMLIB member to define your systems as DFSMSrmm client or server, obtain the necessary TCP/IP address of the server and perform testing to ensure that the TCP/IP configuration is working.

HOMETEST command

Using the TSO HOMETEST command you can see in Figure 6-2 to run the IBM MVS TCP/IP CS V1R4 TCP/IP configuration tester to obtain the TCP/IP information.



```
TSO HOMETEST
```

Figure 6-2 TSO HOMETEST command

Example 6-1 shows the result of the TCP/IP configuration tester.

Example 6-1 *TCP/IP configuration tester output*

```
Running IBM MVS TCP/IP CS V1R6 TCP/IP Configuration Tester
```

```
The FTP configuration parameter file used will be SYSFTPD DD.  
TCP Host Name is: WTSC64
```

```
Using Name Server to Resolve WTSC64  
The following IP addresses correspond to TCP Host Name: WTSC64  
9.12.6.9
```

```
The following IP addresses are the HOME IP addresses defined in PROFILE.TCPIP:  
9.12.6.9  
9.12.6.31  
10.1.101.64  
10.1.101.64  
10.1.101.64  
127.0.0.1
```

```
All IP addresses for WTSC64 are in the HOME list!
```

```
Hometest was successful - all Tests Passed!
```

DISPLAY TCPIP command

You can also use either of the DISPLAY TCPIP MVS commands shown in Figure 6-3 to access the needed TCP/IP information.

```
DISPLAY TCPIP,,NETSTAT,HOME
DISPLAY TCPIP,TCPIP,NETSTAT,HOME
```

Figure 6-3 DISPLAY TCPIP commands

A third way to obtain the needed TCP/IP information is to use the TSO NETSTAT command you can see in Figure 6-4.

```
TSO NETSTAT,HOME
```

Figure 6-4 NETSTAT command

Example 6-2 shows the result of the DISPLAY TCPIP or TSO NETSTAT command.

Example 6-2 *DISPLAY TCPIP or TSO NETSTAT command output*

```
EZD0101I NETSTAT CS V1R6 TCPIP 587
HOME ADDRESS LIST:
LINKNAME:  OSA2CCOLNK
  ADDRESS:  9.12.6.9
  FLAGS:    PRIMARY
LINKNAME:  STAVIPA1LNK
  ADDRESS:  9.12.6.31
  FLAGS:
LINKNAME:  EZASAMEMVS
  ADDRESS:  10.1.101.64
  FLAGS:
LINKNAME:  IQDIOLNK0A016540
  ADDRESS:  10.1.101.64
  FLAGS:
LINKNAME:  EZAXCF63
  ADDRESS:  10.1.101.64
  FLAGS:
LINKNAME:  LOOPBACK
  ADDRESS:  127.0.0.1
  FLAGS:
INTFNAME:  LOOPBACK6
  ADDRESS:  ::1
  TYPE:    LOOPBACK
  FLAGS:
7 OF 7 RECORDS DISPLAYED
```

OMVS console

The third way to obtain TCP/IP information about your configuration is to use the Open MVS console commands. You can start the Open MVS console session using the TSO command OMVS as shown in Figure 6-5 on page 201.

TSO OMVS

Figure 6-5 OMVS command

After you enter the OMVS command, you see the information shown in Example 6-3. You can enter any Open MVS command. To access the TCP/IP configuration, you use the HOSTNAME command.

Example 6-3 Open MVS command HOSTNAME

```
IBM
Licensed Material - Property of IBM
5694-A01 (C) Copyright IBM Corp. 1993, 2004
(C) Copyright Mortice Kern Systems, Inc., 1985, 1996.
(C) Copyright Software Development Group, University of Waterloo, 1989.

All Rights Reserved.

U.S. Government users - RESTRICTED RIGHTS - Use, Duplication, or
Disclosure restricted by GSA-ADP schedule contract with IBM Corp.

IBM is a registered trademark of the IBM Corp.

/usr/lpp/skrb/bin:/usr/lpp/dce/bin:/usr/lpp/ldap/bin:/usr/lpp/lldap/sbin:/usr/lpp
/java/J1.4/bin:/bin:/usr/sbin:.
MHLRES4 @ SC64:/>
```

====> **HOSTNAME**

						INPUT
ESC=¢	1=Help	2=SubCmd	3=HlpRetrn	4=Top	5=Bottom	6=TSO
	7=BackScr	8=Scroll	9=NextSess	10=Refresh	11=FwdRetr	12=Retrieve

Example 6-4 shows the result of the DISPLAY HOSTNAME command.

Example 6-4 Display HOSTNAME output

```
WTSC640E.itso.ibm.com
MHLRES4 @ SC64:/>

====>

ESC=¢  1=Help    2=SubCmd    3=HlpRetrn  4=Top      5=Bottom    6=TSO
        7=BackScr 8=Scroll    9=NextSess 10=Refresh 11=FwdRetr 12=Retrieve
```

To test the TCP/IP connection between your current system and another system, enter the Open MVS command PING hostname command as shown in Example 6-5.

Example 6-5 Open MVS PING command

```
IBM
Licensed Material - Property of IBM
5694-A01 (C) Copyright IBM Corp. 1993, 2004
(C) Copyright Mortice Kern Systems, Inc., 1985, 1996.
(C) Copyright Software Development Group, University of Waterloo, 1989.

All Rights Reserved.

U.S. Government users - RESTRICTED RIGHTS - Use, Duplication, or
```

Disclosure restricted by GSA-ADP schedule contract with IBM Corp.

IBM is a registered trademark of the IBM Corp.

```
/usr/lpp/skrb/bin:/usr/lpp/dce/bin:/usr/lpp/ldap/bin:/usr/lpp/ldap/sbin:/usr/lpp
/java/J1.4/bin:/bin:/usr/sbin:.
MHLRES4 @ SC64:/>
```

====> **PING WTSC630E.itso.ibm.com**

INPUT

ESC=¢	1=Help	2=SubCmd	3=HlpRetrn	4=Top	5=Bottom	6=TSO
	7=BackScr	8=Scroll	9=NextSess	10=Refresh	11=FwdRetr	12=Retrieve

Example 6-6 shows the result of the PING command.

Example 6-6 *Display PING output*

```
MHLRES4 @ SC64:/>ping WTSC630E.itso.ibm.com
CS V1R6: Pinging host WTSC630E.itso.ibm.com (9.12.6.71)
Ping #1 response took 0.001 seconds.
MHLRES4 @ SC64:/>
```

====>

INPUT

ESC=¢	1=Help	2=SubCmd	3=HlpRetrn	4=Top	5=Bottom	6=TSO
	7=BackScr	8=Scroll	9=NextSess	10=Refresh	11=FwdRetr	12=Retrieve

Note: Ensure that TCP/IP definition files are updated to identify the server host name, IP address, and port number.

To leave the Open MVS console environment, use the EXIT command.

Port reservation

Be sure that the port you are specifying in the DFSMSrmm EDGRMMxx PARMLIB member is not used by other tasks. Reserve a port in TCP/IP by updating the TCP/IP parameters as shown in Example 6-7.

Note: There is no need to reserve a PORT in TCP/IP. However, if you do not make a reservation, another task can use the same port number for its requests and your request to use the port number will fail.

Example 6-7 *Sample TCP/IP profile member*

```
; PROFILE.TCPIP
; =====
PORT
    20 TCP OMVS                ; OE FTP Server
        DELAYACKS              ; Delay transmission acknowledgements
    21 TCP FTPMVS1 BIND 9.12.6.9 ; control port
    21 TCP FTPOE1 BIND 9.12.6.31 ; control port
    23 TCP INTCLIEN            ; MVS Telnet Server
    23 TCP OMVS BIND 9.12.6.31  ; OE Telnet Server
    80 TCP OMVS                ; OE Web Server
    111 TCP PMAP                ; Portmap Server
    111 UDP PMAP                ; Portmap Server
    135 UDP LLBD                ; NCS Location Broker
```

```

161 UDP SNMPPD ; SNMP Agent
162 UDP SNMPPQ ; SNMP Query Engine
512 TCP REXECD BIND 9.12.6.9 ; Remote Execution Server
512 TCP OMVS BIND 9.12.6.31 ; Remote Execution Server
514 TCP REXECD BIND 9.12.6.9 ; Remote Execution Server
514 TCP OMVS BIND 9.12.6.31 ; Remote Execution Server
580 UDP NCPROUT ; NCPROUTE Server
750 TCP MVS KERB ; Kerberos
750 UDP MVS KERB ; Kerberos
751 TCP ADM@SRV ; Kerberos Admin Server
751 UDP ADM@SRV ; Kerberos Admin Server
2000 TCP IOASRV ; OSA/SF Server
2049 UDP NFS ; NFS Server
3000 TCP CICS TCP ; CICS Socket
3333 TCP HWS ; IMS/OTMA
3334 TCP HWSB ; IMS/OTMA
3335 TCP EZAIMSJL ; IMS Listener BMP
5001 TCP TPS ; OS/390 TME 10 GEM Connection Service
9000 TCP DFRMM ; DFSMSrmm RMMplex
INCLUDE TCP.COMMON.TCPPARMS(VIPAPORT)

```

NSLOOKUP command

You can use the NSLOOKUP command to specify an individual query in command mode or to issue multiple queries in interactive mode.

To issue a query, use the NSLOOKUP command in the following format shown in Figure 6-6.

```

NSLOOKUP---.-domain_name---.-.-.-.-!SubCommand!
          !-domain_address-!  !-server_name---!
                               !-server_address-!

```

Figure 6-6 NSLOOKUP command syntax

The parameters and subcommands of NSLOOKUP are case sensitive and must be entered in lowercase. Parameter values and domain names are not case sensitive.

Use the following NSLOOKUP command shown in Figure 6-7 to determine the full server name if you specify the server IP address or you can specify the server name to get the server address.

```

TSO NSLOOKUP 9.149.157.65

```

Figure 6-7 NSLOOKUP command

Example 6-8 shows the result of the NSLOOKUP command.

Example 6-8 NSLOOKUP command result

```

Server: ips-de-a.de.ibm.com
Address: 9.165.1.10
Name: stutvs.megacenter.de.ibm.com
Address: 9.149.157.65

```

Command mode query

A command mode query is entered by supplying a complete query, including any desired options and the desired name server, if different from the default name server, with the NSLOOKUP command.

Operands

For a command-mode query, use the following parameters:

DOMAIN_NAME	Queries the name server for information about the current query type of domain_name. The default query type is A (address query).
DOMAIN_ADDRESS	Reverses the components of the address and generates a pointer type (PTR) query to the name server for the "in-addr.arpa" domain mapping of the address to a domain name.
SERVER_NAME	Directs the default name server to map server_name to an IP address and then use the name server at that IP address.
SERVER_ADDRESS	Specifies the IP address of the name server to be queried other than the default name server. A query for the address in the "in-addr.arpa" domain is initially made to the default name server to map the IP address to a domain name for the server.

6.3.2 EDGRMMxx DFSMSrmm options

Update your existing EDGRMMxx PARMLIB member with the CLIENT and SERVER operands and select appropriate values for the suboperands. Also check the CATSYSID and CDSID operands. If these operands are not already set, add them now.

This section describes only the operands related to the DFSMSrmm CLIENT/SERVER support.

CATSYSID

Specify CATSYSID to enable DFSMSrmm catalog synchronization. Use the CATSYSID operand to identify the user catalogs you want tracked when you run the DFSMSrmm EDGHSKP utility with the CATSYNCH operand to exploit catalog status tracking. All the systems that you identify must have the z/OS version that code that supports catalog synchronization. See Chapter 9, "Catalog synchronization" on page 333, for more information.

CATSYSID(*) This means that all catalogs are fully shared. You must specify an asterisk (*) to specify that catalogs are fully shared so that any data set can be processed by DFSMSrmm on any DFSMSrmm subsystem.

CATSYSID(sys_ID_list) This provides a list of DFSMSrmm system IDs that share tape data set user catalogs with this DFSMSrmm subsystem. You can identify up to 16 system IDs for DFSMSrmm subsystems. You must specify a list of system IDs when user catalogs are not shared. Ensure that they are synchronized with the DFSMSrmm CDS before you run inventory management vital record processing. Include IDs for systems that you no longer use but for which you are still retaining tape data sets.

The default is None.

CDSID

The CDSID option specifies the identifier of the CDS that must be used on this system. Specify a value one to eight alphanumeric characters long. When you start DFSMSrmm, the CDSID value is compared to the ID in the CDS control record.

If the IDs match, DFSMSrmm startup continues. If the CDS does not have an ID, DFSMSrmm creates the ID in the control record from the CDSID. If the IDs do not match, DFSMSrmm startup fails and DFSMSrmm issues a message to the operator to select another PARMLIB member. If you do not specify a value for CDSID, you cannot use a CDS that already has an ID in its control record.

See the “Creating or Updating the Control Data Set Control Record” chapter in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for information about how the DFSMSrmm EDGUTIL utility sets the CDS ID.

The default is None.

CLIENT

The CLIENT option specifies the type of system you want to set up. CLIENT is mutually exclusive with SERVER. If neither client nor server are specified, DFSMSrmm starts as a standard system.

SERVERNAME(servername)

SERVERNAME is a required operand when you specify CLIENT. The servername can either be an IP address, a fully qualified domain name, or a server host name.

An IP address can be specified either as a hexadecimal IPv6 address or a numeric IPv4 address. Specify an IPv6 IP address only when the server is running z/OS V1R12 or later release and TCP/IP on the server system has IPv6 dual mode IP stack. For example,

```
CLIENT(SERVERNAME(1080:0:0:0:8:800:200C:417A))
```

specifies an IPv6 IP address. See RFC3513

(at <http://tools.ietf.org/html/rfc3513>)

for the syntax of an IPv6 IP address.

When a domain name or a host name is specified, DFSMSrmm uses the domain name system (DNS) to resolve that name into an IP address. The host name can be a maximum of 63 characters. The host name must contain one or more tokens separated by a period. Each token must be larger than one character. The first character in each token must start with a letter. The remaining characters in each token can be a letter, number, or hyphen. For example,

```
CLIENT(SERVERNAME(RMMPLX1.MAINZ.IBM.COM) PORT(1950))
```

tells DFSMSrmm to start as a client without direct DASD access and to share the tape inventory in access by the RMM server with the host name RMMPLX1 using network IP protocol port 1950.

PORT(PortNumber) Use this operand to specify the port number to be used for IP communication. The PORT operand is required. Specify a value from

1 to 65535. Port numbers 1 to 1023 are reserved. Also, the client port number and server port number must match for the systems to communicate.

The default is None.

SERVER

This option specifies the type of system you want to set up. SERVER is mutually exclusive with CLIENT. Neither SERVER nor CLIENT must be specified when DFSMSrmm is used as a standard system.

PORT(PortNumber) Use this operand to specify the port number to be used for IP communication. The PORT operand is required. Specify a value from 1 to 65535. Port numbers 1 to 1023 are reserved. The port number must be the same for the client system and the server system to establish a network connection.

The default is None.

SERVERTASKS(number)

Use this operand to specify how many DFSMSrmm tasks should be available on the server to handle socket connections from client systems. DFSMSrmm uses this number to determine how many tasks are to be started for processing all client requests on this server. Specify a value from 1 to 999.

The number of local and server tasks you can use and still successfully start DFSMSrmm is limited by the size of the private region above and below 16MB. To start with more tasks, you will require a larger REGION size.

Default: 10.

Recommendation: Specify or accept the default value of 10 server tasks on each server system. Depending on the number of CLIENT systems, this value should be increased to allow 3 tasks per CLIENT. Most of the local tasks are rarely used by DFSMSrmm. You do not need more server tasks than the sum of local tasks across your CLIENT systems.

6.3.3 Updating EDGRMMxx for a client

In the environment shown in Figure 6-1 on page 197, we define four DFSMSrmm client systems: two in SYSPLEX 1 and two in Non-Plex 1. To implement a DFSMSrmm client system, update your EDGRMMxx PARMLIB member to define the CATSYSID, CDSID and CLIENT operands. You can use the same DFSMSrmm client definitions as shown in Figure 6-8 for SYSA and SYSB. All other EDGRMMxx PARMLIB definitions are not shown in this figure.

OPTION	OPMODE(P)	/* operating mode	*/ -
	CATSYSID(SYSA,SYSB)	/* Catalog sysid(s)	*/ -
	CDSID(RMMPLEX1)	/* CDS ID	*/ -
	CLIENT(SERVERNAME(9.12.6.9) PORT(9000))	/* WTSC Server	*/ -
	LOCALTASKS(10)	/* number of subtasks	*/

Figure 6-8 EDGRMMxx PARMLIB options for client SYSA and SYSB

For the both systems SYSC and SYSD, you can use the CATSYSID, CDSID, and CLIENT operand as shown in Figure 6-9. All other EDGRMMxx PARMLIB definitions are not shown in this figure.

OPTION	OPMODE(P)	/* operating mode	*/ -
	CATSYSID(SYSC,SYSD)	/* Catalog sysid(s)	*/ -
	CDSID(RMMPLEX1)	/* CDS ID	*/ -
	CLIENT(SERVERNAME(9.12.6.9) PORT(9000))	/* WTSC Server	*/ -
	LOCALTASKS(10)	/* number of subtasks	*/

Figure 6-9 EDGRMMxx PARMLIB options for client SYSC and SYSD

6.3.4 Updating EDGRMMxx for a server

As shown in Figure 6-1 on page 197, we have only one DFSMSrmm server in SYSPLEX 2 on system SYSG. To implement a DFSMSrmm server system, update your EDGRMMxx PARMLIB member as shown in Figure 6-10 to define the CATSYSID, CDSID and SERVER operands. All other EDGRMMxx PARMLIB definitions are not shown in this figure.

OPTION	OPMODE(P)	/* operating mode	*/ -
	CATSYSID(SYSG,SYSH)	/* Catalog sysid(s)	*/ -
	CDSID(RMMPLEX1)	/* CDS ID	*/ -
	SERVER(PORT(9000) SERVERTASKS(12))	/* for 4 clients	*/ -
	LOCALTASKS(10)	/* number of subtasks	*/

Figure 6-10 EDGRMMxx parmlib options for server SYSG

6.4 Updating the DFSMSrmm CDSID (optional)

Update the CDS control record to specify a unique CDS identifier (CDSID) that must be specified in each EDGRMMxx PARMLIB member using this DFSMSrmm CDS.

6.4.1 Sample EDGUTIL job control

To set or update a CDSID in the DFSMSrmm CDS control record, use the EDGUTIL utility with a PARM of UPDATE.

Note: To create or update the control record, the user of EDGUTIL must have UPDATE or higher RACF access to the DFSMSrmm CDS.

Example 6-9 shows a sample JCL that you can use to set or update the DFSMSrmm CDS control record with a CDSID of RMMPLEX1.

Example 6-9 Sample EDGUTIL job control

```
//UTIL      EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD  SYSOUT=*
//MASTER   DD  DISP=SHR,DSN=RMM.CONTROL.DSET
//SYSIN     DD  *
CONTROL CDSID(RMMPLEX1)
/*
```

CDSID

This option specifies one-to-eight alphanumeric characters that identify the CDS by name. There is no default.

At DFSMSRmm startup time, DFSMSRmm matches this CDSID value with the CDSID operand in parmlib member EDGRMMxx. The CDSID value in EDGUTIL sets or changes the CDS ID. EDGUTIL does not validate the CDSID in the CDS control record. It simply sets the new value into the control record.

If you do not specify an ID for the CDS, you can set it the first time DFSMSRmm is started with this CDS by setting the CDSID operand in EDGRMMxx.

Do not change the CDSID value after you set it, because changing it can affect inventory management. You can change the value only when no data sets exist that were created on the system or when the WHILECATALOG option for vital record specifications is not in use.

6.5 Updating your firewall (optional)

You must update your firewall to ensure that communication between DFSMSRmm clients and servers is allowed only for the defined IP addresses and ports. The DFSMSRmm subsystem does no authentication, encryption, or verification of connect requests received on the server other than to verify that it is a valid DFSMSRmm request and that CDS IDs match. Also consider using RACF to protect the use of the IP addresses defined for DFSMSRmm and limit use of the IP address to the DFSMSRmm started task.

6.6 Starting DFSMSRmm in an RMMplex

After you update the EDGRMMxx PARMLIB member and you optionally check your firewall definitions, start or restart DFSMSRmm on your server and client.

Note: By default, the RMM subsystem address space starts unchanged, as a standard system, unless one of the new PARMLIB command operands is used to identify the system either a client or a server.

6.6.1 Starting and restarting DFSMSRmm on your client and server

To start DFSMSRmm using the default procedure name and PARMLIB member, enter the MVS command shown in Figure 6-11.

```
S DFRMM
```

Figure 6-11 DFSMSRmm start command

If you want, start DFSMSRmm with different parameters than the default. See Figure 6-12 how you can use the MVS command.

```
S DFRMM,M=xx,DSN=parmlib_dataset
```

Figure 6-12 DFSMSRmm start command using options

Where:

M	Specify the suffix of the EDGRMMxx member in PARMLIB.
DSN	Specify a different PARMLIB data set name.

After this command, you see the message shown in Figure 6-13.

```
EDG0105I DFSMSrmm SUBSYSTEM INITIALIZATION COMPLETE
```

Figure 6-13 EDG0105I initialization message

If the subsystem interface was not activated before the command, you see the message shown in Figure 6-14.

```
EDG0103D DFSMSrmm SUBSYSTEM INTERFACE IS INACTIVE - ENTER "IGNORE",  
"CANCEL" OR "RETRY"
```

Figure 6-14 EDG0103D initialization message

Enter **RETRY** to continue with the initialization. After that, you must receive the EDG0105I message.

You can restart DFSMSrmm using the MVS MODIFY command. For example, to restart DFSMSrmm and implement new PARMLIB options, specify the command shown in Figure 6-15.

```
F DFRMM,M=01
```

Figure 6-15 Modify DFRMM command

Where:

M	specifies the suffix of the EDGRMMxx member in PARMLIB.
----------	---

After you enter this command, you see the initialization completion message as shown in Figure 6-16.

```
EDG0105I DFSMSrmm SUBSYSTEM INITIALIZATION COMPLETE
```

Figure 6-16 EDG0105I initialization message after restart

Consideration for the DFSMSrmm server start

When a server subsystem starts, identify some basic information for TCP/IP to the DFSMSrmm subsystem using the EDGRMMxx PARMLIB OPTION SERVER operand. After the startup of the standard subsystem, DFSMSrmm communicates with TCP/IP and prepares to handle DFSMSrmm requests from client systems.

The server verifies that TCP/IP and the specified PORT are available, determines its own default IP address, and informs the operator that initialization is successful or issues error messages. In addition to normal DFSMSrmm subsystem operation, such as processing of local requests, the server waits for and accepts connection requests. It also processes requests from DFSMSrmm client systems. If the server task is unable to start successfully, you can still use DFSMSrmm as a standard subsystem until the server task problem is resolved.

Consideration for the DFSMSrmm client start

When a client subsystem starts, identify some basic information for TCP/IP to the DFSMSrmm subsystem via the EDGRMMxx PARMLIB OPTION client operand.

During startup, the subsystem communicates with TCP/IP and prepares to send DFSMSrmm requests from the client to a server system. The client verifies that TCP/IP is available and that the server can be reached with the specified PORT, determines its own IP address, verifies the CDS ID matches that of the server, and informs the operator that initialization is successful or issues error messages.

DFSMSrmm ignores any parmlib options not required for a client system. The client can connect to only one server system at a time. If the defined server is not available, the client can issue WTOR EDG0358D and wait for either the operator to reply with CANCEL, RETRY, or for the server to be available for connection.

If the server is not available, DFSMSrmm fails any requests with an I/O error that require server processing which are in progress when a refresh (F DFRMM,M=xx) is requested. Existing error recovery in DFSMSrmm can handle this and provide recovery once any new server is available.

DFSMSrmm issues a WTOR when a TCP/IP error occurs. RETRY processing relies on the operator replying to the WTOR. If the error is resolved and the operator has not replied to the WTOR, DFSMSrmm processing automatically continues and cancels the outstanding WTOR. When the client is started, no further verification of the server availability is performed unless a DFSMSrmm request is to be processed.

It may happen that a request is processed and server communication fails or a time out occurs, and retry still cannot process the request. In this case, a WTOR, DFSMSrmm issues the message EDG0358D to describe the error. It prompts the operator to reply CANCEL or RETRY, and automatic retry by DFSMSrmm is possible.

When the CLIENT is started, no further verification of the SERVER availability is performed unless an RMM request is to be processed (no heartbeat is maintained). When a request is processed and SERVER communication fails or a time out occurs, and retry still cannot process the request, a WTOR EDG0358D message is issued that details the error and allows operator involvement in error recovery. CANCEL or RETRY is possible.

6.7 Managing volumes in an RMMplex

You can have different environments in an RMMplex. For example, you can have fully shared catalogs, unshared catalogs, a shared RACF database, or an unshared RACF database. The following section describes the differences.

6.7.1 Catalogs in an RMMplex

You can implement DFSMSrm client/server support regardless of whether your catalogs are fully shared across all of the systems in an RMMplex. However you must correctly identify to DFSMSrmm whether catalogs are shared. It is your choice whether you synchronize the catalogs with the DFSMSrmm CDS. The catalog status for all data sets is maintained in the server system CDS.

Based on unshared catalogs between client and server, the catalogs and the CDS must be synchronized if catalog retention or the UNCATALOG(Y/S) option is used. To synchronize the CDS and the catalogs, use CATSYSID(list_of_sysids) and follow the process for unshared

catalogs. You can run EDGHSKP with CATSYNCH, VERIFY, and EXPROC on the client system.

With unshared catalogs, the UNCATALOG(YES) PARMLIB option, on the server system, cannot be honored for data sets created on the client systems. The server has no way to communicate with the client to initiate uncatalog processing. However, when processing is requested from the client, there is special recognition and handling of the request so that any catalog or RACF profile updates can be initiated on the client system.

DFSMSrmm uncatalogs data sets on a volume, when you specify the EDGRMMxx PARMLIB OPTION UNCATALOG(Y) command under these conditions:

- ▶ A volume is returned to scratch status, and DFSMSrmm uncatalogs all the data sets on the volume.
- ▶ The RMM DELETEVOLUME FORCE subcommand is issued for a volume, and the DFSMSrmm uncatalogs all the data sets on the volume.
- ▶ The RMM CHANGEVOLUME DSNAME subcommand is issued for a volume, and DFSMSrmm uncatalogs all the data sets on the volume. If the data set name specified on the RMM CHANGEVOLUME subcommand matches the data set name on the volume, then DFSMSrmm only uncatalogs subsequent data sets.
- ▶ The RMM DELETEDATASET subcommand is issued for a data set, and DFSMSrmm uncatalogs the data set. Also, DFSMSrmm uncatalogs all data sets recorded on the same volume with higher data set sequence numbers.
- ▶ A tape data set is overwritten, DFSMSrmm uncatalogs the data set. Also, all data sets recorded on the same volume with higher data set sequence numbers are uncataloged.

DFSMSrmm uncatalogs data sets on a volume when you specify the EDGRMMxx PARMLIB OPTION UNCATALOG(S) command. It does this only under the condition that, when a volume is returned to scratch status, DFSMSrmm uncatalogs all the data sets on the volume.

The default is UNCATALOG(N) unless DFSMSrmm is running in protect mode. DFSMSrmm uses the UNCATALOG default of Y when DFSMSrmm is running in protect mode.

Refer to *Chapter 9, "Catalog synchronization" on page 333*, for information about how to synchronize your MVS catalogs.

6.7.2 RACF considerations in an RMMplex

You may have switched on RACF TAPEVOL or TAPEVOL and TAPEDSN on your client and server. You may also have specified in the EDGRMMxx PARMLIB OPTION TPRACF(P) or TPRACF(A) to request DFSMSrmm tape security support. In this case, you can implement DFSMSrmm client/server support regardless of whether your RACF data base is fully shared across all of the systems in an RMMplex. However you must correctly identify to DFSMSrmm whether RACF database is shared.

Note: Check the possibility to use the new SAF tape security. Refere to chapter *Chapter 13, "Tape security with z/OS V1.8" on page 485* for more information how to implementing it.

With unshared RACF data bases, the RACF TAPEVOL profiles and TVTOC entries are maintained only on the server system under these conditions:

- ▶ Confirm that you have returned the volume to its owner and the volume was protected by a RACF TAPEVOL profile.

- ▶ Confirm that you have erased the volume.
- ▶ Confirm that you have initialized the volume.
- ▶ Confirm that you have performed all activities necessary prior to returning the volume to scratch status.
- ▶ A volume is returned to scratch status, and DFSMSrmm deletes the RACF TAPEVOL profile.
- ▶ The RMM DELETEVOLUME FORCE subcommand is issued for a volume, and DFSMSrmm deletes the RACF TAPEVOL profile
- ▶ The RMM CHANGEVOLUME DSNAME subcommand is issued for a volume, and DFSMSrmm deletes all the TVTOC entries for this volume. If the data set name specified on the RMM CHANGEVOLUME subcommand matches the data set name on the volume, then DFSMSrmm only deletes TVTOC entries for subsequent data sets.
- ▶ The RMM DELETEDATASET subcommand is issued for a data set, and DFSMSrmm deletes the TVTOC entry. DFSMSrmm also deletes all TVTOC entries defined for this volumes with higher data set sequence numbers.
- ▶ A tape data set is overwritten, DFSMSrmm deletes the TVTOC entries. Also, all TVTOC entries on the same volume with higher data set sequence numbers are deleted.

Refer to chapter Appendix A, “Security topics” on page 673 for detailed RACF information.

6.7.3 System-managed libraries in an RMMplex

You can perform the following tasks for a system-managed tape library that is known to the client and not the server:

- ▶ Ejecting a volume from a system-managed tape library
- ▶ Adding volumes to a system-managed tape library using STATUS(VOLCAT)
- ▶ Changing volume attributes that are also maintained in the TCDB
- ▶ Running expiration processing
- ▶ Confirm moves for exported stacked volumes
- ▶ Running EDGUTIL VERIFY with the TCDB and optionally the Library Manager

6.7.4 HOUSEKEEP processing in an RMMplex

The utilities provide restricted functions when run on a client or server system. Before running EDGHSKP, you must define several data sets. The data sets that are used by both the EDGHSKP utility and the DFSMSrmm subsystem address space must be pre-allocated and cataloged.

EDGHSKP

You can run all inventory management functions except BACKUP on a client system. However, because there is no direct connection to the CDS, the elapsed times will be longer when run on a client system. We recommend that you run all inventory management functions other than CATSYNCH and EXPROC on either the server or a standard DFSMSrmm system.

Note: When you attempt backup on a client system, the utility ends with RC16 and DFSMSrmm issues message EDG6137E stating that it cannot be run on a client system.

EDGUTIL

When you run EDGUTIL on a client system, the utility can only process a DFSMSrmm CDS that is not in use by DFSMSrmm. Because there is no CDS for a client system, you cannot run with the active CDS. However, you can recover a backup copy of the DFSMSrmm CDS to the client system to run VERIFY(VOLCAT), VERIFY(SMSTAPE), and MEND(SMSTAPE).

When you run EDGUTIL on a server system, you cannot access the TCDB or library for tape libraries that you can only access from the client system. When you run EDGUTIL on a client system and do not specify the MASTER DD, DFSMSrmm issues message EDG6101E and the utility ends with return code 12.

EDGBKUP

You cannot process an active DFSMSrmm CDS on a client system. All functions are available on a client system as long as you provide the DD statements for the CDS and journal. This enables you to use the BACKUP and RESTORE options independent of the DFSMSrmm subsystem. When you run EDGBKUP on a client system and do not specify the MASTER DD or JOURNAL DD, DFSMSrmm issues message EDG6101E and the utility ends with return code 12.

Figure 6-17 shows the correct sequence that you should run your daily housekeeping jobs so that all components are updated correctly and always are in sync.

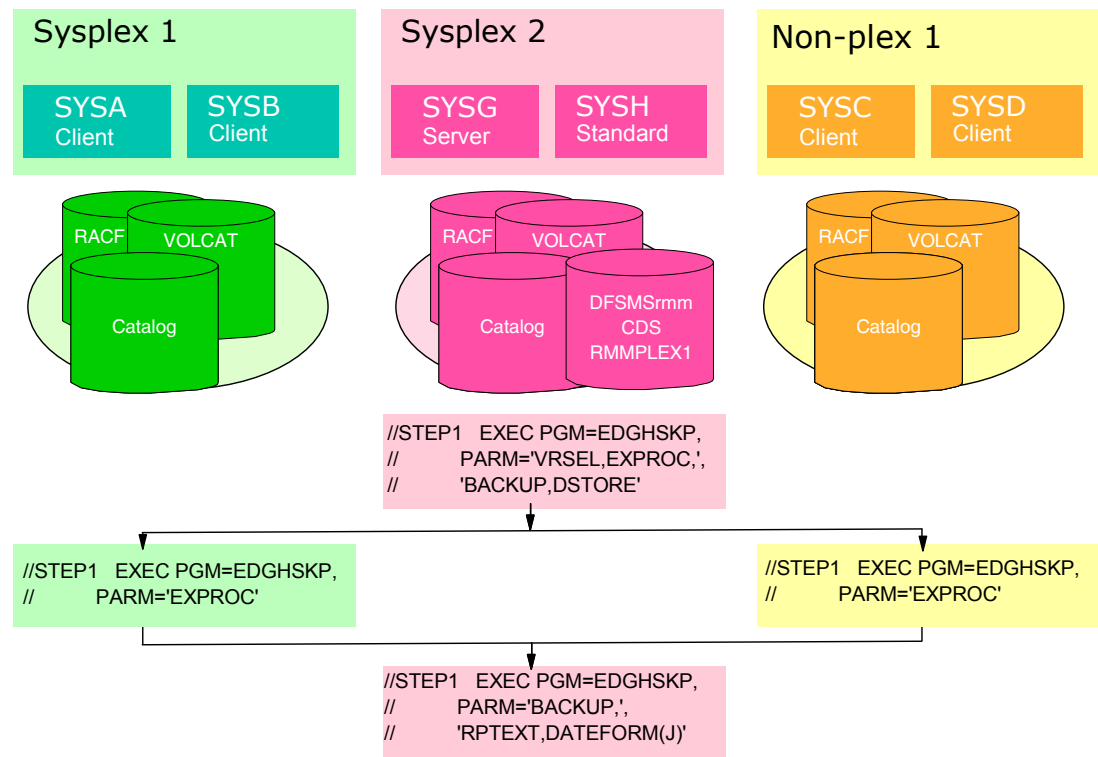


Figure 6-17 DFSMSrmm housekeeping in an RMMplex

6.8 Operator commands

The DFSMSrmm client system processes most requests by communicating with the server but also by processing those local requests which can be completely processed on the client system.

When multiple tasks are being processed, DFSMSrmm maintains a queue in FIFO order. You can use the operator command MODIFY DFRMM,Q A to display the tasks and a summary of the queues. The DFSMSrmm server processes local requests from DFSMSrmm code on the server unchanged. In addition, client requests are accepted and processed synchronously while the requester on the client waits. There is no queue of client requests maintained on the server. The request queues maintained on the server are for local requests only. When you list the active tasks, using QUERY ACTIVE, the active local requests are listed together with the accepted client requests.

The QUERY ACTIVE shown in Figure 6-18 displays the active and queued tasks. To also support IP server tasks, the Q A command lists those subtasks as well and includes them in the count of total subtasks, count of active subtasks, and a list of active subtasks with a summary of the requests they process: sysid, jobname, function code, status, and subtask token.

MODIFY DFRMM,QUERY ACTIVE
or
F DFRMM,Q A

Figure 6-18 DFRMM query active command

In addition, the output from Q A is a changed to be a single, multiline WTO issued as a command response using CART/CONSID. Example 6-10 shows the result of the Query Active command.

Example 6-10 DFSMSrmm status display

EDG1119I DFSMSrmm STATUS IS ACTIVE. JOURNAL ENABLED.
EDG1114I LOCAL TASKS 10, ACTIVE 0, SERVER TASKS 0, ACTIVE 0
EDG1118I 0 QUEUED REQUESTS, INCLUDING 0 NOWAIT 0 CATALOG
EDG1121I DEBUG IS DISABLED, PDA TRACE LEVEL: 1-2-3-4-
EDG1101I DFSMSrmm MODIFY COMMAND ACCEPTED

Where:

sysid	SYSID value from the client, or the local SYSID (local requests)
Jobname	The job name of original requestor; for client requests, the job name on the client system
Function code	The RMM subsystem request function code, from SSOB. There is a new subsystem function code used specifically to handle client requests. All other are unchanged.
Status	IP communication status. ON of; LISTENing, CONNECTing, SENDing, READing, CLOSEing. (IUO: The status is taken from the QMCB)



Tailoring your DFSMSrmm environment

This chapter describes how to implement additional function, which is provided in DFSMSrmm or supports DFSMSrmm, and how to tailor DFSMSrmm to your environment.

The objectives of this chapter are to:

- ▶ Implement additional function that supports DFSMSrmm
- ▶ IBM Tivoli Workload Scheduler
- ▶ Setting up DFSMSrmm Common Time Support
- ▶ Dynamic Installation Exits
- ▶ Activating RACF TAPEVOL and TAPEDSN
- ▶ Maintaining your control data set
- ▶ Initializing and erasing volumes
- ▶ Disposition processing
- ▶ Software products and foreign tapes
- ▶ DFSMSrmm volume pools
- ▶ Deleting non-existent volumes
- ▶ Repairing actions

By the end of this chapter, you will have an understanding of the additional function in DFSMSrmm and how to customize it for your environment.

7.1 IBM Tivoli Workload Scheduler

To manage the resources defined to DFSMSrmm, you need to run the inventory management tasks, also known as *housekeeping* periodically. DFSMSrmm provides the EDGHSKP utility to help you perform inventory management. A job scheduling product, such as IBM Tivoli Workload Scheduler for z/OS (which incorporates Tivoli Operations Planning and Control (OPC)), is helpful to run these tasks.

IBM Tivoli Workload Scheduler for z/OS is a system management product that functions as a batch job scheduler. You can configure the typical DFSMSrmm batch job flow to IBM Tivoli Workload Scheduler for z/OS to run inventory management jobs regularly. This provides the following benefits:

- ▶ Prevents more than one EDGHSKP job from running at the same time
- ▶ Avoids jobs failing because inventory management is already running
- ▶ Prevents EDGUTIL VERIFY jobs from running at the same time as an inventory management jobs
- ▶ Handles the dependencies between jobs

The sample IBM Tivoli Workload Scheduler for z/OS stream included in the DFSMSrmm SAMPLIB library proposes the following steps when running inventory management. The name in parenthesis is the sample member name included in SAMPLIB:

- ▶ Preparation (EDGJHKPA)
- ▶ CDS verify (EDGJVIFY)
- ▶ CDS backup (EDGJBKP1, EDGJBKP2)
- ▶ Inventory management (EDGJDHKP(daily), EDGJWHKP(weekly))
- ▶ Erasing and labeling volumes (EDGJINER)
- ▶ Scratch processing (EDGJEXP)
- ▶ Scratch reporting (EDGJSCR)
- ▶ Ejecting volumes from a system-managed tape library (EDGJEJC)
- ▶ Producing reports (EDGJMOVE)
- ▶ Move confirmation (EDGJCMOV)

Also included in SAMPLIB is the IBM Tivoli Workload Scheduler for z/OS batch loader sample JCL, EDGJLOPC.

7.1.1 IBM Tivoli Workload Scheduler overview

This section introduces the basic IBM Tivoli Workload Scheduler for z/OS concepts, which you need to be familiar with for a better understanding of the samples provided by DFSMSrmm. Note that the Tivoli OPC product has been renamed to *IBM Tivoli Workload Scheduler for z/OS*.

The basic IBM Tivoli Workload Scheduler for z/OS components are:

- ▶ **Calendars** contain the information about the working days and free days for work planning purposes.
- ▶ **Workstations** are the logical place where work occurs. There are three types of workstations:
 - Computer workstations
 - Printer workstations
 - General workstations

The majority of the IBM Tivoli Workload Scheduler for z/OS operations run in computer workstations. The activity that occurs at each workstation is known as *operation*. Each operation has a *job name*, *workstation ID* and *operation number*.

- **Applications** are units of production work that IBM Tivoli Workload Scheduler for z/OS schedules. Each application can be one or more operations.
- **Special resources (optional)** represents any type of limited resources such as data sets, tape drives, or communication lines.

IBM Tivoli Workload Scheduler for z/OS can schedule jobs on a planned basis or dynamically using event-triggered scheduling. The *Event-Triggered Tracking* (ETT) gives you a method of controlling and tracking workload that cannot be planned in advance. Some of the situations that we can consider in this special scheduling are the journal full condition and low-on-scratch situation.

The DFSMSrmm batch loader sample helps you in the definition of applications and special resources. Calendars and workstations must be defined using the IBM Tivoli Workload Scheduler for z/OS dialog.

7.1.2 Scheduling your inventory management tasks

The schedule of inventory management tasks should be defined before running. You can schedule each DFSMSrmm utility to run on its own, or schedule many activities to run together in a sequence. Some activities need to be performed daily like vital records processing, while other activities may be done less frequently.

Table 7-1 shows our recommendation. This may help you to decide how frequently to run DFSMSrmm utilities for your requirement.

Table 7-1 *Schedule inventory management tasks*

Activity	Frequency
Processing vital records daily	Daily
Performing expiration processing	Daily
Performing storage location management	Weekly
Creating an extract data set	Daily
Initializing and erasing volumes	Weekly
Creating security and audit reports	Monthly
Backing up the DFSMSrmm control data set (CDS)	Daily
Backing up the DFSMSrmm journa	Daily
Verifying the CDS	Monthly

Note: If you are using DFSMSrmm z/OS 1.13 or higher you do not need to run VRSEL processing unless any volumes are defined with the VRSEL retention method. Only EXPROC processing is required to handle expiration of all volumes managed by the EXPDT retention method.

We can use the IBM Tivoli Workload Scheduler for z/OS batch loader sample JCL, EDGJLOPC to define the schedule for the activities shown in Table 7-1. The sample JCL defines the following applications based on the execution frequency:

- Daily applications
 - RMMBKP

- RMMHKPD
- RMMPOST
- RMMEXP
- ▶ Weekly applications
 - RMMWEEK
 - RMMMOVE
- ▶ Monthly applications
 - RMMMTHT
- ▶ Other applications
- ▶ RMMVRSVER is run only when a previous application finished with RC=8.

Figure 7-1 shows the flow of the IBM Tivoli Workload Scheduler for z/OS batch loader sample.

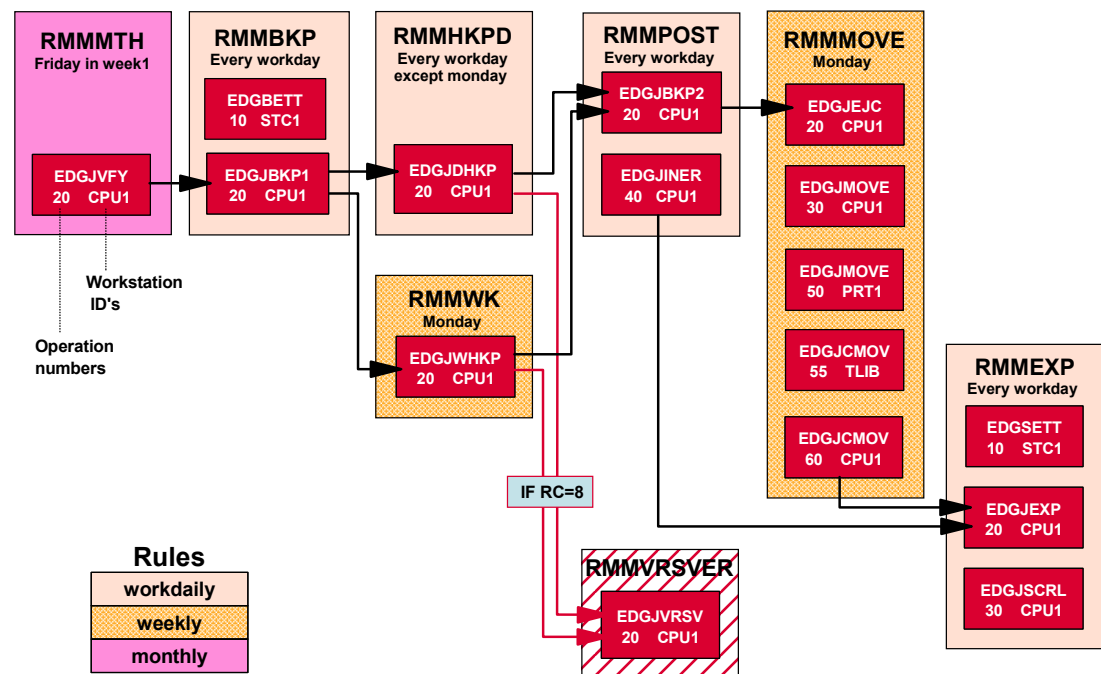


Figure 7-1 DFSMSrmm IBM Tivoli Workload Scheduler for z/OS job flow

In each application, you can see the different operations included. These are the jobs that perform each of the steps introduced at the beginning of the chapter. There are two more jobs: EDGBETT and EDGSETT that are defined under ETT. EDGBETT triggers RMMEXP application in case a low-on-scratch condition is reached, and EDGSETT triggers RMMBKP application in case the journal full threshold is reached.

When you have decided on your schedule, you should customize the EDGJLOPC sample to update the sample schedule to meet your needs. Refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for more details.

The following sections cover the different activities that are performed in each of the steps suggested that comprise the whole inventory management process.

7.1.3 Preparation

This first step is performed by sample job EDGJHKPA. This job executes the EDGPHKPA procedure for allocating the inventory management files needed. Inside the EDGPHKPA procedure there are some others, each allocating one different file:

- ▶ EDGPMSGGA: Allocates the (+1) version of MESSAGE file
- ▶ EDGPVRSR: Allocates the (+1) versions of REPORT and ACTIVITY files
- ▶ EDGPACTA: Allocates the files needed for the ACTIVITY reports:
 - REPORT.RUNINFO
 - REPORT.VRS
 - REPORT.VRS.SUMMARY
 - REPORT.RETDATE
 - REPORT.RETDATE.SUMMARY
 - REPORT.MATCHVRS
 - REPORT.MATCHVRS.SUMMARY
 - REPORT.SUBCHAIN
 - REPORT.SUBCHAIN.SUMMARY
- ▶ EDGPRPTA: allocates the (+1) version of the REPTTEXT file

This procedure also allocates the SCRDATE file.

7.1.4 CDS verify

A sample IBM Tivoli Workload Scheduler for z/OS job EDGJVIFY is provided for verifying the CDS contents. The job invokes a EDGPVIFY procedure, which uses the EDGUTIL program with PARM=VERIFY. The expected completion is the return code 0. You must perform manual recovery before dependent jobs can be run when the return code is greater than 0.

For more information about verifying the CDS, refer to 7.11.1, “Using the VERIFY parameter” on page 268.

7.1.5 CDS backup

Sample jobs EDGJBKP1 and EDGJBKP2 are provided in SYS1.SAMPLIB. These jobs are used to backup the CDS and journal, and to clear the journal after a successful backup. The jobs invoke the EDGPBKUP procedure, which executes EDGHSKP program with PARM='BACKUP(*type*)', with *type* being one of DSS or AMS. If you select DSS, then you can use concurrent copy for backing up the CDS. It also allocates the files needed for the backups.

EDGPMSGC is another procedure inside the jobs, which allocates other files needed for the backup process.

7.1.6 Inventory management

DFSMSrmm manages the removable media resources and performs the inventory management (housekeeping) according to the policies defined to DFSMSrmm as vital record specifications (VRSs).

The inventory management processing includes the following tasks:

- ▶ **Inventory management processing (VRSEL)**
The purpose of vital records processing is to identify the volumes that should be retained and how they should be moved based on the VRSs you defined.

Refer to Chapter 11, “Policy management” on page 375, for detailed information about how VRSs work.

► **Expiration processing (EXPROC)**

This processing is used to expire volumes to scratch status according to the management policy. DFSMSrmm calls the EDGUX200 installation exit when running the expiration processing. This exit is called every time a volume is identified as ready to return to scratch. During expiration processing, DFSMSrmm:

- Checks the expiration date to identify volumes that are ready to expire
- Determines the type of release action needed for the volumes that have reached their expiration date
- Manages the release actions for volumes before returning them to the scratch pool or removing them from the library
- Removes the catalog entry and uncatalogs all parts of the data set across all volumes in the set for a multivolume data set
- Optionally, updates RACF tape security profiles for volumes returning to scratch status
- If UNCATALOG(Y or S) is specified in PARMLIB member EDGRMMxx, DFSMSrmm uncatalogs all data sets for volumes returning to scratch status.
- When the EDGRMMxx PARMLIB option RETAINBY(SET) is in use, DFSMSrmm ensures all volumes in a set are not expired if any one volume does not expire.

► **Storage location management (DSTORE)**

Requests storage location management to set the destination for the volume and optionally assigns the exact shelf location to be used. Storage location management processing assigns volume destinations and sets required volume destinations based on the results of other inventory management functions.

You can select to perform DSTORE by location, specifying the EDGHSKP parameter as shown here:

```
DSTORE(LOCATION(from_location:to_location,...))
```

Note the following explanation:

- `from_location:to_location` indicates a pair of location names separated by a colon. The `from_location` is the current location a volume should move from.
- `to_location` is the name of the required location where a volume should move to. If you omit `to_location`, DFSMSrmm uses “*” as the default.

You can specify one to eight pairs of `from_location:to_location` names. DSTORE processing is performed then for a volume, if it matches to one of the location pairs specified.

The `from_location` and `to_location` names can be specified in one of the following ways:

- **Specific:** A specific location name is one to eight characters. The location names that you specify are not validated against the DFSMSrmm LOCDEF entries or the name of SMS libraries.

For example, SHELF means that just this one location is included.

- **Generic:** A generic location name is one to seven characters followed by any generic characters valid in a single data set qualifier.
 - `ATL*` means that all location names starting with the characters ATL are included.
 - An asterisk (*) means that all locations are included.
 - The notation `ABC%%%` refers to all locations starting with ABC and with any other character up to a total of a 6 character name.

- ▶ **Report extract (RPTEXT)**

When running inventory management, you can choose to request an extract data set, which can be used to create various reports such as volume movement or inventory.

- ▶ **Trial run (VERIFY)**

Trial run vital record processing does not change data set and volume information in the DFSMSrmm CDS. Use trial run vital record processing to analyze the effect that your movement and retention policies have.

To run inventory management functions, sample IBM Tivoli Workload Scheduler for z/OS jobs EDGJDHQP and EDGJWHQP are provided in SYS1.SAMPLIB.

- ▶ EDGJDHQP (daily job): Invokes the following procedures:
 - EDGPHQP: Runs the EDGHSQP program with PARM='VRSEL,EXPROC,RPTEXT'.
 - EDGPMSGC, EDGPVRSR, and EDGPRPTA: Allocate other needed files.
- ▶ EDGJWHQP (weekly job): Invokes the following procedures:
 - EDGPHQP: Runs the EDGHSQP program with PARM='DSTORE,VRSEL,EXPROC,RPTEXT'.
 - EDGPMSGC, EDGPVRSR, and EDGPRPTA: Allocate other needed files.

7.1.7 Erasing and labeling volumes

This step automates the labeling and erasing of tape volumes. Sample IBM Tivoli Workload Scheduler for z/OS job EDGJINERS are provided in SYS1.SAMPLIB. DFSMSrmm searches for volumes with INITIALIZE or ERASE actions pending and labels them after optionally erasing them. The job invokes the EDGPINER procedure, which runs the EDGINERS utility with the required parameters.

Refer to 7.6, "Initializing and erasing volumes" on page 254, for detailed information about erasing and initializing tape volumes.

7.1.8 Scratch processing

This step is needed to process volumes that are returning to scratch status. A sample IBM Tivoli Workload Scheduler for z/OS job is available in SYS1.SAMPLIB member EDGJEXP. This job invokes the EDGPEXP procedure, which runs EDGHSQP with the EXPROC parameter to perform expiration processing. It also allocates some other needed files.

For more information about the expiration processing performed as part of the (EXPROC), refer to 7.1.6, "Inventory management" on page 219.

7.1.9 Scratch reporting

Perform this step when you need to obtain information about volumes in scratch status. Sample job EDGJSCRL is provided in SYS1.SAMPLIB.

This job invokes the following procedures:

- ▶ EDGPRPTX: Creates an updated DFSMSrmm report extract file. For doing this, it runs the EDGHSQP utility with RPTEXT parameter. It allocates the new report extract file using the procedure EDGPRPTA.
- ▶ EDGPSCRL: It uses EDGRPTD reporting utility for generating the latest scratch lists using files SCRLIST and NEWSCL.

7.1.10 Ejecting volumes from a system-managed tape library

This task ejects volumes from system-managed tape libraries when volume movement is required. You should also consider VTS export processing in this step. Similar processing is required for any non-system-managed volumes, which you have in your library.

A sample IBM Tivoli Workload Scheduler for z/OS job is available in SYS1.SAMPLIB member EDGJEJC. This job invokes EDGPEJC procedure. You must execute this procedure once for each library in your installation, changing the library name (FROM), destination (TO), and output station (EJECT) in each case.

The EDGPEJC procedure requires two steps:

1. Create a list of volumes to be moved from a system-managed library. The list is created using the command shown in Figure 7-2.

```
RMM SV VOL(*) OWN(*) LIMIT(*) LOC(&FROM) DEST(&TO) INTRANSIT(N) -  
CLIST("RMM CV", "EJECT(&EJECT)")
```

Figure 7-2 RMM SEARCHVOLUME subcommand to create eject commands

The parameter CLIST allows you to create a set of commands for all the volumes in the generated list.

1. Figure 7-3 shows you the commands generated in the previous step for ejecting the volumes.

```
RMM CV volser EJECT(&EJECT)  
RMM CV volser EJECT(&EJECT)  
....
```

Figure 7-3 RMM CHANGEVOLUME subcommand to eject the volumes

7.1.11 Producing reports

When running inventory management, you can choose to request an extract data set that can be used to create various reports such as volume movement or inventory. The extract data set contains information from the CDS. You can use the extract data set as input to EDGRPTD reporting utility, the EDGRRPTE exec, and the report generator to create reports. The extract data set may contain extended extract records.

See *DFSMSrmm Reporting* for information about using EDGRPTD and EDGRRPTE.

This step is used to generate movement reports. A sample IBM Tivoli Workload Scheduler for z/OS job is available in SYS1.SAMPLIB member EDGJMOVE. It invokes the following procedures:

- ▶ EDGPRPTX: Creates an updated report extract file, running EDGHSKP utility with RPTEXT parameter.
- ▶ EDGPMOVE: Uses the updated report extract file as input to EDGRPTD reporting utility to create movement reports. The produced reports are:
 - RDYTOSCR: Volumes to be moved from locations to home location
 - TOSTRCK: Report sorted by rack number including volumes to be moved:
 - From SHELF to storage locations
 - Between system-managed libraries

- From system-managed libraries to storage locations
- TOSTOWN: Report sorted by owner including volumes to be moved:
 - From SHELF to storage locations
 - Between system-managed libraries
 - From system-managed libraries to storage locations
- FMSTBIN: Report sorted by bin number including volumes to be moved:
 - From storage location to SHELF
 - Between storage locations
 - From storage locations to system-managed libraries
- FMSTOWN: Report sorted by owner including volumes to be moved:
 - From storage location to SHELF
 - Between storage locations
 - From storage locations to system-managed libraries

7.1.12 Move confirmation

Confirmation is needed for volume movement to locations outside the tape library. After the inventory management processing such as vital record processing and the storage location management processing, DFSMSrmm needs your confirmation about volume movement.

A sample IBM Tivoli Workload Scheduler for z/OS job EDGJCMOV is provided in SAMPLIB for performing the move confirmation actions. A global confirmation command is issued, which means that the confirmation is for multiple volumes. The actual confirmation of movement for each volume is done in the next inventory management run.

The procedure EDGPCMOV is invoked, which issues the global confirm move command as shown in Figure 7-4.

```
RMM CV * CMOVE(&FROM,&TO,&TYPE)
```

Figure 7-4 RMM CHANGEVOLUME subcommand to confirm outstanding moves

You should provide location names valid for your installation in the parameters FROM, TO, and TYPE, or use ALL to confirm all outstanding moves.

7.1.13 Additional considerations

There are some additional considerations to take into account for inventory management related tasks.

Managing tape with permanent errors

DFSMSrmm tracks the permanent I/O errors for a volume, and marks it as it needs to be replaced if it is set with a return to scratch release action. DFSMSrmm does not identify those volumes that need the “return to owner” release action.

You can use the RMM SEARCHVOLUME subcommand, the extract data set or the DFSMSrmm ISPF dialog to check for volumes with temporary errors.

Managing open data sets

In the MESSAGE file, DFSMSrmm lists tape data sets that might be open as well as messages describing the expiration processing.

These open data sets might be produced by jobs that are currently active. However, some of them also might result from a job failure, meaning the data set was never closed. Review this list and make the necessary corrections. During vital record processing, DFSMSrmm matches open data sets to VRSs using the reserved data set name mask OPEN, or to VRSs defined with data set name masks.

Other considerations

You can perform some activities only under specific conditions:

- ▶ If you use system-managed tape library in your environment, and there are multiple TCDBs, you must run inventory management expiration processing at least once against each TCDB.
- ▶ When you want to retain data sets based on the catalog information, or if you specify the UNCATALOG with Y or S in EDGRMMxx to indicate that catalog processing is required, you must run inventory management on a system that has access to the catalogs, or you have to enable catalog synchronization.
- ▶ In an unshared user catalog environment, specify OPTION command with CATSYSID operand in the EDGRMMxx to use DFSMSrmm. You must ensure CDS and user catalogs are synchronized and run expiration processing on each different catalog environment.
- ▶ The DFSMSrmm subsystem address space performs all inventory management functions, except backing up the CDS. It is EDGHSKP that backs up the DFSMSrmm CDS.
- ▶ You can continue using RMM TSO subcommands and DFSMSrmm continues to record tape usage, while inventory management functions are running.
- ▶ At least one VRS must be defined in order to run DFSMSrmm vital record processing. The EDGRMMxx PARMLIB OPTION command VRSMIN operand provides control over this processing.

7.2 Setting up DFSMSrmm Common Time Support

Before DFSMSrmm common time support (UTC), also known as GMT, is enabled, all dates and times are stored in the DFSMSrmm control data set in local time. When the control data set is shared, and the sharing systems are set to run in different time zones, the local dates and times in the control data set may be from any of your systems. When you display information or extract records, you need to be aware of how the records were created, on which system, and where they may have been updated in order to interpret the dates and times shown. The same consideration also applies for records created or updated prior to enabling common time support because DFSMSrmm assumes they are times local to the system running the DFSMSrmm subsystem and converts the values based on that assumption.

When you enable common time support, DFSMSrmm maintains the records in the control data set in common time. Most date and time fields are paired together to enable an accurate conversion to and from common time and between different time zones. In some cases, DFSMSrmm has date fields in control data set records, and there is no associated time field. For these date fields, DFSMSrmm uses an internal algorithm that approximates conversion between time zones based on the time zone offsets involved.

Warning: Using the SET system command with either the DATE or the TIME keyword, or both, or replying to message IEA888A to run the system on future or past dates can affect the way that DFSMSrmm calculates local times. In order to get the correct results from DFSMSrmm processing when you need to test with future or past dates, you should alter the TOD clock and keep the time zone offset as before.

7.2.1 How DFSMSrmm using the Date and Time

Dates and times in DFSMSrmm are used mostly for reference, but also to calculate retention periods. Here are some examples:

Journal record prefix	contains date and time that the journal records were written, compared for forward recovery.
Record creation date and time	in any record stored in the DFSMSrmm control data set.
Expiration date	in volume and data set records; shows the date that the entity is expired, but not the time.
Tokens	in control data set records; used for various purposes but usually as a unique value to associate multi- volumes multi-files, or logical volumes and stacked volumes together
Reference dates	in control data set records

Note: Dates and times are always displayed to the user in the same format that they were recorded; in local time

All current uses of date and time in DFSMSrmm use 'TIME DEC' option of the TIME macro; this retrieves the local date and time. DFSMSrmm assumes that all systems that share a control data set (CDS), and therefore the JOURNAL as well, run on a synchronized time source.

Although in theory there is supposed to be a common date/time function available in DFSMSrmm and used for all date conversions, in practice it is used only when a date conversion is required or to get the current date in DFSMSrmm internal format. It is not used where the time is wanted or when the date is wanted not in internal format.

DFSMSrmm uses an internal date format which is derived via TIME DEC from the TOD; a 4 byte, 7 character packed decimal number YYYYDDDs. Using this value DFSMSrmm has no known limit to the dates that can be handled. The internal time is a 7 character packed decimal number also, HHMMSSs. DFSMSrmm does not need to upgrade to using a 128bit timer value, nor have any problems with the TOD clock rolling over in 2043.

7.2.2 Date and Time in a DFSMSrmm Client Server Environment

All dates and times used on the client system are local time. All dates and times used on the server are in local time. When data is retrieved from the DFSMSrmm control data set (CDS) and displayed by subcommand processing the dates and times used are exactly as stored in the DFSMSrmm CDS; no conversion from server time zone to client time zone is performed. Figure 7-5 on page 226 shows you an example when you have a need to use the DFSMSrmm common time support.

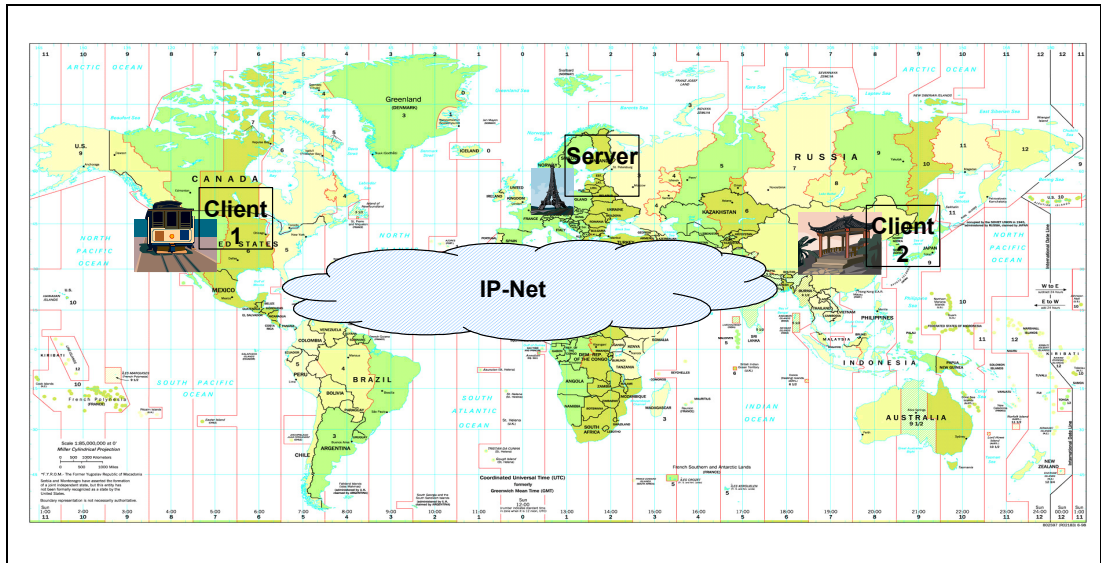


Figure 7-5 DFSMSrmm client server implementation

7.2.3 Enable Common Time Support

Before you can enable the DFSMSrmm common time support you must check that the TOD clocks (Time Of Day) of all systems in the RMMplex are set to UTC (Universal Time, Coordinated) .

To enable common time support, perform the following tasks:

1. Ensure all systems in the RMMplex have toleration maintenance installed or are at z/OSV1R8 or higher, and all applications dependent on the correct date and time information from DFSMSrmm are updated to support the new time zone support if required. DFSMSrmm subcommand output remains in local time, so most applications do not need to change unless they are to exploit the availability of the time zone offset.
2. Ensure the system time of day (TOD) clock is set to GMT on all systems in the RMMplex. It is common practice for the system to use local time based either on the TIMEZONE value in the CLOCKxx member of parmlib or from an external time source.
3. Run the EDGUTIL utility with UPDATE with the UTC(YES) operand on the CONTROL statement of the SYSIN file to enable common time support.

Important: You cannot disable common time support once it is enabled.

UTC(YES) enables DFSMSrmm common time support. Prior to enabling this support all systems in the RMMplex should have toleration maintenance installed or be z/OS V1R8 or higher and applications dependent on the correct date and time information from DFSMSrmm should be updated to support the new time zone support if required.

Figure 7-6 on page 227 shows you the result of a RMM TSO LISTCONTROL if you have DFSMSrmm common time support not enabled.

```

Control record:
Type = MASTER      Create date = 2002/064      Create time = 15:46:09
                  Update date = 2007/082      Update time = 17:49:39
Journal: Utilization = 17% (75% threshold)      STATUS: = ENABLED
CDS:      Utilization = 81%

Exit status:
  EDGUX100 = ENABLED
  EDGUX200 = NONE

Options:
  Stacked Volumes      = NONE
  Extended Bin          = DISABLED
  Common Time          = DISABLED
  CDSID ENQ name       = ENABLED

Last backup:
  Date = 2007/079      Time = 13:25:42
Last journal backup:
  Date = 2007/079      Time = 13:25:42
Last report extract:
  Date = 2007/072      Time = 18:30:39
Last scratch procedure:
  Date =                Time =
Rack numbers           = 6258
LOCAL store bins      = 0
DISTANT store bins    = 0
REMOTE store bins     = 0

Control functions in progress:
Backup      = N  Restore      = N
Verify      = N  Expiration   = N
Report Extract = N  Disaster Store = N
VRS         = N  Synchronize  = N

Client/Server:
  host name =
  IP address =

```

Figure 7-6 Common time support disabled

Use Example 7-1 to enable DFSMSRmm common time support.

Example 7-1 Enable UTC support

```

//UTIL      EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD  SYSOUT=*
//MASTER   DD  DISP=SHR,DSN=RMM.CONTROL.DSET
//SYSIN     DD  *
CONTROL UTC(YES) CDSID(SC70)
/*

```

Where:

CDSID	Specifies one-to-eight alphanumeric characters that identify the control data set by name. There is no default. A CDSID is required in z/OS 1.9 or higher.
UTC	Enables DFSMSRmm common time support.

Once you enable common time support DFSMSRmm starts to record CDS record dates and times in common time and converts existing values, as required, from local times to common time. Figure 7-7 on page 228 shows you an example of a display data set.

Each user can set his own common time zone values to display the date and time information correctly.

Panel Help	
EDGP\$OP1 DFSMSrmm Dialog User Options	
Command ==>	
Date format	JULIAN (American, European, Iso or Julian)
Time zone	MST -04:00:00 (zone offsetHH:MM:SS)
Confirm deletes . . .	YES (Yes or No)
Processing option . .	F F - Foreground, B - Background
Eject option	C C - Convenience, B - Bulk
Variable reuse . . .	Y Y - Yes, N - No
Job statement information:-	
====> //RMMJOB	JOB ,RMM,NOTIFY=&SYSUID,
====> //	MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
====> //*	
====> //*	
Enter END command to save changes, or CANCEL to end without saving.	

Figure 7-7 Set your own common time zone value

All dates and times displayed or entered in the DFSMSrmm dialog are values local to this time zone. To change the time zone you must specify an offset value and a text string to identify that zone to you. The offset value is a time that your selected time zone is ahead of or behind universal time (UTC/GMT). Changes you make only affect future dialog actions and displays.

The report extract data set contains date and time values in the local time of the running system. The extract header record includes a field that lists the time zone offset as shown in Figure 7-8.

H	2007/072 183039SC64	JN-04:00:00
---	---------------------	-------------

Figure 7-8 Report extract data set time zone offset

Recommendation: We recommend that you have the system Time Of Day (TOD) clock set to GMT and enable DFSMSrmm to use UTC. You do this by using the EDGUTIL utility with the UPDATE parameter. Once you do this any newly recorded dates and time will be stored in common time and existing records will be converted to be in all common time as they are updated. You will continue to see dates and times presented in local time because DFSMSrmm handles the conversion from common time to your local time.

Set the TIMEZONE in the CLOCKxx PARMLIB member

TIMEZONE d.hh.mm.ss Specifies the difference between the local time and the Coordinated Universal Time (UTC). If ETRMODE YES and ETRZONE YES are specified (and an operational Sysplex Timer® is available), the system ignores the TIMEZONE parameter.

d	Specifies the direction from UTC. Value Range: E for east of UTC or W for west of UTC. Default: W
hh.mm.ss	Specifies the number of hours (hh) minutes (mm) and seconds (ss) that the local time differs from the UTC. Value Range: The value for hh must be between 00 and 15. The value for mm and ss must be between 00 and 59. mm.ss values are optional. In addition, the combined hh.mm.ss value must be within the range (00:00:00 - 15:00:00). This means that a value like 15.59.59 is not valid because it is outside the range, even though the hh portion is between 00 and 15 and the mm and ss portions are between 00 and 59. If the mm portion or ss portion or both are omitted, a default value of 00 is applied to the omitted portion, and appears in message IEA598I at IPL time. For example, if CLOCKxx contains TIMEZONE W.15, then at IPL time, message IEA598I would indicate: IEA598I TIME ZONE = W.15.00.00 Default: 00.00.00

See Example 7-2 how you can use the DISPLAY T command to display the local time of day and date and the coordinated universal time (UTC) of day and date.

Example 7-2 Displaying the Local and Coordinated Universal Time and Date

D	T
The local time of day and date and the coordinated universal time (UTC) of day and date are to be displayed (message IEE136I) as shown in .	
RESPONSE=SC70 IEE136I LOCAL: TIME=12.56.19 DATE=2007.068 UTC: RESPONSE=TIME=17.56.19 DATE=2007.068	

Figure 7-9 Result of the displaying the Local and Coordinated Universal Time and Date

7.2.4 Potential Problems Using Local Time

1. Systems sharing the CDS but in different time zones will create records using their own local date and time values.
2. Journal records might appear to not be in sequence; however the VSI count and record numbers show the correct sequence. Tokens will vary but will at least always increment.
3. Users will see a date/time and believe it to be in their local time zone, but it could reflect that used on another system.
4. Data sets or volumes could be released early.

7.3 Dynamic Installation Exits

DFSMSrmm now uses the system Dynamic Exit Services to manage calls to the installation exits, determine if exit modules exist, and to provide error handling and recovery. The installation exits DFSMSrmm provides are shown in Table 7-2.

Table 7-2 Sample installation exits IBM provides

Exit Name	Default Exit Routine	SAMPLIB Samples Provided
EDG_EXIT100	EDGUX100	EDGUX100, EDGCVRSX
EDG_EXIT200	EDGUX200	EDGUX200
EDG_EXIT300	EDGUX300	EDGUX300

DFSMSrmm installation exit modules can be loaded from any APF authorized library, the default is the LINKLST

Dynamic Exit Services is used to load and activate the default (EDGUXn00) exit module at initialization time. The default exit is activated in first position. All other exit related processing is handled by Dynamic Exit Services. If you do not use the default exit module (EDGUXn00) you must use PROGxx in z/OS parmlib or the SETPROG operator command, or CSVDYNEX macro.

Important: When definition of the dynamic exits or activation of the default exits encounters a problem DFSMSrmm startup provides the operator with a choice of actions; CANCEL, RETRY, or CONTINUE

7.3.1 DFSMSrmm Installation Exit calls

We have several exit points in Allocation and Open Processing where exit module EDGUX100 gets called to support functions like „Special Date“ or „Sticky Label“. During Inventory Management EDGUX200 gets called before a volume returns to scratch. At one exit point during Open, EDGUX300 provides information about the tape media. See Figure 7-10 on page 230 to understand how DFSMSrmm calls the exit.

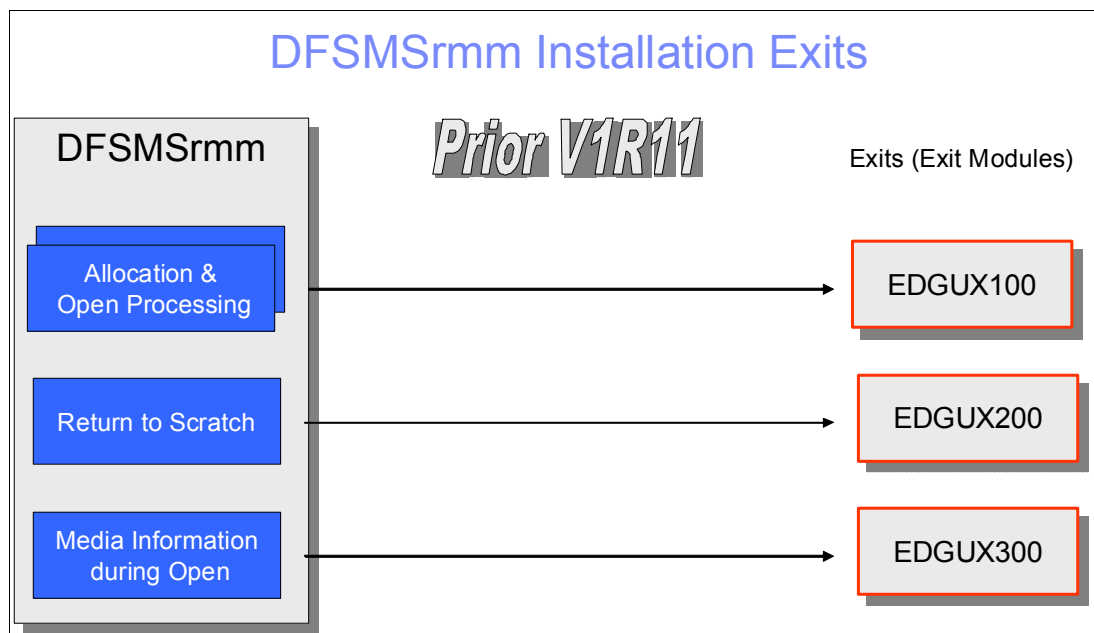


Figure 7-10 Prior z/OS 1.11 DFSMSrmm installation exit call

We go to z/OS V1.11 and the new world of Dynamic Exits and see what has changed and what not.

The blue side on the left, the exits points in DFSMSrmm are still the same. Also, the sample exit modules on the right, which are shipped with DFSMSrmm, (EDGUX100/200/300) have not been changed. The Parameter Lists mapped by EDGPL100/200/300 have not been changed either.

New in the Figure 7-11 on page 231 are the 3 exits in the middle (EDG_EXIT100/200/300) which are controlled by Dynamic Exit Services. DFSMSrmm does no longer call the exit modules directly. Also new is the ability to add more exit modules to a given exit.

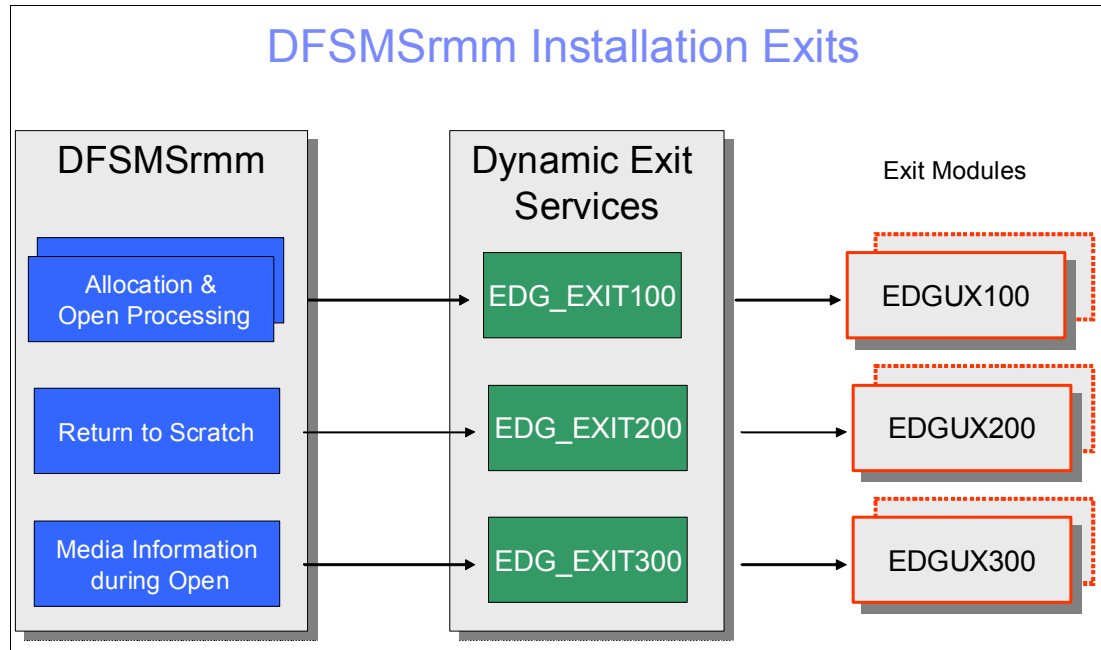


Figure 7-11 z/OS 1.11 DFSMSrmm installation exit call

Note: Even the exit points have not changed, error handling has.

7.3.2 Managing Exit Modules

Prior to z/OS V1.11, the exit modules could not be managed independently. Also, it was not possible to temporarily inactivate an exit module. Instead, the exit module had to be deleted from the LINKLIB before the REFRESH EXITS command was issued. Figure 7-12 on page 232 shows you the use of the DFSMSrmm REFRESH EXITS command. As you can see there is no possibility to specify an exit in the command to refresh only one exit.

Manage Exit Modules



Prior V1R11

- **Add** exit module **EDGUX100**

```
F DFRMM,REFRESH EXITS
```

- **Delete** exit module **EDGUX200**

```
F DFRMM,REFRESH EXITS
```

- **Refresh** exit module **EDGUX300**

```
F DFRMM,REFRESH EXITS
```

- REFRESH EXIT Command always does for all 3 exit modules:
 - Delete module from storage (if exists)
 - Load module from Linklib (if exists)

Figure 7-12 Managing exit modules prior z/OS 1.11

With z/OS V1.11, the MODIFY DFRMM,REFRESH EXIT command is no longer supported. You can use the SETPROG EXIT command instead, which allows to manage the exit modules independently. Remember, that also the EXIT statement of the PROGxx parmlib member can be used to add an exit module. In Figure 7-13 you can see how you can refresh a single DFSMSrmm installation exit using the SETPROG operator command.

Manage Exit Modules



V1R11

- **Add** exit module **EDGUX100**

```
SETPROG EXIT,ADD,EXITNAME=EDG_EXIT100,MODNAME=EDGUX100
```

- **Delete** exit module **EDGUX200**

```
SETPROG EXIT,DELETE,EXITNAME=EDG_EXIT200,MODNAME=EDGUX200
```

- **Refresh** exit module **EDGUX300**

```
SETPROG EXIT,DELETE,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300  
SETPROG EXIT,ADD,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300
```

Figure 7-13 Managing exit modules with z/OS 1.11

Important: If you have installed DFSMSrmm z/OS 1.11 or lower than you should quiesce DFSMSrmm at this time using the following command:

```
MODIFY DFRMM,QUIESCE
```

After the exits are refreshed you must re-start DFSMSrmm.

Also you can do the following:

```
SETPROG EXIT,ADD,EXITNAME=EDGUX100_EXIT,MODNAME=EDGUX101
```

then delete the existing EDGUX100

```
SETPROG EXIT,DEL,EXITNAME=EDGUX100_EXIT,MODNAME=EDGUX100
```

If EDGUX101 is working correctly, then copy it back over EDGUX100, and issue the following commands:

```
SETPROG EXIT,ADD,EXITNAME=EDGUX100_EXIT,MODNAME=EDGUX100
```

```
SETPROG EXIT,DEL,EXITNAME=EDGUX100_EXIT,MODNAME=EDGUX101
```

which will put the RMM environment back to "normal".

Beside deleting and adding an exit module, you can also temporarily inactivate it, but keep in mind, that setting the state to ACTIVE again does not imply a reload of the module from the library as you can see in Figure 7-14.

Manage Exit Modules

V1R11



- **Inactivate** exit module **EDGUX300**

```
SETPROG EXIT,MODIFY,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300,STATE=INACTIVE
```
- **Activate** exit module **EDGUX300**

```
SETPROG EXIT,MODIFY,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300,STATE=ACTIVE
```

Figure 7-14 Deleting and adding exit modules with z/OS 1.11

Note: Inactivate or activate does not reload the exit module from the Library.

Up to V1R11, the Exit status was shown for what are now the exit routines EDGUX100, -200, and -300. Now, the status is shown for the exits EDG_EXIT100, -200, and -300. Nevertheless, the statuses ENABLED, DISABLED and NONE have not changed. Just the exact description of the statuses has been adjusted to:

NONE	exit is not defined or no exit modules exist
ENABLED	at least one active exit module exists

DISABLED

one or more exit modules exist but is none is active.

In Figure 7-15 you can see the status of each installation exit if you are using the TSO RMM LISTCONTROL subcommand.

```
Control record:
Type = MASTER      Create date = 2002/064      Create time = 15:46:09
                   Update date = 2009/229      Update time = 20:05:25
Journal: Utilization = 8% (75% threshold)      STATUS: = ENABLED
CDS:   Utilization = 29%
Exit status:
EDG_EXIT100 = ENABLED
EDG_EXIT200 = NONE
EDG_EXIT300 = NONE
Options:
Stacked Volumes      = NONE
Extended Bin         = DISABLED
Common Time          = ENABLED
CDSID ENQ name       = ENABLED
```

Figure 7-15 LISTCONTROL output

Figure 7-16 shows you the same information when you are using the DFSMSrmm ISPF dialog.

```
DFSMSrmm Control Record Display
Command ==>

Type . . : MASTER      Create date . : 2002/064      Create time . : 15:46:09
                   Update date . : 2009/229      Update time . : 20:05:25
More:      +

Journal utilization . : 8 %      ( 75 % threshold) Status : ENABLED
CDS utilization . . . : 29 %

Exit status:
EDG_EXIT100 . . : ENABLED
EDG_EXIT200 . . : NONE
EDG_EXIT300 . . : NONE
...
Options:
Stacked volumes . . . : NONE
Extended bin . . . . : DISABLED
Common time . . . . . : ENABLED
```

Figure 7-16 DFSMSrmm control panel display

7.3.3 Setting Up the EDG_EXIT100 Routine Environment

To Activate or reactivate the exit routine, you can do one of the following:

- ▶ Use dynamic exit services via operator command.
- ▶ Use the CSVDYNEX macro.
- ▶ Start DFRMM for the first time since an IPL.

Note: Stopping and restarting, or refreshing the DFSMSrmm procedure does not reload or reactivate your exit routine. See *DFSMSrmm Managing and Using Removable Media, SC26-7404*, for more information.

To installing the EDGUX100 default exit routine perform these actions to update or replace the default exit module:

1. Build and install an SMP/E USERMOD to apply the updated source code for the EDGUX100 exit module. Include the necessary JCLIN statements to get the EDGUX100 load module added to the LINKLIB target library. You can install the load module by applying the updated source code using an SMP/E USERMOD as shown in Figure 7-17. Modify the FMID and PRE to reflect the release you are running.

- Allocate a user SAMPLIB data set. In Figure 117 the user SAMPLIB data set is defined as MY.RMM.SRCLIB and allocated to DD card SRCLIB.
- Copy the EDGUX100 source from SAMPLIB to the user SAMPLIB and modify, as needed, for your installation.
- SMP/E RECEIVE the USERMOD.
- SMP/E APPLY the USERMOD. Ensure that a DD card exists for the user SAMPLIB in the APPLY job, or as a DDDEF to SMP/E in the target zone.

After performing these steps, the modified version of the EDGUX100 exit module source code resides in both the user SAMPLIB and SYS1.SAMPLIB. IBM's original copy is only in the distribution libraries at this point. If you accept the USERMOD, only the modified version of the sample exit routine source code exists. The SMP/E target zone reflects RMID indicators of VMRMM01 for all of these records:

- SAMP EDGUX100 RMID=VMRMM01 SYSLIB=SAMPLIB
- SRC EDGUX100 RMID=VMRMM01 SYSLIB=SAMPLIB
- MOD EDGUX100 RMID=VMRMM01 LMOD=EDGUX100
- LMOD EDGUX100 SYSLIB=LINKLIB

The RMID of VMRMM01 for the SAMP record prevents IBM service from being installed. This results in an ID search and notification to you that IBM is the servicing exit source code.

```
//STEP1    EXEC SMPPEMVS,REGION=6120K
//SYSIN    DD  *
SET BDY(GLOBAL) -
RECEIVE
/*
//SMPPTFIN DD DATA,DLM=##
++USERMOD (VMRMM01) REWORK(1992082) .
++VER (Z038) FMID(HDZ11D0) .
++JCLIN .
//EDGUX100 EXEC PGM=IEWL,
//          PARM='LET,NCAL,RENT,REUS,LIST,XREF,REFR'
//SYSLMOD  DD DSN=SYS1.LINKLIB,DISP=SHR
//SRCLIB   DD DSN=MY.RMM.SRCLIB,DISP=SHR
//AEDGMOD1 DD DSN=SYS1.AEDGMOD1,DISP=SHR
//SYSPRINT DD SYSOUT=*
INCLUDE AEDGMOD1(EDGUX100)
ENTRY    EDGUX100
NAME     EDGUX100(R)
++SRC(EDGUX100) TXLIB(SCRLIB) DISTLIB(ASAMPLIB) .
++SAMP(EDGUX100) TXLIB(SCRLIB) DISTLIB(ASAMPLIB) .
##
/*
```

Figure 7-17 Create a SMP/E USERMOD to apply the updated EDGUX100 installation exit

2. Copy the new exit load module into the LNKLST library.

3. Refresh LLA.
4. Refresh the exit by using MVS operator commands.

If DFSMSrmm is running in a shared environment, you must repeat this step on each system.

When you use the default exit module, EDGUX100, for the installation exit EDG_EXIT100 it is loaded and activated as an exit routine, if not already active, by DFSMSrmm when DFSMSrmm is started and stays loaded and active for the life of the IPL. It can be refreshed/deleted at any time by using one of the MVS system operator commands.

You can use any load module name for your exit module because you use PROGxx in z/OS parmlib, SETPROG or CSVDYNEX macro to associate the exit module with the exit. Therefore you do not have to use EDGUX100 as the load module name. However, using the default exit module name simplifies your implementation because DFSMSrmm itself ensures that the exit module is loaded and activated. Figure 7-18 shows you the correct use of the SETGROG MVS command to add the EDG_EXIT100.

```
SETPROG EXIT,ADD,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300
```

Figure 7-18 SETPROG EXIT ADD syntax

The RMM TSO LISTCONTROL subcommand you can see in Figure 7-19 shows you the current status of the three possible DFSMSrmm installation exits.

```
RMM LISTCONTROL

Control record:
Type = MASTER      Create date = 2002/064      Create time = 15:46:09
                  Update date = 2009/235      Update time = 02:00:19
Journal: Utilization = 46% (75% threshold)    STATUS: = ENABLED
CDS:    Utilization = 29%
Exit status:
  EDG_EXIT100 = ENABLED
  EDG_EXIT200 = NONE
  EDG_EXIT300 = ENABLED
Options:
  Stacked Volumes      = NONE
  Extended Bin         = DISABLED
  Common Time          = ENABLED
  CDSID ENQ name       = ENABLED
...
```

Figure 7-19 RMM TSO LISTCONTROL

7.3.4 Controlling the Exit Routine Through the z/OS Dynamic Exits Facility

IBM has defined the EDG_EXITn00 exit to the dynamic exits facility. You can refer to the exit by the name EDG_EXITn00. You can use the EXIT statement of the PROGxx parmlib member, the SET PROG=xx operator command, the SETPROG EXIT operator command, or the CSVDYNEX macro to control this exit and its exit routines. Table 7-20 shows you a sample of the SETPROG to add the EDGUX300 user exit after an IPL.

```
SETPROG EXIT,ADD,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300
```

Figure 7-20 SETPROG EXIT ADD syntax

You can use the ADDABENDNUM and ABENDCONSEC parameters on the CSVDYNEX REQUEST=ADD macro or the ABENDNUM parameter of the SETPROG EXIT operator command to limit the number of times the exit routine abnormally ends before it becomes inactive. An abend is counted when both of the following conditions exist:

- a. The exit routine does not provide recovery, or the exit routine does provide recovery but percolates the error.
- b. The system allows a retry; that is, the recovery routine is entered with bit SDWACLUP off.

By default the system disables the exit routines for EDG_EXIT100 and EDG_EXIT300 after one abend. By default the system does not disable the exit routines for EDG_EXIT200.

You can use the MVS SETPROG command to change the state of an exit routine. In Figure 7-21 we change the status from ACTIVE to INACTIVE.

```
SETPROG EXIT,MODIFY,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300,STATE=INACTIVE
```

Figure 7-21 SETPROG EXIT MODIFY syntax

After you have inactivated the EDG_EXIT300 you can controlling the result using the RMM TSO LISTCONTROL command as shown in Figure 7-22.

```
RMM LISTCONTROL

Control record:
Type = MASTER      Create date = 2002/064      Create time = 15:46:09
                  Update date = 2009/235      Update time = 02:00:19
Journal: Utilization = 46% (75% threshold)    STATUS: = ENABLED
CDS:   Utilization = 29%
Exit status:
  EDG_EXIT100 = ENABLED
  EDG_EXIT200 = NONE
  EDG_EXIT300 = DISABLED
Options:
  Stacked Volumes      = NONE
  Extended Bin         = DISABLED
  Common Time          = ENABLED
  CDSID ENQ name       = ENABLED
...
```

Figure 7-22 RMM TSO LISTCONTROL

7.3.5 Deleting the EDG_EXIT100 Exit Routines

To delete, or selectively delete the exit routines for the EDG_EXIT100 exit from the system, you can use MVS operator commands. If DFSMSrmm is running in a shared environment, you must repeat this step on each system. Figure 7-23 shows you the correct use of the MVS SETPROG command to delete the EDG_EXIT300.

```
SETPROG EXIT,DELETE,EXITNAME=EDG_EXIT300,MODNAME=EDGUX300
```

Figure 7-23 SETPROG EXIT DELETE syntax

Now the status for EDG_EXIT300 is NONE and no longer DISABLED as you can see in Figure 7-24 on page 238.

```

RMM LISTCONTROL

Control record:
Type = MASTER      Create date = 2002/064      Create time = 15:46:09
                  Update date = 2009/235      Update time = 02:00:19
Journal: Utilization = 46% (75% threshold)      STATUS: = ENABLED
CDS:      Utilization = 29%
Exit status:
  EDG_EXIT100 = ENABLED
  EDG_EXIT200 = NONE
  EDG_EXIT300 = NONE
Options:
  Stacked Volumes      = NONE
  Extended Bin         = DISABLED
  Common Time          = ENABLED
  CDSID ENQ name       = ENABLED
...

```

Figure 7-24 RMM TSO LISTCONTROL

7.3.6 Writing an Exit Routine for the EDG_EXIT100 Exit

EDG_EXIT100 exit routines run in SYSTEM KEY 0 or 5 AMODE(31) RMODE(ANY) in the user's address space. KEY 0 is used when a WTO address or an ACERO address is provided, and KEY 5 is used when a JFCB address is provided. The default exit routine, EXGUX100 is loaded and activated via CSVDYNEX macro, the systems' dynamic exits services. Exit modules can be contained in any APF authorized LNKLIST library or a APF authorized library named on CSVDYNEX macro invocation.

7.4 Activating RACF TAPEVOL and TAPEDSN

Instead to implementing RACF TAPEVOL and TAPEDSN you can protect your data sets on tape using the SAF tape security introduced with DFSMS z/OS V1R8. Refere to *Chapter 13, "Tape security with z/OS V1.8" on page 485* for more information.

You can control the actions that DFSMSrmm takes on RACF tape profiles through the DFSMSrmm OPTION TPRACF command and your installation's RACF options. To protect tape volumes, you can specify DFSMSrmm OPTION TPRACF(AUTOMATIC) or TPRACF(PREDEFINED). See 7.4.4, "DFSMSrmm OPTION command operand TPRACF" on page 240, for more information.

Before you execute the RACF commands, activate the RACF TAPEVOL and TAPEDSN classes as shown in Figure 7-25.

```

SETROPTS CLASSACT(TAPEVOL)
SETROPTS TAPEDSN
SETROPTS LIST

```

Figure 7-25 RACF SETROPT commands

Recommendation: We recommend that you do not activate the RACF TAPEVOL and TAPEDSN classes until after you have converted to and successfully used DFSMSrmm for a period of time (for example, three months).

For optimal security, we recommend that you make TAPEVOL and TAPEDSN active with either EDGRMMxx PARMLIB option TPRACF(P) or TPRACF(A) to obtain full protection at the volume and data set level.

Although we discourage this, it is possible to have no tape protection, or to use only TAPEVOL profiles. If you have no tape security, you cannot control the creation and use of data on tape. If you use only TAPEVOL profiles to protect tape volumes, you must maintain the access lists to the TAPEVOL profiles to allow access. Consider the use of TAPEDSN, so that access to tape data is covered by your normal, existing, generic data set profiles.

7.4.1 Before you can start

You can implement RACF TAPEVOL and TAPEDSN only if you have a user with all required authority and access.

Your user ID needs the following attributes to execute all created RACF commands:

- ▶ **SPECIAL:** Specifies that the user is to be allowed to issue all RACF commands with all operands except the operands that requires the AUDITOR attribute
- ▶ **AUDITOR** (Optional): Specifies that the user is to have full responsibility for auditing the use of system resources. An AUDITOR user can control logging of detected accesses to any RACF-protected resources during RACF authorization checking and access to the RACF data set.

You need access to following data set profiles and resources in class FACILITY defined in RACF to create a new report extract data set:

- ▶ ALTER access to data set profile RMM.HSKP.**
- ▶ UPDATE access to data set profile SYS1.PARMLIB or any equivalent PARMLIB you have defined EDGRMMxx member.
- ▶ READ access to STGADMIN.EDG.HOUSEKEEP defined in FACILITY class
- ▶ READ access to STGADMIN.EDG.HOUSEKEEP.RPTEXT defined in FACILITY class

7.4.2 DFSMSrmm RACF tape security support

The objective for DFSMSrmm RACF tape security support is to provide a complete interface with RACF for tapes, so you can use any combination of valid RACF options, including use of any RACF installation exits you still have. You can tell DFSMSrmm not to provide any RACF support by specifying the DFSMSrmm EDGRMMxx PARMLIB OPTION TPRACF(N) command. You can set up the type of processing wanted by requesting automatic TPRACF(A) or predefined TPRACF(P) RACF profiles. You can also use the DFSMSrmm EDGRMMxx PARMLIB VLPOOL RACF command to provide control of RACF at the pool level.

7.4.3 DFSMSrmm automatic tape security support

DFSMSrmm automatic tape security processing assumes that when a tape is mounted for output as a scratch tape, it is not RACF protected. During the open processing for the tape data set, either DFSMSdftp or your installation exits cause some RACF profiles to be created.

DFSMSrmm predefined processing also ensures that when TAPEVOL and TAPEDSN are active, RACF predefined TAPEVOL profiles with an empty TVTOC are created for scratch volumes.

To enforce a DFSMSrmm standard that all tapes are RACF protected when the data set is closed, DFSMSrmm checks that a RACF security profile exists for the volume. If a RACF security profile does not exist, DFSMSrmm creates one and places the owner of the tape in the access list for the TAPEVOL profile with ALTER authority. The process ensures that your current tape security mechanism should continue to work as long as it is based on TAPEVOL profiles.

7.4.4 DFSMSrmm OPTION command operand TPRACF

This specifies the type of RACF tape support that DFSMSrmm provides. Use this operand when you want DFSMSrmm to maintain the security profiles that protect tape volumes. RACF tape support you define can be overridden by RACF tape support you define with the VLPOOL described in 7.4.5, “DFSMSrmm VLPOOL command operand RACF” on page 240.

The TPRACF(NO) option is assumed for any volume serial number containing special characters. To protect tape volumes that use special characters in the volume serial number, use RACF generic TAPEVOL profiles that are outside of DFSMSrmm control. DFSMSrmm honors all other TPRACF options for volume serial numbers that are alphanumeric including national characters. You can also use RACF generic data set profiles to protect data created on the tape volume. DFSMSrmm honors the TPRACF setting when running in all modes.

If you set TPRACF(PREDEFINED) or TPRACF(AUTOMATIC), DFSMSrmm ensures that all non scratch tapes are protected by a discrete RACF TAPEVOL profile by checking that a RACF profile exists, whenever a data set is written on a tape. If a profile does not exist, DFSMSrmm creates one. Therefore, you do not need to use RACF installation exits to set the JCL PROTECT=YES option or specify PROTECT=YES in your JCL. You can use generic data set profiles for all tape data sets without changes to JCL or installation procedures, if you used the VLPOOL command with the RACF(Y) operand, because DFSMSrmm creates a TVTOC when you use the RACF TAPEDSN option.

For both TPRACF(PREDEFINED) and TPRACF(AUTOMATIC), DFSMSrmm ensures TAPEVOL profiles are always deleted during recycling of scratch volumes, if they exist. When you use a volume for output and there is no RACF protection for the volume when you close the data set, DFSMSrmm creates a profile to protect the volume. The owner of the volume is given RACF access to the profile. If TAPEVOL and TAPEDSN are active, DFSMSrmm creates a TVTOC and adds the first file data set entry. The default is TPRACF(N).

For more information about RACF processing and security options, refer to the manual *DFSMSrmm Implementation and Customization Guide*, SC26-7405, and chapter titled “Authorizing DFSMSrmm Users and Ensuring Security.”

7.4.5 DFSMSrmm VLPOOL command operand RACF

Specifies the type of RACF tape support that DFSMSrmm should provide for volumes in the pool you are defining. When you define RACF tape support for volumes in a pool, you must look at the RACF tape support you have defined for your installation with the OPTION TPRACF command described in the chapter “Defining System Options: OPTION,” in the *DFSMSrmm Implementation and Customization Guide*, SG26-7405.

Specify Y if you want DFSMSrmm to create RACF tape profiles for the volumes in the pool. Ensure that OPTION TPRACF(AUTOMATIC or PREDEFINED) is specified. If OPTION

TPRACF(NO) is specified, DFSMSrmm does not create the RACF tape profiles. Only specify Y for tape pools because TAPEVOL profiles are not required for optical volumes.

Specify N if you do not want DFSMSrmm to create RACF tape profiles for the volumes in the pool. If you specify N, DFSMSrmm plays no part in creating or deleting RACF tape profiles, regardless of the system-wide option TPRACF.

The default is RACF(N).

7.4.6 Running DFSMSHsm and DFSMSrmm

How you use the DFSMSHsm SETSYS option TAPESECURITY influences the TPRACF value in the DFSMSrmm PARMLIB member EDGRMMxx. DFSMSHsm uses RACF TAPEVOL profiles to secure its volumes. You can combine the DFSMSHsm method for securing volumes with DFSMSrmm RACF options as described in the chapter “Defining System Options: OPTION” in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, to ensure complete security for your tape data.

Select the appropriate combinations for your environment from the following options:

- ▶ Use RACF with TAPEVOL and TAPEDSN.
- ▶ Use DFSMSHsm TAPE SECURITY with RACF or EXPIRATIONINCLUDE.
- ▶ Use DFSMSrmm TPRACF with A, P, or N and VLPOOL RACF with Y or N.

Recommendations:

- ▶ Run DFSMSHsm with scratch tapes that DFSMSrmm manages so you can have a single scratch pool for all users of tape and gain any available benefits from pre-mounting scratch tapes in cartridge loaders.
- ▶ Use RACF tape security to ensure data security on your system.
- ▶ Set the following suggested DFSMSHsm parameters:

```
SETSYS EXITOFF(ARCTVEXT)           -
      SELECTVOLUME(SCRATCH)         -
      TAPEDELETION(SCRATCH)         -
      TAPESECURITY(RACF EXPIRATION) -
      PARTIALTAPE(MARKFULL)
```

Specify EXITOFF(ARCTVEXT) if DFSMSrmm is your only tape management product because DFSMSHsm always calls the DFSMSrmm programming interface EDGTVEXT if you have installed DFSMSrmm Release 1.4 or later.

7.4.7 Defining RACF profiles

Example 7-3 shows a sample batch job to protect tape volumes and tape data sets using RACF RDEFINE, RALTER, and ADDSD commands.

Example 7-3 *RACF commands for TAPEVOL and TAPEDSN implementation*

```
/* ***** */
/* RACF cmds for DFSMSrmm TAPEVOL and/or TAPEDSN implementation */
/* ***** */
RDEFINE TAPEVOL HSMHSM OWNER(DFHSM) +
      UACC(NONE) AUDIT(ALL(READ))          /* VOL-SET */
      PERMIT HSMHSM CLASS(TAPEVOL) ID(TRESTER) DELETE
RALTER TAPEVOL SC0239 TVTOC                /* MASTER */
ADDSD 'BSYPPS.OPD.STAIR.TSTAIR.E9201' +
```

```

        FILESEQ(1) SETONLY TAPE UNIT(TAPE) VOL(SC0239)
ADDSD 'BSYPPS.OPD.STAIR.DSTAIR.E9201' +
        FILESEQ(2) SETONLY TAPE UNIT(TAPE) VOL(SC0239)
ADDSD 'BSYPPS.OPD.STAIR.DATASETS.E9201' +
        FILESEQ(3) SETONLY TAPE UNIT(TAPE) VOL(SC0239)
PERMIT SC0239 CLASS(TAPEVOL) ID(TRESTER) DELETE
RALTER TAPEVOL HSMHSM ADDVOL(SC0648)                /* HBAC */
RALTER TAPEVOL SC0010 TVTOC                          /* MASTER */
ADDSD 'SCHLUM.RMMDemo.FILE2.VOL12' +
        FILESEQ(1) SETONLY TAPE UNIT(TAPE) VOL(SC0010)
PERMIT SC0239 CLASS(TAPEVOL) ID(TRESTER) DELETE

```

Where:

RDEFINE TAPEVOL HSMHSM

DFSMSHsm protects tapes with RACF by adding them to a RACF tape-volume set. All tapes in DFSMSHsm's RACF tape-volume set share the same access list and auditing controls.

RALTER TAPEVOL SC0239 TVTOC

Create a TVTOC for an existing TAPEVOL profile.

ADDSD 'BSYPPS.OPD.STAIR.TSTAIR.E9201

Add tape data set profiles to the TVTOC for volumes with DFSMSrmm status of MASTER.

PERMIT SC0239 CLASS(TAPEVOL) ID(TRESTER) DELETE

Remove the command executor from the access list.

7.4.8 Final RACF activities

After you define the DFSMSrmm RACF function groups, data set profiles, and resources in RACF FACILITY class, and assign DFSMSrmm a user ID, you must refresh the RACF in-storage profiles.

Example 7-4 shows a batch job to REFRESH the RACF in-storage profiles.

Example 7-4 JCL to refresh RACF in-storage profiles

```

//RACFREFR JOB ,RMM,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
/* *****
/*  Caution
/*  =====
/*  To execute this job you need the RACF "SPECIAL" attribute
/*  *****
//STEP01   EXEC PGM=IKJEFT01
//SYSPRINT DD  SYSOUT=*
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
/* ***** */
/*  FINAL ACTIVITIES                               */
/*  ***** */
        SETROPTS GENERIC(FACILITY) REFRESH
        SETROPTS RACLIST(FACILITY) REFRESH
        SETROPTS GENERIC(DATASET) REFRESH
        SETROPTS GENERIC(STARTED) REFRESH

```

7.5 Maintaining your control data set

DFSMSrmm provides you three different utilities to maintain your DFSMSrmm CDS. The CDS is a VSAM key sequenced data set (KSDS) and should be backed up once a day and reorganized from time to time. In this section, we describe how you can back up your CDS and journal, reorganizing your CDS, and restore your CDS. The utilities provide some more functions, but these functions are not described in this section. For more information about these utilities, refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

The following functions of the DFSMSrmm utilities maintain the CDS that we describe in this section:

EDGHSKP	To back up the CDS and journal
EDGBKUP	To back up the CDS and journal, reorganize the CDS, or to restore the CDS
EDGUTIL	To create or update the CDS control record

7.5.1 Using EDGHSKP

You can use the EDGHSKP utility to back up your DFSMSrmm CDS and journal, and to clear the journal. You do not have to stop any DFSMSrmm function while this utility is running. DFSMSrmm stops updates made to the CDS while the AMS backup function is running. If you are using DSS to back up the CDS, updates are allowed.

The EDGHSKP utility requests that DFSMSrmm is initialized and the task is started.

BACKUP

Specify BACKUP to back up the control data set to back up the journal, or to back up both the control data set and the journal. When you specify BACKUP on the EXEC statement, you must also specify a DD statement. Specify BACKUP DD when you back up the control data set, and JRNLBKUP DD when you back up the journal. You can also specify both DD statements in your EDGHSKP JCL.

Where:

AMS	BACKUP(AMS) is the default. DFSMSrmm uses the access method services REPRO command to perform backup processing. Specify BACKUP(AMS) to prevent DFSMSrmm from updating the control data set during control data set backup processing. Backup cannot be directly to tape.
DSS	Specify BACKUP(DSS) with the DFSMSdss concurrent copy environment set up to permit the update of the DFSMSrmm control data set during backup processing. This ensures that all tape activities can continue during backup processing. You can specify BACKUP(DSS) without setting up the concurrent copy environment. If you specify BACKUP(DSS), backup can be direct to tape.
COPY	Specify BACKUP(COPY) to use DFSMSdss to create a copy of the CDS and optionally, a backup of the journal. The COPY uses

DFSMSdss logical data set copy and enables you to use fast replication services that enables CDS updates to be made while the copy is created. COPY works just as the DSS option, but uses the COPY command instead of the DUMP command with DFSMSdss. The BACKUP DD is optional for COPY. Specify it if you want to direct the copy to a specific volume, or your environment is not SMS-managed.

We recommend that you use DFSMSrmm utilities to backup the CDS and journal data set instead of other backup tools. You can plan batch job using EDGHSKP or EDGBKUP for backup, or set up an automatic backup procedure as well.

You can run DFSMSrmm control data set backup processing using the DFSMSdss DUMP command, or the access method services REPRO command.

When you use DFSMSdss concurrent copy for backup, DFSMSrmm continues to process commands and update the control data set.

When you use the access method services REPRO command or DFSMSdss without concurrent copy for backup, DFSMSrmm does not allow updates to the control data set during control data set backup.

Certain DFSMSrmm requests wait until the backup completes. RMM TSO subcommands that request to update the DFSMSrmm control data set fail. Only actions that read the control data set continue uninterrupted.

To use DFSMSdss concurrent copy, you must authorize the DFSMSrmm batch inventory management backup job to the RACF profile STGADMIN.ADR.DUMP.CNCURRNT.

For journal backup, DFSMSrmm uses IDCAMS to back up the journal even when BACKUP(DSS) is specified. When using EDGHSKP to perform back up, the journal can be cleared.

Note: When DFSMSrmm is active, you do not need to specify the MASTER and JOURNAL DD statements. EDGBKUP obtains the name of the control data set and the journal from the DFSMSrmm subsystem. When the DFSMSrmm subsystem is stopped, quiesced, or when you want to back up a control data set that is not in use by DFSMSrmm, specify the MASTER DD statement and the JOURNAL DD statement.

Example 7-5 shows EDGHSKP JCL using access method services to perform backup processing. The back up copies must be allocated on DASD.

Example 7-5 CDS and journal backup using AMS

```
//CDSHKUP JOB ....
//STEP0001 EXEC PGM=EDGHSKP,PARM='BACKUP'
//MESSAGE DD DISP=SHR,DSN=RMM.MESSAGE.FILE
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=(,CATLG),UNIT=SYSDA,
//        SPACE=(CYL,(100,50),RLSE),
//        DSN=RMM.HSKP.BACKUP(+1),
//        RECFM=VB,BLKSIZE=0,LRECL=9216
//JRNLBKUP DD DISP=(,CATLG),UNIT=SYSDA,
//        SPACE=(CYL,(100,50),RLSE),
//        DSN=RMM.HSKP.JRNLBKUP(+1),
//        RECFM=VB,BLKSIZE=0,LRECL=9248
```

Note: When you using AMS, the back up copy must be allocated on DISK, because until the backup processes the DFSMSrmm control data set (CDS) is locked and no open to a tape data set is possible.

Where:

BACKUP DD	if you want to back up the control data set.
JRNLBKUP DD	if you want to back up the journal.
MESSAGE DD	lists the messages the DFSMSrmm subsystem issues.
SYSPRINT DD	contains the utility program messages.

If you specify both DD statements to back up the CDS and journal, and the function ended without any errors, the journal is cleared at the end.

Example 7-6 shows a sample EDGHSKP JCL using DFSMSdss to perform backup processing. In this case, the output data set can be allocated on TAPE, but in a case of recovery, the actual journal and the latest back up copy of the journal is needed.

Example 7-6 CDS and journal backup using DFSMSdss

```
//CDSHKUP JOB ....
//STEP0001 EXEC PGM=EDGHSKP,PARM='BACKUP(DSS)
//MESSAGE DD DISP=SHR,DSN=RMM.MESSAGE.FILE
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=(,CATLG),UNIT=TAPE,
// LABEL=(1,SL),
// DSN=RMM.HSKP.BACKUP(+1),
// RECFM=VB,BLKSIZE=0,LRECL=9216
//JRNLBKUP DD DISP=(,CATLG),UNIT=TAPE,
// LABEL=(2,SL),VOL=REF=*.BACKUP,
// DSN=RMM.HSKP.JRNLBKUP(+1),
// RECFM=VB,BLKSIZE=0,LRECL=9248
```

Where:

BACKUP DD	if you want to back up the control data set.
JRNLBKUP DD	if you want to back up the journal.
MESSAGE DD	lists the messages the DFSMSrmm subsystem issues.
SYSPRINT DD	contains the utility program messages.

If you specify both DD statements to back up the CDS and journal, and the function ends without any errors, the journal is cleared at the end.

Optionally, you can specify a DSSOPT DD statement to customize the DFSMSdss DUMP or COPY command.

To set up automatic backup for CDS, follow these steps:

1. Set the OPTION command in EDGRMMxx:

- Specify the JOURNALFULL value. The default is 75%, and a zero value means disable the function.
- Specify the name of backup procedure using the BACKUPPROC operand.
- Create a backup procedure in PROCLIB.

Refer to Figure 7-26 on page 246 for the OPTION command in EDGRMMxx.

OPTION			
...			
	BACKUPPROC (EDGBKUP)	/* Name of BACKUP-proc	*/-
	JOURNALFULL(75)	/* Percentage JRNL full	*/-
...			

Figure 7-26 PARMLIB setting for automatic backup

Show Example 7-7 for a sample backup procedure.

Example 7-7 Procedure for backing up CDS and journal

```
//EDGBKUP  PROC  OPT=BACKUP(AMS)
//STEP0001 EXEC  PGM=EDGHSKP,PARM='&OPT'
//MESSAGE  DD   DISP=SHR,DSN=RMM.MESSAGE.FILE
//SYSPRINT DD   SYSOUT=*
//BACKUP   DD   DISP=(,CATLG),UNIT=SYSDA,
//          SPACE=(CYL,(100,50),RLSE),
//          DSN=RMM.HSKP.BACKUP(+1),
//          RECFM=VB,LRECL=9216,BLKSIZE=0,BUFNO=30
//JRNLBKUP DD   DISP=(,CATLG),UNIT=SYSDA,
//          SPACE=(CYL,(100,50),RLSE),
//          DSN=RMM.HSKP.JRNLBKUP(+1),
//          RECFM=VB,LRECL=9248,BLKSIZE=0,BUFNO=30
```

Note: When the inventory management task is running, the DFSMSrmm does not perform automatic backup. If there are multiple DFSMSrmm subsystems sharing CDS, running automatic backup on only one system is better, or define different JOURNALFULL values.

7.5.2 Using EDGBKUP

You can use the EDGBKUP utility to reorganize and restore your DFSMSrmm CDS. Also, you can use this utility to back up your DFSMSrmm CDS and journal, and to clear the journal. You have to stop DFSMSrmm or use the QUIESCE command to quiescing the DFSMSrmm subsystem interface. For more information about QUIESCE, refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

The EDGBKUP utility can be used if DFSMSrmm is stopped or quiesced.

BACKUP

Specify BACKUP to back up the control data set, to back up the journal, or to back up both the control data set and the journal. When you specify BACKUP on the EXEC statement, you must also specify a DD statement. Specify BACKUP DD when you back up the control data set, and JRNLBKUP DD when you back up the journal. You can also specify both DD statements in your EDGHSKP JCL.

COPY

Specify BACKUP(COPY) to use DFSMSdss to create a logical data set copy of the control data set and optionally, a backup of the journal. DFSMSrmm uses DFSMSdss to create the copy of the control data set and IDCAMS to back up the journal. To allow updates to the control data set during backup processing, set up the DFSMSdss concurrent copy environment, virtual concurrent copy environment, or a fast replication with virtual concurrent copy environment. You must have the hardware and software required to

	establish a concurrent copy session, a virtual concurrent copy session, or fast replication with concurrent copy or virtual concurrent copy.
DSS	Specify BACKUP(DSS) to use DFSMSdss to back up the control data set. DFSMSrmm uses DFSMSdss to back up the control data set and IDCAMS to back up the journal. To allow updates to the control data set during backup processing, set up the DFSMSdss concurrent copy environment or virtual concurrent copy environment. You must have the hardware and software required to establish a concurrent copy session or a virtual concurrent copy session.
NREORG	Specify BACKUP(NREORG) to use IDCAMS to back up the control data set and to back up the journal data set.

NREORG is the default.

For journal backup, DFSMSrmm uses IDCAMS to back up the journal even when BACKUP(DSS) is specified. When using EDGHSKP to perform back up, the journal can be cleared.

Example 7-8 shows sample EDGBACK JCL. DFSMSrmm is stopped or quiesced using the access method services to perform backup processing.

Example 7-8 CDS and journal backing up using EDGBKUP with option AMS

```
//CDSHKUP  JOB  ....
//STEP0001 EXEC  PGM=EDGBKUP,PARM='BACKUP'
//SYSPRINT DD   SYSOUT=*
//MASTER   DD   DISP=SHR,DSN=RMM.PROD.CDS
//JOURNAL   DD   DISP=SHR,DSN=RMM.PROD.JRNL
//BACKUP    DD   DISP=(,CATLG),UNIT=SYSDA,
//            SPACE=(CYL,(100,50),RLSE),
//            DSN=RMM.HSKP.BACKUP(+1),
//            RECFM=VB,BLKSIZE=0,LRECL=9216
//JRNLBKUP  DD   DISP=(,CATLG),UNIT=SYSDA,
//            SPACE=(CYL,(100,50),RLSE),
//            DSN=RMM.HSKP.JRNLBKUP(+1),
//            RECFM=VB,BLKSIZE=0,LRECL=9248
```

Where:

BACKUP DD	if you want to back up the control data set.
JOURNAL DD	identifies the DFSMSrmm journal to be used during backup processing.
JRNLBKUP DD	if you want to back up the journal
MASTER DD	identifies the DFSMSrmm control data set.
SYSPRINT DD	contains the utility program messages.

If you specify both DD statements to back up the CDS and journal, and the function ends without any errors, the journal is cleared at the end.

Note: : If DFSMSrmm is stopped or quiesced, code the MASTER and JOURNAL DD if you have requested to back up this files.

REORG

Specify BACKUP(REORG) to reorganize the control data set during backup processing. DFSMSrmm uses IDCAMS to back up the control data set, to optionally back up the journal, and to restore the control data set from the backup.

The DFSMSrmm subsystem must be stopped or quiesced when you reorganize the active control data set. DFSMSrmm reorganizes the control data set by first backing up the control data set and optionally the journal. Then DFSMSrmm restores the control data set from the backup control data set.

Note: Do not use the BACKUP(DSS) parameter to reorganize the control data set.

Before you start the EDGBKUP utility to reorganize the control data set, you have to stop or quiesce the DFSMSrmm subsystem. In our example, we only quiesce the DFSMSrmm subsystem. If you use quiesce, you do not have to stop all your tape activities, nor do you have to switch offline your tape drives.

To quiesce DFSMSrmm, use one of the MVS modify commands shown in Figure 7-27.

```
MODIFY DFRMM,QUIESCE
F DFRMM,QUIESCE
```

Figure 7-27 MVS MODIFY DFSMSrmm command to stop all tape activities

Important: If you have multiple z/OS images, you must repeat the MODIFY command for each z/OS image.

Example 7-9 shows a sample EDGBACK JCL that can be used to reorganize your DFSMSrmm control data set after you see the message that the DFSMSrmm subsystem is quiesced.

Example 7-9 CDS reorganization using EDGBKUP

```
//CDSHKUP JOB ....
//STEP0001 EXEC PGM=EDGBKUP,PARM='BACKUP(REORG)'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//BACKUP DD DISP=(,CATLG),UNIT=SYSDA,
// SPACE=(CYL,(100,50),RLSE),
// DSN=RMM.HSKP.BACKUP(+1),
// RECFM=VB,BLKSIZE=0,LRECL=9216
```

Where:

BACKUP DD	if you want to back up the control data set
MASTER DD	identifies the DFSMSrmm control data set.
SYSPRINT DD	contains the utility program messages.

Example 7-10 shows you how to reorganize the control data set and back up the journal. DFSMSrmm backs up the journal before attempting to restore the control data set because the JRNLBKUP DD and the JOURNAL DD statement are specified in this example.

Example 7-10 CDS reorganization and backing up the journal using EDGBKUP

```
//CDSHKUP JOB ....
```

```
//STEP0001 EXEC PGM=EDGBKUP,PARM='BACKUP(REORG)'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//JOURNAL DD DISP=SHR,DSN=RMM.PROD.JRNL
//BACKUP DD DISP=(,CATLG),UNIT=SYSDA,
// SPACE=(CYL,(100,50),RLSE),
// DSN=RMM.HSKP.BACKUP(+1),
// RECFM=VB,BLKSIZE=0,LRECL=9216
//JRNLBKUP DD DISP=(,CATLG),UNIT=SYSDA,
// SPACE=(CYL,(100,50),RLSE),
// DSN=RMM.HSKP.JRNLBKUP(+1),
// RECFM=VB,BLKSIZE=0,LRECL=9248
```

Where:

BACKUP DD	if you want to back up the control data set.
JOURNAL DD	identifies the DFSMSrmm journal to be used during backup processing.
JRNLBKUP DD	if you want to back up the journal
MASTER DD	identifies the DFSMSrmm control data set.
SYSPRINT DD	contains the utility program messages.

After you check the success of the reorganization, restart the DFSMSrmm subsystem using a MVS modify command as shown in Figure 7-28.

```
MODIFY DFRMM,M=xx
```

Figure 7-28 MVS MODIFY command to restart DFSMSrmm

Important: If you have multiple z/OS images, you must repeat the MODIFY command for each z/OS image. Do not use the /RO *ALL command to restart DFSMSrmm on multiple z/OS images at the same time to minimize the risk of enqueue conditions.

Where:

DFRMM	is the name of the started task
M	specifies the PARMLIB member name that should be used

Note: To restart DFSMSrmm after it is quiesced, the operator must issue the command specified with a member name.

RESTORE

Specify RESTORE to restore the control data set from a backup copy. When you specify the BACKUP DD statement, DFSMSrmm restores the control data set. When you specify the JOURNAL DD statement, forward recovery is started. You can specify both the BACKUP DD statement and the JOURNAL DD statement in the same job step to restore, and forward recover the control data set from journal backups and the journal data set.

The DFSMSrmm subsystem must be stopped or quiesced when you restore the active control data set:

► Restoring the control data set without forward recovery

You can restore your DFSMSrmm control data set without forward recovery. If you do this, you can lose data if the back up copy is not an actual one, and no changes are made since the backup copy was created.

Example 7-11 shows how you can use EDGBACK JCL to restore your DFSMSrmm control data set without forward recovery.

Example 7-11 Restoring a control data set without forward recovery

```
//CDSHKUP JOB ....
//STEP0001 EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//BACKUP DD DISP=SHR,DSN=RMM.HSKP.BACKUP(0)
```

Where:

BACKUP DD	the current actual back up copy of the control data set.
MASTER DD	identifies the DFSMSrmm control data set.
SYSPRINT DD	contains the utility program messages.

Important: Use this example only if you have no need for a forward recovery.

► Restoring and forward recovery the control data set

You can restoring your DFSMSrmm control data set and forward recovery the DFSMSrmm control data set. This is the normal way to have the same information as before the crash.

Example 7-12 shows how you can use EDGBACK JCL to restore your DFSMSrmm CDS and forward recover it from the journal data set.

Example 7-12 Restoring a control data set with forward recovery

```
//CDSHKUP JOB ....
//STEP0001 EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//BACKUP DD DISP=SHR,DSN=RMM.HSKP.BACKUP(0)
//JOURNAL DD DISP=SHR,DSN=RMM.PROD.JRNL
```

Where:

BACKUP DD	the current actual back up copy of the control data set.
JOURNAL DD	identifies the DFSMSrmm journal to be used for forward recovery.
MASTER DD	identifies the DFSMSrmm control data set.
SYSPRINT DD	contains the utility program messages.

Example 7-13 shows how you can use EDGBACK JCL to restore your DFSMSrmm CDS and forward recover it using the backup copy minus one should your current actual DFSMSrmm CDS backup copy be destroyed or if it is unavailable.

Example 7-13 Restoring a control data set from backup copy -1 with forward recovery

```
//CDSHKUP JOB ....
//STEP0001 EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
```

```
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//BACKUP DD DISP=SHR,DSN=RMM.HSKP.BACKUP(-1)
//JOURNAL DD DISP=SHR,DSN=RMM.HSKP.JRNLBKUP(0)
// DD DISP=SHR,DSN=RMM.PROD.JRNL
```

Where:

BACKUP DD	the current actual back up copy of the control data set.
JOURNAL DD	identifies the newest DFSMSrmm journal backup copy and actual DFSMSrmm journal to be used for forward recovery.
MASTER DD	identifies the DFSMSrmm control data set.
SYSPRINT DD	contains the utility program messages.

Attention: If you forward recover using multiple journal data sets, concatenate the journal data sets in the order in which changes were originally made.

Example 7-14 shows how you can use EDGBACK JCL to restore your DFSMSrmm CDS from a backup copy -1 and forward recover it using the latest journal backup and the backup -1. This procedure can be used when the latest DFSMSrmm CDS backup copy is destroyed or not available.

Example 7-14 Restoring a control data set from backup copy - 2 with forward recovery

```
//CDSHKUP JOB ....
//STEP0001 EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//BACKUP DD DISP=SHR,DSN=RMM.HSKP.BACKUP(-2)
//JOURNAL DD DISP=SHR,DSN=RMM.HSKP.JRNLBKUP(-1)
// DD DISP=SHR,DSN=RMM.HSKP.JRNLBKUP(0)
// DD DISP=SHR,DSN=RMM.PROD.JRNL
```

Where:

BACKUP DD	specifies the back up copy -2 of your DFSMSrmm control data set.
JOURNAL DD	identifies the DFSMSrmm journal back up copy - 2, the newest DFSMSrmm journal backup copy and the actual DFSMSrmm journal to be used for forward recovery.
MASTER DD	identifies the DFSMSrmm control data set.
SYSPRINT DD	contains the utility program messages.

Attention: If you forward recover using multiple journal data sets, concatenate the journal data sets in the order in which changes were originally made.

The situation is the same as in the previous example, but the backup is made using BACKUP(DSS), the temporary journal backup, and the later ones should be applied in the correct order. Example 7-15 shows a sample JCL.

Example 7-15 Restoring a control data set from backup copy - 2 with forward recovery

```
//CDSHKUP JOB ....
//STEP0001 EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//BACKUP DD DISP=SHR,DSN=RMM.HSKP.BACKUP(-2)
//JOURNAL DD DISP=SHR,DSN=RMM.HSKP.JRNLBKUP(-1)
```

```
//      DD  DISP=SHR,DSN=RMM.HSKP.JRNLBKUP(0)
//      DD  DISP=SHR,DSN=RMM.PROD.JRNL
```

Where:

BACKUP DD specifies the back up copy - 2 of your DFSMSrmm control data set.

JOURNAL DD specifies the back up copy -2, backup copy -1, the newest back up copy of the DFSMSrmm journal and the actual DFSMSrmm journal.

MASTER DD identifies the DFSMSrmm control data set.

SYSPRINT DD contains the utility program messages.

Example 7-16 shows you the correct JCL you have to use if the DFSMSrmm CDS backup copy was created using EDGSKP(DSS) or EDGBKUP(DSS). In this case always the last backup copy of the journal and the actual journal must be used for forward recovery.

Example 7-16 Restoring a control data set if BACKUP(DSS) was used

```
//CDSREST JOB ....
//EDGBKUP EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//BACKUP DD DISP=SHR,DSN=RMM.HSKP.BACKUP(0)
//JOURNAL DD DISP=SHR,DSN=RMM.HSKP.JRNLBKUP(0)
//      DD DISP=SHR,DSN=RMM.PROD.JRNL
```

Where:

BACKUP DD the current actual back up copy of the control data set.

JOURNAL DD identifies the newest DFSMSrmm journal backup copy and actual DFSMSrmm journal to be used for forward recovery.

MASTER DD identifies the DFSMSrmm control data set.

SYSPRINT DD contains the utility program messages.

Restriction: When backup is taken using BACKUP(DSS) and you restore from the latest control data set backup, both the latest journal backup and the active journal must be used for forward recovery.

If you did not perform forward recovery when you restored the CDS backup, you can perform the recovery with the restored CDS as shown in Example 7-17.

Example 7-17 Forward recovery of restored CDS

```
//CDSREST JOB ....
//EDGBKUP EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//JOURNAL DD DISP=SHR,DSN=RMM.PROD.JRNL
```

Where:

JOURNAL DD specifies the actual DFSMSrmm journal.

MASTER DD identifies the DFSMSrmm control data set.

SYSPRINT DD contains the utility program messages.

7.5.3 EDGUTIL

Use the EDGUTIL utility to create or update the control data set control record. To create or update the control record, the user of EDGUTIL must have UPDATE or higher RACF access to the DFSMSRmm control data set.

After DFSMSRmm is implemented, you only need to use this utility to update the control record if you are implementing new functions, or you must correct one of the counts stored in the control record. In this section we describe how you can use the utility to implement new functions.

CATSYNCH

The CATSYNCH function specifies whether the DFSMSRmm control data set and system catalogs are synchronized:

- | | |
|------------|---|
| NO | CATSYNCH(NO) clears the last synchronization date and time. Specify CATSYNCH(NO) to force synchronization of the DFSMSRmm control data set and user catalogs the next time inventory management is run. |
| YES | CATSYNCH(YES) sets the last synchronization date and time to the current date and time. |

Example 7-18 shows a sample EDGUTIL JCL that you can use to set the last synchronization date and time to the current date and time.

Example 7-18 *Set last synchronization date and time*

```
//CDSUPDT JOB ....
//STEP0001 EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.PROD.CDS
//SYSIN DD *
CONTROL CATSYNCH(YES)
/*
```

Note: Do not specify CATSYNCH(YES) if you have not run the EDGHSKP utility with the CATSYNCH parameter on each system to synchronize the DFSMSRmm control data set with the user catalogs.

EXTENDEDDBIN

The EXTENDEDDBIN function enables DFSMSRmm extended bin support, which allows the reuse of bins at the start of a move:

- | | |
|------------|--|
| YES | When extended bin support is enabled, DFSMSRmm records additional information in the volume and bin record while a volume is moving from or to a bin-managed storage location. |
|------------|--|

Note: Do not enable extended bin support until you have made sure that the same level of code has been installed for all the DFSMSRmm systems that share a control data set.

When it is enabled, extended bin support cannot be disabled.

STACKEDVOLUME

The STACKEDVOLUME function enables DFSMSRmm stacked volume support.

YES

When you enable stacked volume support, DFSMSrmm uses the stacked volume records to manage the movement of the volumes contained in the stacked volumes.

Note: Prior to enabling stacked volume support, ensure all systems using the control data set are at the same release level.

You cannot remove stacked volume support after it is enabled.

7.6 Initializing and erasing volumes

EDGINERS is the DFSMSrmm utility for initializing and erasing volumes. You should use this utility as part of the daily inventory management tasks. You can run EDGINERS in manual or in automatic mode. The recommendation is to run in automatic mode unless you need to make any individual change to a volume.

You can use the LABEL procedure, which runs EDGINERS for initializing or erasing volumes manually. This procedure allows you initialize and erase any tape volume whether it is defined to DFSMSrmm. If you initialize a volume that is not defined to DFSMSrmm, when it is initialized, it is automatically defined. The LABEL procedure is supplied as the EDGLABEL member in the SAMPLIB data set.

You should avoid the manual use of EDGINERS by defining all the volumes to DFSMSrmm previously with the RMM ADDVOLUME command, and setting the pending release action to INIT. The following command adds 100 new scratch volumes starting on A00001. These volumes need to be labeled before using as the INIT(Y) operand indicates. The TSO RMM subcommand shown in Figure 7-29 can be used to add the volumes.

```
RMM AV A00001 COUNT(100) STATUS(SCRATCH) INIT(Y)
```

Figure 7-29 TSO ADDVOLUME subcommand to add 100 volumes

Or, if you have the volumes defined to DFSMSrmm, but not with the pending release action INIT, you can change it by using the command you can see in Figure 7-30.

```
RMM CV A00001 INIT(Y)
```

Figure 7-30 TSO CHANGEVOLUME subcommand to set the init flag

You can also add duplicate volumes to DFSMSrmm to be initialized. However, you can also initialize duplicate volumes using EDGINERS, which automatically add the volume to DFSMSrmm. Use the VOL1 parameter shown in Figure 7-31 if you add a duplicate volume.

```
RMM AV A00001 STATUS(SCRATCH) INIT(Y) VOL1(ABC001)
```

Figure 7-31 Adding a duplicate volume

EDGINERS automatic processing is enabled when the INIT and ERASE options are specified in the control data set. When the volumes have the pending release action INIT or ERASE, then they can be processed automatically by DFSMSrmm. There are some required parameters in the EDGINERS JCL EXEC statement for being able to process any volume:

COUNT, INITIALIZE, ERASE, MEDIANAME, POOL, or LOCATION. When EDGINERS is running as an automatic process, it does not require any operator intervention.

To initialize and erase volumes without operator intervention, you can set up automatic processing by specifying any of these EDGINERS EXEC parameters:

ALVER3	Use ALVER3 to set the EDGINERS processing default value for ISO/ANSI label tapes to version 3.
ALVER4	Use ALVER4 to set the EDGINERS processing default value for ISO/ANSI label tapes to version 4.
BATCH(number_of_batches)	Use BATCH to specify the number of batches of volumes to be processed in a single run of EDGINERS automatic processing. Use the COUNT parameter to specify the number of volumes in each batch. The COUNT is the number of volumes that are initialized or erased before DFSMSrmm verifies the volumes. After DFSMSrmm verifies the volumes in a batch, EDGINERS starts again to initialize or erase the volumes in the next batch. The default for BATCH is BATCH(1). To process all volumes that have actions pending, specify BATCH(0).
COUNT(count)	Use COUNT to specify the number of volumes to initialize or to erase when DFSMSrmm performs automatic processing. Use COUNT with the BATCH parameter to specify the number of volumes in each batch of volumes to be processed. The maximum value that you can specify is 99. The default value is 10.
ERASE(DSE INIT SHRED SHREDDSE)	Use ERASE to request that DFSMSrmm selects volumes that have the erase action pending. If automatic processing is in effect but ERASE is not specified then DFSMSrmm will only select volumes with the initialize action pending.
DSE	Specifies that a Data Secure Erase (DSE) should be attempted. This exploits the tape drive hardware capability to erase data from the volume. This is the default value if the ERASE parameter is omitted or specified without a value.
INIT	Specifies that an ERASE action equates to an INIT action; no secure erase is attempted and the volume is relabelled as if the INIT action had been requested.
SHRED	For encrypted volumes, this value specifies that the Data Key should be made unusable by the drive. For non-encrypted volumes the DSE action is attempted.
SHREDDSE	For encrypted volumes, this value specifies that the Data Key should be made unusable by the drive, and that any non-encrypted residual data on the volume should be subject to DSE. For non-encrypted volumes, the DSE action is attempted.
INITIALIZE	Use INITIALIZE to request that DFSMSrmm selects volumes that have the initialize action pending. If automatic processing is in effect but neither INITIALIZE nor ERASE are specified, then INITIALIZE is the default.

LOCATION(library_name)

Use LOCATION to specify a subset of volumes for automatic processing. The library_name must be the name of a system-managed tape library that is on the running system or SHELF. If you specify LOCATION, you cannot specify MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT.

MEDIANAME(medianame | parmlib_default_medianame)

Use MEDIANAME to specify a subset of volumes for automatic processing. If you specify MEDIANAME, you cannot specify LOCATION, MEDIATYPE, POOL, or RECORDINGFORMAT. If you do not specify LOCATION, MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT.

MEDIATYPE(media

Use MEDIATYPE to specify a subset of volumes for automatic processing. Specifies the volume's physical media type. If you specify MEDIATYPE, you cannot specify LOCATION, MEDIANAME, POOL, or RECORDINGFORMAT. Use one of these:

*	The volume is not a cartridge.
CST	Cartridge System Tape
ECCST	Enhanced Capacity Cartridge System Tape
EHPCT	Extended High Performance Cartridge Tape
HPCT	High Performance Cartridge Tape
MEDIA5/ETC	IBM TotalStorage Enterprise Tape Cartridge
MEDIA6/EWTC	IBM TotalStorage Enterprise WORM Tape Cartridge 3592
MEDIA7/EETC	IBM TotalStorage Enterprise Economy Tape Cartridge 3592
MEDIA8/EEWTC	IBM TotalStorage Enterprise Economy WORM Tape Cartridge 3592
MEDIA9/EXTC	IBM TotalStorage Enterprise Extended Tape Cartridge 3592
MEDIA10/EXWTC	IBM TotalStorage Enterprise Extended WORM Tape Cartridge 3592
MEDIA11/EATC	IBM Enterprise Advanced Tape Cartridge
MEDIA12/EAWTC	IBM Enterprise Advanced WORM Tape Cartridge
MEDIA13/EAETC	IBM Enterprise Advanced Economy Cartridge

POOL(pool_prefix)

Use POOL to specify a subset of volumes for automatic processing. A pool prefix is one-to-five alphanumeric, national, or special characters followed by an asterisk (*). The pool must be one that is defined to DFSMSrmm on the running system. If you specify POOL, you cannot specify LOCATION, MEDIANAME, MEDIATYPE, or RECORDINGFORMAT.

RECORDINGFORMAT(format)

Use RECORDINGFORMAT to specify a subset of volumes for automatic processing. RECORDINGFORMAT specifies the basic recording format for tape volumes.

*	An asterisk indicates that the format is unknown or that the volume is not a tape volume.
18TRACK	Data has been written to the volume in 18-track format. A recording format of 18TRACK is valid with MEDIATYPE(CST) and MEDIATYPE(ECCST) only.

36TRACK	Data has been written to the volume in 36-track format. A recording format of 36TRACK is valid with MEDIATYPE(CST) and MEDIATYPE(ECCST) only.
128TRACK	Data has been written to the volume in 128-track format. A recording format of 128TRACK is valid with MEDIATYPE(EHPCT) and MEDIATYPE(HPCT) only.
256TRACK	Data has been written to the volume in 256-track format. A recording format of 256TRACK is valid with MEDIATYPE(EHPCT) and MEDIATYPE(HPCT) only.
384TRACK	Data has been written to the volume in 384-track format. A recording format of 384TRACK is valid with MEDIATYPE(EHPCT) and MEDIATYPE(HPCT) only.
EFMT1	Data has been written to the volume in EFMT1 (enterprise format 1) recording format. A recording format of EFMT1 is valid with MEDIATYPE(MEDIA5), MEDIATYPE(MEDIA6), MEDIATYPE(MEDIA7), and MEDIATYPE(MEDIA8) only.
EFMT2	Data has been written to the volume in EFMT2 (enterprise format 2) recording format. A recording format of EFMT2 is valid with MEDIATYPE(MEDIA5), MEDIATYPE(MEDIA6), MEDIATYPE(MEDIA7), MEDIATYPE(MEDIA8), MEDIATYPE(MEDIA9), and MEDIATYPE(MEDIA10) only.
EEFMT2	Data has been written to the volume in EEFMT2 (enterprise encrypted format 2) recording format. A recording format of EEFMT2 is valid with MEDIATYPE(MEDIA5), MEDIATYPE(MEDIA6), MEDIATYPE(MEDIA7), MEDIATYPE(MEDIA8), MEDIATYPE(MEDIA9), and MEDIATYPE(MEDIA10) only.
EFMT3	Data has been written to the volume in EFMT3 (enterprise format 3) recording format. A recording format of EFMT3 is valid with MEDIATYPE(MEDIA5, MEDIA6, MEDIA7, MEDIA8, MEDIA9, and MEDIA10) only.
EEFMT3	Data has been written to the volume in EEFMT3 (enterprise encrypted format 3) recording format. A recording format of EEFMT3 is valid with MEDIATYPE(MEDIA5, MEDIA6, MEDIA7, MEDIA8, MEDIA9, and MEDIA10) only.
EFMT4	Data has been written to the volume in EFMT4 (enterprise format 4) recording format. A recording format of EFMT4 is valid with MEDIATYPE(MEDIA9, MEDIA10, MEDIA11, MEDIA12, and MEDIA13) only.
EEFMT4	Data has been written to the volume in EEFMT4 (enterprise encrypted format 4) recording format. A recording format of EEFMT4 is valid with MEDIATYPE(MEDIA9, MEDIA10, MEDIA11, MEDIA12, and MEDIA13) only.
STATUS	Use STATUS to control the kind of tapes that you want DFSMSrmm to initialize or erase. The default for STATUS is NOTMASTER. Specifying STATUS requests automatic processing.
ALL	EDGINERS processes all volumes that have the INITIALIZE or ERASE action pending.
NOTMASTER	EDGINERS processes all volumes in SCRATCH, USER, INIT, ENTRY or PENDING RELEASE status that have the

INITIALIZE or ERASE action pending

NOTMASTER is the default.

SCRATCH

EDGINERS processes volumes in SCRATCH, INIT, ENTRY or PENDING RELEASE status that have the INITIALIZE or ERASE action pending.

VERIFY/NOVERIFY

Use VERIFY to request that DFSMSrmm ask the operator to remount each volume that has been successfully erased or labeled. The volumes are requested in reverse order, and the volume labels read to ensure no operator errors have occurred, for example, a mismatch between the internal label and the external label.

For automatic processing VERIFY is the default. For manual processing NOVERIFY is the default.

WRONGLABEL

Use WRONGLABEL to specify the processing that DFSMSrmm performs when a wrong volume is mounted. WRONGLABEL processing does not apply to NL tapes. For NL tapes, DFSMSrmm issues the WTOR EDG6628A to obtain the volume serial number or rack number for the volume that has been mounted. You can use WRONGLABEL when you are running EDGINERS in automatic mode and manual mode.

FAIL

DFSMSrmm does not prompt the operator to accept a mounted volume that does not match the requested volume. The mount request is rejected, the volume demounted, and DFSMSrmm issues message EDG6661E or message EDG6662E.

IGNORE

When the wrong volume is mounted, DFSMSrmm does not issue any operator prompt. DFSMSrmm issues message EDG6661E or EDG6662E to log the relabeling and processing proceeds. This is an extremely dangerous option and should be used with caution because any volume can be relabeled as long as the requested volume has the INIT action or is not defined to DFSMSrmm. Use of this option requires CONTROL access to RACF FACILITY class resource STGADMIN.EDG.INERS.WRONGLABEL.

PROMPT

When an incorrect volume label is detected by EDGINERS for the mounted volume, the operator is always prompted to confirm the processing to be performed. DFSMSrmm issues message EDG6661E or message EDG6662E, followed by message EDG6663D. Processing continues according to the response to message EDG6663D.

RMMPROMPT

When the volume serial number of the mounted volume that is defined to DFSMSrmm does not match the volume serial number of the requested volume, DFSMSrmm issues message EDG6663D to prompt the operator to confirm processing. If the magnetic volume serial number of the tape is not known to DFSMSrmm, initialization continues as if the tape had no magnetic label. If the volume is known to DFSMSrmm, DFSMSrmm issues messages EDG6662E and EDG6663D to prompt the operator or issues message EDG6661E to log the relabeling.

Example 7-19 shows you sample JCL to initialize the next 20 volumes have a MEDIANAME of 3490 without to remount each volume that has been successfully labeled.

Example 7-19 Sample JCL for automatic processing

```
//AUTOINIT EXEC PGM=EDGINERS,  
//          PARM='MEDIANAME(3490),NOVERIFY,COUNT(20)'  
//SYSPRINT DD  SYSOUT=*  
//TAPE     DD  UNIT=(3490,,DEFER)
```

Note: When you request automatic processing, DFSMSrmm does not process any SYSIN commands you specify.

For more information about EDGINERS, and a complete description of the parameters, refer to *DFSMSrmm Guide and Reference*, SC26-7404.

7.7 Disposition processing

DFSMSrmm disposition processing is optional and provides support for:

- ▶ Providing operators with information to assist them in tasks like moving a tape to a specific location
- ▶ Generating sticky labels
- ▶ Updating the location where a volume resides

DFSMSrmm disposition processing occurs at CLOSE and EOVS for each volume. DFSMSrmm does not provide support for asynchronous processing in a separate job step. To implement this function, refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

To produce labels with DFSMSrmm, you can use the default output, it is a WTO on route code 13, or the OUTPUT JCL statement in the DFSMSrmm started procedure to dynamically allocate a SYSOUT file for each label.

Figure 7-32 shows the default label format for a tape cartridge.

...+....1....+....2....+....3....+....4....+....5....+....6....+....7.	
dsname	_____
userdata	_____
jobname_	crdate_____
	expdt_____
den	

Figure 7-32 Default label format for a tape cartridge

The JCL in Example 7-20 shows this function using the default output for the label.

Example 7-20 Sample JCL to create a sticky label

```
//YCJRES1 JOB MSGCLASS=X,NOTIFY=&SYSUID  
//*  
//STEP1 EXEC PGM=IEBGENER,REGION=4M  
//SYSIN DD DUMMY  
//SYSPRINT DD SYSOUT=*
```

```
//SYSUT1 DD DSN=YCJRES1.DSN9,DISP=SHR
//SYSUT2 DD DSN=YCJRES1.TEST.DISP,DISP=(,KEEP),
//        SPACE=(CYL,(2,2)),VOL=SER=TST108,UNIT=3590
//DISPDD DD *
OB90.TST108.0001.THIS IS A TEST
/*
```

Then you run this sample JCL, you see the message shown in Figure 7-33

```
EDG4054I OB90.TST108.0001.THIS IS A TEST
```

Figure 7-33 EDG4054I disposition message

Figure 7-34 shows the sticky label.

```
YCJRES1.TEST.DISP

YCJRES1      2001/052
              2001/055
              IDRC 00121 027951 FB

TST108 0001 SL      OB90
```

Figure 7-34 Example label format

Where:

dsname	YCJRES1.TEST.DISP
jobname	YCJRES1
crdate	2001/052
expdt	2001/055
comp	IDRC
lrec1	00121
blksize	027951
recf	FB
volser	TST108
seqn	0001
lab	SL
devc	OB90

7.8 Software products and foreign tapes

DFSMSrmm provides support for software products and foreign tapes. To manage software tapes, you can define the product to DFSMSrmm, and DFSMSrmm then tracks the volume as a product volume.

To take advantage of product volume management, you need to set up some DFSMSrmm PARMLIB options, define racks numbers, define products to DFSMSrmm, and define the volumes associated with these products.

7.8.1 Defining software products

To define software products and the volumes associated with them, you can use DFSMSrmm ISPF panels or TSO RMM subcommands. In the examples, we used TSO RMM subcommands. You need to follow these steps:

1. If you want to have a separate range for product volumes, define a new VLPOOL in the EDGRMMxx PARMLIB member. The following statement shows a VLPOOL where the prefix for the product volumes is PP. Figure 7-35

```
VLPOOL DESC('software products') EXPDTCHECK(Y) -  
      MEDIANAME(CARTS) PREFIX(PP*) RACF(N) TYPE(R)
```

Figure 7-35 EDGRMMnn VLPOOL parmlib command

2. Restart DFSMSrmm with the new PARMLIB member.
3. Define some empty rack numbers in the pool as shown in Figure 7-36

```
RMM ADDRACK PP0001 COUNT(100)
```

Figure 7-36 RMM ADDRACK subcommand

4. Define the products that you have or plan to have, using the TSO RMM subcommand shown in Figure 7-37.

```
RMM ADDPRODUCT 5695-DF1 NAME('DFSMS') OWNER(YCJRES1) -  
      LEVEL(V02R10M00) DESCRIPTION('DFSMS Version 2.10')
```

Figure 7-37 RMM ADDPRODUCT subcommand

5. Use the TSO RMM ADDVOLUME subcommand to add volumes. The default for LEVEL is V01R01M00. If your level is different, you must specify it. See Figure 7-38 how you can assign volumes to a product .

```
RMM ADDVOLUME DZ11C0 POOL(PP*) MEDIANAME(CARTS) -  
      FEATCD(1234) STATUS(MASTER) NUMBER(5695-DF1) LEVEL(V02R10M00)
```

Figure 7-38 Assigning a volume to a product

Recommendation: If you use the DFSMSrmm panels to add the volumes, DFSMSrmm uses the following defaults:

- ▶ Status: MASTER
- ▶ Retention period: 90 days
- ▶ Release action: return to owner

7.8.2 Foreign tapes

DFSMSrmm can manage tapes that do not follow your naming convention. These volumes usually are those that are sent to you from another site.

DFSMSrmm manages foreign tapes no differently than any other tapes. It records data set names, provides VRS retention and movement controls, and has expiration release options to

help you identify how to remove the foreign tapes from your library. The only volumes that cannot be defined to DFSMSrmm are those that are duplicates of volumes already defined to DFSMSrmm.

The way you define foreign tapes to DFSMSrmm is basically the same way that you use to define software product tapes.

The following steps take you through an example of how to define foreign tapes:

1. Decide on a range of rack numbers to be used for foreign tapes that enter your installation. Consider whether you want to have a separate range for each type of media. If you have both tape reels and tape cartridges to manage as foreign tapes, and want each to have its own range of rack numbers, you need multiple pool definitions.
2. Define a new VLPOOL in DFSMSrmm PARMLIB member. For example, if the prefix for your foreign volumes is FR, you need to add a statement as shown in Figure 7-39.

```
VLPOOL DESC('foreign cartridges') EXPDTCHECK(Y) -  
      MEDIANAME(CARTS) PREFIX(FR*) RACF(N) TYPE(R)
```

Figure 7-39 EDGRMMnn VLPOOL command to add a new pool for foreign volumes

3. Restart DFSMSrmm with the new PARMLIB member.
4. Define some empty rack numbers in the pool by using the TSO RMM ADDRACK command shown in Figure 7-40.

```
RMM ADDRACK FR0001 COUNT(100)
```

Figure 7-40 Adding 100 rack numbers

In this example, volume BB0001 is retained for 90 days and is then released. The release action, RETURN, prevents the volume from going to the scratch pool. You can use it to identify volumes that should be removed from the library, and either returned to their owner or destroyed.

5. To identify volumes waiting for RETURN release action, use the SEARCHVOLUME subcommand you can see in Figure 7-41.

```
RMM SEARCHVOLUME VOL(*) OWNER(*) LIMIT(*) STATUS(RELEASE) -  
      ACTION(RETURN)
```

Figure 7-41 Search for all volumes with a release action RETURN

6. Use the list generated to pull the volumes from the library and return them to their owner.
7. When the volume is removed from the library, confirm to DFSMSrmm that it has been removed. Use the command you can see in Figure 7-42 on page 262 to confirm the release action for volume BB000.

```
RMM CHANGEVOLUME DZ100B CRLSE(RETURN)
```

Figure 7-42 Confirm the return action

When the RETURN action is confirmed, the volume is deleted from the CDS, and the rack it occupied is now empty.

7.8.3 Bypass DFSMSrmm and SMS processing

You have to bypass SMS and optionally DFSMSrmm if you get a foreign tape with a volser you are using in an IBM system managed ATL or VTS. The reason for bypass SMS is, that if SMS finds the volser in the VOLCAT it sends the allocation to the identified library and ignores the UNIT parameter you have specified in your JCL. You can not mount on a manual drive when a physical volume has the same volume serial number as an SMS managed tape volume.

Example 7-21 shows you sample JCL can be used to bypass DFSMSrmm and SMS to copy the first file of a foreign volume with a duplicate volume serial number to your VTS.

Example 7-21 Sample JCL to bypass DFSMSrmm and SMS

```
//S1      EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD   DISP=(OLD,KEEP),           <=== must be OLD
//        DSN=FOREIGN.TAPE.DATASET.FILE1,
//        UNIT=3490,VOL=SER=V00031,
//        EXPDT=98000,                   <=== bypass DFSMSrmm
//        STORCLAS=DUPT$SMS              <=== bypass SMS
//SYSUT2  DD   DISP=(,CATLG),
//        DSN=SCHLUM.FOREIGN.TAPE.DATASET.FILE1,
//        UNIT=VTS,DCB=*.SYSUT1
//SYSIN   DD   DUMMY
```

Restriction: To bypass DFSMSrmm the user must have read access to one of the resources defined in RACF class FACILITY:

```
STGADMIN.EDG.IGNORE.TAPE.* / STGADMIN.EDG.IGNORE.TAPE.V00031
or
STGADMIN.EDG.IGNORE.TAPE.RMM.* / STGADMIN.EDG.IGNORE.TAPE.RMM.V00031
```

7.9 DFSMSrmm volume pools

A *pool* is a group of rack numbers or volumes that share a common prefix. In DFSMSrmm, there are two categories of pools: *rack* and *scratch*.

- ▶ A **rack pool** is shelf space that can be assigned to hold any volumes. Although you can add scratch volumes to these pools, you cannot normally use these volumes to satisfy non-specific mount requests. A rack pool cannot be used with the DFSMSrmm system-based scratch pooling. You can use rack pools for:
 - Volumes that are temporarily brought into the library but are returned to the owner after a period of time
 - Customer, foreign tapes, and software product volumes
 - Scratch volumes for use with either DFSMSrmm exit-selected scratch pooling, or pools based on storage group.
- ▶ A **scratch pool** is shelf space assigned to hold volumes for use with the DFSMSrmm system-based scratch pooling. The volumes assigned to this shelf space can be used to satisfy scratch requests as long as the volumes are in scratch status. Once the volume has been written to, it becomes a volume with MASTER status, until the volume is returned to scratch status. The volume remains in the same DFSMSrmm system-based scratch pool in that it occupies the same shelf space regardless of status.

DFSMSrmm supports the following basic types of pooling:

- ▶ Pools of shelf space that are based on rack number prefixes. You can create different pools with different characteristics such as the media name and management criteria. You must use shelf space pools to store volumes that do not match your installation selected volume ranges.
- ▶ Pools of volumes that are based on the volume serial number prefix. These volumes do not have a rack number or the rack number matches the volume serial number. Each range identifies different characteristics like media name and management criteria.
- ▶ Scratch pools can be one or more pools of volumes. Scratch pools can be based on name, SMS storage group, prefix, or system.
- ▶ When you pool by storage group and use SMS ACS processing to assign a storage group to a tape data set, or the volume is in a system-managed manual tape library, DFSMSrmm ensures that a volume from the correct storage group is mounted. You can use storage group for scratch pooling for system-managed manual tape libraries and non-system managed tape.

7.9.1 Defining pools

You must define any pool you plan to use with DFSMSrmm in the PARMLIB member EDGRMMxx using the VLPOOL command.

You should define at least one pool using the VLPOOL command in the DFSMSrmm PARMLIB member. Use the pools to assign a media name for shelf space and volumes based on a prefix. Also, define a default pool to ensure that volumes and racks that do not match to a more specific VLPOOL are added with correct media name and pool name.

You can use the following operands in the VLPOOL command:

AUTOSCRATCH(YES|NO)

Use this operand to override the scratch processing release action for volumes in this pool.

DESCRIPTION('pool description')

Describes the pool. Use any description from 1 to 32 characters.

EXPDTCHECK(Y|N|O)

Indicates the way to automate the processing of unexpired tape data sets at the pool level:

- | | |
|----------|---|
| N | if you do not want DFSMSrmm to check or validate the expiration date of tape data sets. This is the recommended value when using DFSMSrmm-managed scratch pools. |
| Y | if you want to ensure that all unexpired tape data sets are never overwritten. |
| O | if you do not want DFSMSrmm to take any action, but allows the operator or automation software to reply as necessary to any WTO (IEC507D). This is the default value. |

MASTEROVERWRITE(ADD|LAST|MATCH|USER)

Specify the VLPOOL MASTEROVERWRITE operand to control how DFSMSrmm allows the overwriting of a volume.

MEDIANAME(media_name)

Indicates a name or media type for all volumes in this pool. Some samples are: CARTS, TAPE, LONG, SHORT, 3480, 3490, 3592.

Any name is valid because DFSMSrmm does not check if it is defined to MVS.

NAME(pool_name)

Specify any name to identify the pool in the operator messages and tape drive displays. You must code the RACK(999) operand of MNTMSG command for modifying the messages adding the pool name.

PREFIX(nnnnn*)

Specifies a generic shelf location used in operator messages, RMM TSO commands, and the ISPF dialog. DFSMSrmm uses the prefix value to assign a newly defined volume to a scratch pool.

RACF(YIN)

Specifies the type of RACF support you want DFSMSrmm to provide for the volumes in the pool. If you want to create RACF tape profiles, then specify Y and be sure that OPTION TPRACF(A I P) is specified.

RELEASEACTION(NOTIFY)

Use this operand with the NOTIFY value to automatically set the NOTIFY release action for all volumes in this pool. If you have an e-mail address for the owner of the volume, DFSMSrmm sends the owner notification that the volume is pending release.

SYSID(system_name)

This operand associates a scratch pool with a particular system. DFSMSrmm matches the value with the SYSID operand of the OPTION command. If the VLPOOL SYSID matches the OPTION SYSID then:

A volume from this pool can be used to satisfy a scratch mount request on this system.

A volume from any other pool where its SYSID also matches the OPTION SYSID can be used to satisfy a scratch request on this system.

A volume from a pool that has no explicit VLPOOL SYSID can be used to satisfy a scratch request on a system if there are no other VLPOOL definitions that have a SYSID matching the OPTION SYSID. The SYSID values are ignored when a storage group is selected by ACS processing and when a pool is set for a non-specific volume request by EDGUX100 processing. The system level pools can still be used as long as SMS ACS processing does not assign a storage group and the EDGUX100 exit does not set a specific pool to be use.

TYPE(SIR)

Specifies the type of pool. Use R for rack pools and S for scratch pools.

Figure 7-43 shows some sample pools definitions.

VLPOOL RACF(N) TYPE(R) EXPDTCHECK(O) MEDIANAME(CARTS)	-
AUTOSCRATCH(YES) MASTEROVERWRITE(USER)	-
RELEASEACTION(NOTIFY)	-
DESCRIPTION('3590 POOL') PREFIX(TST*)	
VLPOOL RACF(N) TYPE(S) EXPDTCHECK(Y) MEDIANAME(CARTS)	-
AUTOSCRATCH(YES) MASTEROVERWRITE(ADD)	-
DESCRIPTION('3490 POOL') PREFIX(HGT*)	
VLPOOL RACF(N) TYPE(S) EXPDTCHECK(Y) MEDIANAME(3490)	-
DESCRIPTION('3490 VTS SCRATCH POOL')	-
PREFIX(V*)	
VLPOOL RACF(N) TYPE(S) EXPDTCHECK(O) MEDIANAME(CARTS)	-
DESCRIPTION('DEFAULT POOL') PREFIX(*)	
VLPOOL RACF(N) TYPE(R) EXPDTCHECK(Y) MEDIANAME(CARTS)	-
DESCRIPTION('SOFTWARE PRODUCTS') PREFIX(PP*)	

Figure 7-43 Sample pools definitions

For more details about the pools, refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

7.9.2 Scratch pooling

With OS/390 V2R10, DFSMSrmm extends the choices you have when implementing scratch pooling. You can use any combination of the following methods and can decide at individual request level which method is used:

- ▶ Use DFSMSrmm system-based pooling VLPOOL command with SYSID. This enforces pooling based on prefix pools.
- ▶ Use DFSMSrmm EDGUX100 installation exit to select a pool prefix. You can use any of the information in the system to make a decision about the pool to be used.
- ▶ Use pre-ACS processing to obtain the DFSMSrmm system based pool prefix, or the EDGUX100 exit selected pool prefix in the MSPPOOL read-only variable.
- ▶ Use SMS ACS processing to base scratch pooling on tape storage group names. This support is provided for non-system managed tape and for system-managed tapes libraries:
 - For system-managed tape libraries, a SG corresponds with a library.
 - For non-system managed tape, a SG corresponds with a scratch pool.

The order in which DFSMSrmm validates the scratch mount is:

1. If it is a system-managed ATL, leave validation to the system
2. Storage group assigned for call to SMS ACS routines
3. Pool assigned by EDGUX100 exit
4. DFSMSrmm system based pooling (VLPOOL)

7.10 Deleting non-existent volumes

When you convert to DFSMSrmm, you may have converted information about volumes that you no longer have. Some vendor tape management systems use databases that must be formatted for each range of volumes to be managed. You may have stopped using some of these ranges but never reformatted the database to remove the volumes.

Running under DFSMSrmm, you have the flexibility to add and delete volumes or other resources at any time. Use the knowledge you have about your existing tape volume ranges to check that you have defined to DFSMSrmm only the volumes and rack numbers that you want to manage.

You can use the DFSMSrmm commands to build lists of commands to delete resources you do not want. When you delete volumes, consider deleting the rack numbers that are left in the empty status.

For example, you know that volumes 100000 through 199999 no longer exist but that they were converted to DFSMSrmm. Use the commands shown in Figure 7-44 to delete the volumes and rack numbers.

```
RMM SEARCHVOLUME VOLUME(1*) OWNER(*) LIMIT(*) -  
  NOLIST CLIST('RMM DV ',' FORCE')  
EXEC EXEC.RMM LIST  
RMM DELETERACK 100000 COUNT(99999)  
RMM DELETERACK 199999 COUNT(1)
```

Figure 7-44 Delete non-existing volumes and racks

Refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for information about the commands used. You can repeat these commands for each range of volumes and rack numbers you want to delete.

You can avoid the need to do cleanup by deleting the volume information during conversion. Use the EDGCNXIT user exit from EDGCNVT to check for ranges of volumes you want deleted, and tell EDGCNVT to skip the volumes and data set records within those ranges.

7.11 Repairing actions

In a system-managed tape library environment, there are three places to record the tape volume information:

- ▶ Library manager database (LM DB)
LM DB records information about all tape volumes in the tape library, including logical volumes in order to control the IBM tape library.
- ▶ Tape configuration database (TCDB)
TCDB is a ICF catalog that contains information of tape volumes, which are in the connected tape libraries. TCDB is always required when you control IBM tape libraries with DFSMS regardless of the existence of DFSMSrmm. TCDB is not needed if you plan to use basic tape library support to control your tape libraries.
- ▶ DFSMSrmm control data set
DFSMSrmm uses this data set to record all volumes, not only tape library volumes, but also non-tape library tape volumes. More information is recorded in CDS than in TCDB. The control data set also contains information about tape data sets.

All tape volume information in the LM DB, TCDB and DFSMSrmm CDS should be consistent. DFSMSrmm provides the EDGUTIL utility for you to maintain the CDS. Also, you can use the ISMF AUDIT function to compare TCDB with LM DB. Figure 7-45 on page 268 shows the difference between the audit process before DFSMSrmm OS/390 Release 10 and with Release 10.

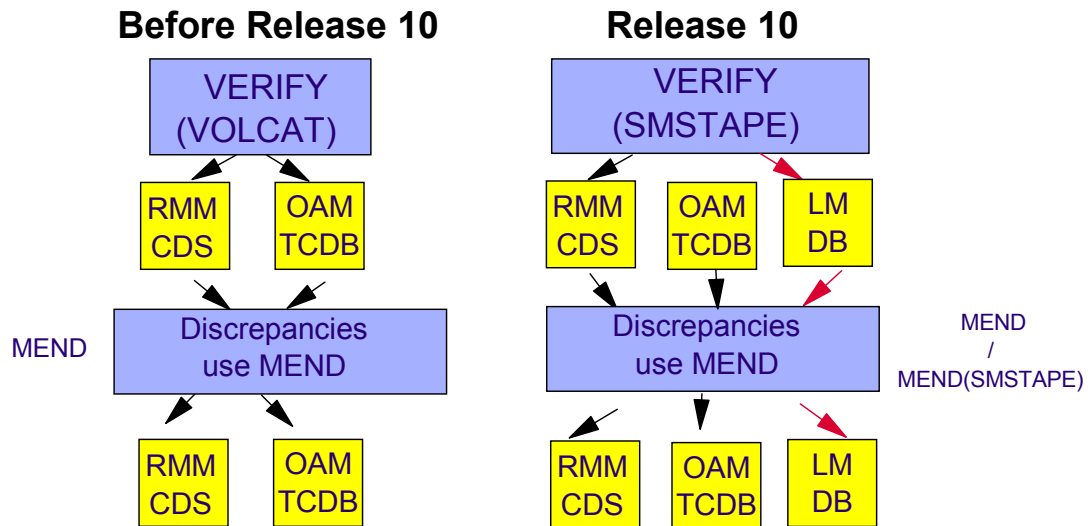


Figure 7-45 Three-way audit of CDS, TCDB and LM DB

7.11.1 Using the VERIFY parameter

You can use the EDGUTIL utility with the VERIFY parameter to verify either active or inactive CDS. Figure 7-46 shows the VERIFY parameter syntax.

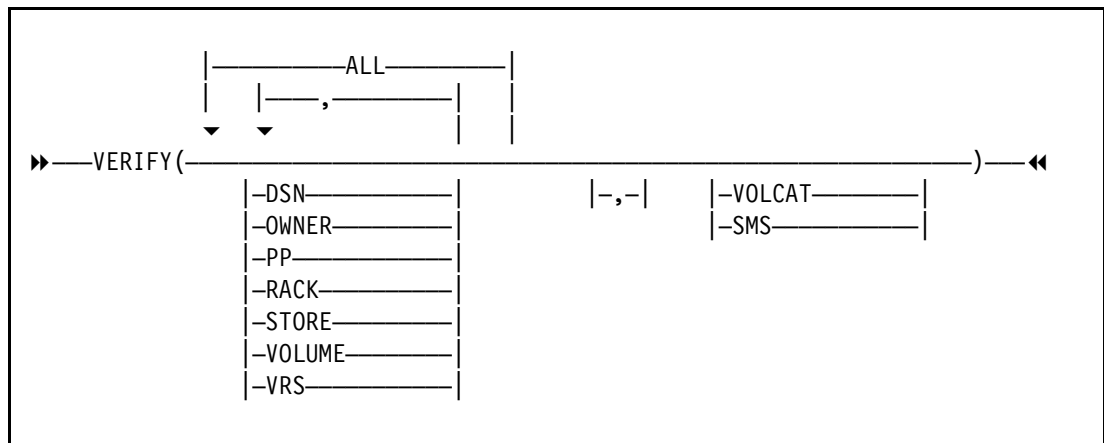


Figure 7-46 EDGUTIL VERIFY syntax diagram

Verify processing can check the consistency between the records inside the DFSMSrmm CDS. In addition, the following actions occur:

- ▶ When you specify VERIFY(VOLCAT), DFSMSrmm checks the information between CDS and TCDB.
- ▶ When you specify VERIFY(SMSTAPE), DFSMSrmm checks the synchronization of the DFSMSrmm CDS with the TCDB, and in addition checks the LM DB if needed.

For VERIFY(SMSTAPE), DFSMSrmm scans both the DFSMSrmm control data set and the TCDB sequentially to ensure that volume records reside in both of them. If any system-managed volumes are only defined to DFSMSrmm CDS or the TCDB, DFSMSrmm checks it against the LM DB.

This function is available with the release 10 enhancement, and is meant to be the replacement of VERIFY(VOLCAT).

Example 7-22 *JCL to verify all information in CDS*

```
//YCJRES4L JOB .....
//UTIL EXEC PGM=EDGUTIL,PARM='VERIFY'
//SYSPRINT DD DSN=RMM.YCJ4.MESSAGE.UTIL,DISP=SHR
//MASTER DD DSN=RMM.PROD.CDS,DISP=SHR
//SYSIN DD DUMMY
```

With the enhancement of DFSMSrmm OS/390 Release 10, you can perform a cross-check against DFSMSrmm, TCDB and LM DB.

By default, all volumes are verified. The SYSIN commands in Figure 7-47 can be used to select the subset of volumes.

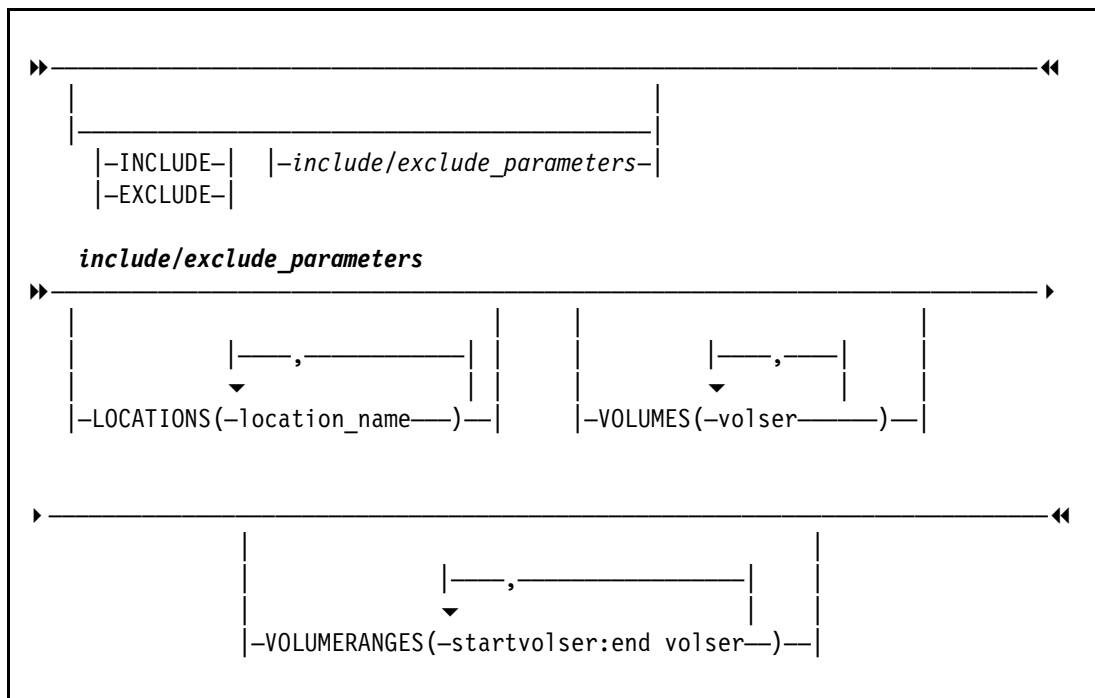


Figure 7-47 EDGUTIL SYSIN parameter syntax

7.11.2 Using the MEND parameter

You can only perform the MEND process with an inactive CDS.

When using the MEND parameter, the same verification as with VERIFY is performed and any error status found is corrected. Information in LM DB or TCDB is not checked before the DFSMSrmm Release 10 enhancement.

If you successfully enabled stacked volume support with the EDGUTIL UPDATE option STACKEDVOLUME(YES), during MEND processing, DFSMSrmm creates stacked volumes in the DFSMSrmm control data set.

When you run MEND for the first time after enabling stacked volume support, DFSMSrmm needs MEDIANAME information for the stacked volumes it creates from the container information. MEDIANAME information is defined in the VLPOOL statements in PARMLIB. To ensure that this is available for EDGUTIL, you must have started DFSMSrmm at least once with the correct VLPOOL definitions before running EDGUTIL MEND.

Figure 7-48 shows the MEND parameter syntax.

Restriction: Do not enable stacked volume support until all systems using the control data set are on a supporting release level.

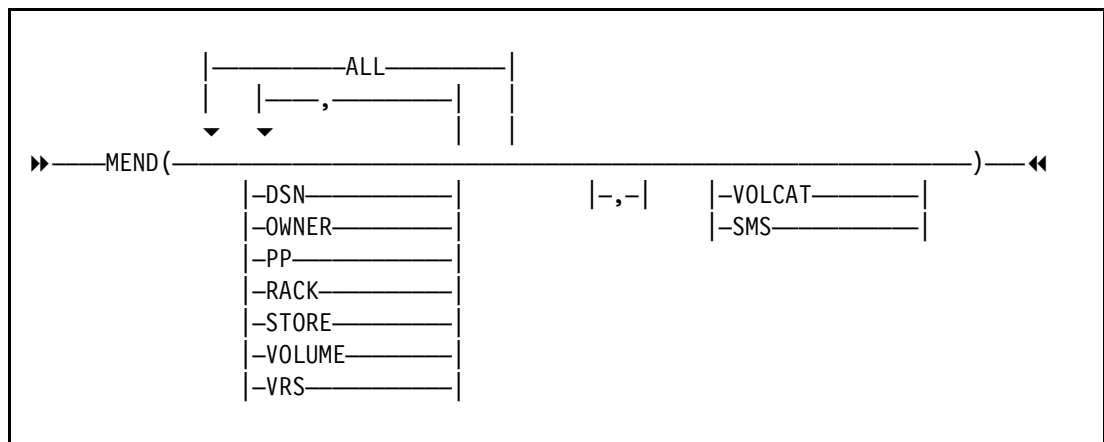


Figure 7-48 EDGUTIL MEND syntax diagram

The default value for MEND operand is ALL, however you can mend a subset of specific record types to increase the performance. Example 7-23 shows a sample JCL to MEND data set and volume records.

Example 7-23 JCL to MEND data set and volume records only

```
//YCJRES4L JOB CLASS=A,MSGCLASS=X,  
//          NOTIFY=&SYSUID,TIME=1440  
//UTIL     EXEC PGM=EDGUTIL,PARM='MEND(VOLUME,DSN)'  
//SYSPRINT DD DSN=RMM.YCJ4.MESSAGE.UTIL,DISP=SHR  
//MASTER  DD DSN=RMM.PROD.CDS,DISP=SHR
```

After MEND processing is completed, you might need to add or change some volume information. Use the RMM CHANGEVOLUME subcommand to set or change any information.

With the DFSMSrmm Release 10 enhancement, MEND processing can also correct CDS information from TCDB or LM DB. Optionally, it can create a stacked volume record in order to support the VTS enhanced support and perform a 3-way audit to check the information about stacked volumes and logical volumes (see Figure 7-49).

3-way audit overview

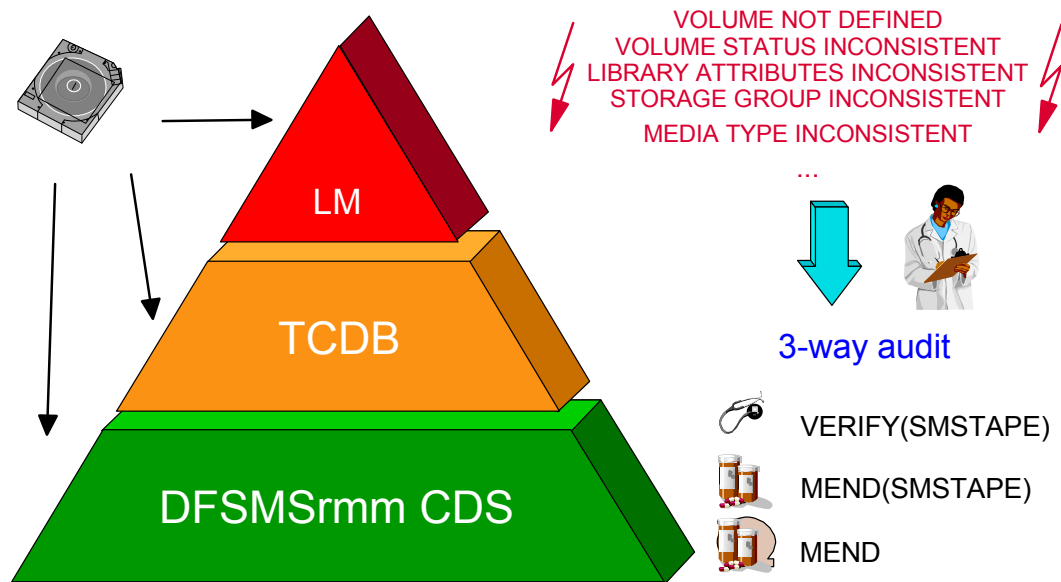


Figure 7-49 DFSMSrmm CDS, Volume Catalog, Library Manager constancy check

When you specify MEND(SMSTAPE), DFSMSrmm performs the same verification action as VERIFY(SMSTAPE) and corrects any error status found. This processing uses information from DFSMSrmm CDS for volume status and storage groups to correct the TCDB and LM DB. Figure 7-50 on page 272 compares the MEND and MEND(SMSTAPE) processing.

MEND(SMSTAPE) versus MEND

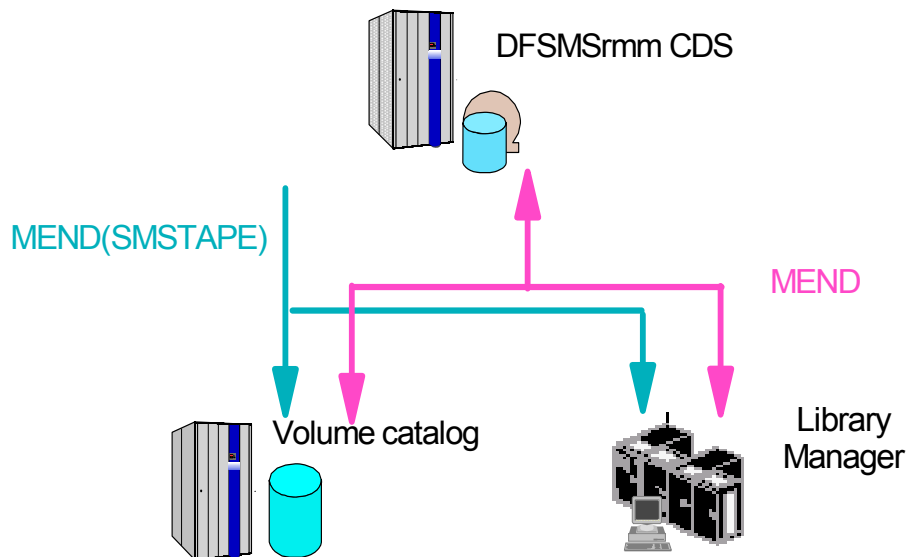


Figure 7-50 *EDGUTIL MEND(SMSTAPE) does not update the CDS*

You can run MEND(SMSTAPE) processing against either an active or inactive CDS. This function is available with the DFSMSrmm Release 10 enhancement.

In z/OS V1.9, DFSMSrmm utility EDGUTIL is no longer considered to be part of inventory management processing. EDGUTIL can run at any time, even in parallel with other EDGUTIL instances. Also a subset of volumes can be selected to be processed on a run of EDGUTIL. An optional SYSIN file allows you to select the subset from the available locations, and volume entries during verification of volumes.

By default, all volumes are verified. The SYSIN commands in Figure 7-51 on page 273 can be used to select the subset of volumes.

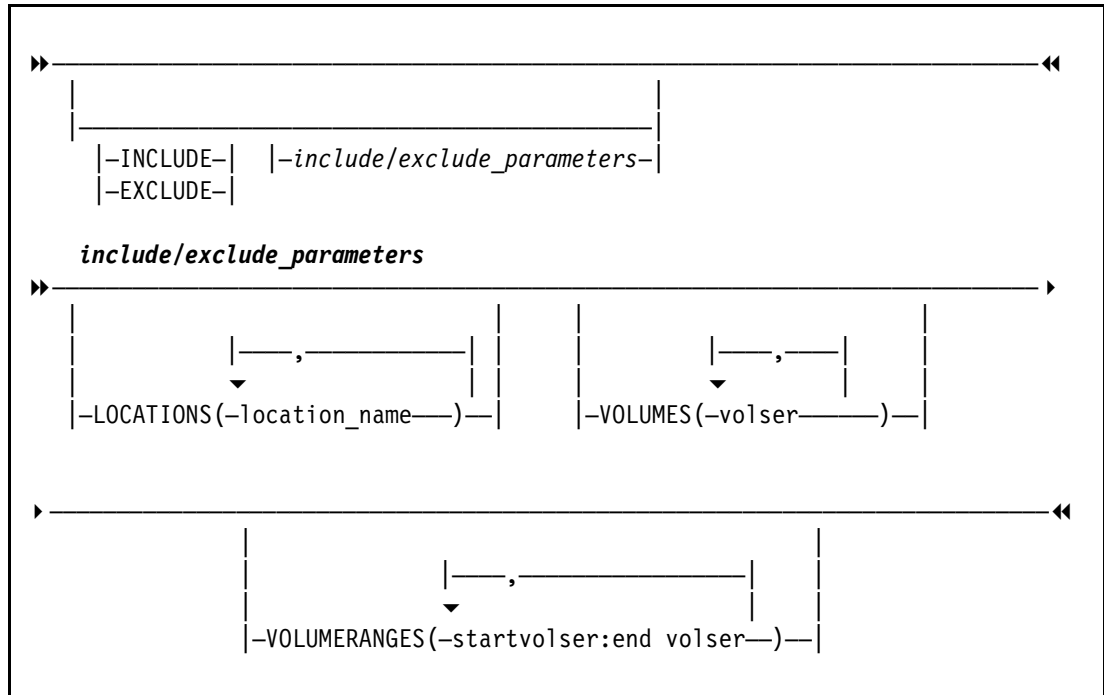


Figure 7-51 EDGUTIL SYSIN parameter syntax

Deferred correcting TCDB and LM database

Correction of the tape configuration database (TCDB) and the library management database (LM) based on the RMM CDS information was previously done by using EDGUTIL with `PARM='MEND(SMSTAPE)'`. The correction was done synchronously to the verify processing, resulting in a long elapsed time.

In z/OS V1R9, by using EDGUTIL with `PARM='VERIFY(SMSTAPE)'` and EDGSPLCS DD and the EDGSPLCS utility:

- ▶ Control statements are generated in the EDGSPLCS output file by EDGUTIL.
- ▶ Volumes can be corrected once EDGUTIL processing is completed by using these statements as an input for the EDGSPLCS utility.

Figure 7-52 is a example of using the deferred TCDB and LM database correction method.

```
//NAID00MN JOB (POK,999),MSGCLASS=T,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC63
//HSKP EXEC PGM=EDGUTIL,PARM='VERIFY(SMSTAPE)'
//EDGSPLCS DD DISP=SHR,DSN=NAID00.SPLCS.DATA
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
INCLUDE VOLUMES(M*,V*,T*)
/*
```

Figure 7-52 EDGUTIL VERIFY(SMSTAPE)

Note: The EDGSPLCS DD:

- Is written during VERIFY(SMSTAPE).
- If specified it causes EDGUTIL to create statements to be used with the EDGSPLCS utility.

Figure 7-53 is the output from SYSPRINT DD.

```
INCLUDE VOLUMES(M*,V*,T*)
EDG6433I STARTING VERIFICATION OF VOLUME RECORDS
EDG6824I VOLUME TST020 IS IN VOLUME CATALOG ERROR STATUS 0004 SECURITY CONFLICT
EDG6846I VOLUME CATALOG UPDATE REQUIRED - STATEMENT WRITTEN TO EDGSPLCS FILE
FOR VOLUME TST020
EDG6824I VOLUME TST021 IS IN VOLUME CATALOG ERROR STATUS 0004 SECURITY CONFLICT
EDG6846I VOLUME CATALOG UPDATE REQUIRED - STATEMENT WRITTEN TO EDGSPLCS FILE
FOR VOLUME TST021
EDG6418W CONTROL DATA SET VERIFY COMPLETED WITH ERRORS
EDG6901I UTILITY EDGUTIL COMPLETED WITH RETURN CODE 4
```

Figure 7-53 SYSPRINT DD output

Where:

EDG6846I

**VOLUME CATALOG UPDATE REQUIRED - STATEMENT
WRITTEN TO EDGSPLCS FILE FOR VOLUME *volser***

Explanation:

You are running the DFSMSrmm EDGUTIL utility to VERIFY SMS information for volumes. DFSMSrmm found inconsistencies between the DFSMSrmm information for the volume and the volume catalog and Library Manager database. This message indicates that the error can be corrected. A control statement has been written to the data set represented by the EDGSPLCS DD statement.

In the message text:

volser

This is the volume serial number of the volume to be corrected.

System action:

DFSMSrmm utility continues. A control statement has been written to the data set specified by the EDGSPLCS DD statement.

Operator response: None.

Application Programmer Response:

Use the EDGSPLCS utility to process the statements produced. You can review the contents of the EDGSPLCS file and if you want to make those corrections, pass the file to the EDGSPLCS INDD file.

Source:

DFSMSrmm

Detecting Module:

EDGUTIL

Figure 7-54 is the corresponding EDGSPLCS generated statements.

S	TST020	LIB1
P	TST021	LIB1

Figure 7-54 EDGSPLCS DD statements

7.11.3 Using EDGSPLCS

You can use the EDGSPLCS utility to issue supported commands to **OAM** for system-managed volumes. DFSMSrmm builds the input commands for this utility automatically during EDGUTIL VERIFY(SMSTAPE) processing and EDGHSKP EXPROC processing when you request them.

You can run multiple copies of EDGSPLCS. Using different parameters, EDGSPLCS can be processing in parallel for multiple libraries, but this utility does not ensure that each parameter is different from any other currently running.

You need RACF ALTER authority to the relevant volume catalog in order to use the EDGSPLCS utility to update the TCDB. For example, if you use just one volume catalog and use the default volume catalog prefix, you need ALTER access to SYS1.VOLCAT.VGENERAL.

Running EDGSPLCS

Example 7-1 shows the sample JCL for execution of EDGSPLCS. The INDD file in the JCL is partly created from the running programs described in “Deferred correcting TCDB and LM database” on page 273.

```
//EDGSPLCS JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
//EDGSPLCS EXEC PGM=EDGSPLCS,PARM='ACTION(S,P)'
//OUTDD DD SYSOUT=*
//INDD DD DISP=SHR,DSN=NAID00.SPLCS.DATA
```

Figure 7-55 EDGSPLCS JCL

You are also able to specify INDD DD * to specify some control cards directly as shown in Figure 7-56.

Restriction: The position of the action characters and the operands for INDD are fixed and cannot be changed.

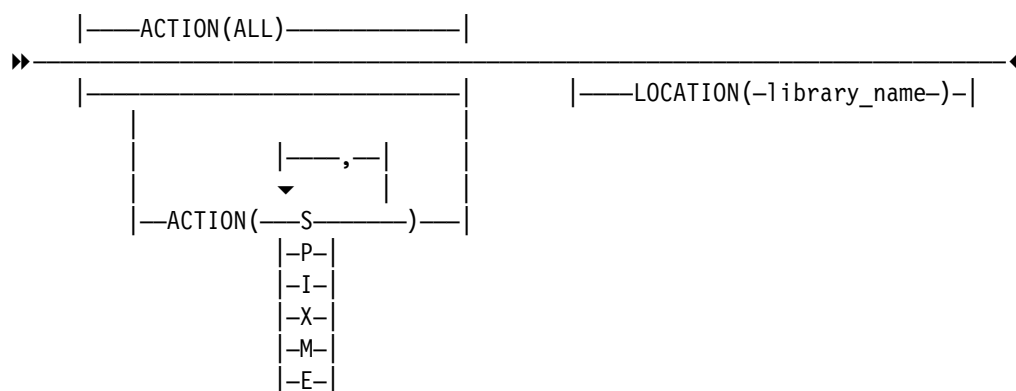
```
//EDGSPLCS JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
//EDGSPLCS EXEC PGM=EDGSPLCS,PARM='ACTION(S,P)'
//OUTDD DD SYSOUT=*
//INDD DD *
P THS000 LIB2
S THS000 LIB2
/*
```

Figure 7-56 EDGSPLCS JCL example with DD *

EXEC Parameters for EDGSPLCS

Example 7-24 shows the execution parameters for EDGSPLCS.

Example 7-24 EDGSPLCS EXEC Parameters



Where:

ACTION(ALL/S/P/I/X/M/E)

Specifies that only the specified requests in the input file are processed. You can optionally provide the name of a library to restrict the processing to only those requests.

You can specify one or more of the possible actions.

S set to **S**cratch status.

P set to **P**rivate status.

I **I**mport volume.

X **eX**port volume.

M **M**anual cartridge entry.

E **E**ject volume.

If you do not specify the ACTION parameter, the default value is ALL.

LOCATION(library_name)

Specifies the name the system-managed library for which the EDGSPLCS utility will process commands during this run. By default, all locations are considered. However, you can select a subset based on the library name using this parameter.

INDD Input File

Figure 7-57 displays an existing file of LRECL 80 containing control statements that direct the processing that EDGSPLCS utility performs. The control statements in INDD have to format shown in Table 7-3 on page 277

Symbol	Explanation	Values
message	Output area for EDGSPLCS	After processing, this area contains a function specific message from the EDGSPLCS utility.

OUTDD Output File

This is the output file that is written by the EDGSPLCS utility. It contains a copy of each of the input control statements, and each statement contains a completion message as shown in Figure 7-58 on page 278. The return and reason codes EDGSPLCS have added starting in column 36 are described in Table 7-4 on page 278

av	volser	options	library	message
12	4	1	2	3
		1	7	6
SV	THS012		LIB2	QVR RETURN CODE = 0004, REASON CODE = 0063.
S	THS013			CUA RETURN CODE = 0012, REASON CODE = 0063.
S	THS015		LIB2	USE ATTRIBUTE NOT CHANGED

Figure 7-58 EDGSPLCS output record format

Refer to “DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries, Chapter 6. Library Control System (LCS) External Services” to find the description of the return and reason codes.

Return Codes for EDGSPLCS

EDGSPLCS issues these return codes shown in Table 7-4.

Table 7-4 EDGSPLCS Return Codes

Return Code	Explanation
0	All requested functions completed successfully. Refer to OUTDD records for individual messages from the input actions.
4	DFSMSrmm encountered a minor error during processing. Refer to OUTDD records for individual messages from the input actions.
8	All requested functions completed successfully. Refer to OUTDD records for At least one requested action was not supported. Refer to OUTDD records for individual messages from the input actions.
12	DFSMSrmm encountered a severe error during processing. DFSMSrmm stops the utility.

Reason Codes for EDGSPLCS

EDGSPLCS issues these reason codes shown in Table 7-5.

Table 7-5 EDGSPLCS Reason Codes

Reason Code	Explanation
0	SUCCESSFUL EXECUTION
2	OAM CONTROL BLOCKS NOT AVAILABLE
3	DELETED WITH MTL SOFTWARE-ONLY

Reason Code	Explanation
4	VOLUME ALREADY SCRATCH
5	VOLUME ALREADY PRIVATE
6	CUA PROCESSING DISABLED
7	CUA PROCESSING NOT PERFORMED FOR THIS VOLUME PER INSTALLATION EXIT REQUEST
8	SCRATCH VOLUME THRESHOLD MESSAGE PROCESSING WAS NOT COMPLETED SUCCESSFULLY
9	REQUIRED TYPE PARAMETER NOT SPECIFIED
10	MUTUALLY EXCLUSIVE REQUIRED PARAMETERS SPECIFIED
11	INVALID TYPE VALUE SPECIFIED
12	REQUIRED FUNC PARAMETER NOT SPECIFIED
13	INVALID FUNC VALUE
14	REQUIRED USE PARAMETER NOT SPECIFIED
15	INVALID USE VALUE
16	REQUIRED VOLUME PARAMETER NOT SPECIFIED
17	INVALID VOLUME VALUE
18	REQUIRED UCBPTR NOT SPECIFIED
19	INVALID UCBPTR VALUE SPECIFIED
20	REQUIRED VOLLIST NOT SPECIFIED
21	INVALID VOLUME LIST VALUE
22	INVALID VALUE IN STORAGE GROUP LIST HEADER
23	REQUIRED LIBRARY NAME NOT SPECIFIED
24	INVALID VOLUME LIST, MIXED MEDIA, REWRITABLE AND WORM VOLUMES
25	INVALID EXPIRATION DATE VALUE
26	LIBRARY NOT DEFINED TO STORAGE GROUP
27	INVALID LIBRARY NAME SPECIFIED
28	INVALID WRITE PROTECT STATUS VALUE
29	INVALID ADDRESS SPECIFIED FOR PARAMETER LIST OR MAPPING MACRO
30	ADDRESS NOT ON WORD BOUNDARY OR LEVEL
31	REQUIRED TAPE DEVICE SELECTION INFORMATION (TDSI) NOT SPECIFIED
32	REQUIRED LIBRARY ID OR LIBRARY NAME NOT SPECIFIED
33	INVALID VALUE SPECIFIED FOR LIBRARY ID
34	INVALID STORAGE GROUP NAME
35	REQUIRED MEDIA TYPE NOT SPECIFIED FOR MCE VOLUME

Reason Code	Explanation
38	INVALID COMPACTION SPECIFIED IN TDSI
39	INVALID SPECIAL ATTRIBUTE SPECIFIED IN TDSI
40	INVALID COMBINATION OF TAPE DEVICE SELECTION VALUES SPECIFIED
41	AMBIGUOUS TDSI COMBINATION SPECIFIED
42	TAPE DEVICE SELECTION VALUE SPECIFIED WHERE NOT ALLOWED OR NOT APPLICABLE
43	INVALID POINTER TO TDSI SPECIFIED
44	INVALID VALUE SPECIFIED FOR DISP KEYWORD
45	LIBRARY NAME AS DEFINED IN VOLUME RECORD NOT FOUND IN TCDB
46	NO ENABLED STORAGE GROUPS
47	NOT ALL VOLUMES ASSOCIATED WITH THE SAME STORAGE GROUP
48	TORAGE GROUP STATE IS NOTCON, DISALL, OR DISNEW
49	NO DEVICE POOLS EXIST TO FULFILL REQUEST FOR TDSI SPECIFICATION
51	SPECIFIC VOLSER REQUEST FOR SCRATCH VOLUME
52	VOLUME(S) RESIDE OUTSIDE LIBRARY
53	LIBRARY FOR SPECIFIED VOLUME NOT DEFINED TO SMS CONFIGURATION
54	SMS STORAGE GROUP WAS NOT OF TYPE TAPE
55	REQUESTED DEVICE DOES NOT RESIDE IN SAME LIBRARY AS REQUESTED VOLUME
56	NO LIBRARIES ASSOCIATED WITH LIST OF STORAGE GROUPS OR LIBRARY IS UNKNOWN
58	FAILURE ACCESSING THE VOLUME RECORD IN THE CATALOG
59	FAILURE ACCESSING THE LIBRARY RECORD IN THE CATALOG
60	FAILURE ACCESSING THE SMS STORAGE GROUP CONSTRUCTS
61	FAILURE ACCESSING HARDWARE CONFIGURATION INFORMATION
62	SPECIFIED LIBRARY IS NOT DEFINED TO ACTIVE SMS CONFIGURATION
63	VOLUME RECORD NOT FOUND FOR REQUESTED VOLUME
64	LIBRARY RECORD NOT FOUND IN TCDB FOR REQUESTED LIBRARY
65	LIBRARY LOGICAL TYPE NOT DEFINED
66	NO DEVICE POOLS TO FULFILL REQUEST FOR SPECIFIED RECORDING TECHNOLOGY
67	NO DEVICE POOLS TO FULFILL REQUEST FOR SPECIFIED MEDIA TYPE
69	REQUEST FAILED BECAUSE VOLUME NOT IN LIBRARY INSTALLATION EXIT
70	VOLUME NOT FOUND IN LIBRARY MANAGER INVENTORY
72	DELETED WITH MTL SOFTWARE-ONLY

Reason Code	Explanation
74	REQUEST FAILED BECAUSE VOLUME SERIAL NUMBER ALREADY EXISTS IN LIBRARY MANAGER INVENTORY
75	UNEXPECTED UCBSCAN ERROR ENCOUNTERED DURING PROCESSING
76	DELETED WITH MTL SOFTWARE-ONLY
77	DELETED WITH MTL SOFTWARE-ONLY
78	ERROR ATTEMPTING TO RETRIEVE VOLUME RECORD
79	ERROR ATTEMPTING TO WRITE VOLUME RECORD
80	ESTAE ROUTINE NOT ESTABLISHED
81	GETMAIN FAILED FOR DEVICE POOL NAMES LIST OR LOCAL WORKING STORAGE
82	ABNORMAL TERMINATION OCCURRED DURING INSTALLATION EXIT (CBRUXCUA) EXECUTION
83	INVALID RETURN CODE OR DATA RETURNED FROM CHANGE USE ATTRIBUTE INSTALLATION EXIT (CBRUXCUA)
84	ABNORMAL TERMINATION OCCURRED DURING EXECUTION
90	CARTRIDGE ENTRY PROCESSING HAS BEEN DISABLED
91	CARTRIDGE ENTRY PROCESSING HAS BEEN SUSPENDED FOLLOWING ERROR INVOKING INSTALLATION EXIT
92	LIBRARY FOR MCE NOT MANUAL TAPE LIBRARY
93	LIBRARY OFFLINE, PENDING OFFLINE, OR NOT OPERATIONAL
94	SPECIFIED VOLUME ALREADY RESIDES IN ANOTHER LIBRARY
95	INSTALLATION EXIT VETOED ENTRY OF VOLUME INTO LIBRARY
96	VOLUME NOT ENTERED INTO MTL BECAUSE INSTALLATION EXIT SAID TO IGNORE THE VOLUME
97	VOLUME OF SAME VOLSER IS KNOWN DASD VOLUME
120	VOLUME IS INELIGIBLE BECAUSE THE TYPE OF MEDIA DEFINED IN THE VOLUME RECORD MAY NOT BE MOUNTED ON SPECIFIED DEVICE
121	VOLUME IS INELIGIBLE BECAUSE THE TYPE OF MEDIA DEFINED IN THE TDSI DOES NOT MATCH MEDIA DEFINED ON THE VOLUME RECORD
122	VOLUME IS INELIGIBLE BECAUSE THE VOLUME RECORD REFLECTS AN ERROR STATUS
123	VOLUME IS INELIGIBLE BECAUSE THE SPECIFIED RECORDING TECHNOLOGY IS INCOMPATIBLE WITH THE VOLUME MEDIA TYPE OR THE SPECIFIED DRIVE TYPE
130	SPECIFIED VOLUME ALREADY RESIDES IN THIS TAPE LIBRARY
131	SCRATCH VOLUME THRESHOLD PROCESSING NOT PERFORMED BECAUSE LIBRARY WAS NOT OPERATIONAL
132	DELETED WITH 3590 SUPPORT
133	DELETED WITH 3590 SUPPORT

Reason Code	Explanation
134	NO TCDB RECORD BUT VOLUME RESIDES IN SPECIFIED LIBRARY
135	LIBRARY MISMATCH, VOLUME RESIDES IN SPECIFIED LIBRARY
136	LIBRARY MISMATCH, VOLUME NOT FOUND IN SPECIFIED LIBRARY
137	LIBRARY MISMATCH, UNABLE TO ACCESS SPECIFIED LIBRARY
138	LIBRARY SCRATCH COUNT NOT UPDATED IN TCDB
139	LIBRARY NAME MISMATCH, SPECIFIED LIB NAME DID NOT MATCH MTL VOLUME RECORD LIB NAME
201	VOLUME ALREADY EJECTED
202	INVALID VALUE FOR EJECT OPTION
203	INVALID VALUE FOR BULK EJECT
204	INVALID TSO USERID SPECIFIED
215	NOT ALL VOLUMES HAVE THE SAME RECORDING TECHNOLOGY
216	INVALID TCDBCHK=NO SPECIFIED WITH MTL LIB NAME
217	AT LEAST ONE OPTIONAL KEYWORD MUST BE SPECIFIED WITH FUNCTION
218	REQUIRED DATATYPE NOT SPECIFIED
219	INVALID POLICY NAME
220	MUTUALLY EXCLUSIVE OPTIONAL KEYWORDS SPECIFIED
221	INVALID STORAGE HEADER AND/OR LENGTH
300	OAM ABEND DURING EJECT REQUEST
302	EJECT REQUEST ALREADY PENDING FOR VOLUME
303	UNABLE TO MAKE USER ADDRESS SPACE NON-SWAPPABLE
304	TCDB ACCESS ERROR IN OAM
305	TCDB AUTHORIZATION ERROR IN OAM
306	OAM INTERNAL ERROR
307	VOLSER NOT IN TCDB
310	MEDIA TYPE OR RECORDING TECHNOLOGY NOT SUPPORTED AT THIS SOFTWARE LEVEL
311	DELETED WITH MTL SOFTWARE-ONLY
312	FUNCTION NOT COMPATIBLE WITH THE LIBRARY
313	VOLUME IS CURRENTLY IN USE
314	IMPORT/EXPORT ALREADY IN PROGRESS OR HOST PROCESSING NOT COMPLETE
315	NOT ENOUGH PHYSICAL DRIVES AVAILABLE IN VTS
316	IMPORT/EXPORT NOT IN PROGRESS

Reason Code	Explanation
317	EMPTY CATEGORY IMPORT: NO IMPORT VOLUMES EXPORT/IMPORT: NO SCRATCH VOLUMES
318	MAXIMUM LOGICALS DEFINED TO LIBRARY
319	NO TCDB RECORD AND VOLUME NOT FOUND IN SPECIFIED LIBRARY
320	NO TCDB RECORD AND UNABLE TO ACCESS SPECIFIED LIBRARY
321	MANUAL CARTRIDGE ENTRY FAILED MEDIA TYPE RETURNED FROM THE INSTALLATION EXIT DOES NOT MATCH THE MEDIA TYPE DEFINED IN THE VOLUME RECORD.
322	COMMAND REJECTED BY THE LIBRARY
323	UNABLE TO RETRIEVE POLICY NAME(S) FROM LIBRARY
324	VOLUME EXPIRE TIME HAS NOT ELAPSED
325	I/O TERMINATED DUE TO TIMEOUT DETECTION
400	OAM INITIALIZED WITH NULL CONFIGURATION
401	LIBRARY NOT ACCESSIBLE, OFFLINE, OR NOT OPERATIONAL
402	VISION SYSTEM NOT OPERATIONAL
403	EJECT PROCESSING HAS BEEN DISABLED BECAUSE AN ERROR IN THE EJECT INSTALLATION EXIT (CBRUXEJC) HAS BEEN DETECTED
404	OAM ADDRESS SPACE NOT AVAILABLE

7.11.4 DFSMSrmm CDS and DFSMShsm consistency checking

As mentioned at the beginning of this chapter, the DFSMSrmm CDS is the most important part of your DFSMSrmm environment. Ensuring availability of your DFSMSrmm controlled data requires ensuring the health of your control data set. This assurance can be achieved by running the standard DFSMSrmm EDGUTIL VERIFY against the DFSMSrmm CDS. You can also run Mainstar: *FastAudit/390* Tape Audit facility, a selectable unit of the Mainstar: *FastAudit/390* product suite.

Mainstar's *FastAudit/390* product is widely known and used in the z/OS and OS/390 environment. The Tape Audit function was developed to ensure that your DFSMSrmm CDS is healthy, in good shape, and that all data controlled by DFSMSrmm can be accessed when needed. One of the specific audits available and performed is the extracts of all references to tape from DFSMShsm CDSs (except ABARS tape) and compared to the DFSMSrmm CDS.

Other DFSMSrmm diagnostics are:

- ▶ Finding tapes in TTOC and MCT/MCD, but not in DFSMSrmm.
- ▶ Finding tapes that are permanent in DFSMSrmm, but not in TTOC.
- ▶ Finding tapes that are in DFSMSrmm, valid in DFSMShsm, but no entries are valid entries.
- ▶ Finding tapes that are in DFSMShsm but expired in DFSMSrmm.
- ▶ Finding tapes that are active DFSMSrmm scratch status.

The audits are performed by submission of a single batch job. This batch job performs a multiple of standard equivalent DFSMShsm command audits without adding the additional increase to the system, DFSMSrmm or your DFSMShsm environments.

The key to finding the errors is to correctly verify and report these errors accurately. Mainstar's Tape Audit accomplishes this by doing a direct catalog lookup for every instance an error is located, meaning the product is bi-directional on finding, verifying and reporting errors. When the errors are verified, Mainstar's *FastAudit/390* provides an ISPF interface for easy and organized viewing of the errors and files in question. The *FastAudit/390* product is shipped with easy to use and understand fixes to assist in correcting the errors.

The customer has the ability to create, save and submit custom fixes too. All errors, discrepancies, fixes, and output are easily accessible and viewed through the product ISPF interface panel for future reference.

For more information about *FastAudit/390*, visit Mainstar's Web site at:

<http://www.mainstar.com>

7.11.5 Using DFSMSrmm with BTLS

You can also use DFSMSrmm to manage volumes in any automated tape library that has special software including an IBM Tape Library dataserver that is managed using Basic Tape Library Support (BTLS).

Both system-managed tape or BTLS support the use of IBM 3494 and 3495 Automated Tape Library Dataservers. DFSMSrmm is integrated with system-managed tape but has no direct interface with BTLS.

You can use DFSMSrmm to work with BTLS to:

- ▶ Select volumes from scratch pools
- ▶ Return volumes to scratch status

Selecting scratch pool

BTLS provides a way of selecting from any one of its eight scratch pools using job names. If you already use the data set name to select a scratch pool, you can use DFSMSrmm to select the scratch pool, and DFSMSrmm tells BTLS which pool is to be used. Use the DFSMSrmm VLPOOL command in PARMLIB to associate an DFSMSrmm scratch pool with the name of a BTLS scratch pool. For example, you can have the following scratch pool definitions, as shown in Example 7-25.

Example 7-25 *Scratch pool definitions*

```
VLPOOL PREFIX(10*) TYPE(S) MEDIANAME(CARTS) NAME(SCRTCH1) -  
    DESCRIPTION('BTLS pool 1') EXPDTCHECK(N)  
VLPOOL PREFIX(2*) TYPE(S) MEDIANAME(3590) NAME(SCRTCH7) -  
    DESCRIPTION('BTLS pool 7') EXPDTCHECK(N)  
VLPOOL PREFIX(*) TYPE(S) MEDIANAME(TAPE) -  
    DESCRIPTION('default pool') EXPDTCHECK(N)
```

To use the data set name to select the scratch pool, you must implement the EDGUX100 installation exit. The *DFSMSrmm Installation and Customization Guide*, SC26-7405, provides details about how to customize, install, and use this exit.

When BTLS issues a mount to the library for a nonspecific volume, DFSMSrmm uses the IGXMSGEX installation exit to return the correct BTLS scratch pool name to BTLS, and BTLS uses that name instead of any pool it may have selected.

Returning volumes to scratch status

Although BTLS tracks the status of volumes, changing them to private status when they are used for a nonspecific mount, it provides no method of deciding when to return the volume to scratch. You use your tape management system to do this. DFSMSrmm provides simple TSO commands that you can use to drive the BTLS LIBRARY command to change a volume back to scratch status. When you have run DFSMSrmm inventory management, you can generate lists of volumes in scratch status and use them as input to the BTLS LIBRARY command to update BTLS volume status. See Example 7-26 for details.

Example 7-26 *BTLS status synchronization*

```
//SYNCBTLS PROC
//TMP      EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
/*
//AMS      EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//LIBIN    DD  DISP=SHR,DSN=userid.EXEC.RMM.CLIST
//          PEND
//S1       EXEC SYNCHBTLS
//TMP.SYSTSIN DD *
RMM SEARCHVOLUME VOLUME(10*) OWNER(*) LIMIT(*) STATUS(SCRATCH) -
NOLIST CLIST
/*
//AMS.SYSIN DD *
LIBRARY SETCAT UNIT(123) CATEGORY(SCRTCH1)
/*
//S7       EXEC SYNCHBTLS
//TMP.SYSTSIN DD *
RMM SEARCHVOLUME VOLUME(2*) OWNER(*) LIMIT(*) STATUS(SCRATCH) -
NOLIST CLIST
/*
//AMS.SYSIN DD *
LIBRARY SETCAT UNIT(123) CATEGORY(SCRTCH7)
/*
//SM EXEC SYNCHBTLS
//TMP.SYSTSIN DD *
RMM SEARCHVOLUME VOLUME(*) OWNER(*) LIMIT(*) STATUS(MASTER) -
LOCATION(SHELF) INTRANSIT(N) NOLIST CLIST
/*
//AMS.SYSIN DD *
LIBRARY SETCAT UNIT(123) CATEGORY(PRIVATE)
/*
//EJECT EXEC SYNCHBTLS
//TMP.SYSTSIN DD *
RMM SEARCHVOLUME VOLUME(*) OWNER(*) LIMIT(*) DEST(VAULT1) -
LOCATION(SHELF) INTRANSIT(Y) NOLIST CLIST
/*
//AMS.SYSIN DD *
LIBRARY SETCAT UNIT(123) CATEGORY(XEJECTB)
/*
```

By customizing the RMM TSO command, you can select volumes from a single scratch pool if necessary so that BTLS only returns a small number of volumes to scratch status. Similarly, you can use DFSMSrmm commands to build eject lists.

7.11.6 Using DFSMSrmm with non-IBM libraries

Whatever tape library is installed in your system, DFSMSrmm can support it through a user interface that tells the library which volumes to return to scratch status and which volumes must be ejected and moved to another storage location.

With DFSMSrmm's reporting and command capabilities it is simple to build this interface, using REXX, DFSORT, DFSMSrmm utilities, and RMM TSO subcommands.

You must analyze what the vendor tape library requires as input for retention and movement processing and then customize DFSMSrmm to provide the library with the correct input.

Be sure to keep the vendor tape library catalog and the DFSMSrmm database synchronized to avoid any mismatch that would cause rejection of tape mounts.

STK tape libraries

This section provides a simple interface example between an STK Silo tape library and DFSMSrmm to eject tapes that need to be moved to another location. After running DFSMSrmm inventory management (storage location management processing DSTORE), you can create a list of all volumes that are moving by using DFSMSrmm commands. Follow these steps:

1. Use the DFSMSrmm commands as shown in Figure 7-59.

```
RMM SV VOLUME(*) DESTINATION( LOCAL) CLIST(' ' -')
RMM SV VOLUME(*) DESTINATION( REMOTE) CLIST(' ' -')
RMM SV VOLUME(*) DESTINATION( DISTANT) CLIST(' ' -')
RMM SV VOLUME(*) DESTINATION( my_loc) CLIST(' ' -')
```

Figure 7-59 Using RMM SEARCHVOLUME subcommands

Note: Unless you exploit the capability of APAR OW30790 and use a pre-allocated RMMCLIST DD name, you can use only one RMM TSO subcommand at a time. This is because each call to the CLIST option overwrites the data that was created previously.

The resulting list, which is stored in the RMMCLIST data set, or the data set prefix.EXEC.RMM.CLIST, consists of VOLSERs only. The data set uses blocked, variable-length records, with a logical record length of 255 if you have not specified any other DCB information. The prefix is the TSO user PROFILE PREFIX that the user has currently defined. If you have specified NOPREFIX, DFSMSrmm allocates a data set using the RACF user ID or the jobname, to avoid using EXEC.RMM.CLIST.

If you are using the RMM TSO subcommands in a batch job, you can set the prefix in front of the RMM TSO subcommand.

2. Convert the list from variable-length records to fixed-length records. HSC/MVS control statement syntax conventions: the control statement for each utility program consists of a command (indicating the utility function) followed by parameters, as applicable, in 80-character card-image records (see the *HSC/MVS Programmer's Guide, Volume 2* provided with the product).
3. Edit this list and adding an EJECT statement in front and close the command. Optionally you can specify a CAP statement as shown in Figure 7-60.


```

EJECT VOLSER( -
    volser1 -
    volser2 -
    volser3 -
    -)
CAP( cap-list )

```

Figure 7-60 Adding the EJECT command to the volume list

Each single EJECT command causes an operator interaction.

4. Run the HSC update utility, SLUADMIN (see the *HSC/MVS System Programmer's Guide, Volume 1* provided with the product). The HSC Initialize Cartridge Utility (SLUADMIN) in Example 7-27 shows a sample JCL that you can use to initialize cartridges with the HSC SLUADMIN utility, by using DFSMSrmm EDGINERS processing.

Example 7-27 JCL for initializing cartridges with EDGINERS

```

//YCJRES1 JOB ....
//TINIT EXEC PGM=SLUADMIN,TIME=1440,PARM=MIXED
//TAPE DD UNIT=(580,1,DEFER),DISP=NEW
/* EDGINERS TAPE UNIT DD
//SLSTAPE DD UNIT=AFF=TAPE INIT CARTRIDGES INIT TAPE
/* UNIT DD.
//SYSIN DD UNIT=VIO,SPACE=(TRK,1) EDGINERS CONTROL DATASET
//SLSPRINT DD SYSOUT=* UTILITY MESSAGES
//SYSPRINT DD SYSOUT=* EDGINERS MESSAGES
//SLSINIT DD *
    INIT LABEL(SL) VOL(*****)
/*
//SLSIN DD *
* WHEN USING THE DFSMSRMM TAPE INITIALIZATION PROGRAM EDGINERS, YOU
* MUST SPECIFY THE 'PROGRAM' PARAMETER ON THE INIT CARTRIDGES
* CONTROL STATEMENT.
INITIALIZE CAP(000) PROGRAM(EDGINERS) OPTION(SCRATCH) CNTLDD(SYSIN)
/*

```

The HSC scratch conversion SLUCONDB program is provided to generate scratch transactions from DFSMSrmm tape management system. SLUCONDB calls the routine SLUDRRMM to read the DFSMSrmm report extract file if RMM is specified in the EXEC JCL statement. This program uses the DFSMSrmm EDGRVEXT macro to map the layout of the extract file. The following list shows the fields that are used:

- ▶ RVTYPE
- ▶ RVVOLSER
- ▶ RVLCDATE
- ▶ RVNAME
- ▶ RVSTATUS
- ▶ RVEXPDT
- ▶ RVEXPDTO
- ▶ RVLABEL

SLUDRRMM processes the extract and passes the volume record information to SLUCONDB, which builds scratch card images to input to SLUADMIN. Volume status scratch (scratch or non scratch) is then updated in the HSC CDS for each volume record in the extract file. Dates on the DFSMSrmm report must be in Julian date format (EDGHSKP run with

PARM ' DATEFORM(J)'). Tapes listed in the DFSMSrmm report without expiration dates are skipped by SLUDRRMM. Example 7-28 shows the DFSMSrmm and HSC synchronization JCL.

Example 7-28 DFSMSrmm and HSC synchronization JCL

```
//JOBSCUP JOB (ACCOUNT),PROGRAMMER
//S1      EXEC PGM=EDGHSKP,PARM='RPTEXT,DATEFORM(J)'
//MESSAGE DD DSN=RMM.MESSAGE.DATASET,DISP=SHR
//RPTEXT  DD DSN=RMM.REPORT.EXTRACT.HSC.ONLY,DISP=SHR
//SYSIN   DD *
          RPTEXT RECORDS(V)
/*
//S2      EXEC PGM=SLUCONDB,
//          PARM=('RMM,SCRPOOL(SL)',,MIXED')
//SLSTMS  DD DSN=RMM.REPORT.EXTRACTHSC.ONLY,DISP=SHR
//SLSSOUT DD DSN=&&SCUPINPT,DISP=(NEW,PASS),UNIT=,SPACE=DCB=
//SLSPRINT DD SYSOUT=*
//S3      EXEC PGM=SLUADMIN
//SLSIN   DD DSN=&&SCUPINPT,DISP=(OLD,DELETE)
//SLSPRINT DD SYSOUT=*
//
```

The SLUCONDB utility, when PARM=RMM is specified, checks in the DFSMSrmm extract file to see if the MEDIANAME of the tapes being returned to scratch is either 3480, 3490 or 3590. For example, if you changed the default media name 3480, and you are using CARTS, you never get your tapes selected as scratch by the utility.

Note: To save space and to speedup the SLUCONDB process it is enough if you are creating an extract file with only V records.

Update the MEDIANAME used in the STK SLUADMIN program by updating the SLUDRRMM source module. After that, compile and link edit it.

Example 7-29 shows you the original program code.

Example 7-29 Original SLUDRRMM macro

```
.....
READRCD EQU *
          LA R2,BUFFER2+4          LOAD ADDRESS OF INPUT BUFFER
          GET INPUT,BUFFER2        READ RECORD
          USING RVEXT,R2           ESTABLISH ADRESSABILITY TO RCD.
          CLI RVTYPE,C'V'          WAS IT A VOLUME RECORD?
          BNE READRCD             NO - READ NEXT RECORD.
          CLC RVNAME,=CL8'3480'    WAS RECORD FOR A 3480 VOL?
          BE READCONT             YES - CHECK STATUS.
          CLC RVNAME,=CL8'3490'    WAS RECORD FOR A 3490 VOL?
          BNE READRCD             NO - READ NEXT RECORD.
          CLC RVNAME,=CL8'3590'    WAS RECORD FOR A 3490 VOL?
          BNE READRCD             NO - READ NEXT RECORD.
          READCONT CLC RVSTATUS,=CL8'STATUS' WAS RECORD IN SCRATCH STATUS?
          BNE READRCD             NO - READ NEXT RECORD.
```

And the modified program code is shown in Example 7-30.

Example 7-30 Modified SLUDRRMM macro

```

.....
READRCD EQU *
        LA    R2,BUFFER2+4           LOAD ADRESS OF INPUT BUFFER
        GET   INPUT,BUFFER2          READ RECORD
        USING RVEXT,R2               ESTABLISH ADRESSABILITY TO RCD.
        CLI   RVTYPE,C'V'            WAS IT A VOLUME RECORD?
        BNE   READRCD                NO - READ NEXT RECORD.
        CLC   RVNAME,=CL8'3480'      WAS RECORD FOR A 3480 VOL?
        BE    READCONT               YES - CHECK STATUS.
        CLC   RVNAME,=CL8'CARTS'     WAS RECORD FOR CARTRIDGES?
        BE    READCONT               YES - CHECK STATUS.
        CLC   RVNAME,=CL8'3490'      WAS RECORD FOR A 3490 VOL?
        BE    READCONT               YES - CHECK STATUS.
        CLC   RVNAME,=CL8'3590'      WAS RECORD FOR A 3590 VOL?
        BNE   READRCD                NO - READ NEXT RECORD.
READCONT CLC   RVSTATUS,=CL8'STATUS' WAS RECORD IN SCRATCH STATUS?
        BNE   READRCD                NO - READ NEXT RECORD.

```

Note: You should define in the routine all the MEDIANAMES that you have specified in the VLPOOL command. In this example, you have used the MEDIANAME 3480, 3490, 3590, or CARTS in different VLPOOL commands.

STK provides a sample insert/delete user exit SLSUX06. This exit enables the HSC to get control when the database is being updated to add or delete a volume. Refer to STK tape libraries documentation for details about how to tailor this exit to work with IBM DFSMSrmm.



System Managed Library partitioning with z/OS V1.10

In this chapter we describe the DFSMSrmm enhancements in DFSMS V1R10 that provide improvements in the areas of partitioning your system managed tape libraries.

In this chapter we discuss the following enhancements:

- ▶ “System Managed Library partitioning”
- ▶ “Reasons to partitioning a SMS managed library”
- ▶ “Partitioning Support”
- ▶ “Using PRTITION and OPENRULE commands”
- ▶ “Conversion from REJECT to PRTITION and OPENRULE”

By the end of this chapter, you will understand how you can partitioning your IBM system managed tape libraries using the PRTITION and OPENRULE commands and how you can convert your REJECT commands.

8.1 System Managed Library partitioning

In z/OS 1.10 new parmlib controls are provided to enable simplified and more powerful partitioning of system managed libraries and controlling of application use of tape volumes. Now You can use PRTITION and OPENRULE commands in the EDGRMM parmlib member to control library partitioning and the use of volumes by applications.

Attention: If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands.

You can partition a system-managed library including a VTS by performing these tasks:

- ▶ Specify the USE operand value on the RMM ADDVOLUME or RMM CHANGEVOLUME subcommands. You can set this value to MVS or VM or both. If you do not specify MVS for a volume, DFSMSrmm prevents the volume from being defined in the volume catalog on this system.
- ▶ Through the use of the PRTITION and OPENRULE parmlib commands, you can simplify the maintenance of the parmlib members as your libraries and volume ranges change. Operands on the OPENRULE and PRTITION commands allow global actions to be set. You can use one or more specific overrides based on volume sets that have different requirements. Typically, you could add a new range of volumes for use by a single partition and only that one system would need to be updated. The OPENRULE and PRTITION commands allow you to define whether they apply to volumes defined to DFSMSrmm or not. You can use operands on the OPENRULE command to automatically ignore volumes, rather than using EXPDT=98000, ACCODE=xCANORES or a customized EDGUX100.
- ▶ Define parmlib member EDGRMMxx REJECT prefixes. You can use REJECT to prevent a volume not defined to DFSMSrmm from being defined in a system-managed tape library. The REJECT ANYUSE(prefix) operand prevents a volume from being defined in the system-managed tape library on the current system. The REJECT OUTPUT(prefix) operand allows you to define the volume to the system-managed tape library but only use the volume for input processing.

In this section we will only describe the use of the new PRTITION and OPENRULE parmlib commands.

Important: You can use operands on the OPENRULE command to automatically ignore volumes rather than using EXPDT=98000, ACCODE=xCANORES or a customized EDGUX100.

8.2 The sequence of processing

The processing path taken depends on whether PRTITION or OPENRULE or REJECT parmlib commands are used. If REJECT is used but neither PRTITION nor OPENRULE are used, processing is unchanged. If you have specified PRTITION and OPENRULE parmlib operands the sequence of processing is:

1. IGNORE processing determines if this request will be ignored or processed by DFSMSrmm
2. Partitioning checking using PRTITION
3. REJECT processing based on OPENRULE

8.3 Reasons to partitioning a SMS managed library

An IBM Automated Tape Library can be shared by different systems in two ways:

- ▶ by logically dividing it into different partitions (**partitioning**)
- ▶ by allowing all attached systems to sequentially access the tape volumes in the library (**sharing**).

In this section we are explain the concepts to partitioning only.

8.3.1 Partitioned library with shared CDS and TCDB

The first example in Figure 8-1 we show you a system managed tape library partitioned by:

- ▶ using CBRUXENT user exits on each system
- ▶ using EDGUX200 user exit on each system
- ▶ different scratch categories for each system in the library manager data base

The EDGHSKP expiration processing is running by system, but all volume in PRIVATE category are shared between the two systems.

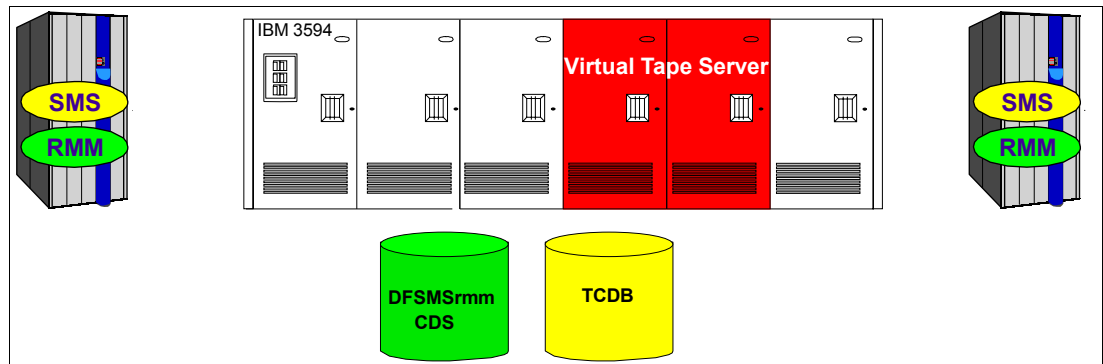


Figure 8-1 Partitioned library with shared CDS and TCDB

8.3.2 Partitioned library with shared TCDB and different CDSs

This example in Figure 8-2 on page 294 we show you a system managed tape library partitioned by:

- ▶ different DFSMSrmm control data set for each system
- ▶ use of REJECT ANYUSE in EDGRMMnn parmlib member
- ▶ different scratch categories for each system in the library manager data base

The EDGHSKP expiration processing is running by CDS, but all volume in PRIVATE category are shared between the two systems. To access this volumes you have to specify one of the two JCL parameters EXPDT=98000 or ACCODE=XCANORES. For both parameters the default shipped EDGUX100 user exit is needed.

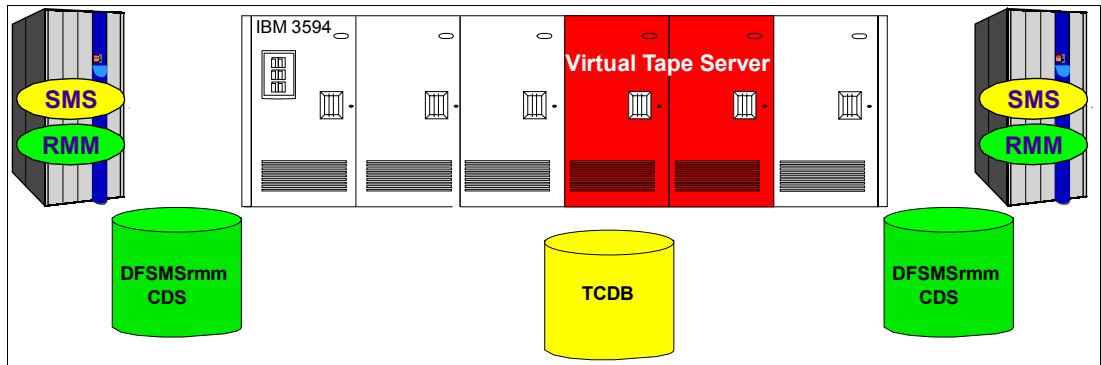


Figure 8-2 Partitioned library with shared TCDB and different CDSs

8.3.3 Partitioned library with unshared CDS and TCDB

In the third example shown in Figure 8-3 you can see a system managed tape library partitioned by:

- ▶ system
- ▶ use of REJECT ANYUSE in EDGRMMnn parmlib member
- ▶ different scratch categories for each system in the library manager data base

The EDGHSKP expiration processing is running by system. All volumes are not shared so you have to define a volume in private status in the TCDB first. To access this volumes you have to specify one of the two JCL parameters EXPDT=98000 or ACCODE=XCANORES. For both parameters the default shipped EDGUX100 user exit is needed.

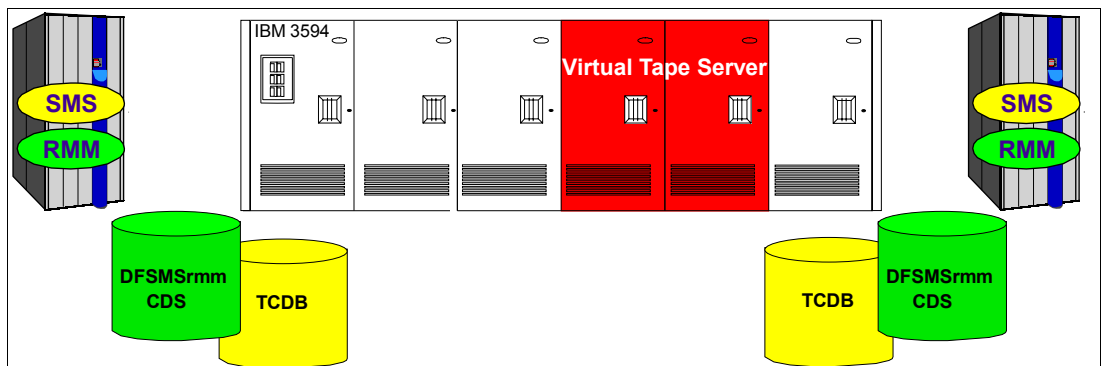


Figure 8-3 Partitioned library with unshared CDS and TCDB

All volumes are not shared so you have to define a volume in private status in the TCDB first before you can use it. Figure 8-4 on page 295 shows you how you can add a volume in the TCDB using AMS commands. To access this volumes you have to specify one of the two JCL parameters EXPDT=98000 or ACCODE=XCANORES. For both parameters the default shipped EDGUX100 user exit is needed.


```
//STEP0000 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
  CREATE VOLUMEENTRY (NAME(VLZTB82) -
    LIBRARYNAME(ATV3494J) -
    LOCATION(LIBRARY) -
    USEATTRIBUTE(PRIVATE) -
    STORAGEEGROUP(ATV3494J) -
    MEDIATYPE(MEDIA2)
/*
```

Figure 8-4 Add and delete a volume entry to the TCDB

After you have successfully processed the volume you have previously defined in the TCDB you should delete the entry from the TCDB using the command as shown in Figure 8-5.

```
//STEP0000 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
  DELETE ('VLZTB82') VOLUMEENTRY PURGE
/*
```

Figure 8-5 Delete a volume entry from the TCDB

8.3.4 Partitioned library with shared CDS and unshared TCDB

The last example in Figure 8-6 we show you a system managed library partitioned by:

- ▶ system
- ▶ using customized version of CBRUXENT user exits on each system
- ▶ different scratch categories for each system in the library manager data base

The EDGHSKP expiration processing is running by system based on the TCDB. All volumes are not shared so you have to define a volume in private status in the TCDB first before you can use it. Figure 8-4 shows you how you can add a volume in the TCDB using AMS commands. After you have successfully processed the volume you have previously defined in the TCDB you should delete the entry from the TCDB using the command as shown in Figure 8-5.

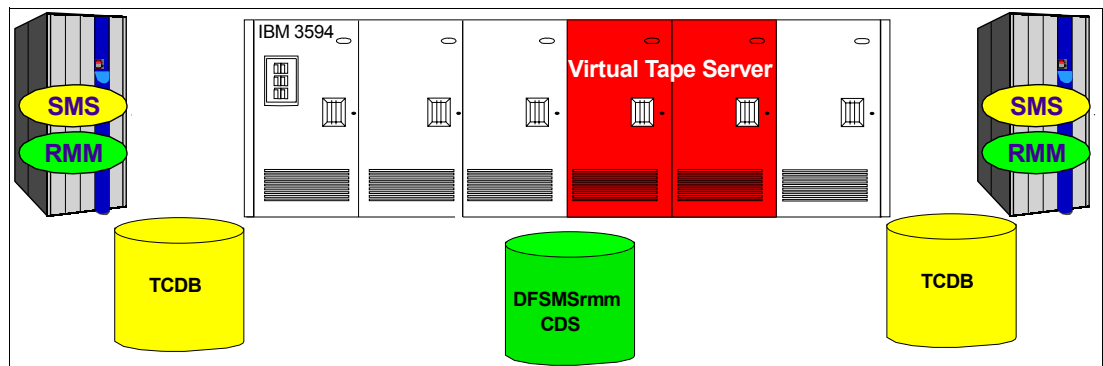


Figure 8-6 Partitioned library with shared CDS and unshared TCDB

8.4 Partitioning Support

Each system in the RMMplex uses its own parmlib options to determine the processing required. For releases prior to z/OS V1R10 the REJECT commands are used for both partitioning and for Open/Close/EOV volume use decisions. On z/OS V1R10 and later releases the REJECT commands are only used if specified and neither PRTITION nor OPENRULE commands are defined. When you define REJECT commands and PRTITION or OPENRULE commands the REJECT commands cause an DFSMSrmm start-up error and you must reply to EDG0239E to continue the start-up or restart DFSMSrmm with a corrected parmlib. If there are no REJECT commands the PRTITION and OPENRULE command defaults are used.

EDG0239E

REJECT COMMAND FOUND IN PARMLIB AND NO LONGER SUPPORTED

- Explanation:** This message is issued during initialization when a REJECT command has been found, but one or more PRTITION and OPENRULE commands are also specified. You must use either REJECT commands or OPENRULE and PRTITION commands.
- System action:** DFSMSrmm initialization stops. This message is followed by message “EDG0215D” on page 42.
- Operator response:** Notify the system programmer. Reply to message EDG0215D as directed. Restart DFSMSrmm when the system programmer has corrected the error.
- System programmer response:**
Do not attempt to use both REJECT commands and PRTITION or OPENRULE commands in parmlib. If you are implementing PRTITION and OPENRULE commands, you must remove the REJECT commands from parmlib. **You can recommend the operator reply ‘Y’ to message EDG0215D to continue and ignore the REJECT commands.**
- Source:** DFSMSrmm
- Detecting Module:** EDGPARM
- Routing Code:** 3
- Descriptor Code:** 3

8.5 Using PRTITION and OPENRULE commands

Using the PRTITION commands, you can control partitioning entry/insert, export/import, eject, and CUA processing for system managed volumes:

- DFSMSrmm partitioning is based on a global setting that you can change. The default is that all system-managed and non-system managed volumes are accepted. This can be represented by the command shown in Figure 8-7.

PRTITION VOLUME(*) TYPE(ALL) SMT(ACCEPT) NOSMT(ACCEPT LOCATION(SHELF))
--

Figure 8-7 Partitioning based on global setting

You can change the global setting so that all volumes are ignored by using the command shown Figure 8-8 on page 297.

```
PRRTITION VOLUME(*) TYPE(ALL) SMT(IGNORE) NOSMT(IGNORE)
```

Figure 8-8 Partitioning based on global setting ignoring all volumes

- Using a global command like the example shown in Figure 8-9

```
PRRTITION VOLUME(*) TYPE(NORMM) SMT(IGNORE) NOSMT(IGNORE)
```

Figure 8-9 Global rejecting all undefined volumes

This global command will to the following:

- All system-managed volumes undefined in the DFSMSrmm CDS are left in insert category to be accepted by another system and non-system-managed volumes are not added automatically to the CDS.

Note: This command will replace the REJECT ANYUSE(*) definition

- The remainder of the default command [PRRTITION VOLUME(*) TYPE(RMM) SMT(ACCEPT) NOSMT(ACCEPT)] that is not overridden by your global commands in parmlib is used to handle TYPE(RMM) volumes.
- With this approach, you must predefine system-managed volumes to DFSMSrmm to enable ownership of volumes during entry/insert processing.
- No PRRTITION commands - When REJECT commands are used, processing is as for earlier releases unless any OPENRULE statements are defined. In the latter case and when no REJECT commands are defined, the defaults are used for partitioning.
- Using selective PRRTITION statements on top of a global command, you can be very specific about which volumes are to become owned by the current system/partition.
- All ACCEPTed volumes that are not defined to DFSMSrmm are added automatically to the DFSMSrmm CDS and owned by this system/partition, unless it is eject processing.
 - For system-managed volumes, 'Owned by this system/partition' means that the volume is defined to DFSMSrmm, has an entry in the TCDB, and the scratch category used by the Library Manager (LM) is set based on your values in this systems DEVSUPxx.
 - The ISMF library default entry status is used for added volumes.
- Pre-defined volumes are only considered when the PRRTITION command specifies TYPE(RMM) or TYPE(ALL). The volume status for the TCDB entry is set by DFSMSrmm from the volume information.

Note: As a result of this flexibility, you should no longer need to customize the CBRUXENT exit.

8.5.1 Defining PRRTITION commands

Example 8-1 shows you the correct use of the EDGRMMnn parmlib member PRRTITION command.

Example 8-1 EDGRMMnn PRRTITION command

ACCEPT/IGNORE Use this operand to identify the action to be taken by DFSMSrmm. The action applies to library entry/insert/import, eject/export, and CUA processing, OPEN processing, and to EXPROC processing. During OPEN processing, once the decision is taken about IGNOREing the volume, the PRTITION checking occurs first, and the action taken before the OPENRULE REJECT/ACCEPT processing occurs. As a result, the action taken by PRTITION can influence the TYPE processing by OPENRULE.

ACCEPT DFSMSrmm processes the volume. During entry/insert import/export, CUA, and OPEN processing, the volume is added to the DFSMSrmm CDS if not yet defined. For NOSMT volumes, the location from the LOCATION operand is used to set the volume current and home locations. During EXPROC processing, the volume is processed for return to scratch. Note that a volume identified as TYPE(NORMM) with an action of ACCEPT, that is added to the CDS, now becomes a TYPE(RMM) volume for subsequent processing including OPENRULE processing. By default, all volumes are processed during EXPROC, and all volumes are accepted during library entry/insert/import or OPEN processing. During OPEN processing and during library entry/insert/import processing, any undefined volumes are automatically added to the DFSMSrmm CDS using the library default volume entry status. During OPEN processing, volumes are added to the CDS only if the current OPMODE is either WARN or PROTECT.

IGNORE DFSMSrmm does not process the volume. During entry/insert/import processing, DFSMSrmm requests that OAM ignore the volume and leave it in the 'insert' category. During CUA and eject/export processing, the request for the volume is ignored. During EXPROC processing, the return to scratch processing is skipped. During OPEN processing:

- For NORMM volumes, the volume is not added to the CDS
- For RMM volumes, processing depends on the OPENRULE entries.

ACCEPT is the default value.

LOCATION(SHELF | LocdefHome)

Use this operand for NOSMT volumes that are of TYPE(NORMM) with action ACCEPT. DFSMSrmm's add volume processing uses the defined location value as the volume's home location and current location.

You can specify either SHELF or the name of a LOCDEF defined location that has a specified TYPE(STORAGE,HOME). DFSMSrmm validates this value against the LOCDEF commands.

Although the operand is only used for NORMM volumes, DFSMSrmm does not prevent you from specifying it on a PRTITION command for volumes of TYPE RMM or ALL.

The default value is SHELF.

TYPE(ALL | RMM | NORMM)

Use to identify the type of volumes selected by the PRTITION command. The volser is used to determine whether the volume is defined in the RMM CDS, and, depending on the function, the VOL1

label and HDR1 can be used. Therefore, DFSMSrmm cannot always determine if the volume is an existing one defined to DFSMSrmm. TYPE is determined as follows:

- ▶ For system-managed library functions such as import/export, insert/entry, CUA, and eject, only the volser is used to determine whether the volume is defined in the RMM CDS.
- ▶ For OPEN processing, the same rules as volume ignore processing are used and the TYPE that is assigned is based on the following:
 - TYPE(RMM)
 1. The volume serial number is defined in the control data set, and there is no HDR1 tape label for the volume.
 2. The volume serial number is defined in the control data set, and no labels are used (NL, NSL, or BLP with an NL tape).
 3. The 17 characters read from the HDR1 label of the mounted volume match the last 17 characters of the data set name in the control data set.–
 - TYPE(NORMM)
 1. The volume serial number is not defined in the control data set.
 2. The volume is defined, but the 17 characters read from the HDR1 label of the mounted volume do not match the last 17 characters of the data set name in the control data set.
- ▶ For EXPROC processing, TYPE(RMM) always applies because EXPROC is driven only by volumes defined to DFSMSrmm.

For each set of volumes, you can code one command with RMM and another with NORMM, but only a single command if ALL is used.

RMM	The volume must already be defined to DFSMSrmm.
NORMM	The volume is not defined to DFSMSrmm.
ALL	Applies to all volumes regardless of whether they are defined to DFSMSrmm. ALL is the default value.

VOLUME/VOLUMERANGE

Use these operands to select volumes that are to be managed by this command. You must specify either VOLUME or VOLUMERANGE, and each command defines a set of one or more volumes. Sets cannot overlap within a TYPE, but a set can be a subset of another and such a subset is more specific. When you code a set with TYPE(RMM) and repeat that set with TYPE(NORMM), you can specify different operands for each set. If you want to use the same operands for the sets, you can do so by coding a type of ALL.

VOLUME	You can specify the volume as fully qualified or a volser prefix ending in *. A fully qualified volume is one to six alphanumeric, national or special characters. A volser prefix is zero to five alphanumeric, national, or special characters ending in an asterisk. Quotes are required for special characters, and the first character must not be blank. Any value ending in *, even if enclosed in quotes, is considered to be a volser prefix.
---------------	--

VOLUMERANGE Use to select a subset of volumes based on starting and ending volser. One to six characters for beginning and end of range are required to be processed, and they can be alphanumeric, national, and special characters. Quotes are required for each value regardless of the use of special characters, and the first character must not be blank. The end of range must not be lower than the start of the range.

We take the PRTITION definitions as shown in Figure 8-11 in our EDGRMMnn parmlib member .

PRTITION	VOLUME(*)	/* added 2008/03/20 NS	*/-
	TYPE(NORMM)	/* undefined volumes in CDS only	*/-
	SMT(IGNORE)	/* ignore sms managed tapes	*/-
	NOSMT(IGNORE)	/* ignore no-sms managed tapes	*/
PRTITION	VOLUMERANGE('THM000': 'THM029')	/* added 2008/03/20 NS	*/-
	TYPE(ALL)	/* defined and undefined volumes	*/-
	SMT(IGNORE)	/* accept sms managed tapes	*/-
	NOSMT(IGNORE)	/* ignore non-sms managed tapes	*/
PRTITION	VOLUME(DB*)	/* added 2008/03/20 NS	*/-
	TYPE(ALL)	/* defined and undefined volumes	*/-
	SMT(IGNORE)	/* ignore sms managed tapes	*/-
	NOSMT(ACCEPT)	/* accept non-sms managed tapes	*/
PRTITION	VOLUME(HE00*)	/* added 2008/03/20 NS	*/-
	TYPE(ALL)	/* defined and undefined volumes	*/-
	SMT(IGNORE)	/* ignore sms managed tapes	*/-
	NOSMT(ACCEPT)	/* accept non-sms managed tapes	*/
PRTITION	VOLUME(J5T*)	/* added 2008/03/20 NS	*/-
	TYPE(ALL)	/* defined and undefined volumes	*/-
	SMT(IGNORE)	/* ignore sms managed tapes	*/-
	NOSMT(ACCEPT)	/* accept non-sms managed tapes	*/
PRTITION	VOLUME(MXX*)	/* added 2008/03/20 NS	*/-
	TYPE(ALL)	/* defined and undefined volumes	*/-
	SMT(IGNORE)	/* ignore sms managed tapes	*/-
	NOSMT(ACCEPT)	/* accept non-sms managed tapes	*/
PRTITION	VOLUME(S0024*)	/* added 2008/03/20 NS	*/-
	TYPE(ALL)	/* defined and undefined volumes	*/-
	SMT(IGNORE)	/* ignore sms managed tapes	*/-
	NOSMT(ACCEPT)	/* accept non-sms managed tapes	*/
PRTITION	VOLUME(S020*)	/* added 2008/03/20 NS	*/-
	TYPE(ALL)	/* defined and undefined volumes	*/-
	SMT(IGNORE)	/* ignore sms managed tapes	*/-
	NOSMT(ACCEPT)	/* accept non-sms managed tapes	*/

Figure 8-11 EDGRMMnn PRTITION definitions

To list the current PRTITION settings you can use the RMM TSO subcommand LISTCONTROL with the new option PRTITION as shon in Figure 8-12.

RMM LISTCONTROL PRTITION
or
RMM LC P

Figure 8-12 LISTCONTROL with option PRTITION

Figure 8-13 shows you the result of the RMM TSO subcommand LISTCONTROL. We used the option PRTITION to get only this information. Each definition is listet twice, one time to schow the type RMM settings and the second time time for the NORMM settings.

Partition Entries:					
Volume or Range	Type	SMT Action	NOSMT Action	Location	
DB*	RMM	IGNORE	ACCEPT	SHELF	
HE00*	RMM	IGNORE	ACCEPT	SHELF	
J5T*	RMM	IGNORE	ACCEPT	SHELF	
MXX*	RMM	IGNORE	ACCEPT	SHELF	
S0024*	RMM	IGNORE	ACCEPT	SHELF	
S020*	RMM	IGNORE	ACCEPT	SHELF	
THM000:THM029	RMM	IGNORE	IGNORE		
*	RMM	ACCEPT	ACCEPT	SHELF	
DB*	NORMM	IGNORE	ACCEPT	SHELF	
HE00*	NORMM	IGNORE	ACCEPT	SHELF	
J5T*	NORMM	IGNORE	ACCEPT	SHELF	
MXX*	NORMM	IGNORE	ACCEPT	SHELF	
S0024*	NORMM	IGNORE	ACCEPT	SHELF	
S020*	NORMM	IGNORE	ACCEPT	SHELF	
THM000:THM029	NORMM	IGNORE	IGNORE		
*	NORMM	IGNORE	IGNORE		

Figure 8-13 Result of the LISTCONTROL PRTITION

Be sure that you have not secified any definition twice, because in this case the DFSMSrmm inialization process is stopped and message EDG0238E following by message EDG0215D is written as shown in Figure 8-14.

EDG0204I DFSMSrmm BEING INITIALIZED FROM MEMBER EDGRMM71 IN RMM.PARMLIB
EDG0238E OVERLAPPING VOLUME SET DEFINED FOR PRTITION TYPE(ALL). THM* OVERLAPS THM*
EDG0215D ERRORS DETECTED IN INITIALIZATION PARAMETERS -
ENTER ~Y~ TO CONTINUE OR ~N~ TO CANCEL

Figure 8-14 Error message until DFSMSrmm inialization

Where:

EDG0238E OVERLAPPING VOLUME SET DEFINED FOR command TYPE(type). value1 OVERLAPS value2

Explanation: A PRTITION or OPENRULE command has specified volume selection operands that conflict with another command.

In the message text:

command Is one of the following: PRTITION, or OPENRULE.

type I s one of the following: ALL, RMM or NORMM.

value1 Is one of the following: a volume serial number, or a volume prefix specified through the VOLUME operand, or a volume range specified with the VOLUMERANGE operand.

System action:	DFSMSRmm initialization stops. This message is followed by message “EDG0215D” on page 42.
Operator response:	Notify the system programmer. Reply to message “EDG0215D” on page 42 as directed. Restart DFSMSRmm when the system programmer has corrected the error.

System programmer response:

Correct the VOLUME and VOLUMERANGE operands so that overlaps do not exist. Volume sets must be unique within command and type; for example, PRTITION TYPE(RMM) with VOLUME(A*) and PRTITION TYPE(RMM) with VOLUMERANGE(A:B99999) is an overlap because the volume range A:B99999 begins within the A* set but extends beyond it. When the message includes TYPE(ALL) the overlap could be with another command specifying a type of ALL or it could be with RMM or NORMM. DFSMSRmm ignores the overlapping values if initialization continues.

Source: DFSMSrmm

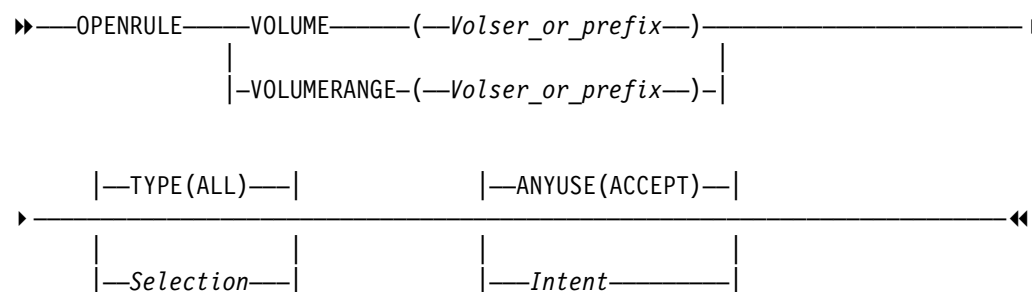
Detecting Module: EDGSUSE

Routing Code: 3

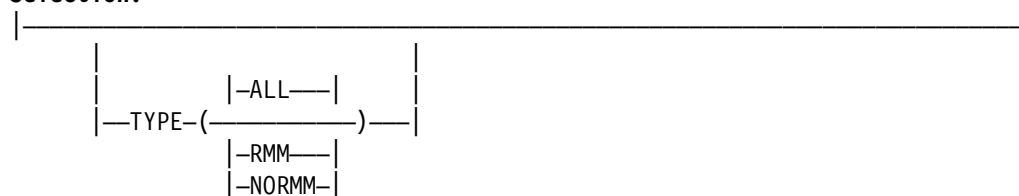
Descriptor Code: 3

Example 8-2 shows you the correct use of the EDGRMMnn parmlib member OPENRULE command.

Example 8-2 EDGRMMnn OPENRULE command



Selection:



Intent:

- ▶ For OPEN processing, the same rules as volume ignore processing are used and the TYPE that is assigned is based on the following:
 - TYPE(RMM)
 1. The volume serial number is defined in the control data set, and there is no HDR1 tape label for the volume.
 2. The volume serial number is defined in the control data set, and no labels are used (NL, NSL, or BLP with an NL tape).
 3. The 17 characters read from the HDR1 label of the mounted volume match the last 17 characters of the data set name in the control data set.–
 - TYPE(NORMM)
 1. The volume serial number is not defined in the control data set.
 2. The volume is defined, but the 17 characters read from the HDR1 label of the mounted volume do not match the last 17 characters of the data set name in the control data set.
- ▶ For EXPROC processing, TYPE(RMM) always applies because EXPROC is driven only by volumes defined to DFSMSrmm.

For each set of volumes, you can code one command with RMM and another with NORMM, but only a single command if ALL is used.

RMM	The volume must already be defined to DFSMSrmm.
NORMM	The volume is not defined to DFSMSrmm.
ALL	Applies to all volumes regardless of whether they are defined to DFSMSrmm. ALL is the default value.

VOLUME/VOLUMERANGE

Use these operands to select volumes that are to be managed by this command. You must specify either VOLUME or VOLUMERANGE, and each command defines a set of one or more volumes. Sets cannot overlap within a TYPE, but a set can be a subset of another and such a subset is more specific. When you code a set with TYPE(RMM) and repeat that set with TYPE(NORMM), you can specify different operands for each set. If you want to use the same operands for the sets, you can do so by coding a type of ALL.

VOLUME	You can specify the volume as fully qualified or a volser prefix ending in *. A fully qualified volume is one to six alphanumeric, national or special characters. A volser prefix is zero to five alphanumeric, national, or special characters ending in an asterisk. Quotes are required for special characters, and the first character must not be blank. Any value ending in *, even if enclosed in quotes, is considered to be a volser prefix.
VOLUMERANGE	Use to select a subset of volumes based on starting and ending volser. One to six characters for beginning and end of range are required to be processed, and they can be alphanumeric, national, and special characters. Quotes are required for each value regardless of the use of special characters, and the first character must not be blank. The end of range must not be lower than the start of the range.

ANYUSE/INPUT/OUTPUT(action)

Use this operand to identify the type of OPEN request issued by the application. It reflects the applications intent towards the data.

INPUT	means that the application is attempting to read from the tape data set
OUTPUT	means that the application is attempting to write to the tape data set
ANYUSE	means that the application is attempting either to read from or write to the tape volume

ACCEPT/IGNORE/REJECT(by)

Use this operand to identify the action to be taken by DFSMSrmm. The action applies to OPEN processing.

ACCEPT	means that DFSMSrmm processes the volume. This attempt to open a file on the tape volume is also subject to DFSMSrmm open-time volume validation and if allowed, the use of the volume and file is recorded by DFSMSrmm. ACCEPT means that the volume is accepted provided that the validation performed by DFSMSrmm at OPEN time allows the volume to be used.
---------------	---

ACCEPT is the default value.

IGNORE	means that if requestor is authorized DFSMSrmm does not process the volume. When the mounted volume matches the requested volume, the use of the volume is ignored by DFSMSrmm; no validation of the volume is performed and there is no recording of the volume or file. The user must be authorized to ignore the volume. DFSMSrmm processing is as if the EDGUX100 exit requested the volume is ignored. Volume ignore processing is attempted if either IGNORE is specified or the EDGUX100 exit requests ignore. For ignore processing to be successful the user must be authorized to ignore the volume. The ignore processing is just as if you coded EXPDT=98000 or ACCODE=xCANORES in the JCL and were using the sample EDGUX100 exit shipped with DFSMSrmm
---------------	--

The use of action IGNORE enables you to ignore any specific or non-specific volume requests including all those for system-managed volumes.

REJECT	means that DFSMSrmm must prevent OPEN processing from allowing use of the volume. For a specific volume this results in the OPEN request failing. For a non-specific volume, the volume is dismounted by the system and another mount request issued.
---------------	---

BY(SPECIFIC/NONSPECIFIC/ANY)

Use this operand to identify for which requests the IGNORE action applies.

SPECIFIC	means that the IGNORE action only applies, if a specific volume is requested. Otherwise, the ACCEPT action is used. NONSPECIFIC means that the IGNORE action only applies if a non-specific request is being processed; the volser in the mount message is either PRIVAT or SCRTCH. Otherwise the ACCEPT action is used.
-----------------	--

ANY means that the IGNORE action applies to all types of request.

ANY is the default value.

BY(SYSID, CATLG) Use this operand for volumes of TYPE RMM to specify that the REJECT action only applies, if the volume is defined to DFSMSrmm, the request is for a specific volume, and the BY condition is not met. Otherwise, the ACCEPT action is used.

When specified for TYPE(NORMM) the operand is parsed successfully but is then ignored and never used by DFSMSrmm because volumes of TYPE(NORMM) cannot have existing tape data set records and are not defined to DFSMSrmm.

You can specify one or more of the values SYSID and CATLG. Use this operand to specify that the REJECT action applies if either applied by SYSID or by CATLG. Otherwise, the ACCEPT action is used.

There is no default value.

SYSID Use this operand to specify that for non-scratch volumes existing tape data sets the use of a volume is to be rejected if the creating SYSID of the first file does not match the current SYSID. This operand might help you if you have a common scratch pool across multiple systems but once a volume is used you only want the volume used/referenced on the system from which the data was created. When the creating SYSID matches the current SYSID the ACCEPT action is used.

CATLG Use this operand to specify that existing tape data sets must be referenced by their catalog entry. When the data set is referenced via its catalog entry the ACCEPT action is used.

DFSMSrmm checks to see if there is an existing data set record defined to RMM and applies this rule if the data set record exists regardless of the disposition specified for the data set. In cases where only the first file is being recorded by DFSMSrmm a data set record does not exist for file 2 onwards, but DFSMSrmm processing assumes that they exist.

Figure 8-16 on page 308 shows you the OPENRULE definitions we have inserted in our existing EDGRMMnn parmlib member. This rules will allow the normal use of all volumes defined to DFSMSrmm "TYPE(RMM)" and all other volumes not defined in the DFSMSrmm control data set can be used if you have the needed access to the STGADMIN.EDG.IGNORE.TAPE.* resource defined in RACF class FACILITY.

```

/* ***** */
/*  OPENRULE commands are added at 2008/03/21      Norbert  */
/*  ***** */
OPENRULE VOLUME(*)          /*      added 2008/03/21 NS      */-
      TYPE(NORMM)           /* volumes not defined in CDS */-
      ANYUSE(IGNORE BY(ANY)) /* bypass RMM processing      */
                          /* ignore all types of request */

OPENRULE VOLUME(*)          /*      added 2008/03/21 NS      */-
      TYPE(RMM)             /* all volumes defined in CDS  */-
      ANYUSE(ACCEPT)        /* allow OPEN processing       */-

```

Figure 8-16 EDGRMMnn parmlib OPENRULE definitions

Figure 8-17 shows you the correct use and the result of the RMM TSO subcommand LISTCONTROL OPENRULE.

```

RMM LICTCONTROL OPENRULE

Openrule Entries:

```

Volume or Range	Type	Input Action Condition	Output Action Condition
-----	-----	-----	-----
*	RMM	ACCEPT	ACCEPT
*	NORMM	ACCEPT	ACCEPT

Figure 8-17 Use and result of the LISTCONTROL OPENRULE

8.5.3 Testing various OPENRULE settings

First of all we have created a new data set on tape using the JCL as shown in Figure 8-18. In the next test cases we have used this data set for input but using different OPENRULE settings. The data set was created on volume THS013, an IBM 3592 volume inside an IBM automated tape library (ATL).

```

//IKJEFT01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD   DISP=SHR,DSN=MHLRES7.RMM.CNTL(OPENR001)
//SYSUT2 DD   DISP=(,KEEP),DSN=MHLRES7.TEST.OPENRULE.RULES,
//           UNIT=ATL3
//SYSUT2 DD   SYSOUT=*
//SYSIN DD   DUMMY

```

Figure 8-18 Sample JCL used to create a single data set on tape

Our conclusion of the test is, that in an unshared tape environment, it means only a single DFSMSrmm control data set, all tapes are defined in this CDS and all tape drives are shared, that only the both OPENRULEs shown in Figure 8-16 and the both RACF definitions shown in Figure 8-19 on page 309 are required.

```

RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.* -
          UACC(NONE) OWNER(SYS1    )

RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.* -
          UACC(NONE) OWNER(SYS1    )

```

Figure 8-19 Basic STGADMIN.EDG.TAPE.IGNORE definitions

Test case 1

Function: Read an existing data sets on a SMS managed tape volume using IEBGENER. The used volume serial number is THS013. That is normal tape processing. Figure 8-20 shows you the JCL we used.

Result: The job ended without any errors, because the user MHLRES7 have access to the data set and the volume can based on the OPENRULE be used on this system.

OPENRULE: All DFSMSrmm defined and not defined volumes will be accepted on this system.

RACF: The user has access to the data set profile protecting this data set. There are no need that the user have access to the special resources
 STGADMIN.EDG.IGNORE.TAPE.RMM.*
 STGADMIN.EDG.IGNORE.TAPE.RMM.*
 defined in RACF class FACILITY.

```

//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//LISTCONT EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//TESTCASE EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.TEST.OPENRULE.RULES,
//          UNIT=ATL3,VOL=SER=THS013
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY

```

Figure 8-20 Test case 1 sample JCL

Where:

RACFCMDS shows the STGADMIN.EDG.* resources defined in RACF class FACILITY.

LISTCONT shows the OPENRULE definitions using the RMM TSO LISTCONTROL command.

TESTCASE Using the IEBGENER utility to read the data set.

Figure 8-21 on page 310 shows you the result of the RACF SEARCH command to see that the resources STGADMIN.EDG.IGNORE.TAPE.RMM.* nor STGADMIN.EDG.IGNORE.TAPE.RMM.* are not defined at this time.

```

READ
  SR CLASS(FACILITY) MASK(STGADMIN.EDG)
STGADMIN.EDG.LISTCONTROL
STGADMIN.EDG.MASTER
READY

```

Figure 8-21 Result of test case 1 RACF listing

The result of the RMM TSO LISTCONTROL subcommand is shown in Figure 8-22 and you can see that there is only one OPENRULE definition for each of the RMM and NORMM managed volumes and that for both definitions all tapes are accepted.

```

READY
  RMM LC OPE
Openrule Entries:

```

Volume or Range	Type	Input Action Condition	Output Action Condition
*	RMM	ACCEPT	ACCEPT
*	NORMM	ACCEPT	ACCEPT

```

READY

```

Figure 8-22 Result of test case 1 LISTCONTROL

In Figure 8-23 we show you the JES message log of this job and you can see there was no error.

```

IEF403I OPENR002 - STARTED - TIME=14.18.28 - ASID=0042 - SC70
- --TIMINGS (MINS.)--
-JOBNAME  STEPNAME  PROCSTEP   RC   EXCP   CPU   SRB   CLOCK  SERV  PG
-OPENR002          RACFCMDS    00    28    .00    .00    .00    340   0
-OPENR002          LISTCONT    00    32    .00    .00    .00    347   0
IEF233A M 0B22,THS013,,OPENR002,TESTCASE,MHLRES7.TEST.OPENRULE.RULES
IEF234E K 0B22,THS013,PVT,OPENR002,TESTCASE
-OPENR002          TESTCASE    00    74    .00    .00    .31    217   0
IEF404I OPENR002 - ENDED - TIME=14.18.47 - ASID=0042 - SC70
-OPENR002 ENDED.  NAME-                TOTAL CPU TIME=    .00  TOTAL

```

Figure 8-23 JES message log of test case 1

Test case 2

Function: Read an existing data sets on a SMS managed tape volume using IEBGENER. The used volume serial number is THS013. That is normal tape processing. Figure 8-24 on page 311 shows you the JCL we used.

Result: The job abended, although the user MHLRES7 have access to the data set residing on this volume the volume cannot be used on this system depend on the OPENRULE definition.

OPENRULE: All DFSMSrmm defined and not defined volumes are ignored on this system.

RACF: The user have access to the data set profile protecting this data set. There are an additionl need to have READ access to the special

recource

STGADMIN.EDG.IGNORE.TAPE.RMM.*

defined in RACF class facility, because the volume is ignored from DFSMSrmm.

```
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//LISTCONT EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//TESTCASE EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.TEST.OPENRULE.RULES,
// UNIT=ATL3,VOL=SER=THS013
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY
```

Figure 8-24 Test case 2 sample JCL

Where:

RACFCMDS show the STGADMIN.EDG.* resources defined in RACF class FACILITY.

LISTCONT show the OPENRULE definitions using the RMM TSO LISTCONTROL command.

TESTCASE Using the IEBGENER utility to read the data set.

Figure 8-25 shows you the result of the RACF SEARCH command to see that the resources STGADMIN.EDG.IGNORE.TAPE.RMM.* nor STGADMIN.EDG.IGNORE.TAPE.RMM.* are defined at this time.

```
READ
  SR CLASS(FACILITY) MASK(STGADMIN.EDG)
STGADMIN.EDG.LISTCONTROL
STGADMIN.EDG.MASTER
READY
```

Figure 8-25 Result of test case 2 RACF listing

The result of the RMM TSO LISTCONTROL subcommand is shown in Figure 8-26 on page 312 and you can see that there is only one OPENRULE definition for RMM and NORMM managed volumes. That means DFSMSrmm must depend on this definition ignore all tape requests on this system.

READY					
RMM LC OPE					
Openrule Entries:					
		Input		Output	
Volume or Range	Type	Action	Condition	Action	Condition
-----	-----	-----	-----	-----	-----
*	RMM	IGNORE	ANY	IGNORE	ANY
*	NORMM	IGNORE	ANY	IGNORE	ANY
READY					

Figure 8-26 Result of test case 2 LISTCONTROL

In Figure 8-27 we show you the JES message log of this job and you can see that the job gets a security violation and an error opening the tape volume.

IEF403I	OPENR003	-	STARTED	-	TIME=14.39.26	-	ASID=0042	-	SC70
-									--TIMINGS (MINS.)--
-JOBNAME	STEPNAME	PROCSTEP	RC	EXCP	CPU	SRB	CLOCK	SERV	PG
-OPENR003		RACFCMDS	00	28	.00	.00	.00	1213	0
-OPENR003		LISTCONT	00	32	.00	.00	.00	312	0
IEF233A	M	OB23,THS013,,OPENR003,TESTCASE,MHLRES7.TEST.OPENRULE.RULES							
ICH408I	USER(MHLRES7)	GROUP(SYS1)	NAME(MARY LOVELACE - RESI)	306					
			STGADMIN.EDG.IGNORE.TAPE.RMM.THS013 CL(FACILITY)						
			INSUFFICIENT ACCESS AUTHORITY						
			FROM STGADMIN.** (G)						
			ACCESS INTENT(READ) ACCESS ALLOWED(NONE)						
EDG4060I	VOLUME THS013	REJECTED. OPENRULE ACTION IGNORE BUT USE OUTSIDE							
EDG4006E	VOLUME THS013 ON OB23	REJECTED FOR USE BY OPENR003, TESTCASE, S							
IEC518I	SOFTWARE	ERRSTAT: REJTMS	OB23,THS013,SL,OPENR003,TESTCASE						
IEC502E	R	OB23,THS013,SL,OPENR003,TESTCASE							
IEC145I	413-08,IFG0194K,OPENR003,TESTCASE,SYSUT1,OB23,,MHLRES7.TEST.OPEN								
IEA848I	DUMP	SUPPRESSED - USER NOT AUTHORIZED BY SAF							
IEF450I	OPENR003	TESTCASE - ABEND=S413 U0000	REASON=00000008	313					
		TIME=14.39.44							
-OPENR003		TESTCASE *S413	40	.00	.00	.30	526	0	
IEF404I	OPENR003	-	ENDED	-	TIME=14.39.44	-	ASID=0042	-	SC70

Figure 8-27 JES message log of test case 2

Where:	
ICH408I	The STGADMIN.** resource is used because there are no more specific RACF resource defined in RACF class FACILITY. DFSMSrmm uses the normal RACF best matching processing to find a matching profile or resource. The user have currently no access tho this definition.
IEC145I	The 413-08 is a result of the reject of the tape volume depent on the OPENRULE definition.
EDG4060I	VOLUME volser REJECTED. OPENRULE ACTION IGNORE BUT USE OUTSIDE OF DFSMSrmm CONTROL NOT AUTHORIZED
Explanation:	The volume matched to an OPENRULE entry that specified an action of IGNORE. The IGNORE action requests that DFSMSrmm ignore this volume. The user must be authorized to request that the

specified volume be ignored. The user was not authorized and the request failed. Message EDG4060I can be accompanied by an ICH408I message that explains the reason for the authorization failure. If the ICH408I message is not issued, the most likely cause for the error is that there is no security profile defined for ignore processing. When there is no profile, DFSMSrmm does not allow a volume to be ignored. You should add one or more security profiles to ensure the correct authorization.

In the message text:

volser The volume serial number of the volume that the user is attempting to use.

System action: If DFSMSrmm is operating in warning mode, the volume specified in this message can be used and DFSMSrmm issues message EDG4004I. If DFSMSrmm is operating in protect mode, DFSMSrmm issues message EDG4005E or EDG4006E.

Operator response: None.

System programmer response:

If the volume should be ignored, define the RACF resource
 - STGADMIN.EDG.IGNORE.TAPE.volser,
 - STGADMIN.EDG.IGNORE.TAPE.NORMM.volser, or
 - STGADMIN.EDG.IGNORE.TAPE.RMM.volser
 and authorize the user. Refer to z/OS DFSMSrmm Implementation and Customization Guide for information about authorizing users.

Source: DFSMSrmm

Detecting Module: EDGOECM

Routing Code: 2,3

Descriptor Code: 3

EDG4006E **VOLUME volser ON rack_number REJECTED FOR USE BY jobname, stepname, ddname; OPEN REQUEST FAILED BY DFSMSrmm**

Explanation: Neither the current volume nor any other tape volume can be used for this mount request.

In the message text:

volser Volume serial number
rack_number Volume shelf location identifier
job_name Name of a job identified to a system.
stepname Name of a step within a job
ddname The data definition name

System action: The tape is rejected, and the job abnormally ends.

Operator response: None.

System programmer response:

Check the specified DD statement for incorrect volume, density, or label parameters. If the DD statement appears correct, review the DFSMSrmm parmlib options to determine if this was a valid occurrence.

Source: DFSMSrmm

Detecting Module: EDGSOCE
Routing Code: 2,3,11
Descriptor Code: 11

Test case 3

Function: Read an existing data sets on a SMS managed tape volume using IEBGENER. The used volume serial number is THS013. That is normal tape processing. Figure 8-28 shows you the JCL we used.
Result: The job ended without an error, although the volume must be ignored by the OPENRULE setting. In this case the user MHLRES7 have READ access to the resoure STGADMIN.EDG.IGNORE.TAPE.RMM.* we now have defined in RACF class facility and have access to the data set residing on the volume.
OPENRULE: All DFSMSrmm defined and not defined volumes are ignored on this system.
RACF: The user has access to the data set profile protecting this data set and have in the minimum READ access to the special resource STGADMIN.EDG.IGNORE.TAPE.RMM.* defined in RACF class facility, because the volume must be ignored by DFSMSrmm depend on the OPENRULE TYPE(RMM) definition for DFSMSrmm managede volumes.

```

//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//LISTCONT EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//TESTCASE EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.TEST.OPENRULE.RULES,
// UNIT=ATL3,VOL=SER=THS013
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY

```

Figure 8-28 Test case 3 sample JCL

Where:

RACFCMDS show the STGADMIN.EDG.* resources defined in RACF class FACILITY.
LISTCONT show the OPENRULE definitions using the RMM TSO LISTCONTROL command.
TESTCASE Using the IEBGENER utility to read the data set.

Figure 8-29 on page 315 shows you the result of the RACF SEARCH command to see that the resource STGADMIN.EDG.IGNORE.TAPE.RMM.* now is defined on this system.

```

READ
  RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.* -
    UACC(NONE) OWNER(SYS1 )
    PERMIT STGADMIN.EDG.IGNORE.TAPE.RMM.* -
      ID(MHLRES7) ACC(UPDATE) CLASS(FACILITY)

  SETROPTS GENERIC(FACILITY) REFRESH
  SETROPTS RACLIST(FACILITY) REFRESH
  SR CLASS(FACILITY) MASK(STGADMIN)
  STGADMIN.EDG.LISTCONTROL
  STGADMIN.EDG.MASTER
STGADMIN.EDG.IGNORE.TAPE.RMM.* (G)
READY

```

Figure 8-29 Result of test case 3 RACF listing

The result of the RMM TSO LISTCONTROL subcommand is shown in Figure 8-30 and you can see that there is only one OPENRULE definition for RMM and NORMMM managed volumes and that for both definitions all tapes must be ignored.

```

READY
  RMM LC OPE
  Openrule Entries:

```

Volume or Range	Type	Input Action Condition	Output Action Condition
*	RMM	IGNORE ANY	IGNORE ANY
*	NORMMM	IGNORE ANY	IGNORE ANY

```

READY

```

Figure 8-30 Result of test case 3 LISTCONTROL

In Figure 8-31 we show you the JES message log of this job and you can see there was no error, because now the resource **STGADMIN.EDG.IGNORE.TAPE.RMM.*** is defined and the user have in minimum **READ** access to this resource. If there is a need to write to a volume ignored by DFSMSrmm the user need in minimum **UPDATE** acces to this resource.

```

IEF403I OPENR004 - STARTED - TIME=14.49.10 - ASID=0042 - SC70
-
--TIMINGS (MINS.)--
-JOBNAME  STEPNAME  PROCSTEP   RC   EXCP   CPU   SRB   CLOCK  SERV  PG
-OPENR004          RACFCMDS    00    127    .00    .00    .00   1477   0
-OPENR004          LISTCONT    00     32    .00    .00    .00   309    0
IEF233A M 0B22,THS013,,OPENR004,TESTCASE,MHLRES7.TEST.OPENRULE.RULES
EDG4061I VOLUME THS013 IGNORED. IGNORE REQUESTED BY OPENRULE ACTION IGNO
IEF234E K 0B22,THS013,PVT,OPENR004,TESTCASE
-OPENR004          TESTCASE    00     76    .00    .00    .96   258    0
IEF404I OPENR004 - ENDED - TIME=14.50.09 - ASID=0042 - SC70

```

Figure 8-31 JES message log of test case 3

Test case 4

Function: Read an existing data sets on a SMS managed tape volume using IEBGENER. The used volume serial number is THS013. That is normal tape processing. Figure 8-32 shows you the JCL we used.

Result: The job ended without an error because the OPENRULE definition is changed to ACCEPT all DFSMSrmm defined volumes and IGNORE only all volumes not defined in the DFSMSrmm control data set.

OPENRULE: All DFSMSrmm defined tape volumes are accepted and all not defined volumes are ignored on this system.

RACF: The user has access to the data set profile protecting this data set. There are no need to have access to the special resources
STGADMIN.EDG.IGNORE.TAPE.RMM.*
STGADMIN.EDG.IGNORE.TAPE.RMM.*
defined in RACF class facility.

```
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//LISTCONT EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//TESTCASE EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.TEST.OPENRULE.RULES,
// UNIT=ATL3,VOL=SER=THS013
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY
```

Figure 8-32 Test case 4 sample JCL

Where:

RACFCMDS shows the STGADMIN.EDG.* resources defined in RACF class FACILITY.

LISTCONT shows the OPENRULE definitions using the RMM TSO LISTCONTROL command.

TESTCASE Using the IEBGENER utility to read the data set.

Figure 8-33 shows you the result of the RACF SEARCH command to see that the resources STGADMIN.EDG.IGNORE.TAPE.RMM.* nor STGADMIN.EDG.IGNORE.TAPE.RMM.* are not defined at this time.

```
READ
  SR CLASS(FACILITY) MASK(STGADMIN.EDG)
STGADMIN.EDG.LISTCONTROL
STGADMIN.EDG.MASTER
READY
```

Figure 8-33 Result of test case 4 RACF listing

The result of the RMM TSO LISTCONTROL subcommand is shown in Figure 8-34 on page 317 and you can see that there is an OPENRULE TYPE(RMM) definition for all DFSMSrmm defined tape volumes with the ACCEPT option that allows the use of volumes

defined in the DFSMSrmm control data set. All other volumes not defined in the DFSMSrmm control data set can only be used when the user have access to the resource

STGADMIN.EDG.IGNORE.TAPE.NORMM.volser

defined in RACF class facility.

READY					
RMM LC OPE					
Openrule Entries:					
		Input		Output	
Volume or Range	Type	Action	Condition	Action	Condition

*	RMM	ACCEPT		ACCEPT	
*	NORMM	IGNORE	ANY	IGNORE	ANY
READY					

Figure 8-34 Result of test case 4 LISTCONTROL

In Figure 8-23 on page 310 we show you the JES message log of this job and you can see there was no error.

IEF403I OPENR005 - STARTED - TIME=15.31.12 - ASID=0042 - SC70									
- --TIMINGS (MINS.)--									
-JOBNAME	STEPNAME	PROCSTEP	RC	EXCP	CPU	SRB	CLOCK	SERV	PG
-OPENR005		RACFCMDS	00	71	.00	.00	.00	1192	0
-OPENR005		LISTCONT	00	32	.00	.00	.00	315	0
IEF233A M OB22,THS013,,OPENR005,TESTCASE,MHLRES7.TEST.OPENRULE.RULES									
IEF234E K OB22,THS013,PVT,OPENR005,TESTCASE									
-OPENR005		TESTCASE	00	74	.00	.00	.29	230	0
IEF404I OPENR005 - ENDED - TIME=15.31.30 - ASID=0042 - SC70									
-OPENR005	ENDED.	NAME-					TOTAL CPU TIME=	.00	TOTAL

Figure 8-35 JES message log of test case 4

Test case 5

- Function:

In the DFSMSrmm control data set we change the data set name DFSMSrmm have automatically recorded at O/C/EOV time for tape volume THS013 to simulate a duplicate tape volume or a foreign tape volume, that means a tape coming from outside. The data set name used in the JCL does no longer matched the data set name DFSMSrmm has recorded for this volume. To read this data set we used IEBGENER too. The used volume serial number is THS013. Figure 8-36 on page 318 shows you the JCL we used.
- Result:

The job abended, although the user MHLRES7 have access to the data set, but the the volume cannot be used on this system depend on the OPENRULE TYPE(NORMM) definition.This rule is used by DFSMSrmm, because the data set name used in our JCL does not matched the data set name DFSMSrmm have recorded for this volume.
- OPENRULE:

All DFSMSrmm defined volumes are accepted by the OPENRULE TYPE(RMM) ACCEPT and can be used under normal conditions but all volumes not defined in the DFSMSrmm control data set are ignored

on this system depend on the OPENRULE TYPE(NORMM) IGNORE definition.

RACF:

The user has access to the data set profile protecting this data set. There is a need to have READ access to the special resource

STGADMIN.EDG.IGNORE.TAPE.NORMM.*

defined in RACF class facility, because the volume is ignored from DFSMSrmm based on the OPENRULE definition TYPE(NORMM), but this resource is not defined at this time.

```
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//LISTCONT EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//TESTCASE EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.TEST.OPENRULE.RULES,
// UNIT=ATL3,VOL=SER=THS013
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY
```

Figure 8-36 Test case 5 sample JCL

Where:

RACFCMDS shows the STGADMIN.EDG.* resources defined in RACF class FACILITY.

LISTCONT shows the OPENRULE definitions using the RMM TSO LISTCONTROL command.

TESTCASE Using the IEBGENER utility to read the data set.

Figure 8-37 shows you the result of the RACF SEARCH command to see that the resources STGADMIN.EDG.IGNORE.TAPE.RMM.* nor STGADMIN.EDG.IGNORE.TAPE.RMM.* are defined at this time. The resource STGADMIN.EDG.FORCE now present because we have a need that we were able to change the O/C/EOV recorded data set name for this volume using the RMM CHANGEVOLUME subcommand with the FORCE option.

```
READ
  SR CLASS(FACILITY) MASK(STGADMIN.EDG)
STGADMIN.EDG.FORCE
STGADMIN.EDG.LISTCONTROL
STGADMIN.EDG.MASTER
READY
```

Figure 8-37 Result of test case 5 RACF listing

The result of the RMM TSO LISTCONTROL and SEARCHDATASET subcommands are shown in Figure 8-38 on page 319 and you can see that there is an OPENRULE definition TYPE(NORMM) for all not DFSMSrmm managed volumes and that definition has an action of IGNORE for any request to these volumes. Also you can see that the data set name recorded for this volume in DFSMSrmm did not match the data set name we are using in our JCL. This volume must be a foreign volume in point of view of DFSMSrmm and so the OPENRULE with TYPE(NORMM) is used.

READY					
RMM LC OPE					
Openrule Entries:					
		Input		Output	
Volume or Range	Type	Action	Condition	Action	Condition

*	RMM	ACCEPT		ACCEPT	
*	NORMM	IGNORE	ANY	IGNORE	ANY
READY					
RMM SD VOLUME(THS013)					
Data set name			Volume Owner	Create date	Seq

MHLRES7.TEST.FROM.EXTERNAL			THS013 MHLRES7	2008/081	1
1 ENTRY LISTED					

Figure 8-38 Result of test case 5 LISTCONTROL and SEARCHDATASET

In Figure 8-39 we show you the JES message log of this job and you can see the job abended and cannot open the volume depend on the OPENRULE definition. Also you can see the missing access to the STGADMIN.EDG.IGNORE.TAPE.NORMM.volser resource defined in RACF class FACILITY.

```

IEF403I OPENR006 - STARTED - TIME=16.03.47 - ASID=0042 - SC70
-
--TIMINGS (MINS.)--
-JOBNAME  STEPNAME  PROCSTEP   RC   EXCP   CPU   SRB  CLOCK  SERV  PG
-OPENR006          RACFCMDS    00    66    .00    .00    .00  1196   0
-OPENR006          LISTCONT    00    32    .00    .00    .00   314   0
IEF233A M 0B23,THS013,,OPENR006,TESTCASE,MHLRES1.TEST.OPENRULE.RULES
ICH408I USER(MHLRES7 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI) 847
  STGADMIN.EDG.IGNORE.TAPE.NORMM.THS013 CL(FACILITY)
  INSUFFICIENT ACCESS AUTHORITY
  FROM STGADMIN.** (G)
  ACCESS INTENT(READ ) ACCESS ALLOWED(NONE )
EDG4060I VOLUME THS013 REJECTED. OPENRULE ACTION IGNORE BUT USE OUTSIDE
EDG4006E VOLUME THS013 ON 0B23 REJECTED FOR USE BY OPENR006, TESTCASE, S
IEC518I SOFTWARE ERRSTAT: REJTMS 0B23,THS013,SL,OPENR006,TESTCASE
IEC502E R 0B23,THS013,SL,OPENR006,TESTCASE
IEC145I 413-08,IFG0194K,OPENR006,TESTCASE,SYSUT1,0B23,,MHLRES1.TEST.OPEN
IEA848I DUMP SUPPRESSED - USER NOT AUTHORIZED BY SAF
IEF450I OPENR006 TESTCASE - ABEND=S413 U0000 REASON=00000008 854
      TIME=16.04.05
-OPENR006          TESTCASE *S413    38    .00    .00    .29   782   0
IEF404I OPENR006 - ENDED - TIME=16.04.05 - ASID=0042 - SC70

```

Figure 8-39 JES message log of test case 5

Note: The STGADMIN.** resource is used because there are no more specific RACF resource defined in RACF class FACILITY. DFSMSrmm uses the normal RACF best matching processing to find a matching profile or resource. The user have currently no access to this definition.

Refere to “Test case 2” on page 310. to see the description of the EDGnnnnc messages.

Test case 6

Function:	In the DFSMSrmm control data set we change the data set name DFSMSrmm have automatically recorded at O/C/EOV time for tape volume THS013 to simulate a duplicate tape volume or a foreign tape volume, that means a tape coming from outside. The data set name used in the JCL does no longer matched the data set name DFSMSrmm has recorded for this volume. To read this data set we used IEBGENER too. The used volume serial number is THS013. Figure 8-40 shows you the JCL we used.
Result:	The job ended without an error, because the user MHLRES7 have access to the data set and have the needed access to the resource STGADMIN.EDG.IGNORE.TAPE.NORMM.* depent on the OPENRULE TYPE(NORMM) IGNORE definition used for all volumes not defined in the DFSMSrmm control data set.
OPENRULE:	All DFSMSrmm defined volumes are accepted by the OPENRULE TYPE(RMM) ACCEPT and can be used under normal conditions but all volumes not defined in the DFSMSrmm control data set are ignored on this system depent on the OPENRULE TYPE(NORMM) IGNORE definition.
RACF:	The user has access to the data set profile protecting this data set and also access to the special recources STGADMIN.EDG.IGNORE.TAPE.NORMM.* defined in RACF class facility, because the volume is ignored from DFSMSrmm based on the OPENRULE TYPE(NORMM) definition.

```
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//LISTCONT EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
//TESTCASE EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.TEST.OPENRULE.RULES,
// UNIT=ATL3,VOL=SER=THS013
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY
```

Figure 8-40 Test case 6 sample JCL

Where:

RACFCMDS	shows the STGADMIN.EDG.* resources defined in RACF class FACILITY.
LISTCONT	shows the OPENRULE definitions using the RMM TSO LISTCONTROL command.
TESTCASE	Using the IEBGENER utility to read the data set.

Figure 8-41 on page 321 shows you the result of the RACF commands we have used to show you that we have deleted and new added in RACF class FACILITY the resource STGADMIN.EDG.IGNORE.TAPE.NORMM.*. This resource is used if a volume is not defined in the DFSMSrmm control data set and the resource STGADMIN.EDG.IGNORE.TAPE.RMM.* will be used if a volume is defined in the

DFSMSrmm control data set . The user MHLRES7 have UPDATE access to the resource STGADMIN.EDG.IGNORE.TAPE.NORMM.*, but only READ access is need to read a volume. It is normal RACF processing that means UPDATE access is more than READ access and solves the request. To the resource STGADMIN.EDG.IGNORE.TAPE.RMM.* we have also defined, but this resource is not used at this time.

```

READY
  RDELETE FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.*
READY
  RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.* -
    UACC(NONE) OWNER(SYS1 )
READY
  PERMIT STGADMIN.EDG.IGNORE.TAPE.RMM.* -
    ID(MHLRES7) ACC(UPDATE) CLASS(FACILITY)
READY

  RDELETE FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.*
READY
  RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.* -
    UACC(NONE) OWNER(SYS1 )
READY
  PERMIT STGADMIN.EDG.IGNORE.TAPE.NORMM.* -
    ID(MHLRES7) ACC(UPDATE) CLASS(FACILITY)
READY

  SETROPTS GENERIC(FACILITY) REFRESH
READY
  SETROPTS RACLIST(FACILITY) REFRESH
READY
  SR CLASS(FACILITY) MASK(STGADMIN)
READY
  STGADMIN.EDG.FORCE
  STGADMIN.EDG.LISTCONTROL
  STGADMIN.EDG.MASTER
  STGADMIN.EDG.IGNORE.TAPE.NORMM.* (G)
  STGADMIN.EDG.IGNORE.TAPE.RMM.* (G)

```

Figure 8-41 Result of test case 6 RACF listing

The result of the RMM TSO LISTCONTROL and SEARCHDATASET subcommands are shown in Figure 8-42 on page 322 and you can see that there is an OPENRULE TYPE(NORMM) definition for all not DFSMSrmm managed volumes and that definition have an action of IGNORE. Any request to this volumes must be ignored by DFSMSrmm. Also you can see that the data set name for this volume recorded in DFSMSrmm not matched the data set name we are using in our JCL. This volume must be a foreign volume in point of view of DFSMSrmm and so the OPENRULE TYPE(NORMM) is used.

```

READY
      RMM LC OPE
Openrule Entries:

```

Volume or Range	Type	Input		Output	
		Action	Condition	Action	Condition

*	RMM	ACCEPT		ACCEPT	
*	NORMM	IGNORE	ANY	IGNORE	ANY

```

READY
      RMM SD VOLUME(THS013)

```

Data set name	Volume Owner	Create date	Seq

MHLRES7.TEST.FROM.EXTERNAL	THS013 MHLRES7	2008/081	1

```

1 ENTRY LISTED

```

Figure 8-42 Result of test case 6 LISTCONTROL and SEARCHDATASET

In Figure 8-43 we show you the JES message log of this job and you can see the job ended without an error depend on the coorect access to the STGADMIN.EDG.IGNORE.TAPE.NORMMM.* resource defined in RACF class FACILITY.

IEF403I OPENR007 - STARTED - TIME=16.12.55 - ASID=004B - SC70									
- --TIMINGS (MINS.)-- --									
-JOBNAME	STEPNAME	PROCSTEP	RC	EXCP	CPU	SRB	CLOCK	SERV	PG
-OPENR007		RACFCMDS	00	164	.00	.00	.00	1854	0
-OPENR007		LISTCONT	00	32	.00	.00	.00	312	0
IEF233A M OB22,THS013,,OPENR007,TESTCASE,MHLRES7.TEST.OPENRULE.RULES									
EDG4061I VOLUME THS013 IGNORED. IGNORE REQUESTED BY OPENRULE ACTION IGNORE									
IEF234E K OB22,THS013,PVT,OPENR007,TESTCASE									
-OPENR007		TESTCASE	00	75	.00	.00	.30	244	0
IEF404I OPENR007 - ENDED - TIME=16.13.14 - ASID=004B - SC70									
-OPENR007	ENDED.	NAME-	TOTAL CPU TIME=			.00	TOTAL EL		

Figure 8-43 JES message log of test case 6

Where:

EDG4061I **VOLUME *volser* IGNORED. IGNORE REQUESTED BY OPENRULE ACTION IGNORE**

Explanation: The DFSMSrmm parmlib options for OPENRULE with an action of IGNORE has requested that this volume be ignored by DFSMSrmm. DFSMSrmm does not record any information about the specified volume and permits the volume's use. This is because the user is authorized to use a volume that is ignored or DFSMSrmm is running in record mode.

In the message text:

volser The volume serial number of the volume that the user is attempting to use.

System action: DFSMSrmm ignores this volume while it remains mounted. DFSMSrmm does not validate the mounted volume and does not

record any information about the current tape usage in the DFSMSrmm control data set.

Operator response: None.
Source: DFSMSrmm
Detecting Module: EDGOECM
Routing Code: 2,3
Descriptor Code: 3

8.6 Converting REJECT Commands

commands in order to use the PRTITION and OPENRULE commands. Review your existing REJECT commands across all system images and identify the sets of volumes to be used on each system. Identify the PRTITION and OPENRULE commands required for each system to minimize parmlib maintenance as your storage requirements change. Until you define one or more PRTITION or OPENRULE commands, both partitioning and rejecting of volumes are controlled by REJECT commands. Any REJECT commands that specify ANYUSE are used for partitioning of undefined volumes, but all REJECT commands are used for rejecting volumes at OPEN time.

When you use either PRTITION or OPENRULE commands, the REJECT commands are no longer used so you must start using both PRTITION and OPENRULE at the same time to avoid loss of function. You have to remove the REJECT commands from parmlib, because they will fail when any PRTITION or OPENRULE commands are defined. When parmlib is processed, DFSMSrmm issues message EDG0239E, followed by message EDG0215D to provide an option to ignore the error or fail and then retry with message EDG0107D with another parmlib member. When you have created your new commands, remove the REJECT commands. The PRTITION and OPENRULE commands can only be used on z/OS V1R10 and later releases. Lower level releases continue to use the REJECT commands. To give you more choice and flexibility, conversion from REJECT commands should not be done one to one and should not be automated. When each REJECT command is converted strictly to an equivalent PRTITION and OPENRULE command, you can end up with too much complexity and duplication. The best approach is to start from scratch and list the basic rules you want to implement. For example, consider the changed function and whether you are impacted by it:

- ▶ For volumes not defined to DFSMSrmm, the REJECT command with ANYUSE causes CUA requests to fail. With the PRTITION command, you can choose between the actions of either ACCEPT or IGNORE. You no longer have the option to fail a CUA request. The ACCEPT action results in the volume being added to DFSMSrmm and the CUA continues. The IGNORE action causes DFSMSrmm to take no action for the CUA request.
- ▶ For non-system-managed volumes that are not defined to DFSMSrmm, the default processing during OPEN did not add the volume to the CDS. The default processing for the PRTITION command is that non-system-managed volumes are added to the CDS.

8.6.1 Examples of Converting REJECT Commands

This section provides examples that show the conversion of REJECT commands to OPENRULE and PRTITION commands.

The sample commands in Figure 8-44 on page 324 shows REJECT being used by both partitioning and for open time rejects. A REJECT with a prefix, when used at open time, applies only to volumes defined to RMM so TYPE(RMM) is used for OPENRULE. However,

partitioning REJECT ANYUSE applies only to volumes NOT defined to RMM, so TYPE(NORMM) is used for PRTITION.

```
REJECT ANYUSE(prefix)

OPENRULE VOLUME(prefix) -
        TYPE(RMM) -
        ANYUSE(REJECT)

PRTITION VOLUME(prefix) -
        TYPE(NORMM) -
        SMT(IGNORE) -
        NOSMT(IGNORE)
```

Figure 8-44 Conversion of REJECT ANYUSE

The sample commands in Figure 8-45 shows REJECT is used only for open time rejects. A REJECT with a prefix, when used at open time, applies only to volumes defined to RMM so TYPE(RMM) is used for OPENRULE. The REJECT OUTPUT allows input processing, so code the INPUT option on the OPENRULE for completeness, although ACCEPT is the default

```
REJECT OUTPUT(prefix)

OPENRULE VOLUME(prefix) -
        TYPE(RMM) -
        OUTPUT(REJECT) -
        INPUT(ACCEPT)
```

Figure 8-45 Conversion of REJECT OUTPUT

The sample commands in Figure 87 shows REJECT is used by both partitioning and for open time rejects. A REJECT with PREFIX(*) applies only to volumes NOT defined to RMM, so TYPE(NORMM) is used for both OPENRULE and for PRTITION.

```
REJECT ANYUSE(*)

OPENRULE VOLUME(*) -
        TYPE(NORMM) -
        ANYUSE(REJECT)

PRTITION VOLUME(*) -
        TYPE(NORMM) -
        SMT(IGNORE) -
        NOSMT(IGNORE)
```

Figure 8-46 Conversion of REJECT ANYUSE(*)

The sample commands in Figure 8-47 on page 325 shows the following:

- ▶ Three systems, SYSA, SYSB, and SYSC, that are each allowed to use only a single range of system-managed volumes (JT, JP, JX).
- ▶ Each system has its own CDS and TCDB.
- ▶ System-managed volumes are not to be automatically defined to DFSMSrmm.
- ▶ Private volumes can be shared.

- ▶ Undefined non-system-managed volumes can be used for input.
- ▶ EDGUX100 is customized to automate the ignoring of private system-managed volumes from other partitions, and the ignoring of undefined non-system-managed volumes. This is based on the RMM API checking if a volume is defined to RMM. The following RACF FACILITY class profiles are also required:
 - STGADMIN.EDG.IGNORE.TAPE.NORMM.JP*
 - STGADMIN.EDG.IGNORE.TAPE.NORMM.JX*

```

/* SYSA example */
REJECT ANYUSE(JP*)
REJECT ANYUSE(JX*)
REJECT ANYUSE(*)

/* Allow read of undefined volumes */
OPENRULE VOLUME(*) -
    TYPE(NORMM) -
    OUTPUT(REJECT)

/* Ignore for input selected system managed volumes if authorized */
OPENRULE VOLUME(JP*) -
    TYPE(NORMM) -
    INPUT(IGNORE) -
    OUTPUT(REJECT)

OPENRULE VOLUME(JX*) -
    TYPE(NORMM) -
    INPUT(IGNORE) -
    OUTPUT(REJECT)

/* Global partition rule - ignore all smt volumes */
PRRTITION VOLUME(*) -
    TYPE(RMM) -
    SMT(IGNORE) -
    NOSMT(ACCEPT)

PRRTITION VOLUME(*) -
    TYPE(NORMM) -
    SMT(IGNORE) NOSMT(IGNORE)

/* This partition owns JT* volumes if predefined */
PRRTITION VOLUME(JT*) -
    TYPE(RMM) -
    SMT(ACCEPT)

```

Figure 8-47 Conversion of REJECT ANYUSE - Three Systems

8.7 Conversion from REJECT to PRRTITION and OPENRULE

If you want to implement the OPENRULE and PRRTITION commands but are already using REJECT commands in parmlib you must plan for conversion.

Until you define one or more PRRTITION or OPENRULE commands both partitioning and rejecting of volumes are controlled by REJECT commands. Any REJECT commands that specify ANYUSE are

used for partitioning of undefined volumes, but all REJECT commands are used for rejecting volumes at OPEN time.

When you use either PRTITION or OPENRULE commands the REJECT commands are no longer used so you must plan to start using both PRTITION and OPENRULE at the same time to avoid loss of function.

You do need to remove the REJECT commands from parmlib because they will be failed when any PRTITION or OPENRULE commands are defined. When parmlib is processed DFSMSrmm issues EDG0239E and then EDG0215D to provide an option to ignore the error or fail and then retry via EDG0107D with another parmlib member. When you have created your new commands, remove the REJECT commands.

The PRTITION and OPENRULE commands can only be used on z/OS V1R10 and later releases. Lower level releases continue to use the REJECT commands.

Conversion from REJECT commands is not best done one to one and so is not something which should be automated. The reason for this is that you have much more choice and flexibility and when each REJECT command is converted strictly to an equivalent PRTITION and OPENRULE command you can end up with too much complexity and also duplication. The best approach is to start from scratch and list the basic rules you want to implement; such as;

- ▶ For partitioning all non defined volumes are ignored
- ▶ For open rules all volumes defined to DFSMSrmm can be used for both input and output

When you have the basic rules you can next identify sets of volumes and how each is to be treated/managed. For example the only volumes to be used on a system/partition might be all those volumes with the 'A' prefix. You can create a command that uses either VOLUME(A*) or VOLUMERANGE('A':A99999) and select the TYPE based on whether you will always pre-define the volumes or allow them to be defined during cartridge entry and OPEN processing.

For the PRTITION commands you must consider the system-managed and non-system managed volumes separately. Be careful not to specify TYPE(RMM/ALL) with NOSMT(IGNORE) unless you really want EXPROC processing to skip the volumes. However, TYPE(RMM/ALL) with SMT(IGNORE) makes sense because DFSMSrmm causes OAM to ignore the volume and leave for another system/partition, and it also causes DFSMSrmm to skip EXPROC without checking if the volume is in the TCDB.

Consider the new function provided with the PRTITION and OPENRULE commands and whether it makes sense to exploit it;

- ▶ You can have separate commands based on whether the volume is defined to DFSMSrmm
- ▶ OPENRULE provides an automated way to ignore volumes at open time without the need to customize
- ▶ EDGUX100 or use EXPDT=98000 in JCL .. OPENRULE also allows you to control use of existing data sets such as enforcing cataloging and reference only from the creating system.

Consider the changed function and whether you are impacted by it;

- ▶ For volumes not defined to DFSMSrmm the REJECT command with ANYUSE caused CUA requests to be failed. With the new PRTITION command you can choose between the actions of either ACCEPT or IGNORE. You no longer have the option to fail a CUA request. The ACCEPT action results in the volume being added to DFSMSrmm and the

CUA continues. The IGNORE action causes DFSMSrmm to take no action for the CUA request.

- For non-system managed volumes which are not defined to DFSMSrmm the default processing during OPEN was not to add the volume to the CDS. The default processing for PRTITION is that non system managed volumes are added to the CDS.

Look at your use of EDGUX100, EDGUX200, and CBRUXENT installation exits to determine if they have been customized and might no longer be needed. The PRTITION and OPENRULE function should allow almost any customization related to partitioning, rejecting volumes, ignoring volumes, to be removed

Your installation exit customization need not be removed immediately but can be run alongside the new function, and only removed when you are happy that it is no longer needed.

CBRUXENT, EDGUX200 - Any existing customization of these exits can very likely be removed and replaced with use of PRTITION parmlib commands. EDGUX100 - Any existing function that controls use of a volume, that might fail open or reject a volume, or enable volumes to be ignored without coding EXPDT=98000, can likely be removed and replaced by use of the OPENRULE parmlib commands.

8.7.1 Partitioned library with shared CDS and TCDB

The first example in Figure 8-48 we show you a system managed tape library partitioned by:

- using CBRUXENT user exits on each system
- using EDGUX200 user exit on each system
- different scratch categories for each system in the library manager data base

The EDGHSKP expiration processing is running by system, but all volume in PRIVATE category are shared between the two systems.

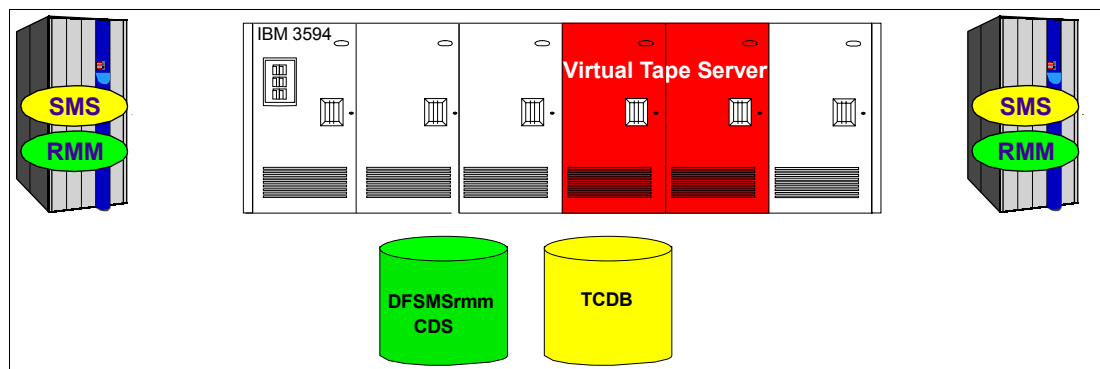


Figure 8-48 Partitioned library with shared CDS and TCDB

In this case all volumes are shared between the two systems and can be used on both systems. All volumes not defined to the DFSMSrmm control data sets should be ignored.

Figure 8-49 shows the new OPENRULE commands must be defined in EDGRMMnn parmlib member to replace the CBRUXENT and EDGUX200 user exit modifications.

OPENRULE VOLUME(*)	/*	added 2008/03/21 NS	*/-
TYPE(NORMM)	/*	volumes not defined in CDS	*/-
ANYUSE(IGNORE BY(ANY))	/*	bypass RMM processing	*/
	/*	ignore all types of request	*/
OPENRULE VOLUME(*)	/*	added 2008/03/21 NS	*/-
TYPE(RMM)	/*	all volumes defined in CDS	*/-
ANYUSE(ACCEPT)	/*	allow OPEN processing	*/

Figure 8-49 Convert a single CDS and TCDB to OPENRULE and PRRTITION commands

8.7.2 Partitioned library with shared TCDB and different CDSs

This example in Figure 8-50 we show you a system managed tape library partitioned by:

- ▶ different DFSMSrmm control data set for each system
- ▶ use of REJECT ANYUSE in EDGRMMnn parmlib member
- ▶ different scratch categories for each system in the library manager data base

The EDGHSKP expiration processing is running by CDS, but all volume in PRIVATE category are shared between the two systems. To access this volumes you have to specify one of the two JCL parameters EXPDT=98000 or ACCODE=XCANORES. For both parameters the default shipped EDGUX100 user exit is needed.

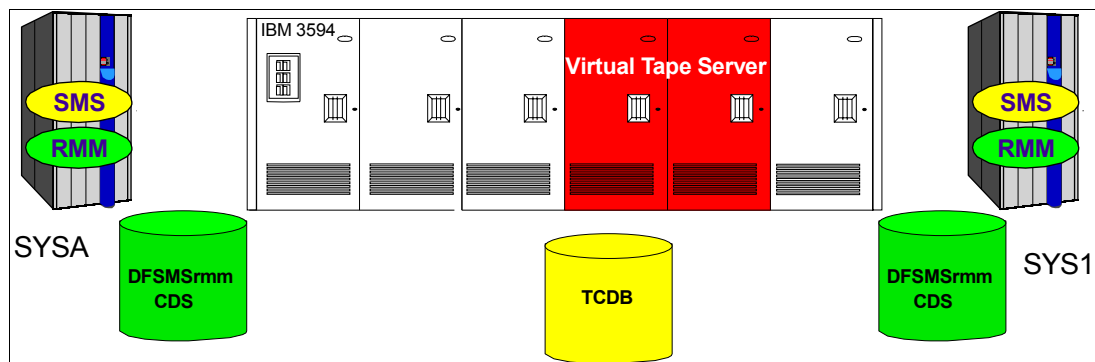


Figure 8-50 Partitioned library with shared TCDB and different CDSs

In this case we have two systems sharing a DFSMSrmm control data set, but each system have a separate tape control data base (TCDB). On system SYSA we are using only volumes starting with a volume prefix of A and on system SYS1 we are using volumes starting with the prefixes 1* and 2*. The use of volumes from the other system is in both cases not allowed.

Figure 8-51 on page 329 shows the new PRRTITION and OPENRULE commands must be defined in EDGRMMnn parmlib member to replace the EDGUX100 user exit. The conversion to this commands allows you to define new volumes in the library manager data base and DFSMSrmm will accept the add of this volumes in the DFSMSrmm control data set if the prefix is matching the prefix we have specified in the PRRTITION command. Also you can add the volumes first in the DFSMSrmm control data set and using the add volume command on the library manager or inserting the volume into the library later.

On system SYSA all tapes with a prefix of A* can be used inside and outside the ATL, but on system SYS1 the use of tape volumes outside the ATL is not allowed.

```

global setting:
OPENRULE VOLUME(*)           /*      added 2008/03/21 NS      */-
      TYPE(NORMM)            /* volumes not defined in CDS  */-
      ANYUSE(IGNORE BY(ANY)) /* bypass RMM processing       */
                          /* ignore all types of request */

OPENRULE VOLUME(*)           /*      added 2008/03/21 NS      */-
      TYPE(RMM)              /* all volumes defined in CDS   */-
      ANYUSE(ACCEPT)         /* allow OPEN processing        */-

PRITITION VOLUME(*)          /*      added 2008/03/21 NS      */-
      TYPE(NORMM)            /* not defined in DFSMSrmm     */-
      SMT(IGNORE)            /* ignore SMS managed volumes   */-
      NOSMT(IGNORE)          /* ignore volumes outside an ATL/VTS */

SYSA:
PRITITION VOLUME(A*)         /*      added 2008/03/21 NS      */-
      TYPE(ALL)              /* volumes defined/not defined in CDS */-
      SMT(IGNORE)            /* ignore SMS managed volumes   */-
      NOSMT(IGNORE)          /* ignore volumes outside an ATL/VTS */

SYS1:
OPENRULE VOLUME(A*)          /*      added 2008/03/21 NS      */-
      TYPE(ALL)              /* volumes defined/not defined in CDS */-
      INPUT(ACCEPT)          /* allow OPEN input processing   */-

PRITITION VOLUME(1*)         /*      added 2008/03/21 NS      */-
      TYPE(ALL)              /* volumes defined/not defined in CDS */-
      SMT(ACCEPT)            /* process the volume           */-
      NOSMT(ACCEPT)          /* process the volume           */-

PRITITION VOLUME(2*)         /*      added 2008/03/21 NS      */-
      TYPE(ALL)              /* volumes defined/not defined in CDS */-
      SMT(ACCEPT)            /* process the volume           */-
      NOSMT(ACCEPT)          /* process the volume           */-

```

Figure 8-51 Convert a single TCDB with two CDSs to OPENRULE and PRITITION commands

8.7.3 Partitioned library with unshared CDS and TCDB

In the third example shown in Figure 8-52 on page 330 you can see a system managed tape library partitioned by:

- by system
- use of REJECT ANYUSE in EDGRMMnn parmlib member
- different scratch categories for each system in the library manager data base

The EDGHSKP expiration processing is running by system. All volumes are not shared so you have to define a volume in private status in the TCDB first. To access this volumes you have to specify one of the two JCL parameters EXPDT=98000 or ACCODE=XCANORES. For both parameters the default shipped EDGUX100 user exit is needed.

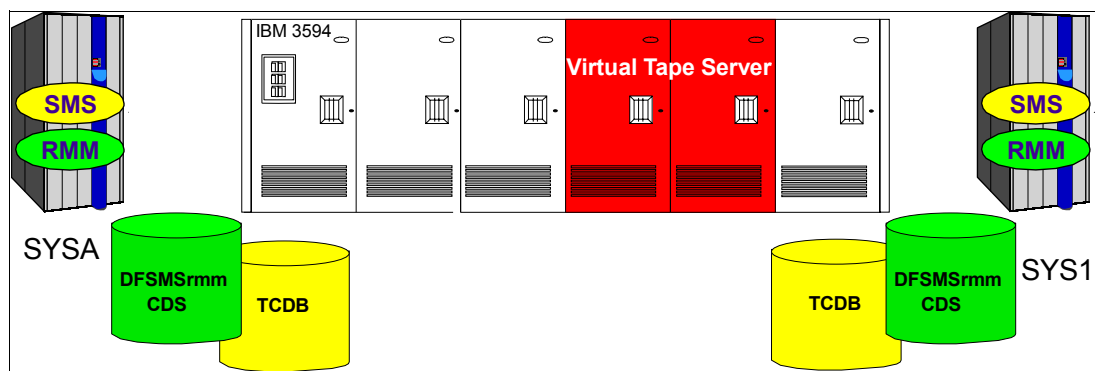


Figure 8-52 Partitioned library with unshared CDS and TCDB

This is a case where REJECT is used by both partitioning and for open time rejects. A REJECT with PREFIX(*) applies only to volumes NOT defined to RMM, so TYPE(NORMMM) is used for both OPENRULE and for PRITITION.

Figure 8-53 shows the new PRITITION and OPENRULE commands must be defined in EDGRMMnn parmlib member to replace the EDGUX100 user exit.

OPENRULE	VOLUME(*)	/* added 2008/03/21 NS	*/-
	TYPE(NORMMM)	/* volumes not defined in CDS	*/-
	ANYUSE(IGNORE BY ANY))	/* bypass RMM processing	*/
		/* ignore all types of request	*/
OPENRULE	VOLUME(*)	/* added 2008/03/21 NS	*/-
	TYPE(RMM)	/* all volumes defined in CDS	*/-
	ANYUSE(ACCEPT)	/* allow OPEN processing	*/
PRITITION	VOLUME(*)	/* added 2008/03/21 NS	*/-
	TYPE(NORMMM)	/* not defined in DFSMSrmm	*/-
	SMT(IGNORE)	/* ignore SMS managed volumes	*/-
	NOSMT(IGNORE)	/* ignore volumes outside an ATL/VTS	*/
SYS1:			
PRITITION	VOLUME(A*)	/* added 2008/03/21 NS	*/-
	TYPE(ALL)	/* volumes defined/not defined in CDS	*/-
	SMT(ACCEPT)	/* process the volume	*/-
	NOSMT(ACCEPT)	/* process the volume	*/
SYS1:			
PRITITION	VOLUME(1*)	/* added 2008/03/21 NS	*/-
	TYPE(ALL)	/* volumes defined/not defined in CDS	*/-
	SMT(ACCEPT)	/* process the volume	*/-
	NOSMT(ACCEPT)	/* process the volume	*/

Figure 8-53 Convert two systems sharing a library to OPENRULE and PRITITION commands

8.7.4 Partitioned library with shared CDS and unshared TCDB

The last example in Figure 8-6 on page 295 we show you a system managed library partitioned by:

- ▶ by system
- ▶ using customized version of CBRUXENT user exits on each system
- ▶ different scratch categories for each system in the library manager data base

The EDGHSKP expiration processing is running by system based on the TCDB. All volumes are not shared so you have to define a volume in private status in the TCDB first before you can use it. Figure 8-4 on page 295 shows you how you can add a volume in the TCDB using AMS commands. After you have successfully processed the volume you have previously defined in the TCDB you should delete the entry from the TCDB using the command as shown in Figure 8-54.

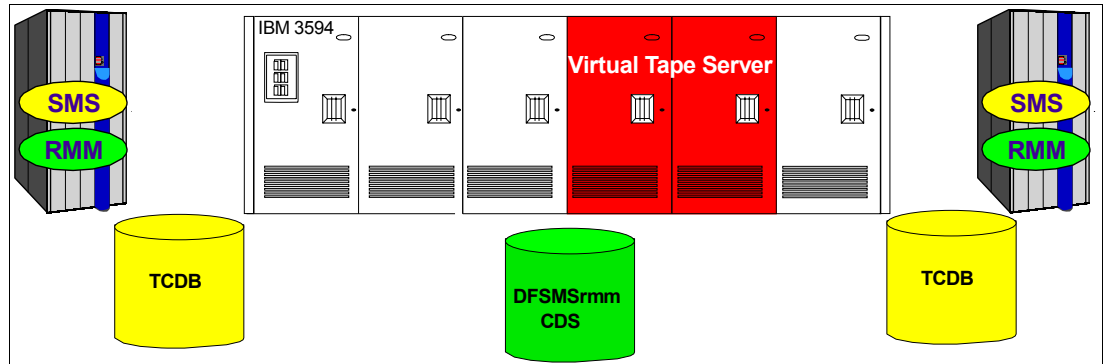


Figure 8-54 Partitioned library with shared CDS and unshared TCDB

This is a case where REJECT is used by both partitioning and for open time rejects. A REJECT with PREFIX(*) applies only to volumes NOT defined to RMM, so TYPE(NORMM) is used for both OPENRULE and for PRITITION.

Figure 8-55 on page 332 shows the new PRITITION and OPENRULE commands must be defined in EDGRMMnn parmlib member to replace the customized CBRUXENT user exit. The conversion to this commands allows you to define new volumes in the library manager data base and DFSMSrmm will accept the add of this volumes in the DFSMSrmm control data set if the prefix is matching the prefix we have specified in the PRITITION command. Also you can add the volumes first in the DFSMSrmm control data set and using the add volume command on the library manager or inserting the volume into the library later.

```

OPENRULE VOLUME(*)           /*      added 2008/03/21 NS      */-
      TYPE(NORMM)            /* volumes not defined in CDS  */-
      ANYUSE(IGNORE BY(ANY)) /* bypass RMM processing       */
                          /*      ignore all types of request */

OPENRULE VOLUME(*)           /*      added 2008/03/21 NS      */-
      TYPE(RMM)              /* all volumes defined in CDS   */-
      ANYUSE(ACCEPT)         /* allow OPEN processing        */-

PRTITION VOLUME(*)           /*      added 2008/03/21 NS      */-
      TYPE(ALL)              /* volumes defined/not defined in CDS */-
      SMT(IGNORE)            /* ignore SMS managed volumes    */-
      NOSMT(IGNORE)          /* ignore volumes outside an ATL/VTS */

SYSA:
PRTITION VOLUME(A*)          /*      added 2008/03/21 NS      */-
      TYPE(ALL)              /* volumes defined/not defined in CDS */-
      SMT(IGNORE)            /* ignore SMS managed volumes    */-
      NOSMT(IGNORE)          /* ignore volumes outside an ATL/VTS */

SYS1:
PRTITION VOLUME(1*)          /*      added 2008/03/21 NS      */-
      TYPE(ALL)              /* volumes defined/not defined in CDS */-
      SMT(IGNORE)            /* ignore SMS managed volumes    */-
      NOSMT(IGNORE)          /* ignore volumes outside an ATL/VTS */

```

Figure 8-55 Convert two TCDBs with one CDS to OPENRULE and PRTITION commands



Catalog synchronization

This chapter explains how to synchronize your MVS user catalogs with DFSMSrmm to reduce catalog locates.

The objectives of this chapter are to:

- ▶ “Overview of catalog synchronization”
- ▶ “Preparing DFSMSrmm catalog synchronization”
- ▶ “Synchronizing with a shared catalog environment”
- ▶ “Synchronizing when catalogs are not shared”
- ▶ “IPV6 Supports ne”

This chapter also provides detailed information about the implementation of the DFSMSrmm catalog synchronization support.

9.1 Overview of catalog synchronization

Install the DFSMSrmm level of code that performs catalog tracking and catalog synchronization (DFSMSrmm OS/390 V2R10 or later) on all systems that share the control data set (CDS). When you have the correct level of DFSMSrmm code installed, DFSMSrmm dynamically records all catalog updates for tape data sets in the DFSMSrmm CDS. DFSMSrmm always tracks tape data set catalog activity independently of the CATSYSID parameter.

When you enable catalog synchronization, DFSMSrmm synchronizes the DFSMSrmm CDS and available user catalogs. Enabling catalog synchronization is normally a one-time task. When the catalog status is synchronized, DFSMSrmm continually tracks and updates the catalog status as shown in Figure 9-1. In this figure, CAS means *catalog address space*.

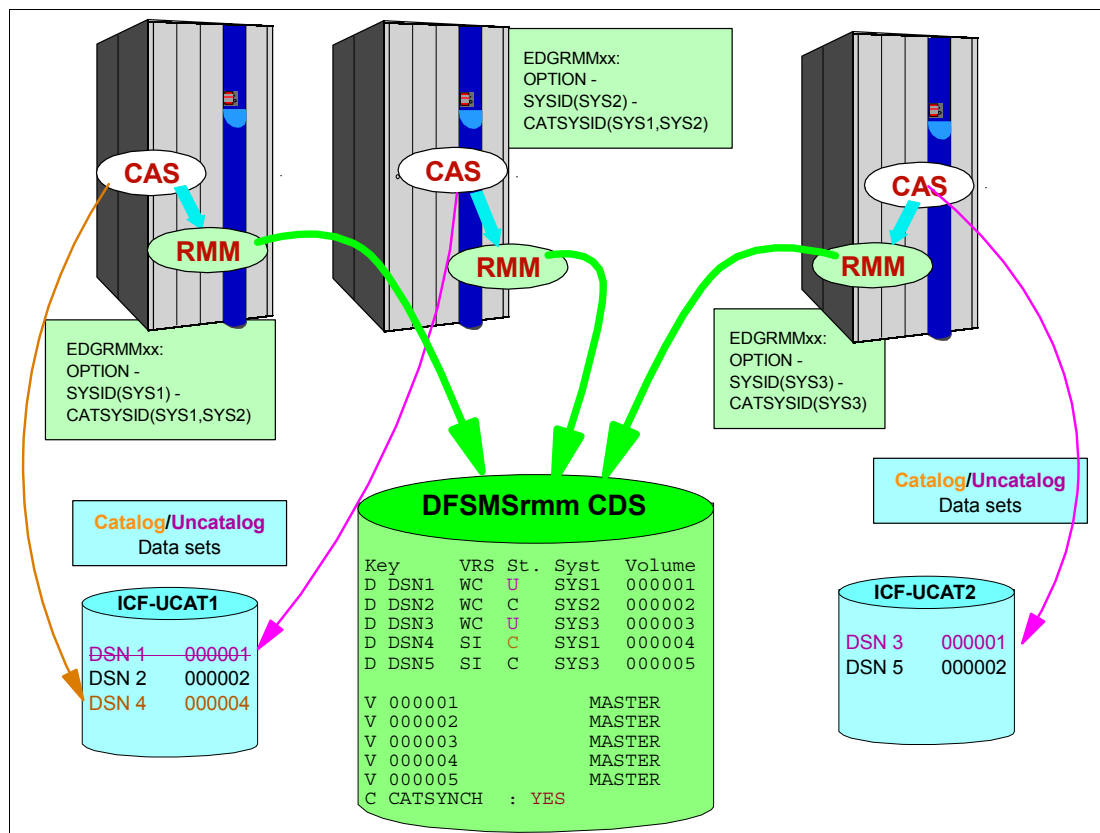


Figure 9-1 Catalog status tracking

You enable catalog synchronization by specifying the DFSMSrmm EDGRMMxx OPTION CATSYSID operand. When you specify the CATSYSID operand, DFSMSrmm uses the catalog search interface to obtain catalog information used for DFSMSrmm vital record processing.

DFSMSrmm performs checking to determine that the catalog search interface catalog environment does not have any errors that might prevent successful processing. DFSMSrmm issues messages EDG2235E, EDG2236I, or EDG2237E when an error is encountered. If you do not specify the DFSMSrmm EDGRMMxx OPTION CATSYSID operand, DFSMSrmm vital record processing uses catalog locates to determine if data sets are still cataloged.

For detailed information about catalog synchronization, refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

9.1.1 DFSMSrmm catalog processing

When you run DFSMSrmm with user catalogs and the DFSMSrmm CDS unsynchronized, DFSMSrmm issues catalog locates as required to check if data sets are cataloged. Catalog locates use the standard catalog search to determine whether a data set is cataloged. Any data sets cataloged using JOBCAT or STEPCAT may not be found to be cataloged because a standard catalog search may not find them.

When you run DFSMSrmm with user catalogs and the DFSMSrmm CDS synchronized, DFSMSrmm does not need to issue catalog locates to find if a data set is cataloged. The catalog status tracked by DFSMSrmm in the CDS is used to determine if a data set is cataloged. When DFSMSrmm tracks catalog status, it does so regardless of whether JOBCAT or STEPCAT is used.

During catalog synchronization, DFSMSrmm uses the Catalog Search Interface (CSI) to retrieve data set catalog information. CSI returns catalog information for all data sets in all catalogs that are in or connected to the master catalog. Because of this, DFSMSrmm can detect a data set is cataloged even if it cannot be found using the standard catalog search.

9.1.2 Reasons to re-synchronize catalog synchronization

Set up automatic synchronization by automating the responses to DFSMSrmm messages **EDG8200E** and **EDG8201E**. Run the EDGUTIL UPDATE with the SYSIN command **CONTROL CDSID(PROD) CATSYNCH(NO)**. You receive these error messages when there is an error in DFSMSrmm catalog processing.

You must re-synchronize the DFSMSrmm CDS and user catalogs under the following conditions:

- ▶ You ran EDGHSKP with CATSYNCH and VERIFY, and differences in catalog status were found. DFSMSrmm clears the catalog synchronization date and time to force you to run CATSYNCH.
- ▶ DFSMSrmm is not active when catalog activity for tape data sets is taking place. DFSMSrmm issues messages EDG8200E and EDG8201E when DFSMSrmm cannot track catalog updates.
- ▶ A DFSMSrmm failure occurs during catalog processing. DFSMSrmm issues messages EDG8200E and EDG8201E when an error is detected in processing.
- ▶ You connect or disconnect user catalogs, which contain entries for tape data sets, to the master catalog.

When you use IDCAMS REPRO MERGECAT for catalog maintenance, you do not need to run a job to re-synchronize the DFSMSrmm CDS and catalogs.

Recommendation: Use EDGUTIL with the UPDATE parameter and with the SYSIN command CONTROL CDSID(PROD) CATSYNCH(NO) parameter when you think the DFSMSrmm CDS and the catalogs are no longer synchronized. This prevents DFSMSrmm from relying on the catalog information in the CDS.

Refer to the “Maintaining the Control Data Set” chapter in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, for more details.

9.2 Preparing DFSMSrmm catalog synchronization

The following discussion uses the configuration shown in Figure 9-2. This diagram shows a complex RMMplex implementation with two sysplexes and one non-plex sharing one single DFSMSrmm CDS. The second non-plex shows a normal shared DFSMSrmm CDS. Each plex has its own direct access storage device (DASD) environment that is not shared between the other plexes. Also, each plex has a unique MVS catalog environment.

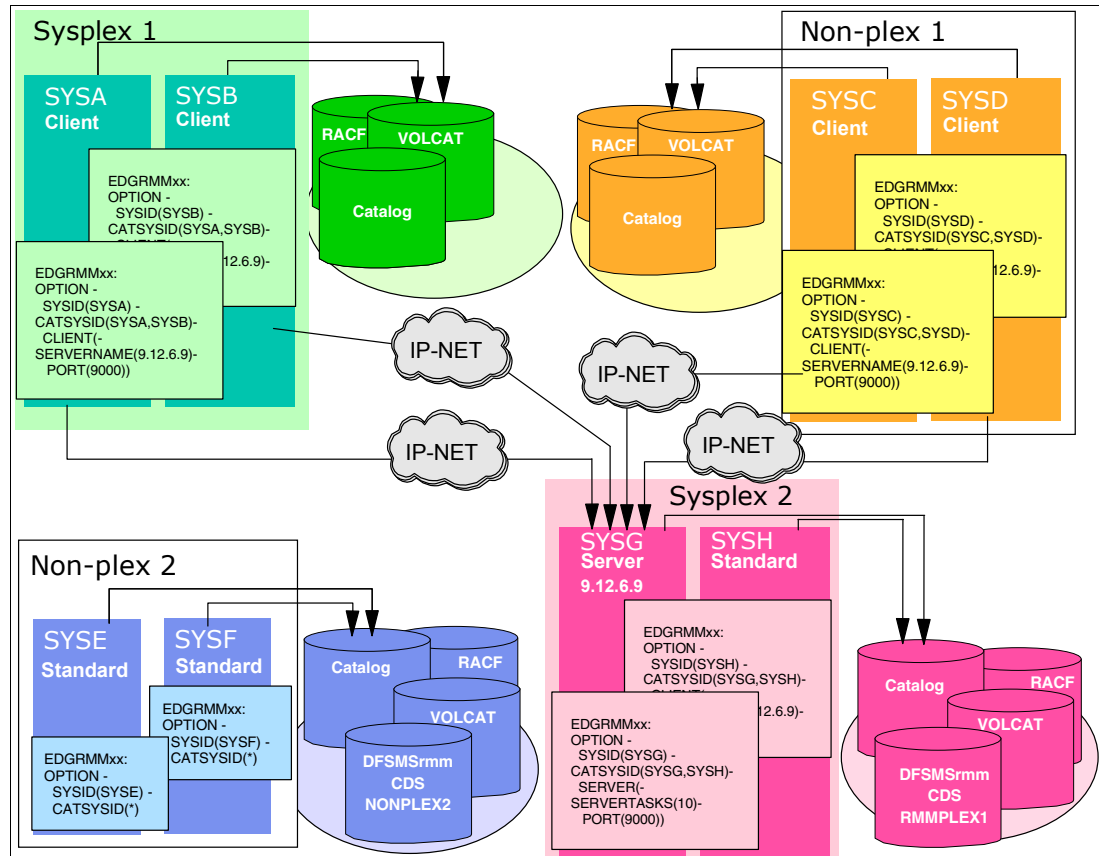


Figure 9-2 RMMplex with six systems sharing one DFSMSrmm CDS

When you enable catalog synchronization, DFSMSrmm synchronizes the DFSMSrmm CDS and available user catalogs. Enabling catalog synchronization is normally a one-time action. When the catalog is synchronized, DFSMSrmm continually tracks and updates the catalog status. You enable catalog synchronization by specifying the DFSMSrmm EDGRMMxx OPTION CATSYSID operand. You can learn more about this operand in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, and the chapter titled “Defining System Options: OPTION.”

Before you start catalog synchronization, check for the following items:

1. Verify that DFSMSrmm has all the authorization to read all ICF user catalogs including the tape volume catalog (TCDB). Normally, the TCDB is named SYS1.VOLCAT.VGENERAL. If you used a name other than the default, you can find your definition in the DEVSUPnn member in SYS1.SAMPLIB.
2. In an environment with unshared catalogs, make sure that for all current and previously used system IDs, there is an entry in the CATSYSID option of one of the systems.

3. Check for data sets with no create system ID, because if there is no create system ID, you can lose data. There may be no create system ID as a result of the data set records being created during a conversion to RMM using an older version of the conversion tools.

9.2.1 Message data set considerations

Normally the allocation of the DFSMSrmm message data set is small because you get only a small amount of information if the daily housekeeping is running. If catalog synchronization is running the first time, you see information for each data set that is changed in this run. After conversion, the catalog status of each data set is unknown and is updated to yes or no in this run.

Allocating a larger message data set

You must allocate a larger DFSMSrmm message data set, because the catalog synchronization process writes a record to the DFSMSrmm message file for each data set that is updated in the DFSMSrmm CDS. You can use the JCL shown in Example 9-1 to allocate a larger message data set. If you have catalogs that are not shared, you have to allocate a larger message data set on each single system or plex on which catalog synchronization must run.

Example 9-1 *Sample JCL to allocate a larger DFSMSrmm message data set*

```
//CATSYNC1 JOB (DE0A047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//* *****
//*          STEP 1  DELETE OLD DFSMSRMM MESSAGE FILE          *
//*          2  ALLOCATE A NEW DFSMSRMM MESSAGE FILE          *
//* *****
//STEP01   EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN     DD   *
DEL RMM.HSKP.MESSAGE.CATSYNC  NONVSAM PURGE
SET MAXCC=0
/*
//IEFBR14   EXEC PGM=IEFBR14
//MESSAGE  DD  DISP=(,CATLG,DELETE),
//          DCB=(LRECL=133,BLKSIZE=6118,RECFM=FBA),
//          DSN=RMM.HSKP.MESSAGE.CATSYNC,
//          SPACE=(TRK,(100,100))
```

Note: After you check your MESSAGE and ACTIVITY data sets and you successfully synchronize your catalogs, you can delete these files.

9.2.2 Activity log data set considerations (optional)

In a normal daily housekeeping run, you see only a small number of records that tell you the changes made in the vital record processing. During the catalog synchronization process, a record for each data set is changed during the run.

Allocating a larger activity log data set

You must allocate a larger DFSMSrmm activity log data set, because the catalog synchronization process writes a record to the DFSMSrmm message file for each data set that is updated in the DFSMSrmm CDS. You can use the JCL shown in Example 9-2 on

page 338 to allocate a larger message data set. If you have catalogs that are not shared, allocate a larger activity log data set on each single system or plex on which the catalog synchronization must run.

Example 9-2 *Sample JCL to allocate a larger DFSMSrmm activity log data set*

```
//CATSYNC1 JOB (DE0A047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//* *****
//*          STEP 1  DELETE OLD DFSMSRMM ACTIVITY LOG          *
//*          2  ALLOCATE A NEW DFSMSRMM ACTIVITY LOG          *
//* *****
//STEP01   EXEC PGM=IDCAMS
//SYSPRINT DD   SYSOUT=*
//SYSIN    DD   *
DEL RMM.HSKP.ACTIVITY.CATSYNC NONVSAM PURGE
SET MAXCC=0
/*
//IEFBR14   EXEC PGM=IEFBR14
//ACTIVITY DD   DISP=(,CATLG,DELETE),
//          DCB=(LRECL=500,BLKSIZE=0,RECFM=VB),
//          DSN=RMM.HSKP.ACTIVITY.CATSYNC,
//          SPACE=(TRK,(100,100))
```

Note: After you check your MESSAGE and ACTIVITY data sets and successfully synchronize your catalogs, you can delete this file.

9.2.3 Journal considerations

The catalog synchronization process also generates many records in the journal data set. Prepare for this by allocating a large journal data set, or process the catalog synchronization run with the journal data set disabled.

Allocating a larger journal data set

Example 9-3 shows sample JCL to create a larger DFSMSrmm journal.

Example 9-3 *Sample JCL to create a larger DFSMSrmm journal*

```
//LCLJRNL JOB ,140.SCHLUMBERGER,NOTIFY=Schlum,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//* *****
//*          STEP 1  DELETE OLD DFSMSRMM JOURNAL          *
//*          2  ALLOCATE A NEW DFSMSRMM JOURNAL          *
//* *****
//STEP01   EXEC PGM=IDCAMS
//SYSPRINT DD   SYSOUT=*
//SYSIN    DD   *
DEL RMM.PROD.JRNL.LARGE NONVSAM PURGE
SET MAXCC=0
/*
//STEP02   EXEC PGM=IEFBR14
//SYSPRINT DD   SYSOUT=*
//JOURNAL DD   DISP=(,CATLG,DELETE),
//          DSN=RMM.PROD.JRNL.LARGE,
```

```
//          UNIT=3390,VOL=SER=DFRMM3,  
//          AVGREC=U,SPACE=(4096,(15000),,CONTIG)
```

Disabling journal data set (optional)

You can disabling the journal if there are no other tape activities at the time you are synchronizing your catalogs. The possibility of disabling the journaling depends on how the journal data set is defined in the DFSMSrmm procedure and in the PARMLIB member EDGRMMxx.

If the journal data set is not defined in the DFSMSrmm procedure, and is defined in the PARMLIB member EDGRMMxx, it is possible to perform a EDGHSKP PARM(VRSEL) run with the journaling disabled:

1. Create a new member EDGRMMyy by copying the existing member EDGRMMxx in PARMLIB.
2. In the new member EDGRMMyy, remove the JRNLNAME statement in the OPTION parameter.
3. Check that you have not specified a JOURNAL DD statement in your DFRMM started task job control.
4. Restart DFSMSrmm with the new EDGRMMyy member, by issuing the operator MODIFY command as shown in .

```
F DFRMM,M=yy
```

Figure 9-3 Figure 9-3MVS command MODIFY DFRMM

When DFSMSrmm is restarted without the journal data set, you receive the messages shown in Figure 9-4 in your system log.

```
EDG0204I DFSMSrmm BEING INITIALIZED FROM MEMBER EDGRMMyy IN SYS1.PARMLIB  
EDG0122I NO JOURNAL FILE ALLOCATED - JOURNALING DISABLED  
EDG0105I DFSMSrmm SUBSYSTEM INITIALIZATION COMPLETE
```

Figure 9-4 DFSMSrmm journal disabled messages

You can also check the status of the journal data set by using the command shown in Figure 9-5.

```
RMM LISTCONTROL CNTL
```

Figure 9-5 RMM LISTCONTROL CNTL subcommand

Example 9-4 shows the output of the LISTCONTROL command.

Example 9-4 LISTCONTROL OPTION output

```
Control record:  
Type = MASTER      Create date = 2009/309      Create time = 18:38:11  
                  Update date = 2010/282      Update time = 14:30:55  
Journal: Utilization = 0% (75% threshold)      STATUS: = DISABLED  
CDS:      Utilization = 87%  
Exit status:                                Options:
```

EDG_EXIT100 = NONE	Stacked Volumes = NONE
EDG_EXIT200 = NONE	Extended Bin = DISABLED
EDG_EXIT300 = ENABLED	Common Time = DISABLED
	CDSID ENQ name = ENABLED
Last backup:	Last expiration processing:
Date = 2010/282 Time = 14:29:58	Date = 2010/280 Time = 16:33:30
Last journal backup:	Last store update:
Date = 2010/282 Time = 14:29:58	Date = Time =
Last report extract:	Last VRS processing:
Date = 2010/280 Time = 16:33:30	Date = 2010/280 Time = 16:33:30
Last scratch procedure:	Last Catalog synchronize:
Date = Time =	Date = Time =
Rack numbers = 6288	Empty racks = 6263
LOCAL store bins = 0	Empty LOCAL bins = 0
DISTANT store bins = 0	Empty DISTANT bins = 0
REMOTE store bins = 0	Empty REMOTE bins = 0
Control functions in progress:	
Backup = N Restore = N	
Verify = N Expiration = N	
Report Extract = N Disaster Store = N	
VRS = N Synchronize = N	
Client/Server:	
host name =	
IP address =	

The Journal file data set name field should be blank when the journal data set is disabled.

DFSMSrmm now runs with the journal data set disabled, and the EDGHSKP PARM(VRSEL) job can start. Running EDGHSKP without journaling also reduces the execution time.

9.2.4 Consideration for create system ID

In an environment with unshared catalogs, check for all current and previously used system IDs. Include them in the CATSYSID values that you have to specify in your EDGRMMxx PARMLIB members.

Example 9-5 shows sample JCL that can be used to find all system IDs that were created previously and currently in use. A backup copy of the current DFSMSrmm CDS is used as input to gather the necessary information.

Example 9-5 *Sample JCL to obtain all system IDs in your RMMplex environment*

```
//CRSYSID JOB (DE0A047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//STEP0001 EXEC PGM=EDGHSKP,
//          PARM=('BACKUP(AMS)')
//SYSPRINT DD SYSOUT=*
//MESSAGE DD DSN=RMM.HSKP.MESSAGE,DISP=SHR
//BACKUP DD DSN=RMM.HSKP.BACKUP(+1),DISP=(,CATLG),
//          UNIT=SYSDA,SPACE=(CYL,(500,50),RLSE),
//          DCB=(LRECL=9216,BLKSIZE=0,REFM=VB,DSORG=PS)
//JRNLBKUP DD DSN=RMM.HSKP.JNLBKUP(+1),DISP=(,CATLG),
//          UNIT=SYSDA,SPACE=(CYL,(500,50),RLSE),
//          DCB=(LRECL=9248,BLKSIZE=0,REFM=VB,DSORG=PS)
/* ***** *
```

```
//STEP0002 EXEC PGM=IEBGENER
//SYSPRINT DD DUMMY
//SYSUT1 DD DSN=RMM.HSKP.MESSAGE,DISP=SHR
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY
/* *****
//STEP0003 EXEC PGM=SORT
//SYSOUT DD SYSOUT=*
//SORTIN DD DISP=SHR,DSN=RMM.HSKP.BACKUP(+1)
//SORTOUT DD DISP=(,CATLG,DELETE),
// DCB=(LRECL=12,BLKSIZE=0,RECFM=VB),
// DSN=RMM.HSKP.ALL.CATSYSID,
// SPACE=(TRK,(1,1))
//SYSIN DD *
      SORT FIELDS=(73,8,CH,A),DYNALLOC
      OMIT COND=(5,1,CH,NE,C'D')
      OUTREC FIELDS=(1,4,73,8)
      SUM FIELDS=NONE
/*
```

Note: The DFSMSrmm CDS backup copy must be created using EDGHSKP,PARM='BACKUP(AMS)'.
In case of conversion, you can use the file used to load the DFSMSrmm control data set.
When the DFSMSrmm journal file is disabled you must remove the JOURNAL DD statement.

Figure 9-6 shows the result of your previous used SORT. You can see that you have an additional create system ID called TEST, and you have data set records without the system information.

```
....
SYSA
SYSB
SYSC
SYSD
SYSG
SYSH
TEST
```

Figure 9-6 List of all used system IDs in the RMMplex environment

Sometimes you see a blank or hex blank lines in the SORTOUT data set. That means that you have data sets without a create system ID information.

Recommendation: The create system ID field specifies the SMF system ID of the system where the data set was created. When you run DFSMSrmm with catalogs that are not all fully shared, DFSMSrmm uses the create system ID of the first file on a volume to determine the system where the volume should return to scratch.

Refer to 9.3, “Synchronizing with a shared catalog environment” on page 343, or to 9.4, “Synchronizing when catalogs are not shared” on page 350, for more information about how you can correct this error.

9.2.5 Disabling catalog synchronization

Use CATSYNCH(NO) to clear the last synchronization date and time stored in the DFSMSrmm control record. After conversion, the catalog synchronization is normally not switched on. Use the JCL shown in Example 9-6 to disable catalog synchronization.

Example 9-6 Sample JCL to disable catalog synchronization

```
//CATSYNCO JOB (DE0A047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//UTIL      EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD SYSOUT=*
//MASTER   DD DISP=SHR,DSN=RMM.CONTROL.DSET
//SYSIN     DD *
           CONTROL CDSID(PROD) CATSYNCH(NO)
/*
```

You can verify that the catalog synchronization is disabled by using the TSO RMM LISTCONTROL subcommand shown in Figure 9-7.

TSO RMM LISTCONTROL

Figure 9-7 RMM LISTCONTROL CNTL subcommand to check catsync status

Example 9-7 shows the result of the TSO RMM LISTCONTROL subcommand. The last catalog synchronization date and time fields should be blank since the function is disabled.

Example 9-7 Output of the TSO RMM LISTCONTROL subcommand

```
Control record:
Type = MASTER      Create date = 2009/309      Create time = 18:38:11
                  Update date = 2010/282      Update time = 14:30:55
Journal: Utilization = 0% (75% threshold) STATUS: = DISABLED
CDS:      Utilization = 87%
Exit status:
  EDG_EXIT100 = NONE
  EDG_EXIT200 = NONE
  EDG_EXIT300 = ENABLED
Options:
  Stacked Volumes      = NONE
  Extended Bin         = DISABLED
  Common Time          = DISABLED
  CDSID ENQ name       = ENABLED
Last backup:
  Date = 2010/282      Time = 14:29:58
Last journal backup:
  Date = 2010/282      Time = 14:29:58
Last report extract:
  Date = 2010/280      Time = 16:33:30
Last scratch procedure:
  Date =               Time =
Rack numbers          = 6288
LOCAL store bins      = 0
DISTANT store bins    = 0
REMOTE store bins     = 0
Control functions in progress:
Backup                = N  Restore                = N
Verify               = N  Expiration              = N
Report Extract       = N  Disaster Store          = N
VRS                  = N  Synchronize             = N
Last expiration processing:
  Date = 2010/280      Time = 16:33:30
Last store update:
  Date =               Time =
Last VRS processing:
  Date = 2010/280      Time = 16:33:30
Last Catalog synchronize:
Date =               Time =
Empty racks           = 6263
Empty LOCAL bins      = 0
Empty DISTANT bins    = 0
Empty REMOTE bins     = 0
```


Client/Server:
host name =
IP address =

If you have an environment with a shared catalog environment, you can continue with the following section, “Synchronizing with a shared catalog environment”. If you have a environment where not all the catalogs are shared, continue with 9.4, “Synchronizing when catalogs are not shared” on page 350.

9.3 Synchronizing with a shared catalog environment

This section explains how you can implement catalog synchronization in an environment with all catalogs shared as shown in Figure 9-8. You can run the jobs on any of the systems sharing the catalog, but you must update the EDGRMMxx PARMLIB member on each system sharing the catalog. In our example, EDGRMMxx is updated on both SYSE and SYSF.

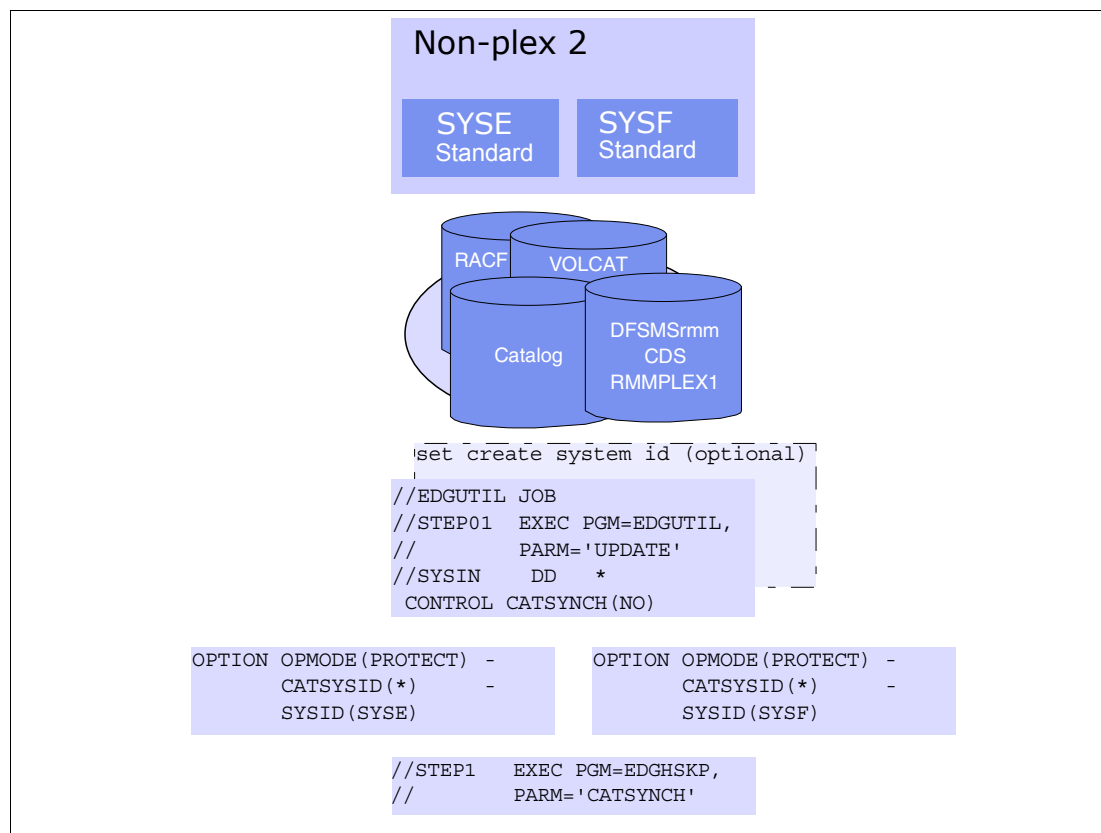


Figure 9-8 Flow to synchronize with a shared catalog environment

You can use shared user catalogs regardless of whether the CDS and user catalogs are synchronized. Synchronize the DFSMSrmm CDS and user catalogs so that DFSMSrmm does not need to locate catalog information for every data set that is managed using catalog control.

Follow these steps to synchronize your catalogs in a fully shared catalog environment:

1. Correct missing create system ID information (optional).

2. Allocate a larger message data set see 9.2.1, “Message data set considerations” on page 337 for more information.
3. Allocate a larger activity log data set as described in “Activity log data set considerations (optional)” on page 337.
4. Run the EDGUTIL UPDATE with the SYSIN command to clear catalog synchronization using the job control shown in Figure 9-9

```
//CATSYNCO JOB (DEOA047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//UTIL      EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT  DD  SYSOUT=*
//MASTER   DD  DISP=SHR,DSN=RMM.CONTROL.DSET
//SYSIN     DD  *
           CONTROL CDSID(PROD) CATSYNCH(NO)
/*
```

Figure 9-9 EDGUTIL to clear catalog synchronization

5. Specify the DFSMSrmm EDGRMMxx PARMLIB OPTION on all systems as shown in Figure 9-10

OPTION	OPMODE(R)	/* Record-Only Mode	*/-
	ACCOUNTING(J)	/* Accounting from JOB	*/-
	BACKUPPROC(EDGBKUP)	/* Name of BACKUP-proc	*/-
	BLP(RMM)	/* DFSMSrmm controls BLP	*/-
	CATRETPD(0012)	/* catalog retention	*/-
	CATSYSID(*)	/* all catalogs shared	*/-
	CDSID(PROD)	/* control data set id	*/-
...			

Figure 9-10 EDGRMMnn parmlib member include CATSYSID command

6. Restart DFSMSrmm using the MVS MODIFY command as you can see in Figure 9-11

```
MODIFY DFRMM,M=01
or
F DFRMM,M=01
```

Figure 9-11 Restatrt DFRMM

7. Run the DFSMSrmm EDGHSKP utility using the JCL shown in Figure 9-10.

```
//LCLHSKP JOB ,140.SCHLUMBERGER,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//STEP01   EXEC PGM=EDGHSKP,PARM='CATSYNCH'
//MESSAGE  DD  DSN=RMM.HSKP.MESSAGE.CATSYNCH,DISP=SHR
//ACTIVITY DD  DSN=RMM.HSKP.ACTIVITY.CATSYNCH,DISP=SHR
```

Figure 9-12 EDGRMMnn parmlib member include CATSYSID command

9.3.1 Setting a create system ID for all data sets have not one

If you have data sets without the create system ID information, and all user catalogs are shared, you can use the JCL shown in Example 9-8 to list all data sets without create system ID information and correct these errors. In this case, you can use the same create system ID for all data sets. We used the OUTREC function to create TSO RMM CHANGEDATASET subcommands directly. In the third step, we run the created commands to correct the errors.

Example 9-8 Sample JCL to list all data sets without create system IDs info

```
//CRSYSID JOB (DE0A047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//DELETE   EXEC PGM=IDCAMS
//SYSPRINT DD DUMMY
//SYSIN    DD *
    DEL RMM.HSKP.WITHOUT.CATSYSID NONVSAM PURGE
    SET MAXCC=0
/*
/* *****
//STEP02   EXEC PGM=EDGHSKP,PARM=(' ,RPTEXT,DATEFORM(J) ')
//SYSPRINT DD SYSOUT=*
//MESSAGE  DD DSN=RMM.HSKP.MESSAGE.DSET,DISP=SHR
//XREPTX   DD DSN=RMM.HSKP.EXTRACTX,DISP=SHR
/* *****
//SORT1    EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//SORTIN   DD DISP=SHR,DSN=RMM.HSKP.EXTRACTX
//SORTOUT  DD DISP=(,PASS,DELETE),
//          DSN=&&SORT1,
//          DCB=*.SORTIN,
//          SPACE=(CYL,(50,50))
//SYSIN    DD *
    SORT FIELDS=(809,44,CH,A,915,4,CH,A),DYNALLOC
    INCLUDE COND=(809,44,CH,NE,C' ',AND,5,1,CH,EQ,C'X')
    OPTION VLSHRT
/*
/* *****
//SORT2    EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//SORTIN   DD DISP=(OLD,DELETE),DSN=&&SORT1
//SORTOUT  DD DISP=(,CATLG,DELETE),
//          DCB=(LRECL=105,BLKSIZE=0,RECFM=VB),
//          DSN=RMM.HSKP.WITHOUT.CATSYSID,
//          SPACE=(TRK,(1,1))
//SYSIN    DD *
    SORT FIELDS=(809,44,CH,A,915,4,CH,A),DYNALLOC
    INCLUDE COND=((809,44,CH,NE,C' '),AND,
                  869,8,CH,EQ,C' ',OR,
                  869,8,CH,EQ,X'0000000000000000')
    OUTREC FIELDS=(1,4,C'          RMM CD ',809,44,
                  C' VOL(' ,9,6,C') SYSID(SYSE) SEQ(' ,
                  915,4,C') FORCE')
    OPTION VLSHRT
/*
/* *****
//CORREC   EXEC PGM=IKJEFT01
```

```
//SYSOUT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYTSIN DD DSN=RMM.HSKP.WITHOUT.CATSYSID,DISP=SHR
```

Note: You only need to verify that the first data set on each volume has a create system ID, because that is the only data set that RMM EXPROC uses. This can result in a much shorter list of data sets to check and commands to run.

9.3.2 Clearing the catalog synchronization

Run the EDGUTIL UPDATE with the SYSIN command CONTROL CDSID(PROD) CATSYNCH(NO) as shown in Example 9-9 to clear the last synchronization date and time.

Note: You can run the EDGUTIL at any time to clear the synchronization date and time whether this information was previously set.

Example 9-9 Sample JCL to clear catalog synchronization

```
//LCLUTILC JOB ,140.SCHLUMBERGER,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//STEP01 EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
          CONTROL CDSID(PROD) CATSYNCH(NO)
/*
```

9.3.3 Updating the EDGRMMxx DFSMSrmm options on all systems

Update the EDGRMMxx PARMLIB member for DFSMSrmm (Example 9-10 on page 347 and Example 9-11 on page 347) to include the CATSYSID operand.

CATSYSID

Specify CATSYSID to enable DFSMSrmm catalog synchronization. Use the CATSYSID operand to identify the user catalogs you want tracked when you run the DFSMSrmm EDGHSKP utility with the CATSYNCH operand to exploit catalog status tracking. All the systems that you identify must have the code that supports catalog synchronization.

CATSYSID(*) means that all catalogs are fully shared. You must specify an asterisk (*) to specify that catalogs are fully shared so that any data set can be processed by DFSMSrmm on any DFSMSrmm subsystem.

CATSYSID(sys_ID_list) provides a list of DFSMSrmm system IDs that share tape data set user catalogs with this DFSMSrmm subsystem. You can identify up to 16 system IDs for DFSMSrmm subsystems. You must specify a list of system IDs when user catalogs are not shared. You must also ensure they are synchronized with the DFSMSrmm CDS before you run inventory management vital record processing. Include IDs for systems that you no longer use but for which you are still retaining tape data sets.

The default is None.

Update the EDGRMM PARMLIB member on system SYSE.

Example 9-10 *EDGRMMxx parmlib member for system SYSE*

OPTION OPMODE(P)	/* operating mode	*/ -
....		-
CATSYSID(*)	/* shared catalogs	*/ -
SYSID(SYSE)	/* system ID	*/ -
....		

Update the EDGRMM PARMLIB member on system SYSE.

Example 9-11 *EDGRMMxx parmlib member for system SYSE*

OPTION OPMODE(P)	/* operating mode	*/ -
....		-
CATSYSID(*)	/* shared catalogs	*/ -
SYSID(SYSE)	/* system ID	*/ -
....		

When the CATSYSID operand is specified in the EDGRMMxx PARMLIB member, restart DFRMM on system SYSE and SYSE to activate this new definition using the command shown in Figure 9-13.

```
MODIFY DFRMM,M=00
or
F DFRMM,M=00
```

Figure 9-13 Modify DFSMSrmm

After you enter this command, you should see the message shown in Figure 9-14.

```
EDG0105I DFSMSrmm SUBSYSTEM INITIALIZATION COMPLETE
```

Figure 9-14 EDG0105I initialization message

9.3.4 Synchronizing your catalogs

Run the DFSMSrmm EDGHSKP utility with the PARM=CATSYNCH parameter as shown in Example 9-12. Do this once on each plex to synchronize all data set records in the CDS with the status from the catalog. Before you run CATSYNCH, allocate larger MESSAGE (see Example 9-1 on page 337) and ACTIVITY (see Example 9-2 on page 338) data sets that are large enough to contain all the messages issued during the catalog synchronization process.

Note: The journal data set should be also large or disabled at this time.

For example, consider when you run CATSYNCH for the first time, perhaps as part of DFSMSrmm implementation, or you run CATSYNCH with VERIFY. In this case, expect messages for each data set that must be updated. Otherwise, the status is different between the catalogs and the DFSMSrmm CDS.

Example 9-12 Sample JCL to synchronize your shared catalogs

```
//LCLHSKP JOB ,140.SCHLUMBERGER,NOTIFY=SCHLUM,
//      MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//STEP01 EXEC PGM=EDGHSKP,PARM='CATSYNCH'
```

```
//MESSAGE DD DSN=RMM.HSKP.MESSAGE.CATSYNC,DISP=SHR
//ACTIVITY DD DSN=RMM.HSKP.ACTIVITY.CATSYNC,DISP=SHR
```

Note: You can run the job on any of the systems SYSE or SYSF.

9.3.5 Checking catalog synchronization

Display DFSMSrmm control information as shown in Figure 9-15 to check if the DFSMSrmm control data set and system catalogs are synchronized.

```
TSO RMM LISTCONTROL
```

Figure 9-15 Check catalog synchronization setting

Example 9-13 shows the output from the RMM LISTCONTROL command.

Example 9-13 Sample LISTCONTROL subcommand output

```
Control record:
Type = MASTER      Create date = 2009/309      Create time = 18:38:11
                  Update date = 2010/282      Update time = 14:30:55
Journal: Utilization = 0% (75% threshold)      STATUS: = DISABLED
CDS:      Utilization = 87%
Exit status:
  EDG_EXIT100 = NONE
  EDG_EXIT200 = NONE
  EDG_EXIT300 = ENABLED
Options:
  Stacked Volumes      = NONE
  Extended Bin          = DISABLED
  Common Time           = DISABLED
  CDSID ENQ name        = ENABLED
Last backup:
  Date = 2010/282      Time = 14:29:58
Last journal backup:
  Date = 2010/282      Time = 14:29:58
Last report extract:
  Date = 2010/280      Time = 16:33:30
Last scratch procedure:
  Date =                Time =
Rack numbers          = 6288
LOCAL store bins      = 0
DISTANT store bins    = 0
REMOTE store bins     = 0
Control functions in progress:
Backup                = N  Restore                = N
Verify                = N  Expiration              = N
Report Extract        = N  Disaster Store          = N
VRS                   = N  Synchronize             = N
Client/Server:
  host name           =
IP address            =
Last expiration processing:
  Date = 2010/280      Time = 16:33:30
Last store update:
  Date =                Time =
Last VRS processing:
  Date = 2010/280      Time = 16:33:30
Last Catalog synchronize:
Date = 2010/280      Time = 16:58:37
Empty racks           = 6263
Empty LOCAL bins      = 0
Empty DISTANT bins    = 0
Empty REMOTE bins     = 0
```

Note: When running EDGHSKP expiration processing or vital record processing, CATSYSID(*) may have been specified in the EDGRMMxx PARMLIB member and the CDS may not be synchronized with the catalogs. In this case, DFSMSrmm initiates catalog synchronization even if you have not specified it.

Look at the message data set. You can see all data sets that are updated in the DFSMSrmm CDS by this run. Example 9-14 shows an example.

Example 9-14 Message data set after catalog synchronization

```

EDG6001I INVENTORY MANAGEMENT STARTING ON 2010/284 AT 14:04:47 -
        PARAMETERS IN USE ARE
        DATEFORM(J),CATSYNCH,DATE(2010/284)
EDG2309I THE PARMLIB OPTIONS CURRENTLY IN USE ARE
        VRSEL(NEW)
        VRSJOBNAME(2)
        VRSMIN(1,INFO)
        VRSCHANGE(INFO)
        VRSDROP(PERCENT(10),INFO) VRSRETAIN(PERCENT(80),INFO)
        EXPDTRDROP(PERCENT(10),INFO)
        SMSTAPE(PURGE(ASIS) UPDATE(EXITS,SCRATCH,COMMAND))
        CATRETPD(6)
        UNCATALOG(Y)
        TPRACF(N)
        NOTIFY(N)
        SYSID(SC70)
        CATSYSID(*)
        RETAINBY(SET)
        MOVEBY(SET)
        GDG(CYCLEBY(GENERATION),DUPLICATE(BUMP))
EDG2234I DFSMSrmm CDS CATALOG STATUS UNKNOWN FOR SCHLUM.RMMDEMO.FILE4.VOL23
        VOLUM.RMMDEMO.FILE4.VOL23 VOLUME SC0010 FILE 3 SYNCHRONIZED TO
        CATALOG STATUS
EDG2234I CONT:- YES
.....
EDG2234I DFSMSrmm CDS CATALOG STATUS YES FOR WWZ038.GSD1997A.DUMP VOLUME
        SC0005 FILE 1 SYNCHRONIZED TO CATALOG STATUS NO
EDG2234I DFSMSrmm CDS CATALOG STATUS YES FOR WWZ038.GSD1997A.DUMP VOLUME
        SC0011 FILE 1 SYNCHRONIZED TO CATALOG STATUS NO
.....
EDG2234I DFSMSrmm CDS CATALOG STATUS NO FOR WWZ038.SAVE.ZOS120.DU01A426
        VOLUME Q17062 FILE 1 SYNCHRONIZED TO CATALOG STATUS YES
EDG2234I DFSMSrmm CDS CATALOG STATUS NO FOR WWZ038.SAVE.ZOS120.DU01A483
        VOLUME Q17030 FILE 1 SYNCHRONIZED TO CATALOG STATUS YES
EDG2307I INVENTORY MANAGEMENT TASK CATSYNCH COMPLETED SUCCESSFULLY
EDG6901I UTILITY EDGHSKP COMPLETED WITH RETURN CODE 0

```

9.3.6 Enabling the journal data set (optional)

When the EDGHSKP run is complete, activate the journal data set again. To activate the journal data set, follow these steps:

1. Restart DFSMSrmm with the EDGRMMxx PARMLIB member, by using the command shown in Figure 9-16

F DFRMM,M=xx

Figure 9-16 Restart DFSMSrmm subsystem

2. Back up the DFSMSrmm CDS and the journal data set, with the EDGHSKP PARM='BACKUP' utility.
DFSMSrmm journal data set is now active, nullified, and synchronized with the DFSMSrmm CDS after the backup. You should see the messages shown in Figure 9-17

```
EDG2121I DFSMSrmm JOURNAL FILE IS NOW CLEARED  
EDG2121I DFSMSrmm JOURNAL FILE IS NOW ENABLED  
EDG6901I UTILITY EDGHSKP COMPLETED WITH RETURN CODE 0
```

Figure 9-17 DFSMSrmm journal enable messages

Note: If the journal was previously used, you must delete it and allocate a new one, because the CDS and JOURNAL control information no longer match.

9.3.7 Catalog re-synchronization with a shared catalog environment

To request catalog re-synchronization repeat the steps between 9.2.5, “Disabling catalog synchronization” on page 342 and 9.3.6, “Enabling the journal data set (optional)” on page 349.

Note: Rerun catalog synchronization if you have manually added volumes in MASTER or USER status with data set information, or you are changing your catalog environment (MERGECAT, IMPORT, EXPORT).

9.4 Synchronizing when catalogs are not shared

This section explains how you can implement catalog synchronization in an environment when catalogs are not shared as shown in Figure 9-18 on page 351. You must run the synchronization jobs on each PLEX once, but the update of the DFSMSrmm control record can run on any system. Also you must update the EDGRMMxx PARMLIB member on each system sharing the catalog. In our example, EDGRMMxx is updated

- ▶ on both SYSA and SYSB with CATSYSID(SYSA,SYSB)
- ▶ on both SYSG and SYSH with CATSYSID(SYSG,SYSH)
- ▶ on both SYSC and SYSD with CATSYSID(SYSC,SYSD).

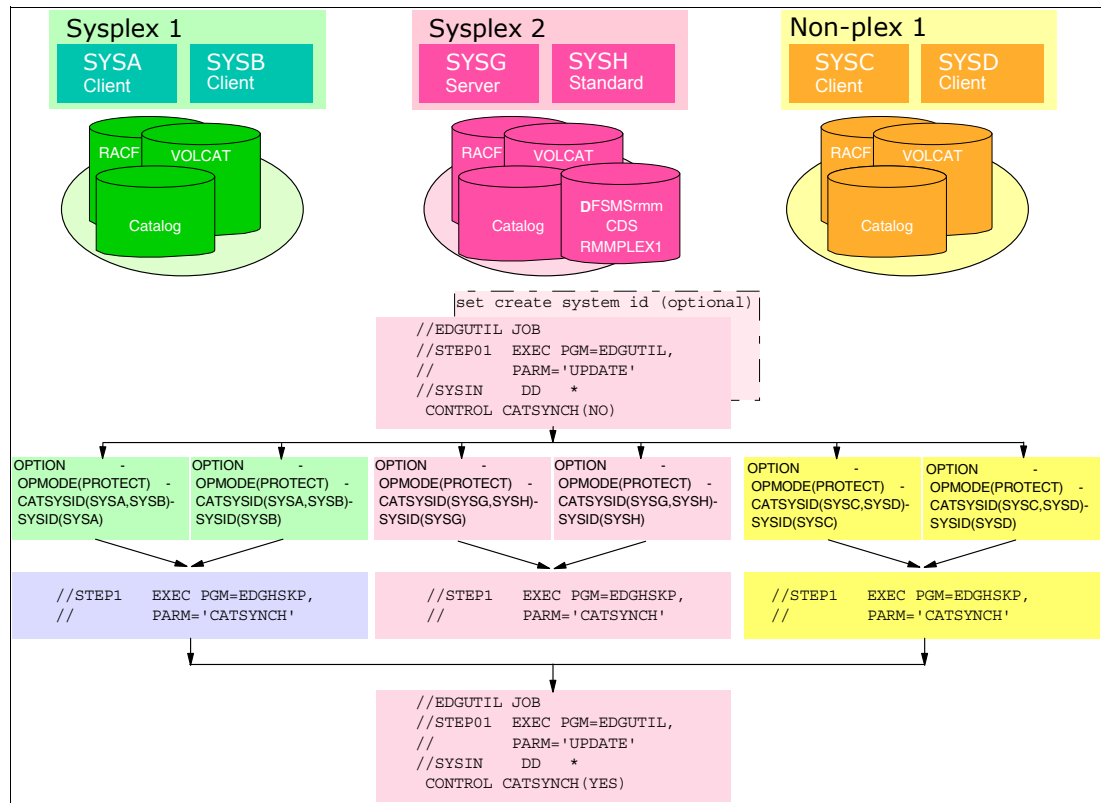


Figure 9-18 Job flow to synchronize when catalogs are not shared

When catalogs are not fully shared, you can have non-GDG and GDG data sets, with the same name, that are cataloged but in different, unshared catalogs. To retain all copies of all data sets, define a retention policy that uses DAYS with COUNT(99999) rather than CYCLES. Using CYCLES can produce unpredictable results for example, the premature expiration of data sets.

To ensure cycles are grouped together, use the job name to further qualify the data sets, using a different job name for each set of cataloged data sets. For example, use JOBNAME(xjzA) for data sets created on system SYSA, and JOBNAME(xjzB) for data sets created on system SYSH. This enables the cycle retention to be based on the scope of the creation system.

Follow these tasks, which are also shown in Figure 9-18, to synchronize your catalogs when your catalog environment is not shared:

1. Correct missing create system ID information (optional).
2. Allocate a larger message data set and a larger activity log data set.
3. Run the EDGUTIL UPDATE with the SYSIN command to clear catalog synchronization using the job control shown in Figure 9-19 on page 352

```
//CATSYNCO JOB (DEOA047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//UTIL     EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD  SYSOUT=*
//MASTER  DD  DISP=SHR,DSN=RMM.CONTROL.DSET
//SYSIN    DD  *
CONTROL CDSID(PROD) CATSYNCH(NO)
/*
```

Figure 9-19 EDGUTIL to clear catalog synchronization

4. Specify the DFSMSrmm EDGRMMxx PARMLIB OPTION on all systems as shown in Figure 9-20.

OPTION	OPMODE(R)	/* Record-Only Mode	*/-
	ACCOUNTING(J)	/* Accounting from JOB	*/-
	BACKUPPROC(EDGBKUP)	/* Name of BACKUP-proc	*/-
	BLP(RMM)	/* DFSMSrmm controls BLP	*/-
	CATRETPD(0012)	/* catalog retention	*/-
	CATSYSID(SYSA,SYSB)	/* catalogs not shared	*/-
	CDSID(PROD)	/* control data set id	*/-
...			

Figure 9-20 EDGRMMnn parmlib member include CATSYSID command

5. Restart DFSMSrmm using the MVS MODIFY command as you can see in Figure 9-21.

```
MODIFY DFRMM,M=01
or
F DFRMM,M=01
```

Figure 9-21 Restatrt DFRMM

6. Run the DFSMSrmm EDGHSKP utility using the JCL shown in Figure 9-22.

```
//LCLHSKP JOB ,140.SCHLUMBERGER,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//STEP01   EXEC PGM=EDGHSKP,PARM='CATSYNCH'
//MESSAGE  DD  DSN=RMM.HSKP.MESSAGE.CATSYNCH,DISP=SHR
//ACTIVITY DD  DSN=RMM.HSKP.ACTIVITY.CATSYNCH,DISP=SHR
```

Figure 9-22 EDGRMMnn parmlib member include CATSYSID command

7. Run the EDGUTIL UPDATE using the SYSIN command as shown in Figure 9-23 on page 353.

```

//CATSYNCO JOB (DE0A047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//UTIL      EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD  SYSOUT=*
//MASTER   DD  DISP=SHR,DSN=RMM.CONTROL.DSET
//SYSIN     DD  *
          CONTROL CDSID(PROD) CATSYNCH(YES)
/*

```

Figure 9-23 EDGUTIL to set catalog synchronization

9.4.1 Setting the create system ID if catalogs are not shared

You can use the information in this section if you have either a normal DFSMSrmm environment when catalogs are not fully shared or you have implemented an RMMplex with catalogs are not fully shared. Example 9-15 shows sample JCL can be used to set a create systemID for all data sets on volumes 5* and 9* without have a create system ID set if you have catalogs that are not fully shared. It also shows how to determine all data sets without create system ID information residing in different volume pools.

TSO RMM CHANGEDATASET subcommands are created with different create system IDs to correct the errors. All volumes with a volume prefix of 5 and 9 are used on SYSA and all other volumes are used on SYSG.

Example 9-15 Sample JCL to determine all data sets without create system ID details

```

//CRSYSID JOB (DE0A047),SCHLUMBERGER,MSGLEVEL=1,MSGCLASS=H,
//          REGION=4M,NOTIFY=&SYSUID
//DELETE    EXEC PGM=IDCAMS
//SYSPRINT DD  DUMMY
//SYSIN     DD  *
          DEL RMM.HSKP.WITHOUT.CATSYSID.SYSA NONVSAM PURGE
          DEL RMM.HSKP.WITHOUT.CATSYSID.SYSG NONVSAM PURGE
          SET MAXCC=0
/*
/* *****
//STEP02    EXEC PGM=EDGHSKP,PARM=(' ,RPTEXT,DATEFORM(J) ')
//SYSPRINT DD  SYSOUT=*
//MESSAGE DD  DSN=RMM.HSKP.MESSAGE.DSET,DISP=SHR
//XREPTX DD  DSN=RMM.HSKP.EXTRACTX,DISP=SHR
/* *****
//SORT1     EXEC PGM=SORT
//SYSOUT DD  SYSOUT=*
//SORTIN DD  DISP=SHR,DSN=RMM.HSKP.EXTRACTX
//SORTOUT DD  DISP=(,PASS,DELETE),
//          DSN=&&SORT1,
//          DCB=*.SORTIN,
//          SPACE=(CYL,(50,50))
//SYSIN     DD  *
          SORT FIELDS=(809,44,CH,A,915,4,CH,A),DYNALLOC
          INCLUDE COND=(809,44,CH,NE,C' ')
          OPTION VLSHRT
/*
/* *****
//SORT2     EXEC PGM=SORT

```

```

//SYSOUT DD SYSOUT=*
//SORTIN DD DISP=(OLD,PASS),DSN=&&SORT1
//SORTOUT DD DISP=(,CATLG,DELETE),
//          DCB=*.SORTIN,
//          DSN=RMM.HSKP.WITHOUT.CATSYSID.SYSA,
//          SPACE=(TRK,(10,1))
/*
/* * include all volumes starting with 5 and 9 *
/*
//SYSIN DD *
        SORT FIELDS=(809,44,CH,A,915,4,CH,A),DYNALLOC
        INCLUDE COND=((9,1,CH,EQ,C'5',OR,9,1,CH,EQ,C'9'),AND,
                      (869,8,CH,EQ,C'          ',OR,
                      869,8,CH,EQ,X'0000000000000000'))

        OPTION VLSHRT
        OUTREC FIELDS=(1,4,C'          RMM CD ',809,44,
                      C' VOL(',9,6,C') SYSID(SYSA) SEQ(',
                      915,4,C') FORCE')

/*
/* *****
//SORT3 EXEC PGM=SORT
//SYSOUT DD SYSOUT=*
//SORTIN DD DISP=(OLD,DELETE),DSN=&&SORT1
//SORTOUT DD DISP=(,CATLG,DELETE),
//          DCB=*.SORTIN,
//          DSN=RMM.HSKP.WITHOUT.CATSYSID.SYSG,
//          SPACE=(TRK,(1,1))
/*
/* * exclude all volumes starting with 5 and 9 *
/*
//SYSIN DD *
        SORT FIELDS=(809,44,CH,A,915,4,CH,A),DYNALLOC
        INCLUDE COND=((9,1,CH,NE,C'5',AND,9,1,CH,NE,C'9'),AND,
                      (869,8,CH,EQ,C'          ',OR,
                      869,8,CH,EQ,X'0000000000000000'))

        OPTION VLSHRT
        OUTREC FIELDS=(1,4,C'          RMM CD ',809,44,
                      C' VOL(',9,6,C') SYSID(SYSG) SEQ(',
                      915,4,C') FORCE')

/*
/* *****
//CORREC EXEC PGM=IKJEFT01
//SYSOUT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYTSIN DD DSN=RMM.HSKP.WITHOUT.CATSYSID.SYSA,DISP=SHR
//          DD DSN=RMM.HSKP.WITHOUT.CATSYSID.SYSG,DISP=SHR

```

Figure 9-24 shows the created TSO RMM CHANGEDATASET subcommands to correct all data set records that do not have the creating-system information available.

RMM CD TAPEA.SAVE.DUMP.D000630 VOL(901254) SYSID(SYSA) SEQ(1) FORCE RMM CD SYSG.SYSOUT.SAVE.D2004 VOL(A05372) SYSID(SYSG) SEQ(1) FORCE

Figure 9-24 TSO RMM subcommands to update data set records

Each system ID used in your environment, as shown in Figure 9-8 on page 343, should be defined in one of your EDGRMMnn PARMLIB members. A volume with data sets on it with a create system ID not defined in any of the CATSYSID operands in the EDGRMMnn PARMLIB members never return to status *scratch*. The expiration processing checks that the create system of the volume matches one of the system IDs defined in the CATSYSID PARMLIB option.

You can use the same procedure to set a system ID for all data sets that have no system ID set to change the create system ID from an old system ID to a current one. For example, you can change all data sets that have a create system ID of TEST to a create system ID of SYSH. To do this, change the statement as shown in Example 9-16.

Example 9-16 Sort statements to change the create system ID

```
//SYSIN DD *
  SORT FIELDS=(809,44,CH,A,915,4,CH,A),DYNALLOC
  INCLUDE COND=((9,1,CH,EQ,C'5',OR,9,1,CH,EQ,C'9'),AND,
                (869,8,CH,EQ,C'TEST'))
  OPTION VLSHRT
  OUTREC FIELDS=(1,4,C' RMM CD ',809,44,
                  C' VOL(',9,6,C') SYSID(SYSH) SEQ(',
                  915,4,C') FORCE')
/*
```

Important: If you have data set records without creating system information or with a create system information not defined in any of the parmlib members, this volumes will never return to scratch.

9.4.2 Clearing the catalog synchronization

Run the EDGUTIL UPDATE with the SYSIN CONTROL CATSYNCH(NO) command as shown in Example 9-17 to clear the last synchronization date and time. You can run this job on any system in the sysplex. In our case, we ran this job on one of the systems in SYSPLEX2 because these systems have direct access to the DFSMSrmm control data set.

Note: You can run the EDGUTIL at any time to clear the synchronization date and time whether this information was previously set.

Example 9-17 Sample JCL to clear catalog synchronization

```
//LCLUTILC JOB ,140.SCHLUMBERGER,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//STEP01 EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
  CONTROL CDSID(PROD) CATSYNCH(NO)
/*
```

9.4.3 Updating the EDGRMMxx DFSMSrmm options

To ensure that DFSMSrmm processes the return to scratch, you must specify the EDGRMMxx PARMLIB OPTION CATSYSID operand with the system IDs on which the volumes were created. DFSMSrmm uses the CATSYSID operand to determine the systems to which volumes can be returned to scratch status.

DFSMSrmm checks the system creation ID for the first file on the volume with the list of IDs that are specified for the CATSYSID operand. If there is a match, DFSMSrmm processes the return to scratch. Update the EDGRMMxx PARMLIB member as shown in Example 9-18 to Example 9-22 to include the CATSYSID operand on each system sharing this DFSMSrmm control data set.

CATSYSID

Specify CATSYSID to enable DFSMSrmm catalog synchronization. Use the CATSYSID operand to identify the user catalogs you want tracked when you run the DFSMSrmm EDGHSKP utility with the CATSYNCH operand to exploit catalog status tracking. All the systems that you identify must have the code that supports catalog synchronization.

CATSYSID(*) means that all catalogs are fully shared. You must specify an asterisk (*) to specify that catalogs are fully shared so that any data set can be processed by DFSMSrmm on any DFSMSrmm subsystem.

CATSYSID(sys_ID_list) provides a list of DFSMSrmm system IDs that share tape data set user catalogs with this DFSMSrmm subsystem. You can identify up to 16 system IDs for DFSMSrmm subsystems. Specify a list of system IDs when user catalogs are not shared. Ensure that they are synchronized with the DFSMSrmm CDS before you run inventory management vital record processing. Include IDs for systems that you no longer use but for which you still retain tape data sets.

The default is None.

Example 9-18 through Example 9-22 show the correct update of the CATSYSID and optionally the SYSID of each of the six systems that share this DFSMSrmm control data set.

Update the EDGRMM PARMLIB member on system SYSA.

Example 9-18 EDGRMMxx PARMLIB member for system SYSA

OPTION	OPMODE(P)	/* operating mode	*/ -
....			-
	CDSID(RMMPLEXP)	/* DFSMSrmm CDS ID (PROD)	*/ -
	CATSYSID(SYSA,SYSB)	/* unshared catalogs	*/ -
	SYSID(SYSA)	/* system ID	*/ -
....			-

Update the EDGRMM PARMLIB member on system SYSB.

Example 9-19 EDGRMMxx PARMLIB member for system SYSB

OPTION	OPMODE(P)	/* operating mode	*/ -
....			-
	CDSID(RMMPLEXP)	/* DFSMSrmm CDS ID (PROD)	*/ -
	CATSYSID(SYSA,SYSB)	/* unshared catalogs	*/ -
	SYSID(SYSA)	/* system ID	*/ -
....			-

Update the EDGRMM PARMLIB member on system SYSC.

Example 9-20 EDGRMMxx PARMLIB member for system SYSC

OPTION OPMODE(P)	/* operating mode	*/ -
....		-
CDSID(RMMPLEXP)	/* DFSMSrmm CDS ID (PROD)	*/ -
CATSYSID(SYSC,SYSD)	/* unshared catalogs	*/ -
SYSID(SYSC)	/* system ID	*/ -
....		-

Update the EDGRMM PARMLIB member on system SYSD.

Example 9-21 EDGRMMxx PARMLIB member for system SYSD

OPTION OPMODE(P)	/* operating mode	*/ -
....		-
CDSID(RMMPLEXP)	/* DFSMSrmm CDS ID (PROD)	*/ -
CATSYSID(SYSC,SYSD)	/* unshared catalogs	*/ -
SYSID(SYSD)	/* system ID	*/ -
....		-

Update the EDGRMM PARMLIB member on system SYSG.

Example 9-22 EDGRMMxx PARMLIB member for system SYSG

OPTION OPMODE(P)	/* operating mode	*/ -
....		-
CDSID(RMMPLEXP)	/* DFSMSrmm CDS ID (PROD)	*/ -
CATSYSID(SYSG,SYSH)	/* unshared catalogs	*/ -
SYSID(SYSG)	/* system ID	*/ -
....		-

Update the EDGRMM PARMLIB member on system SYSH.

When the CATSYSID operand is specified in the EDGRMMxx parmlib member, restart DFRMM on system SYSA and SYSB, SYSC and SYSD, and SYSG and SYSH as shown in Figure 9-25 to activate this new definition.

MODIFY DFRMM,M=00
or
F DFRMM,M=00

Figure 9-25 Restart the DFSMSrmm subsystem

After you enter this command, you should see the message shown in Figure 9-26 on each system.

EDG0105I DFSMSrmm SUBSYSTEM INITIALIZATION COMPLETE

Figure 9-26 DFSMSrmm subsystem completion message

9.4.4 Synchronizing your catalogs

Run the DFSMSrmm EDGHSKP utility with the PARM=CATSYNCH parameter as shown in Example 9-23 on page 358 on each plex (SYSplex1, SYSplex2 and Non-PLEX1). Do this one time to synchronize all data set records in the CDS with the status from the catalog.

Before you run CATSYNCH, allocate larger MESSAGE (see Example 9-1 on page 337) and ACTIVITY (see Example 9-2 on page 338) data sets that are large enough to contain all the messages issued during the catalog synchronization process. Also check the size of your journal or disable it.

The DFSMSrmm subsystem writes a message to each of this data sets for each record that is updated in this run. For example, you may run CATSYNCH for the first time, perhaps as part of DFSMSrmm implementation, or you may run CATSYNCH with VERIFY. In this case, you should expect messages for each data set that must be updated. Otherwise the status will be different between the catalogs and the DFSMSrmm CDS.

Example 9-23 *Sample JCL to synchronize your catalogs are not shared*

```
//LCLHSKP JOB ,140.SCHLUMBERGER,NOTIFY=SCHLUM,
//      MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//STEP01 EXEC PGM=EDGHSKP,PARM='CATSYNCH'
//MESSAGE DD DSN=RMM.HSKP.MESSAGE.CATSYNCH,DISP=SHR
//ACTIVITY DD DSN=RMM.HSKP.ACTIVITY.CATSYNCH,DISP=SHR
```

You have to run the job once on sysplex 1 (systems SYSA or SYSB), once on non-plex 1 (systems SYSC or SYSD), and finally once on sysplex 2 (systems SYSG or SYSH) to synchronize all catalogs currently used in this RMMplex.

Ensure that all jobs run without any errors. Also check your MESSAGE files and ACTIVITY logs for error or warning messages. If you've done these steps, run EDGUTIL with PARM=UPDATE and the sysin operand CONTROL CATSYNCH(YES). This sets the last catalog synchronization date and time in the DFSMSrmm CDS control record as shown in Example 9-24.

Example 9-24 *Sample JCL to set the last catalog synchronization date and time*

```
//LCLUTILC JOB ,140.SCHLUMBERGER,NOTIFY=SCHLUM,
//      MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1)
//STEP01 EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
CONTROL CDSID(PROD) CATSYNCH(YES)
/*
```

Display DFSMSrmm control information as shown in Figure 9-27 to check if the DFSMSrmm control data set and system catalogs are synchronized.

TSO RMM LISTCONTROL

Figure 9-27 Check catalog synchronization setting

Example 9-25 shows the output from the RMM LISTCONTROL command.

Example 9-25 *Sample LISTCONTROL subcommand output*

```
Control record:
Type = MASTER      Create date = 2009/309      Create time = 18:38:11
                  Update date = 2010/282      Update time = 14:30:55
Journal: Utilization = 0% (75% threshold) STATUS: = DISABLED
CDS:      Utilization = 87%
Exit status:                                Options:
```


EDG_EXIT100 = NONE	Stacked Volumes = NONE
EDG_EXIT200 = NONE	Extended Bin = DISABLED
EDG_EXIT300 = ENABLED	Common Time = DISABLED
	CDSID ENQ name = ENABLED
Last backup:	Last expiration processing:
Date = 2010/282 Time = 14:29:58	Date = 2010/280 Time = 16:33:30
Last journal backup:	Last store update:
Date = 2010/282 Time = 14:29:58	Date = Time =
Last report extract:	Last VRS processing:
Date = 2010/280 Time = 16:33:30	Date = 2010/280 Time = 16:33:30
Last scratch procedure:	Last Catalog synchronize:
Date = Time =	 Date = 2010/280 Time = 16:58:37
Rack numbers = 6288	Empty racks = 6263
LOCAL store bins = 0	Empty LOCAL bins = 0
DISTANT store bins = 0	Empty DISTANT bins = 0
REMOTE store bins = 0	Empty REMOTE bins = 0
Control functions in progress:	
Backup = N Restore = N	
Verify = N Expiration = N	
Report Extract = N Disaster Store = N	
VRS = N Synchronize = N	
Client/Server:	
host name =	
IP address =	

Note: When running EDGHSKP expiration processing or vital record processing, CATSYSID(*) may have been specified in the EDGRMMxx PARMLIB member and the CDS may not be synchronized with the catalogs. In this case, DFSMSrmm initiates catalog synchronization even if you have not specified it.

Look at the message data set. You can see all data sets that are updated in the DFSMSrmm CDS by this run. Example 9-26 shows an example.

Example 9-26 Message data set after catalog synchronization

```

EDG6001I INVENTORY MANAGEMENT STARTING ON 2010/284 AT 14:04:47 -
          PARAMETERS IN USE ARE
          DATEFORM(J),CATSYNCH,DATE(2010/284)
EDG2309I THE PARMLIB OPTIONS CURRENTLY IN USE ARE
          VRSEL(NEW)
          VRSJOBNAME(2)
          VRSMIN(1,INFO)
          VRSCHANGE(INFO)
          VRSDROP(PERCENT(10),INFO) VRSRETAIN(PERCENT(80),INFO)
          EXPDTRDOP(PERCENT(10),INFO)
          SMSTAPE(PURGE(ASIS) UPDATE(EXITS,SCRATCH,COMMAND))
          CATRETPD(6)
          UNCATALOG(Y)
          TPRACF(N)
          NOTIFY(N)
          SYSID(SC70)
          CATSYSID(*)
          RETAINBY(SET)
          MOVEBY(SET)
          GDG(CYCLEBY(GENERATION),DUPLICATE(BUMP))

```

```

EDG2234I DFSMSrmm CDS CATALOG STATUS UNKNOWN FOR SCHLUM.RMMDEMO.FILE4.VOL23
        VOLUM.RMMDEMO.FILE4.VOL23 VOLUME SC0010 FILE 3 SYNCHRONIZED TO
        CATALOG STATUS
EDG2234I CONT:- YES
....
EDG2234I DFSMSrmm CDS CATALOG STATUS YES FOR WWZ038.GSD1997A.DUMP VOLUME
        SC0005 FILE 1 SYNCHRONIZED TO CATALOG STATUS NO
EDG2234I DFSMSrmm CDS CATALOG STATUS YES FOR WWZ038.GSD1997A.DUMP VOLUME
        SC0011 FILE 1 SYNCHRONIZED TO CATALOG STATUS NO
....
EDG2234I DFSMSrmm CDS CATALOG STATUS NO FOR WWZ038.SAVE.ZOS120.DU01A426
        VOLUME Q17062 FILE 1 SYNCHRONIZED TO CATALOG STATUS YES
EDG2234I DFSMSrmm CDS CATALOG STATUS NO FOR WWZ038.SAVE.ZOS120.DU01A483
        VOLUME Q17030 FILE 1 SYNCHRONIZED TO CATALOG STATUS YES
EDG2307I INVENTORY MANAGEMENT TASK CATSYNCH COMPLETED SUCCESSFULLY
EDG6901I UTILITY EDGHSKP COMPLETED WITH RETURN CODE 0

```

9.4.5 Enabling the journal data set (optional)

When the EDGHSKP run is complete, activate the journal data set again. To activate the journal data set, follow these steps:

1. Restart DFSMSrmm with the EDGRMMxx PARMLIB member, by using the command shown in Figure 9-28

```
F DFRMM,M=xx
```

Figure 9-28 Restart DFSMSrmm subsystem

2. Back up the DFSMSrmm CDS and the journal data set, with the EDGHSKP PARM='BACKUP' utility. DFSMSrmm journal data set is now active, nullified, and synchronized with the DFSMSrmm CDS after the backup. You should see the messages shown in Figure 9-29

```

EDG2121I DFSMSrmm JOURNAL FILE IS NOW CLEARED
EDG2121I DFSMSrmm JOURNAL FILE IS NOW ENABLED
EDG6901I UTILITY EDGHSKP COMPLETED WITH RETURN CODE 0

```

Figure 9-29 DFSMSrmm journal enable messages

Note: If the journal was previously used, you must delete it and allocate a new one, because the CDS and JOURNAL control information no longer match.

9.4.6 Housekeeping considerations

Figure 9-30 on page 361 shows the correct sequence in which you should run your daily housekeeping jobs so that all components are updated correctly and always are synchronized. The examples show only the most important parameters.

You can run the first and last jobs on any of the systems. However, we recommend that you run these jobs on the server (SYSG) or on the standard system (SYSH) on sysplex 2 that have direct access to the DFSMSrmm control data set. The catalog synchronization, as part

of the expiration processing, must run in each plex separately. This way you have run the EDGHSKP expiration processing on either SYSA or SYSB (sysplex 1), on either system SYSC or SYSD (non-plex 1), and on either system SYSG or SYSH (sysplex 2). The CATSYSID operand in the EDGRMMxx PARMLIB member must be present on each system and must include only system IDs that have access to the available catalogs.

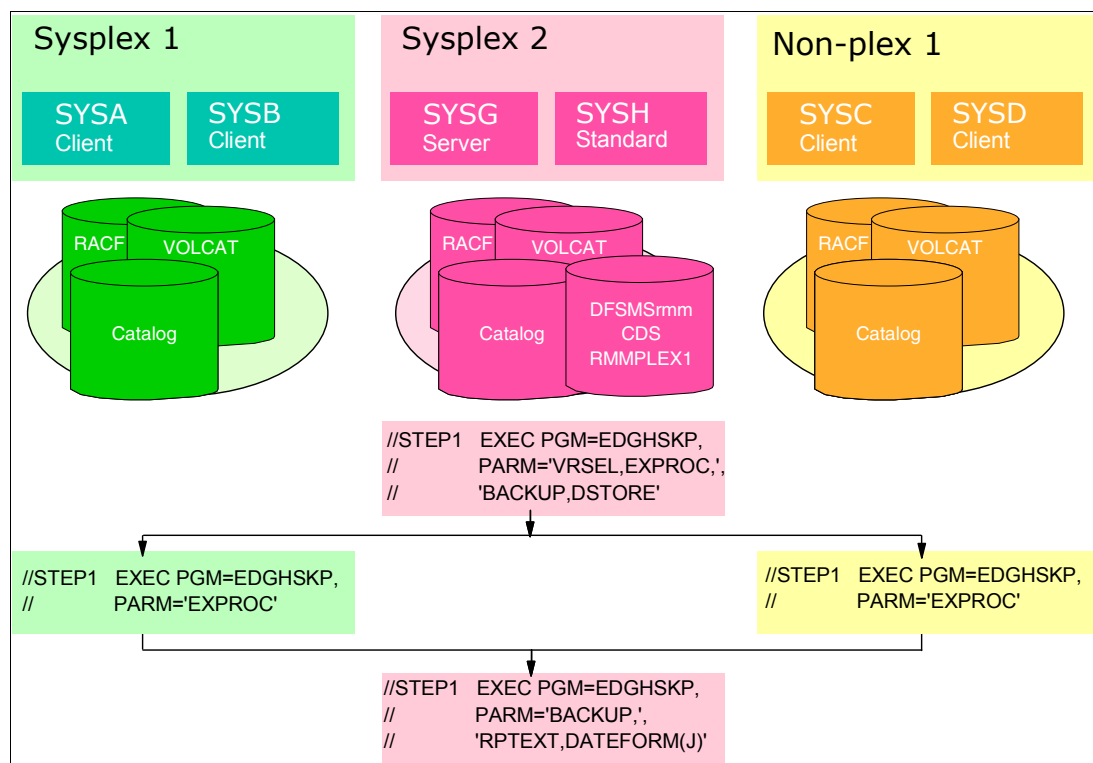


Figure 9-30 DFSMSrmm housekeeping when catalogs are not shared

Run inventory management functions on any one system at a time. However, if you run expiration processing to return volumes to scratch status, DFSMSrmm only processes a subset of volumes.

You must run EDGHSKP expiration processing on multiple systems if you specify the EDGRMMxx PARMLIB OPTION UNCATALOG(Y I S). You should run expiration processing once for each set of user catalogs on a system with access to those user catalogs.

9.4.7 Catalog re-synchronization with a shared catalog environment

To request catalog re-synchronization repeat the steps between 9.4.2, “Clearing the catalog synchronization” on page 355 and 9.4.5, “Enabling the journal data set (optional)” on page 360.

Note: Rerun catalog synchronization if you have manually added volumes in MASTER or USER status with data set information, or you are changing your catalog environment (MERGECAT, IMPORT, EXPORT).

9.5 IPV6 Supports ne

two working strategy

The information in this section is important if you are planning to implementing DFSMSrmm client and server systems or you have implemented it.

DFSMSrmm on z/OS V1R12 and later releases is an IPv6 enabled application that supports both IPv4 and IPv6 sockets. You can continue to use IPv4 on all systems or you can run a mixed environment with one or more V1R12 systems using IPv6 and other systems using IPv4. Once all systems are V1R12, you have the option of moving all systems to IPv6. In a mixed environment, dual-mode IP stacks are required. You can use IP addresses that are compliant to either IPV4 or IPV6.

Note: DFSMSrmm client/server processing is dependent on Internet Protocol V4.

Figure 9-31 shows you an example of a IPV6 address.

1080:0:0:0:8:800:200C:417A

Figure 9-31 IPV6 address



Enabling ISPF Data Set List (DSLIS) Support

To enable direct entry into the DFSMSrmm ISPF dialog from the ISPF Data Set List Utility, use the ISPF Configuration Utility to update the ISPF Configuration Table. To enable this function, select the Enable RM/Tape Commands option. For details of how to use the ISPF Configuration Utility, refer to *z/OS ISPF Planning and Customizing*. Figure 16 on page 50 shows that the data set list support is enabled, as well as showing the default values for the RM/Tape Command EDGRPD34 and Command APPLID EDG. You do not need to change these values.

The following topics are covered:

- ▶ Implementation steps
- ▶ Use the ISPF Configuration Utility
- ▶ Using the ISPF Data Set List Utility Support
- ▶ Move the ISPCFIGU module to the SISPLPA library

10.1 Implementation steps

The ISPF Data Set List support can be implemented for a single user or it can be a installation-wide setting. Following the steps below to implementing ISPF Data Set List support for a single user:

1. Create a new PDS with VB 255 to store the ISPF default settings
2. Under ISPF, call 'TSO ISPCCONF' in any command line
3. Select option 1 *"Create/Modify Settings and Regenerate Keyword File"* in the *ISPF Configuration Utility* primary panel, and added the name to the keyword file dsname field you have allocated before, and added a member name.
You should get message *Keyword file loaded*
4. Select option 3 *"PDF Exits and Other PDF Settings"* in the *ISPF Configuration Utility* primary panel
5. Scroll to the correct place and add '/' to *Enable RM/Tape Commands*
6. Exit the dialog using two times the PF3 key
you should get message *Keyword file saved*
7. Select option 4 *"Build Configuration Table Load Module"*
8. Specify any library in your ISPLLIB concanation as the *"Output Configuration Table Load Module Data Set"*
9. ENTER to load module built
10. Exit ISPF, ensure the specified load library is ISPLLIB concatenated.
11. Start ISPF and it works. (that is just for a single user, not for all users)

To implementing the ISPF Data Set List support for all user you can following the steps as described before, but the configuration table load module should be stored in the SISPLPA library. To load the module to the LPA you must IPL your system. For detailed information defere to 10.4, "Move the ISPCFIGU module to the SISPLPA library" on page 374.

10.2 Use the ISPF Configuration Utility

Use the ISPF Configuration table to change site-wide defaults and to indicate that installation exit routines are provided for some of the ISPF functions. The ISPF functions that allow installation-written exit routines are data set allocation, print utility, data set compression, the data set list utility, member list filter, and data set name change. ISPF checks the configuration table to determine, first, if exit routines are provided, and second, whether those routines are programs or CLISTs. If you specify both a CLIST and a program, ISPF uses the program.

Enter command TSO ISPCCONF to start the ISPF Configuration Utility as shown in Figure 10-1 on page 365. You can chose the command in the ISPF command shell or in any available command line.

Menu Utilities Compilers Options Status Help		
z/OS Primary Option Menu		
Option ==> TSO ISPCCONF		
0	Settings	Terminal and user parameters
1	View	Display source data or listings
2	Edit	Create or change source data
3	Utilities	Perform utility functions
4	Foreground	Interactive language processing
5	Batch	Submit job for language processing
6	Command	Enter TSO or Workstation commands
7	Dialog Test	Perform dialog testing
9	IBM Products	IBM program development products
10	SCLM	SW Configuration Library Manager
11	Workplace	ISPF Object/Action Workplace
12	z/OS System	z/OS system programmer applications
13	z/OS User	z/OS user applications
User ID . : SCHLUM Time. . . : 18:40 Terminal. : 3278 Screen. . : 1 Language. : ENGLISH App1 ID . : PDF TSO logon : IKJACCT TSO prefix: MHLRES5 System ID : SC70 MVS acct. : ACCNT# Release . : ISPF 5.9		
Enter X to Terminate using log/list defaults		

Figure 10-1 Start the ISPF conversion utility

See Figure 10-2 on page 366 how you can select the *Create/Modify Settings and Regenerate Keyword File* after the ISPF conversion utility is up and running.

ISPF Configuration Utility		Enter option
Option ==> 1		
1 Create/Modify Settings and Regenerate Keyword File 2 Edit Keyword File Configuration Table 3 Verify Keyword Table Contents 4 Build Configuration Table Load Module 5 Convert Assembler Configuration Table to Keyword File 6 Build SMP/E USERMOD		
Keyword File Data Set		
Data Set . . .	'RMM.ADDONS.CEEXECVB'	
Member	SETTING	
Configuration Table Assembler Source Data Set		
Data Set . . .		
Member		
Output File Content for Keyword File		
2	1. Include only non-default values	
	2. Include defaults as comments	
	3. Include all values	
Current Configuration Table		
Keyword File :	not available	
Identifier . . :	ISPCFIGU	Level . . . : 480R8001
Compile Date :	2003/04/04	Compile Time :

Figure 10-2 Create/Modify Settings and Regenerate Keyword File selection

After the keyword file is loaded you have to select the *PDF Exits and Other PDF Settings* menu as shown in Figure 10-3 on page 367.

Create/Modify ISPF Configuration		Keyword file loaded
Option ==> 3		
General ISPF Settings		System Profile (ISPSPROF) Settings
1 Editor Settings		6 Log and List Defaults
2 Edit/View/Browse VSAM Settings		7 Terminal and User Defaults
3 PDF Exits and Other PDF Settings		8 Workstation Defaults
4 ISPF Site-wide Defaults		9 Workstation Download Defaults
5 ISPDFTLS, CUA Colors, and Other DM Settings		
Output Keyword File		
Data Set . . . 'RMM.ADDONS.CEEXECVB'		
Member SETTING		
Instructions:		
Enter option to change configuration settings,		
END or EXIT command to generate keyword file, or		
CANCEL command to exit without keyword file generation		

Figure 10-3 PDF Exits and Other PDF Settings selection

The *Modify PDF Configuration Settings* will be displayed in a scrollable panel and you have scroll down until you can see the *DSLST Removable Media Settings* as shown in Figure 10-4 on page 368.

```

                                Modify PDF Configuration Settings
-----
Command ==>

                                More:  -

    2. Use IEBCOPY for PDSEs only

When to use COPY or COPYMOD
2  1. Use COPY if the target block size is equal to or greater than the
    source block size, COPYMOD otherwise
    2. Use COPY if the target block size is equal to the source block size,
    COPYMOD otherwise
    3. Always use COPYMOD

DSLIST Removable Media Settings
Enter "/" to select option
/  Enable RM/Tape Commands

RM/Tape Command  . . %EDGRP34
Command APPLID  . . . EDG

Other PDF Settings
Default PDF Unit . . . . . SYSALLDA
Volume for Migrated Data Sets . . . . . MIGRAT
Delete Command for Migrated Data Sets  HDELETE
Allowed Allocation Units . . . . . ANY
Maximum IEBCOPY Return Code . . . . . 0
Pathname Substitution Character . . . . Ü

Enter "/" to select option
    Allocate Before Uncatalog
/  Verify Expiration Dates
/  Use SuperC Program Interface
    Monitor Edit Macro Commands via the Activity Monitoring Exit
/  Allow SUBMIT from Browse
/  Allow SUBMIT from View
/  Warn when rename target could be a GDG
/  Default Edit/Browse/View member list from Option 3.4
/  Enable View
    Use Panel ISRTSOA in Option 6
    Print using ICF
    Disallow wildcards in the high level qualifier for Data Set List
    Disable all ENQ displays
/  Fail on LMF lock requests

```

Figure 10-4 Modify PDF Configuration Settings

After you have finished your updates you have to verify your new keyword table using selection 3 as shown in than you have to built a new configuration table using selection 4 as you can see in Figure 10-5 on page 369.

```

                                ISPF Configuration Utility
-----
Option ==> 4

1 Create/Modify Settings and Regenerate Keyword File
2 Edit Keyword File Configuration Table
3 Verify Keyword Table Contents
4 Build Configuration Table Load Module
5 Convert Assembler Configuration Table to Keyword File
6 Build SMP/E USERMOD

Keyword File Data Set
  Data Set . . . 'RMM.ADDONS.CEEXECVB'
  Member . . . . SETTING

Configuration Table Assembler Source Data Set
  Data Set . . .
  Member . . . .

Output File Content for Keyword File
2 1. Include only non-default values
   2. Include defaults as comments
   3. Include all values

Current Configuration Table
  Keyword File : not available
  Identifier . : ISPCFIGU           Level . . . : 480R8001
  Compile Date : 2003/04/04        Compile Time :

```

Figure 10-5 Build Configuration Table Load Module

A small popup window will be displayed to specifying the input keyword file data set and member and the output configuration table load module data set.

```

EeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeN
e                               Build Configuration Table Load Module   e
e Command ==>                                                            e
e                                                                    e
e Input Keyword File Data set                                           e
e   Data Set . . . 'RMM.ADDONS.CEEXECVB'                               e
e   Member . . . . SETTING                                              e
e                                                                    e
e Output Configuration Table Load Module Data Set                       e
e   Data Set . . . 'SCHLUM.ISPF.LLIB'                                   e
e                                                                    e
e Optional fields (leave blank for ISPF to use defaults)               e
e   Object data set . . .                                              e
e   Configuration member          (Defaults to ISPCFIGU)               e
e   VSAM member . . . . .      (Defaults to ISPCFIGU)                 e
DssssssssssssssssssssssssssssssssssssssssssssssssssssssssssssssM

```

Figure 10-6 Build Configuration Table Load Module popup window

If your target load library is a concatenated library of your LINKLIST you have to refresh the the LINKLIST as shown in Example 10-1.

Example 10-1 using the LLA refresh command

```
F LLA,REFRESH
```

Figure 10-7 shows you the successfully refresh of your linklist.

```
CSV210I LIBRARY LOOKASIDE REFRESHED
```

Figure 10-7 Successful refresh of linklist

After you have created the new configuration table load module ensure that the specified load library is concatenated to your ISPLLIB, exit ISPF and start ISPF again.

Important: If the DFSMSrmm REXX exec library, normally called SYS1.SEDGEXE1, is not concatenated in your logon procedure you must move the two members EDGRMLIB and EDGRPD34 to a concatenated REXX Exec library and update member EDGRPD34 to specify that the RMM ISPF environment is allocated:

```
UseIspfLibdef = true          /* << true or false          */
```

10.3 Using the ISPF Data Set List Utility Support

After you have enabeled the ISPF Data Set List (DSLIS) support and restarted your ISPF session you can use this support. The line commands supported by DFSMSrmm are I, S, M, and D:

- I** Displays a search results list showing all data sets in the multivolume set for the selected data set.
- S** Displays the individual data set details. DFSMSrmm determines the first file on the selected volume that matches the selected data set. If other data sets of the same name exist on the volume, the wrong details may be displayed. In that case, use the M line command and then the DFSMSrmm I line command from that results list.
- M** Displays a search results list showing all data sets defined to DFSMSrmm that match the selected data set name.
- D** Releases the volume. If the volume is part of a multivolume set, there is the option to release all volumes in the set.

Figure 10-8 on page 371 shows you the use of the display of individual data set details (S).

Menu Options View Utilities Compilers Help		
DSLISLT - Data Sets Matching SCHLUM.LI*		Row 1 of 5
Command ==>		Scroll ==> PAGE
Command - Enter "/" to select action	Message	Volume

	SCHLUM.LIBDUMP	SBOX03
	SCHLUM.LIBDUMP.BIN.XMIT	SBOXFG
	SCHLUM.LINEITEM.PUNCH	SBOX81
I	SCHLUM.LINEITEM.UNLOAD	Info - I TST026
	SCHLUM.LISTDEF.INPUT	SBOX09
***** End of Data Set list *****		

Figure 10-8 Use of the display of individual data set details

The result can be shown in Figure 10-9 on page 372

Panel Help

DFSMSrmm Data Set Details

Command ==>

Data set name . . . : 'SCHLUM.LINEITEM.UNLOAD'
Volume serial . . . : TST026
Owner : SCHLUM

Physical file sequence number . . . : 1
Data set sequence number : 1
More:

Job name : SCHLUMR3U
Step name : STEP010
Program name . . . : DSNUTILB
DD name : SYSREC
Create date : 2003/300
Create time : 14:51:16
Expiration date . . : 2003/300
Original :
System id : SC63

YYYY/DD
YYYY/DD
YYYY/DD

Record format : VB
Block size : 32760
Logical record length : 151
Block count : 36140
Total block count . . : 36140
Percent of volume . . : 2
Device number : 0B92

Last job name . . . : PAOLOR3T
Last step name . . : STEP010
Last program name : IDCAMS
Date last read . . : 2003/302
Date last written : 2003/300

Last DD name : TAPEVOL
Last device number . : 0B92

Retention date . . :
VRS retained . . . : NO

VRS management value :
Management class . . : MCDB22
Data class :
Storage class : SCLIB1
Storage group : SGLIB1

Security name . . . :
Classification . . :

Primary VRS details:
VRS name :
Job name :
Subchain name :

VRS type :
Subchain start date :

Secondary VRS details:
Value or class :
Job name :
Subchain name :

Subchain start date :

Catalog status . . : YES

Figure 10-9 Result of display of individual data set details

Figure 10-10 on page 373 is showing you all data sets defined to DFSMSrmm that match the selected data set name using function “M”.

```

Panel  Help  Scroll
-----
                DFSMSrmm Data Sets (Page 1 of 2)                Row 1 to 1 of 1
Command ===>                                           Scroll ===> PAGE

Enter HELP or PF1 for the list of available line commands
Use the RIGHT command to view other data columns

S  Data set name                                Volume          File
-----
SCHLUM.LINEITEM.UNLOAD                        TST026  SCHLUM  PAOL0R3
***** Bottom of data *****

```

Figure 10-10 Result if you are selecting function M

Use selection D to releases the volume as shown on the data set. If you select this function DFSMSrmm will show you the confirm volume release panel as shown in Figure 10-11.

```

Panel  Help
-----
                DFSMSrmm Confirm Volume Release
Command ===>

Volume . . . . . : TST026      Location . . . . . : LIB1
VOL1 volser . . . :             In container . . . . . :
Volume type . . . : PHYSICAL   Rack number . . . . . : TST026
Media name . . . . : MEDIA3     Expiration date . . . . : 2003/365

                                Original expiration date :

Status . . . . . : USER
Description . . . :
Data set name . . : 'SCHLUM.LINEITEM.UNLOAD'

                                Release actions:
Media type . . . . : HPCT       Return to SCRATCH pool : NO
Label . . . . . : SL           Replace volume . . . . : NO
Density . . . . . : IDRC       Return to owner . . . . : YES
Recording format . : 128TRACK   Initialize volume . . . : NO
Compaction . . . . : YES       Erase volume . . . . . : NO
Attributes . . . . : NONE      Notify owner . . . . . : NO
Availability . . . : PENDING RELEASE Expiry date ignore . . : NO
                                Scratch immediate . . . : NO

Press ENTER to NOFORCE volume, or END command to CANCEL.

```

Figure 10-11 Result if you are selecting function release the volume.

Figure 10-12 on page 374 displays you a search result if you have selected function “I” that includes you all volumes residing on the same tape or all data set residing on the multi volume set.

Panel Help Scroll			
DFSMSrmm Data Sets (Page 1 of 2)			Row 1 to 10 of 10
Command ==>			Scroll ==> PAGE
Enter HELP or PF1 for the list of available line commands			
Use the RIGHT command to view other data columns			
S	Data set name	Volume serial Owner	File seq

	SCHLUM.LINEITEM.UNLOAD	TST026 MHLRES5	1
	SCHLUM.TESTSTAC.TESTMF02	TST026 MHLRES5	2
	SCHLUM.TESTSTAC.TESTMF03	TST026 MHLRES5	3
	SCHLUM.TESTSTAC.TESTMF04	TST026 MHLRES5	4
	SCHLUM.TESTSTAC.TESTMF05	TST026 MHLRES5	5
	SCHLUM.TESTSTAC.TESTMF06	TST026 MHLRES5	6
	SCHLUM.TESTSTAC.TESTMF07	TST026 MHLRES5	7
	SCHLUM.TESTSTAC.TESTMF08	TST026 MHLRES5	8
	SCHLUM.TESTSTAC.TESTMF09	TST026 MHLRES5	9
	SCHLUM.TESTSTAC.TESTMF10	TST026 MHLRES5	10
***** Bottom of data *****			

Figure 10-12 SEARCH volume result

10.4 Move the ISPCFIGU module to the SISPLPA library

After you have installed and verified ISPF, you can enhance its performance by adding the LPA-eligible load modules (in the SISPLPA library) to the LPA list in an LPALSTxx member of PARMLIB. Add those load modules not eligible for LPA (in the SISpload library) to the link list in an LNKSTxx member of PARMLIB. For information about adding data sets to the Link and LPA lists, see z/OS MVS Initialization and Tuning Reference. You can then remove these data sets from the STEPLIB in your TSO LOGON procedure. After adding SISPLPA to LPALST and SISpload to LNKST, specify CLPA as an initial program load (IPL) parameter to force the SISPLPA modules into the link pack area and to have SISpload added as a system link library.

Finally after you have tested all you can move the ISPCFIGU member to your ISPy you have to re-IPL your system or to refresh your linklist as shown in Example 10-2 and then you must take a new login on to your system before you can use this new service.

Example 10-2 using the LLA refresh command

```
F LLA,REFRESH
```

Figure 10-13 shows you the successfully refresh of your linklist.

```
CSV210I LIBRARY LOOKASIDE REFRESHED
```

Figure 10-13 Result of the LLA REFRESH



Policy management

This chapter describes how DFSMSrmm provides policy management for movement and retention at the data set level. Every tape data set can have a policy, and each policy can specify movement as well as retention. The retention and movement policies you define to DFSMSrmm are known as *Vital Record Specifications* (VRS). You use them to indicate how long and where you want to keep data sets or volumes. You also use them to define how volumes are to be moved among the libraries that DFSMSrmm supports, and the storage locations defined for vital records and disaster recovery purposes.

The objectives of this chapter are to:

- ▶ VRS related PARMLIB options
- ▶ VRS types
- ▶ Specifying the VRS parameters
- ▶ Chaining VRSs
- ▶ VRSEL GDG option
- ▶ DFSMS ACS support
- ▶ Modifying VRS
- ▶ Deleting VRS
- ▶ DFSMSrmm VRS policy management simplification
- ▶ Trial run
- ▶ VRS examples
- ▶ Complex VRS chain with three subchains
- ▶ VRS hints and tips

By the end of this chapter, you will understand the different VRS types and how to create them. Sample VRS definitions are provided for a better understanding.

11.1 VRS related PARMLIB options

For DFSMSrmm vital record specification processing, we have several PARMLIB options that can be used for tailoring your environment:

CATRETPD	Specifies the number of hours from creation that a data set should be retained if it has not been cataloged and matches a vital record specification with the WHILECATALOG operand.
CATSYSID	Specify CATSYSID to enable DFSMSrmm catalog synchronization. Use the CATSYSID operand to identify the user catalogs you want tracked when you run the DFSMSrmm EDGHSKP utility with the CATSYNCH operand to exploit catalog status tracking.
GDG	Use the GDG option to specify how generation data groups are handled for cycle retention by VRSEL processing.
MOVEBY	Specify the MOVEBY operand to move volumes that are retained by DFSMSrmm vital record specifications as a set of volumes or as individual volumes.
RETAINBY	Use the RETAINBY option to specify whether DFSMSrmm retains multivolume sets as a set or as individual volumes.
RETENTIONMETHOD	Use this operand to set the system-wide retention method for new tape volume sets created during Open/Close/End-of-Volume (O/C/EOV) processing, and for tape volumes added to the DFSMSrmm CDS.
REUSEBIN	Specifies to control how DFSMSrmm does reuses bins when a volume is moving from a bin.
VRSCCHANGE	Use VRSCCHANGE to specify the action that DFSMSrmm should take during inventory management if you have made any changes to vital record specifications.
VRSDROP	Use VRSDROP to specify a maximum number or percentage of existing VRS-retained volumes that can be dropped from vital records retention and the action to be taken by DFSMSrmm. DFSMSrmm counts the number of VRS-retained physical and logical volumes at the start of vital record selection processing and the number of these dropped by vital record processing. When you specify COUNT, this is an absolute maximum number of volumes that can be dropped by a single run of EDGHSKP VRSEL processing. When you specify PERCENT, this is the maximum percentage of the existing VRS-retained volumes that can be dropped by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing.
VRSJOBNAME	Use VRSJOBNAME to select how DFSMSrmm uses a job name in a vital record specification to match a data set to a movement and retention policy.
VRSMIN	Use VRSMIN to specify a minimum number of vital record specifications required by inventory management vital record processing, and the action to be taken by DFSMSrmm when the minimum number of vital record specifications are not defined.

VRSEL

Use VRSEL to control how DFSMSrmm inventory management vital record processing uses retention and movement information that are defined in vital record specifications.

Restriction: The **VRSEL(OLD)** option is no longer supported. **VRSEL(NEW)** is the only valid value for the VRSEL option.

VRSJOBNAME

Use VRSJOBNAME to select how DFSMSrmm uses a job name in a vital record specification to match a data set to a movement and retention policy. DFSMSrmm records the data set name and job name for new tape files you create. You can define the name of the job that created a data set by using the RMM ADDDATASET subcommand or the CHANGEDATASET subcommand for existing data sets. You can specify a job name and data set name in a vital record specification so that data sets that match the data set name and job name are moved and retained by the policy defined in the vital record specification.

For detailed information, refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

11.1.1 CATRETPD

DFSMSrmm retains the data set for the catalog retention period if the data set has never been cataloged. DFSMSrmm does not retain the data set if DFSMSrmm detected that the data set was cataloged, and then uncataloged during the catalog retention period.

You have to specify a higher catalog retention value than the default if you have a job creating data sets on tape with a long running time, and the data sets expand a volume, or the job creates more than one data set. If the inventory management process is starting while the job is running, you cannot check whether a data set is cataloged, because the data sets are not cataloged until the job is finished. If you have not defined a catalog retention greater than the run time of your jobs, you can lose some data sets, because DFSMSrmm releases them before the job is finished.

Figure 11-1 illustrates the following situation: The default retention in the PARMLIB member is set to zero RETPD(0). A data set name vital record definition that matches all data set exists, but while cataloged only. You have started a long running job at 7:00 PM, creating one or more data sets expanding a single volume. You have not specified a retention parameter in your JCL. Early in the morning at 8:00 AM, 13 hours after your job has started, the daily EDGHSKP with the inventory management function is starting, and checks all data sets for matching vital record specifications.

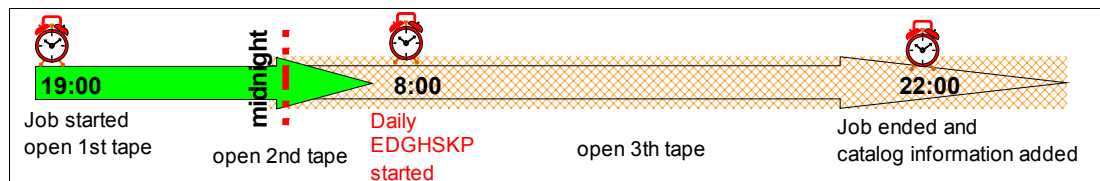


Figure 11-1 How catalog retention works

If you have not specified a CATRETPD parameter in your PARMLIB, the default value of 12 hours is used and the first volume is released, because the data set on the volume has no retention, and is not cataloged so the VRS definition does not match.

If you have specified a CATRETPD parameter in your parmlib with a value of 24 hours, the data set on the first volume is protected by this definition, and no more checks are done. The volume is not released.

11.1.2 GDG option

Use the GDG option to specify how generation data groups are handled for cycle retention by VRSEL processing. Cycle retention includes both the CYCLES and the BYDAYSCYCLE retention types. The correct sequence for determining the retention can be either by using the generation number or the creation order. You can also specify how duplicate generations (generation data sets) are handled and have the flexibility to include or exclude duplicate generations from the cycles count as required by your application processing.

The GDG option has two operands, CYCLEBY and DUPLICATE.

CYCLEBY

Use the CYCLEBY operand to specify whether retention is to be based on generation number or on creation date:

CYCLEBY(GENERATION) Specifies that retention is to be based on the generation number. DFSMSrmm determines the generation number by applying a similar algorithm to that used by Catalog processing. Both the creation order, and the generation number from the data set name are considered, allowing wraps in generation number to be correctly handled.

CYCLEBY(GENERATION) is the default.

CYCLEBY(CRDATE) Specifies that only the creation sequence is to be used to determine the retention.

DUPLICATE

Use the DUPLICATE operand to specify how VRSEL processing handles duplicate generations. You can specify one of:

- ▶ Count duplicate generations
- ▶ Keep duplicate generations, but do not count them
- ▶ Bump duplicate generations from the current subchain
- ▶ Drop duplicate generations from VRS retention

Duplicate generations are determined within a single VRS sub chain and only if the generation and the generation it duplicates are not already dropped for another reason. For GENERATION based cycles, the duplicate generations are determined in generation (data set name), then creation order. For creation date based cycles, the duplicate generations are determined only in creation order, so they can be detected only when they are created consecutively.

Duplicates are processed within the context of the matching VRS chain or sub chain depending on the DUPLICATE option. DROP is within the context of the VRS chain and all others are within the context of the sub chain. A duplicate generation is considered a duplicate for cycles retention only if the generation it duplicates is retained by the current VRS sub chain.

DUPLICATE(BUMP)	Specifies that a duplicate generation is to be bumped by the current VRS sub chain and considered for retention by a subsequent VRS sub chain. DUPLICATE(BUMP) is the default.
DUPLICATE(COUNT)	Specifies that a duplicate generation is to be treated like a non-duplicate generation regarding CYCLE and BYDAYSCYCLE processing.
DUPLICATE(DROP)	Specifies that a duplicate generation is to be dropped from VRS retention without further consideration.
DUPLICATE(KEEP)	Specifies that a duplicate generation is considered as either the same CYCLE or the same BYDAYSCYCLE depending on the VRS retention type and regardless of the duplicate generation creation date.

Before we look at the different handling options which can be specified, we want to explain, under which circumstances VRSEL applies the GDG(DUPLICATE()) operand.

- Same generation number as previous.
 - Consider GDG(CYCLEBY(CRDATE)) – a higher generation might have already been created before the same generation has been created again (like the example shown in Figure 11-2).
- Same matching VRS as previous (same subchain).
 - Reasons why data sets with the same name and generation number match to a different VRS could be:
 - DELETE VRS, OPEN VRS, ABEND VRS
 - VRS with Management Class
 - VRS with JOBNAME()
 - DUPLICATE processing only applies within a single VRS sub chain.
- VRS retention type is CYCLES or BYDAYSCYCLE
- Met other retention criteria (e.g. UNTILEXPIRED)

Creation Sequence	Data Set Name	Matching VRS	Processing sequence	Duplicate
	A.B.G0003V00	DSN('A.B.*') CYCLES		YES
	A.B.G0003V00	DSN('DELETE') DAYS		NO (3)
	A.B.G0003V00	DSN('A.B.*') CYCLES		NO (1)
	A.B.G0004V00	DSN('A.B.*') CYCLES		NO (1)
	A.B.G0003V00	DSN('A.B.*') CYCLES		NO (1)
	A.B.G0003V00	DSN('DELETE') DAYS		NO (2,3)

Figure 11-2 Process data set as „Duplicate Generation

The table in Figure 11-3 on page 380 shows the different results of the DUPLICATE operands BUMP, DROP, KEEP and COUNT.

Let's assume, we have 3 generation data sets in a cycle group, with the „red“ generation 3 being processed as a so called „duplicate“.

The matching VRS has 2 subchains, the first one keeps 2 cycles, the second one has a DAYS retention.

- BUMP**

The duplicate does not get retained by the CYCLES subchain, but the next subchain retains it (assuming the retention criteria are met).
- DROP**

The duplicate is dropped from the whole VRS chain.
- KEEP**

The duplicate gets retained by the CYCLES subchain as the same cycle as it's original, so that 3 data sets are kept with COUNT(2).
- COUNT**

The duplicate gets also retained by the CYCLES subchain, however as it's own cycle, so that generation 2 cannot be kept in this sub-chain anymore.

DUPLICATE processing works basically the same way for CYCLES and BYDAYSCYCLE retention.

For BYDAYSCYCLE is true:

If the duplicate is created on the same day as it's original, KEEP and COUNT make no difference. Because it's the nature of BYDAYSCYCLE that data sets created on the same day are treated as one cycle

VRS Subchains	GDG DUPLICATE(...)			
	BUMP	DROP	KEEP	COUNT
CYCLES COUNT(2)	A.B.G0003V00	A.B.G0003V00	A.B.G0003V00	A.B.G0003V00
	A.B.G0002V00	A.B.G0002V00	A.B.G0003V00 A.B.G0002V00	A.B.G0003V00
DAYS COUNT(100)	A.B.G0003V00			A.B.G0002V00

Figure 11-3 Different results of the DUPLICATE operands BUMP, DROP, KEEP and COUNT

11.1.3 MOVEBY

You use this parameter to move volumes that are retained by DFSMSrmm vital record specifications as a set of volumes or as individual volumes:

- **VOLUME:** Specify VOLUME when you want to move each volume individually.
- **SET:** Specify SET if you want; DFSMSrmm moves a multivolume set as an aggregate. When you retain by SET, if any volume in a set is to be moved by a vital record specification, all volumes in the set are moved to the same location.

Figure 11-4 on page 381 shows how DFSMSrmm works with MOVEBY(SET) specification if you have a set of tree volumes with tree data sets on it, and two of the data sets should be moved to different locations.

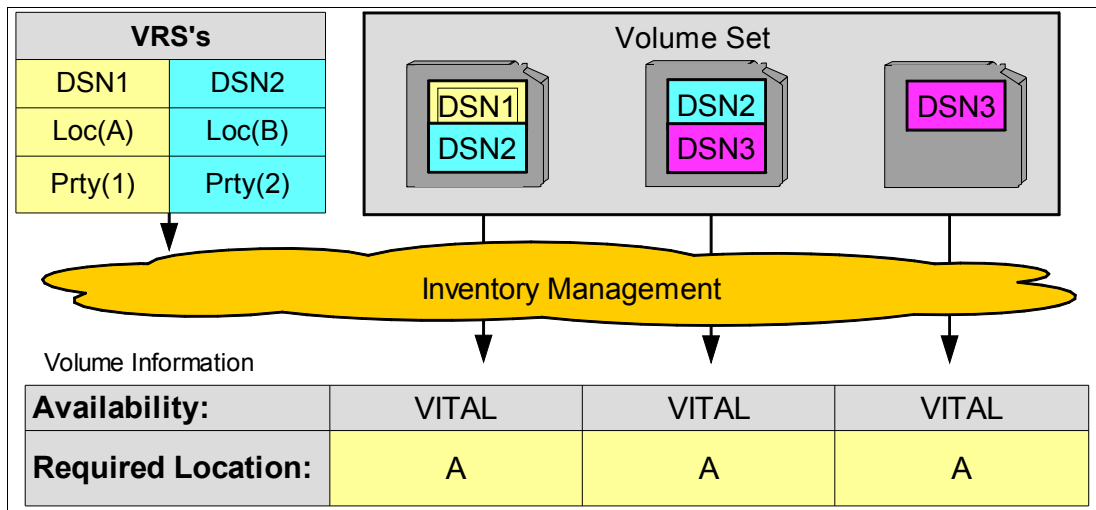


Figure 11-4 *RETAINBY(SET) MOVEBY(SET)*

The second volume is moved to location A because the VRS definition for data set A has a higher priority than the VRS definition for data set B. The third volume in the set is moved along with the other two volumes because it is part of the set.

11.1.4 RETAINBY

This operand allows you to work with the volume set as an aggregate:

- **VOLUME:** Specify VOLUME when you want to retain each volume individually.
- **SET:** Specify SET if you want DFSMSrmm to retain a multivolume set as an aggregate. When you retain by SET, if any volume in a set is retained by a vital record specification, all volumes in the set are retained as vital records.

When a volume in a set is only retained because it is a member of a set, DFSMSrmm sets an indicator as shown in Figure 11-5.

Set retained . . . : YES

Figure 11-5 *RETAINBY set indicator*

Figure 11-6 shows how DFSMSrmm works with the RETAINBY(SET) specification if you have a set of three volumes with tree data sets on it.

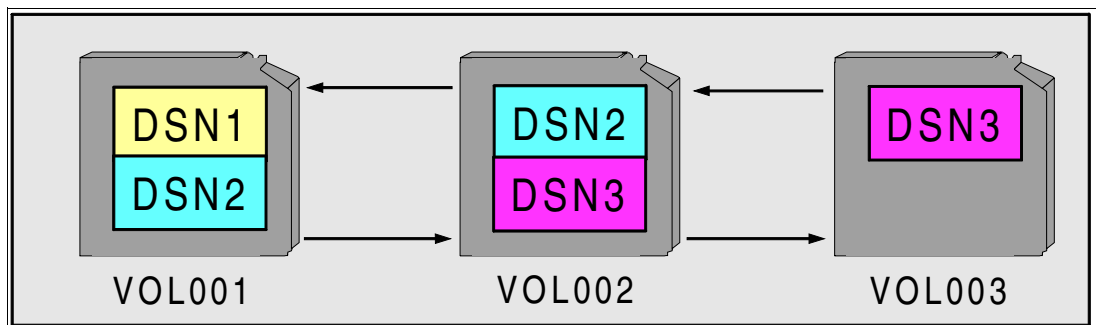


Figure 11-6 *Volume set*

Figure 11-7 shows VOL001 with EXPDT +10 days, VOL002 with EXPDT +20 days, and VOL003 with EXPDT -1 day. After the housekeeping, all the volumes are retained and VOL003 sets the Set Retained field to YES, because it is retained only by being a member of a set.

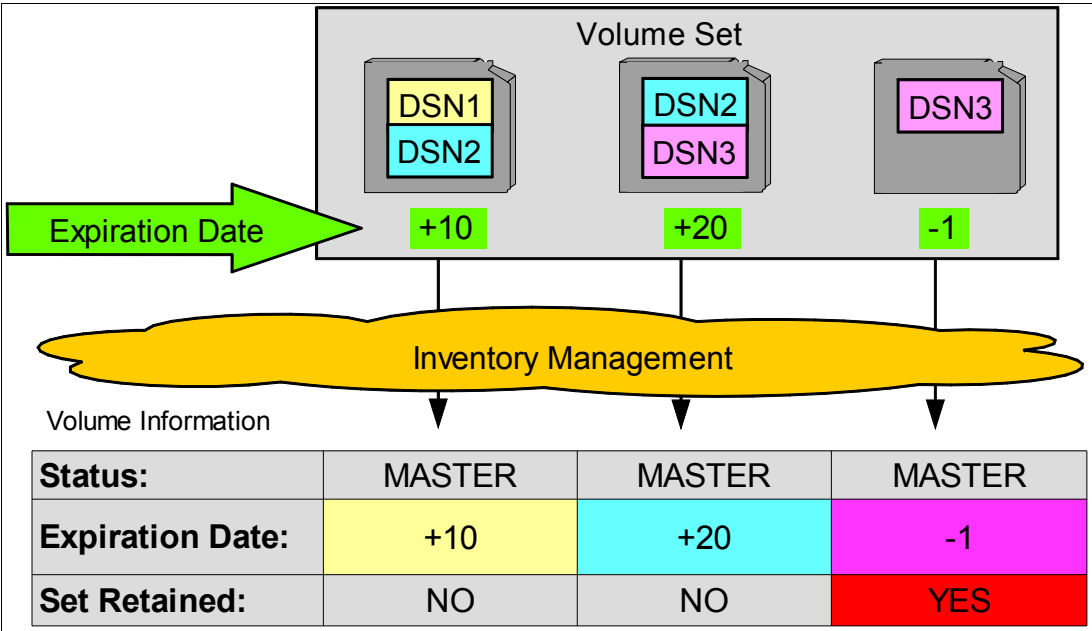


Figure 11-7 RETAINBY(SET) - EXPDT

If the volumes are retained by VRS, you see the result shown in Figure 11-8.

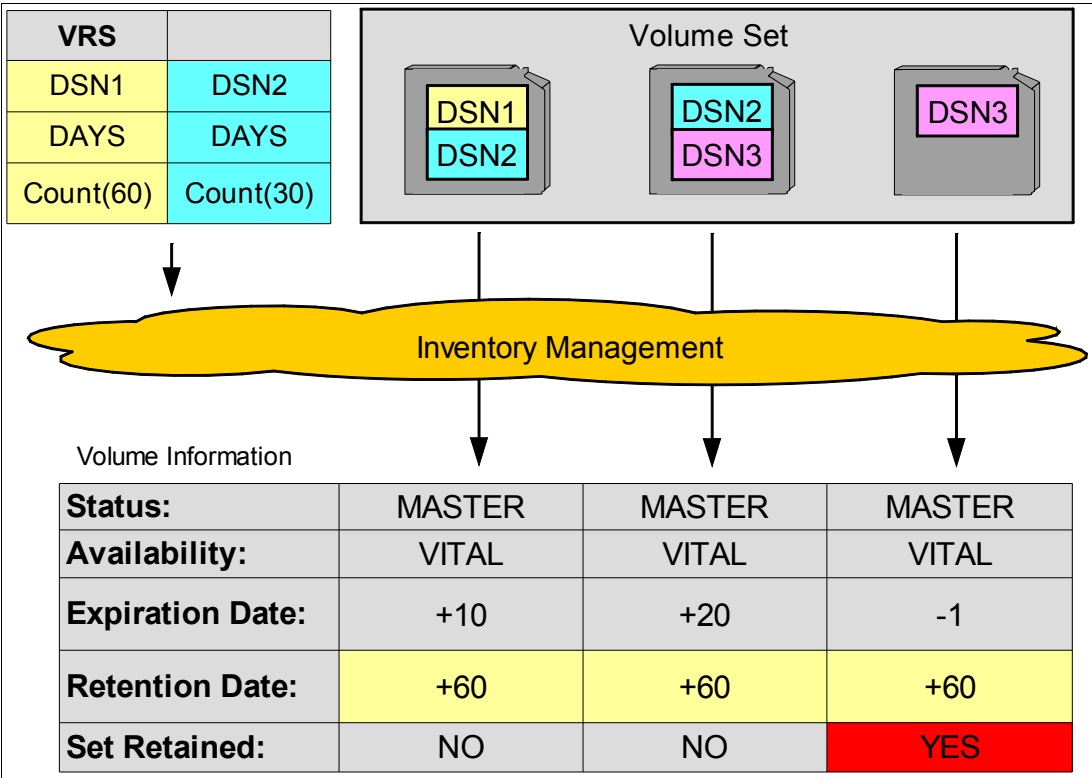


Figure 11-8 RETAINBY(SET) - VRS

11.1.5 REUSEBIN

Use this operand to control how DFSMSrmm reuses bins when a volume is moving from a bin.

CONFIRMMOVE

When a volume moves out of a bin, DFSMSrmm does not reuse this bin until the volume move has been confirmed. In this case, you have to define more bins as shown in Figure 11-9.

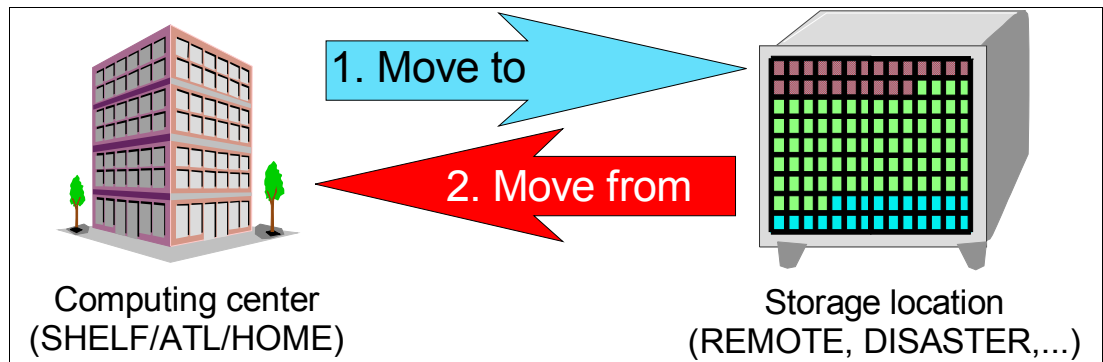


Figure 11-9 REUSE bin with CONFIRMMOVE operand

STARTMOVE

When a volume moves out of a bin, DFSMSrmm does not reuse this bin until the volume move has been confirmed.

A bin can be reused as soon as a volume starts moving out of a bin. Extended bin support must be enabled before you can use this operand. This parameter works only if you have enabled the extended bin support. For more information, refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405. The bins are reused in the same processing run as shown in Figure 11-10.

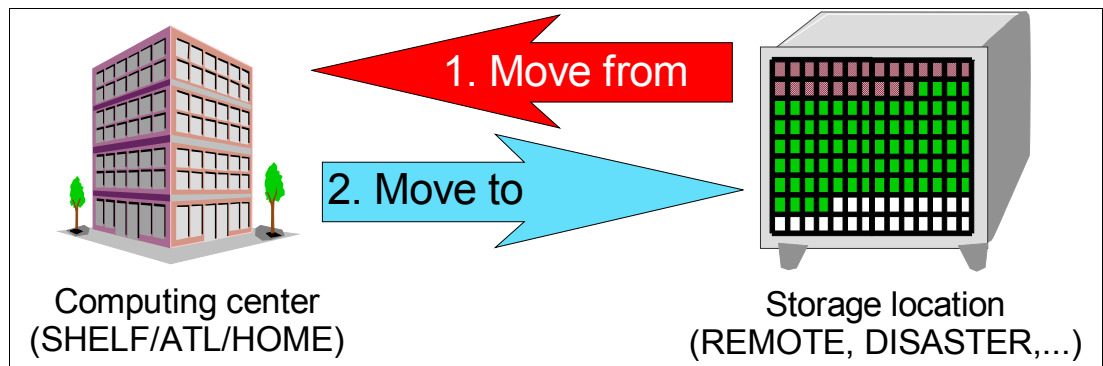


Figure 11-10 REUSE bin with STARTMOVE operand

11.1.6 VRSCCHANGE

Specify the action that DFSMSrmm should take during inventory management if you have made any changes to vital record specifications since the last run:

- **INFO:** You see a message in the job log and no additional processing or actions are required when vital record specification changes occur.

- **VERIFY:** Any changes made to vital record specifications must be verified by running EDGHSKP vital record processing using the VERIFY parameter. DFSMSrmm must issue a return code zero before EDGHSKP vital record processing can be performed to update the control data set.

Note: Check the ACTIVITY log of the vital record processing trial run, and check all changes made in this run before you start the life update.

11.1.7 VRSJOBNAME

Use this option to tell DFSMSrmm which of the two possible masks DSNNAME or JOBNAME should be checked first:

- **1:** If you specify one, the jobname or jobname mask is checked first, and the data set name or data set name mask is checked second.
- **2:** If you specify two, the data set name or data set name mask is checked first, and the jobname or jobname mask is checked second.

As part of vital record processing, DFSMSrmm checks that data set name masks and job name masks used in vital record specifications are specified according to DFSMSrmm naming conventions.

DFSMSrmm compares the data set, job name, and volume information recorded in the control data set with information in the VRSs to determine which data sets and volumes to retain and the processing required. This includes any volumes with special JCL specified expiration dates used by your installation.

DFSMSrmm builds a matching string with the data set and job name. The data set and job name position depends on the value of the VRSJOBNAME parameter. DFSMSrmm selects the most specific VRS.

For example, we define the VRSs as shown in Figure 11-11.

```
1. RMM ADDVRS DSN('ZE*.TEST.T---R--') WHILECATALOG
2. RMM ADDVRS DSN('*ERB.TEST.T---R--') DAYS COUNT(150)
3. RMM ADDVRS DSN('*ERB.TEST.T---R--') CYCLES COUNT(10) -
   JOBNAME('ZERB%%')
```

Figure 11-11 VRS examples using wild card characters

We also run job ZERB0A, which creates the data set ZERB.TEST.T0023R45.

Depending on the VRSJOBNAME parameter, we get different results.

Using the VRSJOBNAME(1) option, we have the following syntax string as shown in Figure 11-12:

```
ZERB0A!!ZERB.TEST.T0023R45
```

Figure 11-12 VRSJOBNAME(1)

In this case, VRS (c) matches the string, because it matches first by job name and second by data set name.

Using VRSJOBNAME(2), we have the following syntax string as shown in Figure 11-12:

```
ZERB.TEST.T00023R45!!ZERB0A
```

Figure 11-13 VRSJOBNAME(1)

In this case, VRS (a) matches the string, because it matches by data set name having an alphanumeric character (Z) on the first position.

11.1.8 VRSMIN

Specify a minimum number of vital record specifications required by inventory management vital record processing, and the action to be taken by DFSMSrmm when the minimum number of vital record specifications are not defined. DFSMSrmm counts the vital record specifications used by vital record processing. DFSMSrmm does not count vital record specifications that are deleted during vital record processing.

- ▶ **FAIL:** Issues message EDG2229I to the MESSAGE file and stops inventory management processing. Processing ends with return code 8.
- ▶ **INFO:** Issues message EDG2229I to the MESSAGE file and processing continues.
- ▶ **WARN:** Issues message EDG2229I to the MESSAGE file and sets a minimum return code of 4. Processing continues.

Tip: After conversion, set the minimum number of vital record specifications that you get, and set the processing action to FAIL.

11.1.9 VRSEL

Control how DFSMSrmm inventory management vital record processing uses retention and movement information that is defined in vital record specifications.

- ▶ **OLD:**

Attention: This option is no longer supported.

- ▶ **NEW:** Specify NEW if you want DFSMSrmm to process:
 - Retention information in NAME vital record specifications
 - Release options defined in vital record specifications
 - Vital record specifications chained using the ANDVRS operand

The following example shows you how VRSEL(NEW) works:

- ▶ We first define the following data set and management value VRSs as shown in Figure 11-14.

```
1. RMM ADDVRS DSNAME('YCJRES.TEST')      -  
              DAYS COUNT(100)             -  
              LOCATION(REMOTE)             -  
2. RMM ADDVRS DSNAME('CYCL004')           -  
              CYCLES COUNT(4) LOCATION(DISTANT) -  
              STORENUMBER(4)
```

Figure 11-14 Defining two data set name VRS definitions

- Figure 11-15 shows some data sets that were created that have a management value of CYCL004 assigned.

(1)	Data set name	YCJRES.TEST	creation date	120 days ago
(2)	Data set name	YCJRES.TEST	creation date	119 days ago
(3)	Data set name	YCJRES.TEST	creation date	118 days ago
(4)	Data set name	YCJRES.TEST	creation date	90 days ago
(5)	Data set name	YCJRES.TEST	creation date	89 days ago
(6)	Data set name	YCJRES.TEST	creation date	88 days ago

Figure 11-15 Data set created using a management value of CYCL004

- When VRSEL(NEW) is in use, both VRSs are processed. Data sets numbers (4), (5) and (6) are retained in location REMOTE by the data set VRS, and number (3) in DISTANT location by the name VRS. DFSMSrmm retains the data set (3) by the secondary VRS (MV VRS) after the primary VRS (data set VRS) does not retain the data set any more.

With VRSEL(NEW), both VRSs primary and secondary are processed in parallel on each housekeeping run. DFSMSrmm evaluates both VRSs and then determines which policy to apply.

When VRSEL(NEW) is in use, the volume expiration date is not used during data set name VRS UNTILEXPIRED processing when a data set matches both a primary VRS and a secondary VRS. For example, if a retention type of UNTILEXPIRED is defined in the primary data set name VRS, and in the secondary management value (name vital record specification) VRS there is a retention type of DAYS with COUNT(30) defined, then the volume is retained by the secondary management value VRS. This definition retains the data set for 30 more days before being expired.

If UNTILEXPIRED is used in a secondary VRS, the volume expiration date is used to determine whether the retention condition is true.

There is no restriction on the definition of either the primary or secondary VRS; both can be vital record specification chains containing any of the possible ADDVRS operands.

11.1.10 VRSDROP

Use VRSDROP to specify a maximum number or percentage of existing VRS retained volumes that can be dropped from vital records retention, and the action to be taken by DFSMSrmm. DFSMSrmm counts the number of VRS retained volumes at the start of vital record selection processing and the number of these dropped by vital record processing. When you specify COUNT this is an absolute maximum number of volumes that can be dropped by a single run of EDGHSKP VRSEL processing. When you specify PERCENT this is the maximum percentage of the existing VRS retained volumes that can be dropped by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing.

Figure 11-16 on page 387 shows you the correct use of the VRSDRAIN parameter option.

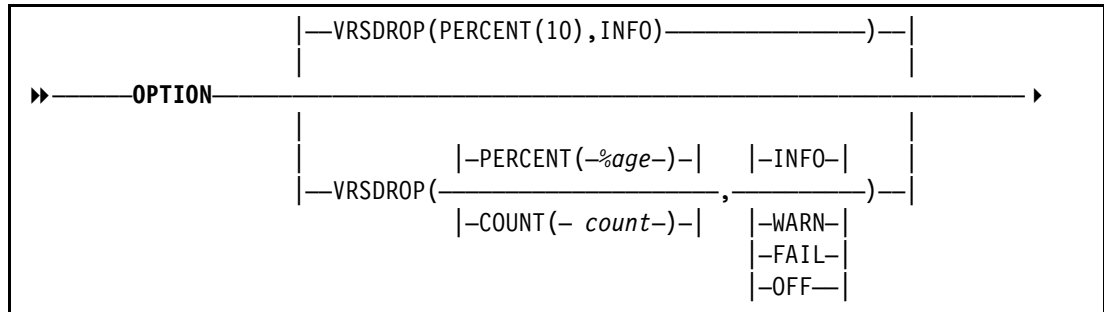


Figure 11-16 EDGRMMnn OPTION operand VRSDROP

Where:

COUNT	this is an absolute maximum number of volumes that can be dropped by a single run of EDGHSKP VRSEL processing.
PERCENT	this is the maximum percentage of the existing VRS-retained volumes that can be dropped by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing. VRSDROP processing is intended to provide limited checking for volumes that are already VRS-retained. It considers how many of these existing VRS-retained volumes are removed from VRS retention. Those volumes dropped from VRS-retention can be set pending release, depending on VRS release options and the volume expiration date. They can also become EXPDT retained and on the next run of EXPROC, they will be considered by EXPDTPROP limit processing. count can be 0 to 2,147,483,647. Percent can be 0 to 100. The default is PERCENT(10).
action	Specify action to control the action DFSMSrmm takes during processing and when the value is exceeded. Action can be FAIL, INFO, OFF, or WARN.
FAIL	Issues messages EDG2242I and EDG2244I to the MESSAGE file. When the value is exceeded, DFSMSrmm stops inventory management processing prior to making CDS updates and in addition, message EDG2310I is issued, and processing ends with return code 12.
INFO	Issues messages EDG2242I and EDG2244I to the MESSAGE file and processing continues.
OFF	Processing of this function is turned off.
WARN	Issues messages EDG2242I and EDG2244I to the MESSAGE file. When the value is exceeded, DFSMSrmm sets a minimum return code of 4 and processing continues.

11.1.11 VRSRETAIN

Use VRSRETAIN to specify a minimum number or percentage of newly assigned volumes that are to be retained by vital records retention, and the action to be taken by DFSMSrmm. A newly assigned volume is one which has a volume assignment date and time which is higher than the run date and time of the previous VRSEL processing. DFSMSrmm counts the number of newly assigned volumes at the start of vital record selection processing and the

number of these that become VRS retained by vital record processing. When you specify count this is an absolute minimum number of volumes that are to be retained by a single run of EDGHSKP VRSEL processing. When you specify a percentage this is the minimum percentage of the newly assigned volumes that are to be retained by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing.

Figure 11-17 shows you the correct use of the VRSRETAIN parameter option.

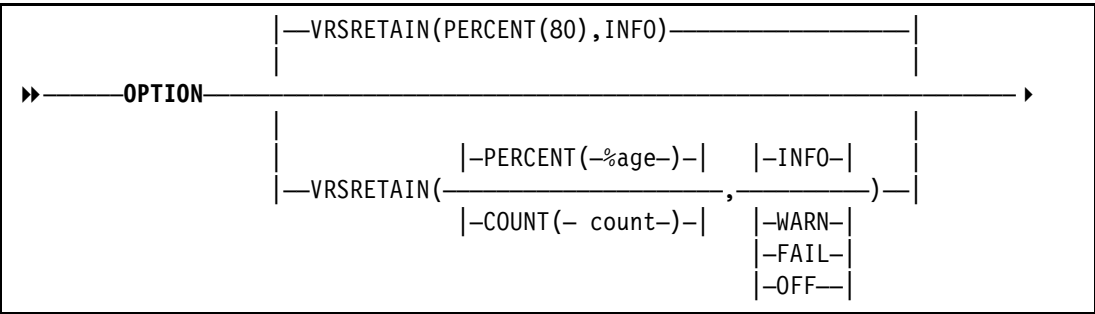


Figure 11-17 EDGRMMnn OPTION operandVRSRETAIN

Where:

COUNT

this is an absolute minimum number of volumes that are to be retained by a single run of EDGHSKP VRSEL processing. A newly assigned volume is one that has a volume assignment date and time that is higher than the run date and time of the previous VRSEL processing and that is not VRS-retained. DFSMSrmm counts the number of newly assigned physical and logical volumes at the start of vital record selection processing and the number of these that become VRS retained by vital record processing.

PERCENT

this is the minimum percentage of the newly assigned volumes that are to be retained by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing. VRSRETAIN processing is intended to provide limited checking for volumes containing newly created data sets that have not yet been processed by inventory management VRSEL. The data sets on the volumes have been created since the start of the last completed VRSEL run. It considers how many of these volumes become VRS-retained during the new VRSEL run. Those volumes which become VRS-retained will be considered by the VRSDROP limit checking in future VRSEL runs. Those volumes that are not retained by VRS can be set pending release, depending on VRS release options and the volume expiration date. They can also become EXPDT retained and on the next run of EXPROC, they will be considered by EXPDTRDROP limit processing. count can be 0 to 2,147,483,647. Percent can be 0 to 100. The default is PERCENT(80).

action

Specify action to control the action DFSMSrmm takes during processing and when the value is exceeded. Action can be one of:

FAIL	Issues messages EDG2243I and EDG2245I to the MESSAGE file. When the value is exceeded, DFSMSrmm stops inventory management processing prior to making CDS updates and in addition, message EDG2310I is issued, and processing ends with return code 12.
INFO	Issues messages EDG2243I and EDG2245I to the MESSAGE file and processing continues.
OFF	Processing of this function is turned off.
WARN	Issues messages EDG2243I and EDG2245I to the MESSAGE file. When the value is exceeded, DFSMSrmm sets a minimum return code of 4 and processing continues.

11.2 VRS types

DFSMSrmm uses policies to control the retention of data sets and volumes, and to determine a volume's required location. The retention and movement policies specified in the VRS are processed during DFSMSrmm inventory management (housekeeping).

You can define expiration and retention defaults for your installation, using the DFSMSrmm PARMLIB member EDGRMMxx.

There are three types of VRS:

- ▶ Data set VRS
- ▶ Volume VRS
- ▶ Name VRS

To define VRS, you can use DFSMSrmm ISPF panels or TSO subcommands. In the examples, we used TSO subcommands. To define VRS using panels, go to the DFSMSrmm Add Vital Record Specification panel, shown in Figure 11-18.

Panel Help	

DFSMSrmm Add Vital Record Specification	
Command ==>	
Specify one of the following:	
Data set mask . . 'DELETEx'	
Job name mask . .	Add data set filter VRS . . (Yes or No)
Volume serial . .	(May be generic)
VRS name	

Figure 11-18 DFSMSrmm Add Vital Record Specification panel in z/OS release 1.8 or higher

11.2.1 Data set VRS

A data set vital record specification defines the retention criteria by setting the total number of data set cycles to be retained, or the total number of days during which a data set should be retained. A data set *cycle* is a new version of the data set, so the number of data set cycles is equivalent to the number of data set versions.

You can combine or merge retention and movement policies, so you need to define more than one vital record specification for this data set. In this case, we call the *primary* vital record specification to the one that matches on:

- ▶ Data set name
- ▶ Data set name mask
- ▶ Management class name, or
- ▶ Vital record specification management value

The *secondary* vital record specification matches on:

- ▶ Management class, or
- ▶ Vital record specification management value

A vital record specification *management value* is a single qualifier name you define in a vital record specification to tell DFSMSrmm how to manage and retain your tape data sets. The vital record specification management value can be up to eight alphanumeric characters, and must begin with an alphabetic character.

Figure 11-19 shows how DFSMSrmm performs vital record processing if you used PARMLIB option VRSEL(NEW).

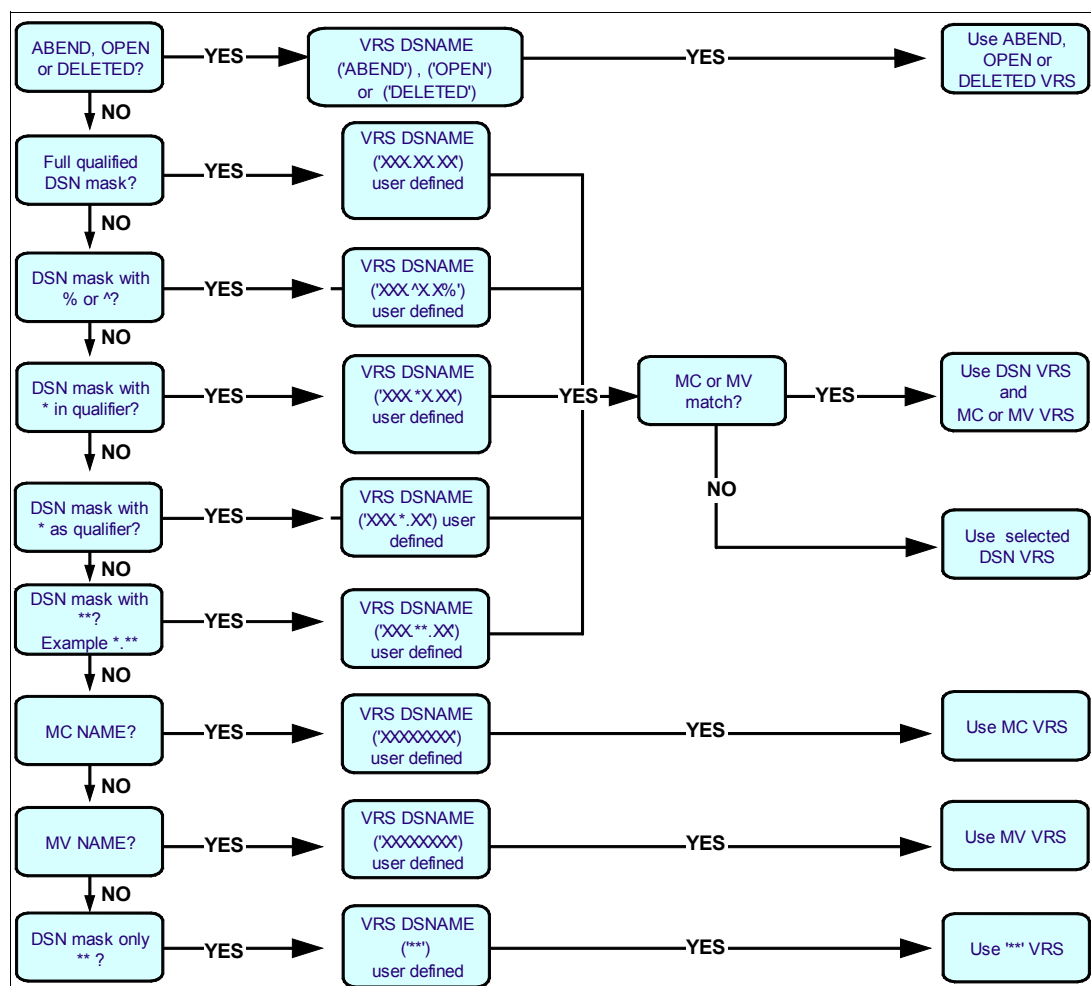


Figure 11-19 Selecting matching VRS for data sets (new)

VRSEL(NEW) provides such new and improved functions as:

- ▶ Mixed retention types: NEXTVRS to different type
- ▶ Combined retention: ANDVRS
- ▶ Release options: EXPIRYDATEIGNORE and SCRATCHIMMEDIATE
- ▶ Enable use of primary and secondary VRSs
- ▶ Management class and VRS management value are equivalents, and can be secondary VRS policy

Matching data sets

- ▶ Fully qualified data set names/job name

The following VRS retains all copies of the data set DATA.SET.BACKUP for five days. NOGDG is the default.

```
RMM ADDVRS DSNAME('DATA.SET.BACKUP') NOGDG DAYS COUNT(5)
```

- ▶ Data set name/job name masks

You can use standard wild cards: %, *, ** and ¬.

Note: If a ¬ (not sign) is used in a data set name, the name is treated as a pseudo GDG entry.

** indicates to select all data sets. You can use this mask to define a vital record specification to define a retention criteria for data sets that are not covered by other vital record specifications.

The data set name masks *.** and ** match to all data sets not covered by a more specific vital record specification. You can use these data set name masks to define a system-wide release option.

- ▶ Generation data groups (GDGs)
- ▶ You can use data set name or data set name masks with the GDG operand. In this case, the VRS only match GDG data sets.
- ▶ The following VRS definition retains the last three versions of the GDG named DATA.SET.GDG:

```
RMM ADDVRS DSNAME('DATA.SET.GDG') GDG CYCLE COUNT(3)
```

- ▶ But, it does not retain the non-GDG data set DATA.SET.GDG. You can also define a VRS definition using the not sign (¬) wild card character to retain only GDGs as shown here:

```
RMM ADDVRS DSNAME('DATA.SET.GDG.G¬¬¬¬V¬¬') CYCLE COUNT(3)
```

The following VRS definition retains all GDGs starting with DATA.TEST:

```
RMM ADDVRS DSNAME('DATA.TEST.**') GDG CYCLES COUNT(99999)
```

The following VRS definition retains all GDGs and non-GDGs starting with DATA.TEST:

```
RMM ADDVRS DSNAME('DATA.TEST.**') NOGDG CYCLES COUNT(99999)
```

- ▶ Management class (**MC**)

You can specify a management class name in a vital record specification. The MC is assigned by the SMS ACS routines. You can find more information in 11.7, “DFSMS ACS support” on page 421.

The following example defines a VRS using a management class name to retain a data set with MC 'DAYS003'. This MC is assigned to the data set by your installation through an

ACS routine. The operand DAYS COUNT(3) indicates the data set is to be retained for three days. Define a management class.

```
RMM ADDVRS DSNAME('DAYS003') DAYS COUNT(3)
```

Note: There is no difference in the VRS definitions between a management class and VRS management value. Only the tools (SMS or EDGUX100) setting the value are different.

► VRS management value (**MV**)

For data sets that do not belong to a MC, but need to be managed by special expiration dates, you can define a vital record specification management value. To assign MV to data sets, use the DFSMSrmm supplied EDGUX100 installation exit.

The following example defines a VRS to retain a data set with special date 99000. The vital record specification MV 'CATALOG' is assigned by your installation in an installation exit such as the EDGUX100 user exit. The WHILECATALOG operand indicates the data set is to be retained while it is cataloged.

```
RMM ADDVRS DSNAME('CATALOG') WHILECATALOG
```

► Status **ABEND**

To control the retention of data sets closed as a result of an abnormal end in a task, you can define a VRS using a reserved data set name mask ABEND. For example, to retain the data set one day after the last reference, use the command shown in the following example:

```
RMM ADDVRS DSNAME('ABEND') LASTREFERENCEDAYS COUNT(1)
```

Note: You can define several ABEND VRSs to have different retention for data sets created using different JOBNAMEs:

```
RMM ADDVRS DSNAME('ABEND') JOBNAME(*) DAYS COUNT(1)
RMM ADDVRS DSNAME('ABEND') JOBNAME(TSM*) CYCLES COUNT(99999)
RMM ADDVRS DSNAME('ABEND') JOBNAME(HSM*) CYCLES COUNT(99999)
```

This retains all data sets created by using the job name such as HSMA and HSMB, or TSM1 permanently, and you have to release them manually.

► Status **DELETED**

To control the data sets created with a disposition of delete. DFSMSrmm provides support for managing deleted tape data sets. DFSMSrmm checks the normal disposition at CLOSE time and if this is 'DELETE', a 'deleted' flag is recorded in the data set record. Subsequent use of a data set cannot change this. However, you can use the CHANGEDATASET command to reset the flag.

You enable the management function by defining one or more DELETED VRSes. Deleted data sets that do not match to a DELETED VRS are matched to other VRSes as normal.

```
RMM AS DSNAME('DELETED') DAYS COUNT(0) -
    RELEASE(EXPIRYDATEIGNORE,SCRATCHIMMEDIATE)
RMM AS DSNAME(mask) JOBNAME(DELETED) DAYS COUNT(0) -
    RELEASE(EXPIRYDATEIGNORE,SCRATCHIMMEDIATE):
```

Note: You can define several DELETED VRs to have different retention for data sets created using different JOBNAMEs:

```
RMM ADDVRS DSNAME('DELETED') JOBNAME(*) DAYS COUNT(1)
      RELEASE(EXPIRYDATEIGNORE,SCRATCHIMMEDIATE)
RMM ADDVRS DSNAME('***') JOBNAME(TSM*) CYCLES COUNT(99999)
RMM ADDVRS DSNAME('***') JOBNAME(HSM*) CYCLES COUNT(99999)
```

This retains all data sets created by using the job name such as HSMA and HSMB or TSM1 permanently, and you have to release them manually.

► Status **OPEN**

To control the retention of data sets left open during a system failure, or that are in use during DFSMSrmm inventory management, use the reserved data set name mask OPEN. For example, to retain the data set two days after the last reference, use the command as shown in the following example:

```
RMM ADDVRS DSNAME('OPEN') LASTREFERENCEDAYS COUNT(2)
```

Note: You can define several OPEN VRs to have different retention for data sets created using different JOBNAMEs:

```
RMM ADDVRS DSNAME('ABEND') JOBNAME(*) DAYS COUNT(1)
RMM ADDVRS DSNAME('ABEND') JOBNAME(TSM*) CYCLES COUNT(99999)
RMM ADDVRS DSNAME('ABEND') JOBNAME(HSM*) CYCLES COUNT(99999)
```

This retains all data sets created by using the job name such as HSMA and HSMB or TSM1 permanently, and you have to release them manually.

Figure 11-20 shows in which order DFSMSrmm matches the data set name VRs.

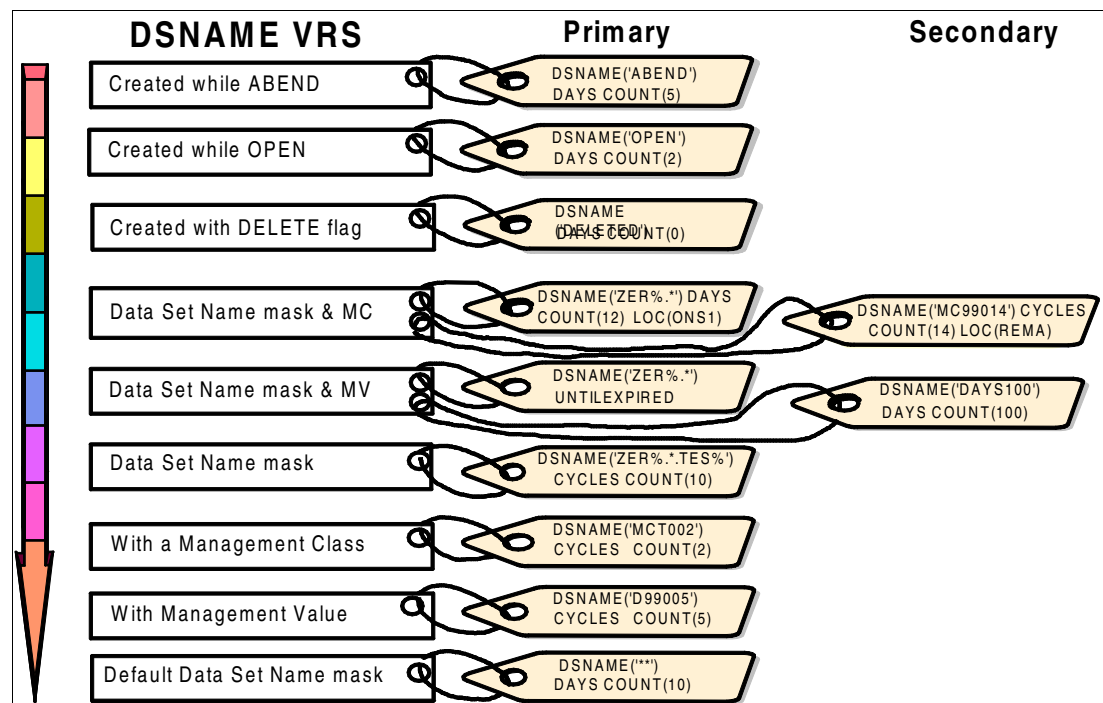


Figure 11-20 Data set name matching order

11.2.2 Volume VRS

A volume vital record specification defines the retention criteria by setting the total number of volumes to be retained, or the total number of days during which a volume should be retained.

Matching volumes

Volume vital record specifications are defined using a specific or generic volume serial number. The matching rules for volumes are:

- Specific volume serial number

Specify the volume serial number to retain only this volume. The following example shows how you can retain a specific volume for five days:

```
RMM ADDVRS VOLUME(TST100) LOCATION(HOME) COUNT(5)
```

- Generic volumes

You can specify generic volumes using “*”. The following example shows the RMM TSO ADDVRS subcommand that you can use to retain ten volumes matching the prefix TST:

```
RMM ADDVRS VOLUME(TST*) LOCATION(REMOTE) COUNT(10)
```

11.2.3 Name VRS

Name vital record specifications can define additional movement criteria, optionally additional retention policy information. When a name VRS contains only movement information, it cannot exceed the retention criteria specified in the previous VRS, and it is called a *location* name VRS. We recommend that you do not use location name VRSs, but instead always specify a retention type and a COUNT value. You can define name vital record specifications for creating data set or volume VRS *chains*.

Specify the DFSMSrmm PARMLIB OPTION VRSEL(OLD) operand to define movement information only. Specify VRSEL(NEW) to define either retention or movement information, or both.

The following example shows the definition of a VRS chain where the second VRS is a name VRS with different retention criteria:

```
RMM ADDVRS DSN('DATA.SET.**') COUNT(3) CYCLES NEXTVRS(SECVRS)  
RMM ADDVRS NAME(SECVRS) EXTRADAYS COUNT(10)
```

For more details about chaining vital record specifications, refer to 11.4, “Chaining VRSs” on page 399.

11.3 Specifying the VRS parameters

Table 11-1 shows the different parameters that you can specify in a VRS definition depending on the type of VRS that you are defining.

Table 11-1 Specifying VRS

Types of VRS and retention	DATA SET	VOLUME	NAME
Match using	- Data set name - Management class - VRS management value - GDG - ABEND - OPEN	- Volume serial number - Generic volser	
Retention types	- CYCLES - BYDAYSCYCLE - DAYS - LASTREFERENCE-DAYS - WHILECATALOG - UNTILEXPIRED	- Days - Volumes	- CYCLES - BYDAYSCYCLE - DAYS - LASTREFERENCE-DAYS - WHILECATALOG - UNTILEXPIRED - EXTRADAYS
Limits	- COUNT - DELETEDATE	- COUNT - DELETEDATE	- COUNT - DELETEDATE
Movement	- STORENUMBER - LOCATION - PRIORITY - DELAY	- STORENUMBER - LOCATION - PRIORITY - DELAY	- STORENUMBER - LOCATION
Release options	- SCRATCH-IMMEDIATE - EXPIRYDATE-IGNORE		
Chain using	- NEXTVRS - ANDVRS	- NEXTVRS	- NEXTVRS - ANDVRS
Chained from			- DSN VRS - VOLUME VRS - NAME VRS

11.3.1 DSNAME and NAME retention types

DFSMSrmm calculates the retention date for data sets based on a combination of the COUNT operand value and the retention type operands. The retention type operands for data sets and names are:

► CYCLES

For GDG data sets, indicate the number of generations that DFSMSrmm should be kept. To retain three versions of the GDG ITSO.GDG.DATA.SET, define the VRS as shown in the following example:

```
RMM ADDVRS DSNAME('ITSO.GDG.DATA.SET') GDG CYCLES COUNT(3)
```

For non-GDG data sets, each occurrence of a data set is a cycle. The following VRS retains three occurrences of the data set ITSO.NONGDG.DATA.SET.

```
RMM ADDVRS DSNAME('ITSO.NONGDG.DATA.SET') CYCLES COUNT(3)
```

► **BYDAYSCYCLE**

DFSMSRmm considers all the data sets created in a single day is a cycle.

► **DAYS**

This operand indicates a period of elapsed days from the creation date. The following command shows how you can retain data sets with the high level qualifier YCJRES created in the last 10 days:

```
RMM ADDVRS DSNAME('YCJRES.**') DAYS COUNT(10)
```

► **LASTREFERENCEDAYS**

This operand indicates a period of elapsed days since the data set was last read or written to. You can define a VRS by using:

```
RMM ADDVRS DSNAME('YCJRES.**') LASTREFERENCEDAYS COUNT(10)
```

This retains all data sets beginning by YCJRES last referenced during the last 10 days.

► **WHILECATALOG**

You can use this operand to retain data sets while they are cataloged.

► **UNTILEXPIRED**

A data set now optionally can be managed separately for retention. A vital record specification management value or a SMS management class can identify retention criteria, and the data set name can be used to identify movement criteria. Up to two VRSs can be used to provide policy information for a data set.

When new data is created, and no vital record specification management value or management class is assigned, because the jobs do not specify keyword dates and are not SMS managed, VRSEL processing finds only the data set VRS to match to, which specifies UNTILEXPIRED. The following example shows how you can define them.

```
RMM ADDVRS DSNAME('**') UNTILEXPIRED
```

DFSMSRmm checks the volume expiration date, and the UNTILEXPIRED option makes DFSMSRmm drop the data set from retention.

The volume expiration date is also used if the following conditions are true and the PARMLIB option VRSEL(NEW) is specified.

- When a data set matches to a single VRS
- When UNTILEXPIRED is used in the secondary VRS

The VRS enhancements introduced with APAR OW30969 enable until expired retention to be based on any type of retention. When a primary and secondary VRS match to a data set, the use of UNTILEXPIRED in the primary uses the secondary VRS chain to determine overall retention. If there is no secondary VRS, or UNTILEXPIRED is used in a secondary VRS or a VRS management value VRS, and PARMLIB option VRSEL(NEW) is used, only the volume expiration date is used. If VRSEL(OLD) is specified in PARMLIB, UNTILEXPIRED retention uses the volume expiration date or catalog status to determine for how long to retain a data set.

Any policy used for retention or movement, or both retention and movement, can be a chain of multiple VRSs, where each VRS in a chain can use a different type of retention. Both the primary and secondary VRS can be VRS chains, offering enough flexibility to create quite complex retention and movement policies.

A secondary VRS can be only a VRS management class or a VRS management value.

11.3.2 Special NAME retention type

If you have specified to use a NEXTVRS in chain, you can use the EXTRADAYS operand on it. On a NEXTVRS you can specify the number of days since a name vital record specification started to retain the data set. The number of extra days is specified with the COUNT operand.

EXTRADAYS is the number of days since the NAME vital record specification started to retain the data set. The number of days depends on when the previous vital record specification stopped retaining the data set, and the time when vital record processing is run. This option can only be used on a NEXTVRS and not a primary VRS.

11.3.3 Volume retention types

DFSMSrmm calculates the retention date for volumes based on a combination of the COUNT operand value and the retention type operands. The retention type operands for volumes are:

- **BY DAYS**

By number of elapsed days since creation, which indicates that a volume is retained until the specified number of days since creation has elapsed.

To retain volume V20001 for 31 days in a storage location, type the following command:

```
RMM ADDVRS VOLUME(V20001) DAYS COUNT(31) LOCATION(DISTANT)
```

- **BY NUMBER OF VOLUMES**

- Indicates how many volumes should be retained. For example, to retain the latest 50 volumes matching the generic volume serial number, TST, in a storage location, type the following command:

```
RMM ADDVRS VOLUME(TST*) COUNT(50) LOCATION(DISTANT)
```

11.3.4 Retention limits

You can define the limits for a volume retention or for a VRS to be considered as valid. The parameters that you can use to specify the limits are:

- **COUNT**

Specifies a retention amount, based on the retention type chosen: number of days or cycles for data sets VRS, and number of volumes for volume VRS. The COUNT default is 99999.

You can specify COUNT with DAYS to indicate the number of days that DFSMSrmm retains your data set, as shown in the following example:

```
RMM ADDVRS DSNAME('YCJRES.**') DAYS COUNT(10)
```

Or, you can specify COUNT with CYCLES to indicate the number of cycles of the data set that DFSMSrmm retains, as shown in the following example:

```
RMM ADDVRS DSNAME('YCJRES.BACKUP.CYCLE') CYCLES COUNT(8)
```

Note: A value of 99999 indicates that DFSMSrmm retains all cycles of a data set, or all volumes specified.

- **DELETEDATE**

Specifies the date when DFSMSrmm deletes the VRS. The default value is 1999/365, which indicates that DFSMSrmm never deletes the vital record specification. When that date is reached, all matching data sets and volumes are eligible for matching to other, less

specific VRSs, and if no longer VRS is retained, the volumes are eligible for release processing.

See the following example. How you can define a VRS retains the data set until the VRS deletion date (2005/170) is reached, or the data set is uncataloged.

```
RMM ADDVRS DSNAME('YCJKRES1.DELETE.DATE') -  
          DELETEDATE(2005/170) WHILECATALOG
```

Note: If you have a DELETEDATE specified, and the date is reached, all data sets matching this VRS are never passed to another VRS.

11.3.5 Movement policies

You can define policies to provide retention information for data sets and volumes that must be moved through multiple locations before they expire. You define such policies by creating a VRS chain, and using the following parameters:

► **STORENUMBER**

For data set VRS, this specifies how many days to retain a data set, how many data set cycles or versions to retain in the location where the volume is. For volume VRS specifies how many volumes to retain, or how many days to retain a volume in the location specified in the parameter LOCATION. The number specified must be less than or equal to COUNT.

► **LOCATION**

Specifies a location where the volume should be retained. Use HOME if you want the volume to be returned to its home location. Use CURRENT to avoid moving volumes.

► **PRIORITY**

When multiple data sets on the same volume are retained by VRSs, and each VRS contains a different destination, DFSMSrmm decides where to move the volume according to priority number. The default value 0 means that the priority specified in the LOCDEF parameter on PARMLIB should be used.

► **DELAY**

Specifies the number of days that a data set or volume is retained in its current location before it is sent to a storage location.

11.3.6 Release options

Specifies special release options to be processed after a data set is dropped from VRS, and the volume is released. Release options take effect only for data sets once they have been retained by a VRS.

► **SCRATCHIMMEDIATE**

If you specify this option, DFSMSrmm returns the volume to scratch status in a single run of inventory management (EXPROC processing) when the only pending release action for the volume is to return to scratch, and you have specified VRSEL(NEW).

► **EXPIRYDATEIGNORE**

If you specify this option, DFSMSrmm ignores the volume expiration date when VRSEL(NEW) is used and no other vital VRS matches to the data set.

11.4 Chaining VRSs

A *VRS chain* is composed by a volume or data set VRS, and all of the name VRSs chained from it. For chaining two or more VRS, you must use one of the two following operands:

- **NEXTVRS**

Specify NEXTVRS(name) to point to another VRS, and create a chain of VRSs. With NEXTVRS, DFSMSrmm processes each vital record specification in the chain separately.

You can only chain volume VRSs with NEXTVRS operand.

- **ANDVRS**

Specify ANDVRS to create a chain of VRSs where all the retention conditions must be true for the data sets to be retained. DFSMSrmm uses the STORENUMBER and LOCATION from the first VRS in the VRS chain.

You can combine VRSs using NEXTVRS and ANDVRS operands; these combinations are called VRS chain and VRS subchain:

- VRS chain is a volume or data set VRS and all name VRSs chained from it.
- VRS subchain is a data set VRS, name VRS with retention information, or ANDVRS group, and all the VRSs chained from it, without including the NEXTVRS in the chain that contains retention information.

Both a VRS chain and subchain may comprise one or more VRSs. When a data set name VRS is chained only to name VRSs that have no retention information, there is no difference between a VRS chain and a VRS subchain. The following examples explain the differences between chain and subchain VRS.

Figure 11-21 shows a VRS chain with three subchains, they are a data set VRS with NEXTVRS, and two-name VRSs with retention information.

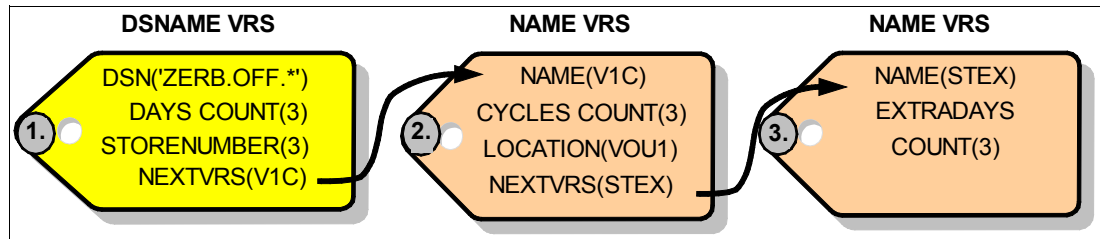


Figure 11-21 VRS chain with three subchains

Figure 11-22 shows a VRS chain with two subchains, it is a data set VRS with a ANDVRS that form one subchain, and a name VRS joined with NEXTVRS command with retention information.

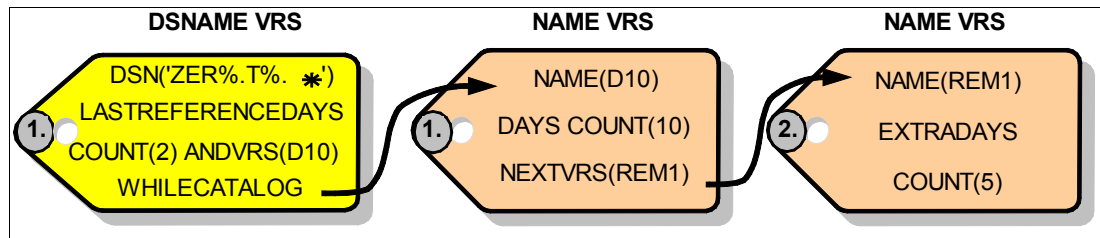


Figure 11-22 VRS chain with two subchains

Figure 11-23 shows a VRS chain with a single subchain; the VRS are joined by ANDVRS commands.

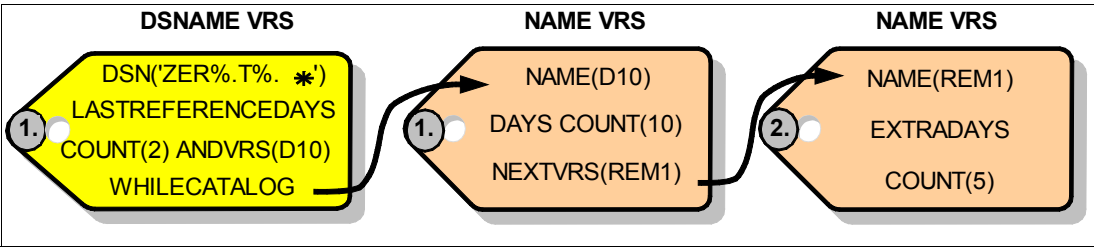


Figure 11-23 VRS chain using NEXTVRS

Figure 11-24 shows a VRS chain with two subchains: the data set VRS and the first name VRS form a subchain, because the name VRS does not have retention information. The second name VRS is a different VRS subchain.

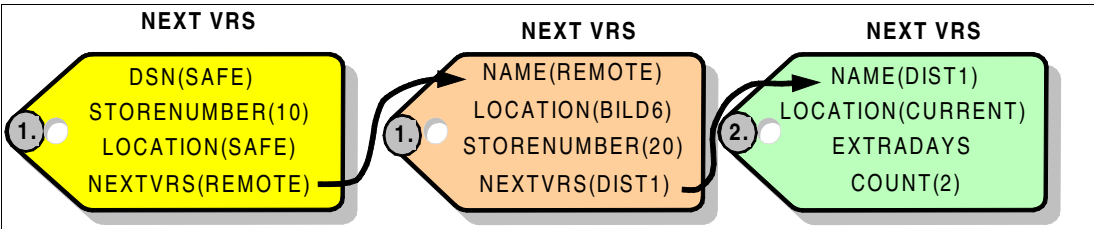


Figure 11-24 VRS chain with two subchains

11.4.1 VRS chain and subchain

The VRS is key to how DFSMSrmm manages your tape data sets and volumes; retention and movement are driven by VRSs. There are many ways to build VRS chains and subchains. Even if you are able to meet all your retention and movement needs, your approach should be to keep the VRS chain and subchain as simple as possible.

There are some terms you should be familiar with before we discuss building VRS chains and subchains:

Data Set Group	All the data sets with a name that matches a VRS are treated as a single vital retention group. You can use a GDG base name when defining a VRS to retain volumes. You must not supply the generation data set group suffix. You must specify CYCLES if you want DFSMSrmm to manage the data sets as a data set group. If you are using GDG version numbering, DFSMSrmm only keeps the latest version of each generation.
VRS chain	A volume or data set VRS and all of the name VRSs chained from it
VRS subchain	A data set name VRS, NAME VRS with retention information, or AND VRS group and all the VRSs chained from it, up to but not including the next VRS in the chain that contains retention information. Both a VRS chain and subchain may comprise one or more VRSs. When a data set name VRS is chained only to NAME VRSs that have no retention information, there is no difference between a VRS chain and a VRS subchain.

Note: A data set VRS can only be used as the first definition in a chain. A data set VRS cannot be used in the NEXTVRS operand.

11.5 VRSEL GDG option

New DFSMSrmm parmlib options provide flexibility in how tape generation data sets are managed for cyclic retention.

Attention: Non generation data sets are not reflected by this new function.

Use the GDG option to specify how generation data groups are handled for cycle retention by VRSEL processing. Cycle retention includes both the CYCLES and the BYDAYSCYCLE retention types. The correct sequence for determining the retention can be either by using the generation number or the creation order. You can also specify how duplicate generations are handled and have the flexibility to include or exclude duplicates from the cycles count as required by your application processing.

The correct syntax to the new GDG OPTION command shown in Figure 11-25.

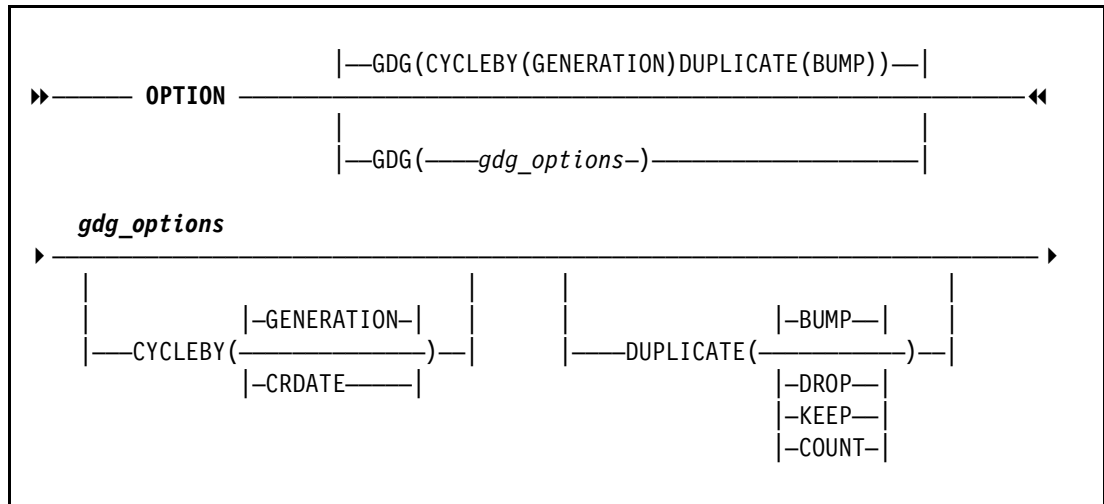


Figure 11-25 GDG option command syntax

Where:

GDG

Use the GDG option to specify how generation data groups are handled for cycle retention by VRSEL processing. Cycle retention includes both the CYCLES and the BYDAYSCYCLE retention types. The correct sequence for determining the retention can be either by using the generation number or the creation order. You can also specify how duplicate generations (generation data sets) are handled and have the flexibility to include or exclude duplicate generations from the cycles count as required by your application processing.

The GDG option has two operands, CYCLEBY and DUPLICATE.

Use the CYCLEBY operand to specify whether retention is to be based on generation number or on creation date:

CYCLEBY(GENERATION)

Specifies that retention is to be based on the generation number. DFSMSrmm determines the generation number by applying a similar algorithm to that used by Catalog processing. Both the creation order, and the generation number from the data set name

are considered, allowing wraps in generation number to be correctly handled.

CYCLEBY(GENERATION) is the default.

CYCLEBY(CRDATE) Specifies that only the creation sequence is to be used to determine the retention.

Use the DUPLICATE operand to specify how VRSEL processing handles duplicate generations. You can specify one of:

- ▶ Count duplicate generations
- ▶ Keep duplicate generations, but do not count them
- ▶ Bump duplicate generations from the current subchain
- ▶ Drop duplicate generations from VRS retention

Duplicate generations are determined within a single VRS sub chain and only if the generation and the generation it duplicates are not already dropped for another reason. For GENERATION based cycles, the duplicate generations are determined in generation (data set name), then creation order. For creation date based cycles, the duplicate generations are determined only in creation order, so they can be detected only when they are created consecutively.

Duplicates are processed within the context of the matching VRS chain or sub chain depending on the DUPLICATE option. DROP is within the context of the VRS chain and all others are within the context of the sub chain. A duplicate generation is considered a duplicate for cycles retention only if the generation it duplicates is retained by the current VRS sub chain.

DUPLICATE(BUMP) Specifies that a duplicate generation is to be bumped by the current VRS sub chain and considered for retention by a subsequent VRS sub chain.

DUPLICATE(BUMP) is the default.

DUPLICATE(COUNT) Specifies that a duplicate generation is to be treated like a non-duplicate generation regarding CYCLE and BYDAYSCYLE processing.

DUPLICATE(DROP) Specifies that a duplicate generation is to be dropped from VRS retention without further consideration.

DUPLICATE(KEEP) Specifies that a duplicate generation is considered as either the same CYCLE or the same BYDAYSCYCLE depending on the VRS retention type and regardless of the duplicate generation creation date.

Before we look at the different handling options which can be specified, we want to explain, under which circumstances VRSEL applies the GDG(DUPLICATE()) operand.

1. Same generation number as previous.

- Consider GDG(CYCLEBY(CRDATE)) – a higher generation might have already been created before the same generation has been created again (like the example shown in Figure 11-26 on page 403).
2. Same matching VRS as previous (same subchain).
 - Reasons why data sets with the same name and generation number match to a different VRS could be:
 - DELETE VRS, OPEN VRS, ABEND VRS
 - VRS with Management Class
 - VRS with JOBNAME()
 - DUPLICATE processing only applies within a single VRS sub chain.
 3. VRS retention type is CYCLES or BYDAYSCYCLE
 4. Met other retention criteria (e.g. UNTILEXPIRED)

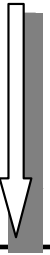
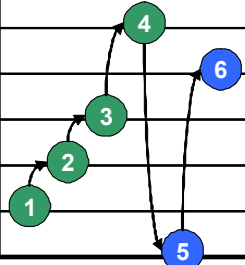
Creation Sequence	Data Set Name	Matching VRS	Processing sequence	Duplicate
	A.B.G0003V00	DSN('A.B.*') CYCLES		YES
	A.B.G0003V00	DSN('DELETE') DAYS		NO (3)
	A.B.G0003V00	DSN('A.B.*') CYCLES		NO (1)
	A.B.G0004V00	DSN('A.B.*') CYCLES		NO (1)
	A.B.G0003V00	DSN('A.B.*') CYCLES		NO (1)
	A.B.G0003V00	DSN('DELETE') DAYS		NO (2,3)

Figure 11-26 Process data set as „Duplicate Generation

The table in Figure 11-27 on page 404 shows the different results of the DUPLICATE operands BUMP, DROP, KEEP and COUNT.

Let's assume, we have 3 generation data sets in a cycle group, with the „red“ generation 3 being processed as a so called „duplicate“.

The matching VRS has 2 subchains, the first one keeps 2 cycles, the second one has a DAYS retention.

BUMP	The duplicate does not get retained by the CYCLES subchain, but the next subchain retains it (assuming the retention criteria are met).
DROP	The duplicate is dropped from the whole VRS chain.
KEEP	The duplicate gets retained by the CYCLES subchain as the same cycle as it's original, so that 3 data sets are kept with COUNT(2).
COUNT	The duplicate gets also retained by the CYCLES subchain, however as it's own cycle, so that generation 2 cannot be kept in this sub-chain anymore.

DUPLICATE processing works basically the same way for CYCLES and BYDAYSCYCLE retention.

For BYDAYSCYCLE is true:

If the duplicate is created on the same day as it's original, KEEP and COUNT make no difference. Because it's the nature of BYDAYSCYCLE that data sets created on the same day are treated as one cycle

VRS Subchains	GDG DUPLICATE(...)			
	BUMP	DROP	KEEP	COUNT
CYCLES COUNT(2)	A.B.G0003V00	A.B.G0003V00	A.B.G0003V00	A.B.G0003V00
	A.B.G0002V00	A.B.G0002V00	A.B.G0003V00 A.B.G0002V00	A.B.G0003V00
DAYS COUNT(100)	A.B.G0003V00			A.B.G0002V00

Figure 11-27 Different results of the DUPLICATE operands BUMP, DROP, KEEP and COUNT

Use the RMM TSO LISTCONTROL OPTION to get the information which parmlib option is currently set. Figure 11-28 shows you the correct use of the command.

RMM LISTCONTROL OPTION

Figure 11-28 LISTCONTROL OPTION

In Figure 11-29 on page 405 you can see the result of the LISTCONTROL command.

```

System options:
System options:
PARMLIB Suffix = 02
Operating mode = P      Retention period: Default = 0      Maximum = NOLIMIT
                           Catalog = 6      hours
                           Retention method: Method = VRSEL
Control data set name      = RMM.CONTROL.DSET
Journal file data set name = RMM.JOURNAL.DSET
Journal threshold          = 75%      Journal transaction = NO
Catalog SYSID              = Notset
Scratch procedure name     = EDGXPROC
Backup procedure name      = EDGCDSBK
IPL date check = N      Date format = J      RACF support = N
SMF audit = 0      SMF security = 42      CDS id = SC70
MAXHOLD value = 100      Lines per page = 54      System ID = SC70
BLP = RMM      TVEXT purge = RELEASE Notify = N
                           days = 0
Uncatalog = Y      VRS job name = 2      Message case = M
MASTER overwrite= USER      Accounting = J      VRS selection = NEW
VRS change = INFO      GDG duplicate = BUMP      GDG cycle by = GENERATION
VRSMIN action = INFO      VRSMIN count = 1
VRSDROP action = INFO      VRSDROP count = 0      percent = 10
VRSRETAIN action= INFO      VRSRETAIN count= 0      percent = 80
EXPDTDROP action= INFO      EXPDTDROP count= 0      percent = 10
Disp DD name = DISPDD      Disp msg ID = EDG4054I
Retain by = SET      Move by = SET      CMDAUTH Owner = NO
PREACS = NO      SMSACS = NO      CMDAUTH Dsn = YES
Reuse bin = CONFIRMMOVE      Media name = 3480
                           Local tasks = 10

PDA: ON
Block count = 0      Block size = 0      Log = ON
SMSTAPE:
Update scratch = YES      Update command = YES      Update exits = YES
Purge = ASIS
Client/Server:
Subsystem type = STANDARD Port = 0
Server      Server tasks = 0
host name =
IP address =

```

Figure 11-29 Result of LISTCONTROL

in your activity log you can now find two additional fields as shown in Figure 11-30 that shows you the active GDG definitions at this run.

```

-----1-----+...7-----+-----8-----+-----9-----+-----0-----+-----1-----+-----2-
H   2009/223  1...SC70  NSS           0 10I           0 80I           0 10IGB

```

Figure 11-30 Activity header record

Where:

```

GDGCYCLEBY      on column 120
ACTRC_HDR_GDGCYCLEBY  DS  C      GDG(CYCLEBY())

```

```

ACTRC_HDR_GDGC_GENERATION EQU C'G'
ACTRC_HDR_GDGC_CRDATE     EQU C'C'

```

```

GDGDuplicate      on column 121
ACTRC_HDR_GDGDuplicate DS  C      GDG(Duplicate())
ACTRC_HDR_GDGD_Bump   EQU C'B'
ACTRC_HDR_GDGD_DROP   EQU C'D'
ACTRC_HDR_GDGD_Keep   EQU C'K'
ACTRC_HDR_GDGD_Count  EQU C'C'

```

11.5.1 Generation data set and DSN vital record specification creation

To test and show you how this new functionality works we have created 12 data sets on tape and will use always this data sets in our test cases. See Table 11-2 for the details of the created data sets. So we have created each generation data set twice with different creation days. You can also see that the generation data sets G0003V00 are created after the generation data sets G0004V00.

Table 11-2 Created data set to test new GDG parmlib option

DSNAME	VOLSER	CRDATE
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	NS0001	2009/200
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	NS0011	2009/201
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	NS0002	2009/207
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	NS0012	2009/208
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	NS0003	2009/221
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	NS0013	2009/222
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	NS0004	2009/214
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	NS0014	2009/215
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	NS0005	2009/228
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	NS0015	2009/229
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	NS0006	2009/235
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	NS0016	2009/236

Second we have a DSNAME vital record specification with the attributes you can see in Figure 11-31 on page 407 to mach the data sets shown in Table 11-2.


```

                                DFSMSrmm Display Data Set VRS
Command ==>

Data set mask . : 'RMM.TEST.NEW.GDG.PARMLIB.OPTION.**'          GDG . : NO
  Job name mask . :

Count . . . : 5                      Retention type . . . . . : CYCLES
                                   While cataloged . . . . . : NO
Delay . . . : 0    Days                Until expired . . . . . : NO

Location . . . . . : HOME
Number in location : 5
Priority . . . . . : 0

                                   Release options:
Next VRS in chain . : GDGOPT          Expiry date ignore . . . : NO
  Chain using . . . :                  Scratch immediate . . . . : NO

Owner . . . . . : SCHLUM
Description . . : RETAIN AND MOVE DATA SETS
Last reference  :                      ( YYYY/DDD  HH:MM:SS )
Delete date . . : 1999/365    ( YYYY/DDD )

Last Change information:
  Date . . . . . : 2010/289          Time . : 20:45:17      System : SC70
  User change date : 2010/289        Time . : 20:45:17      User ID : MHLRES7

```

Figure 11-31 DSNAMES vital record specification to test new GDG parmlib option

To show the differences better we have added a management value/management class vital record specification too. See Figure 11-32 to get the details of this additional DSNAMES vital record specification.

DFSMSrmm Display Name VRS		
Command ==>		
Name	: GDGOPT	
Count	: 6	Retention type : CYCLES
		While cataloged : NO
		Until expired : NO
Location	: HOME	
Number in location	: 6	
Next VRS in chain . .	:	
Chain using . . .	:	
Owner	: SCHLUM	
Description . . .	: TEST NEW GDG PARMLIB OPTION	
Last reference	: (YYYY/DDD HH:MM:SS)	
Delete date . . .	: 1999/365 (YYYY/DDD)	
Last Change information:		
Date	: 2011/335	Time . . : 12:49:20 System : SC70
User change date	: 2011/335	Time . . : 12:49:20 User ID : MHLRES7

Figure 11-32 Management value / management class VRS definition

11.5.2 CYCLEBY GENERATION and DUPLICATE BUMP

Figure 11-33 Shows you the normal GDG processing as it was used before. It matched the GDG parmlib option GDG(CYCLEBY(GENERATION) DUPLICATE(BUMP)) and it is the default if you have not specified the GDG option in your EDGRMMnn parmlib. You can also specify CYCLEBY(CRDATE) to fulfil the old processing if you have created the data sets in sequence.

Creation Sequence	Data Set Name	Processing Sequence for GDG(CYCLEBY(...))	
		GENERATION	CRDATE
	A.B.G0001V00		
	A.B.G0002V00		
	A.B.G0003V00		
	A.B.G0004V00		
	A.B.G0005V00		

Figure 11-33 Generations created consecutively

In our first test case we have create 12 files and we have specified a DSNAME vital record specification with the option CYCLES and a count of 5 to show you how does it works. In Figure 11-34 you can see the detail DSNAME VRS options.

We have created each generation data set twice to show you that always only the most actual generation data set is retained and the older one not. Table 11-2 on page 406 shows you all

the data set we have created and Figure 11-34 shows you how a generation data set is retained and how not. As you can see only one circumstance of the same generation data set is retained.

REMOVABLE MEDIA MANAGER			VITAL RECORDS RETENTION REPORT							PAGE		1		
Copyright IBM Corp. 1993,2007			-----							TIME 14:07:04 DATE		2009/223		
JOB MASK	DATA SET OR VOLUME MASK		OWNER	TYPE	RETN	C	X	DELETE	DLY	COUNT	STNUM	LOCATION	RLSE	LASTREF
	RMM.TEST.NEW.GDG.PARMLIB.OPTION.**		SCHLUM	DSN	CYCLES	N	N	1999/365	0	5	5	HOME		2009/223
	GDGOPT		MHLRES7	NEXT	CYCLES	N	N	1999/365		6	6	HOME		2009/223
JOB NAME	DATA SET NAME		FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT	REQUIRED	PRTY	RETDAT	RETNAM		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0016	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0006	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0015	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0005	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0014	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0004	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0013	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0003	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0012	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0002	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT			
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0011	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT			
NUMBER OF DATA SETS RETAINED (GROUP STORE) =												11	0	

Figure 11-34 Result of test case one

In Table 11-3 on page 409 you can see the drop reason and why a data set is retained.

Table 11-3 Details of test case one

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0001	2009/200	no	12	cycles exceeded	duplicate GDS
RMM.....G0001V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 6
RMM.....G0002V00	NS0002	2009/207	next	10	duplicate GDS	CYCL/00006 5
RMM.....G0002V00	NS0012	2009/208	prim	9	CYCL/00005 5	
RMM.....G0003V00	NS0003	2009/221	next	8	duplicate GDS	CYCL/00006 4
RMM.....G0003V00	NS0013	2009/222	prim	7	CYCL/00005 4	
RMM.....G0004V00	NS0004	2009/214	next	6	duplicate GDS	CYCL/00006 3
RMM.....G0004V00	NS0014	2009/215	prim	5	CYCL/00005 3	
RMM.....G0005V00	NS0005	2009/228	next	4	duplicate GDS	CYCL/00006 2
RMM.....G0005V00	NS0015	2009/229	prim	3	CYCL/00005 2	
RMM.....G0006V00	NS0006	2009/235	next	2	duplicate GDS	CYCL/00006 1
RMM.....G0006V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.3 CYCLEBY CRDATE and DUPLICATE BUMP

If, however, generation data sets are created out of sequence, you can use the GDG(CYCLEBY(CRDATE)) operand of the parmlib OPTION command to specify if VRSEL should process the data sets in generation sequence from highest number to lowest, or you

can specify GDG(CYCLEBY(CRDATE)) to process the data sets in creation date sequence starting from the youngest. Figure 11-35 shows you the sequence the data sets are created.

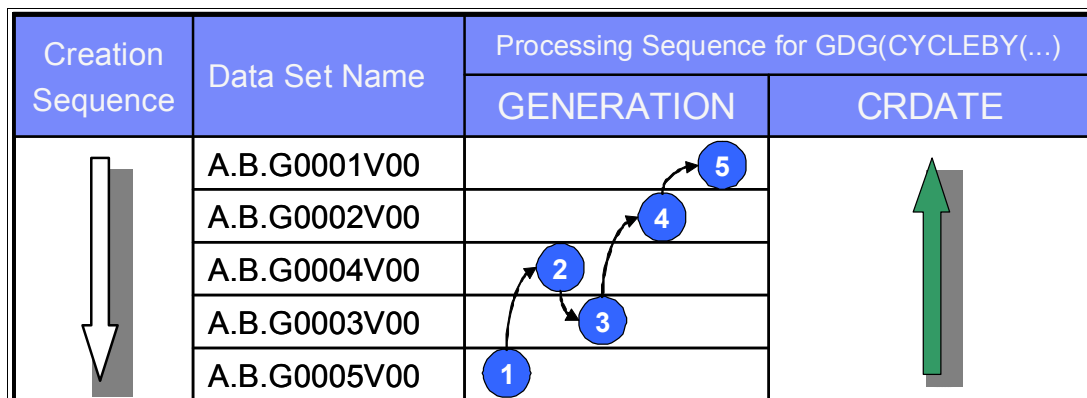


Figure 11-35 Generations out of sequence

In our second test case we have used the same 12 files as before, but we have changed the GDG option from CYCLESBY(GENERATION) to CYCLESBY(CRDATE). Figure 11-36 on page 410 shows you the result of our test.

REMOVABLE MEDIA MANAGER		VITAL RECORDS RETENTION REPORT						PAGE 1				
Copyright IBM Corp. 1993,2007								TIME 14:09:45 DATE 2009/223				
JOB MASK	DATA SET OR VOLUME MASK	OWNER	TYPE	RETN	C X	DELETE	DLY	COUNT	STNUM	LOCATION	RLSE	LASTREF
	RMM.TEST.NEW.GDG.PARMLIB.OPTION.**	SCHLUM	DSN	CYCLES	N N	1999/365	0	5	5	HOME		2009/223
	GDGOPT	MHLRES7	NEXT	CYCLES	N N	1999/365		6	6	HOME		2009/223
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT	REQUIRED	PRTY	RETDATE		RETNAME
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0016	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0006	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0015	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0005	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0013	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0003	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0014	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0004	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0012	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0002	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0011	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT	
NUMBER OF DATA SETS RETAINED (GROUP STORE) =										11	0	

Figure 11-36 Result of test case two

In Table 11-4 you can see the drop reason and why a data set is retained. As you can see generation data set G0003V00 is retained before generation data set G0004V00 depend on the creation date.

Table 11-4 Details of test case two

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0001	2009/200	no	12	cycles exceeded	duplicate GDS
RMM.....G0001V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 6
RMM.....G0002V00	NS0002	2009/207	next	10	duplicate GDS	CYCL/00006 5
RMM.....G0002V00	NS0012	2009/208	prim	9	CYCL/00005 5	
RMM.....G0003V00	NS0003	2009/221	next	6	duplicate GDS	CYCL/00006 3

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0003V00	NS0013	2009/222	prim	5	CYCL/00005 3	
RMM.....G0004V00	NS0004	2009/214	next	8	duplicate GDS	CYCL/00006 4
RMM.....G0004V00	NS0014	2009/215	prim	7	CYCL/00005 4	
RMM.....G0005V00	NS0005	2009/228	next	4	duplicate GDS	CYCL/00006 2
RMM.....G0005V00	NS0015	2009/229	prim	3	CYCL/00005 2	
RMM.....G0006V00	NS0006	2009/235	next	2	duplicate GDS	CYCL/00006 1
RMM.....G0006V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.4 CYCLEBY GENERATION and DUPLICATE BUMP with generation wrap

Now we have simulated the generation wrap up, that means we have reached the maximum count of 9999 of a generation data set and the next generation is created with one.

In our third test case we have create 12 files like the files created before as you can see in Table 11-5 on page 411, but we have started by the generation data set G9997V00 so that we get a generation wrap. In Figure 11-31 on page 407 you can see the detail DSNAME VRS options and in Figure 11-32 on page 408 you can se the NAME VRS details.

Table 11-5 Created data set with a generation wrap to test new GDG parmlib option

DSNAME	VOLSER	CRDATE
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9997V00	NS0001	2009/200
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9997V00	NS0011	2009/201
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9998V00	NS0002	2009/207
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9998V00	NS0012	2009/208
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9999V00	NS0003	2009/221
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9999V00	NS0013	2009/222
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	NS0004	2009/214
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	NS0014	2009/215
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	NS0005	2009/228
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	NS0015	2009/229
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	NS0006	2009/235
RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	NS0016	2009/236

Figure 11-37 illustrates a generation wrap that means you have reached the maximum of 9999 for generation data set and the data set create gets the number 0001.

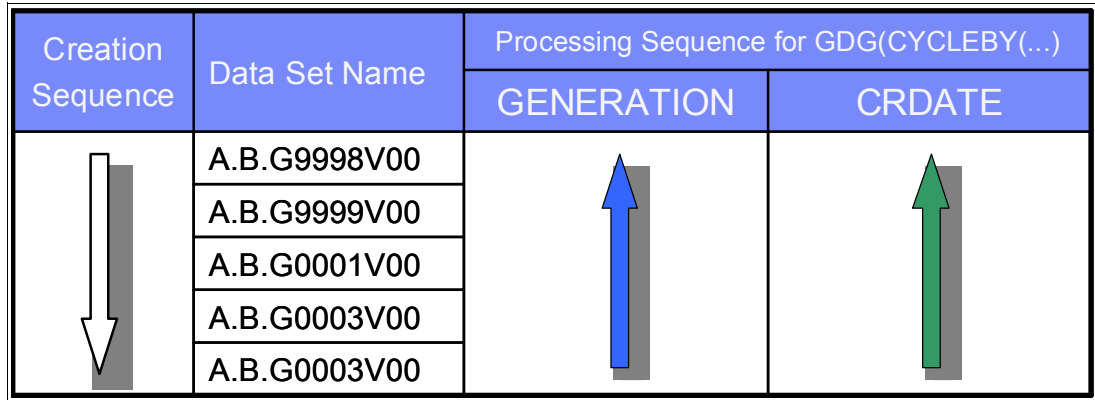


Figure 11-37 Generation wrap

As you can see that like the first test case always only the most actual generation data set is retained by the primary VRS definition and the older one not. Figure 11-38 on page 412 shows you all the data set retained in this test case.

REMOVABLE MEDIA MANAGER		VITAL RECORDS RETENTION REPORT										PAGE	1
Copyright IBM Corp. 1993,2007												TIME 14:08:14 DATE	2009/223
JOB MASK	DATA SET OR VOLUME MASK	OWNER	TYPE	RETN	C X	DELETE	DLY	COUNT	STNUM	LOCATION	RLSE	LASTREF	
	RMM.TEST.NEW.GDG.PARMLIB.OPTION.**	SCHLUM	DSN	CYCLES	N N	1999/365	0	5	5	HOME		2009/223	
	GDGOPT	MHLRES7	NEXT	CYCLES	N N	1999/365		6	6	HOME		2009/223	
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT	REQUIRED	PRTY	RETDATE		RETNAME	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0016	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0006	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0015	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0005	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0014	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0004	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9999V00	1	1	NS0013	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9999V00	1	1	NS0003	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9998V00	1	1	NS0012	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9998V00	1	1	NS0002	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT		
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9997V00	1	1	NS0011	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT		
NUMBER OF DATA SETS RETAINED (GROUP STORE) =										11	0		

Figure 11-38 Result of test case three

In Table 11-6 you can see the drop reason and why a data set is retained.

Table 11-6 Details of test case three

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G9997V00	NS0001	2009/200	no	12	cycles exceeded	duplicate GDS
RMM.....G9997V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 6
RMM.....G9998V00	NS0002	2009/207	next	10	duplicate GDS	CYCL/00006 5
RMM.....G9998V00	NS0012	2009/208	prim	9	CYCL/00005 5	
RMM.....G9999V00	NS0003	2009/221	next	8	duplicate GDS	CYCL/00006 4
RMM.....G9999V00	NS0013	2009/222	prim	7	CYCL/00005 4	
RMM.....G0001V00	NS0004	2009/214	next	6	duplicate GDS	CYCL/00006 3
RMM.....G0001V00	NS0014	2009/215	prim	5	CYCL/00005 3	

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0005	2009/228	next	4	duplicate GDS	CYCL/00006 2
RMM.....G0001V00	NS0015	2009/229	prim	3	CYCL/00005 2	
RMM.....G0001V00	NS0006	2009/235	next	2	duplicate GDS	CYCL/00006 1
RMM.....G0001V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.5 CYCLEBY CRDATE and DUPLICATE BUMP with generation wrap

In our fourth test case we have used the same 12 files as before, but we have changed the GDG option from CYCLESBY(GENERATION) to CYCLESBY(CRDATE). As you can see in Figure 11-39 on page 413 the generation data set G9999V00 is retained before generation data set G0001V00 depend on the creation date.

REMOVABLE MEDIA MANAGER Copyright IBM Corp. 1993,2007		VITAL RECORDS RETENTION REPORT					PAGE 1	
JOB MASK	DATA SET OR VOLUME MASK	OWNER	TYPE	RETN	C X	DELETE	DLY COUNT	STNUM LOCATION RLSE
								TIME 14:08:14 DATE 2009/223
								LASTREF
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT	REQUIRED PRY RETDATE RETNAME
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0016	1	SCHLUM	SHELF	LIB2 4800 CYCL/00005 * 2009/223
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0006	1	SCHLUM	SHELF	LIB2 4800 CYCL/00006 GDG0PT 2009/223
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0015	1	SCHLUM	SHELF	LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0005	1	SCHLUM	SHELF	LIB2 4800 CYCL/00006 GDG0PT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9999V00	1	1	NS0013	1	SCHLUM	SHELF	LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9999V00	1	1	NS0003	1	SCHLUM	SHELF	LIB2 4800 CYCL/00006 GDG0PT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0014	1	SCHLUM	SHELF	LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0004	1	SCHLUM	SHELF	LIB2 4800 CYCL/00006 GDG0PT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9998V00	1	1	NS0012	1	SCHLUM	SHELF	LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9998V00	1	1	NS0002	1	SCHLUM	SHELF	LIB2 4800 CYCL/00006 GDG0PT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G9997V00	1	1	NS0011	1	SCHLUM	SHELF	LIB2 4800 CYCL/00006 GDG0PT
NUMBER OF DATA SETS RETAINED (GROUP STORE) =								11 0

Figure 11-39 Result of test case four

In Table 11-7 you can see the drop reason and why a data set is retained. As you can see generation data set G9999V00 is retained before generation data set G0001V00 depend on the creation date.

Table 11-7 Details of test case four

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G9997V00	NS0001	2009/200	no	12	cycles exceeded	duplicate GDS
RMM.....G9997V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 6
RMM.....G9998V00	NS0002	2009/207	next	10	duplicate GDS	CYCL/00006 5
RMM.....G9998V00	NS0012	2009/208	prim	9	CYCL/00005 5	
RMM.....G9999V00	NS0003	2009/221	next	6	duplicate GDS	CYCL/00006 3
RMM.....G9999V00	NS0013	2009/222	prim	5	CYCL/00005 3	

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0004	2009/214	next	8	duplicate GDS	CYCL/00006 4
RMM.....G0001V00	NS0014	2009/215	prim	7	CYCL/00005 4	
RMM.....G0001V00	NS0005	2009/228	next	4	duplicate GDS	CYCL/00006 2
RMM.....G0001V00	NS0015	2009/229	prim	3	CYCL/00005 2	
RMM.....G0001V00	NS0006	2009/235	next	2	duplicate GDS	CYCL/00006 1
RMM.....G0001V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.6 CYCLEBY GENERATION and DUPLICATE DROP

In our fifth test we have change the GDG parmlib option to GDG(CYCLEBY(GENERATION) DUPLICATE(DROP)) and you can see in Figure 11-40 on page 414 that only the newer version of each generation data set is retained. Depend on the DROP option we have specified the older versions are dropped by cycles exceeded.

REMOVABLE MEDIA MANAGER		VITAL RECORDS RETENTION REPORT				PAGE 1	
Copyright IBM Corp. 1993,2007						TIME 14:09:45 DATE 2009/223	
JOB MASK	DATA SET OR VOLUME MASK	OWNER	TYPE	RETN	C X	DELETE	DLY COUNT STNUM LOCATION RLSE LASTREF
RMM.TEST.NEW.GDG.PARMLIB.OPTION.**	SCHLUM	DSN	CYCLES	N N	1999/365	0	5 5 HOME 2009/223
GDGOPT	MHLRES7	NEXT	CYCLES	N N	1999/365	6	6 HOME 2009/223
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT REQUIRED PRTY RETDATE RETNAME
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0016	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0015	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0014	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0013	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0012	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0011	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
NUMBER OF DATA SETS RETAINED (GROUP STORE) =							6 0

Figure 11-40 Result of test case five

In Table 11-8 you can see the drop reason and why a data set is retained.

Table 11-8 Details of test case five

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0001	2009/200	no	12	cycles exceeded	duplicate GDS
RMM.....G0001V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 1
RMM.....G0002V00	NS0002	2009/207	no	10	duplicate GDS	
RMM.....G0002V00	NS0012	2009/208	prim	9	CYCL/00005 5	
RMM.....G0003V00	NS0003	2009/221	no	8	duplicate GDS	
RMM.....G0003V00	NS0013	2009/222	prim	7	CYCL/00005 4	
RMM.....G0004V00	NS0004	2009/214	not	6	duplicate GDS	
RMM.....G0004V00	NS0014	2009/215	prim	5	CYCL/00005 3	

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0005V00	NS0005	2009/228	no	4	duplicate GDS	
RMM.....G0005V00	NS0015	2009/229	prim	3	CYCL/00005 2	
RMM.....G0006V00	NS0006	2009/235	no	2	duplicate GDS	
RMM.....G0006V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.7 CYCLEBY CRDATE and DUPLICATE DROP

In our sixth test we have change the GDG parmlib option to GDG(CYCLEBY(GENERATION) DUPLICATE(DROP)) and you can see in Figure 11-41 on page 415 that only the newer version of each generation data set is retained on the DROP specification. Depend on the DROP option we have specified the older versions are dropped by cycles exceeded, but generation data set 4 will be retained before generation data set 3.

REMOVABLE MEDIA MANAGER		VITAL RECORDS RETENTION REPORT						PAGE		1		
Copyright IBM Corp. 1993,2007								TIME 14:09:45 DATE		2009/223		
JOB MASK	DATA SET OR VOLUME MASK	OWNER	TYPE	RETN	C X	DELETE	DLY	COUNT	STNUM	LOCATION	RLSE	LASTREF
	RMM.TEST.NEW.GDG.PARMLIB.OPTION.**	SCHLUM	DSN	CYCLES	N N	1999/365	0	5	5	HOME		2009/223
	GDGOPT	MHLRES7	NEXT	CYCLES	N N	1999/365		6	6	HOME		2009/223
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT	REQUIRED	PRTY	RETDTE	RETNAME	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0016	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0015	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0013	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0014	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0012	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*	
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0011	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT	
NUMBER OF DATA SETS RETAINED (GROUP STORE) =											6	0

Figure 11-41 Result of test case six

In Table 11-9 you can see the drop reason and why a data set is retained. As you can see generation data set G0003V00 is retained before generation data set G0004V00 depend on the creation date.

Table 11-9 Details of test case six

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0001	2009/200	no	12	cycles exceeded	duplicate GDS
RMM.....G0001V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 1
RMM.....G0002V00	NS0002	2009/207	no	10	duplicate GDS	
RMM.....G0002V00	NS0012	2009/208	prim	9	CYCL/00005 5	
RMM.....G0003V00	NS0003	2009/221	no	6	duplicate GDS	
RMM.....G0003V00	NS0013	2009/222	prim	5	CYCL/00005 4	
RMM.....G0004V00	NS0004	2009/214	no	8	duplicate GDS	
RMM.....G0004V00	NS0014	2009/215	prim	7	CYCL/00005 3	

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0005V00	NS0005	2009/228	no	4	duplicate GDS	
RMM.....G0005V00	NS0015	2009/229	prim	3	CYCL/00005 2	
RMM.....G0006V00	NS0006	2009/235	no	2	duplicate GDS	
RMM.....G0006V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.8 CYCLEBY GENERATION and DUPLICATE KEEP

In our seventh test we have change the GDG parmlib option to GDG(CYCLEBY(GENERATION) DUPLICATE(KEEP)) and you can see in Figure 11-42 on page 416 that both versions of the current five generation data sets are retained by the primary VRS definition and that both versions of the oldest generation data sets are retained by the next VRS definition GDGOPT.

REMOVABLE MEDIA MANAGER Copyright IBM Corp. 1993,2007		VITAL RECORDS RETENTION REPORT				PAGE 1	
JOB MASK	DATA SET OR VOLUME MASK	OWNER	TYPE	RETN	C X	DELETE	DLY COUNT STNUM LOCATION RLSE LASTREF
	RMM.TEST.NEW.GDG.PARMLIB.OPTION.**	SCHLUM	DSN	CYCLES	N N	1999/365	0 5 5 HOME 2009/223
	GDGOPT	MHLRES7	NEXT	CYCLES	N N	1999/365	6 6 HOME 2009/223
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT REQUIRED PRTY RETDATE RETNAME
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0016	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0006	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0015	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0005	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0014	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0004	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0013	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0003	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0012	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0002	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0011	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0001	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
NUMBER OF DATA SETS RETAINED (GROUP STORE) =							12 0

Figure 11-42 Result of test case seven

In Table 11-10 you can see that all 12 data sets are retained.

Table 11-10 Details of test case seven

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0001	2009/200	next	12	cycles exceeded	CYCL/00006 1
RMM.....G0001V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 1
RMM.....G0002V00	NS0002	2009/207	next	10	CYCL/00005 5	
RMM.....G0002V00	NS0012	2009/208	prim	9	CYCL/00005 5	
RMM.....G0003V00	NS0003	2009/221	prim	8	CYCL/00005 4	
RMM.....G0003V00	NS0013	2009/222	prim	7	CYCL/00005 4	
RMM.....G0004V00	NS0004	2009/214	prim	6	CYCL/00005 3	

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0004V00	NS0014	2009/215	prim	5	CYCL/00005 3	
RMM.....G0005V00	NS0005	2009/228	prim	4	CYCL/00005 2	
RMM.....G0005V00	NS0015	2009/229	prim	3	CYCL/00005 2	
RMM.....G0006V00	NS0006	2009/235	prim	2	CYCL/00005 1	
RMM.....G0006V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.9 CYCLEBY CREATION and DUPLICATE KEEP

In our eight test we have change the GDG parmlib option to GDG(CYCLEBY(CREATION) DUPLICATE(KEEP)) and you can see in Figure 11-43 on page 417 that both versions of the last five generation data sets are retained by the primary VRS definition and that both versions of the oldest generation data sets are retained by the next VRS definition GDGOPT, but generation data set 4 will be retained before generation data set 3 depend on the creation date.

REMOVABLE MEDIA MANAGER Copyright IBM Corp. 1993,2007		VITAL RECORDS RETENTION REPORT				PAGE 1	
JOB MASK DATA SET OR VOLUME MASK		-----				TIME 14:09:18 DATE 2009/223	
		OWNER	TYPE	RETN	C X	DELETE	DLY COUNT STNUM LOCATION RLSE LASTREF
RMM.TEST.NEW.GDG.PARMLIB.OPTION.**		SCHLUM	DSN	CYCLES	N N	1999/365	0 5 5 HOME 2009/223
GDGOPT		MHLRES7	NEXT	CYCLES	N N	1999/365	6 6 HOME 2009/223
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT REQUIRED PRTY RETDATE RETNAME
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0016	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0006	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0015	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0005	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0013	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0003	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0014	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0004	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0012	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0002	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0011	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0001	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
NUMBER OF DATA SETS RETAINED (GROUP STORE) =						12 0	

Figure 11-43 Result of test case eight

In Table 11-11 you can see that all 12 created data sets are retained. As you can see generation data set G0003V00 is retained before generation data set G0004V00 depend on the creation date.

Table 11-11 Details of test case eight

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0001	2009/200	next	12	cycles exceeded	CYCL/00006 1
RMM.....G0001V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 1
RMM.....G0002V00	NS0002	2009/207	prim	10	CYCL/00005 5	
RMM.....G0002V00	NS0012	2009/208	prim	9	CYCL/00005 5	

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0003V00	NS0003	2009/221	prim	6	CYCL/00005 3	
RMM.....G0003V00	NS0013	2009/222	prim	5	CYCL/00005 3	
RMM.....G0004V00	NS0004	2009/214	prim	8	CYCL/00005 4	
RMM.....G0004V00	NS0014	2009/215	prim	7	CYCL/00005 4	
RMM.....G0005V00	NS0005	2009/228	prim	4	CYCL/00005 2	
RMM.....G0005V00	NS0015	2009/229	prim	3	CYCL/00005 2	
RMM.....G0006V00	NS0006	2009/235	prim	2	CYCL/00005 1	
RMM.....G0006V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.10 CYCLEBY GENERATION and DUPLICATE COUNT

In our ninth test we have change the GDG parmlib option to GDG(CYCLEBY(GENERATION) DUPLICATE(COUNT)) and you can see in Figure 11-44 on page 418 that the current newest created 5 data sets are retained by the primary VRS definition and that additional 6 versions of the next current created generation data sets are retained by the next VRS definition GDGOPT and the oldest generation data set is dropped by cycles exceeded.

REMOVABLE MEDIA MANAGER		VITAL RECORDS RETENTION REPORT						PAGE		1	
Copyright IBM Corp. 1993,2007		-----						TIME 14:09:18 DATE		2009/223	
JOB MASK	DATA SET OR VOLUME MASK	OWNER	TYPE	RETN	C X	DELETE	DLY COUNT	STNUM	LOCATION	RLSE	LASTREF
	RMM.TEST.NEW.GDG.PARMLIB.OPTION.**	SCHLUM	DSN	CYCLES	N N	1999/365	0	5	5 HOME		2009/223
	GDGOPT	MHLRES7	NEXT	CYCLES	N N	1999/365		6	6 HOME		2009/223
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT	REQUIRED	PRTY	RETDATE	RETNAME
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0016	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0006	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0015	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0005	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0014	1	SCHLUM	SHELF	LIB2	4800	CYCL/00005	*
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0004	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0013	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0003	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0012	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0002	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0011	1	SCHLUM	SHELF	LIB2	4800	CYCL/00006	GDGOPT
NUMBER OF DATA SETS RETAINED (GROUP STORE) =									11	0	

Figure 11-44 Result of test case six

In Table 11-12 you can see that 11 data sets are retained.

Table 11-12 Details of test case nine

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0001	2009/200	no	12	cycles exceeded	cycles exceeded
RMM.....G0001V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 6
RMM.....G0002V00	NS0002	2009/207	next	10	cycles exceeded	CYCL/00006 5
RMM.....G0002V00	NS0012	2009/208	next	9	cycles exceeded	CYCL/00006 4

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0003V00	NS0003	2009/221	next	8	cycles exceeded	CYCL/00006 3
RMM.....G0003V00	NS0013	2009/222	next	7	cycles exceeded	CYCL/00006 2
RMM.....G0004V00	NS0004	2009/214	next	6	cycles exceeded	CYCL/00006 1
RMM.....G0004V00	NS0014	2009/215	prim	5	CYCL/00005 5	
RMM.....G0005V00	NS0005	2009/228	prim	4	CYCL/00005 4	
RMM.....G0005V00	NS0015	2009/229	prim	3	CYCL/00005 3	
RMM.....G0006V00	NS0006	2009/235	prim	2	CYCL/00005 2	
RMM.....G0006V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.5.11 CYCLEBY CREATION and DUPLICATE COUNT

In our tenth test we have change the GDG parmlib option to GDG(CYCLEBY(GENERATION) DUPLICATE(COUNT)) and you can see in Figure 11-45 on page 419 that the current newest created 5 data sets are retained by the primary VRS definition and that additional 6 versions of the next current created generation data sets are retained by the next VRS definition GDGOPT and the oldest generation data set is dropped by cycles exceeded. Depend on the creation date generation data set 4 will be retained before generation data set 3.

REMOVABLE MEDIA MANAGER		VITAL RECORDS RETENTION REPORT				PAGE 1	
Copyright IBM Corp. 1993,2007		-----				TIME 14:09:18 DATE 2009/223	
JOB MASK	DATA SET OR VOLUME MASK	OWNER	TYPE	RETN	C X	DELETE	DLY COUNT STNUM LOCATION RLSE LASTREF
RMM.TEST.NEW.GDG.PARMLIB.OPTION.**		SCHLUM	DSN	CYCLES	N N	1999/365	0 5 5 HOME 2009/223
GDGOPT		MHLRES7	NEXT	CYCLES	N N	1999/365	6 6 HOME 2009/223
JOB NAME	DATA SET NAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT REQUIRED PRTY RETDATE RETNAME
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0016	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0006V00	1	1	NS0006	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0015	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0005V00	1	1	NS0005	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0013	1	SCHLUM	SHELF LIB2 4800 CYCL/00005 *
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0003V00	1	1	NS0003	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0014	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0004V00	1	1	NS0004	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0012	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0002V00	1	1	NS0002	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
TESTGDG	RMM.TEST.NEW.GDG.PARMLIB.OPTION.G0001V00	1	1	NS0011	1	SCHLUM	SHELF LIB2 4800 CYCL/00006 GDGOPT
NUMBER OF DATA SETS RETAINED (GROUP STORE) =							11 0

Figure 11-45 Result of test case ten

In Table 11-13 you can see that 11 data sets are retained. As you can see generation data set G0003V00 is retained before generation data set G0004V00 depend on the creation date.

Table 11-13 Details of test case ten

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0001V00	NS0001	2009/200	no	12	cycles exceeded	cycles exceeded
RMM.....G0001V00	NS0011	2009/201	next	11	cycles exceeded	CYCL/00006 6

DSNAME	VOLSER	CRDATE	VRS	Cycle no	Reason for (non) retention	
					primary	next
RMM.....G0002V00	NS0002	2009/207	next	10	cycles exceeded	CYCL/00006 5
RMM.....G0002V00	NS0012	2009/208	next	9	cycles exceeded	CYCL/00006 4
RMM.....G0003V00	NS0003	2009/221	next	6	cycles exceeded	CYCL/00006 3
RMM.....G0003V00	NS0013	2009/222	prim	5	CYCL/00005 5	
RMM.....G0004V00	NS0004	2009/214	next	8	cycles exceeded	CYCL/00006 2
RMM.....G0004V00	NS0014	2009/215	next	7	cycles exceeded	CYCL/00006 1
RMM.....G0005V00	NS0005	2009/228	prim	4	CYCL/00005 4	
RMM.....G0005V00	NS0015	2009/229	prim	3	CYCL/00005 3	
RMM.....G0006V00	NS0006	2009/235	prim	2	CYCL/00005 2	
RMM.....G0006V00	NS0016	2009/236	prim	1	CYCL/00005 1	
next these data sets are retained by the next VRS definition						

11.6 Managing DFSMSHsm tapes

The interface between DFSMSHsm and DFSMSrmm is automatically in place when you install DFSMS, so you do not need to take any action to activate it, or to call it.

You can use DFSMSrmm PARMLIB OPTION TVEXTPURGE operand to control the processing that DFSMSHsm performs when it releases a volume. You can set this volume to pending release or set the volume expiration date to the current date. These are the different options for this operand:

► NONE

DFSMSrmm takes no action for volumes to be purged.

► RELEASE

DFSMSrmm releases volume to be purged according to the release actions set for the volume. You must run expiration processing to return a volume to scratch status.

Example 11-1 Adding a VRS to retain DFSMSHsm managed data sets

```
RMM ADDVRS DSNAME('mprefix.HMIGTAPE.DATASET') COUNT(99999) CYCLES
RMM ADDVRS DSNAME('mprefix.BACKTAPE.DATASET') COUNT(99999) CYCLES
RMM ADDVRS DSNAME('mprefix.COPY.HMIGTAPE.DATASET') COUNT(99999) CYCLES
RMM ADDVRS DSNAME('mprefix.COPY.BACKTAPE.DATASET') COUNT(99999) CYCLES
```

These VRSs protect all your volumes until DFSMSHsm releases the tapes; in this case the volumes are set to pending release, and with the next expiration processing, to scratch.

► EXPIRE(days)

Use the EXPIRE(days) option to set the volume expiration date to the current date plus days for volumes to be purged. Use the EXPIRE(days) parameter when you use the EXPDT retention method, using days as a way to delay expiration of the volume. When using the VRSEL retention method, you can optionally use this operand in combination with vital record specifications that use the UNTILEXPIRED retention type. First, the EXPIRE(days) is applied to set a new volume EXPDT. Next, by running VRSEL you can

then optionally extend retention using the extra days retention type. For example, specify EXPIRE(0) when using a VRS with extra days retention or you can use a non-zero EXPIRE(days) value and avoid using an extra days retention VRS.

days is the number of days for which DFSMSrmm retains the volume before considering it for release. The value is a one to four digit decimal number and is added to today's date to compute the new expiration date. If the value exceeds the maximum retention period (MAXRETPD) it is reduced to the MAXRETPD value. The default value for days is 0. That is, TVEXTPURGE(EXPIRE) is the same as TVEXTPURGE(EXPIRE(0)).

11.7 DFSMS ACS support

Before OS/390 V2R10, you could assign a management value to the data set created on a any tape volume. For doing this, DFSMSrmm calls to EDGUX100 exit at O/C/EOV time to assign a MV to the data set. During VRSEL processing, this MV assigned to the data set is used to assign a VRS. Figure 11-46 on page 421 shows this process. EDGUX100 also can be used to modify the WTO mount message.

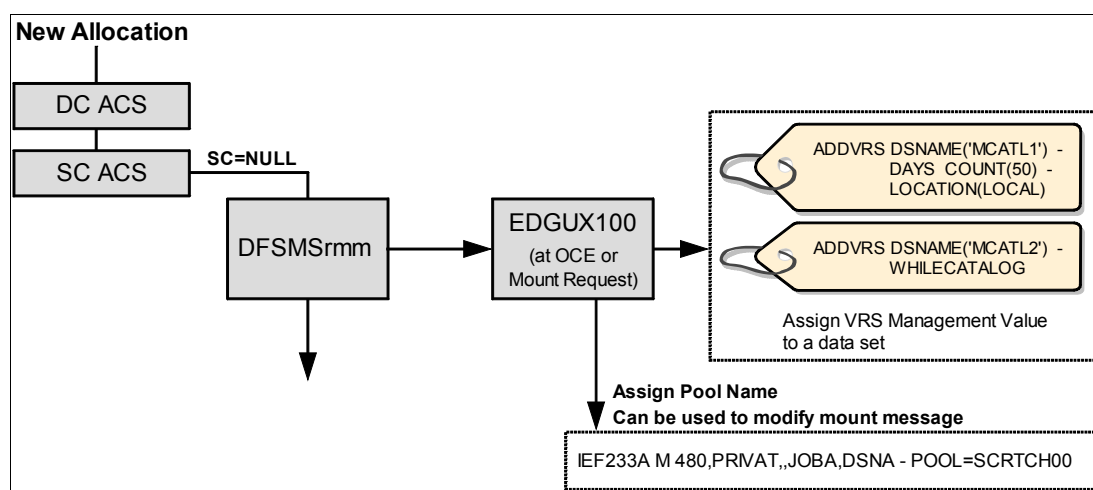


Figure 11-46 Support before OS/390 V2R10 for non-system-managed tapes

Some enhancements were introduced with DFSMSdftp prior to OS/390 R10 to add pre-ACS support using the SMS exit IGDACSXT, which gets control before the ACS routines. On OS/390 R10 DFSMSrmm added both pre-ACS support to support the DFSMSdftp processing, and DFSMSrmm ACS support to enable you to use the management class for VRS policy management.

For more details about this function and how to implement this enhancement, refer to Chapter 12, “DFSMSrmm ACS support” on page 447.

11.8 Modifying VRS

All the modifications to VRS definitions can be made by TSO subcommands or by DFSMSrmm ISPF panels. Here we use only DFSMSrmm ISPF panels. To see a description of TSO subcommands, refer to the *DFSMSrmm Guide and Reference*, SC26-7404.

DFSMSrmm does not provide a RMM TSO CHANGEVRS subcommand for modifying an existing VRS. You can use a combination of DELETEVRS and ADDVRS subcommands for this. Another way is to use the ISPF dialog.

11.9 Deleting VRS

Before you delete a VRS, you must be sure that you want all the data set or volumes protected by this VRS to be released. You can verify this in the DFSMSrmm VRSs panel (Figure 11-47 on page 422). Select **M** before the VRS you are planning to delete and press Enter.

DFSMSrmm VRSs (Page 1 of 3)				Row 1 to 10 of 10	
Command ==>				Scroll ==> CSR	
The following line commands are valid: C, D, L, M, O, S.					
Use the RIGHT command to view other data columns.					
S	Volume/Data set/Name specification	Job name	Type	Location	Priority
-	-----	-----	-----	-----	-----
	*		DSN	HOME	0
	D99000		DSN	HOME	0
	M98031		DSN	HOME	0
	OPEN		DSN	HOME	0
	SAFE		DSN	REMOTE	0
M	YCJRES%.T*.*		DSN	HOME	0
	YCJRES1.CAT*.**		DSN	HOME	0

Figure 11-47 DFSMSrmm VRSs panel

DFSMSrmm shows all the data sets matched by this VRS (Figure 11-48) and you can check if you want to release all of them. The listed data sets may be retained by other more specific VRSs; you can check which VRSs retains a data set by using the **L** line command from the returned list.

DFSMSrmm Data Sets				Row 1 to 4 of 4	
Command ==>				Scroll ==> CSR	
The following line commands are valid: C, D, I, L, O, V.					
S	Data set name	Volume Serial	Owner	File Create Seq	Date
-	-----	-----	-----	-----	-----
	YCJRES1.TEST1.VRS2	TST016	YCJRES1	1	2001/059
	YCJRES1.TEST1.VRS2	TST023	YCJRES1	1	2001/060

Figure 11-48 DFSMSrmm Data Sets panel

To delete the VRS, you can use the ISPF dialog, or use the DELETEVRS command.

11.10 DFSMSrmm VRS policy management simplification

In this new release the VRS processing changed and the RMM TSO subcommand and the reporting are enhanced as following:

1. Separation of Data Set Name Mask from the Policy
2. Release Options Applied if VRS Matched
3. Special ABEND, DELETED and OPEN via DSNAME match
4. Find unused VRSs

5. Incomplete VRS chains - dummy VRS '*broken*'
6. Toleration and removal of really old functions.
7. Conversion to DFSMSrmm from other tape management systems

Attention: The DFSMSrmm parmlib option VRSEL(OLD) will be removed in a future release. The recommendation is to migrate from VRSEL(OLD) to VRSEL(NEW) before moving to z/OS V1R8. Each time you run VRSEL processing and VRSEL(OLD) is in use the new message EDG2317E, and a minimum return code of 4 are issued

11.10.1 Separation of Data Set Name Mask from the Policy

Figure 11-49 shows the structure of a DSNAME VRS and how the concept of separating the data set name mask from the policy itself is put into effect.

The Data Set name VRS now allows a COUNT of zero so that this first VRS in a chain has no retention specification. The NEXTVRS in the chain and subsequent VRSs now specify the entire policy

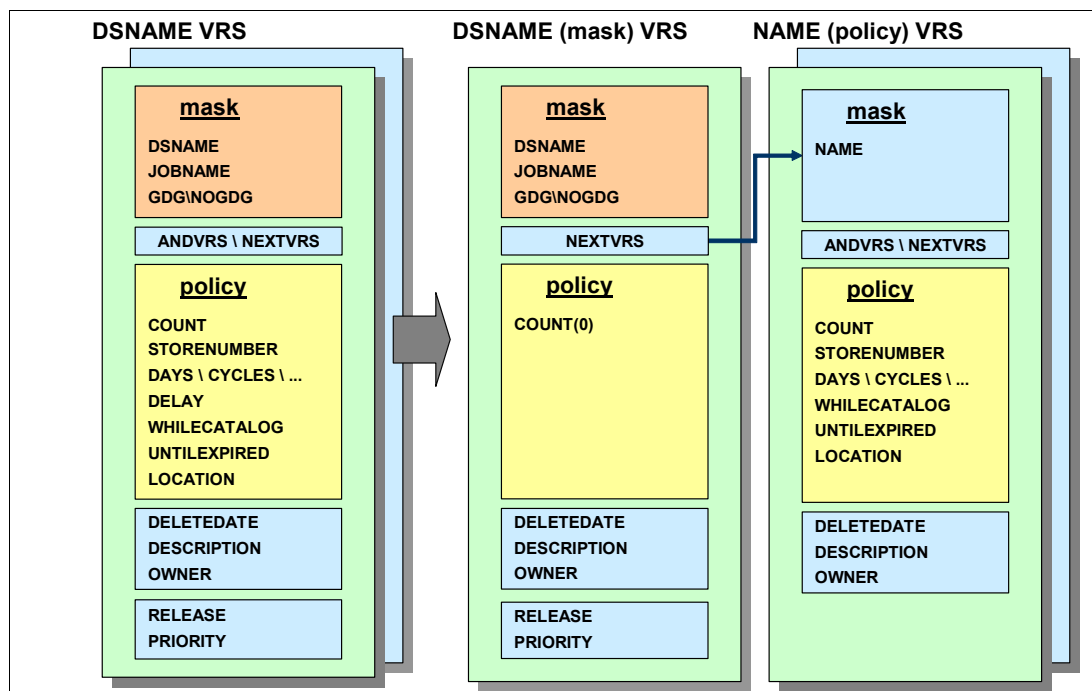


Figure 11-49 DSNAME VRS concept

Setting up DFSMSrmm retention and movement policies can require large numbers of VRSs to be created because each data set name mask VRS also contains the initial retention and movement information.

Separating the data set name mask from the policy itself as shown in Figure 11-50 on page 424 enables clear and well-defined service levels to be set up for tape management.

These policy/service-level VRSs can then be easily modified as required without changing the filters that select them.

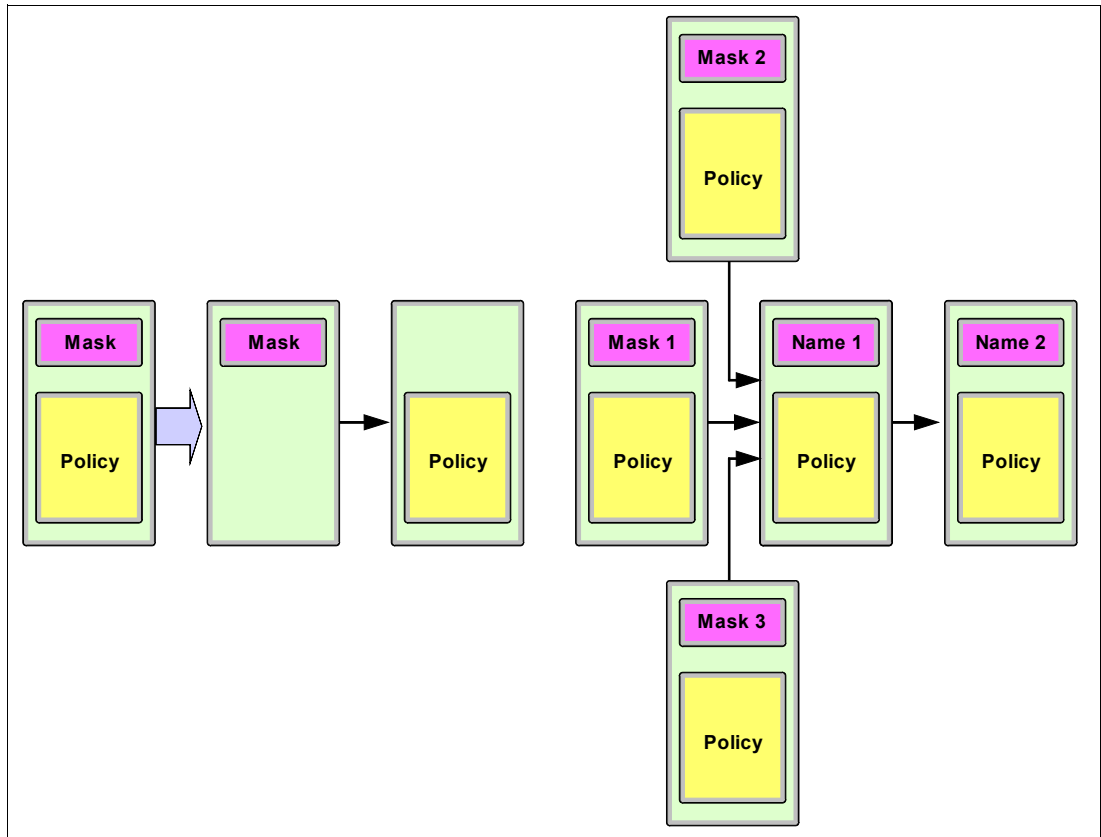


Figure 11-50 Separation of DSNAME mask from the policy

Note: VRSEL(NEW) is required to implementing the “Separating the data set name mask from the policy” processing.

If the customer runs DFSMSrmm with VRSEL(OLD), for each VRS where COUNT is set to 0 DFSMSrmm will issue the message EDG2225I (see chapter ‘New functions only supported with VRSEL(NEW)’); the processing continues, DFSMSrmm sets a minimum return code of 4, and ignores the VRS; no data sets can match to or use the VRS.

11.10.2 Release Options Applied if VRS Matched

Release options for matching VRSs can be applied regardless of whether the data set actually is ever VRS retained.

The combination of COUNT(0) and release option changes ensures a Volume can be scratched the same day a Data Set on it is created. See Figure 11-51 on page 425 that shows you how you can specify COUNT(0) in a DSNAME VRS definition.

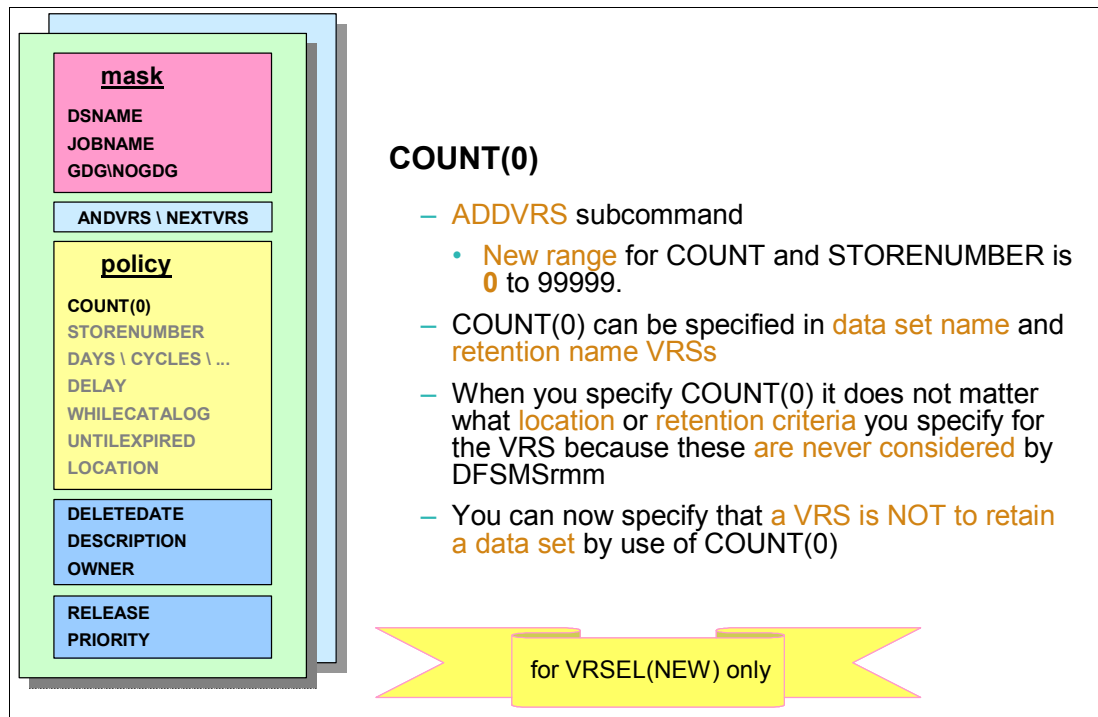


Figure 11-51 VRS count zero processing

When a data set is no longer retained by a vital record specification, DFSMSrmm releases the volume on which the data set resides only if no data set and the volume is retained by a vital record specification. If you use the DFSMSrmm EDGRMMxx parmlib OPTION command VRSEL(NEW) option and the RMM ADDVRS RELEASE(EXPIRYDATEIGNORE) operand, DFSMSrmm ignores the volume expiration date and uses information in a vital record specification to control retention. There are two special RELEASE options available and you can select one or both of these options together:

- ▶ EXPIRYDATEIGNORE
- ▶ SCRATCHIMMEDIATE

DFSMSrmm does not immediately return a volume to scratch status or to its owner when a volume reaches its expiration date and is not retained by a vital record specification. You must run expiration processing two times to return a volume to scratch status or to its owner. The first run of expiration processing sets the volume status to pending release. The second run of expiration processing completes the return. Running expiration processing two times give you time to make changes to the volume status before the volume is released.

Note: Sometimes DFSMSrmm cannot make the return in a single run, for example there may be other release actions required.

If you do not need to run expiration processing in two runs, specify the DFSMSrmm EDGRMMxx parmlib OPTION command VRSEL(NEW) option and the RMM ADDVRS RELEASE(SCRATCHIMMEDIATE) operand. This enables you to return volumes to scratch in a single run of expiration processing. For more information about the RELEASE option refer to the *DFSMSrmm Implementation and Customization Guide*, SC26-7405

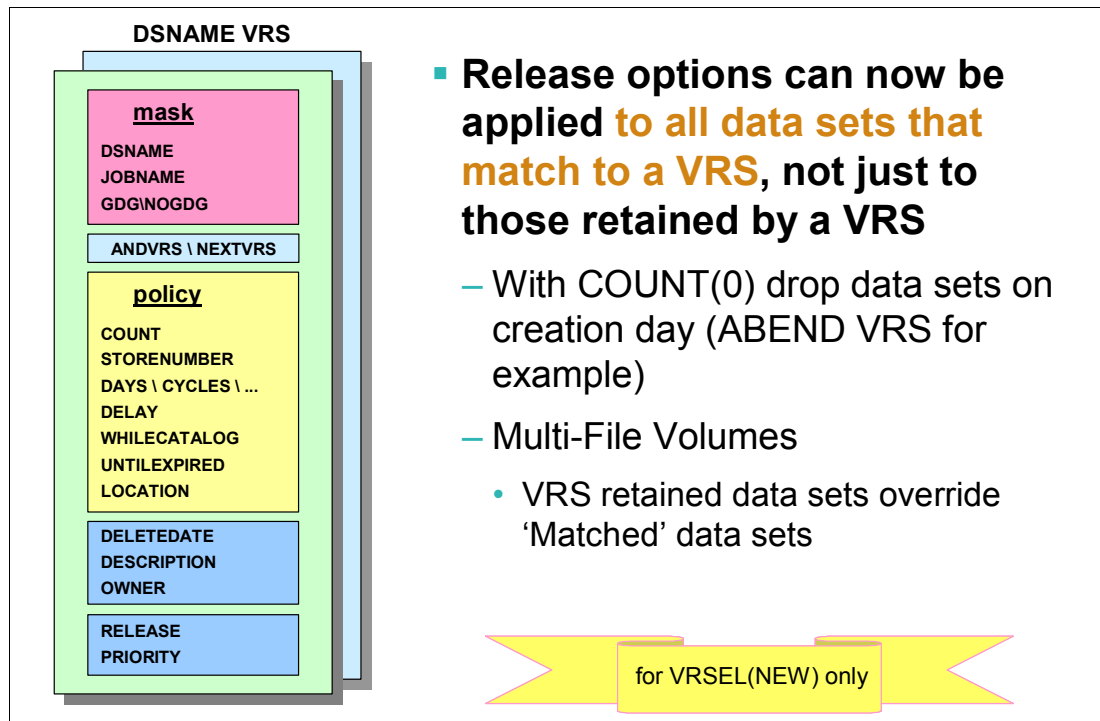


Figure 11-52 VRS release option

11.10.3 Special ABEND, DELETED and OPEN via DSNAME match

The support for ABEND and OPEN is extended to allow selection of the appropriate policy using the data set name mask. You can use the reserved data set or job names ABEND and OPEN to specify policies for:

- ▶ data sets closed as a result of an abnormal end (ABEND flag in the data set record ON) in a task
- ▶ data sets that are left open (OPEN flag in the volume record ON) or are in use during inventory management.

Figure 11-53 shows you how you can specify the new reserved ABEND and OPEN masks in the ADD VRS subcommand.

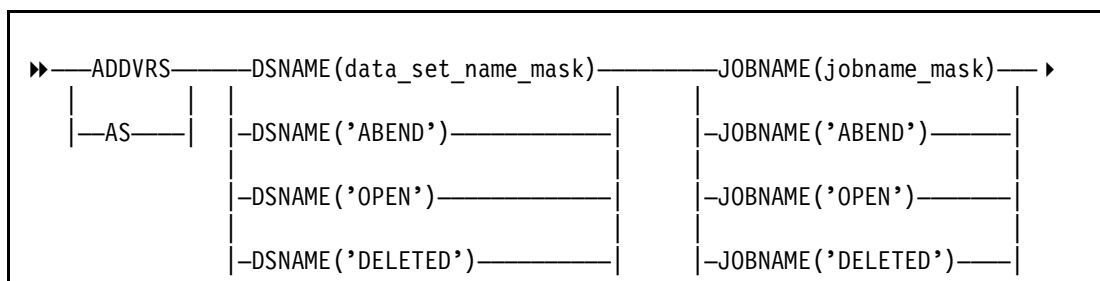


Figure 11-53 Add new ABEND or OPEN VRS

This allows you to use either data set name masks or job name masks, to manage open or abend data sets.

The data set name mask or job name can even be used to match to data sets via management class or vital record specification management value as long as the data set name mask specified is not more than a single qualifier.

11.10.4 Find unused VRSs

Now DFSMSrmm helps you manage your VRSs by VRSEL maintains last reference date (DLR) and last reference time (TLR) for each VRS. This new function includes following new functions too:

- ▶ counts the number of unused VRSs
- ▶ identifies which VRS policy chains are not being used

You can use these information to identify and delete VRSs which are no longer required.

The VRS last reference date and time is now externalized in the report extract file, in the output of the LISTVRS subcommand and the last reference date is also shown in the REPORT file as shown in Figure 11-54. Also the REPORT file will contain a list of VRS chains not used in this VRSEL run

VITAL RECORDS RETENTION REPORT

REMOVABLE MEDIA MANAGER

VIT

PAGE

120

(C) IBM CORPORATION 1993,2006

TIME 11:21:02 DATE 03/22/2006

JOB MASK DATA SET OR VOLUME MASK

OWNER

TYPE RETN

C X

DELETE

DLY COUNT STNUM

LOCATION RLSE

LASTREF

.GDG..ICRMT.**

X002

DSN

CYCLES Y N 12/31/1999

0 99999 99999 LOCAL

03/2

JOB NAME DATA SET NAME

2ndVRS

2ndNAMED FSEQ

DSEQ

VOLUME VSEQ OWNER

CURRENT

REQUIRED PRTY

RETDAT

RE

X015IJIC DT04.GDG.DSND806.ICRMT.SYSSTR.G0001V00

12

12 L01699

1 X015

LOCAL

LOCAL

300

WHILECATLG *

NUMBER OF DATA SETS RETAINED (GROUP STORE) =

1

1

X015IJIC DT04.GDG.DSND806.ICRMT.SYSUSER.G0001V00

11

11 L01699

1 X015

LOCAL

LOCAL

300

WHILECATLG *

NUMBER OF DATA SETS RETAINED (GROUP STORE) =

1

1

X015IJIC DT04.GDG.DSND806.ICRMT.SYSVIEWS.G0001V00

13

13 L01699

1 X015

LOCAL

LOCAL

300

WHILECATLG *

NUMBER OF DATA SETS RETAINED (GROUP STORE) =

UNUSED VRS CHAINS REPORT

REMOVABLE MEDIA MANAGER

UNUS

PAGE

121

(C) IBM CORPORATION 1993,2006

TIME 11:21:02 DATE 03/22/2006

JOB MASK DATA SET OR VOLUME MASK

OWNER

TYPE RETN

C X

DELETE

DLY COUNT STNUM

LOCATION RLSE

LASTREF

*.BACKUP.**

LIB

DSN

CYCLES Y N 12/31/1999

0 99999 99999 LOCAL

01/2

*.IMGCPY.IMSDB.**

LIB

NAME

DAYS N Y 12/31/1999

0 99999 99999 HOME

12/1

VITAL.**

LIB

DSN

CYCLES Y N 12/31/1999

0 99999 99999 CURRENT

IX

05/2

LASTREF

01/23/1992

12/12/2004

05/27/2003

Figure 11-54 Vital records retention report

The MESSAGE file will contain an additional message that shows the number of VRSs which are not used by this VRSEL run.

The maintenance of the VRS last reference date and time is supported only if you have specified VRSEL(NEW). In the REPORT file LASTREF column is listed in any case:

- VRSEL(NEW) it will be filled with dates
- VRSEL(OLD) it will be filled with blanks)

11.10.5 VRS last reference date

After some time the number of vital record specifications (VRS) may be grown to a number that is hard to comprehend, especially for no longer used VRSeS. Now with DFSMSrmm V1R13, users can list a VRS based on the last reference date.

The SEARCHVRS subcommand is used to create a list of vital record specifications. Figure 11-55 shows an overview of the SEARCHVRS subcommand with the two new operands, LASTREFDATE and LASTCHANGAEDATE. Use the start_date operand to specify a date range for your search. A date_range consists of a start date and an end date. Each date can be an absolute date in either yyyy/ddd or yyddd format, or it can be a relative value from which DFSMSrmm calculates the date.

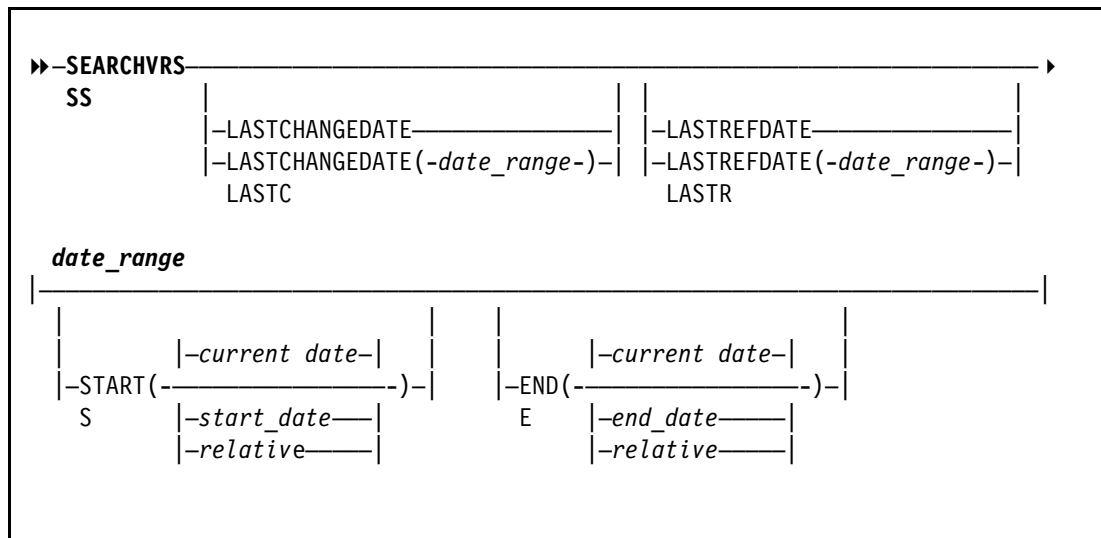


Figure 11-55 SEARCHVRS subcommand using the LASTREFDATE and LASTCHANGEDATE

Where:

LASTCHANGEDATE(date_range)

Lists the vital record specifications whose last changed date matches the specified date criteria. The date criteria can be specified using the START and END suboperands.

LASTREFDATE(date_range)

Lists the vital record specifications based on their last reference date. The last reference date of the vital record specification record is the date of the last inventory management VRSEL run that used this VRS to retain a data set or volume.

date_range

to lists the data sets whose creation date matches the specified date. In the following list *nn* can be one of:

- LASTCHANGEDATE
- LASTREFDATE

The default is the current date.

ccccDATE

Only data sets whose creation date is the current date are listed

ccccDATE(START)

Only data sets whose creation date is the current date are listed

ccccDATE(START())

Only data sets whose creation date is the current date are listed

ccccDATE(START(start_date))

Only data sets whose creation date is on or after the specified

start date are listed, where start_date is either an absolute date or relative date.

ccccDATE(END) Only data sets whose creation date is the current date are listed

ccccDATE(END()) Only data sets whose creation date is the current date are listed

ccccDATE(END(end_date))

Only data sets whose creation date is on or before the specified end date are listed, where end_date is either an absolute date or relative date. Note that because START defaults to the current date, the specified end date equal to or greater than the current date when START is omitted.

CRDATE(START(start_date)END(end_date))

Only data sets whose creation date is within the range delimited by the specified start and end dates are listed, where both start_date and end_date are either an absolute date or relative date. The specified end date equal to or greater than the specified start date.

Each of the start_date and end_date values can be absolute or relative dates.

Absolute dates are specified as either *yyyy/ddd* or *yyddd* format. For example, January 3, 2011 may be specified as 2011/003 or 11003.

Relative Dates are specified as a number of days, months, or years prior to the current date.

-0 specifies the current day, current month, current year.

-n specifies that the date is n days before the current date

-nM specifies that the date is n months before the current month and the current day in the month is as the current date.

-nY specifies that the date is n years before the current year and the current day in the year is as the current date.

The value range for n is 0 to 99999, with a required leading “-” and an optional suffix of M or Y.

Examples to list data sets whose creation date is:

Today specify SD CRDATE

Three days ago specify SD CRDATE(START(-3) END(-3))

Before January 1, 2000 specify SD CRDATE(START(0000/001) END(1999/365))

On or after January 2, 2005 Specify
SD CRDATE(START(2005/002))

RMM ISPF panel updates

Figure 11-56 on page 430 shows the RMM ISPF panel, Display Data Set VRS, with the new fields.

Panel Help	

DFSMSrmm Display Data Set VRS	
Command ==>	
Data set mask . . . : 'SCHLUM.RMMTEST.MOVE.**'	GDG . . : NO
Job name mask . . . :	
Count : 99999	Retention type : CYCLES
	While cataloged : YES
Delay : 0 Days	Until expired : NO
Location : REMOTE	
Number in location : 1	
Priority : 0	
	Release options:
Next VRS in chain . . :	Expiry date ignore . . . : NO
Chain using . . . :	Scratch immediate : NO
Owner : SCHLUM	
Description . . . : RETAIN AND MOVE DATA SETS	
Last reference : 2011/326 13:26:39 (YYYY/DDD HH:MM:SS)	
Delete date . . : 1999/365 (YYYY/DDD)	
Last Change information:	
Date : 2011/326	Time . . : 13:26:40 System : SC70
User change date : 2010/289	Time . . : 20:45:17 User ID : MHLRES7

Figure 11-56 DFSMSrmm VRS Display Data Set VRS panel

Where:

Last Reference display the date and time of the last inventory management VRSEL run which used the VRS to retain a data set or volume.

Last change information display details of the most recent change to the record.

Date displays the last change date.

Time displays the last change time.

System displays the ID of the system on which the last change occurred.

User change date displays the date of the most recent change by a user, other than by a DFSMSrmm internal function.

User change time displays the time of the most recent change by a user, other than by a DFSMSrmm internal function.

User ID displays the ID of the user who caused the most recent change. If the most recent change was made by DFSMSrmm processing the ID starts with an asterisk (*). Internal IDs include these values:

*OAM - DFSMSrmm system managed tape support

*HKP - Inventory management

11.10.6 Incomplete VRS chains - dummy VRS '*broken*'

If VRSEL processing finds an incomplete chain it is reported with EDG2230I, but the processing continues. In this case DFSMSrmm sets a minimum return code of 4 and any data set that match to this incomplete chain is retained by a special Name VRS '*broken*'. Figure 11-57 shows you an example of an incomplete VRS chain.

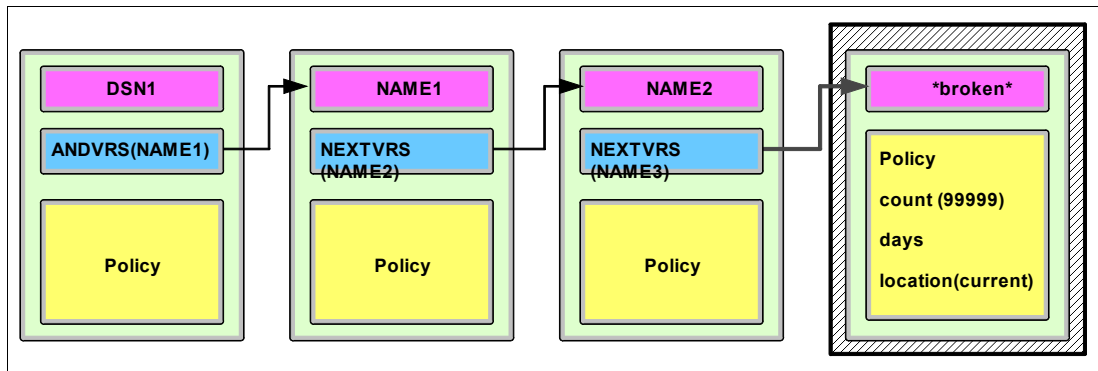


Figure 11-57 Incomplete VRS chain

A description of the two new messages you can get if the VRSEL processing detects some errors:

EDG2230I NEXTVRS name_vrs DOES NOT EXIST. CHAINING vrs_type VRS IS vrs_mask.

Where:

vrs_name
vrs_type

vrs_mask

the NAME vital record specification defined by NEXTVRS.

DSN - DSNAME vital record specification

VOL - VOLUME vital record specification

NAME - NAME vital record specification

the mask that uniquely identifies the vital record specification with the chaining error. For DSNAME vital record specifications the mask includes the data set name and optionally job name.

Severity:

Information.

Explanation:

During inventory management vital record processing, DFSMSrmm checks all vital record specification chains by following the chain using the NEXTVRS values. The vital record specification displayed in the message does not exist in the DFSMSrmm control data set.

Source:

DFSMSrmm.

Detecting Module:

EDGVREC0.

System Action:

Processing continues and DFSMSrmm sets a minimum return code of 4. For VRSEL(OLD) DFSMSrmm retains additional data sets or volumes, in the home location, up to the COUNT value specified in the initial VRS in the chain. For VRSEL(NEW) additional data sets are retained in the current location, permanently.

Operator Response: None.

Sysprogr Response: Add the missing vital record specification or correct the NEXTVRS value specified on the vital record specification displayed in the message.

Routing Codes: n/a

Descriptor Codes: None.

EDG2317E MIGRATION FROM VRSEL(OLD) TO VRSEL(NEW) IS RECOMMENDED.

Where:

Severity: Warning.

Explanation: During inventory management vital record processing, DFSMSrmm checks the VRSEL option which you have defined in the EDGRMMxx parmlib. The VRSEL(OLD) option will be removed in a future release of z/OS. Please migrate to using VRSEL(NEW).

Source: DFSMSrmm.

Detecting Module: EDGMHKP.

System Action: Processing continues. A minimum return code of 4 is set.

Operator Response: None.

Sysprogr Response: Plan a migration to VRSEL(NEW). Refer to the migration planning steps documented in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

Routing Codes: 11

Descriptor Codes: None.

11.10.7 Toleration and removal of really old functions

In addition IBM recommends that you perform a migration to VRSEL(NEW). Refer to the migration planning steps documented in the DFSMSrmm Implementation & Customization Guide.

Prevents use of very old VRS operands STARTNUMBER, LOCATION(BOTH), and STORENUMBER(xx,yy)

The toleration APAR OA13355 includes a ++HOLD(ACTION) and requires that VRSs are cleaned up before you are able to run EDGHSKP with VRSEL on a z/OS V1R8 system.

The cleanup actions are documented under message EDG2221E. EDG2221E sets return code 12 instead of 4.

Important: As long as you do not implement COUNT(0) or JOBNAME(ABEND\OPEN) you can run VRSEL processing on either a toleration system or z/OS V1R8.

Error messages

A description of the new message you get if the VRSEL processing detects some errors:

EDG2222E type VRS FOR mask SPECIFIES UNSUPPORTED OPTIONS - SOME RETENTION OPTIONS IGNORED.

Where:

type type is the type of vital record specification. It can be one of:
DSN - DSNNAME type vital record specification
VOL - VOLUME type vital record specification

mask is the vital record specification data set name or volume serial number

Severity: Information.

Explanation: During vital record processing, DFSMSrmm found a vital record specification that contains unsupported options. The unsupported options are STARTNUMBER and LOCATION(BOTH).

Source: DFSMSrmm.

Detecting Module: EDGVREC0.

System Action: Processing ends. A return code of 12 is set.

Operator Response: None.

Sysprogr Response: You must replace the vital record specification with other vital record specifications that provide the retention options you require. For example, if you use LOCATION(BOTH) you can replace it with use of the NEXTVRS operand. This example shows first an unsupported vital record specification and then the equivalent supported vital record specifications you might use.
Unsupported:

```
RMM ADDVRS DSNAME(data_set_name_mask) -  
    CYCLES COUNT(5) LOCATION(BOTH) -  
    STORENUMBER(2,1)
```

Supported:

```
RMM ADDVRS DSNAME(data_set_name_mask) -  
    CYCLES COUNT(5) LOCATION(LOCAL) -  
    STORENUMBER(2) NEXTVRS(DIST1C)  
RMM ADDVRS NAME(DIST1C) LOCATION(DISTANT) -  
    STORENUMBER(1)
```

If you use STARTNUMBER you can replace it with the use of the NEXTVRS operand. This example shows first an unsupported vital record specification and then the equivalent supported vital record specifications you might use.

Unsupported:

```
RMM ADDVRS DSNAME(data_set_name_mask) -  
    CYCLES COUNT(3) LOCATION(VAULT1) -  
    STORENUMBER(2) STARTNUMBER(1)
```

Supported:

```
RMM ADDVRS DSNAME(data_set_name_mask) -  
    CYCLES COUNT(3) LOCATION(HOME) -  
    STORENUMBER(1) NEXTVRS(VLT12C)  
RMM ADDVRS NAME(VLT12C) LOCATION(VAULT1) -  
    STORENUMBER(2)
```

In addition IBM suggests that you perform a migration to VRSEL(NEW). See the migration planning steps documented in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

Routing Codes: 11

Descriptor Codes: 7

11.10.8 Conversion to DFSMSrmm from other tape management systems

Before release V1R8 DFSMSrmm converted only DSN, LABEL and JOB information to UXTABLE entries. Now it also supports MGMTCLAS and ABEND keywords as shown in Figure 11-58 on page 434 too.

```
{           }{           {keyword           } }
{           }{           {preferred date}} [,J=create.jobname ]
{D=dsname [-] }{ ,LABEL=EXPDT={Julian date  } } [,JOB=create.jobname]
{DSN=dsname [-] }{ ,LABEL=RETPD=nnnn           } [,JOBNAME=create.jobname]
MGMTCLAS=smsclass { ,LABEL=WRETPD=nnnn           } [,SELECT=ALL           ]
M=smsclass      { ,ABEND=RETPD=nnnn           }
                { ,ABEND=WRETPD=nnnn           }
                { ,ABEND=EXPDT={keyword           } }
                {preferred date}
                {Julian date  }
```

Figure 11-58 MGMTCLAS and ABEND RDS keywords support

EDGCRFMT and EDGRSRDS conversion programs

If you have a CA-1 retention data set (RDS) entry than this RDS entry is converted in two steps to create a UXTABLE entry that can be used in the EDGUX100 user exit:

1. EDGCRFMT creates input for EDGCSRDS
2. EDGCSRDS will built the UXTABLE entries

Example 11-2 shows you some CA-1 retention data set (RDS) entries

Example 11-2 CA-1 retention data set (RDS) entries

```
M=SMSCLAS1,LABEL=RETPD=2010/011,JOB=TEST
MGMTCLAS=SMSCLAS2,LABEL=RETPD=2020/011
DSN=RXXXX.XXXX,ABEND=RETPD=2010/001,JOB=TESTTEST
DSN=RXXXX.XXXX,LABEL=RETPD=2010/001,JOB=TESTTEST
DSN=R3650.*,LABEL=RETPD=3650,JOB=COMBIN*
```

This CA-1 retention data set entries are converted to UXTABLE as shown in Figure 11-59 on page 435. For more information about a CA-1 to DFSMSrmm conversion refer to the *Converting to DFSMSrmm from CA-1* redbook. In this redbook the conversion is described in detail.

```

* start of RDS entries
  EDGCVRSR DSN=RXXXX.XXXX.G%%%%V%%, X
    JOB=ABEND, X
    RO=001, X
    RETPD=2010
  EDGCVRSR DSN=RXXXX.XXXX, X
    JOB=ABEND, X
    RO=001, X
    RETPD=2010
  EDGCVRSR DSN=RXXXX.XXXX.G%%%%V%%, X
    JOB=TESTTEST, X
    RO=001, X
    RETPD=2010
  EDGCVRSR DSN=RXXXX.XXXX, X
    JOB=TESTTEST, X
    RO=001, X
    RETPD=2010
  EDGCVRSR DSN=R3650.*, X
    JOB=COMBIN*, X
    RO=NO, X
    RETPD=3650
  EDGCVRSR DSN=SMSCLAS1, X
    JOB=TEST, X
    RO=011, X
    RETPD=2010
  EDGCVRSR DSN=SMSCLAS2, X
    RO=011, X
    RETPD=2020
* default RP value
  EDGCVRSR DSN=*, X
    RO=NO, X
    RETPD=3

```

Figure 11-59 Sample UXTABLE entries

EDGCSVDS conversion program

EDGCSVDS converts the vaulting policies from the VPDD into K-Records representing the DFSMSrmm storage location movement policies.

Processing is changed to implement data set filter VRSs for all DSN entries found in the VPDD. Instead of storing data set and job names in memory until the vault statements are reached, and then looping through them to create K records, we can now create a K record for the filter as we read the entry from the VPDD and no longer have memory limitations in EDGCSVDS.

EDGCNVT conversion program

EDGCNVT is updated to allow COUNT(0) in the count field of the EDGCKREC record. This program converts the output of the data extraction programs to DFSMSrmm format (CDS records or ADDVRS commands as shown in Figure 11-60 on page 436).

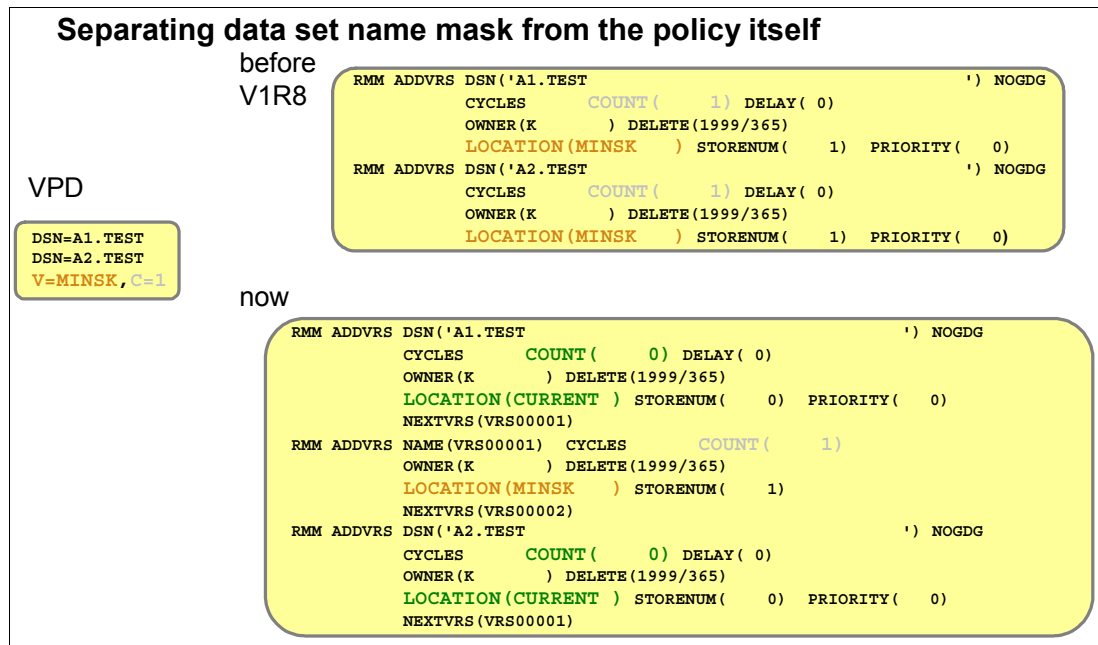


Figure 11-60 EDGCNVT RMM TSO ADDVRS commands

11.11 Trial run

DFSMSrmm provides the EDGHSKP utility to help you perform inventory management. When you specify the VERIFY parameter, then DFSMSrmm performs a trial run of vital record processing, so you can see how vital record processing changes affect the data set and volume information before the CDS is updated. Figure 11-61 contains a sample JCL for inventory management trial run processing.

```

//HSKP EXEC PGM=EDGHSKP,PARM='VRSEL,VERIFY,DATE(+5)'
//MESSAGE DD DISP=SHR,DSN=HSKP.MESSAGES
//REPORT DD DISP=SHR,DSN=HSKP.VRS.REPORT
//ACTIVITY DD DISP=SHR,DSN=HSKP.ACTIVITY

```

Figure 11-61 Inventory management trial run

The parameter DATE(+5) adds five days to current day to perform a trial run in this date.

The activity file is not intended to be a report, it just contains information about changes made to data sets during vital record processing. DFSMSrmm provides a sample job EDGJACTP in SAMPLIB that shows how to selectively format and print fields.

The output for this job are nine data sets containing the following reports:

- ▶ RUNINFO report. Data set with run control information
- ▶ VRS report. Data sets with changed V/R status
- ▶ VRSS report. Summary by status
- ▶ RETDATE report. Data sets with new retention date
- ▶ RETDS report. Summary by new date
- ▶ MATCHVRS report. Data sets with new matching VRS
- ▶ MATCHVS report. Summary by new matching VRS
- ▶ SUBCHN report. Data sets retained by new part of chain
- ▶ SUBCHNS report. Summary by new retaining subchain

For more information about these reports and the EDGJACTP job, refer to *DFSMSrmm Reporting*, SC26-7406.

11.12 VRS examples

The following sections provide examples of a VRS chain and subchain to explain the difference between these concepts.

11.12.1 ADDVRS operand defaults

In the examples, we use the default values for the operands of the ADDVRS subcommand listed in Table 11-14.

Table 11-14 Operands of ADDVRS subcommand

Operand	Default value
COUNT	99999
DELAY	0
DELETEDATE	1999/365
LOCATION	HOME
PRIORITY	0
STORENUMBER	99999

11.12.2 Adding a single VRS

In the first example (see Figure 11-62), we are using a single VRS to retain and move a data set. The data sets shown in Figure 11-63 on page 438 are retained only by this PRIMARY VRS.

The tasks for this DSN VRS are:

- ▶ Retain ten cycles of each matching data set group.
- ▶ All instances of a data set created on a single calendar day are considered to be a single cycle.
- ▶ The newest five cycles must be stored in the outside storage location SAVE, and all others in their HOME location.

Primary DSN VRS		
RMM	ADDVRS	-
	DSN('MARY.**')	-
	BYDAYSCYCLES	-
	COUNT(10)	-
	STORENUMBER(5)	-
	LOCATION(SAVE)	-

Figure 11-62 Single VRS example

Figure 11-63 shows the data sets and the location retained by the single VRS definition.

Data set name	Creation date	Location	Creation occurrence
MARY.TSO.DATA.BACKUP	2002/123	HOME	-9
MARY.TSO.DATA.BACKUP	2002/124	HOME	-8
MARY.TSO.DATA.BACKUP	2002/125	HOME	-7
MARY.TSO.DATA.BACKUP	2002/126	HOME	-6
MARY.TSO.DATA.BACKUP	2002/127	HOME	-5 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/127	HOME	-5 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/130	SAVE	-4
MARY.TSO.DATA.BACKUP	2002/131	SAVE	-3 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/131	SAVE	-3 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/132	SAVE	-2
MARY.TSO.DATA.BACKUP	2002/133	SAVE	-1
MARY.TSO.DATA.BACKUP	2002/134	SAVE	0

Figure 11-63 Single VRS Retention and location results

11.12.3 VRS chain with one NEXTVRS operand

In the second example (Figure 11-64), we are using the NEXTVRS operand to have an additional LOCATION definition. The request for this VRS is almost the same as the one in Figure 11-62 on page 437. The difference is that the newest five cycles must always be in the HOME location.

The tasks for this VRS specification are as follows:

- ▶ Retain ten cycles of each matching data set group.
- ▶ All instances of a data set created on a single calendar day are considered to be a single cycle.
- ▶ The newest cycle (0) must be stored in the HOME location. You do not need to specify the LOCATION(HOME) operand, because HOME is the default.
- ▶ The next five cycles (-1 to -5) must be stored in the outside storage location SAVE.
- ▶ All other cycles (-6 to -9) should be retained in the HOME location.

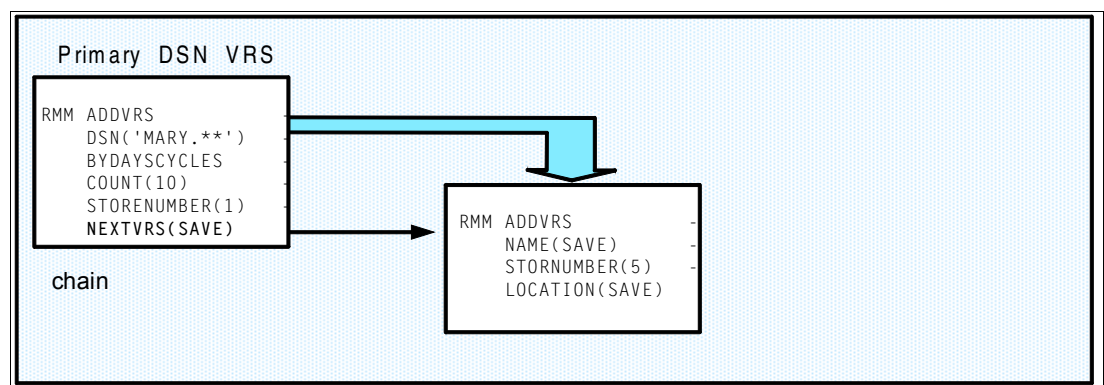


Figure 11-64 VRS chain with one NEXTVRS operand

Figure 11-65 on page 439 shows the data sets and their location retained by this VRS definition.

Data set name	Creation date	Location	Creation occurrence
MARY.TSO.DATA.BACKUP	2002/123	HOME	-9
MARY.TSO.DATA.BACKUP	2002/124	HOME	-8
MARY.TSO.DATA.BACKUP	2002/125	HOME	-7
MARY.TSO.DATA.BACKUP	2002/126	HOME	-6
MARY.TSO.DATA.BACKUP	2002/127	SAVE	-5 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/127	SAVE	-5 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/130	SAVE	-4
MARY.TSO.DATA.BACKUP	2002/131	SAVE	-3 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/131	SAVE	-3 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/132	SAVE	-2
MARY.TSO.DATA.BACKUP	2002/133	SAVE	-1
MARY.TSO.DATA.BACKUP	2002/134	HOME	0

Figure 11-65 Single VRS using the NEXTVRS operand results

VRS chain with two NEXTVRS definitions

The third example (see Figure 11-66) shows a VRS with two NEXTVRS definitions to have more than one outside LOCATION definition. The only difference between the first and the second example is that the data sets must be moved to a secondary storage location.

The tasks for this VRS specification are:

- ▶ Retain ten cycles of each matching data set group.
- ▶ All instances of a data set created on a single calendar day are considered to be a single cycle.
- ▶ The newest cycles (0) must be stored in the HOME location. You do not need to specify the LOCATION(HOME) operand, because HOME is the default.
- ▶ The next three cycles (-1 to -3) must be stored in the outside storage location SAVE.
- ▶ The cycles (-4 to -6) must be stored in the outside storage location MUNICH.
- ▶ All other cycles (-7 to -9) should be retained in the HOME location.

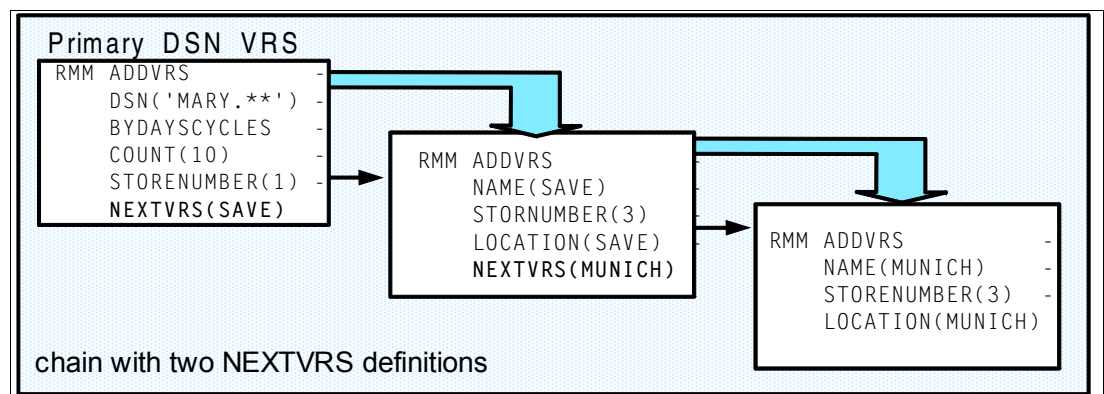


Figure 11-66 VRS chain with two NEXTVRS operands

Figure 11-67 shows the data sets and their location retained by this VRS definition.

Data set name	Creation date	Location	Creation occurrence
MARY.TSO.DATA.BACKUP	2002/123	HOME	-9
MARY.TSO.DATA.BACKUP	2002/124	HOME	-8
MARY.TSO.DATA.BACKUP	2002/125	HOME	-7
MARY.TSO.DATA.BACKUP	2002/126	MUNICH	-6
MARY.TSO.DATA.BACKUP	2002/127	MUNICH	-5 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/127	MUNICH	-5 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/130	MUNICH	-4
MARY.TSO.DATA.BACKUP	2002/131	SAVE	-3 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/131	SAVE	-3 BYDAYSCYCLE
MARY.TSO.DATA.BACKUP	2002/132	SAVE	-2
MARY.TSO.DATA.BACKUP	2002/133	SAVE	-1
MARY.TSO.DATA.BACKUP	2002/134	HOME	0

Figure 11-67 Single VRS using two NEXTVRS operand results

VRS chain with two NEXTVRS operands with retention information

Figure 11-68 shows a management class (MC) or a management value (MV) VRS using two NEXTVRS operands to have more than one retention criterion. All data sets retained by this VRS are retained in the HOME location. This example is a VRS chain with two additional subchains.

The tasks for this VRS specification are:

- ▶ Retain a data set for five calendar days by specifying the operand DAYS.
- ▶ After five calendar days, retain the data set as long as it is cataloged by specifying the WHILECATALOG operand.
- ▶ If the data set is no longer cataloged, retain the data set for one additional day by specifying the EXTRADAYS operand.

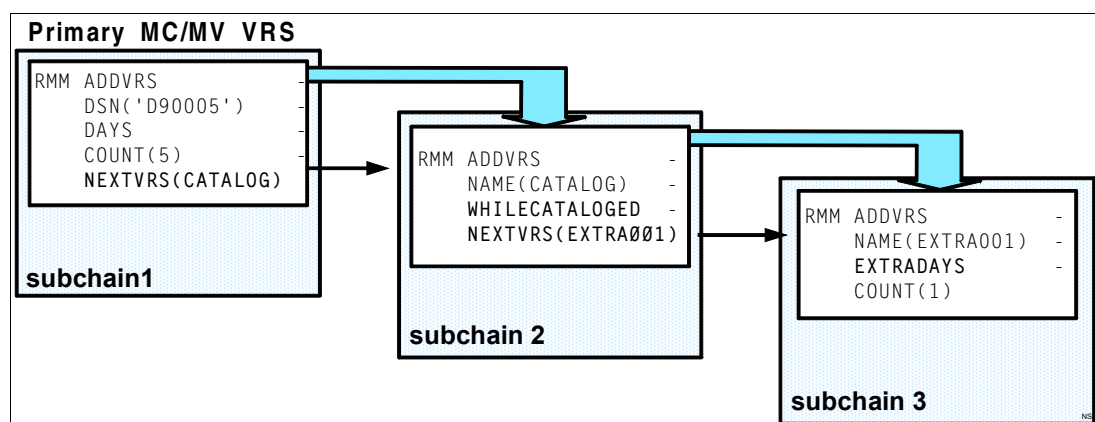


Figure 11-68 VRS chain with retention information example

In Figure 11-69, the generation data group (GDG) is defined with a limit of 10. Generation data set MARY.TSO.DATA.G0011V00 was uncataloged manually yesterday, and the generation data set MARY.TSO.DATA.G0010V00 was uncataloged manually last week. The first of the two MARY.TSO.DATA.G0013V00 generation data sets was created incorrectly and not cataloged.

Data set name	Creation date	Location	Data set cataloged	Retention criteria
MARY.TSO.DATA.G0009V00	2002/123	HOME	Y	WHILECATALOGED
MARY.TSO.DATA.G0010V00	2002/124	HOME	N	not retained
MARY.TSO.DATA.G0011V00	2002/125	HOME	N	EXTRADAYS
MARY.TSO.DATA.G0012V00	2002/126	MUNICH	Y	WHILECATALOGED
MARY.TSO.DATA.G0013V00	2002/127	MUNICH	N	not retained
MARY.TSO.DATA.G0013V00	2002/127	MUNICH	Y	WHILECATALOGED
MARY.TSO.DATA.G0014V00	2002/130	MUNICH	Y	DAYS
MARY.TSO.DATA.G0015V00	2002/131	SAVE	Y	DAYS
MARY.TSO.DATA.G0015V00	2002/131	SAVE	Y	DAYS
MARY.TSO.DATA.G0016V00	2002/132	SAVE	Y	DAYS
MARY.TSO.DATA.G0017V00	2002/133	SAVE	Y	DAYS
MARY.TSO.DATA.G0018V00	2002/134	HOME	Y	DAYS

Figure 11-69 Single VRS using two retention NEXTVRS operands results

11.12.4 VRS chain with two ANDVRS operands

The fifth example (see Figure 11-70) shows a complex VRS with two ANDVRS operands in order to have more than one retention criterion for retaining a data set. If you specify ANDVRS, all definitions must be true to retain the data set. If any definition is not true, the data set is no longer retained by this VRS chain.

The tasks for this VRS are:

- ▶ Retain a data set for three cycles.
- ▶ Only do so if the data set has been used in the last six days.
- ▶ Data set should be moved to the outside storage location STOREX until the volume expiration date is reached.

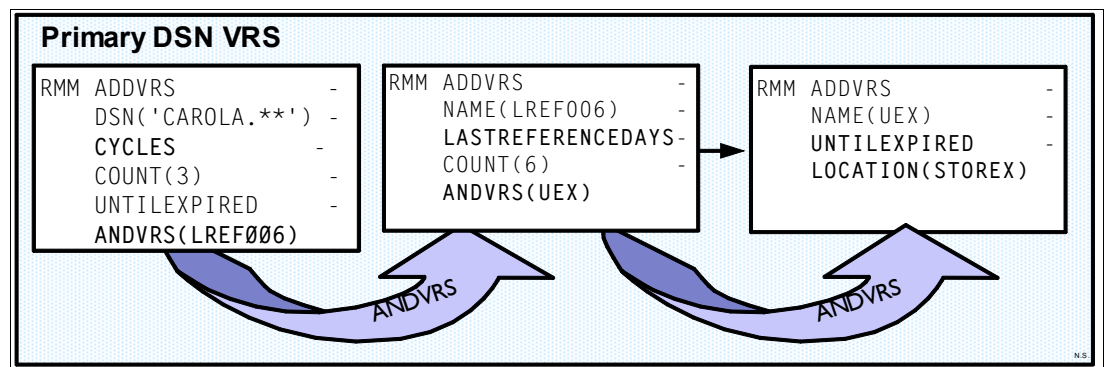


Figure 11-70 VRS chain with two ANDVRS operands example

Figure 11-71 on page 442 shows the data sets and their location retained by the VRS definition in Figure 11-70. Only the generation data sets G0016V00, G0017V00, and G0018V00 are retained by the VRS because we specified CYCLES COUNT(3).

Data set name	Creation date	Exp. date	Last used date	Retention criteria		
				WC	LR	UEX
MARY.TSO.DATA.G0009V00	2002/123	2002/123	2002/123	N	N	N
MARY.TSO.DATA.G0010V00	2002/124	2002/124	2002/124	N	N	N
MARY.TSO.DATA.G0011V00	2002/125	2002/125	2002/231	N	N	N
MARY.TSO.DATA.G0012V00	2002/126	2002/126	2002/126	N	N	Y
MARY.TSO.DATA.G0013V00	2002/127	2002/127	2002/127	N	N	Y
MARY.TSO.DATA.G0013V00	2002/127	2002/127	2002/132	N	Y	Y
MARY.TSO.DATA.G0014V00	2002/130	2002/130	2002/130	N	N	Y
MARY.TSO.DATA.G0015V00	2002/131	2002/131	2002/100	N	Y	Y
MARY.TSO.DATA.G0015V00	2002/131	2002/131	2002/131	N	N	Y
MARY.TSO.DATA.G0016V00	2002/132	2002/132	2002/132	Y	Y	Y <===
MARY.TSO.DATA.G0017V00	2002/133	2002/133	2002/133	Y	Y	Y <===
MARY.TSO.DATA.G0018V00	2002/134	2002/134	2002/134	Y	Y	Y <===

Figure 11-71 Single VRS using two retention NEXTVRS operands results

11.12.5 VRS chain with two subchains

Figure 11-72 shows a complex VRS with an ANDVRS definition, and a NEXTVRS VRS with retention information. The tasks for this VRS are:

- ▶ Retain a data set for three cycles.
- ▶ Only do so if the data set has been used in the last six calendar days.
- ▶ If the data set is no longer retained by the first subchain, the data set must be retained for five additional days.

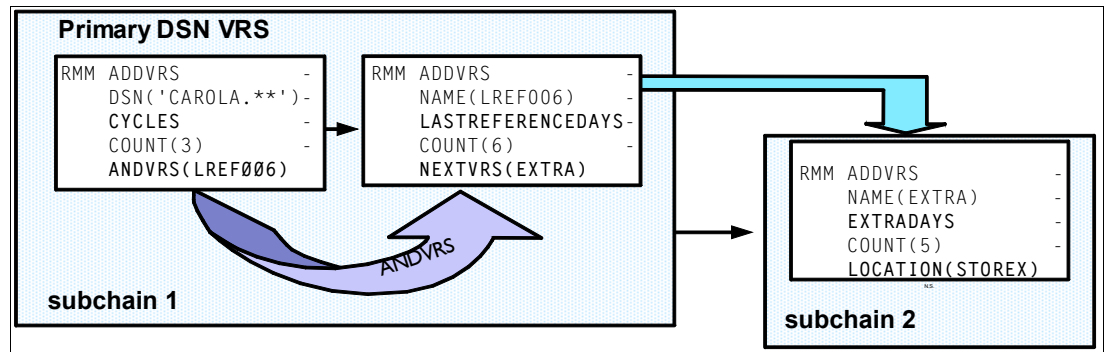


Figure 11-72 VRS chain with Two Subchains Example

Figure 11-73 on page 443 shows the data sets and their location retained by the VRS definition in Figure 11-73 on page 443. The generation data sets G0016V00, G0017V00, and G0018V00 are retained by the first subchain, and the generation data set G0015V00 is retained and moved to the storage location STOREX by the secondary subchain.

Data set name	Creation date	Exp. date	Last used date	Retention criteria		
				WC	LR	*
MARY.TSO.DATA.G0009V00	2002/123	2002/123	2002/123	N	N	
MARY.TSO.DATA.G0010V00	2002/124	2002/124	2002/124	N	N	
MARY.TSO.DATA.G0011V00	2002/125	2002/125	2002/231	N	N	
MARY.TSO.DATA.G0012V00	2002/126	2002/126	2002/126	N	N	
MARY.TSO.DATA.G0013V00	2002/127	2002/127	2002/127	N	N	
MARY.TSO.DATA.G0013V00	2002/127	2002/127	2002/132	N	N	
MARY.TSO.DATA.G0014V00	2002/130	2002/130	2002/130	N	N	
MARY.TSO.DATA.G0015V00	2002/131	2002/131	2002/100	N	N	
MARY.TSO.DATA.G0015V00	2002/131	2002/131	2002/131	N	Y	<=== 2
MARY.TSO.DATA.G0016V00	2002/132	2002/132	2002/132	Y	Y	<=== 1
MARY.TSO.DATA.G0017V00	2002/133	2002/133	2002/133	Y	Y	<=== 1
MARY.TSO.DATA.G0018V00	2002/134	2002/134	2002/134	Y	Y	<=== 1

* = retained by VRS subchain

Figure 11-73 ANDVRS and NEXTVRS with retention operand results

11.13 Complex VRS chain with three subchains

Figure 11-74 on page 444 shows a complex VRS with a primary DSN VRS that has two NEXTVRS definitions followed by two subchains with retention information. The second subchain has a NEXTVRS definition, and the third subchain is using the ANDVRS operand.

The tasks for this VRS are:

1. The newest 20 cycles are retained by the first subchain.
 - The newest five cycles are always in the HOME location.
 - The next five cycles are moved to the storage location SAVE.
 - The cycles -10 to -14 are retained in the storage location MUNICH.
 - The last five cycles are retained in the HOME location.
2. After the data sets are no longer retained by the first subchain, we retain all data sets for 365 calendar days since creation:
 - The first 60 days, retain the data sets in the HOME location.
 - Then retain the data sets for 14 days in the storage location MAINZ.
 - The last 291 days, retain the data sets in the HOME location.
3. The third subchain retains the data set only under the following conditions:
 - If we have up to 30 cycles.
 - And if the data set has been read or written to in the last 400 calendar days.

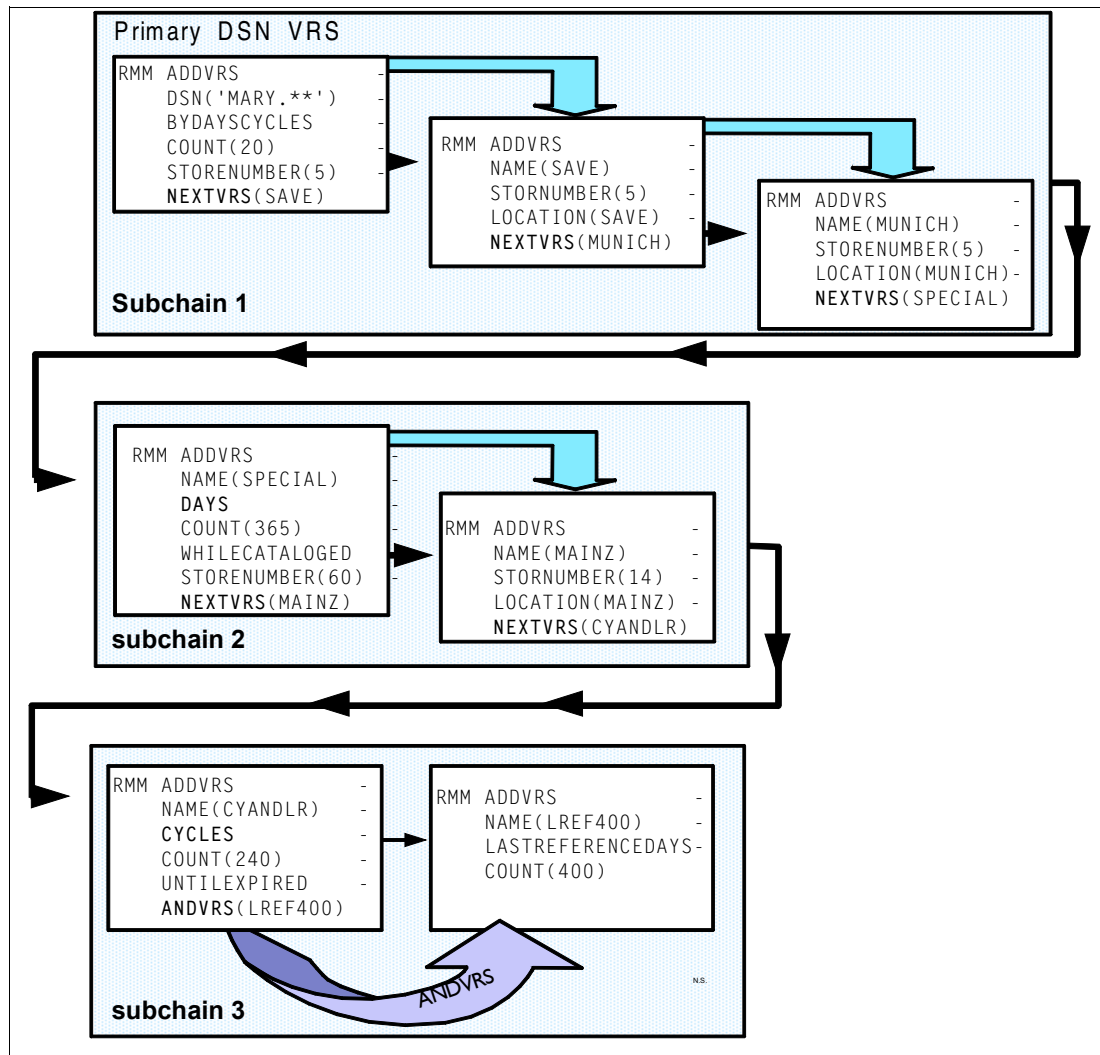


Figure 11-74 VRS chain with three subchains example

11.14 VRS hints and tips

With OS/390 V2R10 and VRSEL(NEW) parameters, system-managed and non-system-managed tape data sets can be managed using ACS routines. This allows you to simplify VRS specifications and centralize the data set specifications in the ACS routines.

You can benefit from the new VRS support introduced by VRSEL(NEW) following these recommendations:

- Move data set name masks to ACS filter list.

You may have the following VRSs

```

RMM ADDVRS DSNAME('DATA.SET.**') COUNT(31) DAYS
RMM ADDVRS DSNAME('YJRES.SET.**') COUNT(31) DAYS

```

If so, you can assign a MC LDATE031 through ACS routines to these data set name masks, and add the following VRS as shown here:

```

RMM ADDVRS DSNAME('LDATE031') COUNT(31) DAYS

```

- Use one VRS chain to implement one service level.

For example, you can assign through the ACS routines the same management class (CATALOG) to all the data sets that you want to retain while they are cataloged. Then you can add a data set name VRS for this MC as shown here:

```
RMM ADDVRS DSNAME('CATALOG') WHILECATALOG
```

The rest of the VRSs with the same specification can be removed.

- Code MC ACS routine to process allocations for system-managed tapes and for non-system-managed tapes.

All the data set specifications, system-managed or non-system-managed, tape data sets are to be managed in the same way, using the RMMVRS environment call for non-SMS tapes and regular allocations for SMS tapes.

- Assign policies by name. Move decisions out of EDGUX100.

ACS routines are easier to change and maintain than exits, so we recommend that you include all data set specifications in the ACS routines, and remove them from EDGUX100 exit.

The migration from MV to MC is transparent. The MC specification overwrites MV specification (refer to Figure 11-20 on page 393) and the data sets are retained by policy, for example:

- a. We create the VRS as shown here:

```
RMM ADDVRS DSNAME('CYCL005') CYCLES COUNT(5)
```

- b. We create the data sets 1, 2, and 3 as shown in Figure 11-75 on page 445 with a management value of CYCL005 assigned by EDGUX100.
- c. We add the same specification in the EDGUX100 to ACS routines, assigning to this data sets a MC=CYCL005, and we create the data sets 4, 5 and 6 in Figure 11-75 on page 445. They have MC=CYCL005 and MV=CYCL005.
- d. After that, we remove the specification in EDGUX100, and we create the data set 7 in Figure 11-75 on page 445. This data set has only MC=CYCL005.
- e. DFSMSrmm retains five versions of the data sets: 7, 6, 5, 4 and 3, although numbers 1 and 2 have only MV specification.

- | |
|--|
| <ol style="list-style-type: none"> 1. Data set name YCJRES.TEST1 creation date 33 days ago with MV 2. Data set name YCJRES.TEST1 creation date 32 days ago with MV 3. Data set name YCJRES.TEST1 creation date 31 days ago with MV 4. Data set name YCJRES.TEST1 creation date 29 days ago with MV and MC 5. Data set name YCJRES.TEST1 creation date 28 days ago with MV and MC 6. Data set name YCJRES.TEST1 creation date 25 days ago with MV and MC 7. Data set name YCJRES.TEST1 creation date 20 days ago with MC |
|--|

Figure 11-75 Data set examples

- Move entries in UXTABLE to ACS routines.

The EDGUX100 exit is shipped by DFSMSrmm as SAMPLIB member EDGCVRSX. This sample exit uses a dynamic table that is initially built by the sample job EDGJSRDS. This table is called UXTABLE, and it is a list of assembler macro statements that is easily customized to add or remove entries. You can add your pooling decisions to the table by modifying the entries built by EDGCSRDS, or by adding new entries.

If you are using UXTABLE to assign policies to data sets, you can eliminate this table assigning a MC through ACS routines to each data set name or data set name mask with an entry in the UXTABLE.



DFSMSrmm ACS support

Starting with DFSMSrmm V2.10, you can use your SMS ACS routines to assign a VRS management class and to select a specific scratch pool. This chapter explains the steps to update your SMS environment to allow a management class to be used in place of the VRS management value assigned in the EDGUX100 user exit, and a storage group name to be used as a scratch pool name.

The objectives of this chapter are to:

- Understand the interaction between DFSMSrmm and SMS
- Learn how DFSMSrmm calls SMS in both an SMS-managed environment and a non-SMS-managed environment

Update the SMS configuration to move decisions from the EDGUX100 exit to SMS

12.1 DFSMSrmm SMS ACS support

With DFSMSrmm V2R10 or later, you can use your SMS ACS routines to select a specific scratch pool and storage group to be assigned to create new data on tape, and set a management class to retain and move a data set on tape. You move your policy and scratch pool decisions to your SMS ACS routines as shown in Figure 12-1. For non-system managed tapes, DFSMSrmm calls the ACS routines to allow a management class and storage group assignment. For SMS managed tapes, SMS calls the ACS routines directly.

To enable DFSMSrmm ACS support, you must have the SMS subsystem active and have a valid SMS configuration.

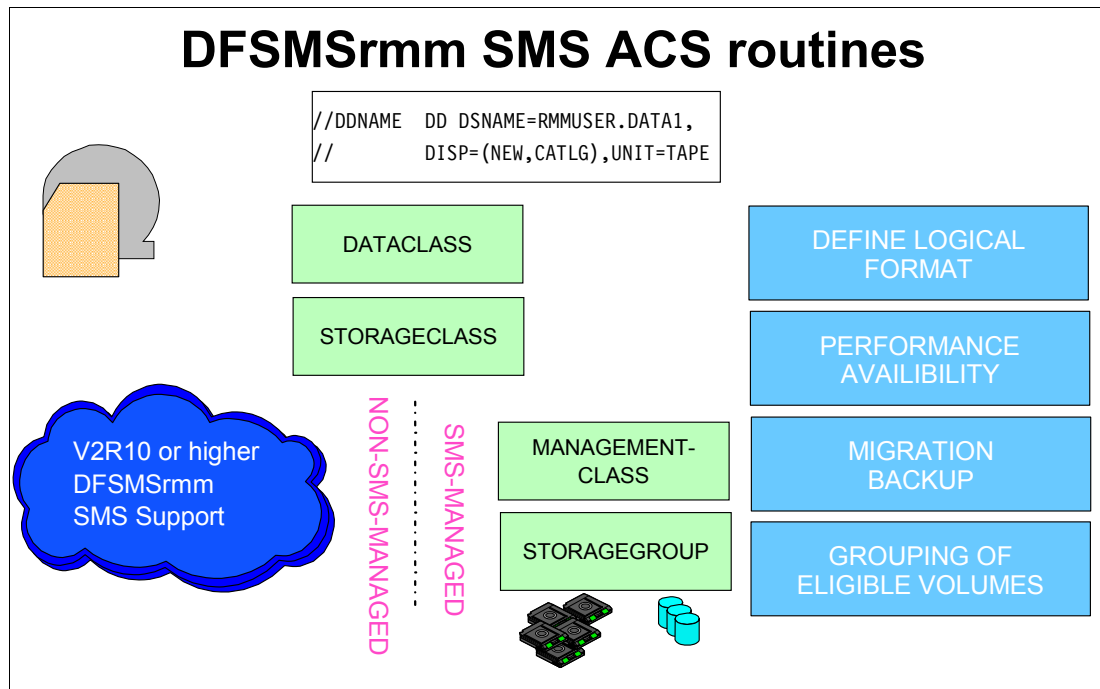


Figure 12-1 SMS ACS support with DFSMSrmm OS/390 V2R10 or later

With OS/390 V2R10 or later, SMS ACS routines are always called by DFSMSrmm if you have a tape mount outside a SMS managed tape library.

ACS processing for non-system managed tapes can be tailored by the PARMLIB member EDGRMMxx OPTION subparameters PREACS and SMSACS. Figure 12-2 shows the syntax of the EDGRMMxx OPTION subparameter.

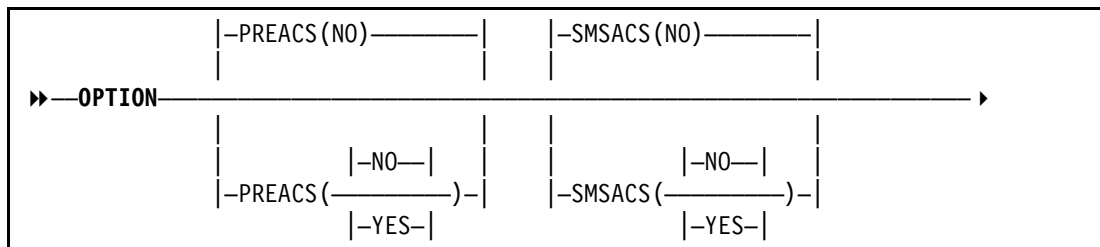


Figure 12-2 DFSMSrmm ACS processing PARMLIB options

The PREACS and SMSACS subparameters are further explained here:

► **PREACS**

Specify the PREACS operand to control whether DFSMSRmm-supplied values and EDGUX100 installation exit-supplied values are input to SMS pre-ACS processing.

Where:

NO	Specify NO to avoid DFSMSRmm pre-ACS processing using the DFSMSRmm EDGUX100 installation exit.
YES	Specify YES to enable DFSMSRmm pre-ACS processing using the DFSMSRmm EDGUX100 installation exit.

► **SMSACS**

The scratch pooling and policy assignment rules are per default coded in EDGUX100 user exit. You can move your decisions to the SMS ACS routines, where you can use ACS input variables as a base for the management class and storage group assignment.

Where:

NO	Specify NO to prevent DFSMSRmm from calling the SMS ACS processing to obtain management class and storage group names. DFSMSRmm system-based scratch pooling, and VRS management values based on the EDGUX100 installation exit are used.
YES	Specify YES to enable DFSMSRmm calls to the SMS ACS processing to obtain management class and storage group names. If values are returned by the SMS ACS routines, the values are used instead of the DFSMSRmm and EDGUX100 decisions.

We recommend that you move pooling decisions and VRS management value assignment out of the EDGUX100 user exit.

Recommendation: Do not implement PREACS processing. Also do not use the EDGUX100 user exit except for the following reasons:

- Cartridge loader control (ACL)
- Data set recording
- Sticky label customization
- Request ignore of volumes

12.1.1 How ACS support works

Use ACS routines for scratch pooling based on tape storage group names. Using ACS processing to set a storage group name overrides all other pool selection methods.

DFSMSRmm provides support for non-system-managed tape and for system-managed manual tape libraries. This support enables pooling at the individual volume level. You assign a storage group name to each volume by using DFSMSRmm TSO subcommands, or by using pooling information that you define with the DFSMSRmm EDGRMMxx PARMLIB VLPOOL command (see “Tailoring your EDGRMMxx PARMLIB member” on page 469).

DFSMSRmm calls ACS routines passing environment information, including the pool identified by DFSMSRmm system-based pooling. The ACS routine can optionally set a storage group name, which overrides the DFSMSRmm system-based pool.

12.1.2 ACS support for non-system managed volumes

In a non-system-managed library, DFSMSRmm supports SMS ACS routines in different ways:

- DFSMSrmm directly calls the ACS routines if you specified SMSACS(YES) in the EDGRMMxx PARMLIB member to allow a management class and storage group to be assigned. Figure 12-3 shows how it works.

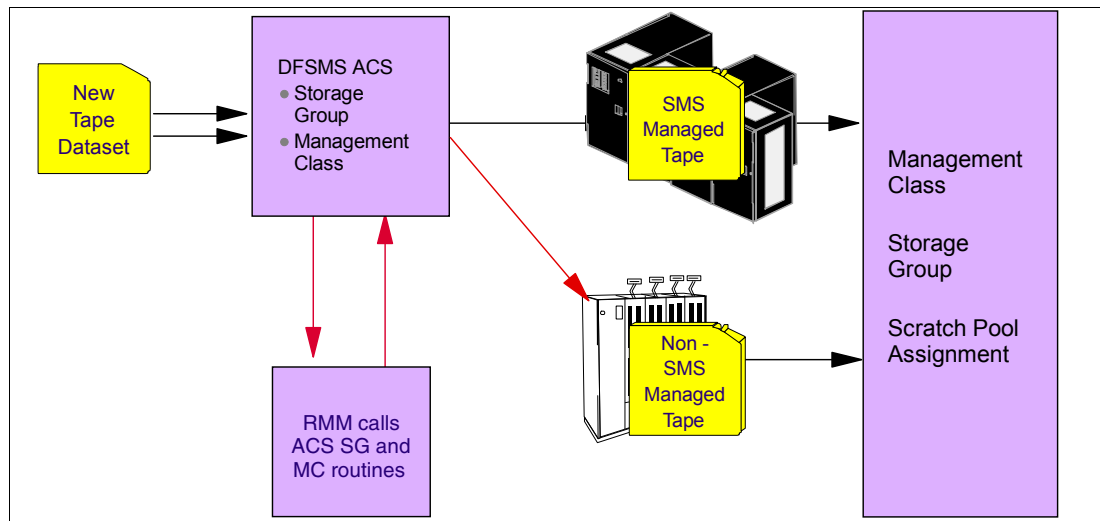


Figure 12-3 How DFSMSrmm SMS direct calls for non-system managed volumes

- You can use the EDGUX100 user exit to call SMS ACS routines to assign a VRS management class value and a scratch pool if you have specified PREACS(YES) as shown in Figure 12-4.

Use pre-ACS processing to obtain the DFSMSrmm system-based pool or the EDGUX100 installation exit pool prefix as an input value to the ACS routines in the MSPOL read-only variable. During pre-ACS processing, DFSMSrmm does not make the RMMPOOL environment call to the ACS routine. During pre-ACS processing for new allocations, DFSMSrmm performs the following actions:

- Uses the VLPOOL definitions to select a default DFSMSrmm pool using DFSMSrmm system-based pooling. DFSMSrmm sets a pool prefix if a specific pool is selected.
- Calls the EDGUX100 installation exit to obtain a pool prefix value. If a pool prefix value is returned, the pool prefix value returned by the EDGUX100 installation exit overrides the DFSMSrmm selected pool.
- Returns the selected value in the MSPOL read-only variable if the MSPOL variable is not already set by the pre-ACS exit.

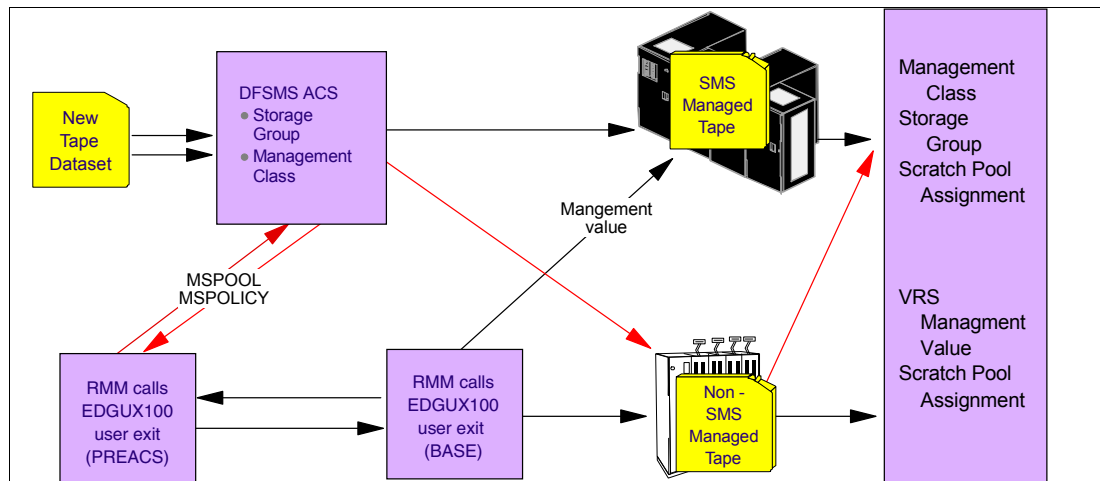


Figure 12-4 DFSMSrmm EDGUX100 user exit call with PREACS(YES)

- You can use both the DFSMSrmm direct call and the user exit EDGUX100 call together to take the decision as shown in Figure 12-5.

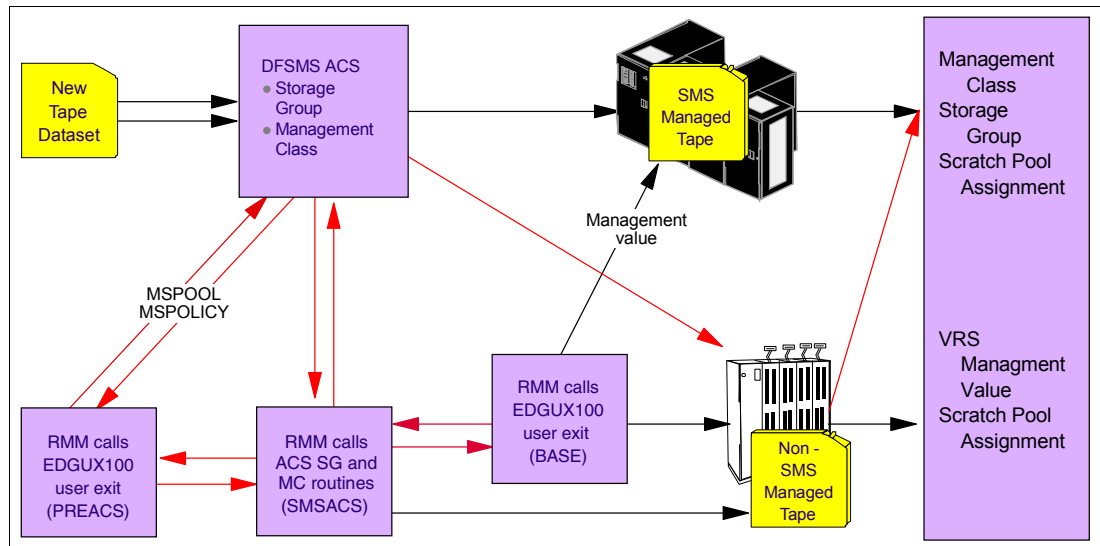


Figure 12-5 DFSMSrmm SMS direct and EDGUX100 user exit call

Note: To enable DFSMSrmm ACS support, you must have the SMS subsystem active and have a valid SMS configuration.

If you are using PREACS processing and have installed the EDGUX100 user exit, DFSMSrmm processes the functions in following sequence:

1. ACS processing calls the PRE-ACS exit.
2. DFSMSrmm calls EDGUX100 and sets MSPOOL and MSPOLICY.
3. ACS decides whether the entity is system managed. During processing, the four PRE-ACS read-only variables can be used as input to the decision.
4. For non-system managed tape, we enter DFSMSrmm ACS support. ACS processing is called for storage group processing, and again for management class processing.
5. DFSMSrmm calls EDGUX100 for its basic exit function.

12.1.3 Non-system managed tape libraries

A non-system managed tape library is all the volumes, shelves, and drives that are not in an automated tape library dataserwer or manual tape library. You might know this as the traditional tape library in a data center, or as an automated environment that is not system-managed. DFSMSrmm provides complete tape management functions for the volumes and shelves in this traditional tape library.

All tape media and drives supported by z/OS are supported in this environment. Use DFSMSrmm to fully manage all types of tapes in a non-system managed tape library, including 3420 reels, 3480, 3490, 3590, and 3592 cartridge system tapes.

You can also use DFSMSrmm to manage volumes in any automated tape library that has special software including an IBM Tape Library dataserwer that is managed using Basic Tape Library Support (BTLS) or an VTFM.

Most non-IBM libraries do not provide support for system-managed tape, so we consider them to be the traditional tape libraries.

The IBM direction is to replace DFSMSrmm exit function with an alternative, either the SMS policy or PARMLIB option.

12.1.4 SMS read-only variables

DFSMSrmm calls the ACS routines to request the assignment of storage group and management class names for non-system managed tape data sets. Table 12-1 lists the read-only variables that are set for DFSMSrmm requests.

Table 12-1 SMS read-only variables set by DFSMSrmm

Variables			
&ACCT_JOB	&ACCT_STEP	&ACSENVIR	&DD
&DSN	&DSORG	&DSTYPE	&EXPDT
&FILENUM	&GROUP	&HLQ	&JOB
&LABEL	&LIBNAME	&LLQ	&NQUAL
&PGM	&STORGRP	&SYSNAME	&SYSPLEX
&UNIT	&USER	&XMODE	

Note: The SMS pre-ACS exit variables &MSPOOL, &MSPOLICY, &MSDEST, and &MSPARM read-only variables can be used by a tape management system only.

When the &ACSENVIR variable is set to RMMPOOL, DFSMSrmm requests SMS to return a storage group name. DFSMSrmm requests that both the management class and storage group ACS routines are run. With a combination of these routines, you can decide whether you want SMS to return a storage group value, and what the value will be. If a storage group name is returned, it must be a valid tape storage group name. Using different &ACSENVIR values helps you differentiate between DFSMSrmm requests for a storage group name, and allocation requests for system-managed data sets.

When the &ACSENVIR variable is set to RMMVRS, DFSMSrmm requests SMS to return a management class name. DFSMSrmm requests that only the management class ACS routine is run. Using different &ACSENVIR values helps you differentiate between DFSMSrmm requests for a management class name and allocation requests for system-managed data sets.

Using the read-only variables

Most ACS variables are read-only. Read-only variables contain data set and system information, and they reflect what is known at the time of the allocation request. You can use read-only variables in comparison operations, but you cannot change their values.

Note: The read-only variables are case sensitive.

► &ACCT_JOB

The accounting information from the JOB statement.

Type: Literal
Max value: 142 characters

► **&ACCT_STEP**

The accounting information from the EXEC statement. This information is refreshed for each step in the job.

Type: Literal
Max value: 142 characters

► **&ACSENVIR**

The environment on which the ACS routine was invoked, one of:

RECALL	For data set recall operations
RECOVER	For data set recover operations
RENAME	For data set alter rename operations
RMMPOOL	For DFSMSrmm requests for a storage group name
RMMVRS	For DFSMSrmm requests for a management class name
CONVERT	For data set convert in place operations
ALLOC	For new data set allocations (this is the default)
STORE	OSREQ object store environment
CHANGE	OSREQ object change environment
CTTRANS	OSMC object class transition environment
other	Installation exit can set its own value before re-invoking ACS

Type: Literal
Max value: 8 characters

► **&DD**

DDNAME in the DD statement of the data set.

Type: Literal
Max value: 8 characters

► **&DSN**

The name of the data set or collection for which ACS processing is taking place. For VSAM data sets, only the cluster name is passed to the ACS routine; the component names are not.

If the data set has an absolute or relative generation number, it is stripped from &DSN. The generation number is the low-level qualifier of the data set name.

Type: Literal
Max value: 44 characters

► **&DSORG**

The data set organization, one of:

PS	Physical sequential
PO	Partitioned
VS	VSAM organization
DA	BDAM organization
null	No value specified

Type: Literal
Max value: 2 characters

► **&DSTYPE**

The data set type, one of:

GDS	One generation data set of a generation data group, or any data set allocated with a relative generation number (such as A.B.C(+1)) or an absolute generation number (such as A.B.C.G0000V00).
PERM	Standard permanent data sets
TEMP	Temporary data sets
null	None of the above

Type: Literal

Max value: 8 characters

► **&EXPDT**

The expiration date in the form of YYYYDDD where YYYY is the four-digit number for the year. The maximum allowable value for yyyy is 9799. DDD is the three-digit number for the day of the year from 1 - 366.

Exception: Expiration dates of 99365 and 99366 are considered “NEVER-SCRATCH” dates.

Type: Literal

Max value: 7 characters

► **&FILENUM**

The value of the FILENUM ACS read-only variable. This variable corresponds to the data set sequence number on the JCL LABEL parameter. The default is 1. This field is optional.

Type: Numeric

Max value: 4 characters

► **&GROUP**

The RACF-defined group to which you are connected, or the group specified in the GROUP keyword on the JCL JOB statement. If the environment is recall or recover, &GROUP is set only if the requester of the recall or recover is not a DFSMSHsm authorized user. When DFSMSHsm invokes the ACS routines, &GROUP is the group associated with &USER.

Type: Literal

Max value: 8 characters

► **&HLQ**

The high-level (first) qualifier of the data set or collection name.

Type: Literal

Max value: 8 characters

► **&JOB**

The job name, the started task name, or the TSO/E user ID from the JOB statement, depending on the execution mode (&XMODE).

Type: Literal

Max value: 8 characters

► **&LABEL**

The value of the LABEL ACS read-only variable. This variable corresponds to the label field of the JCL LABEL parameter. Allowable values are NL, AL, SL, NSL, SUL, AUL, BLP, LTM or blank. The default is IBM Standard Label. This field is optional.

Type: Literal

Max value: 3 characters

► **&LIBNAME**

The name for the LIBNAME ACS read-only variable, can contain a 1 to 8 character tape library name. This field is optional.

Type: Literal

Max value: 8 characters

► &LLQ

The low-level (last) qualifier of the data set or collection name.

Type: Literal

Max value: 8 characters

► &MSPDEST

The destination, specified in data set name format, for a tape management system-driven tape allocation. This value is specified through the AMS pre-ACS installation exit. The data set name format lets you specify a sequence of destinations to be identified, where each qualifier is a specific destination. For example, a data set vaulted first at location OUTD and then sent to OLTS can have an MSPDEST of 'OUTD.OLTS'. The actual values depend on the support provided by your tape management system.

Type: Alphanumeric

Max value: 44 characters

► &MSPARM

Additional information related to a tape management system-driven tape allocation. This is a variable length field that can be indexed. The value is specified through an external exit.

Type: Alphanumeric

Max value: 256 bytes, including a 2-byte length field for each value specified

► &MSPOLICY

The name of a management policy associated with tape data for a tape management system-driven allocation. You can use the DFSMSrmm EDGUX100 installation exit to set MSPOLICY to a VRS management value name. You can also set the value of this variable using the SMS pre-ACS installation exit, or allow your tape management system to set it using the pre-ACS installation exit.

Type: Alphanumeric

Max value: 8 characters

► &MSPOOL

A tape pool name associated with the data set being allocated. In a system-managed tape environment with scratch pool support, you can use this variable to specify a default storage group, where the tape storage group is equivalent to the tape pool specified in the variable. If you use the DFSMSrmm EDGUX100 installation exit, you can set this variable to the pool name or prefix determined by the DFSMSrmm scratch pool processing. This variable can also be set through the pre-ACS installation exit.

Type: Alphanumeric

Max value: 8 characters

► &NQUAL

The number of qualifiers in the data set or collection name.

Type: Numeric

Max value: 22

► &PGM

The name of the program the system is running.

Type: Literal
Max value: 8 characters

► **&SYSNAME**

Specifies the system name of the system on which the ACS routine is executing. This field is optional.

Type: Literal
Max value: 8 characters

► **&SYSPLEX**

Specifies the Parallel Sysplex® name of the system on which the ACS routine is executing. This field is optional.

Type: Literal
Max value: 8 characters

► **&UNIT**

IBM-supplied or installation defined generic name for a device type (for example, 3380, SYSDA).

Type: Literal
Max value: 8 characters

A slash (/) preceding a four digit number represents a unit address (for example, /3380).

When you allocate a tape data set with DISP=MOD, and no unit information is specified in the JCL, this variable is blank and SMS might attempt to manage the tape data set as a DASD-resident data set.

► **&USER**

The user ID of the person allocating the data set. When DFSMSHsm invokes the ACS routines, &USER is either the requestor of the recall or recover, or the user ID of the DFSMSHsm address space. If the environment is recall or recover, the variable is set only if the requestor of the recall or recover is not a DFSMSHsm authorized user.

Type: Literal
Max value: 8 characters

► **&XMODE**

The execution mode in which the data set is being allocated, one of:

BATCH Batch execution mode
TSO TSO execution mode
TASK A started address space

Type: Literal
Max value: 8 characters

12.1.5 Implementing SMS ACS processing

You can move your scratch pool and policy assignment decisions to the SMS ACS routines, where you can use ACS input variables as a base for management class and storage group assignment. In the EDGRMMxx PARMLIB member specify **SMSACS(YES)** so that DFSMSrmm calls the SMS ACS routines for pooling decisions and policy assignment.

Before you enable SMS ACS support in your installation, you must make some changes in your SMS ACS routines.

Defining your SMS environment

To define a SMS environment for non-system managed tapes using the ISMF dialog as shown in Figure 12-6, use the steps in the following sections.

```

                                ISMF PRIMARY OPTION MENU - z/OS DFSMS V1 R12
Selection or Command ==>

0  ISMF Profile                - Specify ISMF User Profile
1  Data Set                    - Perform Functions Against Data Sets
2  Volume                      - Perform Functions Against Volumes
3  Management Class            - Specify Data Set Backup and Migration Criteria
4  Data Class                  - Specify Data Set Allocation Parameters
5  Storage Class               - Specify Data Set Performance and Availability
6  Storage Group               - Specify Volume Names and Free Space Thresholds
7  Automatic Class Selection   - Specify ACS Routines and Test Criteria
8  Control Data Set            - Specify System Names and Default Criteria
9  Aggregate Group             - Specify Data Set Recovery Parameters
10 Library Management          - Specify Library and Drive Configurations
11 Enhanced ACS Management    - Perform Enhanced Test/Configuration Management
C  Data Collection              - Process Data Collection Function
G  Report Generation           - Create Storage Management Reports
L  List                        - Perform Functions Against Saved ISMF Lists
P  Copy Pool                   - Specify Pool Storage Groups for Copies
R  Removable Media Manager     - Perform Functions Against Removable Media
X  Exit                        - Terminate ISMF
Use HELP Command for Help; Use END Command or X to Exit.
```

Figure 12-6 ISMF primary option menu

Defining a tape library

If you do not have such a library, or do not want to associate the SMS storage groups with an existing library, you must define a new library to ISMF. You can do this using the ISMF define library application, but you specify a made-up library ID. For other data fields required for tape library definition, you can select any values you would like. When you do this, OAM issues a CBR3006I message at startup time.

Note: You can skip this section if you do not make the storage group assignments.

1. Choose option **10** (Library Management) on the ISMF PRIMARY OPTION MENU panel (see Figure 12-7) to display the LIBRARY MANAGEMENT SELECTION MENU.
2. Choose option **3** (Tape Library) to display the TAPE LIBRARY APPLICATION SELECTION as shown in Figure 12-7.

```

                                LIBRARY MANAGEMENT SELECTION MENU
Enter Selection or Command ==>

1  Optical Library             - Optical Library Configuration
2  Optical Drive               - Optical Drive Configuration
3  Tape Library                 - Tape Library Configuration

Use HELP Command for Help; Use END Command to Exit.
```

Figure 12-7 ISMF library selection menu

Define a system-managed tape library with a made-up library ID using the Tape Library Application Selection panel as shown in Figure 12-8.

```

                                TAPE LIBRARY APPLICATION SELECTION

Command ==>

To Perform Library operations, Specify:

CDS Name . . . . . 'SYS1.SMS.SCDS'
                                (1 to 44 Character Data Set Name or 'Active')
Library Name . . . . . $VTFM      (For Tape Library list, fully or
                                Partially Specified or * for all)

Select one of the following options :
3  1. List   - Generate a list of Libraries
    2. Display - Display a Library
    3. Define - Define a Library
    4. Alter  - Alter a Library

If List option is chosen,
    Enter "/" to select option      Respecify View Criteria
                                    Respecify Sort Criteria

Use ENTER to Perform Selection;
Use HELP Command for Help; Use END Command to Exit.

```

Figure 12-8 Specifying a library name to SMS for a non-system managed library

Note: In the Library Name field, enter the SMS friendly name for your tape library. This name relates your tape library to your SMS tape storage group, which you define later. There is a minor restriction when you name the library. That is the first character of a library name must not be V, or one of the DFSMSrmm-defined locations (LOCAL, REMOTE, DISTANT).

3. In the tape library define panel (see Figure 12-9), specify a Library ID, the Entry Default Use Attribute and Eject Defaults. Press Enter to perform verification. Click **END** to save and exit the dialog.

Description is a 120-byte field that allows you to enter a description of the library definition for use by the installation. There are no restrictions on its content. The use of the description field is optional.

```

                                TAPE LIBRARY DEFINE                                Page 1 of 4

Command ==>

SCDS Name . : SYS1.SMS.SCDS
Library Name : $VTFM

To Define Library, Specify:

Description ==>
==>
Library ID . . . . . FFFFF      (00001 to FFFFF)
Console Name . . . . .
Entry Default Data Class . . . .
Entry Default Use Attribute . . P      (P=PRIVATE or S=SCRATCH)
Eject Default . . . . . P      (P=PURGE or K=KEEP)

Use ENTER to Perform Verification; Use DOWN Command to View next Panel;
Use HELP Command for Help; Use END Command to Save and Exit; CANCEL to Exit.

```

Figure 12-9 Defining a new SMS tape library (part 1 of 4)

Note: You can use any library ID not used in your installation.

4. In the second panel you can specify a scratch threshold for each media type as shown in Figure 12-10.

TAPE LIBRARY DEFINE		Page 2 of 4
Command ==>		
SCDS Name . : SYS1.SMS.SCDS		
Library Name : \$VTFM		
Media Type:	Scratch Threshold	
Media1:	0	(0 to 999999)
Media2:	0	(0 to 999999)
Media3:	0	(0 to 999999)
Media4:	0	(0 to 999999)
Media5:	0	(0 to 999999)
Media6:	0	(0 to 999999)
Media7:	0	(0 to 999999)
Media8:	0	(0 to 999999)
Media9:	0	(0 to 999999)
Media10:	0	(0 to 999999)
Media11:	0	(0 to 999999)
Media12:	0	(0 to 999999)
Media13:	0	(0 to 999999)
Use ENTER to Perform Verification; Use DOWN Command to View next Panel; Use HELP Command for Help; Use END Command to Save and Exit; CANCEL to Exit.		

Figure 12-10 Defining a new SMS tape library (part 1 of 4)

5. Check your initial online status settings if you have more than one system sharing this SMS environment. To do so, scroll down the current panel that is displayed and set the Initial Online Status for all displayed systems to **YES** and press Enter. See Figure 12-11.

TAPE LIBRARY DEFINE		Page 3 of 4	
Command ==> Y			
SCDS Name . : SYS1.SMS.SCDS			
Library Name : \$VTFM			
Initial Online Status (Yes, No, or Blank):			
SC63	SC64	SC65	SC70
====>	====>	====>	====> YES
Warning:			
When you connect a tape library to a system group rather than a system, you lose the ability to vary that library online or offline to the individual systems in the system group. It is strongly recommended that the tape library be connected to individual systems only.			
Use ENTER to Perform Verification; Use DOWN Command to View next Panel; Use HELP Command for Help; Use END Command to Save and Exit; CANCEL to Exit.			

Figure 12-11 Defining a new SMS tape library (part 2 of 3)

6. Restart your OAM dynamically by using the MVS MODIFY command shown in Figure 12-12 on page 460.

```
MODIFY OAM,RESTART
```

Figure 12-12 Restart OAM address space

Where:

OAM is the name of the OAM started task.

Figure 12-13 shows you the messages you receive after you successfully restart OAM.

```
CBR3006I Library $VTFM with Library ID FFFFF unknown in I/O
configuration.
CBR3002E Library $VTFM no longer usable.
CBR0097I OAM restart completed.
```

Figure 12-13 OAM restart messages

7. Use the MVS DISPLAY SMS command shown in Figure 12-14 to see the newly defined library.

```
D SMS,LIB(ALL),DETAIL
```

Figure 12-14 Display the SMS managed tape libraries status

Figure 12-15 shows the result of the MVS DISPLAY SMS command.

```
CBR1110I OAM library status: 389
TAPE      LIB  DEVICE  TOT  ONL  AVL  TOTAL  EMPTY  SCRTCH  ON  OP
LIBRARY   TYP  TYPE    DRV  DRV  DRV  SLOTS  SLOTS  VOLS
LIB1      AL   349-4L1   4    0    0   1445   32     6   Y   Y
$VTFM     UNK                0    0    0     0     0     0   N   N
```

Figure 12-15 Output of a DISPLAY SMS command

Note: Deleting a tape library using ISMF panels has no effect on the TCDB. Instead, the library definition is removed only from the specified SCDS. To delete a tape library from the TCDB, use the IDCAMS DELETE command.

Defining a tape storage group

To use SMS storage groups, first define tape storage groups using ISMF. To defined tape storage groups you need to have at least one system managed tape library defined to ISMF. See “Defining a tape storage group” on page 460.

The storage group type TAPE is provided to classify tape cartridges in DFSMS. A tape storage group consists of tape libraries and the tape cartridges associated with them. A tape storage group can contain one to eight tape libraries specified by their library name, and one tape library can contain more than one tape storage group.

Note: You can skip this section if you do not make storage group assignments.

To define a Storage Group, follow these steps:

1. Choose option **6** (Storage Group) on the ISMF PRIMARY OPTION MENU to display the STORAGE GROUP APPLICATION SELECTION menu, as shown in Figure 12-6 on page 457.
2. On that panel, specify the Storage Group Name and a Storage Group Type. The Storage Group Type is **TAPE** in our case.
3. Choose option **2** (Define) to display the TAPE STORAGE GROUP DEFINE panel, as shown in Figure 12-16.

STORAGE GROUP APPLICATION SELECTION	
Command ===>	
To perform Storage Group Operations, Specify:	
CDS Name	'SYS1.SMS.SCDS'
	(1 to 44 character data set name or 'Active')
Storage Group Name	VTMSG (For Storage Group List, fully or partially specified or * for all)
Storage Group Type	TAPE (VIO, POOL, DUMMY, COPY POOL BACKUP, OBJECT, OBJECT BACKUP, or TAPE)
Select one of the following options :	
2 1. List	- Generate a list of Storage Groups
2. Define	- Define a Storage Group
3. Alter	- Alter a Storage Group
4. Volume	- Display, Define, Alter or Delete Volume Information
If List Option is chosen,	
Enter "/" to select option	Respecify View Criteria
	Respecify Sort Criteria
Use ENTER to Perform Selection;	
Use HELP Command for Help; Use END Command to Exit.	

Figure 12-16 *Specifying a storage group name to SMS*

4. Figure 12-17 on page 462 shows the Tape Storage Group Define panel. The SCDS Name and Storage Group Name fields are output fields containing the SCDS and storage group names, which you specified in the Storage Group Application Selection panel. The Description field is an optional field of 120 characters in which you can describe the tape storage group. ISMF primes the field with blanks, which are the default.

The Library Names field is used to specify the tape libraries that own the volumes within this storage group. One to eight library names can be associated with a tape storage group. At least one library name must be specified when defining a tape storage group. The library name in the tape storage group definition must also be defined in the same SCDS.

Press Enter to perform verification.

Type the END command to save the newly defined tape storage group and return to the Storage Group Application Selection panel.

```

                                TAPE STORAGE GROUP DEFINE
Command ==>

SCDS Name . . . . . : SYS1.SMS.SCDs
Storage Group Name   : VTFMSG

To DEFINE Storage Group, Specify:

Description ==> FOR THE IBM VIRTUAL TAPE FACILITY FOR MAINFRAME (VTFM)
                ==>

Library Names (1 to 8 characters each):
    ==> $VTFM    ==>          ==>          ==>
    ==>          ==>          ==>          ==>

DEFINE      SMS Storage Group Status . . Y (Y or N)

Use ENTER to Perform Verification and Selection;
Use HELP Command for Help; Use END Command to Save and Exit; CANCEL to Exit.

```

Figure 12-17 Defining a new SMS storage group

- Repeat steps 1 on page 461 through step 4 on page 461 for each storage group you have to define.

Note: For each different DFSMSrmm volume pool, you select by using SMS ACS routines that you have to define a SMS storage group.

- Check your SMS storage group status settings if you have more than one system sharing this SMS environment. To do so, in the DEFINE SMS Storage Group Status field in the panel shown in Figure 12-18, specify **Y** and then press Enter. The settings for each system should match the settings of your SMS tape libraries.

```

                                SMS STORAGE GROUP STATUS ALTER                                Page 1 of 2
Command ==>

SCDS Name . . . . . : SYS1.SMS.SCDs
Storage Group Name : VTFMSG
Storage Group Type : TAPE
To ALTER Storage Group System/
Sys Group Status, Specify:

System/Sys      SMS SG   System/Sys      SMS SG   ( Possible SMS SG Status
Group Name      Status   Group Name      Status   for each:
-----
SC63            ==> NOTCON SC64            ==> NOTCON   - Pool SG Type
SC65            ==> NOTCON SC70          ==> ENABLE   - Tape SG Type
                ==>          ==>          ==>          NOTCON, ENABLE,
                ==>          ==>          ==>          DISNEW, QUIALL, QUINEW
                ==>          ==>          ==>          - Copy Pool Backup SG Type
                ==>          ==>          ==>          NOTCON, ENABLE )
                ==>          ==>          ==>          * SYS GROUP = sysplex
                ==>          ==>          ==>          minus Systems in the
                ==>          ==>          ==>          Sysplex explicitly
                ==>          ==>          ==>          defined in the SCDS

Use ENTER to Perform Verification; Use DOWN Command to View next Panel;
Use HELP Command for Help; Use END Command to Save and Exit; CANCEL to Exit.

```

Figure 12-18 SMS STORAGE GROUP STATUS DEFINE panel

If you successfully defined all your storage groups, you can display them by selecting option 1 (Generate a List Storage Group) in the storage group selection menu (see Figure 12-16 on

page 461). Specify an asterisk (*) in the STORAGE GROUP NAME field to show all defined storage groups in your SMS environment. Figure 12-19 shows you the result of the list.

```

                                STORAGE GROUP LIST
Command ==>                                Scroll ==> HALF
                                           Entries 1-1 of 1
                                           Data Columns 3-6 of 48

CDS Name : SYS1.SMS.SCDS

Enter Line Operators below:

      LINE      STORGRP SG          VIO      VIO      AUTO
      OPERATOR   NAME   TYPE        MAXSIZE  UNIT  MIGRATE
----(1)----  --(2)---  -----(3)-----  --(4)--  (5)-  --(6)---
              VTFMSG  TAPE              -----  ---  -----
              -----  -----  BOTTOM  OF  DATA  -----  -----

```

Figure 12-19 Listing all defined SMS storage groups

Before you activate the new settings validate it using the ISPF dialog. Select 8 on the ISMF primary selection panel (see Figure 12-6 on page 457) to see the CDS APPLICATION SELECTION panel shown in Figure 12-20 than select option4.

```

                                CDS APPLICATION SELECTION      ACTIVATION SCHEDULED
Command ==>

To Perform Control Data Set Operations, Specify:
CDS Name . . 'SYS1.SMS.SCDS'
                                (1 to 44 Character Data Set Name or 'Active')

Select one of the following Options:
  4 1. Display      - Display the Base Configuration
    2. Define       - Define the Base Configuration
    3. Alter        - Alter the Base Configuration
    4. Validate     - Validate the SCDS
    5. Activate     - Activate the CDS
    6. Cache Display - Display CF Cache Structure Names for all CF Cache Sets
    7. Cache Update - Define/Alter/Delete CF Cache Sets
    8. Lock Display  - Display CF Lock Structure Names for all CF Lock Sets
    9. Lock Update  - Define/Alter/Delete CF Lock Sets
If CACHE Display is chosen, Enter CF Cache Set Name . . *
If LOCK Display is chosen, Enter CF Lock Set Name . . . *
                                (1 to 8 character CF cache set name or * for all)
Use ENTER to Perform Selection;
Use HELP Command for Help; Use END Command to Exit.

```

Figure 12-20 Validate the SCDS definitions

Figure 12-21 on page 464 shows you how you can specify a listing data set to get all the messages. Finally press enter.

```

                                VALIDATE ACS ROUTINES OR ENTIRE SCDS
Command ==>

To Perform Validation, Specify:

SCDS Name . . . . . 'SYS1.SMS.SCDS'
                                (1 to 44 Character Data Set Name)

ACS Routine Type . . *   (DC=Data Class, MC=Management Class, SC=Storage
                                Class, SG=Storage Group, *=Entire SCDS)

Listing Data Set . . LISTING
                                (1 to 44 Character Data Set Name)
Use ENTER to Perform Validation;
Use HELP Command for Help; Use END Command to Exit.

```

Figure 12-21 Process SCDS validation

If the validation is finish the result is displayed as shown in Figure 12-22

```

BROWSE   MHLRES7.LISTING                               Line 00000000 Col 001 080
Command ==>                                           Scroll ==> PAGE
***** Top of Data *****
                                VALIDATION RESULTS

VALIDATION RESULT:  VALIDATION SUCCESSFUL - WARNINGS DETECTED
SCDS NAME:          SYS1.SMS.SCDS
ACS ROUTINE TYPE:   *
DATE OF VALIDATION: 2010/10/17
TIME OF VALIDATION: 16:58

IGD06023I STORAGE GROUP MHLEAV IS NOT REFERENCED BY THE STORAGE GROUP ACS ROUTINE
IGD06023I STORAGE GROUP SGMHL02 IS NOT REFERENCED BY THE STORAGE GROUP ACS ROUTINE
IGD06023I STORAGE GROUP SGMHL04 IS NOT REFERENCED BY THE STORAGE GROUP ACS ROUTINE
IGD06023I STORAGE GROUP TPCDB2 IS NOT REFERENCED BY THE STORAGE GROUP ACS ROUTINE

```

Figure 12-22 Validation result messages

After all errors are fixed go back to the the CDS APPLICATION SELECTION panel (see Figure 12-20 on page 463)

You activate the new SMS configuration dynamically using the MVS SETSMS command as shown in Figure 12-23.

```

SETSMS SCDS(SYS1.SMS.SCDS)

```

Figure 12-23 Activate you new SMS settings

Where:

SCDS is your current used SMS source data set.

You see the IDG008I message as shown in Figure 12-24 after successful activation.

```

RESPONSE=SC65      IDG008I NEW CONFIGURATION ACTIVATED FROM SCDS
RESPONSE=SYS1.SMS.SCD

```

Figure 12-24 Activate SMS setting messages

Use the DISPLAY SMS command as shown in Figure 12-25 to display the new defined manual tape library.

```
D SMS,LIB(ALL),DETAIL
```

Figure 12-25 Display SMS Libary command

Figure 12-26 shows you the result of the display SMS command.

```
CBR1110I OAM library status: 116
TAPE      LIB  DEVICE      TOT  ONL  AVL   TOTAL   EMPTY  SCRTCH  ON  OP
LIBRARY   TYP  TYPE       DRV  DRV  DRV   SLOTS   SLOTS   VOLS
$VTFM     UNK                0    0    0       0       0       0  Y  N
LIB1      UNK                0    0    0     1445     1427     0  Y  N
LIB2      AL   3584-L22     16    0    0       260       84      22  Y  Y
```

Figure 12-26 Outout of the display SMS library command

Defining management classes

Because no management class attributes are available for tape cartridges, defining management classes is optional. You have to define policies for management classes by defining vital record specifications. Use the RMM subcommand ADDVRS with the DSNAME operand, or the DFSMSrmm Add Data Set VRS panel in the DFSMSrmm ISPF dialog to define vital record specifications. Use the data set name masks that match the management class names you have defined.

Note: You can skip this section if you do not make management class assignments.

In our examples we define a management class to move the special meaning expiration date decision from the EDGUX100 user exit to SMS ACS. We define a management classes of M99000 (catalog control). You can change the first qualifier from “M” to “D” if you migrate from EDGUX100 to SMS ACS, and you have previously defined all the vital record specifications to match the special meaning expiration dates starting with a “D”.

Note: There are no differences in the VRS definitions to match to a management class, or to match a VRS management value.

To define a management class, the following steps are required:

1. Choose option **3** (Management Class) on the ISMF PRIMARY OPTION MENU panel as shown in Figure 12-6 on page 457 to display the MANAGEMENT CLASS APPLICATION SELECTION panel.
2. To define a management class, you must specify a **CDS Name** and a **Management Class Name** on the panel shown in Figure 12-27 on page 466. The CDS Name must be the name of an SCDS. ISMF primes the field with the last used name. The default is the quoted word 'ACTIVE', which represents the currently active configuration, but you cannot define or alter management classes to the 'ACTIVE' configuration. In the Management Class Name field, you must specify the name of the management class that you are defining.

MANAGEMENT CLASS APPLICATION SELECTION		Page 1 of 2
Command ==>		
To perform Management Class Operations, Specify:		
CDS Name	'SYS1.SMS.SCDS'	(1 to 44 character data set name or 'Active')
Management Class Name . . .	M99000	(For Management Class List, fully or partially specified or * for all)
Select one of the following options :		
3	1. List	- Generate a list of Management Classes
	2. Display	- Display a Management Class
	3. Define	- Define a Management Class
	4. Alter	- Alter a Management Class
If List Option is chosen,		
Enter "/" to select option	Respecify View Criteria	Respecify Sort Criteria
Use ENTER to Perform Selection; Use DOWN Command to View next Selection Panel; Use HELP Command for Help; Use END Command to Exit.		

Figure 12-27 ISMF management class selection

- Choose Option **3** (DEFINE) on the panel to display the MANAGEMENT CLASS DEFINE panel shown in Figure 12-28.

MANAGEMENT CLASS ALTER		Page 1 of 5
Command ==>		
SCDS Name : SYS1.SMS.SCDS		
Management Class Name : M99000		
To ALTER Management Class, Specify:		
Description ==> Special meaning expdt 99000 whilecatalog ==> for DFSMSrmm only		
Expiration Attributes		
Expire after Days Non-usage . .	NOLIMIT	(1 to 9999 or NOLIMIT)
Expire after Date/Days	NOLIMIT	(0 to 9999, yyyy/mm/dd or NOLIMIT)
Retention Limit	NOLIMIT	(0 to 9999 or NOLIMIT)
Use ENTER to Perform Verification; Use DOWN Command to View next Panel; Use HELP Command for Help; Use END Command to Save and Exit; CANCEL to Exit.		

Figure 12-28 ISMF Panel: Management Class Define

You can use the optional description field to describe the management class. All other fields are not used for a management class defined for DFSMSrmm use.

Tip: You do not have to define any parameters in the ISMF definition panels, because the management class attributes are not used by DFSMSrmm.

You can restart your SMS system dynamically by using the MVS SETSMS command command shown in Figure 12-29.

```
SETSMS SCDS(SYS1.SMS.SCDS)
```

Figure 12-29 Activate you new SMS settings

Where:

SCDS is your current used SMS source data set.

You see the IDG008I message as shown in Figure 12-30 after successful activation.

```
RESPONSE=SC65      IDG008I NEW CONFIGURATION ACTIVATED FROM SCDS
RESPONSE=SYS1.SMS.SCD
```

Figure 12-30 Activate SMS setting messages

Tailoring your SMS management class routines

Update your ACS routine to use an appropriate filter list and logic to assign management classes to data sets on tape. When a volume is opened, DFSMSrmm records the management class name you assign to new tape data sets using your ACS routine.

Example 12-1 shows a management class ACS routine that maps some of the special meaning expiration dates, for example 99000, to a management class name. If there is no match, a management class of CATALOG is set.

Example 12-1 Sample ACS management class routine

```
PROC 1 &MGMTCLAS
/*****
/* DEFINE FILTER FOR TAPE UNITS */
/*****
FILTLIST TAPEUNITS INCLUDE(TAPE*,T3420*,T3480*,T3490*,
                           '3420','3480','3490',
                           T3590*,'3590','3592')
/*****
/* DATA SET NAME FILTLIST DEFINITIONS */
/*****
FILTLIST MCDB20 INCLUDE(DB2P.DSNDB%.*.%0*.*,
                       DB2T.DSNDB%.*.T0*.*,
                       DB2D.DSNDB%.*.T0*.*)

FILTLIST MCDB21 INCLUDE(DB2P.DSNDB%.*.M1*.*,
                       DB2D.DSNDB%.*.T1*.*,
                       DB2T.DSNDB%.*.T1*.*)

FILTLIST MCDB22 INCLUDE(DB2D.DSNDB%.*.T2*.*,
                       DB2T.DSNDB%.*.T2*.*)
/*****
/* Support special meaning expiration dates */
```

```

/*****/
IF &UNIT = &TAPEUNITS THEN DO
  WRITE 'MC ACS GETS CONTROL TO SET EXPDT MC'

  SELECT
    WHEN (&EXPDT = '1998001') SET &MGMTCLAS = 'M98001'
    WHEN (&EXPDT = '1999002') SET &MGMTCLAS = 'M98002'
    WHEN (&EXPDT = '1999003') SET &MGMTCLAS = 'M98003'

    WHEN (&EXPDT = '1990000') SET &MGMTCLAS = 'M99000'
    WHEN (&EXPDT = '1999000') SET &MGMTCLAS = 'M99000'

    WHEN (&EXPDT = '1999001') SET &MGMTCLAS = 'M99001'
    WHEN (&EXPDT = '1999002') SET &MGMTCLAS = 'M99002'
    WHEN (&EXPDT = '1999003') SET &MGMTCLAS = 'M99003'
    WHEN (&EXPDT = '1999004') SET &MGMTCLAS = 'M99004'

    OTHERWISE SET &MGMTCLAS = 'CATALOG'

  END
END
/*****/
/* SELECT VALID MANAGEMENT CLASS */
/*****/

SELECT
/*****/
/* Accept special meaning expdt */
/*****/
WHEN (&MGMTCLAS = '' && &DSN = &MCDB20) SET &MGMTCLAS = 'MCDB20'

WHEN (&DSN = &MCDB21) SET &MGMTCLAS = 'MCDB21'

/*****/
/* Accept any expdt */
/*****/
WHEN (&EXPDT = '' && &DSN = &MCDB20) SET &MGMTCLAS = 'MCDB20'

END
END

```

Important: If you allow the use of special meaning expiration dates in your computing center, you should specify the entire range of these dates in order to not lose data.

This example can be used for both system managed and non-system managed tape volumes. If you use the example for non-system managed tape volumes only, you must include the code shown in Example 12-2.

Example 12-2 Sample ACS management class routine for tape only

```

IF &ACSENVIR = 'RMMVRS' THEN
  SELECT
    WHEN (&EXPDT = '1990000') SET &MGMTCLAS = 'M99000'
    WHEN (&EXPDT = '1998001') SET &MGMTCLAS = 'M98001'
    ....

```

```

        WHEN (&EXPDT = '1999000') SET &MGMTCLAS = 'M99000'
        WHEN (&EXPDT = '1999002') SET &MGMTCLAS = 'M99002'
    ....
END

```

Tailoring your SMS storage group routines

To assign a storage group, you must have the SMS subsystem active, and have a valid SMS configuration.

You use the ACS routines to process the special calls that DFSMSrmm makes to the SMS subsystem for ACS processing. DFSMSrmm requests that the management class and storage group routines are run. The environment variable is set to RMMPOOL so that you can differentiate allocation requests for system-managed data sets from requests by DFSMSrmm for a storage group name. When DFSMSrmm calls the ACS routines with the &ACSENVIR variable, set to either RMMPOOL or RMMVRS; the storage class (&STORCLAS variable) is set to the word USERMM.

Attention: You must use the EDGUX100 exit to request DFSMSrmm to disable the tape drive cartridge loader to prevent specific scratch pool requests from emptying the loaders through volume rejection. This function is not available using SMS ACS routines to select scratch pools.

Example 12-3 shows a simple example to use a specific scratch pool if the high-level qualifier is MHLRES1 or MHLRES4 for the new tape data set. Both volume pools must be defined in DFSMSrmm.

To make your decisions, you can use all read only variables described in 12.1.4, “SMS read-only variables” on page 452.

Example 12-3 *ACS storage group routine for non-system managed tape*

```

IF &ACSENVIR = 'RMMVRS' THEN EXIT

IF &ACSENVIR = 'RMMPOOL' THEN DO
    WRITE 'SG ACS GETS CONTROL &ACSENVIR=' &ACSENVIR
    SELECT
        WHEN (&HLQ = 'MHLRES1') SET &STORGRP = 'PL3490'
        WHEN (&HLQ = 'MHLRES4') SET &STORGRP = 'LOAN'
    END
    EXIT
END

```

Tip: The SMS ACS management class routine is called first so you can use this routine to make much more powerful decisions in this routine, and to use the result in the SMS ACS storage group routine.

Tailoring your EDGRMMxx PARMLIB member

Specify SMSACS(YES) in the EDGRMMxx PARMLIB member as shown in Figure 12-31 on page 470 to control whether DFSMSrmm calls SMS ACS processing to enable the use of storage group and management class values with DFSMSrmm.

Add new VLPOOL commands or update your previous defined VLPOOL commands in the EDGRMMxx PARMLIB member (see Figure 12-31 on page 470) to specify the NAME

operand. If the NAME is a valid storage group name, it is used as the default STORGRP value for all volumes added into this pool range. You can choose another storage group name when adding a volume by using the STORGRP operand on the ADDVOLUME command.

The same NAME value can be used on multiple VLPOOL commands enabling you to group multiple prefix ranges into a single logical pool without the need to specify STORGRP on ADDVOLUME.

By naming a specific storage group on ADDVOLUME, you can select at the individual volume level to which pool it belongs.

OPTION	OPMODE (R)	/* Record-Only Mode	*/	-
				
	SMSACS (YES)	/* enable MV ACS processing	*/	-
				
VLPOOL	PREFIX (VT*)			-
	TYPE (R)			-
	RACF (Y)			-
	NAME (VTFM)			-
	EXPDTCHECK (Y)			-
	DESCRIPTION ('FOREIGN CARTRIDGES')			
VLPOOL	PREFIX (L*)			-
	TYPE (S)			-
	RACF (Y)			-
	NAME (LOAN)			-
	EXPDTCHECK (Y)			-
	DESCRIPTION ('TAPES FOR LOAN LOC PROCESSING')			

Figure 12-31 EDGRMMxx PARMLIB member to support SMS ACS processing

Note: For manual tape libraries, DFSMSrmm uses the assigned storage group as the pool name to validate that a mounted scratch volume is from the requested pool.

To restart DFSMSrmm, use the modify command as shown in Figure 12-32 to implement updates to the data set, and avoid stopping and restarting DFSMSrmm.

F DFRMM,M=xx

Figure 12-32 Restart DFSMSrmm to use the new parmlib settings

Where:

xx is the suffix of the EDGRMMxx PARMLIB member.

You can also restart DFSMSrmm using another PARMLIB data set member with parameters to implement changes.

Figure 12-33 shows you the the messages after DFSMSrmm is successfully restarted.

```
EDG1101I DFSMSrmm MODIFY COMMAND ACCEPTED
EDG0204I DFSMSrmm BEING INITIALIZED FROM MEMBER EDGRMM03 IN RMM.PARMLIB
EDG0105I DFSMSrmm SUBSYSTEM INITIALIZATION COMPLETE
```

Figure 12-33 DFSMSrmm subsystem re-start messages

Adding volumes to the volume pool

You can add new volumes to DFSMSrmm using the TSO RMM ADDVOLUME subcommand, or you can use the TSO RMM CHANGEVOLUME subcommand to update details for volumes already defined to DFSMSrmm. For both functions, you can also use the RMM ISPF dialog.

Use the RMM TSO ADDVOLUME subcommand shown in Figure 12-34 to add 10 new scratch volumes.

```
RMM ADDVOLUME LOAN01 COUNT(10) STATUS(SCRATCH) INIT(Y) POOL(VT*)
```

Figure 12-34 RMM ADDVOLUME subcommand

Where:

COUNT	specifies the number of volumes to be added.
STATUS	specifies the volume status.
INIT	specifies whether the volume must be initialized before it can be used.
POOL	specifies a POOL where DFSMSrmm stores the volume in the removable media library

Note: When you do not specify a storage group name, DFSMSrmm assigns a storage group name by using the matching EDGRMMxx VLPOOL NAME value. If the VLPOOL NAME value is a valid storage group name, DFSMSrmm uses the VLPOOL NAME value as the default value for volumes added to the pool.

If you are using previously defined volumes, use the RMM TSO CHANGEVOLUME subcommand shown in Figure 12-35 to update some volume details.

```
RMM CHANGEVOLUME VT0001 POOL(VT*) STORAGEGROUP(VTFM)
```

Figure 12-35 RMM CHANGEVOLUME subcommand

Here, STORAGEGROUP specifies the SMS-defined storage group to which the volume belongs. POOL specifies a POOL where DFSMSrmm stores the volume in the removable media library.

Defining your vital record specifications

For all SMS management classes you have defined and set in your SMS ACS routines, you must add a vital record specification (VRS) in DFSMSrmm to define retention and movement policies. See Chapter 5, “Retaining and Moving Your Volumes” in *DFSMSrmm Guide and Reference*, SC26-7404, for more information about DFSMSrmm VRS processing. A management class vital record specification must be defined as a data set vital record

specification, and the data set name mask must match the management class name defined in SMS.

You can add new vital record specifications to DFSMSrmm using the TSO RMM ADDVRS subcommand, or you can also use the RMM ISPF dialog.

Use the commands as shown in Figure 12-36 to define all new data set vital record specifications used in the sample ACS management class routine.

```
/* define VRs for special meaning expdt processing 98001 to 98365 */
RMM ADDVRS DATASETNAME('M98001') COUNT(1) LASTREFERENCEDAYS
RMM ADDVRS DATASETNAME('M98002') COUNT(2) LASTREFERENCEDAYS
RMM ADDVRS DATASETNAME('M98003') COUNT(2) LASTREFERENCEDAYS
/* define VRs for special meaning expdt processing 99000 and 99000 */
RMM ADDVRS DATASETNAME('M99000') CYCLES COUNT(99999) WHILECATALOGED
/* define VRs for special meaning expdt processing 99001 to 99364 */
RMM ADDVRS DATASETNAME('M99001') COUNT(1) CYCLES
RMM ADDVRS DATASETNAME('M99002') COUNT(2) CYCLES
RMM ADDVRS DATASETNAME('M99003') COUNT(3) CYCLES
RMM ADDVRS DATASETNAME('M99004') COUNT(4) CYCLES
/* define VRs for all other used management classes */
RMM ADDVRS DATASETNAME('MCDB20') COUNT(20) BYDAYCYCLES WHILECATALOG -
    UNTILEXPIRED SCRATCHIMMEDIATE NEXTVRS(SAVE)
RMM ADDVRS NAME(SAVE) LOCATION(SAVE) COUNT(5)
RMM ADDVRS DATASETNAME('MCDB21') COUNT(21) CYCLES WHILECATALOG -
    UNTILEXPIRED SCRATCHIMMEDIATE NEXTVRS(EXTRA)
RMM ADDVRS NAME(EXTRA) LOCATION(CURRENT) EXTRADAYS(2)
```

Figure 12-36 Defining new data set name vital record specifications

12.1.6 Migration considerations

If you are already using DFSMSrmm, use this new function to migrate your EDGUX100 management value and volume pool selection functions to the DFSMSrmm SMS ACS support as described previously.

Move pooling decisions out of EDGUX100 user exit

You can implement DFSMSrmm scratch pooling using ACS routines based on storage group names. DFSMSrmm provides support for non-system-managed tape and for system-managed manual tape libraries. This support enables pooling at the individual volume level. You assign a storage group name to each volume by:

- ▶ Using DFSMSrmm TSO subcommands
- ▶ Using pool information that you define with the DFSMSrmm EDGRMMxx PARMLIB VLPOOL command

You use the ACS routines to process the special calls that DFSMSrmm makes to the SMS subsystem for ACS processing. DFSMSrmm requests that the management class and storage group routines are run.

Moving VRS management decisions out of EDGUX100 user exit

You use the ACS routines to process the special calls that DFSMSrmm makes to the SMS subsystem for ACS processing. In replacement of the EDGUX100 user exit VRS management value, DFSMSrmm requests that the management class routine is run.

For each new tape data set, DFSMSrmm processing follows the steps described here depending on the circumstances (see Figure 12-37):

- ▶ Pre-ACS processing (see “Pre-ACS support as a way to migrate” on page 473 for details):
 - IGDACSXT exit is called, you can implement this exit to set values to &MSPOLICY, &MSDEST, &MSPOOL, and &MSPARM ACS read-only variables before ACS routines are called. Then your ACS routines can refer to these variables.
 - DFSMSrmm calls EDGUX100 installation exit to enable a VRS management value to be returned. The selected value is returned in the &MSPOLICY variable.
- ▶ OPEN processing after a device has been allocated:
 - DFSMSrmm calls SMS ACS processing to proceed the management class ACS routine passing environment information.
 - If a management class is not returned from ACS processing, DFSMSrmm calls the EDGUX100 installation exit to enable a VRS management value to be returned.
 - The specified management class or VRS management value is recorded by DFSMSrmm at the data set level.

Attention: The environment variable &ACSENVIR is set to RMMVRS so that you can differentiate allocation requests for system managed data sets from requests by DFSMSrmm for a management class name.

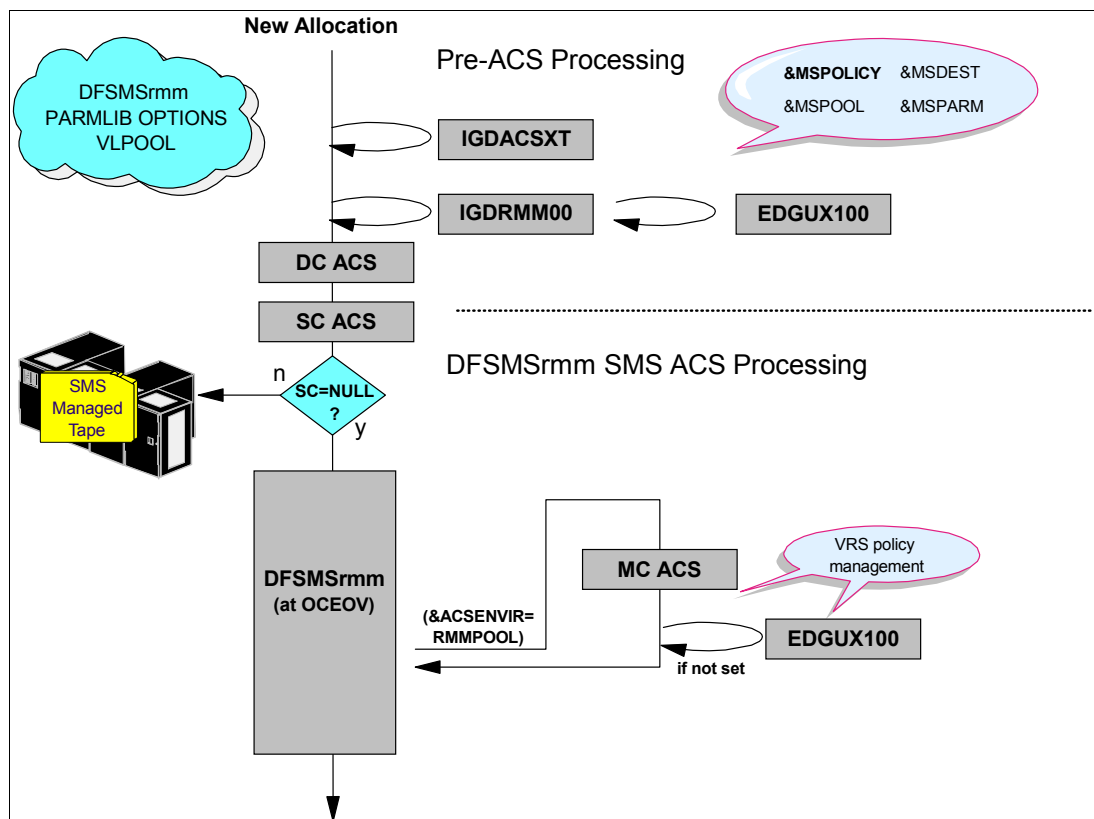


Figure 12-37 DFSMSrmm V2R10 SMS ACS VRS policy management support

Pre-ACS support as a way to migrate

Using the SMS pre-ACS calls to DFSMSrmm, you can use any existing DFSMSrmm and EDGUX100 exit scratch pool and EDGUX100 exit policy assignment decisions as input to

your SMS ACS logic. This enables you to direct new data set allocations to the correct media. You can use SMS ACS to decide if data sets are to be system-managed or non-system-managed. For system-managed tape data sets, you can decide to which library or library type you want the allocation directed. This can help you with VTS implementation.

You can use the pre-ACS interface to provide additional information like vault destination or pool information to the ACS routines. You can use the EDGUX100 installation exit for pre-ACS processing. When called during pre-ACS processing, values selected by the exit are used as input to the ACS routine.

DFSMSrmm always attempts to pass values for the &MSPOOL and &MSPOLICY ACS read-only variables. If you do not use an EDGUX100 exit, the &MSPOOL variable is set to the DFSMSrmm system-based scratch pool decision. If you do not use an EDGUX100 exit, there is never a variable set for &MSPOLICY. DFSMSrmm sets a pre-ACS variable only if the variable has not yet been set using the pre-ACS exit. Since DFSMSrmm gets control after the installation exit, any vendor or customer decisions take precedence.

DFSMSrmm can set the following ACS read-only variables:

- **Pool name &MSPOOL**

Used to specify a tape pool name associated with a data set being allocated. In a system managed tape library, this variable might be used to specify a default storage group where the storage group is equivalent to the tape pool specified in the library.

- **Policy name &MSPOLICY**

Used to specify a management policy related to a system-managed library

If you only want to use &MSPOLICY and &MSPOOL variables, it is enough to code the EDGUX100 installation exit. EDGUX100 gets control right after IGDACSXT returns the control to the system, and it sets the values only if IGDACSXT did not set them.

In this way, you can use EDGUX100 assignment decisions as input to your SMS ACS logic to enable you to direct new data set allocations to the correct media. The pre-ACS routine passes the address of the ACS installation exit control block ACERO to the EDGUX100 installation exit. The ACERO is mapped by the IGDACERO macro and is the input to the pre-ACS installation exit IGDACSXT. For more information about how you can use the ACERO refer to *z/OS DFSMS Installation Exits*, SC26-7396.

You can use any of the values in the ACERO as input to the EDGUX100 installation exit. The sample EDGUX100 exit uses these values:

- **ACEROJOB:** What is the job name
- **ACERODSN:** What is the data set name
- **ACEROEXP:** What is the expiration date. The expiration date is used only if the retention period is not set.
- **ACERORTP:** What is the retention period

The values are used to perform PL100_CAN_IGNORE, PL100_CAN_VRS, and PL100_CAN_POOL processing. All other functions are performed only when the EDGUX100 is not called by the pre_ACS routine.

If your installation's version of the EDGUX100 installation exit provides values for scratch pooling (PL100_POOL) or vital record specification management value (PL100_VRS), DFSMSrmm updates the ACS input values for MSPOOL and MSPOLICY when they have not already been set by the IGDACSXT pre-ACS installation exit.

Do I use pre-ACS or DFSMSrmm SMS ACS processing

Our intention is to enable you to move function out of the EDGUX100 installation exit. We recommend that you do not use the PREACS option in the EDGRMMxx PARMLIB member.

You can use the existing decisions coded in your EDGUX100 exit to input into your SMS ACS logic. Within system-managed tape, you can decide which library or library type you would like the allocation to be directed. This can help with VTS implementation. Do this by using the pre-ACS processing.

Implement or customize the sample EDGUX100 exit (or the EDGCVRSX sample of EDGUX100) to feed pool and VRS management value information into ACS. Base your ACS processing decisions on the MSPPOOL and MSPOLICY values that come from DFSMSrmm scratch pool and EDGUX100 exit processing. You can plan to move the EDGUX100 exit decisions into your SMS ACS routines to enable the EDGUX100 exit processing to be ignored or removed.

Using SMS ACS processing, you can consolidate scratch pooling and policy assignment decisions in a single place whether you use system managed tape.

12.2 Partitioning a SMS library

Depending on how you want to partition your library, there are different methods. In any case, you need to use the following resources:

- ▶ Scratch categories. DEVSUPxx PARMLIB member
- ▶ DFSMSrmm PARMLIB control: REJECT ANYUSE(prefix)
- ▶ DFSMSrmm exits: EDGUX100 and EDGUX200
- ▶ OAM exit: CBRUXENT

When a cartridge enters the tape library, it may be accepted and assigned a corresponding category by a system, or it may be rejected depending on the partition configuration.

Figure 12-38 on page 476 shows the procedure a system used to decide to accept or reject a volume.

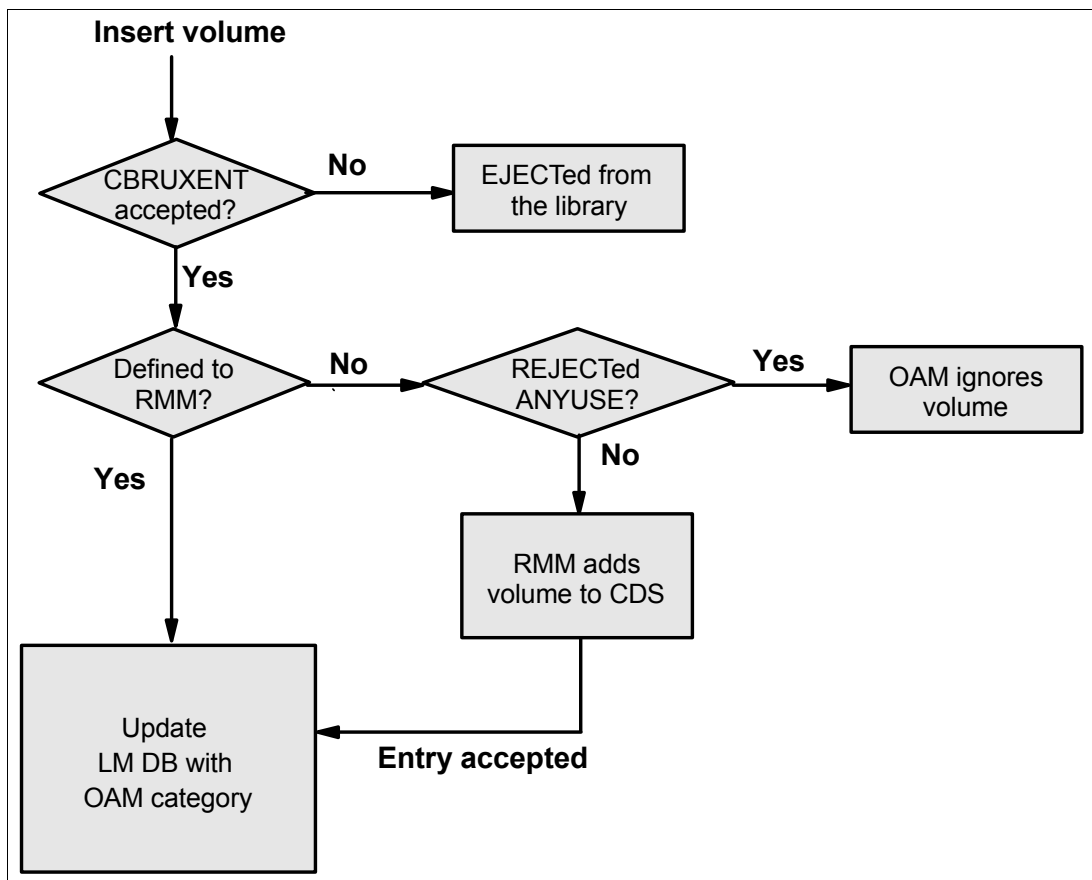


Figure 12-38 Volumes enter a tape library

Note: When you change the CBRUXENT exit, you need to assemble and link edit the module after each maintenance of it.

You base your acceptance of the volumes on your definitions in PARMLIB. The REJECT shown in Figure 12-39 operand allows you to select the volumes or the volume prefixes allowed in each system:

REJECT ANYUSE(A*)	on SYSTEM B
REJECT ANYUSE(B*)	on SYSTEM A

Figure 12-39 REJECT ANYUSE definitions

With these sentences you are preventing the use of volumes A* on SYSTEM B and volumes B* on SYSTEM A.

The following sections document the different library partitioning scenarios that we found. In each section, you learn how partitioning is implemented.

Partitioning a library between MVS and other systems

You can use DFSMSrmm to manage all your volumes. MVS volumes are managed automatically, and volumes from other systems are managed manually. Each time you add a

volume, specify whether this volume will be used from MVS or from another system by using the TSO ADDVOLUME subcommand operand shown in Figure 12-40.

```
USE(MVS)
or
USE(VM)
or
USE(IRMM)
```

Figure 12-40 RMM ADDVOLUME subcommand with the USE attribute

See Figure 12-41 how you can add VM0001 volume to be used only from a VM system.

```
RMM ADDVOLUME VM0001 DENSITY(3480) OWNER(YCJRES)-
STATUS(USER) USE(VM) MEDIANAME(3480)
```

Figure 12-41 RMM ADDVOLUME subcommand for a volume used on a VM system

MVS uses only its owns volumes, and DFSMSrmm manages these volumes automatically.

If you prefer, DFSMSrmm has information about the status of the VM volumes; you need to change this information manually using commands.

Partitioning a library with one CDS and one TCDB

The scenario is shown in Figure 12-42. We have one library and two systems sharing the same CDS and the same TCDB.

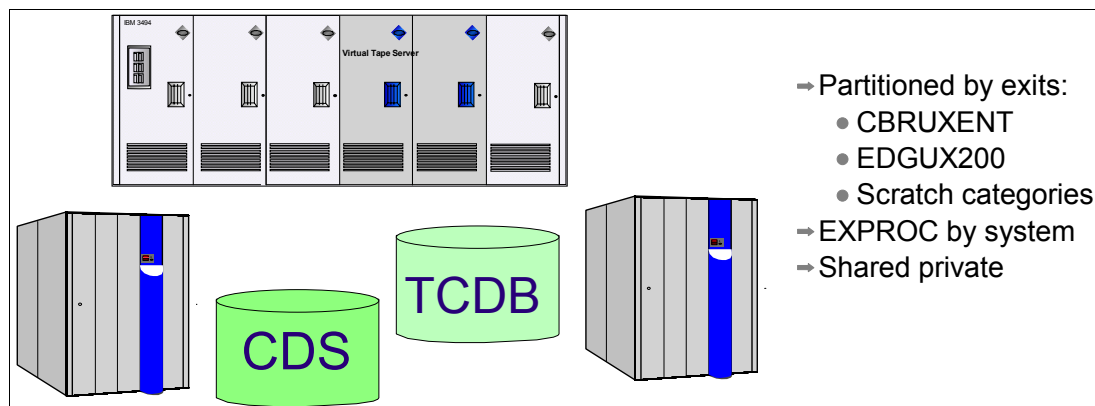


Figure 12-42 Scenario with one CDS and one TCDB

Both the TCDB and CDS are shared between systems; you need to modify CBRUXENT and EDGUX200 exit to partition the library. When a volume that was moved out of the library comes back again because a record for this volume exists in the CDS, then the REJECT command in EDGRMMxx does not work.

This kind of partition offers you a convenient way to share private volumes among systems for all the volume entries are in the same TCDB. No additional consideration is introduced by this partition, because both CDS and TCDB contain the information of all SMS volumes.

You need to run EXPROC processing on each system name, and use a customized EDGUX200 exit which prevents volumes returning to scratch on the wrong system. The

decisions about which volumes to process should match those coded in the customized CBRUXENT exit.

Partitioning a library with two CDSs and one TCDB

The scenario is shown in Figure 12-43. We have one library, two systems, each system with its own CDS and the same TCDB.

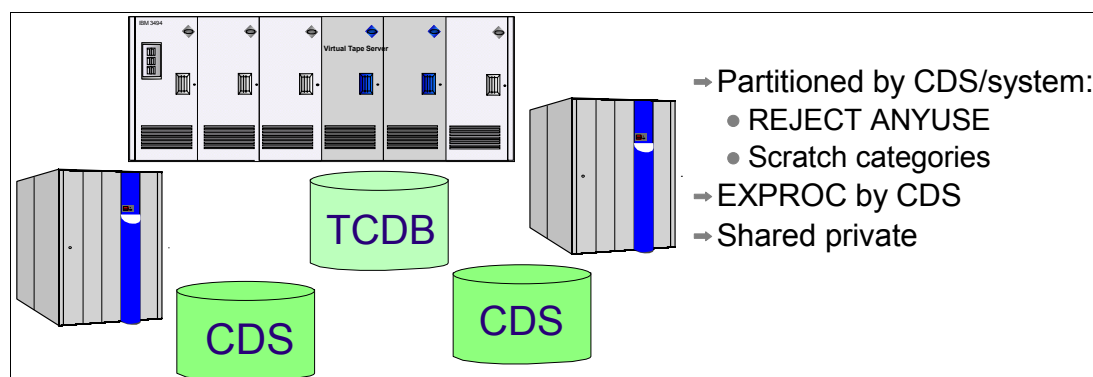


Figure 12-43 Scenario with two CDSs and one TCDB

The tape library is partitioned with CDS. You can use a selective REJECT command in EDGRMMxx for DFSMSrmm to partition tape libraries.

You can access all volumes, because the same TCDB is used by both systems. If you want to use private volumes from the other system, specify EXPDT=98000 since it is not recorded by the CDS in the system in which you run the job.

You need to run inventory management on each system by CDS.

Partitioning a library with two CDSs and two TCDBs

The scenario is shown in Figure 12-44. We have one library and two systems. Each system has its own CDS and TCDB.

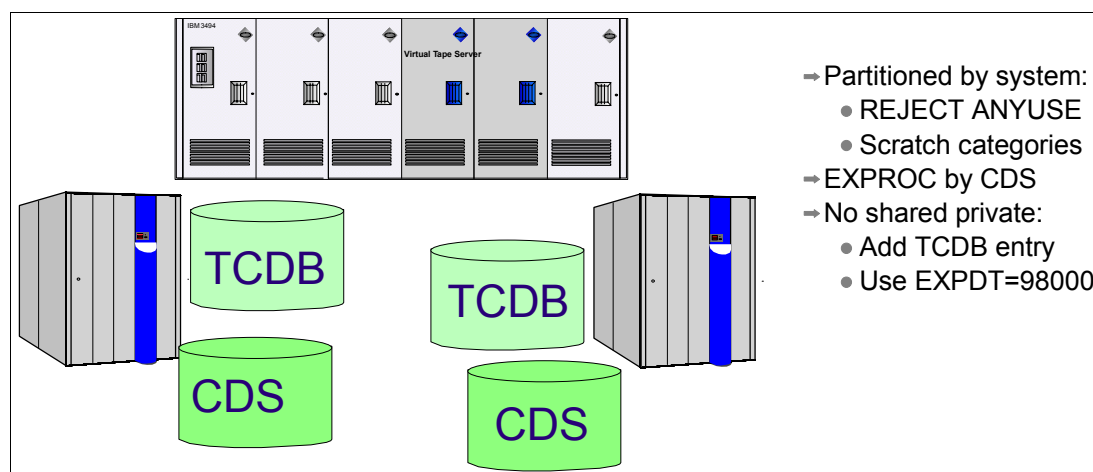


Figure 12-44 Scenario with two CDSs and two TCDBs

Since each system has its own CDS, you can use REJECT ANYUSE(*) to prevent the OAM address space to record information of volumes that you do not plan to use in the specific system.

Since private volumes are not shared by systems, define the entry in TCDB when you plan to use a private volume of the other system. Also specify EXPDT=98000 when using these volumes, because there is no information about them in CDS. After use, remember to delete the volume entries in TCDB.

You need to run the inventory management task by CDS.

Partitioning a library with one CDS and two TCDBs

This scenario is shown in Figure 12-45. We have one library, two systems, each system with its TCDB and sharing one CDS.

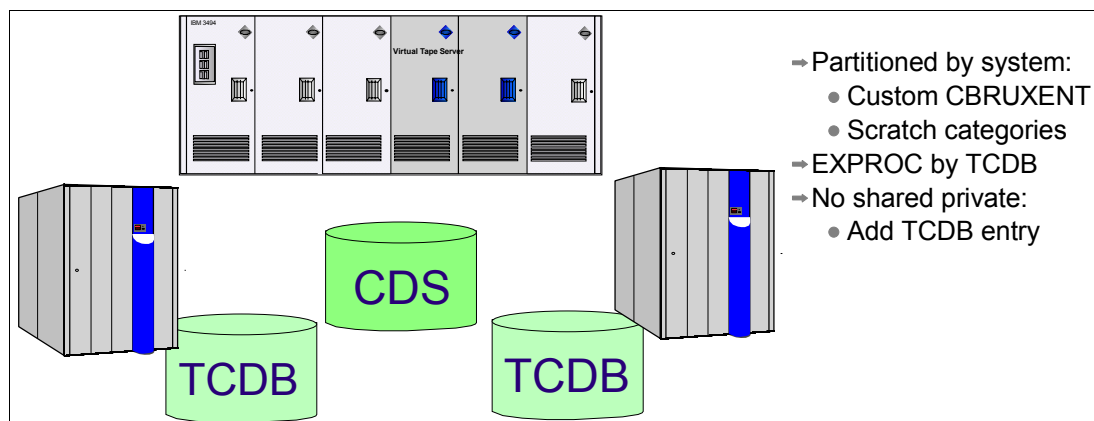


Figure 12-45 Scenario with two TCDBs and one CDS

The CDSs are shared among systems, and the tape library is partitioned with TCDB. You need to modify CBRUXENT to partition a library. If you use the selective REJECT command in EDGRMMxx, it may result in a volume being recorded in the wrong TCDB. When a volume is moved out of the library and then comes back again because one record for this volume exists in the CDS, the REJECT command in EDGRMMxx does not work.

Private volumes are not shared because each system has its own TCDB. If you want to use private volumes from another system, you must use add volume entry into the TCDB using the ASM utility. You need to delete the entries after work to avoid potential problems.

Although only one CDS is used, inventory management tasks should be run on each system, once for each TCDB, because DFSMSrmm can only return volumes to scratch on a system which has access to the TCDB containing the volume.

12.3 Using DFSMSrmm with virtual tape solutions

Virtual tape solutions are always related with the use of an automated tape library, system managed or non-system managed. Therefore, virtual tape solutions have a different way of working than the traditional tapes. This is why we want to make a separate section for explaining how they work, and how DFSMSrmm provides special support for these solutions and their specific functions.

The DFSMSrmm support for virtual solutions is basically the same for IBM Virtual Tape Server and for other implementations, but there are some specifics for VTS. The interface between DFSMSrmm and OAM is built-in and automated; the interface for other virtual tape solutions must be driven by DFSMSrmm commands.

DFSMSrmm supports virtual tape server import/export in different ways depending on whether DFSMSrmm stacked volume support is enabled. When you enable stacked volume support, DFSMSrmm tracks logical volumes and stacked volumes.

To verify that the stacked volume support is enabled, go to the DFSMSrmm Control Record Display panel shown in Figure 12-46.

DFSMSrmm Control Record Display

Command ==>

More:

+

Type . . : MASTER

Create date . : 2009/309

Create time . : 18:38:11

Update date . : 2010/291

Update time . : 12:55:05

Journal utilization . : 0 % (75 % threshold)

Status : ENABLED

CDS utilization . . . : 19 %

Exit status:

Options:

EDG_EXIT100 . . : ENABLED

Stacked volumes . . . : NONE

EDG_EXIT200 . . : ENABLED

Extended bin . . . : ENABLED

EDG_EXIT300 . . : ENABLED

Common time : DISABLED

Figure 12-46 DFSMSrmm Control Record Display panel

If it is not enabled, use the EDGUTIL program, as shown in Figure 12-47.

```
//EDGUTIL JOB <JOB CARD PARAMETERS>
//*
//CREATE EXEC PGM=EDGUTIL,
//      PARM='UPDATE'
//MASTER DD DISP=SHR,DSN=RMM.CONTROL.DSET
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
CONTROL CDSID(PROD) STACKEDVOLUME(YES)
/*
```

Figure 12-47 EDGUTIL example

12.3.1 VTS support for stacked volumes

Prior to OS/390 R10 DFSMSrmm, VTS support was limited to new logical volume type, with the ability to record the container volume for any volume defined to DFSMSrmm. Stacked volumes are not defined nor managed as a stacked volume for DFSMSrmm.

With OS/390 V2R10, DFSMSrmm supports stacked volumes defined to DFSMSrmm. If you do not define stacked volumes, DFSMSrmm defines them automatically at export time if stacked volume support is enabled.

To define stacked volumes, you can use ISPF panels or the RMM TSO subcommand shown in .

```
RMM ADDVOLUME ST0000 TYPE(STACKED) STATUS(MASTER) LOCATION(LIBVTS)
```

Figure 12-48 RMM ADDVOLUME subcommand to add a stacked volume

Note: You cannot remove stacked volume support after it is enabled. We recommend that you enable stacked volume support whether you have a VTS.

When you define a stacked volume, DFSMSrmm checks the TCDB and the library manager. If the volume is not known to the library manager, the volume location is set to SHELF and the destination is set to the VTS library name.

Figure 12-49 shows the VTS support for stacked volumes.

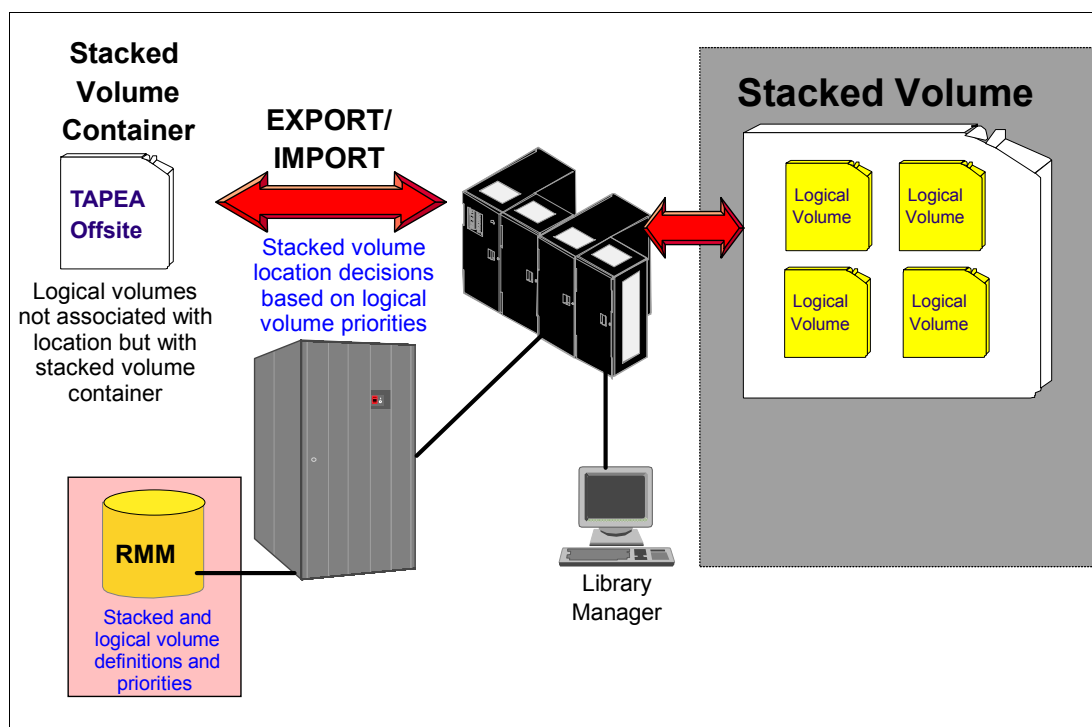


Figure 12-49 VTS support for stacked volumes

At export time, the DFSMSrmm control data set shows that logical volumes are on stacked volumes (field: In container), and stacked volumes are known and managed by DFSMSrmm.

The movement of the stacked volumes is the specified for the logical volumes. DFSMSrmm drives the stacked volume movement based on the required location of the contained volume.

Export processing

The example explains how to export a data set IBMUSER.TAPE3490.PRUACS on logical volume MWW000 without previously defining the stacked volume:

1. Define the vital record specification for the data sets you want exported as shown in Figure 12-50 on page 482

```
RMM ADDVRS DSNAME('IBMUSER.TAPE3490.**') DAYS COUNT(3) -  
LOCATION(REMOTE)
```

Figure 12-50 RMM ADDVRS subcommand

2. Run inventory management VRSEL processing to determine which logical volumes should be moved and to set the required location for each volume. For our volume, you see the information in Figure 12-51.

Location:	Current	Destination	Old	Required	Home
Name	= LIB2	REMOTE			LIB2
Type	= AUTO	STORE			AUTO

Figure 12-51 VRSEL processing output

3. See Figure 12-52 to create a list of volumes to export.

```
RMM SV VOLUME(*) LIMIT(*) REQUIRED(REMOTE) LOCATION(ATLBA999) -  
CLIST('',' REMOTE')
```

Figure 12-52 RMM SEARCHVOLUME subcommand

4. Start the export process as described in the *DFSMS OAM Planning, Installation, and Storage Administrator Guide for Tape Libraries*, SC35-0427. During this process, DFSMSrmm updates the field “in container” in the logical volume.

The stacked volume **DD0813** is added to DFSMSrmm, with a volume type of STACKED.

5. Run inventory management DSTORE processing to set the destination for the volume.
6. Eject the stacked volume from the VTS from the Library Manager.
7. Confirm that the stacked volume has been moved by using the command shown in Figure 12-53

```
RMM CHANGEVOLUME DD0813 CONFIRM MOVE
```

Figure 12-53 RMM CHANGEVOLUME subcommand to confirm a volume move

Import processing

DFSMSrmm supports the importing of logical volumes that may be defined to DFSMSrmm. During import processing for a logical volume, DFSMSrmm automatically adds the volume information to the DFSMSrmm control data set or leaves the volume to be processed on another system.

The following steps explain how to import a volume:

1. Use the library manager to transfer stacked volume from the “un-assign” category to the “import” category.
 - a. Create an import list. You can create this list using stacked volumes or logical volumes. To build a list of stacked volumes, use the command shown in Figure 12-54 on page 483.

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) TYPE(STACKED)-
DESTINATION(lib_vts) CLIST
```

Figure 12-54 RMM SEARCHVOLUME subcommand for a single destination

- b. To build a list of logical volumes to import from a single stacked volume, use the command shown in Figure 12-55.

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(*) CONTAINER(DD0813)-
TYPE(LOGICAL) CLIST
```

Figure 12-55 RMM SEARCHVOLUME subcommand for a single container volume

- c. To build a list of logical volumes to import from multiple stacked volumes, use the command shown in Figure 12-56.

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(*) CONTAINER(*)-
TYPE(LOGICAL) REQUIRED(lib_vts) CLIST
```

Figure 12-56 RMM SEARCHVOLUME subcommand with the required operand

2. Start the import process. If the volume is already known to DFSMSrmm, it is accepted for processing if it is defined as an exported logical volume, otherwise it is rejected.

During this process, DFSMSrmm removes the stacked volume association from the logical volume.

Table 12-2 shows the information that DFSMSrmm tracks in import and export processing for stacked volumes.

Table 12-2 *Stacked volume information*

Field	Export	Import
Status	NOT EMPTY	EMPTY
Assign Date	Set at start	Set at end
Stacked Count	Increment	Decrement
Use Count	Increment at start	
Usage	Sum of contained volumes	Sum of contained volumes
Security Level	Highest	
Las Read Date		Set
Last Write Date	Set	

You can use DFSMSrmm logical and stacked volumes together with DFSMSrmm commands for ANY virtual tape solution. See 12.3.2, “Stacked volume support for non-VTS” on page 484.

12.3.2 Stacked volume support for non-VTS

You can use stacked volume support for virtual tape solutions different to VTS. The difference is that you must provide to DFSMSrmm the information that OAM provides for VTS during export and import processing.

In this case, you must define to DFSMSrmm stacked and virtual volumes using the RMM commands shown in Figure 12-57.

```
RMM ADDVOLUME stacked_vol TYPE(STACKED) LOCATION(SHELF)
RMM ADDVOLUME logical_vol TYPE(PHYSICAL/LOGICAL) -
    STATUS(MASTER) CONTAINER(stacked_vol)
```

Figure 12-57 RMM ADDVOLUME subcommands

DFSMSrmm is notified by OAM for import and export processing, and can obtain movement completion information from LM; you must do all this using commands.

If you have already defined virtual and stacked volumes, you can add a virtual volume to a container using CV CONTAINER(name). CONTAINER(' ') removes the logical volume from container.

Figure 12-58 shows how DFSMSrmm manages stacked and container, and logical volumes in the export process.

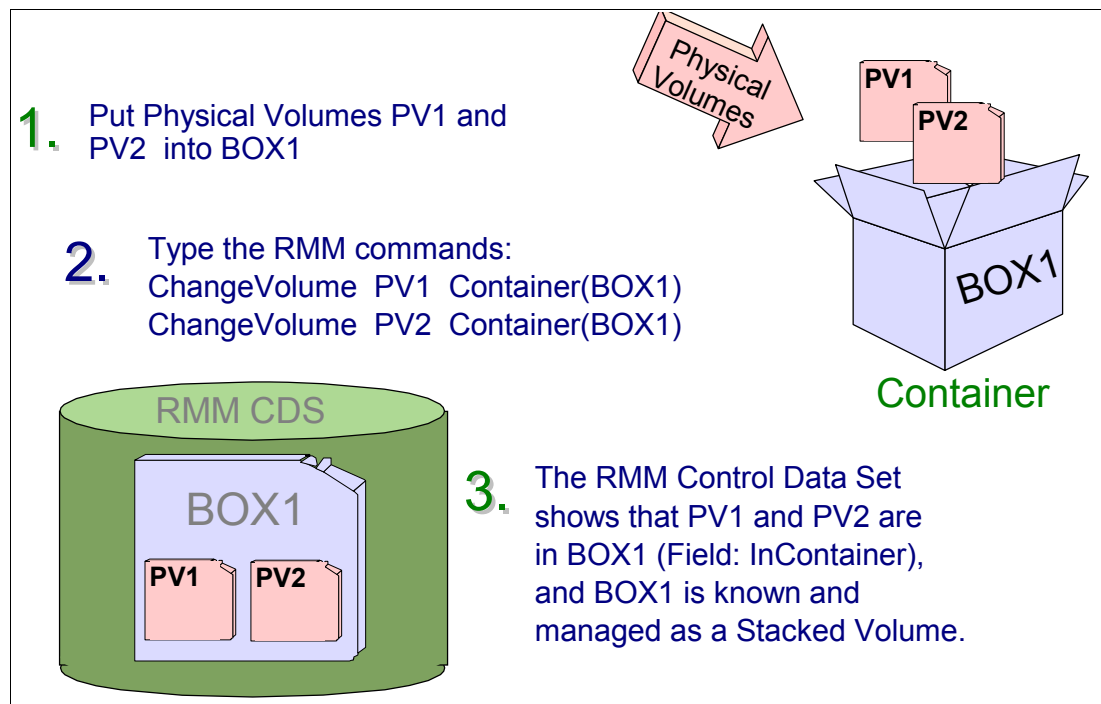


Figure 12-58 Export process for non-VTS



Tape security with z/OS V1.8

z/OS DFSMS V1.8 provides new options for securing tape data sets using System Authorization Facility (SAF). These are designed to allow you to define profiles to protect data sets on tape using the DATASET class without the need to activate the TAPEDSN option or the TAPEVOL class. DFSMS also provides options you can use to specify that all data sets on a tape volume should have common authorization and that users are authorized to overwrite existing files on a tape volume.

In this chapter we describe the options available in z/OS DFSMS V1.8 for tape data set security and how to implement it.

This chapter contains the following sections:

- ▶ Tape data set authorization
- ▶ Implementation
- ▶ Removing TAPEVOL and TAPEDSN processing
- ▶ Error messages
- ▶ Testing various security settings

13.1 Tape data set authorization

z/OS DFSMS V1.8 provides new options for securing tape data sets using the system authorization facility (SAF), to allow you to protect data sets on tape using the RACF DATASET class without the need to activate the TAPEDSN option or the TAPEVOL class. This new tape data set authorization checking allows you to specify that all data sets on a tape volume should have common authorization. You can specify whether users are authorized to overwrite existing files on a tape volume. In addition, you can specify that a user must have access to the first file on a tape volume to add additional files on that tape volume.

13.1.1 Recommendations for tape security

For optimum tape security use the combined capabilities of DFSMSrmm, DFSMSdfp, and RACF. It is recommended you specify the following parameters:

- ▶ In DEVSUPxx parmlib member:
 - TAPEAUTHDSN=YES
 - TAPEAUTHF1=YES
 - TAPEAUTHRC4=FAIL
 - TAPEAUTHRC8=FAIL
- ▶ In EDGRMMxx parmlib member:
 - OPTION TPRACF(N)
- ▶ In RACF:
 - SETROPTS NOTAPEDSN NOCLASSACT(TAPEVOL)

The combination of DFSMSrmm, DFSMSdfp, and RACF ensures:

- ▶ Full 44 character data set name validation.
 - ▶ Validation that the correct volume is mounted.
 - ▶ Control the overwriting of existing tape data sets.
 - ▶ Management of tape data set retention.
 - ▶ Control over the creation and destruction of tape volume labels.
 - ▶ No limitations caused by RACF TAPEVOL profile sizes and TVTOC limitations.
 - ▶ All tape data sets on a volume have a common authorization.
 - ▶ Use of generic DATASET profiles, enabling common authorization with DASD data sets.
 - ▶ Authorization for all tape data sets regardless of the tape label type.
 - ▶ Authorization for the use of bypass label processing (BLP).
 - ▶ Exploitation of RACF 'erase on scratch' support.
 - ▶ Use of DFSMSrmm FACILITY class profiles for data sets unprotected by RACF.
- Your authorization to use a volume outside of DFSMSrmm control through 'ignore' processing also enables authorization to the data sets on that volume.

To aid in the migration to this recommended environment, DFSMSrmm provides the TPRACF(CLEANUP) option, and DEVSUPxx provides TAPEAUTHRC8(WARN) and TAPEAUTHRC4(ALLOW).

The function in DFSMSdfp does not replace all the functional capabilities that the RACF TAPEDSN option, TAPEVOL class, and TVTOC provide. However, together with the functions that DFSMSrmm provides, you do have equivalent capability. The enhanced DFSMSdfp function addresses the authorization requirements for tape data sets and relies on your use of a tape management system such as DFSMSrmm to perform the following operations:

- ▶ Verify full 44 character data set names.
- ▶ Control the overwriting of existing tape files.
- ▶ Handle tape data set retention.

- ▶ Control the creation and destruction of tape labels.

Important: With the z/OS DFSMS V1.8 tape security implementation, as opposed to a TAPEVOL and TAPEDSN implementation, you can do the following:

- ▶ write more than 500 data sets to one tape or one tape volume set
- ▶ a tape volume set can expand more than 42 tape volumes
- ▶ write duplicate data set names to one tape or one tape volume set

13.1.2 Overview of the TAPEVOL and TAPEDSN processing

The enhancement in authority checking that is available with z/OS DFSMS V1.8 takes into account that today most customers are using a tape management system. A tape management system like DFSMSrmm verifies during reuse of data sets and tape volumes the data set names that were put in place at creation time. This ensures that the same data set resources are always being checked in RACF whenever a particular data set is opened.

How protection of tape data sets works pre-z/OS DFSMS V1.8r

Processing in detail differs depending on

- ▶ the setting of SETROPTS TAPEDSN / NOTAPEDSN
- ▶ the RACF TAPEVOL class being active or not
- ▶ the label type of the tape volume
- ▶ the requirement of bypass label processing (BLP)

SETROPTS NOTAPEDSN NOCLASSACT(TAPEVOL)

Tape data sets are not protected if neither RACF option TAPEDSN is being set nor the RACF TAPEVOL class is active. This is true regardless of label type or BLP requirement.

SETROPTS TAPEDSN and SETROPTS CLASSACT(TAPEVOL)

If the RACF TAPEVOL class is active as well as the RACF option TAPEDSN being set, tape data are protected at the data set level.

In case of label type SL or AL, during open of an existing data set a REQUEST=AUTH is performed in the DATASET class with DSNTYPE=T:

1. If there is a matching TAPEVOL class profile the caller's authority according to the type of open is being checked (READ for input and UPDATE for output).
 - If the caller's authority at the volume level is sufficient, no further checks are performed.
 - Else if the TAPEVOL profile contains a TVTOC, the data set name is verified at the full length of 44 bytes.
 - If the verification was successful, RACF looks for a matching data set profile and determines if the caller's authority is sufficient.
 - If verification was not successful the request ends with return code 8 (denied).
 - If the TAPEVOL profile contains no TVTOC see 2
2. Else if there is no matching TAPEVOL profile, RACF looks for a matching DATASET class profile and determines if the caller is authorized sufficiently (READ for input and UPDATE for output).

Note: Without a TVTOC there is no verification of the full 44 byte length of the data set name.

With a label type of SL or AL, during open of a new data set a REQUEST=AUTH as well as a REQUEST=DEFINE is performed in the DATASET class with DSNTYPE=T;

- ▶ If the caller wants to append another data set
 - a. If there is a matching TAPEVOL class profile the caller must have at least UPDATE authority otherwise the request ends with return code 8 (denied)
 - b. RACF is looking for a matching data set profile and determines that the caller is at least authorized for UPDATE access
- ▶ If the caller wants to overwrite an existing data set
 - if the new data set name is different then the existing data set
 - either the security retention period of that particular existing data set (stored in the protecting DATASET profile) and of all of the following existing data sets must have expired
 - or the caller must have at least UPDATE authority to the volume
 - RACF is looking for the matching profile of the new data set name and determines that the caller is at least authorized for UPDATE access.
 - if the new data set name is *not* different then the caller must have UPDATE authority
 - either for the matching DATASET profile
 - or for the matching TAPEVOL profile
- ▶ If the TAPEVOL profile contains a TVTOC then for the new data set an entry in the TVTOC is maintained.

Note: When a TAPEVOL class profile contains a TVTOC some restrictions apply:

- ▶ The maximum number of entries for data sets that a TVTOC can contain is 500.
- ▶ The maximum number of volumes that any data set on the tape with an entry in the TVTOC can span is 42.
- ▶ From a tape volume set you cannot delete a tape volume if a TVTOC entry indicates that there is a protected data set on the volume.

In case of nonlabeled tapes (NL)

- ▶ during open for input the caller must have READ authority
 - either to the volume
 - or, if there is a TVTOC for the volume, to the matching data set profile
- ▶ during open for output the caller must have at least UPDATE authority to both
 - the volume
 - and the data set.

In case of nonstandard label tapes (NSL) data management does not do authorization checking.

In case of a request for BLP during open of a data set the caller must be authorized for the ICHBLP resource in the FACILITY class.

SETROPTS TAPEDSN and SETROPTS NOCLASSACT(TAPEVOL)

If only the RACF option TAPEDSN is being set while the TAPEVOL class is not active, tape data is protected at the data set level.

In case of label type SL, AL or NL during open of a data set for reading, the caller needs at least READ authority for the matching DATASET class profile

In case of label type SL, AL or NL during open of a data set for writing, the caller needs at least

- ▶ UPDATE authority to open an existing data set
- ▶ ALTER authority to create a new data set

for the matching DATASET class profile.

In case of nonstandard label tapes (NSL) data management does not do authorization checking.

In case of a request for BLP during open of a data set the caller must be authorized for the ICHBLP resource in the FACILITY class.

SETROPTS NOTAPEDSN and SETROPTS CLASSACT(TAPEVOL)

If the RACF option TAPEDSN is not being set while the TAPEVOL class is active, tape data are protected at the tape volume level.

In the case of label type SL, AL or NL during open of a data set for reading, the caller needs at least READ authority for the matching TAPEVOL class profile.

In the case of label type SL, AL or NL during open of a data set for writing, the caller needs at least UPDATE authority for the matching TAPEVOL class profile.

In the case of nonstandard label tapes (NSL) data management does not do authorization checking.

In the case of a request for bypass label processing during open of a data set the caller must be authorized for the ICHBLP resource in the FACILITY class.

Summary

In order to achieve the highest level of security you should choose an environment that provides discrete TAPEVOL class profiles with TVTOCs as well as the SETROPTS TAPEDSN setting. This environment provides the following advantages:

- ▶ Full 44 character data set name validation
- ▶ Use of generic DATASET class profiles, enabling common authorization with DASD data sets
- ▶ Volume level control in order to guarantee exclusive volume use for applications

13.1.3 How the z/OS DFSMS V1.8 tape data set authority checking works

The alternate authority checking is enabled by modifying the contents of member DEVSUPxx in the parmlib concatenation. Four new keywords are available:

TAPEAUTHDSN

YES

Enables tape authorization checks in the DATASET class but without DSTYPE=T.

The system uses the data set name specified in the allocation or JCL to check your authorization to read or write the specified file. In addition, the system determines the RACF ease-on-scratch setting from the RACF profile and passes it to your tape management system.

When you request bypass label processing (BLP) and the mounted volume uses standard labels, OPEN issues the authorization check that the user is authorized to use BLP. This processing uses the existing ICHBLP resource in the RACF FACILITY class. When you specify TAPEAUTHDSN = YES only, it replaces the check that RACF makes as part of tape volume authorization checking.

NO

Indicates that OPEN processing to issue RACROUTEs as it did before based on the options set in RACF such as SETROPTS TAPEDSN and SETROPTS CLASSACT(TAPEVOL).

This is the default setting.

TAPEAUTHF1

YES

Enables additional tape authorization checks in the DATASET class for existing files on the same tape volume when any other file on the tape volume is opened.

This function depends on the tape management system returning the 44 character data set name and data set sequence number to OPEN/EOV through the IFGTEP during the Volume Mount exit Volume Security function. If no data set name is returned by the tape management system, processing is as if this keyword had not been specified.

Although intended to enable an additional authorization check for the first data set when any other data set on the tape volume is opened, the implementation allows your tape management system to request one or more additional authorization checks when any data set on a tape volume is opened. Each additional data set name and data set sequence number returned results in an additional RACROUTE. Do not use this function unless you have a tape management system and it can return a data set name and data set sequence number. A data set sequence number is the label number normally specified in the JCL LABEL keyword and stored in the catalog.

When TAPEAUTHDSN=YES is in use, any additional RACROUTE matches that issued for TAPEAUTHDSN except for the data set name and data set sequence number. Otherwise TAPEAUTHF1 uses a RACROUTE that matches that used for SETROPTS TAPEDSN. When neither TAPEAUTHDSN nor SETROPTS TAPEDSN is in use, TAPEAUTHF1 support is not provided.

NO

Disables additional tape authorization checks in the DATASET class for existing files on the same tape volume when any other file on the tape volume is opened.

This is the default setting.

TAPEAUTHRC4

ALLOW

This applies to authorization checks in the DATASET class, and applies only to the results of TAPEAUTHDSN = YES and TAPEAUTHF1 = YES processing.

Allows accessing of data sets that are not protected by a security profile. RC4' refers to the return code value of 4 returned from SAF as a result of the RACROUTE issued by OPEN/CLOSE/EOV. A return code of 4 in general means that the resource is not protected.

FAIL

Denies accessing of data sets that are not protected by a security profile. Use this setting in a PROTECTALL(FAIL) environment. This is the default setting.

TAPEAUTHRC8

WARN

Enables warning mode for all tape authorization checks in the DATASET class as a result of TAPEAUTHDSN = YES and TAPEAUTHF1 = YES processing. Allows accessing of data sets that typically cannot be accessed. RACF issues an ICH408I message to indicate why access is not allowed; however OPEN/EOV allows access.

FAIL

Denies accessing of data sets according to the result of the check in DATASET class. This is the default setting.

In Table 13-1 you can see the different PROTECTALL settings in RACF and the equivalent settings of the new TAPEAUTHRC4 and TAPEAUTHRC8 options for securing tape data sets using System Authorization Facility (SAF).

Table 13-1 Compare RACF and DEFSUP protection

RACF settings	DEVSUP settings	Action
PROTECTALL(NONE)	n/a	none
PROTECTALL(WARN)	TAPEAUTHRC4(ALLOW) TAPEAUTHRC8(WARN)	Enables warning mode
PROTECTALL(FAIL)	TAPEAUTHRC4(FAIL) TAPEAUTHRC8(FAIL)	Denies accessing of data sets: ▶ that are not protected ▶ according to the result of the check

Figure 13-1 on page 492 shows you the RACF checking sequence when you have set the new tape security options TAPEAUTHDSN and TAPEAUTHF1 and activated it. If you have not specified to bypass DFSMSrmm processing by using the JCL parameter EXPDT=98000 or ACCCODE=XCANORES then the RACF checking will be made. The data sets must be RACF protected and the user must have access to the data set profile protecting the data sets.

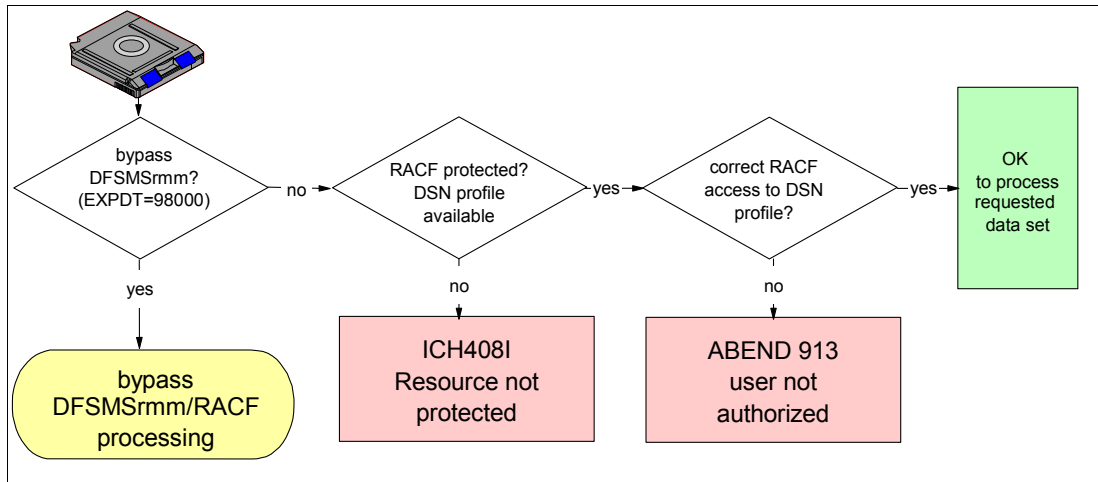


Figure 13-1 New Tape Security flowchart

If you have specified to bypass DFSMSrmm processing by using the JCL parameter EXPDT=98000 or ACCODE=XCANORES DFSMSrmm will make some additional security checks as shown in Figure 13-2. Instead of have different RACF resources to check the bypass processing for volumes defined or not defined in the DFSMSrmm control data set you can specify the resource STGADMIN.EDG.IGNORE.TAPE.* in class FACILITY.

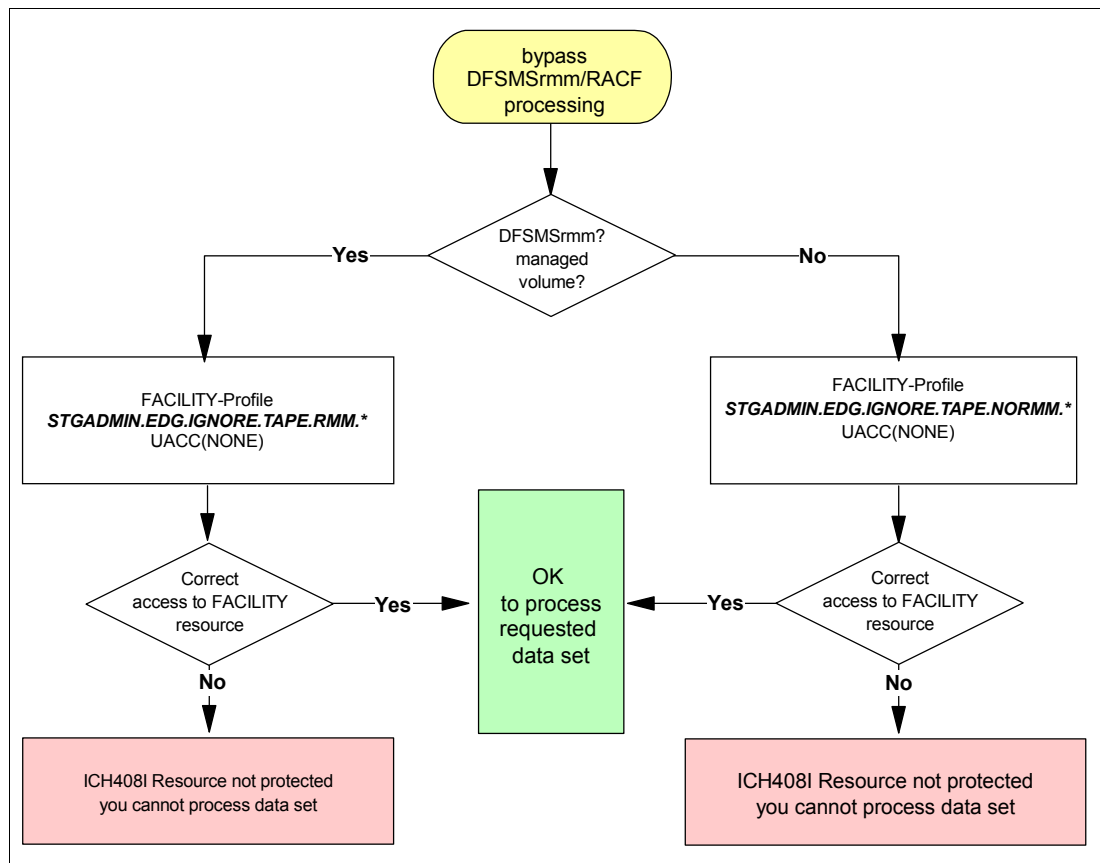


Figure 13-2 Bypass DFSMSrmm processing flowchart

Note: You can specify a profile in RACF class FACILITY for each single volume by using the full volume serial number at the end of the resource name like

STGADMIN.EDG.IGNORE.TAPE.RMM.V12345

or for a range of volumes using the asterisk as part of the volume serial number like

STGADMIN.EDG.IGNORE.TAPE.NORMM.ABC*

for all three different kinds of resources

13.2 Implementation

In this section we will describe how you can implement the new tape data set authorization. This section describes the new implementation of tape data set authorization using the new DEVSUP settings to protect data sets on tape using the DATASET class.

Note: Before you are starting this new tape data set authorization implementation you should search your DFSMSrmm control data set and check that for all data sets residing on tape, at a minimum, the high level qualifier is defined to RACF.

You should protect the use of the set MVS DEVSUP command so that only a small number of people have access to this resource and are able to modify your tape data set authorization settings. Example 13-1 shows you how you can protect the use of the MVS DEVSUP command.

Example 13-1 Protect MVS SET DEVSUP command

```
//MHLRES5C JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*
/* *****
/* * TESTING Tape Data Set Authorization *
/* *****
//DELETE EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
RDEF OPERCMDS MVS.SET.DEVSUP UACC(NONE)
PE MVS.SET.DEVSUP CL(OPERCMDS) ID(MHLRES5 MHLRES2) ACC(NONE)
SETR REFRESH RACLIST(OPERCMDS)
/*
```

Note: Refresh the RACF OPERCMDS because the OPERCMDS are in the RACLIST class.

Use the MVS SET DEVSUP command in Example 13-2 to test your RACF protection of the MVS SET DEVSUP command.

Example 13-2 Unauthorized use of set DEVSUP

```
/T DEVSUP=18
```

This MVS SET COMMAND is not working because the user MHLRES5 is not permitted to use this command. Figure 13-3 shows you the RACF error messages you receive.

```
RESPONSE=SC64
  ICH408I USER(MHLRES5 ) GROUP(SYS1      ) NAME(MARY LOVELACE - RESI)
    MVS.SET.DEVSUP CL(OPERCMD5)
    INSUFFICIENT ACCESS AUTHORITY
    ACCESS INTENT(UPDATE ) ACCESS ALLOWED(NONE  )
```

Figure 13-3 RACF error messages

13.2.1 Check all high level qualifiers on tape

You should check that each high level qualifier on volumes in DFSMSrmm status MASTER or USER is defined to RACF as a user or group. You can create different reports to check the high level qualifiers currently used:

- ▶ only on volumes in status SCRATCH
- ▶ only on volumes have a status other than SCRATCH
- ▶ using all volumes in any volume status.

13.2.2 Update DEVSUPnn parmlib member

Update the DEVSUPxx member in your parmlib to specifying your installation default for device support options. DEVSUPxx is processed during the NIP phase of IPL. After IPL, you can use system command SET DEVSUP=XX to activate the DEVSUP changes.

To enable the new tape data set protection add the following parameters to the DEVSUPxx parmlib member:

TAPEAUTHDSN	to enable tape authorization checks in the DATASET class
TAPEAUTHF1	enables additional tape authorization checks in the DATASET class for existing files on the same tape volume when any other file on the tape volume is opened.
TAPEAUTHRC4	use this keyword to control PROTECTALL processing for tape data sets. This applies to the results of RACROUTE processing when both TAPEAUTHDSN=YES and when TAPEAUTHF1=YES are specified.
TAPEAUTHRC8	use this keyword as an aid to the implementation of TAPEAUTHDSN and TAPEAUTHF1. Provides a managed and controlled implementation of tape authorization checks in the DATASET class, and applies only to the results of TAPEAUTHDSN=YES and TAPEAUTHF1=YES processing.

Note: While TAPEAUTHRC4=FAIL and TAPEAUTHRC8=FAIL are specified to implement the z/OS DFSMS V1.8 tape data set security enhancement, the TAPEAUTHRC4=ALLOW and TAPEAUTHRC8=WARN options are provided to ease the migration to the new tape security implementation. It is recommended you initially specify TAPEAUTHRC4=ALLOW and TAPEAUTHRC8=WARN.

Figure 13-4 on page 495 shows a sample DEVSUPxx PARMLIB member including the default for compaction for new data sets written to 3480, 3490 or 3590 tape subsystems. The

category used for this system for MEDIA1, MEDIA2, MEDIA3 and MEDIA5 tapes in status scratch. The PRIVATE category for tapes in private status and the ERROR category for all volumes that OAM detects any error. This member also includes the final settings for the new tape data set authorization implementation after you have successfully tested this new function.

```
COMPACT = YES,                                /* INSTALLATION DEFAULT FOR IDRC */
MEDIA1 = 0021,
MEDIA2 = 0022,
MEDIA3 = 0023,
MEDIA5 = 0025,
ERROR = 002E,
PRIVATE = 002F,
TAPEAUTHDSN = YES,
TAPEAUTHF1 = YES,
TAPEAUTHRC4 = FAIL,
TAPEAUTHRC8 = FAIL
```

Figure 13-4 Sample DEVSUPxx parmlib member

Where:

- COMPACT** specifies the default for compaction for new data sets written to 3480, 3490 or 3590 tape subsystems
- MEDIAx** used to specify category codes for library partitioning
- ERROR** used to specify category codes for library partitioning
- PRIVATE** used to specify category codes for library partitioning
- TAPEAUTHDSN=YES** Enables tape authorization checks in the DATASET class but without DSTYPE=T.
DSTYPE=T indicates to RACF that the check is for data set on a tape volume and that special RACF tape data set and a tape volume processing is to be performed. Without DSTYPE=T RACF authorization checking considers only profiles in the DATASET class.
The system uses the data set name specified in the allocation or JCL to check your authorization to read or write the specified file. In addition, the system determines the RACF erase-on-scratch setting from the RACF profile and passes it to your tape management system.
Use this option only when you have a tape management system, such as DFSMSrmm, installed and actively checking that the 44 character data set name specified by the user matches the data set name on tape. Without a tape management system, tape data set open processing can only validate the last 17 characters of the data set name against the tape volume labels.
When you request bypass label processing (BLP) and the mounted volume uses standard labels, OPEN issues the authorization check that the user is authorized to use BLP. This processing uses the existing ICHBLP resource in the RACF FACILITY class. When you specify TAPEAUTHDSN=YES only, it replaces the check that RACF makes as part of tape volume authorization checking.

TAPEAUTHF1=YES	Enables additional tape authorization checks in the DATASET class for existing files on the same tape volume when any other file on the tape volume is opened. This function depends on the tape management system returning the 44 character data set name and data set sequence number to OPEN/EOV through the IFGTEP during the Volume Mount exit Volume Security function; if no data set name is returned by the tape management system, processing is as if this keyword had not been specified. Although intended to enable an additional authorization check for the first data set when any other data set on the tape volume is opened, the implementation allows your tape management system to request one or more additional authorization checks when any data set on a tape volume is opened. Each additional data set name and data set sequence number returned results in an additional RACROUTE. Do not use this function unless you have a tape management system and it can return a data set name and data set sequence number. A data set sequence number is the label number normally specified in the JCL LABEL keyword and stored in the catalog. When TAPEAUTHDSN=YES is in use, any additional RACROUTE matches that issued for TAPEAUTHDSN except for the data set name and data set sequence number. Otherwise TAPEAUTHF1 uses a RACROUTE that matches that used for SETROPTS TAPEDSN. When neither TAPEAUTHDSN nor SETROPTS TAPEDSN is in use, TAPEAUTHF1 support is not provided.
TAPEAUTHRC4	Denies accessing of data sets that are not protected by a security profile.
TAPEAUTHRC8	Denies accessing of data sets that typically cannot be accessed.

Use the MVS SET DEVSUP command to implementing the new tape data set security settings. Figure 13-5 shows the successful result of the command.

```

T DEVSUP=18
IEE252I MEMBER DEVSUP18 FOUND IN SYS1.PARMLIB
IEE536I DEVSUP  VALUE 18 NOW IN EFFECT
IEA253I DEVSUP 3480X RECORDING MODE DEFAULT IS COMPACTION.
IEA253I DEVSUP ISO/ANSI TAPE LABEL VERSION DEFAULT IS V3
IEA253I DEVSUP TAPE OUTPUT DEFAULT BLOCK SIZE LIMIT IS 32760
IEA253I DEVSUP COPYSDB DEFAULT IS INPUT
IEA253I DEVSUP TAPEAUTHDSN: YES
IEA253I DEVSUP TAPEAUTHF1: YES
IEA253I DEVSUP TAPEAUTHRC4: FAIL
IEA253I DEVSUP TAPEAUTHRC8: FAIL

```

Figure 13-5 Result of the set DEVSUP command

Restriction: Only the new tape security parameters are updated using the SET DEVSUP command, all existing parameters are not updated and cannot be changed without an IPL.

13.3 Removing TAPEVOL and TAPEDSN processing

In this section we provide instructions for changing from using RACF TAPEVOL and TAPEDSN to the new tape data set authorization checking.

13.3.1 Check and modify your RACF Settings

First check your RACF settings using the RACF SETR LIST command in a TSO session as shown in Example 13-3.

Example 13-3 RACF SETR LIST command used in TSO

```
Command ==> SETR LIST
```

Example 13-4 shows you the same command used in a batch job.

Example 13-4 RACF SETR LIST command used in a batch job.

```
//SCHLUM JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*
/* *****
/* * TESTING Tape Data Set Authorization *
/* *****
//SETRLIST EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
SETR LIST
/*
```

The following figures show the output with the current RACF settings. Figure 13-6 shows you the RACF *ATTRIBUTES*, *STATISTICS* and *ACTIVE CLASSES*.

```
ATTRIBUTES = INITSTATS WHEN(PROGRAM -- BASIC)
STATISTICS = DATASET DASDVOL GDASDVOL GTERMINL O2OMPE TAPEVOL TERMINAL
ACTIVE CLASSES = DATASET USER GROUP ACCTNUM ACICSPCT APPCLU APPCPORT APPCSERV
APPCTP APPL BCICSPCT CBIND CCICSCMD CDT CONSOLE CPSMOBJ
CPSMXMP CSFKEYS CSFSERV DASDVOL DCICSDCT DIGTCERT DIGTRING
DSNR ECICSDCT EJBROLE FACILITY FCICSFCT FIELD FSSEC GCICSTRN
GCPSMOBJ GCSFKEYS GDASDVOL GEJBROLE GMQADMIN GSDSF GXFACILI
HCICSFCT IBMOPC ILMADMIN JCICSJCT JESJOBS JESSPOOL KCICSJCT
KEYSMSTR LOGSTRM MCICSPPT MQADMIN NCICSPPT NETCMDS NETSPAN
NODES NODMBR OPERCMDS O2OMPE PCICSPSB PMBR PRINTSRV PROGRAM
PTKTDATA PTKTVAL QCICSPSB RACFVARS RACGLIST RRSFDATA
RVARSMBR SCICSTST SDSF SERVAUTH SERVER STARTED STORCLAS
SURROGAT SYSMVIEW TAPEVOL TCICSTRN TMEADMIN TSOAUTH TSOPROC
UCICSTST UNIXPRIV VCICSCMD VTAMAPPL WRITER XFACILIT
```

Figure 13-6 RACF ATTRIBUTES, STATISTICS and ACTIVE CLASSES

In Figure 13-7 on page 498 you can see the active RACF *GENERIC PROFILE CLASSES*.

```

GENERIC PROFILE CLASSES = DATASET ACCTNUM ACICSPCT AIMS ALCSAUTH APPCLU
                           APPCPORT APPCSERV APPCSI APPCTP APPL CACHECLS
                           CBIND CCICSCMD CIMS CONSOLE CPSMOBJ CPSMXMP
                           CSFKEYS CSFSERV DASDVOL DBNFORM DCEUIDS DCICSDCT
                           DEVICES DIGTCERT DIGTCRIT DIGTNMAP DIGTRING
                           DIRACC DIRAUTH DIRECTRY DIRSRCH DLFCLASS DSNADM
                           DSNR EJBROLE FACILITY FCICSFCT FIELD FILE FIMS
                           FSOBJ FSSEC GMBR IBMOPC ILMADMIN INFOMAN IPCOBJ
                           JAVA JCICSJCT JESINPUT JESJOBS JESSPOOL KEYSMSTR
                           LDAPBIND LFSCCLASS LOGSTRM MCICSPPT MDSNBP MDSNCL
                           MDSNDB MDSNJR MDSNPK MDSNPN MDSNSC MDSNSG MDSNSM
                           MDSNSP MDSNTB MDSNTS MDSNUF MDSNUT MGMTCLAS
                           MQADMIN MQCHAN MQCMDS MQCONN MQNLIST MQPROC
                           MQQUEUE NDSLINK NETCMDS NETSPAN NODES NODMBR
                           NOTELINK NVASAPDT OIMS OPERCMDS OZOMPE PCICSPSB
                           PERFGRP PIMS PMBR PRINTSRV PROCACT PROCESS
                           PROPCNTL PSFMPL PTKTDATA PTKTVAL RACFVARS RACGLIST
                           RMTOPS RODMMGR ROLE RRSFDATA RVARSMBR SCDMBR
                           SCICSTST SDSF SECLMBR SERVAUTH SERVER SFSCMD
                           SIMS SMESAGE SOMDOBS STARTED STORCLAS SUBSYSNM
                           SURROGAT SYSMVIEW TAPEVOL TCICSTRN TEMPDSN
                           TERMINAL TIMS TMEADMIN TSOAUTH TSOPROC UNIXMAP
                           UNIXPRIV VMBATCH VMBR VMCMD VMMAC VMMDISK VMNODE
                           VMPOSIX VMRDR VMSEGMT VTAMAPPL VXMBR WRITER
                           XFACILIT

```

Figure 13-7 RACF GENERIC PROFILE CLASSES

Figure 13-8 on page 499 shows you the current active RACF *GENERIC COMMAND CLASSES*.

```

GENERIC COMMAND CLASSES = DATASET ACCTNUM ACICSPCT AIMS ALCSAUTH APPCLU
                           APPCPORT APPCSERV APPCSI APPCTP APPL CACHECLS
                           CBIND CCICSCMD CIMS CONSOLE CPSMOBJ CPSMXMP
                           CSFKEYS CSFSERV DASDVOL DBNFORM DCEUIDS DCICSDCT
                           DEVICES DIGTCERT DIGTCRIT DIGTNMAP DIGTRING
                           DIRACC DIRAUTH DIRECTRY DIRSRCH DLFCLASS DSNADM
                           DSNR EJBROLE FACILITY FCICSFCT FIELD FILE FIMS
                           FSOBJ FSSEC GMBR IBMOPC ILMADMIN INFOMAN IPCOBJ
                           JAVA JCICSJCT JESINPUT JESJOBS JESSPOOL KEYSMSTR
                           LDAPBIND LFSCCLASS LOGSTRM MCICSPPT MDSNBP MDSNCL
                           MDSNDB MDSNJR MDSNPK MDSNPN MDSNSC MDSNSG MDSNSM
                           MDSNSP MDSNTB MDSNTS MDSNUF MDSNUT MGMTCLAS
                           MQADMIN MQCHAN MQCMDS MQCONN MQNLIST MQPROC
                           MQQUEUE NDSLINK NETCMDS NETSPAN NODES NODMBR
                           NOTELINK NVASAPDT OIMS OPERCMD5 O2OMPE PCICSPSB
                           PERFGRP PIMS PMBR PRINTSRV PROCACT PROCESS
                           PROPCNTL PSFMPL PTKTDATA PTKTVAL RACFVARS RACGLIST
                           RMTOPS RODMMGR ROLE RRSFDATA RVARSMBR SCDMBR
                           SCICSTST SDSF SECLMBR SERVAUTH SERVER SFSCMD
                           SIMS SMESAGE SOMDOBSJ STARTED STORCLAS SUBSYSNM
                           SURROGAT SYSMVIEW TAPEVOL TCICSTRN TEMPDSN
                           TERMINAL TIMS TMEADMIN TSOAUTH TSOPROC UNIXMAP
                           UNIXPRIV VMBATCH VMBR VMCMD VMMAC VMMDISK VMNODE
                           VMPOSIX VMRDR VMSEGMT VTAMAPPL VXMBR WRITER
                           XFACILIT

```

Figure 13-8 RACF GENERIC COMMAND CLASSES

Figure 13-8 shows you the RACF GENLIST, GLOBAL CHECKING and the settings of the RACLIST classes.

```

GENLIST CLASSES = NONE
GLOBAL CHECKING CLASSES = O2OMPE
SETR RACLIST CLASSES = ACCTNUM APPCPORT APPCSERV APPCTP APPL CBIND CDT
                        CSFKEYS CSFSERV DIGTCERT DIGTRING FACILITY FIELD
                        ILMADMIN JESSPOOL NETCMDS NODES OPERCMD5 O2OMPE
                        PRINTSRV PTKTDATA PTKTVAL RACFVARS RRSFDATA SDSF
                        SERVAUTH SERVER STARTED SURROGAT SYSMVIEW TSOAUTH
                        TSOPROC UNIXPRIV VTAMAPPL WRITER XFACILIT

```

Figure 13-9 RACF RACLIST CLASSES

In Figure 13-10 on page 500 you can see that the tape data set protection is active (TAPEDSN) and that the high level qualifier “*PASSWORD*” will be added to each single level qualifier data set that is used on this system.

```

GLOBAL=YES RACLIST ONLY = NONE
AUTOMATIC DATASET PROTECTION IS NOT IN EFFECT
ENHANCED GENERIC NAMING IS IN EFFECT
REAL DATA SET NAMES OPTION IS INACTIVE
JES-BATCHALLRACF OPTION IS INACTIVE
JES-XBMALLRACF OPTION IS INACTIVE
JES-EARLYVERIFY OPTION IS ACTIVE
PROTECT-ALL OPTION IS NOT IN EFFECT
TAPE DATA SET PROTECTION IS ACTIVE
SECURITY RETENTION PERIOD IN EFFECT IS 9999 DAYS.
ERASE-ON-SCRATCH IS INACTIVE
SINGLE LEVEL NAME PREFIX IS PASSWORD
LIST OF GROUPS ACCESS CHECKING IS ACTIVE.
INACTIVE USERIDS ARE NOT BEING AUTOMATICALLY REVOKED.
NO DATA SET MODELLING BEING DONE.
PASSWORD PROCESSING OPTIONS:
    PASSWORD CHANGE INTERVAL IS 180 DAYS.
    PASSWORD MINIMUM CHANGE INTERVAL IS 0 DAYS.
    MIXED CASE PASSWORD SUPPORT IS NOT IN EFFECT
    NO PASSWORD HISTORY BEING MAINTAINED.
    USERIDS NOT BEING AUTOMATICALLY REVOKED.
    NO PASSWORD EXPIRATION WARNING MESSAGES WILL BE ISSUED.
    NO INSTALLATION PASSWORD SYNTAX RULES ARE PRESENT.
DEFAULT RVARV PASSWORD IS IN EFFECT FOR THE SWITCH FUNCTION.
DEFAULT RVARV PASSWORD IS IN EFFECT FOR THE STATUS FUNCTION.
SECLABEL CONTROL IS NOT IN EFFECT
GENERIC OWNER ONLY IS IN EFFECT
COMPATIBILITY MODE IS NOT IN EFFECT
MULTI-LEVEL QUIET IS NOT IN EFFECT
MULTI-LEVEL STABLE IS NOT IN EFFECT
NO WRITE-DOWN IS NOT IN EFFECT
MULTI-LEVEL ACTIVE IS NOT IN EFFECT
CATALOGUED DATA SETS ONLY, IS NOT IN EFFECT
USER-ID FOR JES NJEUSERID IS : ????????
USER-ID FOR JES UNDEFINEDUSER IS : ++++++++
PARTNER LU-VERIFICATION SESSIONKEY INTERVAL DEFAULT IS "NEVER EXPIRES".
ADDCREATOR IS NOT IN EFFECT
KERBLVL = 1
MULTI-LEVEL FILE SYSTEM IS NOT IN EFFECT
MULTI-LEVEL INTERPROCESS COMMUNICATIONS IS NOT IN EFFECT
MULTI-LEVEL NAME HIDING IS IN EFFECT
SECURITY LABEL BY SYSTEM IS NOT IN EFFECT
PRIMARY LANGUAGE DEFAULT : ENU
SECONDARY LANGUAGE DEFAULT : ENU

```

Figure 13-10 other RACF related information

In our example above you can see that the RACF TAPEVOL class is active and TAPE DATA SET PROTECTION is set so we will deactivate both functions using the RACF commands as shown in Example 13-5.

Example 13-5 Deactivate the RACF TAPEVOL class and TAPE DATA SET PROTECTION

```

//MHLRES5C JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
//*

```

```

/* *****
/* * TESTING Tape Data Set Authorization *
/* *****
//DELETE EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
SETROPTS NOCLASSACT(TAPEVOL)
SETROPTS NOTAPEDSN
SETR REFRESH GENERIC(TAPEVOL)
/*

```

13.3.2 Check and modify your DFSMSHsm settings

Finally you should check your DFSMSHsm settings and switch off the TAPESECURITY if this option is set. Figure 13-11 shows you a sample ARCCMDxx parmlib member where and you can see that the SETSYS TAPESECURITY(RACF) option is used.

The use of the TAPESECURITY(RACF) means that DFSMSHsm protects each backup, migration, and dump tape with RACF. DFSMSHsm also protects alternate backup and migration tapes generated as a result of TAPECOPY processing. The RACF subparameter does not support backup or migration of password-protected data sets.

Note: The **RACF** option of the **SETSYS TAPESECURITY** command directs DFSMSHsm to automatically add RACF protection to scratch tapes.

```

...
/*****
/*          DFSMSHSM RACF SPECIFICATIONS          */
/*****
/*
SETSYS          /* DO NOT PUT RACF-INDICATION      */ -
  NORACFIND      /* ON BACKUP AND MIGRATION              */
                  /* COPIES OF DATA SETS.                */
SETSYS          /* USE RACF TO PROVIDE TAPE SECURITY */ -
  TAPESECURITY(RACF EXPIRATIONINCLUDE) /* USE 99365 EXPDT */
SETSYS          /* DO NOT ALLOW ERASE-ON-SCRATCH      */ -
  NOERASEONSCRATCH /* ON ANY DFSMSHSM BACKUP              */
                  /* VERSIONS AND MIGRATION COPIES      */
SETSYS          /* BACKUP DISCRETE RACF PROFILES      */ -
  PROFILEBACKUP
/*
...

```

Figure 13-11 Sample DFSMSHsm PARMLIB member

If the TAPESECURITY is set to RACF change your ARCCMDxx parmlib member as shown in Figure 13-12 to remove the RACF setting. Restart your DFSMSShsm so that the new setting is used.

```

...
/*****
/*              DFSMSHSM RACF SPECIFICATIONS              */
/*****
/*
/*
/*
/*
/* DO NOT PUT RACF-INDICATION          */ -
/* ON BACKUP AND MIGRATION              */ -
/* COPIES OF DATA SETS.                */ -
/*
/*
/* USE 99365 TO WIRTE  AN EXPDT         */ -
/* TAPESECURITY(EXPIRATIONINCLUDE)      */ -
/* TO EACH TAPE                        */ -
/*
/* DO NOT ALLOW ERASE-ON-SCRATCH        */ -
/* ON ANY DFSMSHSM BACKUP                */ -
/* VERSIONS AND MIGRATION COPIES        */ -
/*
/* BACKUP DISCRETE RACF PROFILES        */ -
/* PROFILEBACKUP                        */ -
/*
/*
...

```

Figure 13-12 Sample DFSMSShsm PARMLIB member with RACF setting

Note: You must re-start you DFSMSShsm, because there is no SETSYS command available to switch of the RACF setting.

13.3.3 Cleanup your TAPEVOL profiles using DFSMSrmm settings

Modify your DFSMSrmm settings in EDGRMMxx if you have specified that DFSMSrmm maintains the security profiles that protect tape volumes specified in TPRACF OPTION operand.

TPRACF

Check the type of RACF tape support you have selected. If you have specified TPRACF(AUTOMATIC) or TPRACF(PREDEFINED) you should now change this setting to one of the following:

TPRACF(CLEANUP) DFSMSrmm ensures that TAPEVOL profiles and discrete tape DATASET profiles are deleted during recycling of scratch volumes and existing TAPEVOL profiles are deleted when volumes are deleted from the DFSMSrmm CDS. When you use this option, DFSMSrmm never creates any RACF profiles for you. This processing is only provided for VLPOOLS with RACF(Y). TPRACF(CLEANUP) is intended to be used when you are changing how tape data sets are protected. For example, you no longer wish to use TAPEVOL profiles and are enabling the use of DATASET profiles, TPRACF(CLEANUP) can be used for this occasion. When you

specify TPRACF(CLEANUP), DFSMSrmm deletes RACF tape profiles for any volumes in your installation based on the following VLPOOL values:

- VLPOOL RACF(N), DFSMSrmm does no processing of tape profiles for volumes in the pool at any time.
- VLPOOL RACF(Y), RACF tape profiles are deleted when RMM CHANGEVOLUME, or DELETEVOLUME subcommands are issued. DFSMSrmm deletes TAPEVOL and discrete tape data set profiles during recycling of scratch tapes if the profiles exist.

TPRACF(NONE) DFSMSrmm will not manipulate an TAPEVOL profile in any circumstances.

Example 13-6 shows you the update to your EDGRMMxx parmlib member OPTION command if you would like DFSMSrmm to clean up your security TAPEVOL profiles.

Example 13-6 Update the OPTION command

OPTION	OPMODE(R)	/* Record-Only Mode	*/ -
	ACCOUNTING(J)	/* Accounting from JOB	*/ -
	BACKUPPROC(EDGBKUP)	/* Name of BACKUP-proc	*/ -
	BLP(RMM)	/* DFSMSrmm controls BLP	*/ -
	CATRETPD(0012)	/* catalog retention	*/ -
	CATSYSID(*)	/* all catalogs shared	*/ -
	CDSID(PROD)	/* control data set id	*/ -
	COMMANDAUTH(OWNER)	/* type of authorization	*/ -
	DATEFORM(J)	/* Date format	*/ -
	DISPDDNAME(LOANDD)	/* DISP ctrl DD card	*/ -
	DISPMSGID(EDG4054I)	/* DISP message number	*/ -
	DSNAME(RMM.PROD.CDS)	/* CDS data set name	*/ -
	IPLDATE(N)	/* IPL date checking	*/ -
	JRNLNAM(RMM.PROD.JRNL)	/* JRNL data set name	*/ -
	JOURNALFULL(75)	/* Percentage JRNL full	*/ -
	LINECOUNT(054)	/* Lines per page	*/ -
	MASTEROVERWRITE(LAST)	/* Overwriting of a vol	*/ -
	MAXHOLD(100)	/* Number of I/O oper.	*/ -
	MAXRETPD(NOLIMIT)	/* Maximum retention	*/ -
	MEDIANAME(3480)	/* spec. how to move vols	*/ -
	MOVEBY(VOLUME)	/* spec. how to move vols	*/ -
	MSG(M)	/* case for message txt	*/ -
	NOTIFY(Y)	/* Notify volume owners	*/ -
	PDA(OFF)	/* PDA is disabled	*/ -
	PDABLKCT(255)	/* number of blocks	*/ -
	PDABLKSZ(31)	/* blocksize in K	*/ -
	PDALOG(OFF)	/* PDA output disabled	*/ -
	PREACS(NO)	/* Disable EDGUX100 ACS pr.	*/ -
	RETAINBY(VOLUME)	/* spec. how to retain vols	*/ -
	RETPD(0005)	/* Default retention	*/ -
	REUSEBIN(STARTMOVE)	/* reuse BIN as soon as pos.	*/ -
	SCRATCHPROC(EDGXPROC)	/* ATL/MTL procedure	*/ -
	SMFAUD(248)	/* SMF audit records	*/ -
	SMFSEC(249)	/* SMF security records	*/ -
	SMSACS(YES)	/* enable MV ACS processing	*/ -
	SMSTAPE(UPDATE(EXITS,SCRATCH,COMMAND),PURGE(YES))	ATL	*/ -
	SYSID(EGZB)	/* Name of the system	*/ -

TPRACF(CLEANUP)	/* RACF tape support	*/ -
TVEXTPURGE(EXPIRE)	/* set an expiration date	*/ -
UNCATALOG(N)	/* Catalog support	*/ -
VRSCCHANGE(INFO)	/* No additional action	*/ -
VRSEL(NEW)	/* New VRS processing	*/ -
VRSJOBNAME(2)	/* DATASETNAME/JOBNAME	*/ -
VRSMIN(0000000100,WARN)	/* Warn if < 100 VRSs	*/

Example 13-7 shows you the correct setting of the of the VLPOOL command in the EDGRMMnn parmlib member.

Example 13-7 Update the VLPOOL command

VLPOOL	PREFIX(TST*)	-
	TYPE(S)	-
	DESCRIPTION('LOGISCHE VTS VOLUMES')	-
	MEDIANAME(3490)	-
	RACF(Y)	-
	EXPDTCHECK(N)	/* NOT TO CHECK OR VALIDATE */
VLPOOL	PREFIX(E*)	-
	TYPE(S)	-
	DESCRIPTION('Enhanced 3590 cartridges')	-
	MEDIANAME(3490)	-
	RACF(Y)	-
	EXPDTCHECK(N)	/* NOT TO CHECK OR VALIDATE */
VLPOOL	PREFIX(M*)	-
	TYPE(S)	-
	DESCRIPTION('MEDIA5 cartridges')	-
	MEDIANAME(3490)	-
	RACF(Y)	-
	EXPDTCHECK(N)	/* NOT TO CHECK OR VALIDATE */

Where:

RACF(Y) specifies that you want DFSMSrmm to delete RACF tape profiles for the volumes in the pool.

13.3.4 Cleanup your TAPEVOL profiles using commands

If you would like to cleanup your security server directly you can use the TSO RMM SEACHVOLUME subcommand with the CLIST operand to create RACF delete TAPEVOL commands. After you have checked the commands you can execute them and delete all your existing TAPEVOL profiles directly. Example 13-8 shows you sample JCL to create the RDELETE statements.

Example 13-8 Create RACF RDELETE commands

```
//LCLRDEL JOB ,RMM,NOTIFY=&SYSUID,
//          MSGCLASS=H,CLASS=0,MSGLEVEL=(1,1),REGION=0M
//CLEANUP DD PGM=IDCAMS
//SYSPRINT DD DUMMY
//SYSIN DD *
DELETE RMM.DELETE.RACF.TAPEVOL.TST NONVSAM PURGE
/*
//LCLSVSM EXEC PGM=IKJEFT01,DYNAMNBR=99
//SYSTSPRT DD SYSOUT=*
```

```
//RMMCLIST DD  DISP=(,CATLG),DSN=RMM.DELETE.RACF.TAPEVOL.TST,
//          SPACE=(TRK,(45,45),RLSE),LRECL=80,RECFM=FB,
//          UNIT=SYSDA
//SYSTSIN DD  *
RMM SV VOLUME(TST*) LIMIT(*) OWNER(*) -
      CLIST('RDELETE TAPEVOL ',')
/*
//
```

Where:

RMMCLIST DD the RMM TSO SEARCH subcommands with the CLIST operand is writing to this data set to store the commands.

CLIST(prefix_string,suffix_string)

specifies a CLIST to create a data set of executable commands or to prepare an import list. You can edit the data set to remove any volumes you do not want in the list. Then you can run the CLIST at your convenience.

DFSMSrmm returns the volume serial number for each record if you do not specify (prefix_string and suffix_string). When the volume serial number contains special characters the value is returned within quotation marks.

You can add RMM TSO subcommands and operands to the records in the CLIST data set by specifying (prefix_string and suffix_string). These text strings cannot exceed 255 characters. Separate the prefix_string and suffix_string using a blank or a comma between the text strings. Insert blanks in the prefix and suffix values to prevent DFSMSrmm from concatenating the strings with the data that DFSMSrmm returns. To enter a null prefix_string, add a pair of separator characters such as " to the text string (for example, CLIST("',' suffix_string')).

Figure 13-13 shows you how the RACF commands created using the job above.

```

RDELETE TAPEVOL TST000
RDELETE TAPEVOL TST001
RDELETE TAPEVOL TST002
RDELETE TAPEVOL TST003
RDELETE TAPEVOL TST004
RDELETE TAPEVOL TST005
RDELETE TAPEVOL TST006
RDELETE TAPEVOL TST007
RDELETE TAPEVOL TST008
RDELETE TAPEVOL TST009
RDELETE TAPEVOL TST010
RDELETE TAPEVOL TST011
RDELETE TAPEVOL TST012
RDELETE TAPEVOL TST013
RDELETE TAPEVOL TST014
RDELETE TAPEVOL TST015
RDELETE TAPEVOL TST016
RDELETE TAPEVOL TST017
RDELETE TAPEVOL TST018
RDELETE TAPEVOL TST019
RDELETE TAPEVOL TST020
RDELETE TAPEVOL TST021
RDELETE TAPEVOL TST022

```

Figure 13-13 Sample RACF RDELETE commands

After you have checked the commands you can execute them using the JCL shown in Example 13-9.

Example 13-9 Execute RACF RDELETE commands

```

//LCLRDEL JOB ,RMM,NOTIFY=&SYSUID,
//          MSGCLASS=H,CLASS=0,MSGLEVEL=(1,1),REGION=0M
//EXECUTE EXEC PGM=IKJEFT01,DYNAMNBR=99
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
EX 'RMM.DELETE.RACF.TAPEVOL.TST'
/*
//

```

13.4 Error messages

Until the TAPEVOL profiles are deleted you may see some security violations and you should check your system log from time to time for messages.

Figure 13-14 on page 507 shows message IEC150I which is generated if you have not the correct access to create a given data set.

```

JOB22272 IEC150I 913-60,IFG0196T,GENER99,STEP01,SYSUT2,0B91,TST026,DTAUS
JOB22272 IEA995I SYMPTOM DUMP OUTPUT 928
SYSTEM COMPLETION CODE=913 REASON CODE=00000060
TIME=20.17.17 SEQ=02226 CPU=0000 ASID=002A
PSW AT TIME OF ERROR 075C1000 80C49CBE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C49CB8 - 41003B7A 0A0D41F0 38BE56F0
GR 0: 00000000_00C49F84 1: 00000000_A4913000
2: 00000000_000097F4 3: 00000000_00C4940A
4: 00000000_007DD1F8 5: 00000000_007DD58C
6: 00000000_007DD534 7: 00000000_007DD58C
8: 00000000_007DD554 9: 00000000_007C71A8
A: 00000000_007DBC00 B: 00000000_00C4CB04
C: 00000000_80C4CBE4 D: 00000000_007DD4B8
E: 00000000_80C49542 F: 00000010_00000060
END OF SYMPTOM DUMP
JOB22272 IEF450I GENER99 STEP01 - ABEND=S913 U0000 REASON=00000060 929

```

Figure 13-14 IEC150I Message

IEC150I **913-rc,mod,jjj,sss, ddname[-#],dev,ser,dsname(member)**

Where:

<i>rc</i>	Associates this message with system completion code 913 and with the return code.
<i>jjj</i>	The job name.
<i>sss</i>	The step name.
<i>ddname[-#]</i>	DDname (followed by a concatenation number if it is part of a concatenation and not the first DD statement in the concatenation).
<i>dev</i>	The device number.
<i>ser</i>	The volume serial number.
<i>mod</i>	The name of the module in which the error occurred.
<i>dsname(member)</i>	The data set name. Member name if specified. The explanation for the hex return code is as follows: For RACF errors, see message IDC3009I for the return code.

Severity:

Explanation:

Information.
The error occurred during 1) the processing of an OPEN macro instruction or during end-of-volume for a password-protected data set after the operator attempted to enter a password in response to message IEC301A or 2) the processing of an OPEN macro instruction involving a checkpoint data set. A VSAM data set is being opened with a DCB instead of an ACB.

The explanation for the hex return code is as follows: For RACF errors, see message IDC3009I for the return code.

Return Code

Explanation

60

One of the following occurred:

- The user is not authorized to define a data set with the specified name. The specified data set name and file sequence indicator do not match the corresponding names in the Tape Volume Table of Contents (TVTOC).
- The user is not authorized to access this data set.

Attention: In this case the volumes gets the volume error status of “SECURITY CONFLICT” in the VOLCAT and is protected for future use until you have reset this situation, if the volume is SMS managed:

LINE	VOLUME USE	VOLUME	CHECKPT	LIBRARY	STORAGE
OPERATOR	SERIAL ATTR	ERROR STATUS	VOLUME	NAME	GRP NAME
---(1)---	-(2)-- --(3)--	----- (4)-----	--(5)--	--(6)---	--(7)---
	TST020 SCRATCH	SECURITY CONFLICT	NO	LIB1	*SCRTCH*
-----	-----	-----	-----	-----	-----
		BOTTOM OF DATA			

Figure 13-15 shows you an ICH408I message you get now for tapes. You currently get this message for data sets residing on DASD volumes if you do not have the correct access to the specified security resource in RACF facility class.

```

JOB22265 IEF233A M 0B91,PRIVAT,SL,GENER99,STEP01,MHLRES1.TESTSTAC.TESTYF01
JOB22265 ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
781
      MHLRES1.TESTSTAC.TESTYF01 CL(DATASET ) VOL(TST020)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES1.** (G)
      ACCESS INTENT(UPDATE ) ACCESS ALLOWED(NONE )
JOB22265 IEC518I SOFTWARE ERRSTAT: RACFPROT 0B91,TST020,SL,GENER99,STEP01
JOB22265 IEC502E RK 0B91,TST020,SL,GENER99,STEP01
JOB22265 IEC150I
913-38,IFG0194F,GENER99,STEP01,SYSUT2,0B91,,MHLRES1.TESTSTAC.T
JOB22265 IEA995I SYMPTOM DUMP OUTPUT 785
      SYSTEM COMPLETION CODE=913 REASON CODE=00000038
      TIME=19.54.16 SEQ=02221 CPU=0000 ASID=002A
      PSW AT TIME OF ERROR 075C1000 80C49CBE ILC 2 INTC 0D
      NO ACTIVE MODULE FOUND
      NAME=UNKNOWN
      DATA AT PSW 00C49CB8 - 41003B7A 0A0D41F0 38BE56F0
      GR 0: 00000000_00C49F84 1: 00000000_A4913000
      2: 00000000_000097F4 3: 00000000_00C4940A
      4: 00000000_007C51F8 5: 00000000_007C558C
      6: 00000000_007C5534 7: 00000000_007C558C
      8: 00000000_007C5554 9: 00000000_007C3040
      A: 00000000_007DBCE0 B: 00000000_00C4CB04
      C: 00000000_80C4CBE4 D: 00000000_007C54B8
      E: 00000000_80C49542 F: 00000010_00000038
      END OF SYMPTOM DUMP
JOB22265 IEF450I GENER99 STEP01 - ABEND=S913 U0000 REASON=00000038 786
      TIME=19.54.16
JOB22265 - --TIMINGS (MINS.)--
JOB22265 -JOBNAME STEPNAME PROCSTEP RC EXCP CPU SRB CLOCK SERV
JOB22265 -GENER99 STEP01 *S913 51 .00 .00 .56 362
. . . . .

```

Figure 13-15 Normal ICH408I security violation

Note: If the volume is a SMS managed volume the status of the volume is not changed in the VOLCAT and the volume can be continually accessed.

13.5 Testing various security settings

The following jobs give you an overview of how the different tape security settings work and the error messages you get if your job abends. Table 13-2 shows you the different settings we have tested.

Table 13-2 Different RACF, DEVSUP and DFSMSrmm settings

Test case / success	RACF settings					TAPEAUTH settings in the DEVSUPnn				DFSMSrmm ²	
	TAPEVOL	TAPEDSN	THM00n ⁴	MHLRES1.**	MHLRES7.**	DSN	F1	RC4	RC8	TPRACF	RACF
All test cases between 1 and 19 are without the use of the new TAPEAUTH settings in the DEVSUPnn member											
Create two new data sets on a tape volume, but at this time there are no DEVSUP settings											
1 ·	ACTIVE	ACTIVE	N/A ¹	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
Read the previously created data sets with different RACF settings and different access to the DATASET and TAPEVOL profile											
2 ·	ACTIVE	ACTIVE	ALTER	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
3 ·	ACTIVE	ACTIVE	ALTER	NONE	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
4 ·	ACTIVE	ACTIVE	NONE	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
5 ✂	ACTIVE	ACTIVE	NONE	NONE	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
6 ·	N/A ¹	ACTIVE	N/A ¹	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
7 ✂	N/A ¹	ACTIVE	N/A ¹	NONE	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
8 ·	N/A ¹	N/A ¹	N/A ¹	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
9 ·	N/A ¹	N/A ¹	N/A ¹	ALTER	NONE	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
Create an additional file to the previously created two tape data set with different RACF settings											
10 ·	ACTIVE	ACTIVE	ALTER	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
11 ✂	ACTIVE	ACTIVE	ALTER	ALTER	NONE	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
12 ·	ACTIVE	N/A ¹	ALTER	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
13 ·	ACTIVE	N/A ¹	ALTER	ALTER	NONE	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
14 ·	N/A ¹	ACTIVE	ALTER	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
15 ✂	N/A ¹	ACTIVE	ALTER	ALTER	NONE	N/A ¹	N/A ¹	N/A ¹	N/A ¹	A ³	Y
Read the previously created data sets, but at this time the DFSMSrmm RACF support is switched of											
16 ·	ACTIVE	ACTIVE	ALTER	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	N	N
17 ·	ACTIVE	ACTIVE	ALTER	ALTER	NONE	N/A ¹	N/A ¹	N/A ¹	N/A ¹	N	N
18 ·	N/A ¹	ACTIVE	N/A ¹	ALTER	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	N	N
19 ✂	N/A ¹	ACTIVE	N/A ¹	NONE	ALTER	N/A ¹	N/A ¹	N/A ¹	N/A ¹	N	N
All test cases below are now tested with the new TAPEAUTH settings in the DEVSUPnn member and the RACF CLASS TAPEVOL is not active and option TAPEDSN is inactive.											
Create two new data sets on a tape volume, but at this time RACF TAPEVOL and TAPEDSN are inactive and the DFSMSrmm RACF support is switched of											
20 ·	N/A ¹	N/A ¹	N/A ¹	ALTER	ALTER	YES	YES	FAIL	Fail	N	N
Read the previously created data sets with different RACF access to MHLRES1.** and MHLRES7.**											
21 ·	N/A ¹	N/A ¹	N/A ¹	ALTER	ALTER	YES	YES	FAIL	FAIL	N	N
22 ✂	N/A ¹	N/A ¹	N/A ¹	NONE	ALTER	YES	YES	FAIL	FAIL	N	N

Test case / success	RACF settings					TAPEAUTH settings in the DEVSUPnn				DFSMSrmm ²	
	TAPEVOL	TAPEDSN	THM00n ⁴	MHLRES1.**	MHLRES7.**	DSN	F1	RC4	RC8	TPRACF	RACF
23 ✓	N/A ¹	N/A ¹	N/A ¹	ALTER	NONE	YES	YES	FAIL	FAIL	N	N
24 ✓	N/A ¹	N/A ¹	N/A ¹	NONE	NONE	YES	YES	FAIL	FAIL	N	N
Create an additional file MHLRES7.RACF.TEST2.FILEn with different RACF access to MHLRES1.** and MHLRES5.**											
25 ·	N/A ¹	N/A ¹	N/A ¹	ALTER	ALTER	YES	YES	FAIL	FAIL	N	N
26 ✓	N/A ¹	N/A ¹	N/A ¹	NONE	ALTER	YES	YES	FAIL	FAIL	N	N
27 ✓	N/A ¹	N/A ¹	N/A ¹	ALTER	NONE	YES	YES	FAIL	FAIL	N	N
28 ✓	N/A ¹	N/A ¹	N/A ¹	NONE	NONE	YES	YES	FAIL	FAIL	N	N
Read the previously created data sets with different RACF access to MHLRES1.** and MHLRES7.**											
29 ·	N/A ¹	N/A ¹	N/A ¹	ALTER	ALTER	YES	NO	FAIL	FAIL	N	N
30 ✓	N/A ¹	N/A ¹	N/A ¹	NONE	ALTER	YES	NO	FAIL	FAIL	N	N
31 ✓	N/A ¹	N/A ¹	N/A ¹	ALTER	NONE	YES	NO	FAIL	FAIL	N	N
32 ✓	N/A ¹	N/A ¹	N/A ¹	NONE	NONE	YES	NO	FAIL	FAIL	N	N
Create an additional file MHLRES1.RACF.TEST2.FILEn with different RACF access to MHLRES1.** and MHLRES5.*											
33 ·	N/A ¹	N/A ¹	N/A ¹	ALTER	ALTER	YES	NO	FAIL	FAIL	N	N
34 ·	N/A ¹	N/A ¹	N/A ¹	NONE	ALTER	YES	NO	FAIL	FAIL	N	N
35 ✓	N/A ¹	N/A ¹	N/A ¹	ALTER	NONE	YES	NO	FAIL	FAIL	N	N
36 ✓	N/A ¹	N/A ¹	N/A ¹	NONE	NONE	YES	NO	FAIL	FAIL	N	N
¹ N/A means that this function is not activated at this time or no TAPEVOL profile exists ² The DFSMSrmm settings are stored in the current active DFSMSrmm EDGRMMnn PARMLIB member. In this table we will show only the tape security related two DFSMSrmm operands we have modified for our test cases TPRACF is an operand of the OPTION command and RACF is an option of the VLPOOL command ³ "A" is the abbreviation of the TPRACF(AUTOMATIC) processing ⁴ THM001 is the TAPEVOL profile protecting the volume we have used to create the two data sets for test case 1 to 19											

13.5.1 Test case 1

Function: Create two new tape sets on a volume in status scratch. Program IEBGENER is used to copy a member of a library.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE1

Result: The job eneded without any errors and DFSMSrmm has created a new RACF TAPEVOL profile for volume THM001 and the user MHLRES5 have ALTER access to it.

Example 13-10 shows you the JCL and the settings we have used to create the two data sets.

Example 13-10 Sample JCL used for test case 1

```
//RACFTS01 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
//JOBPARM SYSAFF=SC70
//* *****
//* * /F DFRMM,M=R2 *
//* *****
//*
//* RMM OPTIONS:
//* TPRACF(A)
```



```

//*          RACF(Y)
//*
//*
//*          DEVSUP
//*          TAPEAUTHDSN=N/A
//*          TAPEAUTHF1=N/A
//*          TAPEAUTHRC4=N/A
//*          TAPEAUTHRC8=N/A
//*
//*          RACF
//*          MHLRES1.**          ACC(ALTER)
//*          MHLRES5.**          ACC(ALTER)
//*          MHLRES7.**          ACC(ALTER)
//*
//*          FUNCTION
//*          CREATE TWO NEW DATA SETS
//*          MHLRES1.RACF.TEST1.FILE1
//*          MHLRES7.RACF.TEST1.FILE2
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR CLASSACT(TAPEVOL)
  SETR TAPEDSN
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR REFRESH GENERIC(DATASET)
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN  DD  *
  DELETE MHLRES1.RACF.TEST1.FILE1 NONVSAM NOSCRATCH
  DELETE MHLRES7.RACF.TEST1.FILE2 NONVSAM NOSCRATCH
  SET MAXCC=0
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD  DISP=(,CATLG),DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,99),
//          UNIT=ATL3
//SYSIN  DD  DUMMY
//*-----*
//STEP1 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD  LABEL=(02,SL),DSN=MHLRES7.RACF.TEST1.FILE2,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=(,RETAIN,,REF=*.STEP01.SYSUT2)
//SYSIN  DD  DUMMY

```

Example 13-11 on page 512 shows you the most important results of the RACF commands.

Example 13-11 RACF Command results of test case 1

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES5.**') ALL
INFORMATION FOR DATASET MHLRES5.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES5	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-------	-------	------------------	---------	-------

```

-----
00      MHLRES7      NONE      NO      NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER      SYS1      NON-VSAM

NO INSTALLATION DATA

      SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.2 ITest case 2

Function: Read the two previously created tape sets. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE1

Result: The job ended without any errors, because the user have ALTER access to the TAPEVOL profile and to the two profiles in class DATASET.

Example 13-12 shows you the JCL and the settings we have used to read the two data sets.

Example 13-12 Sample JCL used for test 2

```

//RACFTS02 JOB (999,POK),MSGLEVEL=1,
//          NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(A)
//*      RACF(Y)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=N/A
//*      TAPEAUTHF1=N/A
//*      TAPEAUTHRC4=N/A
//*      TAPEAUTHRC8=N/A
//*
//*      RACF
//*      MHLRES1.**      ACC(ALTER)
//*      MHLRES5.**      ACC(ALTER)
//*      MHLRES7.**      ACC(ALTER)
//*      THM001          ACC(ALTER)

```

```

/*          TAPEVOL          ACTIVE
/*          TAPEDSN          ACTIVE
/*
/*      FUNCTION
/*          READ THE PREVIOUSLY CREATED DATA SETS
/*              MHLRES1.RACF.TEST1.FILE1
/*              MHLRES7.RACF.TEST1.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
    SETR CLASSACT(TAPEVOL)
    SETR TAPEDSN
    PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACC(ALTER)
    SETR REFRESH RACLIST(TAPEVOL)
    RL TAPEVOL THM001 ALL
    PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
    PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
    LD DATASET('MHLRES1.**') ALL
    LD DATASET('MHLRES7.**') ALL
    SETR REFRESH GENERIC(DATASET)
    SETR LIST
/*
//STEP01  EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//STEP02  EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//SUBMIT  EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF03)
//SYSUT2  DD  SYSOUT=(A,INTRDR)
//SYSIN   DD  DUMMY

```

Example 13-13 shows you the most important results of the RACF commands.

Example 13-13 RACF Commands result of test case 2

```

RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5      NONE              ALTER        NO

```

USER	ACCESS	ACCESS COUNT
----	-----	-----
MHLRES5	ALTER	000000
STC	ALTER	000001

LD DATASET('MHLRES1.**') ALL
 INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-----	-----	-----	-----	-----
00	MHLRES1	NONE	NO	NO

AUDITING

 FAILURES(READ)

NOTIFY

 NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

 NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
 INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-----	-----	-----	-----	-----
00	MHLRES7	NONE	NO	NO

AUDITING

 FAILURES(READ)

NOTIFY

 NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

 NO SECURITY LEVEL

13.5.3 Test case 3

Function: Read the two previously created tape sets, but in this case the user has no access to the first data set. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE1

Result: The job ended without any errors, because the user have ALTER access to the TAPEVOL profile and to the second data set. This is only possible, because after RACF have checked the access to the data set profile it will check the access to the TAPEVOL profile. If one of two checks ended with a return code zero the access is allowed.

Example 13-14 shows you the JCL and the settings we have used to read the two data sets.

Example 13-14 Sample JCL used for test 3

```
//RACFTS03 JOB (999,POK),MSGLEVEL=1,
//          NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(A)
/*      RACF(Y)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**          ACC(NONE)
/*      MHLRES5.**          ACC(ALTER)
/*      MHLRES7.**          ACC(ALTER)
/*      THM001              ACC(ALTER)
/*      TAPEVOL              ACTIVE
/*      TAPEDSN              ACTIVE
/*
/*      FUNCTION
/*      READ THE PREVIOUSLY CREATED DATA SETS
/*      MHLRES1.RACF.TEST1.FILE1
/*      MHLRES7.RACF.TEST1.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD  *
SETR CLASSACT(TAPEVOL)
SETR TAPEDSN
PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACC(ALTER)
SETR REFRESH RACLIST(TAPEVOL)
RL TAPEVOL THM001 ALL
PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
```

```

LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF04)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-15 shows the most important results of the RACF commands.

Example 13-15 RACF Commands result of case 3

```

RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5      NONE              ALTER        NO

USER    ACCESS    ACCESS COUNT
-----
MHLRES5 ALTER        000000
STC     ALTER        000001

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE              NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE

```

```

-----
      NONE          SYS1          NON-VSAM
-----
NO INSTALLATION DATA

      SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES7      NONE          NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER          SYS1          NON-VSAM

NO INSTALLATION DATA

      SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.4 Test case 4

Function: Read the two previously created tape sets, but in this case the user has no access to the TAPEVOL profile. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE1

Result: The job ended without any errors, because the user have ALTER access to the data set profiles and there is no check for the access to the TAPEVOL profile.

Example 13-16 shows you the JCL and the settings we have used to read the two data sets.

Example 13-16 Sample JCL used for test 4

```

//RACFTS04 JOB (999,P0K),MSGLEVEL=1,
//          NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(A)
//*      RACF(Y)
//*
//*

```



```

/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**      ACC(ALTER)
/*      MHLRES5.**      ACC(ALTER)
/*      MHLRES7.**      ACC(ALTER)
/*      THM001          ACC(NONE)
/*      TAPEVOL          ACTIVE
/*      TAPEDSN          ACTIVE
/*
/*      FUNCTION
/*      READ THE PREVIOUSLY CREATED DATA SETS
/*      MHLRES1.RACF.TEST1.FILE1
/*      MHLRES7.RACF.TEST1.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD   SYSOUT=*
//SYSTSIN DD   *
    SETR CLASSACT(TAPEVOL)
    SETR TAPEDSN
    PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(NONE)
    SETR REFRESH RACLIST(TAPEVOL)
    RL TAPEVOL THM001 ALL
    PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
    PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
    SETR REFRESH GENERIC(DATASET)
    LD DATASET('MHLRES1.**') ALL
    LD DATASET('MHLRES7.**') ALL
    SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD   SYSOUT=*
//SYSUT1 DD   DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2 DD   DUMMY
//SYSIN DD   DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD   SYSOUT=*
//SYSUT1 DD   DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2 DD   DUMMY
//SYSIN DD   DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD   SYSOUT=*
//SYSUT1 DD   DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF05)
//SYSUT2 DD   SYSOUT=(A,INTRDR)
//SYSIN DD   DUMMY

```

Example 13-17 shows you the most important results of the RACF commands.

Example 13-17 RACF Commands of test case 4

```

RL TAPEVOL THM001 ALL
CLASS      NAME

```

```

-----
TAPEVOL      THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
 00    MHLRES5      NONE              NONE         NO

USER      ACCESS    ACCESS COUNT
-----
MHLRES5    NONE      000000
STC        ALTER      000001

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
 00    MHLRES1      NONE          NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
  ALTER      SYS1          NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
 00    MHLRES7      NONE          NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
  ALTER      SYS1          NON-VSAM

```

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

13.5.5 Test case 5

Function: Read the two previously created tape sets, but in this case the user has no access to the TAPEVOL profile and no access to the first file. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE1

Result: In this case we get a security violation for the first data set, because we have no access to the data set profile nor to the TAPEVOL profile. The second data set can be read without any errors.

Example 13-16 on page 518 shows you the JCL and the settings we have used to read the two data sets.

Example 13-18 Sample JCL used for test 5

```
//RACFTS05 JOB (999,P0K),MSGLEVEL=1,
//          NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*          TPRACF(A)
/*          RACF(Y)
/*
/*      DEVSUP
/*          TAPEAUTHDSN=N/A
/*          TAPEAUTHF1=N/A
/*          TAPEAUTHRC4=N/A
/*          TAPEAUTHRC8=N/A
/*
/*      RACF
/*          MHLRES1.**          ACC(ALTER)
/*          MHLRES5.**          ACC(ALTER)
/*          MHLRES7.**          ACC(ALTER)
/*          THM001              ACC(NONE)
/*          TAPEVOL              ACTIVE
/*          TAPEDSN              ACTIVE
/*
/*      FUNCTION
/*          READ THE PREVIOUSLY CREATED DATA SETS
/*              MHLRES1.RACF.TEST1.FILE1
/*              MHLRES7.RACF.TEST1.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD    SYSOUT=*
```

```

//SYSTSIN DD *
  SETR CLASSACT(TAPEVOL)
  SETR TAPEDSN
  PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(NONE)
  SETR REFRESH RACLIST(TAPEVOL)
  RL TAPEVOL THM001 ALL
  PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF06)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-19 shows the most important result of the RACF commands.

Example 13-19 RACF Commands of test case 5

```

RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5      NONE                NONE         NO

USER    ACCESS    ACCESS COUNT
-----
MHLRES5  NONE        000000
STC      ALTER      000001

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----

```

00 MHLRES1 NONE NO NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-------------	----------------	--------------

-----	-----	-----
-------	-------	-------

NONE	SYS1	NON-VSAM
------	------	----------

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-------	-------	------------------	---------	-------

-----	-----	-----	-----	-----
-------	-------	-------	-------	-------

00	MHLRES7	NONE	NO	NO
----	---------	------	----	----

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-------------	----------------	--------------

-----	-----	-----
-------	-------	-------

ALTER	SYS1	NON-VSAM
-------	------	----------

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-20 you can see that we get a security violation for file one.

Example 13-20 Test case 5 job output

....

IEF237I DMY ALLOCATED TO SYSIN

ICH408I USER(MHLRES5) GROUP(SYS1) NAME(MARY LOVELACE - RESI)

THM001 CL(TAPEVOL)

INSUFFICIENT ACCESS AUTHORITY

ACCESS INTENT(READ) ACCESS ALLOWED(NONE)

```

ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
MHLRES1.RACF.TEST1.FILE1 CL(DATASET ) VOL(THM001)
INSUFFICIENT ACCESS AUTHORITY
FROM MHLRES1.** (G)
ACCESS INTENT(READ ) ACCESS ALLOWED(NONE )
IEC150I 913-60,IFG0194F,RACFTS05,STEP01,SYSUT1,0B23,,MHLRES1.RACF.TEST1.FILE1
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000060
TIME=18.11.17 SEQ=00982 CPU=0000 ASID=0039
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
E: 00000000/00000000_80C50762 F: 00000002/00000010_00000060
END OF SYMPTOM DUMP
IEF472I RACFTS05 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000060
....

```

13.5.6 Test case 6

Function:	Read the two previously created tape sets, but in this case the RACF CLASS TAPEVOL is inactive. Program IEBGENER is used to read the files.
Data set names:	MHLRES1.RACF.TEST1.FILE1 MHLRES7.RACF.TEST1.FILE1
Result:	The job ended without any errors, because the user have ALTER access to both data set profiles. The RACF TAPEVOL profile is not checked.

Example 13-21 shows you the JCL and the settings we have used to read the two data sets.

Example 13-21 Sample JCL used for test 6

```

//RACFTS06 JOB (999,POK),MSGLEVEL=1,
//          NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(A)
//*      RACF(Y)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=N/A
//*      TAPEAUTHF1=N/A
//*      TAPEAUTHRC4=N/A
//*      TAPEAUTHRC8=N/A
//*
```

```

/*      RACF
/*      MHLRES1.**      ACC(ALTER)
/*      MHLRES5.**      ACC(ALTER)
/*      MHLRES7.**      ACC(ALTER)
/*      THM001          NO ACCESS
/*      TAPEVOL          INACTIVE
/*      TAPEDSN          ACTIVE
/*
/*      FUNCTION
/*      READ THE PREVIOUSLY CREATED DATA SETS
/*      MHLRES1.RACF.TEST1.FILE1
/*      MHLRES7.RACF.TEST1.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR NOCLASSACT(TAPEVOL)
  SETR TAPEDSN
  PE THM001 CLASS(TAPEVOL) ID(MHLRES5) DELETE
  SETR REFRESH RACLIST(TAPEVOL)
  RL TAPEVOL THM001 ALL
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF07)
//SYSUT2  DD  SYSOUT=(A,INTRDR)
//SYSIN   DD  DUMMY

```

Example 13-22 shows the most important results of the RACF commands.

Example 13-22 RACF Commands of test case 6

```

  SETR NOCLASSACT(TAPEVOL)
READY
  SETR TAPEDSN
WARNING: TAPEDSN OPTION ACTIVE, TAPEVOL CLASS IS NOT ACTIVE

```

READY

RL TAPEVOL THM001 ALL

CLASS NAME

TAPEVOL THM001

LEVEL	OWNER	UNIVERSAL ACCESS	YOUR ACCESS	WARNING
-------	-------	------------------	-------------	---------

-----	-----	-----	-----	-----
-------	-------	-------	-------	-------

00	MHLRES5	NONE	ALTER	NO
----	---------	------	-------	----

USER	ACCESS	ACCESS COUNT
------	--------	--------------

----	-----	-----
------	-------	-------

STC	ALTER	000001
-----	-------	--------

LD DATASET('MHLRES1.**') ALL

INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-------	-------	------------------	---------	-------

-----	-----	-----	-----	-----
-------	-------	-------	-------	-------

00	MHLRES1	NONE	NO	NO
----	---------	------	----	----

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-------------	----------------	--------------

-----	-----	-----
-------	-------	-------

ALTER	SYS1	NON-VSAM
-------	------	----------

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL

INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-------	-------	------------------	---------	-------

-----	-----	-----	-----	-----
-------	-------	-------	-------	-------

00	MHLRES7	NONE	NO	NO
----	---------	------	----	----

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
ALTER	SYS1	NON-VSAM
NO INSTALLATION DATA		
SECURITY LEVEL		

NO SECURITY LEVEL		

13.5.7 Test case 7

Function: Read the two previously created tape sets, but in this case the RACF CLASS TAPEVOL is inactive and the user have no access to the first data set. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE1

Result: In this case we get a security violation for the first data set, because we have no access to the data set profile and the TAPEVOL profile is not checked, because RACF CLASS TAPEVOL is inactive. The second data set can be red without any errors.

Example 13-23 shows you the JCL and the settings we have used to red the two data sets.

Example 13-23 Sample JCL used for test 7

```
//RACFTS07 JOB (999,P0K),MSGLEVEL=1,
//          NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(A)
/*      RACF(Y)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**          ACC(NONE)
/*      MHLRES5.**          ACC(ALTER)
/*      MHLRES7.**          ACC(ALTER)
/*      THM001              NO ACCESS
/*      TAPEVOL              INACTIVE
/*      TAPEDSN              ACTIVE
/*
/*      FUNCTION
/*      READ THE PREVIOUSLY CREATED DATA SETS
/*      MHLRES1.RACF.TEST1.FILE1
/*      MHLRES7.RACF.TEST1.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
```

```

//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR NOCLASSACT(TAPEVOL)
  SETR TAPEDSN
  RL TAPEVOL THM001 ALL
  PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
//STEP01  EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//STEP02  EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//SUBMIT  EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF08)
//SYSUT2  DD  SYSOUT=(A,INTRDR)
//SYSIN   DD  DUMMY

```

Example 13-24 shows the most important result of the RACF commands.

Example 13-24 RACF Commands of test case 7

```

  SETR NOCLASSACT(TAPEVOL)
READY
  SETR TAPEDSN
WARNING: TAPEDSN OPTION ACTIVE, TAPEVOL CLASS IS NOT ACTIVE
READY

  RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
  00    MHLRES5      NONE                ALTER        NO

USER    ACCESS    ACCESS COUNT
-----
STC     ALTER     000001

LD DATASET('MHLRES1.**') ALL

```

INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
NONE	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-25 you can see the security violation we get for the first file, but in this case only for the data set profile.

Example 13-25 Test case 7 job output

....
IEF237I DMY ALLOCATED TO SYSIN

```

ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
MHLRES1.RACF.TEST1.FILE1 CL(DATASET ) VOL(THM001)
INSUFFICIENT ACCESS AUTHORITY
FROM MHLRES1.** (G)
ACCESS INTENT(READ ) ACCESS ALLOWED(NONE )
IEC150I 913-60,IFG0194F,RACFTS07,STEP01,SYSUT1,0B23,,MHLRES1.RACF.TEST1.FILE1
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000060
TIME=18.13.50 SEQ=00983 CPU=0000 ASID=0039
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
E: 00000000/00000000_80C50762 F: 00000002/00000010_00000060
END OF SYMPTOM DUMP
IEF472I RACFTS07 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000060
....

```

13.5.8 Test case 8

Function:	Read the two previously created tape sets, but in this case both RACF CLASSES TAPEVOL and TAPEDSN are inactive. Program IEBGENER is used to read the files. In this case you have all needed RACF acces and the job ended without any error as expected.
Data set names:	MHLRES1.RACF.TEST1.FILE1 MHLRES7.RACF.TEST1.FILE1
Result:	The job eneded without any errors, because there are no security checks.

Example 13-26 shows you the JCL and the settings we have used to red the two data sets.

Example 13-26 Sample JCL used for test 8

```

//RACFTS08 JOB (999,POK),MSGLEVEL=1,
//          NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(A)
//*      RACF(Y)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=N/A
//*      TAPEAUTHF1=N/A
//*      TAPEAUTHRC4=N/A
//*      TAPEAUTHRC8=N/A
//*

```

```

/*      RACF
/*      MHLRES1.**      ACC(ALTER)
/*      MHLRES5.**      ACC(ALTER)
/*      MHLRES7.**      ACC(ALTER)
/*      TAPEVOL          INACTIVE
/*      TAPEDSN          INACTIVE
/*      THM001          NO ACCESS
/*
/*      FUNCTION
/*      READ THE PREVIOUSLY CREATED DATA SETS
/*      MHLRES1.RACF.TEST1.FILE1
/*      MHLRES7.RACF.TEST1.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR NOCLASSACT(TAPEVOL)
  SETR NOTAPEDSN
  RL TAPEVOL THM001 ALL
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//SUBMIT  EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF09)
//SYSUT2  DD  SYSOUT=(A,INTRDR)
//SYSIN   DD  DUMMY

```

Example 13-27 shows the most important result of the RACF commands.

Example 13-27 RACF Commands of test case 8

```

  SETR NOCLASSACT(TAPEVOL)
READY
  SETR NOTAPEDSN

  RL TAPEVOL THM001 ALL
CLASS      NAME

```

```

-----
TAPEVOL      THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
 00    MHLRES5      NONE              ALTER        NO

USER      ACCESS  ACCESS COUNT
-----
STC        ALTER    000001

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
 00    MHLRES1      NONE          NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
  ALTER      SYS1          NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL
-----
NO SECURITY LEVEL
LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
 00    MHLRES7      NONE          NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
  ALTER      SYS1          NON-VSAM

NO INSTALLATION DATA

```

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-26 you can see that there was no security violation.

13.5.9 Test case 9

Function: Read the two previously created tape sets, both RACF CLASSES TAPEVOL and TAPEDSN are inactive and the user have no access to the data set profile that protect the second file. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE1

Result: The job ended without any errors, because there are no security checks.

Example 13-28 shows you the JCL and the settings we have used to red the two data sets.

Example 13-28 Sample JCL used for test 9

```
//RACFTS09 JOB (999,P0K),MSGLEVEL=1,
//          NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(A)
/*      RACF(Y)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**          ACC(NONE)
/*      MHLRES5.**          ACC(ALTER)
/*      MHLRES7.**          ACC(ALTER)
/*      THM001              NO ACCESS
/*      TAPEVOL              INACTIVE
/*      TAPEDSN              INACTIVE
/*
/*      FUNCTION
/*      READ THE PREVIOUSLY CREATED DATA SETS
/*      MHLRES1.RACF.TEST1.FILE1
/*      MHLRES7.RACF.TEST1.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD  *
SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
```

```

RL TAPEVOL THM001 ALL
PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF10)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-29 shows the most important result sof the RACF commands.

Example 13-29 RACF Commands of test case 9

```

SETR NOCLASSACT(TAPEVOL)
READY
SETR NOTAPEDSN

RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5      NONE                ALTER        NO

USER    ACCESS    ACCESS COUNT
-----
STC     ALTER      000001

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE                NO       NO

```



```

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
   00   MHLRES7          NONE          NO      NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.10 Test case 10

Function: Create an additional third new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library.

Data set names: MHLRES7.RACF.TEST1.FILE3

Result: The job ended without any errors , because we have ALTER access to the RACF TAPEVOL profile for volume THM001 and the user MHLRES5 have ALTER access to the data set profile.

Example 13-30 on page 536 shows you the JCL and the settings we have used to create the third data set.

Example 13-30 Sample JCL used for test 10

```
//RACFTS10 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(Y)
/*      RACF(Y)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**          ACC(ALTER)
/*      MHLRES5.**          ACC(ALTER)
/*      MHLRES7.**          ACC(ALTER)
/*      THM001              ACC(ALTER)
/*      TAPEVOL             ACTIVE
/*      TAPEDSN             ACTIVE
/*
/*      FUNCTION
/*      CREATE AN ADDITIONAL NEW DATA SET
/*      MHLRES7.RACF.TEST1.FILE3
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD  *
SETR CLASSACT(TAPEVOL)
SETR TAPEDSN
PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
SETR REFRESH RACLIST(TAPEVOL)
PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR REFRESH GENERIC(DATASET)
SETROPTS LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN DD  *
DELETE MHLRES7.RACF.TEST1.FILE3 NONVSAM NOSCRATCH
SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD  LABEL=(03,SL),DSN=MHLRES7.RACF.TEST1.FILE3,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=SER=THM001
//SYSIN DD  DUMMY
```

```

//*-----*
//SUBMIT EXEC PGM=IEBGGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF11)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-31 shows the most important results of the RACF commands.

Example 13-31 RACF Commands of test case 10

```

RL TAPEVOL THM001
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5      NONE                ALTER        NO

USER      ACCESS  ACCESS COUNT
-----
STC        ALTER      000005
MHLRES5    ALTER      000000

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE                NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER        SYS1          NON-VSAM

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES7      NONE                NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY

```

```

-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.11 Test case 11

Function: Create an additional fourth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library.

Data set names: MHLRES7.RACF.TEST1.FILE4

Result: In this case we get a security violation for the new fourth data set we would like to create it, because we have no access to the fourth data set profile.

Example 13-32 shows you the JCL and the settings we have used to create the third data set.

Example 13-32 Sample JCL used for test 11

```

//RACFTS11 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(Y)
//*      RACF(Y)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=N/A
//*      TAPEAUTHF1=N/A
//*      TAPEAUTHRC4=N/A
//*      TAPEAUTHRC8=N/A
//*
//*      RACF
//*      MHLRES1.**          ACC(ALTER)
//*      MHLRES5.**          ACC(ALTER)
//*      MHLRES7.**          ACC(NONE)
//*      THM001              ACC(ALTER)
//*      TAPEVOL              ACTIVE
//*      TAPEDSN              ACTIVE
//*
//*      FUNCTION
//*      CREATE AN ADDITIONAL NEW DATA SETS
//*      MHLRES7.RACF.TEST1.FILE4
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *

```

```

SETR CLASSACT(TAPEVOL)
SETR TAPEDSN
PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
SETR REFRESH RACLIST(TAPEVOL)
RL TAPEVOL THM001 ALL
PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR REFRESH GENERIC(DATASET)
SETRPTS LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
DELETE MHLRES7.RACF.TEST1.FILE4 NONVSAM NOSCRATCH
SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD LABEL=(04,SL),DSN=MHLRES7.RACF.TEST1.FILE4,
// DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
// VOLUME=SER=THM001
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF12)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-33 shows the most important results of the RACF commands.

Example 13-33 RACF Commands of tase case 11

```

RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5      NONE              ALTER        NO

USER    ACCESS    ACCESS COUNT
-----
STC     ALTER      000005
MHLRES5 ALTER      000000

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----

```

```

00      MHLRES1          NONE          NO          NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER      SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

      LD DATASET('MHLRES7.**') ALL
      INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00      MHLRES7          NONE          NO          NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE      SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

In the output of the job in Example 13-34 you can see that there was no security violation.

Example 13-34 Test case 11 job output

```

.....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1      ) NAME(MARY LOVELACE - RESI)
      MHLRES7.RACF.TEST1.FILE4 CL(DATASET ) VOL(THM001)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES7.** (G)
      ACCESS INTENT(UPDATE ) ACCESS ALLOWED(NONE )

```

```

IEC150I 913-60,IFG0194F,RACFTS11,STEP01,SYSUT2,0B23,,MHLRES7.RACF.TEST1.FILE4
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000060
TIME=08.00.54 SEQ=01017 CPU=0000 ASID=002C
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
      2: 00000000/00000000_0000C3E4 3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B51F8 5: 00000000/00000000_007B558C
      6: 00000000/00000000_007B5534 7: 00000000/00000000_007B558C
      8: 00000000/00000000_007B5554 9: 00000000/00000000_007B3328
      A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B54B8
      E: 00000000/00000000_80C50762 F: 00000002/00000010_00000060
END OF SYMPTOM DUMP
IEF472I RACFTS11 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000060
....

```

13.5.12 Test case 12

Function: Repeate the creation of the additional fourth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library.

Data set names: MHLRES7.RACF.TEST1.FILE4

Result: The job eneded without any errors , because we have access to the RACF TAPEVOL profile for volume THM00, although the user MHLRES5 have ALTER access to the data set profile this access is not checked because RACF class TAPEDSN is inactive.

Example 13-35 shows you the JCL and the settings we have used to create the third data set.

Example 13-35 Sample JCL used for test 12

```

//RACFTS12 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//* RMM OPTIONS:
//* TPRACF(Y)
//* RACF(Y)
//*
//* DEVSUP
//* TAPEAUTHDSN=N/A
//* TAPEAUTHF1=N/A
//* TAPEAUTHRC4=N/A
//* TAPEAUTHRC8=N/A
//*
//* RACF
//* MHLRES1.** ACC(ALTER)
//* MHLRES5.** ACC(ALTER)
//* MHLRES7.** ACC(ALTER)
//* THM001 ACC(ALTER)
//* TAPEVOL ACTIVE

```

```

/*          TAPEDSN          INACTIVE
/*
/*      FUNCTION
/*          CREATE AN ADDITIONAL NEW DATA SET
/*          MHLRES7.RACF.TEST1.FILE4
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR CLASSACT(TAPEVOL)
  SETR NOTAPEDSN
  PE THM001CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
  SETR REFRESH RACLIST(TAPEVOL)
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR REFRESH GENERIC(DATASET)
  SETROPTS LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN  DD  *
  DELETE MHLRES7.RACF.TEST1.FILE4 NONVSAM NOSCRATCH
  SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD  LABEL=(04,SL),DSN=MHLRES7.RACF.TEST1.FILE4,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=SER=THM001
//SYSIN  DD  DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF13)
//SYSUT2 DD  SYSOUT=(A,INTRDR)
//SYSIN  DD  DUMMY

```

Example 13-36 shows you the most important result of the RACF commands.

Example 13-36 RACF Commands of test case 12

```

RL TAPEVOL THM001ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5      NONE                ALTER        NO

USER    ACCESS    ACCESS COUNT
-----

```


STC ALTER 000005
MHLRES5 ALTER 000000

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

13.5.13 Test case 13

Function: Create an additional fifth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library.

Data set names: MHLRES7.RACF.TEST1.FILE4

Result: The job ended without any errors , because the RACF option NOTAPEDSN class is being set and the user MHLRES5 have ALTER access to the TAPEVOL.

Example 13-37 shows you the JCL and the settings we have used to create the third data set.

Example 13-37 Sample JCL used for test 13

```
//RACFTS13 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(Y)
/*      RACF(Y)
/*
/*
/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**          ACC(NONE)
/*      MHLRES5.**          ACC(ALTER)
/*      MHLRES7.**          ACC(ALTER)
/*      THM001              ACC(ALTER)
/*      TAPEVOL              ACTIVE
/*      TAPEDSN              INACTIVE
/*
/*      FUNCTION
/*      CREATE AN ADDITIONAL NEW DATA SETS
/*      MHLRES7.RACF.TEST1.FILE5
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD  *
SETR CLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE THM001CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
SETR REFRESH RACLIST(TAPEVOL)
PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR REFRESH GENERIC(DATASET)
SETROPTS LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN DD  *
```

```

DELETE MHLRES7.RACF.TEST1.FILE5 NONVSAM NOSCRATCH
SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD LABEL=(05,SL),DSN=MHLRES7.RACF.TEST1.FILE5,
// DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
// VOLUME=SER=THM001
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF14)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-38 shows you the most important result of the RACF commands.

Example 13-38 RACF Commands of test case 13

```

RL TAPEVOL THM001ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5      NONE                ALTER        NO

USER      ACCESS   ACCESS COUNT
-----
STC        ALTER      000005
MHLRES5    ALTER      000000

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE                NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER        SYS1          NON-VSAM

NO INSTALLATION DATA

```

```

                                SECURITY LEVEL
                                -----
NO SECURITY LEVEL

                                SECURITY LEVEL
                                -----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER    UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES7      NONE           NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE      SYS1          NON-VSAM

NO INSTALLATION DATA

                                SECURITY LEVEL
                                -----
NO SECURITY LEVEL

```

13.5.14 Test case 14

Function: Create an additional sixth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library.

Data set names: MHLRES7.RACF.TEST1.FILE6

Result: The job ended without any errors , because the user MHLRES5 have ALTER access to the data set profile and RACF class TAPEVOL is incative.

Example 13-39 shows you the JCL we have used

Example 13-39 Sample JCL used for test 14

```

//RACFTS14 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(Y)
//*      RACF(Y)
//*
//*

```

```

/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**          ACC(ALTER)
/*      MHLRES5.**          ACC(ALTER)
/*      MHLRES7.**          ACC(ALTER)
/*      TAPEVOL             INACTIVE
/*      TAPEDSN             ACTIVE
/*      THM001              ACC(ALTER)
/*
/*      FUNCTION
/*      CREATE AN ADDITIONAL NEW DATA SETS
/*      MHLRES7.RACF.TEST1.FILE6
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR NOCLASSACT(TAPEVOL)
  SETR TAPEDSN
  PE THM001CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
  SETR REFRESH RACLIST(TAPEVOL)
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR REFRESH GENERIC(DATASET)
  SETROPTS LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN  DD  *
  DELETE MHLRES7.RACF.TEST1.FILE6 NONVSAM NOSCRATCH
  SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD  LABEL=(06,SL),DSN=MHLRES7.RACF.TEST1.FILE6,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=SER=THM001
//SYSIN  DD  DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF15)
//SYSUT2 DD  SYSOUT=(A,INTRDR)
//SYSIN  DD  DUMMY

```

Example 13-40 on page 548 shows the most important result of the RACF commands.

Example 13-40 RACF Commands

```

SETR NOCLASSACT(TAPEVOL)
READY
SETR TAPEDSN
WARNING: TAPEDSN OPTION ACTIVE, TAPEVOL CLASS IS NOT ACTIVE

```

```

RL TAPEVOL THM001ALL
CLASS      NAME
-----
TAPEVOL    THM001

```

LEVEL	OWNER	UNIVERSAL ACCESS	YOUR ACCESS	WARNING
00	MHLRES5	NONE	ALTER	NO

USER	ACCESS	ACCESS COUNT
STC	ALTER	000005
MHLRES5	ALTER	000000

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

```

AUDITING
-----
FAILURES(READ)

```

```

NOTIFY
-----
NO USER TO BE NOTIFIED

```

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

```

NO INSTALLATION DATA

```

```

                SECURITY LEVEL
-----
NO SECURITY LEVEL

```

```

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

```

AUDITING
-----
FAILURES(READ)

```

```

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER      SYS1      NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.15 Test case 15

Function: Create an additional seventh new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library.

Data set names: MHLRES7.RACF.TEST1.FILE7

Result: In this case we get a security violation for the new seventh data set we would like to create it, because we have no access to the data set profile and the RACF CLASS TAPEVOL is inactive.

Example 13-41 shows you the JCL and the settings we have used to create the third data set.

Example 13-41 Sample JCL used for test 15

```

//RACFTS15 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(Y)
//*      RACF(Y)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=N/A
//*      TAPEAUTHF1=N/A
//*      TAPEAUTHRC4=N/A
//*      TAPEAUTHRC8=N/A
//*
//*      RACF
//*      MHLRES1.**      ACC(ALTER)
//*      MHLRES5.**      ACC(ALTER)
//*      MHLRES7.**      ACC(NONE)
//*      THM001          ACC(ALTER)
//*      TAPEVOL          INACTIVE
//*      TAPEDSN          ACTIVE
//*
//*      FUNCTION
//*      CREATE AN ADDITIONAL NEW DATA SETS
//*      MHLRES7.RACF.TEST1.FILE7
//*
//RACFCMDS EXEC PGM=IKJEFT01

```

```

//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR NOCLASSACT(TAPEVOL)
  SETR TAPEDSN
  PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
  SETR REFRESH RACLIST(TAPEVOL)
  RL TAPEVOL THM001 ALL
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR REFRESH GENERIC(DATASET)
  SETROPTS LIST
/*
//CLEANUP  EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN    DD  *
  DELETE MHLRES7.RACF.TEST1.FILE6 NONVSAM NOSCRATCH
  SET MAXCC=0
/*
/*-----*
//STEP01   EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1   DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2   DD  LABEL=(07,SL),DSN=MHLRES7.RACF.TEST1.FILE7,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=SER=THM001
//SYSIN    DD  DUMMY

```

Example 13-42 shows you the most important result of the RACF commands.

Example 13-42 RACF Commands of test case 14

```

  SETR NOCLASSACT(TAPEVOL)
READY
  SETR TAPEDSN
WARNING: TAPEDSN OPTION ACTIVE, TAPEVOL CLASS IS NOT ACTIVE
READY
  PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
READY
  SETR REFRESH RACLIST(TAPEVOL)
RACLIST REFRESH of class TAPEVOL ignored. The class is not active yet.
SETROPTS command complete.

```

```

  RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
  00    MHLRES5      NONE              ALTER        NO

USER    ACCESS    ACCESS COUNT
-----
STC     ALTER     000005

```


MHLRES5 ALTER 000000

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
NONE	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-43 on page 552 you can see that there was no security violation.

Example 13-43 Test case 15 job output

```
....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1    ) NAME(MARY LOVELACE - RESI)
      MHLRES7.RACF.TEST1.FILE7 CL(DATASET ) VOL(THM001)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES7.** (G)
      ACCESS INTENT(UPDATE ) ACCESS ALLOWED(NONE    )
IEC150I 913-60,IFG0194F,RACFTS15,STEP01,SYSUT2,0B23,,MHLRES7.RACF.TEST1.FILE7
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913  REASON CODE=00000060
TIME=08.49.02 SEQ=01037 CPU=0000 ASID=0039
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2  INTC 0D
      NO ACTIVE MODULE FOUND
      NAME=UNKNOWN
      DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
      AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
          2: 00000000/00000000_0000C3E4 3: 00000000/00000000_00C5062A
          4: 00000000/00000000_007B51F8 5: 00000000/00000000_007B558C
          6: 00000000/00000000_007B5534 7: 00000000/00000000_007B558C
          8: 00000000/00000000_007B5554 9: 00000000/00000000_007B3328
          A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
          C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B54B8
          E: 00000000/00000000_80C50762 F: 00000002/00000010_00000060
      END OF SYMPTOM DUMP
IEF472I RACFTS15 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000060
....
```

13.5.16 Test case 16

Function: Read the six previously created tape sets, but in this case both RACF CLASSES TAPEVOL and TAPEDSN are inactive. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE2
MHLRES7.RACF.TEST1.FILE3
MHLRES7.RACF.TEST1.FILE4
MHLRES7.RACF.TEST1.FILE5
MHLRES7.RACF.TEST1.FILE6

Result: The job ended without any errors, because no security checks are being performed.

Example 13-28 on page 533 shows you the JCL and the settings we have used to read the two data sets.

Example 13-44 Sample JCL used for test 2

```
//RACFTS16 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//* *****
//* * /F DFRMM,M=02 *
//* *****
//*
//* RMM OPTIONS:
```

```

//*      TPRACF(N)
//*      RACF(N)
//*
//*
//*      DEVSUP
//*      TAPEAUTHDSN=N/A
//*      TAPEAUTHF1=N/A
//*      TAPEAUTHRC4=N/A
//*      TAPEAUTHRC8=N/A
//*
//*      RACF
//*      MHLRES1.**      ACC(ALTER)
//*      MHLRES5.**      ACC(ALTER)
//*      MHLRES5.**      ACC(ALTER)
//*      THM001          ACC(ALTER)
//*      TAPEVOL          ACTIVE
//*      TAPEDSN          ACTIVE
//*
//*      FUNCTION
//*      READ ALL PREVIOUSLY CREATED DATA SETS
//*      MHLRES1.RACF.TEST1.FILE1
//*      MHLRES7.RACF.TEST1.FILE2
//*      MHLRES7.RACF.TEST1.FILE3
//*      MHLRES7.RACF.TEST1.FILE4
//*      MHLRES7.RACF.TEST1.FILE5
//*      MHLRES7.RACF.TEST1.FILE6
//*      MHLRES7.RACF.TEST1.FILE7
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD  *
  SETR CLASSACT(TAPEVOL)
  SETR TAPEDSN
  PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
  SETR REFRESH RACLIST(TAPEVOL)
  RL TAPEVOL THM001 ALL
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR REFRESH GENERIC(DATASET)
  SETROPTS LIST
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2 DD  DUMMY
//SYSIN DD  DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)

```

```

//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP03 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE3,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(3,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP04 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE4,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(4,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP05 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE5,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(5,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP06 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE6,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(6,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP07 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE7,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(7,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF17)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-45 shows you the most important result of the RACF commands.

Example 13-45 RACF Commands of test case 16

```

RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5    NONE                ALTER        NO

```

USER	ACCESS	ACCESS COUNT
----	-----	-----
STC	ALTER	000005
MHLRES5	ALTER	000000

LD DATASET('MHLRES1.**') ALL
 INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-----	-----	-----	-----	-----
00	MHLRES1	NONE	NO	NO

AUDITING

 FAILURES(READ)

NOTIFY

 NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

 NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
 INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-----	-----	-----	-----	-----
00	MHLRES7	NONE	NO	NO

AUDITING

 FAILURES(READ)

NOTIFY

 NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

13.5.17 Test case 17

Function: Read the six previously created tape sets, but in this case RACF CLASS TAPEVOL is active and option TAPEDSN is being set. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE2
MHLRES7.RACF.TEST1.FILE3
MHLRES7.RACF.TEST1.FILE4
MHLRES7.RACF.TEST1.FILE5
MHLRES7.RACF.TEST1.FILE6

Result: The job ended without any errors, though the user has no access to the data set profile MHLRES1.**. In this situation RACF checks that the user has UPDATE access to the TAPEVOL profile.

Example 13-46 shows you the JCL and the settings we have used to read the two data sets.

Example 13-46 Sample JCL used for test 17

```
//RACFTS17 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**      ACC(NONE)
/*      MHLRES5.**      ACC(ALTER)
/*      MHLRES7.**      ACC(ALTER)
/*      THM001          ACC(ALTER)
/*      TAPEVOL          ACTIVE
/*      TAPEDSN          ACTIVE
/*
/*      FUNCTION
/*      READ ALL PREVIOUSLY CREATED DATA SETS
/*      MHLRES1.RACF.TEST1.FILE1
/*      MHLRES7.RACF.TEST1.FILE2
/*      MHLRES7.RACF.TEST1.FILE3
/*      MHLRES7.RACF.TEST1.FILE4
/*      MHLRES7.RACF.TEST1.FILE5
/*      MHLRES7.RACF.TEST1.FILE6
/*      MHLRES7.RACF.TEST1.FILE7
/*
//RACFCMDS EXEC PGM=IKJEFT01
```

```

//SYSTSPRT DD   SYSOUT=*
//SYSTSIN  DD   *
  SETR CLASSACT(TAPEVOL)
  SETR TAPEDSN
  PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
  SETR REFRESH RACLIST(TAPEVOL)
  RL TAPEVOL THM001 ALL
  PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR REFRESH GENERIC(DATASET)
  SETROPTS LIST
/*
/*-----*
//STEP01  EXEC PGM=IEBGENER
//SYSPRINT DD   SYSOUT=*
//SYSUT1  DD   DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2  DD   DUMMY
//SYSIN   DD   DUMMY
/*-----*
//STEP02  EXEC PGM=IEBGENER
//SYSPRINT DD   SYSOUT=*
//SYSUT1  DD   DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2  DD   DUMMY
//SYSIN   DD   DUMMY
/*-----*
//STEP03  EXEC PGM=IEBGENER
//SYSPRINT DD   SYSOUT=*
//SYSUT1  DD   DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE3,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(3,SL)
//SYSUT2  DD   DUMMY
//SYSIN   DD   DUMMY
/*-----*
//STEP04  EXEC PGM=IEBGENER
//SYSPRINT DD   SYSOUT=*
//SYSUT1  DD   DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE4,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(4,SL)
//SYSUT2  DD   DUMMY
//SYSIN   DD   DUMMY
/*-----*
//STEP05  EXEC PGM=IEBGENER
//SYSPRINT DD   SYSOUT=*
//SYSUT1  DD   DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE5,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(5,SL)
//SYSUT2  DD   DUMMY
//SYSIN   DD   DUMMY
/*-----*
//STEP06  EXEC PGM=IEBGENER
//SYSPRINT DD   SYSOUT=*
//SYSUT1  DD   DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE6,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(6,SL)
//SYSUT2  DD   DUMMY

```

```
//SYSIN DD DUMMY
/*-----*
//STEP07 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE7,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(7,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF18)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY
```

Example 13-47 shows you the most important result of the RACF commands.

Example 13-47 RACF Commands of test case 17

```
RL TAPEVOL THM001 ALL
CLASS NAME
-----
TAPEVOL THM001
```

LEVEL	OWNER	UNIVERSAL ACCESS	YOUR ACCESS	WARNING
00	MHLRES5	NONE	ALTER	NO

```
USER ACCESS ACCESS COUNT
----
STC ALTER 000005
MHLRES5 ALTER 000000
```

```
LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)
```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

```
AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS CREATION GROUP DATASET TYPE
-----
NONE SYS1 NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL
-----
```


NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

13.5.18 Test case 18

Function: Read the six previously created tape sets, but in this case RACF CLASS TAPEVOL is inactive and option TAPEDSN is being set. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE2
MHLRES7.RACF.TEST1.FILE3
MHLRES7.RACF.TEST1.FILE4
MHLRES7.RACF.TEST1.FILE5
MHLRES7.RACF.TEST1.FILE6

Result: The job ended without any errors, because the user have access to the data set profile and the RACF TAPEVOL class is inactive.

Example 13-48 shows you the JCL and the settings we have used to read the two data sets.

Example 13-48 Sample JCL used for test 18

```
//RACFTS18 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*
/*      DEVSUP
```

```

//*      TAPEAUTHDSN=N/A
//*      TAPEAUTHF1=N/A
//*      TAPEAUTHRC4=N/A
//*      TAPEAUTHRC8=N/A
//*
//*      RACF
//*      MHLRES1.**      ACC(ALTER)
//*      MHLRES5.**      ACC(ALTER)
//*      MHLRES7.**      ACC(ALTER)
//*      THM001          ACC(ALTER)
//*      TAPEVOL          INACTIVE
//*      TAPEDSN          ACTIVE
//*
//*      FUNCTION
//*      READ ALL PREVIOUSLY CREATED DATA SETS
//*      MHLRES1.RACF.TEST1.FILE1
//*      MHLRES7.RACF.TEST1.FILE2
//*      MHLRES7.RACF.TEST1.FILE3
//*      MHLRES7.RACF.TEST1.FILE4
//*      MHLRES7.RACF.TEST1.FILE5
//*      MHLRES7.RACF.TEST1.FILE6
//*      MHLRES7.RACF.TEST1.FILE7
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR NOCLASSACT(TAPEVOL)
  SETR TAPEDSN
  PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
  SETR REFRESH RACLIST(TAPEVOL)
  RL TAPEVOL THM001 ALL
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR REFRESH GENERIC(DATASET)
  SETROPTS LIST
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//STEP03 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*

```

```

//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE3,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(3,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP04 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE4,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(4,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP05 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE5,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(5,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP06 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE6,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(6,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP07 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE7,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(7,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF19)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-49 shows you the most important result of the RACF commands.

Example 13-49 RACF Commands of test case 18

```

RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001

LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     MHLRES5     NONE              ALTER        NO

USER    ACCESS    ACCESS COUNT
-----
STC     ALTER      000005
MHLRES5 ALTER      000000

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

13.5.19 Test case 19

Function: Read the six previously created tape sets, but in this case RACF CLASS TAPEVOL is inactive and option TAPEDSN is being set. Program IEBGENER is used to read the files.

Data set names: MHLRES1.RACF.TEST1.FILE1
MHLRES7.RACF.TEST1.FILE2
MHLRES7.RACF.TEST1.FILE3
MHLRES7.RACF.TEST1.FILE4
MHLRES7.RACF.TEST1.FILE5
MHLRES7.RACF.TEST1.FILE6

Result: We get a security violation, because we have no access to the data set profile MHLRES1.** and the RACFCLASS TAPEVOL is inactive.

Example 13-50 shows you the JCL and the settings we have used to read the two data sets.

Example 13-50 Sample JCL used for test 19

```
//RACFTS19 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=N/A
/*      TAPEAUTHF1=N/A
/*      TAPEAUTHRC4=N/A
/*      TAPEAUTHRC8=N/A
/*
/*      RACF
/*      MHLRES1.**          ACC(NONE)
/*      MHLRES5.**          ACC(ALTER)
/*      MHLRES7.**          ACC(ALTER)
/*      THM001              ACC(ALTER)
/*      TAPEVOL              INACTIVE
/*      TAPEDSN              ACTIVE
/*
/*      FUNCTION
/*      READ ALL PREVIOUSLY CREATED DATA SETS
/*      MHLRES1.RACF.TEST1.FILE1
/*      MHLRES7.RACF.TEST1.FILE2
/*      MHLRES7.RACF.TEST1.FILE3
/*      MHLRES7.RACF.TEST1.FILE4
/*      MHLRES7.RACF.TEST1.FILE5
/*      MHLRES7.RACF.TEST1.FILE6
/*      MHLRES7.RACF.TEST1.FILE7
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD  *
SETR NOCLASSACT(TAPEVOL)
SETR TAPEDSN
PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
```

```

SETR REFRESH RACLIST(TAPEVOL)
RL TAPEVOL THM001 ALL
PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR REFRESH GENERIC(DATASET)
SETRPTS LIST
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.RACF.TEST1.FILE1,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(1,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE2,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP03 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE3,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(3,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP04 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE4,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(4,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP05 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE5,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(5,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP06 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE6,
// VOL=(,RETAIN,,,SER=(THM001)),LABEL=(6,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP07 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST1.FILE7,

```

```
//          VOL=(,RETAIN,,,SER=(THM001)),LABEL=(7,SL)
//SYSUT2   DD    DUMMY
//SYSIN     DD    DUMMY
```

Example 13-51 shows you the most important result of the RACF commands.

Example 13-51 RACF Commands

```
SETR NOCLASSACT(TAPEVOL)
READY
SETR TAPEDSN
WARNING: TAPEDSN OPTION ACTIVE, TAPEVOL CLASS IS NOT ACTIVE
READY
PE THM001 CLASS(TAPEVOL) ID(MHLRES5) ACCESS(ALTER)
READY
SETR REFRESH RACLIST(TAPEVOL)
RACLIST REFRESH of class TAPEVOL ignored. The class is not active yet.
SETRPTS command complete.
```

```
RL TAPEVOL THM001 ALL
CLASS      NAME
-----
TAPEVOL    THM001
```

LEVEL	OWNER	UNIVERSAL ACCESS	YOUR ACCESS	WARNING
00	MHLRES5	NONE	ALTER	NO

USER	ACCESS	ACCESS COUNT
STC	ALTER	000005
MHLRES5	ALTER	000000

```
LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)
```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

```
AUDITING
-----
FAILURES(READ)
```

```
NOTIFY
-----
NO USER TO BE NOTIFIED
```

YOUR ACCESS	CREATION GROUP	DATASET TYPE
NONE	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-52 you can see that there was no security violation.

Example 13-52 Test case 19 job output

ICH70001I MHLRES5 LAST ACCESS AT 18:37:04 ON SUNDAY, AUGUST 5, 2007

....

IEF237I DMY ALLOCATED TO SYSIN

ICH408I USER(MHLRES5) GROUP(SYS1) NAME(MARY LOVELACE - RESI)

MHLRES1.RACF.TEST1.FILE1 CL(DATASET) VOL(THM001)

INSUFFICIENT ACCESS AUTHORITY

FROM MHLRES1.** (G)

ACCESS INTENT(READ) ACCESS ALLOWED(NONE)

IEC150I 913-60,IFG0194F,RACFTS19,STEP01,SYSUT1,0B23,,MHLRES1.RACF.TEST1.FILE1

IEA995I SYMPTOM DUMP OUTPUT

SYSTEM COMPLETION CODE=913 REASON CODE=00000060

TIME=18.39.36 SEQ=00996 CPU=0000 ASID=0039

PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D

NO ACTIVE MODULE FOUND

NAME=UNKNOWN

DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0

AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000

2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A

4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4

6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4

8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8

A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C

C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0

E: 00000000/00000000_80C50762 F: 00000002/00000010_00000060

END OF SYMPTOM DUMP

IEF472I RACFTS19 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000060
....

13.5.20 Test case 20

This test case and all following test cases are under the condition that we have specified different settings of the new TAPEAUTH settings in the DEFSUPnn member and both RACF CLASSES TAPEVOL and TAPEDSN are inactive.

Function: Create two new tape sets on a tape volume in status scratch. Program IEBGENER is used to write the files. TAPEAUTHDSN and TAPEAUTHF1 are set and option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES1.RACF.TEST2.FILE1
MHLRES7.RACF.TEST2.FILE2

Result: The job ended without any errors, because the user have access to the data set profiles.

Example 13-53 shows you the JCL and the settings we have used to create the two data sets.

Example 13-53 Sample JCL used for test 20

```
//RACFTS20 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//* RMM OPTIONS:
//* TPRACF(N)
//* RACF(N)
//*
//*
//* DEVSUP
//* TAPEAUTHDSN=YES
//* TAPEAUTHF1=YES
//* TAPEAUTHRC4=FAIL
//* TAPEAUTHRC8=FAIL
//*
//* RACF
//* MHLRES1.** ACC(ALTER)
//* MHLRES5.** ACC(ALTER)
//* MHLRES7.** ACC(ALTER)
//* TAPEVOL INCATIVE
//* TAPEDSN INCATIVE
//*
//* FUNCTION
//* CREATE TWO NEW DATA SETS
//* MHLRES1.RACF.TEST2.FILE1
//* MHLRES7.RACF.TEST2.FILE2
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
PE 'MHLRES5.**' ACC(ALTER) ID(MHLRES5)
```

```

PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES5.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
DELETE MHLRES1.RACF.TEST2.FILE1 NONVSAM NOSCRATCH
DELETE MHLRES7.RACF.TEST2.FILE2 NONVSAM NOSCRATCH
SET MAXCC=0
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD DISP=(,CATLG),DSN=MHLRES1.RACF.TEST2.FILE1,
// VOL=(,RETAIN,,99),
// UNIT=ATL3
//SYSIN DD DUMMY
/*-----*
//STEP1 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD LABEL=(02,SL),DSN=MHLRES7.RACF.TEST2.FILE2,
// DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
// VOLUME=(,RETAIN,,REF=*.STEP01.SYSUT2)
//SYSIN DD DUMMY
/*-----*

```

13.5.21 Test case 21

Function: Red the two previously created tape sets. Program IEBGENER is used to read the files. TAPEAUTHDSN and TAPEAUTHF1 are set and option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES1.RACF.TEST2.FILE1
MHLRES7.RACF.TEST2.FILE2

Result: The job ended without any errors, because the user have access to the data set profiles.

Example 13-54 shows you the JCL we have used for test case 21.

Example 13-54 Sample JCL used for test 21

```

//RACFTS21 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/* RMM OPTIONS:
/* TPRACF(N)
/* RACF(N)
/*
/*
/* DEVSUP
/* TAPEAUTHDSN=YES

```

```

//*      TAPEAUTHF1=YES
//*      TAPEAUTHRC4=FAIL
//*      TAPEAUTHRC8=FAIL
//*
//*      RACF
//*      MHLRES1.**      ACC(ALTER)
//*      MHLRES7.**      ACC(ALTER)
//*      TAPEVOL          INCATIVE
//*      TAPEDSN          INCATIVE
//*
//*      FUNCTION
//*      CREATE TWO NEW DATA SETS
//*      MHLRES1.RACF.TEST2.FILE1
//*      MHLRES7.RACF.TEST2.FILE2
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
    SETR NOCLASSACT(TAPEVOL)
    SETR NOTAPEDSN
    PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
    PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
    SETR REFRESH GENERIC(DATASET)
    LD DATASET('MHLRES1.**') ALL
    LD DATASET('MHLRES7.**') ALL
    SETR LIST
/*
//STEP01  EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES1.RACF.TEST2.FILE1,
//          VOL=(,RETAIN,,,SER=(THM002)),LABEL=(1,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
//*-----*
//STEP02  EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES7.RACF.TEST2.FILE2,
//          VOL=(,RETAIN,,,SER=(THM002)),LABEL=(2,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
//*-----*
//SUBMIT  EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF22)
//SYSUT2  DD  SYSOUT=(A,INTRDR)
//SYSIN   DD  DUMMY

```

Example 13-55 shows you the most important result of the RACF commands.

Example 13-55 test case 21 job output

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----

```

```

00    MHLRES1          NONE          NO          NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
    ALTER      SYS1          NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER    UNIVERSAL ACCESS  WARNING  ERASE
-----
00    MHLRES7          NONE          NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
    ALTER      SYS1          NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

CATEGORIES

```

13.5.22 Test case 22

```

Function:      Red the two previously created tape sets. Program IEBGENER is
                used to read the files. TAPEAUTHDSN and TAPEAUTHF1 are set and
                option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names:  MHLRES1.RACF.TEST2.FILE1
                  MHLRES7.RACF.TEST2.FILE2

```

Result: We got two security violations, because the user have no access to the data set profile MHLRES1.** that protects the first file on the volume. So we could no access any data set on the volume.

Example 13-56 shows you the JCL we have used for test case 22.

Example 13-56 Sample JCL used for test 22

```
//RACFTS22 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=YES
/*      TAPEAUTHF1=YES
/*      TAPEAUTHRC4=FAIL
/*      TAPEAUTHRC8=FAIL
/*
/*      RACF
/*      MHLRES1.**          ACC(NONE)
/*      MHLRES7.**          ACC(ALTER)
/*      TAPEVOL             INCATIVE
/*      TAPEDSN             INCATIVE
/*
/*      FUNCTION
/*      CREATE TWO NEW DATA SETS
/*      MHLRES1.RACF.TEST2.FILE1
/*      MHLRES7.RACF.TEST2.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD  *
SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES1.RACF.TEST2.FILE1,
//          VOL=(,RETAIN,,,SER=(THM002)),LABEL=(1,SL)
//SYSUT2 DD  DUMMY
//SYSIN DD  DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES7.RACF.TEST2.FILE2,
//          VOL=(,RETAIN,,,SER=(THM002)),LABEL=(2,SL)
```

```
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
//*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF23)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY
```

Example 13-57 shows you the most important result of the RACF commands.

Example 13-57 List of the RACF profile in class TAPEVOL

```
LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)
```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

```

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE          SYS1          NON-VSAM

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)
```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

```

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
```

NO SECURITY LEVEL

In the output of the job in Example 13-56 you can see the security violation for the first file we got.

Example 13-58 Test case 9 job output

```
....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1      ) NAME(MARY LOVELACE - RESI)
      MHLRES1.RACF.TEST2.FILE1 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES1.** (G)
      ACCESS INTENT(READ  ) ACCESS ALLOWED(NONE  )
IEC150I 913-38,IFG0194F,RACFTS22,STEP01,SYSUT1,0B23,,MHLRES1.RACF.TEST2.FILE1
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913  REASON CODE=00000038
TIME=16.34.36  SEQ=00892  CPU=0000  ASID=0039
PSW AT TIME OF ERROR 075C1000  80C50EDE  ILC 2  INTC 0D
      NO ACTIVE MODULE FOUND
      NAME=UNKNOWN
      DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
      AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
              2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
              4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
              6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
              8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
              A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
              C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
              E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
      END OF SYMPTOM DUMP
IEF472I RACFTS22 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1      ) NAME(MARY LOVELACE - RESI)
      MHLRES1.RACF.TEST2.FILE1 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES1.** (G)
      ACCESS INTENT(READ  ) ACCESS ALLOWED(NONE  )
IEC150I 913-38,IFG0194F,RACFTS22,STEP02,SYSUT1,0B23,,MHLRES7.RACF.TEST2.FILE2
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913  REASON CODE=00000038
TIME=16.35.22  SEQ=00893  CPU=0000  ASID=0039
PSW AT TIME OF ERROR 075C1000  80C50EDE  ILC 2  INTC 0D
      NO ACTIVE MODULE FOUND
      NAME=UNKNOWN
      DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
      AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
              2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
              4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
              6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
              8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
              A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
              C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
              E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
      END OF SYMPTOM DUMP
```

IEF472I RACFTS22 STEP02 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....

13.5.23 Test case 23

Function: Red the two previously created tape sets. Program IEBGENER is used to read the files. TAPEAUTHDSN and TAPEAUTHF1 are set and option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES1.RACF.TEST2.FILE1
MHLRES7.RACF.TEST2.FILE2

Result: Although we have access to the first file on the volume we got a security violation, because the user have no access to the data set profile MHLRES7.** to read the second file.

Example 13-59 shows you the JCL we have used for test case 22.

Example 13-59 Sample JCL used for test 23

```
//RACFTS23 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=YES
/*      TAPEAUTHF1=YES
/*      TAPEAUTHRC4=FAIL
/*      TAPEAUTHRC8=FAIL
/*
/*      RACF
/*      MHLRES1.**          ACC(ALTER)
/*      MHLRES7.**          ACC(NONE)
/*      TAPEVOL             INCATIVE
/*      TAPEDSN             INCATIVE
/*
/*      FUNCTION
/*      CREATE TWO NEW DATA SETS
/*      MHLRES1.RACF.TEST2.FILE1
/*      MHLRES7.RACF.TEST2.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
  SETR NOCLASSACT(TAPEVOL)
  SETR NOTAPEDSN
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
```



```

//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.RACF.TEST2.FILE1,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(1,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST2.FILE2,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF24)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-60 shows you the most important result of the RACF commands.

Example 13-60 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER        SYS1          NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES7      NONE             NO        NO

```

```

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

In the output of the job in Example 13-61 you can see the security violation we got for the second file.

Example 13-61 Test case 9 job output

```

...
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
      MHLRES7.RACF.TEST2.FILE2 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES7.** (G)
      ACCESS INTENT(READ ) ACCESS ALLOWED(NONE )
IEC150I 913-38,IFG0194F,RACFTS23,STEP02,SYSUT1,0B23,,MHLRES7.RACF.TEST2.FILE2
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000038
TIME=16.36.09 SEQ=00894 CPU=0000 ASID=002C
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
      2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
      6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
      8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
      A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
      E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
IEF472I RACFTS23 STEP02 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....

```

13.5.24 Test case 24

Function: Red the two previously created tape sets. Program IEBGENER is used to read the files. TAPEAUTHDSN and TAPEAUTHF1 are set and option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES1.RACF.TEST2.FILE1
MHLRES7.RACF.TEST2.FILE2

Result: We got two security violations, because the user have no access to the both data set profiles MHLRES1.** and MHLRES7.**.

Example 13-62 shows you the JCL we have used for test case 22.

Example 13-62 Sample JCL used for test 24

```
//RACFTS24 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=YES
/*      TAPEAUTHF1=YES
/*      TAPEAUTHRC4=FAIL
/*      TAPEAUTHRC8=FAIL
/*
/*      RACF
/*      MHLRES1.**          ACC(NONE)
/*      MHLRES7.**          ACC(NONE)
/*      TAPEVOL             INCATIVE
/*      TAPEDSN             INCATIVE
/*
/*      FUNCTION
/*      CREATE TWO NEW DATA SETS
/*      MHLRES1.RACF.TEST2.FILE1
/*      MHLRES7.RACF.TEST2.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD  *
SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES1.RACF.TEST2.FILE1,
//          VOL=(,RETAIN,,,SER=(THM002)),LABEL=(1,SL)
//SYSUT2 DD  DUMMY
```

```
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST2.FILE2,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF25)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY
```

Example 13-63 shows the most important result of the RACF commands.

Example 13-63 Result of the RACF commands

```
LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)
```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

```
AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE           SYS1           NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)
```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

```
AUDITING
-----
FAILURES(READ)

NOTIFY
```

```

-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE           SYS1           NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

```

In the output of the job in Example 13-64 you can see that we got two security violations. One for file 1 and one for file 2.

Example 13-64 Test case 24 job output

```

....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1      ) NAME(MARY LOVELACE - RESI)
      MHLRES1.RACF.TEST2.FILE1 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES1.** (G)
      ACCESS INTENT(READ  ) ACCESS ALLOWED(NONE  )
IEC150I 913-38,IFG0194F,RACFTS24,STEP01,SYSUT1,0B23,,MHLRES1.RACF.TEST2.FILE1
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913  REASON CODE=00000038
TIME=10.40.55  SEQ=01052  CPU=0000  ASID=002C
PSW AT TIME OF ERROR 075C1000  80C50EDE  ILC 2  INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_0A913000
      2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
      6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
      8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
      A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
      E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
IEF472I RACFTS24 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1      ) NAME(MARY LOVELACE - RESI)
      MHLRES7.RACF.TEST2.FILE2 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES7.** (G)
      ACCESS INTENT(READ  ) ACCESS ALLOWED(NONE  )
IEC150I 913-38,IFG0194F,RACFTS24,STEP02,SYSUT1,0B23,,MHLRES7.RACF.TEST2.FILE2
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913  REASON CODE=00000038
TIME=10.41.42  SEQ=01053  CPU=0000  ASID=002C
PSW AT TIME OF ERROR 075C1000  80C50EDE  ILC 2  INTC 0D
NO ACTIVE MODULE FOUND

```

```

NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
      2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
      6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
      8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
      A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
      E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
....

```

13.5.25 Test case 25

Function: Create an additional third new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library. TAPEAUTHDSN and TAPEAUTHF1 are set and option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES7.RACF.TEST2.FILE3

Result: The job ended without any errors , because we have access to both data set profiles protecting the first file on the volume and the new file we are creating.

Example 13-65 shows you the JCL and the settings we have used to create the third data set.

Example 13-65 Sample JCL used for test 25

```

//RACFTS25 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(N)
//*      RACF(N)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=YES
//*      TAPEAUTHF1=YES
//*      TAPEAUTHRC4=FAIL
//*      TAPEAUTHRC8=FAIL
//*
//*      RACF
//*      MHLRES1.**          ACC(ALTER)
//*      MHLRES7.**          ACC(ALTER)
//*      TAPEVOL             INCATIVE
//*      TAPEDSN             INCATIVE
//*
//*      FUNCTION
//*      CREATE AN ADDITIONAL NEW DATA SET
//*      MHLRES7.RACF.TEST2.FILE3
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *

```

```

SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
PE 'MHLRES5.**' ACC(ALTER) ID(MHLRES5)
PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
DELETE MHLRES7.RACF.TEST2.FILE3 NONVSAM NOSCRATCH
SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD LABEL=(03,SL),DSN=MHLRES7.RACF.TEST2.FILE3,
// DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
// VOLUME=SER=THM002
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF26)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-66 shows you the most important result of the RACF commands.

Example 13-66 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER        SYS1              NON-VSAM

NO INSTALLATION DATA

```

```

                                SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----  -
  00    MHLRES7          NONE           NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----  -
  ALTER      SYS1           NON-VSAM

NO INSTALLATION DATA

                                SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.26 Test case 26

Function: Create an additional fourth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library. TAPEAUTHDSN and TAPEAUTHF1 are set and option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES7.RACF.TEST2.FILE4

Result: We got a security violation and the file is not created, because we have no access to the RACF data set profile MHLRES1.** that protects the first file on the volume

Example 13-67 shows you the JCL we have used for test case 26.

Example 13-67 Sample JCL used for test 9

```

//RACFTS26 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(N)
//*      RACF(N)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=YES
//*      TAPEAUTHF1=YES

```



```

//*      TAPEAUTHRC4=FAIL
//*      TAPEAUTHRC8=FAIL
//*
//*      RACF
//*      MHLRES1.**      ACC(NONE)
//*      MHLRES7.**      ACC(ALTER)
//*      TAPEVOL          INCATIVE
//*      TAPEDSN          INCATIVE
//*
//*      FUNCTION
//*      CREATE AN ADDITIONAL NEW DATA SET
//*      MHLRES7.RACF.TEST2.FILE4
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR NOCLASSACT(TAPEVOL)
  SETR NOTAPEDSN
  PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN  DD  *
  DELETE MHLRES7.RACF.TEST2.FILE4 NONVSAM NOSCRATCH
  SET MAXCC=0
/*
//*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD  LABEL=(04,SL),DSN=MHLRES7.RACF.TEST2.FILE4,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=SER=THM002
//SYSIN  DD  DUMMY
//*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF27)
//SYSUT2 DD  SYSOUT=(A,INTRDR)
//SYSIN  DD  DUMMY

```

Example 13-68 shows the most important result of the RACF commands.

Example 13-68 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-----	-----	-----	-----	-----
00	MHLRES1	NONE	NO	NO

```

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
   00   MHLRES7          NONE          NO      NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

In the output of the job in Example 13-69 you can see that we got a security violation toalthough the data set profile that protects the first file on the volume.

Example 13-69 Test case 26 job output

```

.....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
      MHLRES1.RACF.TEST2.FILE1 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES1.** (G)
      ACCESS INTENT(UPDATE ) ACCESS ALLOWED(NONE )

```

```

IEC150I 913-38,IFG0194F,RACFTS26,STEP01,SYSUT2,0B23,,MHLRES7.RACF.TEST2.FILE4
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000038
TIME=16.39.27 SEQ=00897 CPU=0000 ASID=0039
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
      2: 00000000/00000000_0000C3E4 3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B51F8 5: 00000000/00000000_007B558C
      6: 00000000/00000000_007B5534 7: 00000000/00000000_007B558C
      8: 00000000/00000000_007B5554 9: 00000000/00000000_007B3328
      A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B54B8
      E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
IEF472I RACFTS26 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....

```

13.5.27 Test case 27

Function: Recreate an additional fourth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library. TAPEAUTHDSN and TAPEAUTHF1 are set and option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES7.RACF.TEST2.FILE4

Result: We got a security violation and the file is not created, because we have no access to the RACF data set profile MHLRES7.** that protects the data set we would like to create.

Example 13-70 shows you the JCL we have used for test case 26.

Example 13-70 Sample JCL used for test 9

```

//RACFTS27 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//* RMM OPTIONS:
//* TPRACF(N)
//* RACF(N)
//*
//* DEVSUP
//* TAPEAUTHDSN=YES
//* TAPEAUTHF1=YES
//* TAPEAUTHRC4=FAIL
//* TAPEAUTHRC8=FAIL
//*
//* RACF
//* MHLRES1.** ACC(ALTER)
//* MHLRES7.** ACC(NONE)
//* TAPEVOL INCATIVE
//* TAPEDSN INCATIVE
//*

```

```

/*      FUNCTION
/*      CREATE AN ADDITIONAL NEW DATA SET
/*      MHLRES7.RACF.TEST2.FILE4
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
    SETR NOCLASSACT(TAPEVOL)
    SETR NOTAPEDSN
    PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
    PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
    SETR REFRESH GENERIC(DATASET)
    LD DATASET('MHLRES1.**') ALL
    LD DATASET('MHLRES7.**') ALL
    SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN    DD  *
    DELETE MHLRES7.RACF.TEST2.FILE4 NONVSAM NOSCRATCH
    SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2  DD  LABEL=(04,SL),DSN=MHLRES7.RACF.TEST2.FILE4,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=SER=THM002
//SYSIN    DD  DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF28)
//SYSUT2  DD  SYSOUT=(A,INTRDR)
//SYSIN    DD  DUMMY

```

Example 13-71 shows you the most important result of the RACF commands.

Example 13-71 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
NONE	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-72 you can see that there was no security violation reading the first file, but we was not able to read the second file because we have no access to the profile that protects this file.

Example 13-72 Test case 27 job output

```

.....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1      ) NAME(MARY LOVELACE - RESI)
      MHLRES7.RACF.TEST2.FILE4 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES7.** (G)
      ACCESS INTENT(UPDATE ) ACCESS ALLOWED(NONE      )
IEC150I 913-38,IFG0194F,RACFTS27,STEP01,SYSUT2,0B23,,MHLRES7.RACF.TEST2.FILE4
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913  REASON CODE=00000038
TIME=16.40.27  SEQ=00898  CPU=0000  ASID=002C
PSW AT TIME OF ERROR 075C1000  80C50EDE  ILC 2  INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0

```

```

AR/GR 0: A3280FB6/00000000_00C511A4    1: 00000000/00000000_A4913000
      2: 00000000/00000000_0000C3E4    3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B51F8    5: 00000000/00000000_007B558C
      6: 00000000/00000000_007B5534    7: 00000000/00000000_007B558C
      8: 00000000/00000000_007B5554    9: 00000000/00000000_007B3328
      A: 00000000/00000000_007CE390    B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C    D: 00000000/00000000_007B54B8
      E: 00000000/00000000_80C50762    F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
IEF472I RACFTS27 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....

```

13.5.28 Test case 28

Function: Rereate an additional fourth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library. TAPEAUTHDSN and TAPEAUTHF1 are set and option for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES7.RACF.TEST2.FILE4

Result: We got a security violation, because we have no access to the RACF data set profile MHLRES1.** that protects the first file on the volume nor to the profile MHLRES7.** that protects the data set we would like to create it.

Example 13-73 shows you the JCL we have used for test case 28.

Example 13-73 Sample JCL used for test 28

```

//RACFTS28 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(N)
//*      RACF(N)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=YES
//*      TAPEAUTHF1=YES
//*      TAPEAUTHRC4=FAIL
//*      TAPEAUTHRC8=FAIL
//*
//*      RACF
//*      MHLRES1.**          ACC(NONE)
//*      MHLRES7.**          ACC(NONE)
//*      TAPEVOL             INCATIVE
//*      TAPEDSN             INCATIVE
//*
//*      FUNCTION
//*      CREATE AN ADDITIONAL NEW DATA SET
//*      MHLRES7.RACF.TEST2.FILE4
//*
//RACFCMD5 EXEC PGM=IKJEFT01
//SYSTSPRT DD   SYSOUT=*
//SYSTSIN  DD   *

```

```

SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
DELETE MHLRES7.RACF.TEST2.FILE4 NONVSAM NOSCRATCH
SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD LABEL=(04,SL),DSN=MHLRES7.RACF.TEST2.FILE4,
// DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
// VOLUME=SER=THM002
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF29)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-74 shows the most important result of the RACF commands.

Example 13-74 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
NONE         SYS1             NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

```

```

-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----  -
00     MHLRES7      NONE             NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE      SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

In the output of the job in Example 13-75 you can see that there was not able to create the new data set.

Example 13-75 Test case 28 job output

```

.....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
      MHLRES7.RACF.TEST2.FILE4 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES7.** (G)
      ACCESS INTENT(UPDATE ) ACCESS ALLOWED(NONE )
IEC150I 913-38,IFG0194F,RACFTS28,STEP01,SYSUT2,0B23,,MHLRES7.RACF.TEST2.FILE4
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000038
TIME=16.41.27 SEQ=00899 CPU=0000 ASID=002C
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
      2: 00000000/00000000_0000C3E4 3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B51F8 5: 00000000/00000000_007B558C
      6: 00000000/00000000_007B5534 7: 00000000/00000000_007B558C
      8: 00000000/00000000_007B5554 9: 00000000/00000000_007B3328
      A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B54B8
      E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038

```



```
END OF SYMPTOM DUMP
IEF472I RACFTS28 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....
```

13.5.29 Test case 29

Function: Red the two previously created tape sets. Program IEBGENER is used to read the files. TAPEAUTHDSN is set to YES but TAPEAUTHF1 is set to NO. The options for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES1.RACF.TEST2.FILE1
MHLRES7.RACF.TEST2.FILE2

Result: The job ended without any errors, because the user have access to both data set profiles protecting this files.

Example 13-76 shows you the JCL we have used for test case 29.

Example 13-76 Sample JCL used for test 29

```
//RACFTS29 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=YES
/*      TAPEAUTHF1=YES
/*      TAPEAUTHRC4=FAIL
/*      TAPEAUTHRC8=FAIL
/*
/*      RACF
/*      MHLRES1.**          ACC(ALTER)
/*      MHLRES7.***        ACC(ALTER)
/*      TAPEVOL             INCATIVE
/*      TAPEDSN             INCATIVE
/*
/*      FUNCTION
/*      Read the two DATA SETS
/*      MHLRES1.RACF.TEST2.FILE1
/*      MHLRES7.RACF.TEST2.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
```

```

/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.RACF.TEST2.FILE1,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(1,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST2.FILE2,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF30)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-77 shows the most important result of the RACF commands.

Example 13-77 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER        SYS1           NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES7      NONE             NO        NO

```

```

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.30 Test case 30

Function: Red the two previously created tape sets. Program IEBGENER is used to read the files. TAPEAUTHDSN is set to YES but TAPEAUTHF1 is set to NO. The options for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES1.RACF.TEST2.FILE1
MHLRES7.RACF.TEST2.FILE2

Result: We got a security violation, because the user have no access to the data set profile MHLRES1.**. The second file can be red without any error.

Example 13-78 shows you the JCL we have used for test case 30.

Example 13-78 Sample JCL used for test 30

```

//RACFTS30 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*   RMM OPTIONS:
//*   TPRACF(N)
//*   RACF(N)
//*
//*   DEVSUP
//*   TAPEAUTHDSN=YES
//*   TAPEAUTHF1=YES
//*   TAPEAUTHRC4=FAIL
//*   TAPEAUTHRC8=FAIL
//*
//*   RACF
//*   MHLRES1.**          ACC(NONE)
//*   MHLRES7.**          ACC(ALTER)
//*   TAPEVOL             INCATIVE
//*   TAPEDSN             INCATIVE
//*

```

```

/*      FUNCTION
/*      CREATE TWO NEW DATA SETS
/*      MHLRES1.RACF.TEST2.FILE1
/*      MHLRES7.RACF.TEST2.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
    SETR NOCLASSACT(TAPEVOL)
    SETR NOTAPEDSN
    PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
    PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
    SETR REFRESH GENERIC(DATASET)
    LD DATASET('MHLRES1.**') ALL
    LD DATASET('MHLRES7.**') ALL
    SETR LIST
/*
//STEP01  EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES1.RACF.TEST2.FILE1,
//          VOL=(,RETAIN,,,SER=(THM002)),LABEL=(1,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//STEP02  EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES7.RACF.TEST2.FILE2,
//          VOL=(,RETAIN,,,SER=(THM002)),LABEL=(2,SL)
//SYSUT2  DD  DUMMY
//SYSIN   DD  DUMMY
/*-----*
//SUBMIT  EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1  DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF31)
//SYSUT2  DD  SYSOUT=(A,INTRDR)
//SYSIN   DD  DUMMY

```

Example 13-79 shows the most important result of the RACF commands.

Example 13-79 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

```

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES1	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
NONE	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-----	-----	-----	-----	-----
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
ALTER	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-80 you can see that there was a security violation for the first file.

Example 13-80 Test case 30 job output

```

.....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
      MHLRES1.RACF.TEST2.FILE1 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES1.** (G)
      ACCESS INTENT(READ ) ACCESS ALLOWED(NONE )
IEC150I 913-38,IFG0194F,RACFTS30,STEP01,SYSUT1,0B23,,MHLRES1.RACF.TEST2.FILE1
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000038
TIME=16.47.20 SEQ=00901 CPU=0000 ASID=0039
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000

```

```

2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
IEF472I RACFTS30 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....

```

13.5.31 Test case 31

Function: Red the two previously created tape sets. Program IEBGENER is used to read the files. TAPEAUTHDSN is set to YES but TAPEAUTHF1 is set to NO. The options for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES1.RACF.TEST2.FILE1
MHLRES7.RACF.TEST2.FILE2

Result: We got a security violation, because the user have no access to the data set profile MHLRES7.**. The first file can be red without any error.

Example 13-81 shows you the JCL we have used for test case 31.

Example 13-81 Sample JCL used for test 9

```

//RACFTS31 JOB (999,P0K),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//* RMM OPTIONS:
//* TPRACF(N)
//* RACF(N)
//*
//* DEVSUP
//* TAPEAUTHDSN=YES
//* TAPEAUTHF1=YES
//* TAPEAUTHRC4=FAIL
//* TAPEAUTHRC8=FAIL
//*
//* RACF
//* MHLRES1.** ACC(ALTER)
//* MHLRES7.** ACC(NONE)
//* TAPEVOL INCATIVE
//* TAPEDSN INCATIVE
//*
//* FUNCTION
//* CREATE TWO NEW DATA SETS
//* MHLRES1.RACF.TEST2.FILE1
//* MHLRES7.RACF.TEST2.FILE2
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *

```

```

SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.RACF.TEST2.FILE1,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(1,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST2.FILE2,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF32)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-82 shows the most important result of the RACF commands.

Example 13-82 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER        SYS1             NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL
-----

```

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
NONE	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-83 you can see that we got a security violation for data set MHLRES7.RACF.TEST2.FILE2.

Example 13-83 Test case 31 job output

```
.....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
MHLRES7.RACF.TEST2.FILE2 CL(DATASET ) VOL(THM002)
INSUFFICIENT ACCESS AUTHORITY
FROM MHLRES7.** (G)
ACCESS INTENT(READ ) ACCESS ALLOWED(NONE )
IEC150I 913-38,IFG0194F,RACFTS31,STEP02,SYSUT1,0B23,,MHLRES7.RACF.TEST2.FILE2
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000038
TIME=16.49.21 SEQ=00902 CPU=0000 ASID=002C
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
```


IEF472I RACFTS31 STEP02 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....

13.5.32 Test case 32

Function: Red the two previously created tape sets. Program IEBGENER is used to read the files. TAPEAUTHDSN is set to YES but TAPEAUTHF1 is set to NO. The options for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES1.RACF.TEST2.FILE1
MHLRES7.RACF.TEST2.FILE2

Result: We got a security violation, because the user have no access to both data set profiles MHLRES1.** and MHLRES7.**. No file can be red.

Example 13-84 shows you the JCL we have used for test case 31.

Example 13-84 Sample JCL used for test 32

```
//RACFTS32 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*      DEVSUP
/*      TAPEAUTHDSN=YES
/*      TAPEAUTHF1=YES
/*      TAPEAUTHRC4=FAIL
/*      TAPEAUTHRC8=FAIL
/*
/*      RACF
/*      MHLRES1.**          ACC(NONE)
/*      MHLRES7.**          ACC(NONE)
/*      TAPEVOL             INCATIVE
/*      TAPEDSN             INCATIVE
/*
/*      FUNCTION
/*      CREATE TWO NEW DATA SETS
/*      MHLRES1.RACF.TEST2.FILE1
/*      MHLRES7.RACF.TEST2.FILE2
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
  SETR NOCLASSACT(TAPEVOL)
  SETR NOTAPEDSN
  PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
```

```

//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES1.RACF.TEST2.FILE1,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(1,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*
//STEP02 EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES7.RACF.TEST2.FILE2,
// VOL=(,RETAIN,,,SER=(THM002)),LABEL=(2,SL)
//SYSUT2 DD DUMMY
//SYSIN DD DUMMY
/*-----*

```

Example 13-85 shows you the most important result of the RACF commands.

Example 13-85 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE      SYS1             NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES7      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY

```

```

-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE           SYS1           NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

```

In the output of the job in Example 13-86 you can see that we got a security violation for both data set profiles protecting this files.

Example 13-86 Test case 9 job output

```

....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
      MHLRES1.RACF.TEST2.FILE1 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES1.** (G)
      ACCESS INTENT(READ ) ACCESS ALLOWED(NONE )
IEC150I 913-38,IFG0194F,RACFTS32,STEP01,SYSUT1,0B23,,MHLRES1.RACF.TEST2.FILE1
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000038
TIME=11.52.12 SEQ=01054 CPU=0000 ASID=002C
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_0A913000
      2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
      6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
      8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
      A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
      E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
IEF472I RACFTS32 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1 ) NAME(MARY LOVELACE - RESI)
      MHLRES7.RACF.TEST2.FILE2 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES7.** (G)
      ACCESS INTENT(READ ) ACCESS ALLOWED(NONE )
IEC150I 913-38,IFG0194F,RACFTS32,STEP02,SYSUT1,0B23,,MHLRES7.RACF.TEST2.FILE2
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000038
TIME=11.53.03 SEQ=01055 CPU=0000 ASID=002C
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND

```

```

NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
      2: 00000000/00000000_0000C380 3: 00000000/00000000_00C5062A
      4: 00000000/00000000_007B4410 5: 00000000/00000000_007B47A4
      6: 00000000/00000000_007B474C 7: 00000000/00000000_007B47A4
      8: 00000000/00000000_007B476C 9: 00000000/00000000_007B56B8
      A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
      C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B46D0
      E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
IEF472I RACFTS32 STEP02 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....

```

13.5.33 Test case 33

Function: Create an additional forth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library. TAPEAUTHDSN is set to YES but TAPEAUTHF1 is set to NO. The options for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES7.RACF.TEST2.FILE5

Result: The job ended without any errors , because we have access to both data set profiles protecting the first file on the volume and the new file we are creating.

Example 13-87 shows you the JCL we have used for test case 33.

Example 13-87 Sample JCL used for test 33

```

//RACFTS33 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//* RMM OPTIONS:
//* TPRACF(N)
//* RACF(N)
//*
//* DEVSUP
//* TAPEAUTHDSN=YES
//* TAPEAUTHF1=YES
//* TAPEAUTHRC4=FAIL
//* TAPEAUTHRC8=FAIL
//*
//* RACF
//* MHLRES1.** ACC(ALTER)
//* MHLRES7.** ACC(ALTER)
//* TAPEVOL INCATIVE
//* TAPEDSN INCATIVE
//*
//* FUNCTION
//* CREATE AN ADDITIONAL NEW DATA SET
//* MHLRES7.RACF.TEST2.FILE4
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*

```

```

//SYSTSIN DD *
  SETR NOCLASSACT(TAPEVOL)
  SETR NOTAPEDSN
  PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES5.**' ACC(ALTER) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
  DELETE MHLRES7.RACF.TEST2.FILE4 NONVSAM NOSCRATCH
  SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD LABEL=(04,SL),DSN=MHLRES7.RACF.TEST2.FILE4,
//        DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//        VOLUME=SER=THM002
//SYSIN DD DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF34)
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY

```

Example 13-88 shows you the most important result of the RACF commands.

Example 13-88 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER       SYS1              NON-VSAM

NO INSTALLATION DATA

```

```

                                SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----  -
00     MHLRES7      NONE              NO       NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER        SYS1          NON-VSAM

NO INSTALLATION DATA

                                SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.34 Test case 34

Function: Create an additional fifth new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library. TAPEAUTHDSN is set to YES but TAPEAUTHF1 is set to NO. The options for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES7.RACF.TEST2.FILE5

Result: The job ended without any errors, though we have no access to the first file on the volume. This is not checked, because the option TAPEAUTHF1 is not set to yes.

Example 13-89 shows you the JCL we have used for test case 33.

Example 13-89 Sample JCL used for test 34

```

//RACFTS34 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
//*JOBPARM SYSAFF=SC70
//*
//*      RMM OPTIONS:
//*      TPRACF(N)
//*      RACF(N)
//*
//*      DEVSUP
//*      TAPEAUTHDSN=YES

```

```

//*      TAPEAUTHF1=YES
//*      TAPEAUTHRC4=FAIL
//*      TAPEAUTHRC8=FAIL
//*
//*      RACF
//*      MHLRES1.**      ACC(NONE)
//*      MHLRES7.**      ACC(ALTER)
//*      TAPEVOL          INCATIVE
//*      TAPEDSN          INCATIVE
//*
//*      FUNCTION
//*      CREATE AN ADDITIONAL NEW DATA SET
//*      MHLRES7.RACF.TEST2.FILE5
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
  SETR NOCLASSACT(TAPEVOL)
  SETR NOTAPEDSN
  PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(ALTER) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN  DD  *
  DELETE MHLRES7.RACF.TEST2.FILE5 NONVSAM NOSCRATCH
  SET MAXCC=0
/*
//*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD  LABEL=(05,SL),DSN=MHLRES7.RACF.TEST2.FILE5,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=SER=THM002
//SYSIN  DD  DUMMY
//*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF35)
//SYSUT2 DD  SYSOUT=(A,INTRDR)
//SYSIN  DD  DUMMY

```

Example 13-90 shows you the most important result of the RACF commands.

Example 13-90 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----

```

```

00      MHLRES1          NONE          NO          NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      NONE          SYS1          NON-VSAM

NO INSTALLATION DATA

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00      MHLRES7          NONE          NO          NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
      ALTER          SYS1          NON-VSAM

NO INSTALLATION DATA

                        SECURITY LEVEL
-----
NO SECURITY LEVEL

```

13.5.35 Test case 35

Function: Create an additional sixh new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library. TAPEAUTHDSN is set to YES but TAPEAUTHF1 is set to NO. The options for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.

Data set names: MHLRES7.RACF.TEST2.FILE5

Result: We got a security violation and the file is not created, because we have no access to the RACF data set profile MHLRES7.** that protects the data set we would like to create it.

Example 13-91 on page 607 shows you the JCL we have used for test case 33.

Example 13-91 Sample JCL used for test 35

```
//RACFTS35 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
/*      RACF(N)
/*
/*
/*      DEVSUP
/*      TAPEAUTHDSN=YES
/*      TAPEAUTHF1=YES
/*      TAPEAUTHRC4=FAIL
/*      TAPEAUTHRC8=FAIL
/*
/*      RACF
/*      MHLRES1.**          ACC(ALTER)
/*      MHLRES7.**          ACC(NONE)
/*      TAPEVOL             INCATIVE
/*      TAPEDSN             INCATIVE
/*
/*      FUNCTION
/*      CREATE AN ADDITIONAL NEW DATA SET
/*      MHLRES7.RACF.TEST2.FILE6
/*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN  DD  *
SETR NOCLASSACT(TAPEVOL)
SETR NOTAPEDSN
PE 'MHLRES1.**' ACC(ALTER) ID(MHLRES5)
PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
SETR REFRESH GENERIC(DATASET)
LD DATASET('MHLRES1.**') ALL
LD DATASET('MHLRES7.**') ALL
SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN    DD  *
DELETE MHLRES7.RACF.TEST2.FILE6 NONVSAM NOSCRATCH
SET MAXCC=0
/*
/*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD  LABEL=(06,SL),DSN=MHLRES7.RACF.TEST2.FILE6,
//          DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//          VOLUME=SER=THM002
//SYSIN    DD  DUMMY
/*-----*
//SUBMIT EXEC PGM=IEBGENER,COND=EVEN
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD  DISP=SHR,DSN=MHLRES5.RACF.CNTL(RACF36)
```

```
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSIN DD DUMMY
```

Example 13-92 shows you the most important result of the RACF commands.

Example 13-92 Result of the RACF commands

```
LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES1      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
ALTER        SYS1           NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----
00     MHLRES7      NONE             NO        NO

AUDITING
-----
FAILURES(READ)

NOTIFY
-----
NO USER TO BE NOTIFIED

YOUR ACCESS  CREATION GROUP  DATASET TYPE
-----
NONE         SYS1           NON-VSAM

NO INSTALLATION DATA

                SECURITY LEVEL
-----
```

NO SECURITY LEVEL

In the output of the job in Example 13-93 you can see that we got a security violation to data set profile MHLRES7.**.

Example 13-93 Test case 35 job output

```
....
IEF237I DMY  ALLOCATED TO SYSIN
ICH408I USER(MHLRES5 ) GROUP(SYS1    ) NAME(MARY LOVELACE - RESI)
      MHLRES7.RACF.TEST2.FILE6 CL(DATASET ) VOL(THM002)
      INSUFFICIENT ACCESS AUTHORITY
      FROM MHLRES7.** (G)
      ACCESS INTENT(UPDATE ) ACCESS ALLOWED(NONE )
IEC150I 913-38,IFG0194F,RACFTS35,STEP01,SYSUT2,0B23,,MHLRES7.RACF.TEST2.FILE6
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913  REASON CODE=00000038
TIME=17.20.50  SEQ=00905  CPU=0000  ASID=002C
PSW AT TIME OF ERROR 075C1000  80C50EDE  ILC 2  INTC 0D
      NO ACTIVE MODULE FOUND
      NAME=UNKNOWN
      DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
      AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
          2: 00000000/00000000_0000C3E4 3: 00000000/00000000_00C5062A
          4: 00000000/00000000_007B51F8 5: 00000000/00000000_007B558C
          6: 00000000/00000000_007B5534 7: 00000000/00000000_007B558C
          8: 00000000/00000000_007B5554 9: 00000000/00000000_007B3328
          A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
          C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B54B8
          E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
      END OF SYMPTOM DUMP
IEF472I RACFTS35 STEP01 - COMPLETION CODE - SYSTEM=913 USER=0000 REASON=00000038
....
```

13.5.36 Test case 36

Function:	Recreate the additional sixh new tape sets on the previous used tape volume. Program IEBGENER is used to copy a member of a library. TAPEAUTHDSN is set to YES but TAPEAUTHF1 is set to NO. The options for TAPEAUTHRC4 and TAPEAUTHRC8 is set to FAIL.
Data set names:	MHLRES7.RACF.TEST2.FILE5
Result:	We got a security violation and the file is not created, because we have no access to the RACF data set profile MHLRES7.** that protects the data set we would like to create it. The first file protection is not checked, because the TAPEAUTHF1 checking is not set

Example 13-94 shows you the JCL we have used for test case 36.

Example 13-94 Sample JCL used for test 36

```
//RACFTS36 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID
/*JOBPARM SYSAFF=SC70
/*
/*      RMM OPTIONS:
/*      TPRACF(N)
```

```

//*          RACF(N)
//*
//*
//*          DEVSUP
//*          TAPEAUTHDSN=YES
//*          TAPEAUTHF1=YES
//*          TAPEAUTHRC4=FAIL
//*          TAPEAUTHRC8=FAIL
//*
//*          RACF
//*          MHLRES1.**          ACC(NONE)
//*          MHLRES7.**          ACC(NONE)
//*          TAPEVOL             INCATIVE
//*          TAPEDSN             INCATIVE
//*
//*          FUNCTION
//*          CREATE AN ADDITIONAL NEW DATA SET
//*          MHLRES7.RACF.TEST2.FILE6
//*
//RACFCMDS EXEC PGM=IKJEFT01
//SYSTSPRT DD  SYSOUT=*
//SYSTSIN DD   *
  SETR NOCLASSACT(TAPEVOL)
  SETR NOTAPEDSN
  PE 'MHLRES1.**' ACC(NONE) ID(MHLRES5)
  PE 'MHLRES7.**' ACC(NONE) ID(MHLRES5)
  SETR REFRESH GENERIC(DATASET)
  LD DATASET('MHLRES1.**') ALL
  LD DATASET('MHLRES7.**') ALL
  SETR LIST
/*
//CLEANUP EXEC PGM=IDCAMS
//SYSPRINT DD  SYSOUT=*
//SYSIN DD     *
  DELETE MHLRES7.RACF.TEST2.FILE6 NONVSAM NOSCRATCH
  SET MAXCC=0
/*
//*-----*
//STEP01 EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=*
//SYSUT1 DD   DISP=SHR,DSN=MHLRES5.RACF.CNTL(TEXT)
//SYSUT2 DD   LABEL=(06,SL),DSN=MHLRES7.RACF.TEST2.FILE6,
//           DISP=(,CATLG,DELETE),UNIT=ATL3,RETPD=02,
//           VOLUME=SER=THM002
//SYSIN DD   DUMMY
//*-----*

```

Example 13-95 shows you the most important result of the RACF commands.

Example 13-95 Result of the RACF commands

```

LD DATASET('MHLRES1.**') ALL
INFORMATION FOR DATASET MHLRES1.** (G)

LEVEL  OWNER      UNIVERSAL ACCESS  WARNING  ERASE
-----

```

00 MHLRES1 NONE NO NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
NONE	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

LD DATASET('MHLRES7.**') ALL
INFORMATION FOR DATASET MHLRES7.** (G)

LEVEL	OWNER	UNIVERSAL ACCESS	WARNING	ERASE
-----	-----	-----	-----	-----
00	MHLRES7	NONE	NO	NO

AUDITING

FAILURES(READ)

NOTIFY

NO USER TO BE NOTIFIED

YOUR ACCESS	CREATION GROUP	DATASET TYPE
-----	-----	-----
NONE	SYS1	NON-VSAM

NO INSTALLATION DATA

SECURITY LEVEL

NO SECURITY LEVEL

In the output of the job in Example 13-96 you can see that we got a security violation for the data set profile MHLRES7.** only.

Example 13-96 Test case 36 job output

....
IEF237I DMY ALLOCATED TO SYSIN
ICH408I USER(MHLRES5) GROUP(SYS1) NAME(MARY LOVELACE - RESI)
MHLRES7.RACF.TEST2.FILE6 CL(DATASET) VOL(THM002)
INSUFFICIENT ACCESS AUTHORITY
FROM MHLRES7.** (G)

ACCESS INTENT(UPDATE) ACCESS ALLOWED(NONE)
IEC150I 913-38,IFG0194F,RACFTS36,STEP01,SYSUT2,0B23,,MHLRES7.RACF.TEST2.FILE6
IEA995I SYMPTOM DUMP OUTPUT
SYSTEM COMPLETION CODE=913 REASON CODE=00000038
TIME=17.21.52 SEQ=00906 CPU=0000 ASID=002C
PSW AT TIME OF ERROR 075C1000 80C50EDE ILC 2 INTC 0D
NO ACTIVE MODULE FOUND
NAME=UNKNOWN
DATA AT PSW 00C50ED8 - 41003B7A 0A0D41F0 38BE56F0
AR/GR 0: A3280FB6/00000000_00C511A4 1: 00000000/00000000_A4913000
2: 00000000/00000000_0000C3E4 3: 00000000/00000000_00C5062A
4: 00000000/00000000_007B51F8 5: 00000000/00000000_007B558C
6: 00000000/00000000_007B5534 7: 00000000/00000000_007B558C
8: 00000000/00000000_007B5554 9: 00000000/00000000_007B3328
A: 00000000/00000000_007CE390 B: 00000000/00000000_00C53B2C
C: 00000000/00000000_80C53C0C D: 00000000/00000000_007B54B8
E: 00000000/00000000_80C50762 F: 00000002/00000010_00000038
END OF SYMPTOM DUMP
....



Report Generator

This chapter describes how you can use the DFSMSrmm Report Generator DFSMSrmm Report Generator to create customized reports. The DFSMSrmm Report Generator is an ISPF application that you use to create reports to show the status of the resources that DFSMSrmm manages for you. These resources include volumes, data sets, racks, owners, and the retention and movement policies that are established for your installation.

The objectives of this chapter are to:

- ▶ “Report Generator Overview”
- ▶ “Setting up the Report Generator for Your Installation”
- ▶ “Create a Report”

At the end of this chapter you are familiar to use the DFSMSrmm Report Generator.

14.1 Report Generator Overview

The DFSMSrmm report generator is an Interactive System Productivity Facility (ISPF) application that you can use to create reports. The report generator:

- ▶ Provides reports that you can run as-is or that you can modify as you wish. You can use samples to create reports for volumes, data sets, racks, owners, and the retention and movement policies that are established for your installation. You can modify these samples to create tailored reports. DFSMSrmm ships samples in SYS1.SAMPLIB. See “Running a Report Generator Report” on page 7 to run one of these reports.
- ▶ Generates job control language (JCL) that is based on specifications that you use to submit the report jobs. The generation of JCL depends on the report type and therefore the macros that map the data records. The generation knows, based on the macro name and keyword options used, whether to generate a DCOLLECT jobstep, a DFSMSshm FSR reformat, a DFSMSrmm extract, or a copy of SMF records.
- ▶ Includes samples for reporting from DCOLLECT and DFSMSshm data.

The DFSMSrmm Report Generator is updated to support keywords for assembler macros from which report types are derived, and to add new built-in data extract steps. This is in support of new SMF record types from DFSMSrmm, and for DFSMSshm and DCOLLECT reporting.

Any new report type or report definitions created on z/OS V1R10 which include macro keyword information can be used by a lower level release, however the macro keywords are ignored and are removed if the report type or report definition is changed or updated. The original input report type or report definition is unaffected and can be reused later on a supporting release to exploit the macro keywords.

Recommendation: Use z/OS V1R10 or later release to update/customize report types and reports which require macro keywords to be specified. Once the report JCL is generated you can run that JCL on any supported release.

14.2 Setting up the Report Generator for Your Installation

Following the steps have to be used to setting up the DFSMSrmm report generator before you can use them. If you have defined the DFSMSrmm Report Generation settings you can skip this section and go directly to 14.3, “Create a Report” on page 620.

1. Customize the EDGRMAIN EXEC. The default name of the library you can find this REXX Exec SYS1.SEDGEXE1. The REXX variable names that you can customize all start with the characters 'cedggrdl'.
 - a. Define the installation library name and optionally customize the product library name in EXEC EDGRMAIN. There is no installation library name in the EXEC, so you must add the name.
 - b. Update the default naming convention in EXEC EDGRMAIN for the user library name and the JCL library name, if necessary.

Figure 14-1 on page 615 shows you the section of the EDGRMAIN EXEC that you can customize. To allow a shared library for example that is used by all users you must remove the 4 lines that are strikethrough and add the highlighted one. In our example we will use the two libraries:

- RMM.REPORT.LIB

– RMM.REPORT.JCL

```

/* Initialise Report library names */                /*$09A*/
address "ISPEXEC" "VGET ZPREFIX"                    /*$09A*/
If length(zprefix) = 0 then                          /*$10C*/
  edggpref = sysvar('SYSUID')                       /*$09A*/
else                                                  /*$10C*/
  edggpref = zprefix                                /*$10C*/

  edggpref = "RMM"                                   /*NSCH*/

cedggrdlu = "'edggpref!!'.REPORT.LIB'" /* User Library      $10C*/
cedggrdlj = "'edggpref!!'.REPORT.JCL'" /* User JCL Library   $10C*/
cedggrdlp = "'SYS1.SAMPLIB'"          /* Product Library    $10C*/
cedggrdli = ""                        /* Installation Library $10A*/

```

Figure 14-1 Change prefix in EDGRMAIN REXX exec

2. Starting the RMM ISPF dialog from the ISMF PRIMARY OPTION MENU by entering the selection “R” to call the Removable Media Manager as shown in Figure 14-2. The are selection available in the ISMF PRIMARY OPTION MENU depend on the on the “User Mode Selection” that can be changed by seeting up your ISMF profile. Use the selection “O” to change your profile if there is a need.

Panel	Help																														
ISMF PRIMARY OPTION MENU - z/OS DFSMS V1 R10 Enter Selection or Command ==> R Select one of the following options and press Enter: <table> <tr> <td>0</td> <td>ISMF Profile</td> <td>- Change ISMF User Profile</td> </tr> <tr> <td>1</td> <td>Data Set</td> <td>- Perform Functions Against Data Sets</td> </tr> <tr> <td>2</td> <td>Volume</td> <td>- Perform Functions Against Volumes</td> </tr> <tr> <td>3</td> <td>Management Class</td> <td>- Specify Data Set Backup and Migration Criteria</td> </tr> <tr> <td>4</td> <td>Data Class</td> <td>- Specify Data Set Allocation Parameters</td> </tr> <tr> <td>5</td> <td>Storage Class</td> <td>- Specify Data Set Performance and Availability</td> </tr> <tr> <td>9</td> <td>Aggregate Group</td> <td>- Specify Data Set Recovery Parameters</td> </tr> <tr> <td>L</td> <td>List</td> <td>- Perform Functions Against Saved ISMF Lists</td> </tr> <tr> <td>R</td> <td>Removable Media Manager</td> <td>- Perform Functions Against Removable Media</td> </tr> <tr> <td>X</td> <td>Exit</td> <td>- Terminate ISMF</td> </tr> </table> Use HELP Command for Help; Use END Command to Exit.		0	ISMF Profile	- Change ISMF User Profile	1	Data Set	- Perform Functions Against Data Sets	2	Volume	- Perform Functions Against Volumes	3	Management Class	- Specify Data Set Backup and Migration Criteria	4	Data Class	- Specify Data Set Allocation Parameters	5	Storage Class	- Specify Data Set Performance and Availability	9	Aggregate Group	- Specify Data Set Recovery Parameters	L	List	- Perform Functions Against Saved ISMF Lists	R	Removable Media Manager	- Perform Functions Against Removable Media	X	Exit	- Terminate ISMF
0	ISMF Profile	- Change ISMF User Profile																													
1	Data Set	- Perform Functions Against Data Sets																													
2	Volume	- Perform Functions Against Volumes																													
3	Management Class	- Specify Data Set Backup and Migration Criteria																													
4	Data Class	- Specify Data Set Allocation Parameters																													
5	Storage Class	- Specify Data Set Performance and Availability																													
9	Aggregate Group	- Specify Data Set Recovery Parameters																													
L	List	- Perform Functions Against Saved ISMF Lists																													
R	Removable Media Manager	- Perform Functions Against Removable Media																													
X	Exit	- Terminate ISMF																													

Figure 14-2 Selecting the Removable Media Manager on the ISMF Menu Panel

If you an “ End User (EU)” you can skip the next information and go directly to step 4. If you a Storage Administrator (SA) continue with step 2.

3. From the **REMOVABLE MEDIA MANAGER (DFSMSrmm)** ISPF primary menu as shown in Figure 14-3 on page 616 choose 5 to select the **DFSMSrmm Command Menu**. Also you can type REPORT on the option line to go directly to the primary **DFSMSrmm Report Generator** ISPF selection panel. In this case you can continue with step 4.

```

Panel  Help
-----
                REMOVABLE MEDIA MANAGER (DFSMSrmm) - z/OS V1R10
Option ==> 5

0  OPTIONS      - Specify dialog options and defaults
1  USER        - General user facilities
2  LIBRARIAN    - Librarian functions
3  ADMINISTRATOR - Administrator functions
4  SUPPORT      - System support facilities
5  COMMANDS    - Full DFSMSrmm structured dialog
6  LOCAL        - Installation defined dialog
X  EXIT        - Exit DFSMSrmm Dialog

Enter selected option or END command. For more info., enter HELP or PF1.

```

Figure 14-3 Selecting the Options Option on the DFSMSrmm User Menu Panel

4. Select R from the **DFSMSrmm Command Menu** as shown in Figure 14-4 to get the primary **DFSMSrmm Report Generator** ISPF selection panel.

```

Panel  Help
-----
                DFSMSrmm Command Menu - z/OS V1R10
Option ==> R

0  OPTIONS      - Specify dialog options and defaults
1  VOLUME       - Volume commands
2  RACK         - Rack and bin commands
3  DATA SET    - Data set commands
4  OWNER        - Owner commands
5  PRODUCT      - Product commands
6  VRS         - Vital record specifications
7  CONTROL      - Display system control information
R  REPORT      - Report generator

Enter selected option or END command. For more info., enter HELP or PF1.

```

Figure 14-4 Selecting the Options Option on the DFSMSrmm User Menu Panel

5. If you are using the DFSMSrmm Report Generator first time you can set up your user options selecting "0" for **Specify dialog options and defaults** as showing in Figure 14-5.

```

Panel  Help
-----
                DFSMSrmm Report Generator
Option ==> 0

0  OPTIONS      - Specify dialog options and defaults
1  REPORT       - Work with reports
2  REPORT TYPE  - Work with report types
3  REPORTING TOOL - Work with reporting tools

Enter selected option or END command. For more info., enter HELP or PF1.

```

Figure 14-5 Selecting the Options Option on the DFSMSrmm User Menu Panel

6. In the **DFSMSrmm Dialog Options Menu** panel select the option “1” **Specify processing options** first as shown in Figure 14-6.

Panel	Help
DFSMSrmm Dialog Options Menu	
Option ==> 1	
1	USER - Specify processing options
2	SORT - Specify list sort options
3	REPORT - Specify report options
Enter selected option or END command. For more info., enter HELP or PF1.	

Figure 14-6 Selecting the Options Option on the DFSMSrmm User Menu Panel

7. Figure 14-7 shows you the **DFSMSrmm Dialog User Options** panel to specify the JOBCARDS should be used for all jobs generated by DFSMSrmm.

Panel	Help
DFSMSrmm Dialog User Options	
Command ==>	
Date format	JULIAN (American, European, Iso or Julian)
Time zone	LOCAL (zone offsetHH:MM:SS)
Confirm deletes . . .	YES (Yes or No)
Processing option . .	F F - Foreground, B - Background
Eject option	C C - Convenience, B - Bulk
Variable reuse . . .	Y Y - Yes, N - No
Job statement information:-	
====> //SCHLUMXX JOB ,RMM,NOTIFY=&SYSUID,	
====> // MSGCLASS=H,CLASS=0,MSGLEVEL=(1,1),REGION=0M	
====> /* ----- generated by DFSMSrmm -----	
====> /*JOBPARM SYSAFF=SC70	
Enter END command to save changes, or CANCEL to end without saving.	

Figure 14-7 Specifying a jobcard on the DFSMSrmm Dialog User Options Panel

8. After you are back to the **DFSMSrmm Dialog Options Menu** panel select the option 3 **Specify report options** as shown in Figure 14-8 on page 618.

Panel	Help
DFSMSrmm Dialog Options Menu	
Option ==> 3	
1 USER	- Specify processing options
2 SORT	- Specify list sort options
3 REPORT	- Specify report options
Enter selected option or END command. For more info., enter HELP or PF1.	

Figure 14-8 Selecting the report options on the DFSMSrmm Dialog Options Menu Panel

9. In the **DFSMSrmm Report Options** panel you have to specify the installation library that you want to use as your user library as show in Figure 14-9. If you do not allocate the library, DFSMSrmm automatically allocates the library by using a primary space and secondary space of 10 tracks and 50 directory blocks.

Panel	Help
DFSMSrmm Report Options	
Command ==>	
Report definition libraries:	
User	'MHLRES7.REPORT.LIB'
Installation	
Product	'SYS1.SAMPLIB'
User report JCL library . . 'MHLRES7.REPORT.JCL'	
DFSMSrmm allocates user libraries if they do not exist.	

Figure 14-9 Selecting the report options on the DFSMSrmm Dialog Options Menu Panel

Where:

Report Definition Libraries

this fields enable you to specify or change the names of the libraries containing the report definitions, report types and reporting tools.

The format of the report definition library members is fixed length, 80 byte records. You can use partitioned data sets or libraries. The DFSMSrmm Product Library is normally SYS1.SAMPLIB and contains report definitions and samples shipped with the product. Your installation defines the Installation library. You can name a User library. If you do not create the User library, DFSMSrmm automatically creates it for you. The information in the Report Definition Libraries is used in an hierarchical order; first the User library, then the Installation library and finally the Product library. When there are duplicate entries the first occurrence in the hierarchy is used and others are ignored. Report type definitions are stored in the EDGGRTD member of each library and the reporting tool definitions are stored in the EDGGTOOL member.

Possible values:
Any data set name

Default values are the library names defined in the EDGRMAIN exec:

User 'MHLRES7.REPORT.LIB'
Installation . .
Product 'SYS1.SAMPLIB'

User Report JCL Library

this field enables you to specify or change the name of the library containing the report JCL generated by DFSMSrmm. The format of the user report JCL library is fixed length, 80 byte records. You can use partitioned data sets or libraries.

The User Report JCL Library is required for you to generate and submit reports.

You must name a User Report JCL Library. If you do not create the User Report JCL Library, DFSMSrmm automatically creates it for you.

Possible values:
Any data set name

Default value is the library name defined in the EDGRMAIN exec:

'MHLRES7.REPORT.JCL'

Return to the **DFSMSrmm Dialog Options Menu** using one of the following options:

- Enter END command to save changes
- CANCEL to end without saving.

Now you have defined all needed options and you can leave the **DFSMSrmm Dialog Options Menu** to come back to the **DFSMSrmm Report Generator** primary option menu.

Note: Set up the access lists for the libraries. Provide READ authority to the users of the installation libraries and the product libraries.

10. At the end we will show you the current supported tools can by used to create reports. Figure 14-8 on page 618. shows you that today the ICETOOL and SYNCTOOL are the only tools are supported. You can ADD, CHANGE or DELETE a reporting tool at any time.

Panel Help				
DFSMSrmm Reporting Tools			Row 1 to 2 of 2	
Command ==>			Scroll ==> PAGE	
The following line commands are valid: A,C, and D				
S Reporting tool	Exec	Skeleton	Colspace	Group sort

ICETOOL	EDGRGGEN	EDGSGICE	3	U
SYNCTOOL	EDGRGGEN	EDGSGSYN	3	U

Figure 14-10 Selecting the report options on the DFSMSrmm Dialog Options Menu Panel

14.3 Create a Report

1. Select the Report Definition panel to customize report definitions that are shipped with the product. The report definition is a report file that contains all of the information that is needed to run a report. Each report definition in the product library, installation library, or user library contains the report type information, reporting tool information, the data fields that are used in the report, and the sort order of the records. The report selection criteria specify the subset of records that are used for a report. The reporting tool is a REXX EXEC that builds control statements to create reports that use a reporting utility, such as DFSORT's ICETOOL. You can change the reporting tool at any time.

Select "1" from the **DFSMSrmm Report Generator** primary option menu as shown in Figure 14-11 to create a new report or work with existing reports.

Panel	Help
DFSMSrmm Report Generator	
Option ==> 1	
0	OPTIONS - Specify dialog options and defaults
1	REPORT - Work with reports
2	REPORT TYPE - Work with report types
3	REPORTING TOOL - Work with reporting tools
Enter selected option or END command. For more info., enter HELP or PF1.	

Figure 14-11 Selecting the REPORT Option on the DFSMSrmm report Generator menu panel

2. From the **DFSMSrmm Report Definition Search** panel shown in Figure 14-11 you can specify that all existing reports are listed or only reports generated by a specific user id. Also you can specify which library should be searched for. If you do not specify any selection DFSMSrmm will search for all available reports in the User and Product libraries when you press enter.

Panel Help	
DFSMSrmm Report Definition Search	
Command ==>	
Report name . .	May be generic. Leave blank for all reports.
User id	Leave blank for all user ids.
Libraries (enter S):	Select one or more library.
S User	Default are all defined libraries.
Installation	
S Product	
The following line commands will be available when the list is displayed:	
A - Add a new report definition	D - Delete a report definition
G - Generate and save the JCL	J - Edit and manually submit the JCL
L - List macro assembly results	M - Macros for report type are browsed
N - Copy a report definition	S - Display or change the report definition
T - Select a reporting tool	

Figure 14-12 Selecting the Options Option on the DFSMSrmm User Menu Panel

Where:

Report name

Use the Report Name field to search for report definitions by name. The report definition name is the name of a member in one of the report definition libraries (User, Installation or Product). The least generic report name that is used for the search in the Product library is EDGG*.

Possible values:

- 1 to 8 alphanumeric or national characters for a specific report name
- 1 to 7 alphanumeric or national characters, followed by a *, for a generic report name
- * for all report names, which is the default
- The first character must be an alphabetic or national character.

User ID

Use the User ID field to specify the creation or last update user id of report definitions.

Possible values:

- 1 to 8 alphanumeric or national characters for a specific user id
- * for all user ids, which is the default
- The first character must be an alphabetic or national character.

Libraries:

Select one or more of the User, Installation or Product report definition libraries to specify which libraries are searched for report definitions. DFSMSrmm searches the libraries in the hierarchy shown so that duplicate report definition names in different libraries are skipped. If you add any members to the product library, use member names that start with the EDGG prefix.

The default is to search all defined libraries.

- You get a list of all available reports shown in the **DFSMSrmm Report Definitions** panel in form of a table as shown in Figure 14-13. There are now 16 DFSMSHsm related reports available. To show the details for the DCOLLECT BACKUP DATA report use the “S” line command in front of this report.

Panel Help		DFSMSrmm Report Definitions		Row 1 to 23 of 40
Command ==>		Scroll ==> PAGE		
The following line commands are valid: A,D,G,J,L,M,N,S, and T				
S	Name	Report title	Report type	User id
-	-	-	-	-
	ARCGAB01	ABARS ABACKUP Statistics	DFSMSHsm ABARS Report	HSM
	ARCGAR01	ABARS ARECOVER Statistics	DFSMSHsm ABARS Report	HSM
S	ARCGDB01	DCOLLECT BACKUP DATA	DFSMSHsm DCOLLECT BACKUP	HSM
	ARCGDD01	DCOLLECT DASD CAPACITY PLANNIN	DFSMSHsm DCOLLECT DASD CAP	HSM
	ARCGDM01	DCOLLECT MIGRATION DATA	DFSMSHsm DCOLLECT MIGRATION	HSM
	ARCGDT01	DCOLLECT TAPE CAPACITY PLANNIN	DFSMSHsm DCOLLECT TAPE CAP	HSM
	ARCGS001	Statistics for DFSMSHsm	DFSMSHsm FSR-SMF Records	HSM
	ARCGS002	Statistics for Backup	DFSMSHsm FSR-SMF Records	HSM
	ARCGS003	Statistics for Migration	DFSMSHsm FSR-SMF Records	HSM
	ARCGS004	Statistics for Recall	DFSMSHsm FSR-SMF Records	HSM
	ARCGS005	Statistics for Recovery	DFSMSHsm FSR-SMF Records	HSM
	ARCGS006	Statistics for Volume Dump	DFSMSHsm FSR-SMF Records	HSM
	ARCGS007	Statistics for Restore from Du	DFSMSHsm FSR-SMF Records	HSM
	ARCGS008	Statistics for FRBACKUP	DFSMSHsm FSR-SMF Records	HSM
	ARCGS009	Statistics for FRRecover	DFSMSHsm FSR-SMF Records	HSM
	ARCGS010	DFSMSHsm Thrashing Report	DFSMSHsm FSR-SMF Records	HSM
	EDGGAUD1	SMF Audit of Volumes by Volser	SMF Records for Volumes	RMM
	EDGGAUD2	SMF Audit of Volume by Rack	SMF Records for Volumes	RMM
	EDGGAUD3	SMF42 Audit of Volumes by Vols	SMF42 Records for Volumes	RMM
	EDGGAUD4	SMF42 Audit of Volume by Rack	SMF42 Records for Volumes	RMM
	EDGGDCDS	Data Sets by Storage Group	DFSMS DCOLLECT for Data Sets	RMM
	EDGGDSNM	Mixed Case data sets Retained	Extended Extract Records	DFRMM1
	EDGGREPL	Volumes to be replaced	Extended Extract Records	RMM
	EDGGREPV	Volumes to be replaced based o	Extended Extract Records	RMM
	EDGGR01	Scratch tapes by volume serial	Extended Extract Records	RMM
	EDGGR02	List of SCRATCH Volumes by Dat	Extended Extract Records	RMM
	EDGGR03	Inventory List by Volume Seria	Extended Extract Records	RMM
	EDGGR04	Inventory List by Dataset Name	Extended Extract Records	RMM
	EDGGR06	Inventory of Volumes by Locati	Extended Extract Records	RMM
	EDGGR07	Inventory of Dataset by Locati	Extended Extract Records	RMM
	EDGGR08	Inventory of Bin by Location	Extended Extract Records	RMM
	EDGGR09	Datasets in Loan Location	Extended Extract Records	RMM
	EDGGR10	Volumes in Loan Location	Extended Extract Records	RMM
	EDGGR11	List MultiVolume and MultiFile	Extended Extract Records	RMM
	EDGGR12	Movement Report by Dataset	Extended Extract Records	RMM
	EDGGR13	Movement Report by Bin	Extended Extract Records	RMM
	EDGGR14	Movement Report by Volume Seri	Extended Extract Records	RMM
	EDGGR15	Volume Inventory Including Vol	Extended Extract Records	RMM
	EDGGSEC1	Report of Accesses to Secure V	SMF Security Records	RMM
	EDGGSEC2	SMF42 Report of Accesses to Se	SMF42 Security Records	RMM

Figure 14-13 DFSMSrmm Report Definitions showing all available default reports

Note: You can also add new report types for data other than data that are created by DFSMSrmm. For example, the report types shipped with the report generator include types for DCOLLECT and DFSMSHsm reporting. The report type contains information about a specific type of record in an input data set, the Assembler language macro that defines the record format, and basic record selection criteria. For example, the report type "Extract Records for Data Sets" in the product library contains information about the data set record in the extract data set, the EDGRDXT mapping macro, and the minimum subset definition of records that are used in the report. Report types contain only the base information from which report definitions are created.

- To modify an existing Report use the "N" Copy a report definition line selection in front of the report you would like to modify. In the pop up panel you must define a new name for this report as shown Figure 14-14

Panel Help

DFSMSrmm Report Definitions

Row 1 to 33 of 41

Command ==>

Scroll ==> PAGE

The following line commands are valid: A,D,G,J,L,M,N,S, and T

S Name	Report title	Report type	User id
- - - - -	+ - - - -	+ - - - -	- - - - -
ARCGAB01	!	!	HSM
ARCGAR01	!	!	HSM
N ARCGDB01	!	! P	MHLRES7
ARCGDD01	! Enter the report name MYOWNREP	! CAP	HSM
ARCGDM01	!	! TION	HSM
ARCGDT01	!	! CAP	HSM
ARCGS001	+ - - - -	+ s	HSM
ARCGS002	Statistics for Backup	DFSMSshm FSR-SMF Records	HSM

Figure 14-14 Specify a new report name

- You can select one of the default reports or you can create your own report. To show how you can create your own report we will show you an existing report and give you some information what can be change. We have selected the *MYOWNREP DCOLLECT BACKUP DATA DFSMSHsm DCOLLECT BACKUP* report as shown in Figure 14-13 on page 622 to get the detailed information in the panel **DFSMSrmm Report Definition - MYOWNREP** as shown in Figure 14-15 on page 624.

Panel Help

DFSMSrmm Report Definition - MYOWNREP

Row 1 to 18 of 40

Command ==>

Scroll ==> PAGE

Report title . . . DCOLLECT BACKUP DATA

Report footer . . .

Reporting tool . : ICETOOL

Report width: 260

Use END to save changes or CANCEL

Select a field name with S to specify a field selection criterion

S	CO	SO	Field name	Column header text	CW	Len	Typ
*	1	1A	UBDATE	LAST BU DATE	12	4	N
	2	3A	UBTIME	LAST BU TIME	12	4	B
	3	2A	UBDSNAM	DSN	44	44	C
	4		UBFRVOL	1st SRC VOL	11	6	C
	5		UBDATCL	DC NAME	30	30	C
	6		UBMGTCCL	MC NAME	30	30	C
	7		UBSTGCL	SC NAME	30	30	C
	8		UBALLSP	ORIG ALLOC (KB)	15	4	N
	9		UBUSESP	USER DATA (KB)	14	4	N
	10		UBDSIZE	BACKUP DS (KB)	14	4	N
*			DCURCTYP	RECORD TYPE FOR THIS RECORD	27	2	C
			DCUOUTH	DATA COLLECTION OUTPUT RECORD	29	24	C
			...				

Figure 14-15 DFSMSrmm Report Definition panel

The MYOWNREP report have 10 data columes and is sorted by date first, data set name and time next.

Where:

- S

Selection column

Enter S in this selection column to list all fields with existing selection criteria, and to optionally add the newly selected field to the list to enable criteria to be entered.

Fields for which a selection criterion already exists, are marked with an asterisk in this column.
- CO

Column Order

Use this data column to specify the order and the grouping of the columns in your report. The column order is left to right across the report page.

Enter G to specify fields you want to group. You must then specify the sort order for the fields within the group.

Duplicate numbers are automatically resolved and renumbered from the bottom to the top of the list. Grouped field names appear on the top of the list by sort order. The grouped field names are the primary sort key of the record.

Grouped field names are listed in the report header. A page break is forced each time the contents of a grouping field name changes. Grouped field names are typically fields that have the same contents over several report pages. (E.g. the location and destination name in a movement report).

	Possible values: A decimal number between 1 and 99 or G .
SO	Sort Order Use this data column to specify the Sort Order and the direction for the selected field names to change the sequence in which records are presented in the report. Grouped field names are the primary sort key of the record. The sort order for grouped field names can only be specified within the range of the number of grouped field names. The default sort direction for grouped field names is ascending. Possible values: A decimal number between 1 and 99 , followed by A - Ascending or D - Descending.
Field Name	The Field Name data column displays the name of a field in the mapping macro specified in the report type definition.
Column header text	Use the Column Header Text data column to specify the text used as the column header . The column header text, the column width and the field length are dependent on each other Changing the column header text sets the column width to the maximum of the column width, the field length and the column header text. Changing the column width sets the length of the column header text to the column width. If the column width is less than the length of the field definition the output in the report is truncated to the column width. Truncation removes trailing characters for character fields, and leading characters for numeric fields. Possible values: 1 to 37 characters.
CW	Column Width Use the Column Width data column to specify the length of the column in the report. The column header text, the column width and the field length are dependent on each other. Changing the column header text will set the column width to the maximum of the column width, the field length and the column header text. Changing the column width will set the length of the column header text to the column width. If the column width is less than the length of the field definition the output in the report will be truncated to the column width. Possible values: A decimal number between 1 and 99 .
Len	The Field Length data column displays the length of the field definition. If the column width is less than the length of the field definition the output in the report will be truncated to the column width.
Type	The Field Type data column displays the type of the field name. Use this information when specifying compare values. Possible values: C - Character N - Numeric B - Bitstring

- To add a new column to the report you can specify a “S” in the selection column and the position the column should be written in the report. In our example we have added the **DS size compressed in KB** information to the existing columns and this information should be shown as column 11 at the end of each row in the report. You can see our selection in Figure 14-16.

Panel Help							
DFSMSrmm Report Definition - MYOWNREP		Row 35 to 40 of 40					
Command ==>		Scroll ==> PAGE					
Report title . . . DCOLLECT BACKUP DATA							
Report footer . .							
Reporting tool . : ICETOOL		Report width: 260					
Use END to save changes or CANCEL							
Select a field name with S to specify a field selection criterion							
S	CO	SO	Field name	Column header text	CW	Len	Typ
-	-	-	-	-	-	-	-
			UBBKLN	BLOCK LENGTH	12	2	C
			UBFLAG2	INFORMATION FLAG #2	19	1	B
			UBRECSP	RECOVER SPACE ESTIMATE(KB)	26	4	N
			UB_USER_DATASIZE	DS size if not compressed in KB	31	4	N
S 11			UB_COMP_DATASIZE	DS size compressed in KB	24	4	N
			UBBDSIE	END OF DCUBCDS	14	1	C

Figure 14-16 DFSMSrmm Report Definition panel

- Use END to save changes and return to the **DFSMSrmm Report Definition** panel or press enter to tailoring your selection **DFSMSrmm Report Criteria** panel as shown in Figure 14-17. If you have used the END skip the next information and go directly to step 15.

Panel Help			
DFSMSrmm Report Criteria - MYOWNREP		Row 1 to 3 of 3	
Command ==>		Scroll ==> PAGE	
Report title : DCOLLECT BACKUP DATA			
Use END to save changes or CANCEL			
The following line commands are valid: B,D,N,P,R,T, and I (for details)			
Comparison operators: EQ =, NE <>, GT >, GE >=, LT <, LE <=, IN, BW, SE, SN			
Conjunction: AND, OR, AND(,)AND			
S	Field name	Op Compare value(s)	Conj Len Typ
-	-----	-----	-----
	DCURCTYP	EQ B	2 C
	UBDATE	LE &TODAY	4 N
-	UB_COMP_DATASIZE	---	4 N

Figure 14-17 DFSMSrmm Report Generation panel

Where:

S Use the S data column to specify the logical order of the criteria, or to request the Details panel.

Possible values:

- B** Bottom - Move this entry to the bottom.
- D** Delete - Delete this entry.
- I** Detail - Add or change information details, for example substring parameters
- N** Next - Move this entry down by one.
- P** Previous - Move this entry up by one.
- R** Repeat - Repeat this entry.
- T** Top - Move this entry to the top.

8. To generate the job control for this report return to the **DFSMSrmm Report Definitions** panel and use “G” in front of the report to get the **DFSMSrmm Report Generation** panel as shown in Figure 14-18 to change some variables before the file tailoring is runing to create and store the jobcontrol.

Panel	Help
DFSMSrmm Report Generation - MYOWNREP	
Command ==>	
Enter or change the skeleton variables for the generated JCL:	
Input data set	'MHLRES5.PEDCOL.EAVTST.G0003V00'
Date format	YYYYDDD
(American, European, Iso, Julian, or free form)	
Required if you use variable dates (&TODAY) in your selection criteria.	
Create report data . .	N (Y/N)
Choose Y if you want an extract step included into your generated JCL.	
Additional skeleton variables, if an extract step is included:	
Skeleton Variable_1 . .	
Skeleton Variable_2 . .	
Skeleton Variable_3 . .	
The skeleton selection depends on the reporting macro . . . : ARCUTILP	
and macro keyword . . : TYPE=B	
Enter END command to start the report generation or CANCEL	

Figure 14-18 DFSMSrmm Report Generation panel

Where:

Input Data Set

specifies the name of the report input file. This field is optional for a report type definition, but required for the generation of the report JCL.

If an extract step is included into the reporting JCL, obviously the output file name of this step is also represented by this Input Data Set parameter. In case of an RMM extract step the file must be preallocated.

Possible values:

Any sequential file name

Specify GDG data sets as follows:

'RMM.EXTRACT(0)'

Date Format

Specify the Date Format if you use variable dates with &TODAY ... in the selection criteria. For RMM extract data use the date format specified when creating the data with the RMM report extract inventory management step.

When you compare dates remember that a format such as ISO or JULIAN enables easy comparison because of the year, month, day hierarchy.

Possible values:

AMERICAN	dates in format MM/DD/YYYY
EUROPEAN	dates in format DD/MM/YYYY
ISO	dates in format YYYY/MM/DD
JULIAN	dates in format YYYY/DDD

free form

The free form has a maximum length of 20 bytes and contains DD and MM (alternatively DDD), and YY or YYYY or CYY. These key characters may be surrounded by fill-in characters. Examples:

YYYYDDD	this form would fit for packed decimal data
CYYDDD	the C (century) is set to 1 for years after 2000
DD.MM.YY	dot as fill-in character
' YY"MM"DD'	three leading blanks and ' as fill-in character

The default value JULIAN is specified in the DFSMSrmm Dialog User Option (option 0.1).

Create Report Data

If you want to include the step which creates the Report Data, then choose Y.

Additional skeleton variables may be needed for the extract JCL.

Possible values:

N	The Create Report Data step is not included.
Y	The Create Report Data step is included.

Skeleton Variables

Use the Skeleton Variable fields to help with customization of the optional job step which creates the report data.

Unless the skeleton variables are already defined in the report definition, the system tries to determine some skeleton variables based on the applied macro name. For example: if an RMM report extract macro is applied, then

Skeleton Variable_1	will contain DATEFORM(x)
Skeleton Variable_2	will contain a suggestion for the RMM reportextractmessagedatasetname.

Possible values:

1 to 50 characters

9. Use END to save your changes and return to the **DFSMSrmm Report Definition** panel. You get the information where the JCL is stored as shown in Figure 14-19 on page 629.

Panel Help

DFSMSrmm Report Definitions

Row 1 to 22 of 40

Command ==>

Scroll ==> PAGE

The following line commands are valid: A,D,G,J,L,M,N,S, and T

S Name	Report title	Report type	User id

ARCGAB01	ABARS ABACKUP Statistics	DFSMSshm ABARS Report	HSM
ARCGAR01	ABARS ARECOVER Statistics	DFSMSshm ABARS Report	HSM
MYOWNREP	DCOLLECT BACKUP DATA	DFSMSshm DCOLLECT BACKUP	MHLRES7
ARCGDD01	DCOLLECT DASD CAPACITY PLANNIN	DFSMSshm DCOLLECT DASD CAP	HSM
...			
EDGGAUD1	SMF Audit of Volumes by Volser	SMF Records for Volumes	RMM
EDGGAUD2	SMF Audit of Volume by Rack	SMF Records for Volumes	RMM
EDGGAUD3	_____ s		RMM
EDGGAUD4	! Report JCL MYOWNREP stored on 'MHLRES7.REPORT.JCL' ! s		RMM
EDGGDCDS	_____ Sets		RMM
EDGGDSNM	Mixed Case data sets Retained	Extended Extract Records	DFRMM1
....			

Figure 14-19 DFSMSrmm Report Generation panel with stored message

Now you can see the stored report as member ARCD01 in the library 'MHLRES7.REPORT.JCL' as shown in Figure 14-20. This is the library you have defined in **Report definition libraries: USER** as described in section 14.2, "Setting up the Report Generator for Your Installation" on page 614 Step 8.

Panel Help			
EDIT		MHLRES7.REPORT.JCL	Row 00001 of 00002
Command ===>		Scroll ===> PAGE	
	Name	Prompt	Size Created Changed ID
-----	MYOWNREP		89 2008/04/02 2008/04/02 07:20:08 MHLRES7
-----	EDGGSEC1		89 2008/03/12 2008/03/12 21:26:16 MHLRES7
End			

Figure 14-20 User report definition library

- Use the "J" *Edit and manually submit a report JCL* line command in front of the new generated DCOLLECT BACKUP DATA report to edit the JCL member as shown in Example 14-1. You can see that the **DS size compressed in KB column** is added to this report. Specify SUBMIT and press enter to submitting the job

Example 14-1 Sample DCOLLECT BACKUP DATA report

File Edit Edit_Settings Menu Utilities Compilers Test Help			
EDIT		MHLRES7.REPORT.JCL(MYOWNREP) - 01.00	Columns 00001 00072
Command ===>		SUBMIT	Scroll ===> PAGE
***** Top of Data *****			
//RMMJOBS1 JOB (999,POK),MSGLEVEL=1,NOTIFY=&SYSUID			
/*JOBPARM SYSAFF=SC70			
/*			
/*			
/*			

```

/*****
/**      SKELETON MEMBER EDGSGICE
/**      TAILORED BY THE RMM REPORT GENERATOR
/*****
/*****
/**      DATE CALCULATION STEP
/**      COMPARE VALUES CONTAINING ARE CALCULATED BASED ON RUN DATE
/**      COMPARE VALUES CONTAINING ARE CALCULATED BASED ON RUN DATE
//DATECONV EXEC PGM=IKJEFT01,PARM='%EDGRGDAT'
//SYSPROC DD DISP=SHR,DSN=SYS1.SEDGEXE1
//SYSTSPRT DD SYSOUT=*
//DATEFMT DD *
DATE PATTERN:YYYYDDD
//INCLIN DD *
OPTION VLSHRT,VLSCMP
INCLUDE COND=((9,2,CH,EQ,C'B'),
              AND,
              (85,4,PD,LE,&TODAY-000D))
INREC FIELDS=(1,4,
              85,4,C' ',
              81,4,C' ',
              29,44,C' ',
              213,6,C' ',
              91,30,C' ',
              155,30,C' ',
              123,30,C' ',
              193,4,C' ',
              197,4,C' ',
              77,4,C' ',
              209,4,C' ')
SORT FIELDS=(5,4,PD,A,
            15,44,CH,A,
            10,4,CH,A)
//INCLOUT DD DSN=&INCL,UNIT=SYSALLDA,SPACE=(TRK,(1,1)),DISP=(,PASS),
//          DCB=(RECFM=FB,LRECL=80)
//SYSTSIN DD DUMMY
/*****
/**
/**      following the new positions in record after inrec
/**      (variable record length needs a rdw field in column 1 to 4)
/**
/*****
/**      UBDATC          : orig pos: 00081. position after inrec:    5
/**      UBTIME          : orig pos: 00077. position after inrec:   10
/**      UBDSNAM         : orig pos: 00025. position after inrec:   15
/**      UBFRVOL         : orig pos: 00209. position after inrec:   60
/**      UBDATCL         : orig pos: 00087. position after inrec:   67
/**      UBMGTCL         : orig pos: 00151. position after inrec:   98
/**      UBSTGCL         : orig pos: 00119. position after inrec:  129
/**      UBALLSP         : orig pos: 00189. position after inrec:  160
/**      UBUSESP         : orig pos: 00193. position after inrec:  165
/**      UBDSIZE         : orig pos: 00073. position after inrec:  170
/**      UB_COMP_DATASIZE : orig pos: 00205. position after inrec:  175
/**
//WRITE1 EXEC PGM=ICETOOL,REGION=OM

```



```

//SYSPRINT DD SYSOUT=*
//TOOLMSG DD SYSOUT=*
//DFSMSG DD SYSOUT=*
//INDD DD DSN=MHLRES5.PEDCOL.EAVTST.G0003V00,
// DISP=SHR
//OUTDD DD SYSOUT=*
//TEMP DD UNIT=SYSALLDA,SPACE=(TRK,(5,25))
//TOOLIN DD *
SORT FROM(INDD) TO(TEMP) USING(INCL)
DISPLAY FROM(TEMP) LIST(OUTDD) -
TITLE('DCOLLECT BACKUP DATA') -
PAGE DATE(4MD/) TIME -
HEADER('LAST BU DATE') ON(5,4,PD,A0) -
HEADER('LAST BU TIME') ON(10,4,HEX) -
HEADER('DSN') ON(15,44,CH) -
HEADER('1st SRC VOL') ON(60,6,CH) -
HEADER('DC NAME') ON(67,30,CH) -
HEADER('MC NAME') ON(98,30,CH) -
HEADER('SC NAME') ON(129,30,CH) -
HEADER('ORIG ALLOC (KB)') ON(160,4,FI,A0) -
HEADER('USER DATA (KB)') ON(165,4,FI,A0) -
HEADER('BACKUP DS (KB)') ON(170,4,FI,A0) -
HEADER('DS size compressed in KB') ON(175,4,FI,A0) -
BLANK -
TOTAL(' ')
/** /* &INCL CONTAINS THE SORT $02A*/
/** /* INCLUDE STATEMENTS, MODIFIED $02A*/
/** /* IN STEP DATECONV $02A*/
//INCLCNTL DD DSN=&INCL,DISP=(OLD,PASS) /*$02C*/

```

11. Submit and run the job to get the result as shown in Figure 14-21. The column width can be changed in panel **DFSMSrmm Report Definition - MYOWNREP** as described in step 4 in this section.

DCOLLECT BACKUP DATA - 1 - 2008/04/03 05:15:10			
LAST BU DATE	LAST BU TIME	DSN	
2007047	18054049	SYS1.SMS.ACDS.GW	
2007053	12395692	YYY.MHLRES2.MVOL.DATA	
2007053	12405017	YYY.SRCHFOR.LIST	
2007053	12405453	YYY.SUPERC.LIST	
2007054	18561271	MHLRES2.DCOLLECT.D987	
2007058	15290805	YYY.DCOLLECT.ESDS0	
2007058	15294019	YYY.DCOLLECT.ESDS01	
2007058	15301437	YYY.DCOLLECT.ESDS1	
2007061	13334149	TESTFR.SUPERC.LIST	
2007071	17392521	TESTFR.CNTL.JCL	
2007071	17531956	TESTFR.CNTL.JCL	
....			

Figure 14-21 Sample DCOLLECT BACKUP DATA report

12. You can change your settings to reduce the column width and the sequence the columns are shown in the report. In the **DFSMSrmm Report Definitions** panel use the “S” line command in front of the report to get the **DFSMSrmm Report Definition - MYOWNREP**

detail panel. In your example in Figure 14-22 we have changed the column width of the LAST BU DATE, LAST BU TIME and DS size compressed in KB. Also we have changed the column sequence so we have moved the DS size compressed in KB from column 11 to column 3 and have added one to all columns between 3 and 10. That means the column 3 is now column 4, column 4 is now column 5 and so on.

Panel Help

DFSMSrmm Report Definition - MYOWNREP

Row 1 to 28 of 40

Command ==>

Scroll ==> PAGE

Report title . . . DCOLLECT BACKUP DATA

Report footer . . .

Reporting tool . : ICETOOL

Report width: 271

Use END to save changes or CANCEL

Select a field name with S to specify a field selection criterion

S	CO	SO	Field name	Column header text	CW	Len	Typ
*	1	1A	UBDATE	LBU DATE	8	4	N
	2	3A	UBTIME	LBU time	8	4	B
	3		UB_COMP_DATASIZE	DS size	7	4	N
	4	2A	UBDSNAM	DSN	44	44	C
	5		UBFRVOL	1st SRC VOL	11	6	C
	6		UBDATCL	DC NAME	30	30	C
	7		UBMGTC	MC NAME	30	30	C
	8		UBSTGCL	SC NAME	30	30	C
	9		UBALLSP	ORIG ALLOC (KB)	15	4	N
	10		UBUSESP	USER DATA (KB)	14	4	N
	11		UBDSIZE	BACKUP DS (KB)	14	4	N
*			DCURCTYP	RECORD TYPE FOR THIS RECORD	27	2	C
...							

Figure 14-22 Changed DFSMSrmm Report Definition for report MYOWNREP

Note: The column header text cannot exceed the column width you have specified in Len.

- Use END to ave your changes and return to the **DFSMSrmm Report Definitions** panel. Select “G” Generate and save the JCL, than “ J” Edit and manually submit the JCL as described in step 7 to step 9 in this section und submit the job. In Figure 14-23 you can see the changes. For example the DS size column is now before the DSN column.

DCOLLECT BACKUP DATA		- 1 -	2008/04/03	06:08:07
LBU DATE	LBU time	DS size	DSN	
-----	-----	-----	-----	
2007047	18054049	0	SYS1.SMS.ACDS.GW	
2007053	12395692	0	YYY.MHLRES2.MVOL.DATA	
2007053	12405017	0	YYY.SRCHFOR.LIST	
...				

Figure 14-23 Changed DFSMSrmm Report Definition for report MYOWNREP



Splitting and merging your DFSMSrmm CDS

The control data set contains information about the resources managed by DFSMSrmm. Normally, the resources are all those used on a single system or in a set of systems with a shared direct access storage device (DASD) environment. If you ever split these systems or merge them together because of changes in workload, you may also want to do the same to the DFSMSrmm control data set.

The objectives of this chapter are to:

- ▶ Understand when to split or merge a DFSMSrmm CDS
- ▶ Learn the process to successfully split or merge a DFSMSrmm CDS

By the end of this chapter, you will understand when to split or merge DFSMSrmm CDSs and the process to successfully accomplish the task.

15.1 When to merge or split

You should not normally have to merge or split the contents of the DFSMSrmm control data set. However, either during conversion or when you are changing the number of MVS systems you run, you may want to merge or split the contents of the DFSMSrmm control data set.

15.1.1 During conversion

When you convert from an existing database to an DFSMSrmm control data set, you may want to use an existing DFSMSrmm control data set. You may have one system already converted to DFSMSrmm, and now a second is to be converted, but you want to share the control data set with the first system.

During conversion, you add data produced by EDGCNVT to an existing control data set. You do not split the conversion data stream into multiple control data sets. See 15.5.1, “Converting to an existing CDS” on page 652, for details about how you merge the data you are converting into an existing DFSMSrmm control data set.

15.1.2 Changing the number of systems

If you change the number of systems or control data sets that you have, and do not delete the information, merge or split the contents of the DFSMSrmm control data set.

If you increase the number of control data sets, you split the control data set. See 15.5, “Splitting the CDS” on page 646, for details about how you split an existing control data set into multiple DFSMSrmm control data sets.

If you decrease the number of control data sets, you merge control data sets. See 15.4, “Merging the CDS” on page 643, for details about how you merge multiple DFSMSrmm control data sets.

15.1.3 Considerations

When you merge or split a control data set, consider the timing of events as well as the contents of the control data set.

DFSMSrmm must be inactive, and you must not perform tape processing when you are processing the control data set. When you stop DFSMSrmm, back up existing control data sets and ensure that you have recovery plans in place in case you are unable to complete the process successfully. Also when you stop DFSMSrmm, do not use the OPT=RESET function, because it allows tapes to be processed without DFSMSrmm’s knowledge.

When merging or splitting a control data set, it is important to understand the contents of the control data set. For example, if you merge two control data sets, you cannot merge if the same resource is defined in each control data set. You must analyze the contents of the control data set to ensure that there are no duplicate resources. The following sections describe each resource type in the control data set and explain what you can do to handle duplicate resources.

Table 15-1 on page 635 lists the record types.

Table 15-1 *Key types and record types*

Key type	Record type
C	Control
CA	Action
D	Data set
E	Empty rack
F	Scratch rack
K	VRS
O	Owner
P	Product
R	Empty bin
S	In-use bin
U	In-use rack
V	Volume

15.1.4 Control record

Each DFSMSrmm control data set contains a control record. The control record contains the dates and times of when inventory management functions last ran, and the counts of rack numbers and bin numbers for built-in storage locations.

You can merge the control record from a single control data set and copy it to all target control data sets if you are splitting a control data set. In both cases, you must use EDGUTIL with PARM=MEND to correct the record counts after processing is completed.

The control record also contains the CDSID value. If you use this value to identify a control data set, you must ensure that each control data set you create uses a different value. Use EDGUTIL with PARM='UPDATE' after processing is completed and specify the CDSID value, using the SYSIN DD CONTROL statement.

15.1.5 Action record

Action records contain information about the librarian actions that are outstanding. They include the release actions and any volume movements that are required. Omit these records, because there is a strong chance that they exist in each control data set, and the records are rebuilt by DFSMSrmm during inventory management.

15.1.6 Data set record

A data set record exists for each volume on which the data set has been written. Each record is uniquely identified by the VOLSER and physical file number of the data set. When merging control data sets, refer to the information about volume records in 15.1.12, "Volume record" on page 640 for an explanation of how to handle duplicate volumes and the data sets on them.

When you split a control data set, you must copy all data set records to each target control data set, and then use the facilities of DFSMSrmm to delete those data set records that are not required. The DFSMSrmm EDGUTIL utility determines which are required by using the volume information you have copied into each target control data set. Use the PARM=MEND option to delete the unnecessary data set records from the control data set.

15.1.7 VRS record

The VRS records contain the retention and movement policies for your data. If you have multiple control data sets, it is likely that you have the same policies defined in each control data set for some types of data. The policies are identified by data set name and job name, VOLSER, and the VOLSER prefix.

When you merge control data sets, you must check for and resolve any duplicate policies. If you have a policy for the same data set name and job name combination in more than one control data set, you should check that the retention and movement defined are the same. If they are the same, you can use the policy from any control data set. If they are not the same, you must resolve the conflict before merging the control data sets.

One way to resolve the conflict is to ensure that one control data set contains the complete set of VRS policies. Identify the control data set with most of the VRSs you require in the merged control data set. Also, identify the VRSs that are unique in the other control data sets, and define them in the one control data set.

When you split a control data set, you copy the complete set of VRS records to all target control data sets. This action may result in having VRSs that are unused in some control data sets, but it does not cause a problem in DFSMSrmm processing. When DFSMSrmm is started using the new control data set, you can use the DELETEVRS subcommand to delete the unused VRSs.

15.1.8 Owner record

There is one owner record for each identified user that owns a DFSMSrmm resource such as a volume or VRS. There can also be owner records for users that own no resources. Perhaps you have defined these so that the information can be used within your installation.

The owner record contains detailed information about the user and the count of volumes owned. For users who own volumes, DFSMSrmm uses multiple owner records to provide directed retrieval of volume and data set information during search commands. You must consider the owner details and the owner volume records when splitting and merging the control data set.

Whether you merge or split control data sets, you must use a SORT program to select the owner records, remove the additional owner records that contain just volume information, and remove the duplicate owner records. After the records are loaded into a control data set, use the EDGUTIL utility with the PARM=MEND option to set the correct counts of owned volumes and build the owner volume records for owned volumes.

When you merge control data sets, check for duplicate owners across the control data sets being merged. Duplicates are not a problem, because they can be handled as long as the owner record identifies the same person or task in each control data set. If the duplicate owners are the same user, determine which control data set has the correct owner details, such as address and contact information.

Ensure that at least one control data set has the correct owner details for the users, and use the owner details as the primary source of input to the merge. If the duplicate owners are really different users, you must decide how to handle that situation. You may have to assign different user IDs or define a new owner ID and transfer the owned resources to the new owner ID. Your objective is to have no duplicate owners, or to have any duplicates be for the same user in each case.

15.1.9 Product record

A product record identifies a single product and software level, and the volumes associated with that product. Whether you are merging or splitting a control data set, the existence of product records is a problem that must be resolved.

When you merge control data sets, there must be no duplicate product records. If there are, you must delete the duplicates and redefine the product information after the merge is completed. For each duplicate product that must be deleted, list the volumes for that product. To list the volumes, assuming that all product information is deleted from a single control data set, use the command shown in Figure 15-1.

```
RMM SEARCHPRODUCT NUMBER(*) LIMIT(*) NOLIST -  
  CLIST('RMM LISTPRODUCT ')  
EXEC EXEC.RMM LIST
```

Figure 15-1 RMM SEARCHPRODUCT subcommand

To delete all product information from a single control data set, enter: the command shown in Figure 15-2.

```
RMM SEARCHPRODUCT NUMBER(*) LIMIT(*) NOLIST -  
  CLIST('RMM DELETEPRODUCT ',' NORELEASE')  
EXEC EXEC.RMM LIST
```

Figure 15-2 RMM SEARCHPRODUCT subcommand to delete all PP records

After you merge the control data sets, you can add product information by using the RMM CHANGEVOLUME subcommand you can see in Figure 15-3.

```
RMM CHANGEVOLUME DZ11C0 NUMBER(5695-DF1) FEATCD(A001) -  
  LEVEL(V01R03M00)
```

Figure 15-3 RMM SEARCHPRODUCT subcommand to delete all PP records

When you split a control data set, the easiest way to handle product records is to target all product volumes to a single control data set. If this is not possible, you can choose either of the following actions:

- ▶ Delete the product information and add it after the control data set is split.
- ▶ Individually copy the product records by key to the correct target control data set. You can do this only if all volumes for a product are targeted to the same control data set.

15.1.10 Rack numbers record

There is a single rack record for each shelf location you have defined to DFSMSrmm. There must be at least one for each volume defined, but there can be more if you also defined empty rack numbers for use when volumes are added to the control data set. **The rack number represents the volume's external VOLSER.**

There are three rack number records, each using a unique key to aid retrieval by DFSMSrmm. During the copy operation, the three key ranges must be copied, but you need to consider only the rack numbers that you have defined.

When you merge control data sets, you must not have any duplicate rack numbers. Any duplicates must be deleted. If the duplicate already contains a volume, you can reassign the volume to another rack number by using the RMM CHANGEVOLUME subcommand. See Figure 15-4 how you can use either of the following different ways to reassign a volume to a new rack number.

```
RMM CHANGEVOLUME PP1234 POOL(A*)
RMM CHANGEVOLUME PP1234 RACK(A00001)
```

Figure 15-4 RMM CHANGEVOLUME subcommand

In the first command, DFSMSrmm picks an empty rack in pool A*. In the second example, you select the empty rack to be used.

Before you clean up any duplicate rack numbers, refer to 15.1.12, "Volume record" on page 640 for an explanation of how to handle duplicate volumes.

When you split a control data set, you split the rack numbers in the same way that you split the volume ranges into the target control data sets. You decide which volumes are copied to each control data set, and ensure that the rack numbers with which they are associated are also copied to the same target control data set.

During the copy process, you identify the rack numbers using key ranges to include or exclude the appropriate rack numbers. The significant part of the key of rack number records is the first 16 characters. The key must be specified in hexadecimal, because the second byte is hex zero.

When copying ranges of volumes, you can use the COUNT keyword on the REPRO statement. However, you must specify a hexadecimal key for the TOKEY and the FROMKEY. The key includes an 8-character media name that matches the VLPOOL media name as well as the rack number. Example 15-1 shows an example of copying the rack numbers for the volumes in the 001000 to 001999 range.

Example 15-1 *Copying rack numbers*

```
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'C500F3F4F8F0404040F0F0F1F0') -
      TOKEY(X'C500F3F4F8F0404040F0F0F1F9F9F9')
/* COPY racks from E 3480 001000 to E 3480 001999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'C600F3F4F8F0404040F0F0F1F0') -
      TOKEY(X'C600F3F4F8F0404040F0F0F1F9F9F9')
/* COPY racks from F 3480 001000 to F 3480 001999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'E400F3F4F8F0404040F0F0F1F0') -
```



```
TOKEY(X'E400F3F4F8F0404040F0F0F1F9F9F9')  
/* COPY racks from U 3480 001000 to U 3480 001999 */
```

The first character of the key is either E, F, or U, the media name starts in position 3, and the rack number starts in position 11. Only copy a range of volumes to a single target control data set.

15.1.11 Bin numbers record

There is a single bin record for each storage shelf location you have defined to DFSMSrmm.

There are two different bin number records, each using a unique key to aid retrieval by DFSMSrmm. During the copy operation, the two key ranges must be copied, but you only need to consider the bin numbers that you have defined.

When you merge control data sets, you must not have any duplicate bin numbers. Any duplicates must be deleted. If the duplicate already contains a volume, you can reassign the volume to another bin number by using the RMM CHANGEVOLUME subcommand. You can reassign a volume to a new bin number by typing the following command you can see in Figure 15-5.

```
RMM CHANGEVOLUME A00123 BIN(000029)
```

Figure 15-5 RMM CHANGEVOLUME subcommand

You can also remove the volume from the storage location, free up the bin number, and delete the unused bin number. Use the following command to delete duplicate bin number X00022 in location VAULT1 of media name CARTS you see in Figure 15-6.

```
RMM CHANGEVOLUME A00123 LOCATION(HOME) CMOVE  
RMM DELETEBIN X00022 MEDIANAME(CARTS) LOCATION(VAULT1)
```

Figure 15-6 RMM CHANGEVOLUME subcommand to confirm outstanding moves

The next time you run inventory management, the volume is reassigned to the storage location, and another bin number is assigned.

Note: When you need to reassign bin numbers, consider how offsite movements are managed in your installation. If you depend on the volume being in a specific slot in a storage location, ensure that any volumes you change are physically moved in the storage location.

When you split a control data set, you split the bin numbers according to how you want to handle storage locations in the split environment. If the volumes in the storage locations are split so that each target control data set contains volumes for each shelf-managed storage location, handling the splitting of bin numbers is very complex. We suggest that you use DFSMSrmm commands to change the location of all volumes and confirm their return to their home location. This approach frees the in-use bin numbers.

Do not copy the bin numbers to the target control data sets. Instead, add empty bin numbers after the split and use inventory management VRSEL and DSTORE processing to assign new bin numbers to the volumes after the split processing is completed.

Targeting all volumes in a single storage location for the same control data set simplifies the copying of bin numbers. During the copy process, you identify the bin numbers using key ranges to include or exclude the appropriate bin numbers. The significant part of the key of bin number records is the first 24 characters. The key can be specified in character form.

When copying ranges of bin numbers, specify a TOKEY and the FROMKEY because a range of bin numbers can include both empty and in-use bin numbers. The key includes an 8-character media name, which matches the LOCDEF media name as well as the bin number, and the location name. Example 15-2 shows an example of copying the bin numbers for the bin numbers in storage location VLT1, media name CARTS, and the V10001 to V10500 range.

Example 15-2 *Sample REPRO statements for copying bin numbers*

```

REPRO INFILE(SOURCE) OUTFILE(OUTS1)      -
      FROMKEY(C'RUVLT1  CARTS  V10001') -
      TOKEY(C'RUVLT1  CARTS  V10500')
REPRO INFILE(SOURCE) OUTFILE(OUTS1)      -
      FROMKEY(C'SUVLT1  CARTS  V10001') -
      TOKEY(C'SUVLT1  CARTS  V10500')

```

The first two characters of the key are either RU or SU, the storage location is in position 3, the media name starts in position 11, and the bin number starts in position 19. Only copy a range of bin numbers to a single target control data set.

15.1.12 Volume record

The volume record contains detailed information about a volume and its contents. It also identifies the data sets that are on the volume and volume chaining information. Some fields in the records identify the existence of other records in the control data set.

When moving volume records from one control data set to another, you must be sure to move the related records as well. These include the data set records, owner record, rack record, bin records, and product record. If you do not move them, the EDGUTIL utility can identify the missing records, and with the MEND option, creates the missing records with one exception which is the product records that cannot be re-created.

When you merge control data sets, you should not have any duplicate volumes. If you do, you must either skip the duplicate volumes or delete them from the source control data set before starting the merge process. You should understand why there are duplicate volumes, and based on your understanding, select the best approach for dealing with them.

If you have duplicate volumes, there can also be duplicate data set records for those volumes. If you delete the volumes, you also delete the data set records.

When you split a control data set, you will most probably split the volumes according to VOLSER ranges. During the copy process, you identify the volumes using key ranges to include or exclude the appropriate volumes. The significant part of the key of volume records is the first eight characters. The key must be specified in hexadecimal as the second byte is hex zero. When copying ranges of volumes, you can use the COUNT keyword on the REPRO statement. Otherwise, you must specify a hexadecimal key for the TOKEY as well.

Example 15-3 on page 641 shows REPRO statements for copying the volumes in the 001000 - 001999 range.

Example 15-3 *Sample REPRO statements for copying ranges of volumes*

```
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -  
      FROMKEY(X'E50F0F0F1') COUNT(1000)
```

The first character of the key is V, and the VOLSER starts in position 3. Only copy a range of volumes to a single target control data set.

15.2 PARMLIB options

The PARMLIB options that are affected by the splitting and merging of control data sets are:

- ▶ LOCDEF
- ▶ VLPOOL
- ▶ REJECT
- ▶ OPENRULE
- ▶ PRITITION

15.2.1 LOCDEF

These options identify your locations to DFSMSrmm. When you are merging control data sets, consolidate the LOCDEF options from all control data sets you are merging to ensure that you have no duplicates. If you have duplicates, ensure that they are identified as the same location type, and with the same media names.

When you split a control data set, each system that will use one of the target control data sets should have defined to it the locations where the volumes in the target control data set reside. You can simply include all LOCDEFs in each new system, but you have the option to remove those that are not required.

15.2.2 VLPOOL

These options identify the ranges of rack numbers and pools of shelf space that you have in your library. If your library is not changing, we recommend that you do not change your current VLPOOL options.

If you merge control data sets and your existing VLPOOL definitions are different for each existing control data set, you must merge the definitions together to cover the merged sets of volumes and rack numbers. Ensure that the operands specified on each VLPOOL are correct when the same pool prefix is defined in more than one source environment.

When you split a control data set, we recommend that you take the existing VLPOOL definitions and use them on each of the new control data sets.

15.2.3 REJECT

These options identify the ranges of volumes to be rejected for use on the system. The reject specifies the rack number prefix and should match the VLPOOL definitions that you have.

Where you have a tape library shared by multiple systems, define the same VLPOOL definitions to all systems. Then use the REJECT definitions to prevent systems from using volumes that are for use on other systems.

The actions you take for the REJECT definitions should match how you handle the VLPOOL definitions.

Attention: If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRITITION and OPENRULE commands.

15.2.4 OPENRULE

These options identify special processing for DFSMSrmm to perform during OPEN processing. It applies equally to both system-managed and non-system managed volumes.

In a shared CDS RMMplex, these commands allow you to control sharing of volumes between systems while still maintaining a single CDS. OPENRULE allows you to specify what cannot be used as well as what can be used.

15.2.5 PRITITION

These options identify special processing for DFSMSrmm to perform during library entry, export/import, eject, and CUA processing, during OPEN processing, and EXPROC processing. The PRITITION commands apply to all volumes, not only system-managed volumes. However, only system-managed volumes benefit for the functions such as library insert, CUA, and eject.

15.3 Running IDCAMS REPRO

When using IDCAMS REPRO to copy records to the target control data set, you may receive messages indicating that records are duplicates, or that there are no input records:

- Return code 4 from a REPRO command with this message:

```
IDC11465I DELIMITERS WERE SPECIFIED, BUT NO RECORDS PROCESSED.
```

The only key range for which this should occur is P for product records. Expect this message if you do not have any product definitions in the control data set.

- Return code 8 or 12 from a REPRO command with messages:

```
IDC3302I ACTION ERROR ON DFRMM.MASTER.MERGE  
IDC3308I ** DUPLICATE RECORD X'ccccccXXXX'
```

You should not receive these messages if you follow the recommendations made earlier. After four duplicate records, IDCAMS stops processing the records, so the copy operation is incomplete. You must resolve the duplicates by taking the actions recommended in 15.1.3, “Considerations” on page 634, and then rerun the merge or split job.

15.4 Merging the CDS

Use the following steps to perform the merge of two control data sets. SYS1 is the system on which the merge is being performed, and SYS2 is the second system from which the control data set data is being merged.

1. Refer to 15.1.3, “Considerations” on page 634, and use the information to analyze the control data sets that are being merged. If any potential problems or duplicate records are identified, make the recommended changes before continuing.
2. Stop DFSMSrmm on all systems affected by the merging of the control data sets. You cannot use tapes while you merge control data sets.
3. Back up the control data set on SYS2 and send the backup to SYS1. Run the backup job shown in Example 15-4 on page 644. If you have shared DASD between SYS1 and SYS2, you can skip this step.
4. Restore the SYS2 control data set on SYS1. Run the restore job shown in Example 15-5 on page 644. If you have shared DASD between SYS1 and SYS2, you can skip this step.
5. Run the merge job shown in Example 15-6 on page 644, including these steps:
 - a. **BACKS1**: Backs up the control data set used for SYS1
 - b. **COPYO**: Copies the owner records from both source control data sets to temporary files for input to step SORTO
 - c. **SORTO**: Removes any duplicate owner records found in SYS2 input and deletes the owner volume records
 - d. **DEFINE**: Deletes and then redefines a target control data set
 - e. **COPY**: Copies most of the records from both source control data sets
For VRS and product records, only one control data set is used as input, and a later step copies from the second control data set. This is done because these are the most likely records to still have duplicates.
 - f. **COPYK2**: Copies the VRS records from SYS2
Expect duplicates to be found if you use the same VRS definitions on both control data sets. You can remove this step if you know that all VRS definitions were in SYS1.
 - g. **COPYP2**: Copies the product records from SYS2
 - h. **MEND**: Validates the newly merged control data set and makes any corrections to record relationships
It also builds the owner volume records, corrects the count of owned volumes, and corrects the rack number and bin number counts in the control record.
 - i. **VERIFY**: Verifies that the control data set is logically sound and ready to useIf the merge job is successful, you are ready to use the merged control data set.
6. Update SYS1.PARMLIB member EDGRMMxx as indicated in Table 15-2.

Table 15-2 *Commands for changing EDGRMMxx*

Command	Required change
OPTION	Update DSNAME to identify the new control data set name.
LOCDEF	Update to reflect all locations used in the merged control data set.
VLPOOL	Update to reflect the new set of pools.
REJECT	Update to reflect any changes in VLPOOL definitions.

7. Start the DFSMSrmm procedure using the new PARMLIB member.
8. Check that the merged information is good by using DFSMSrmm commands, and the ISPF dialog to retrieve and display information.
9. Run any commands that are identified to be run in 15.1, "When to merge or split" on page 634.
10. Run DFSMSrmm inventory management on the new control data set. During this run, DFSMSrmm VRSEL and DSTORE processing identifies volumes to be moved to the correct location. If any moves are identified, use EDGRPTD to produce the picking lists and actions of any moves that you know are still required. The inventory management run should execute all inventory management functions including backup.

Example 15-4 shows the JCL for backing up the control data set on SYS2.

Example 15-4 *JCL to back up the control data set on SYS2*

```
//BACKS2 EXEC PGM=EDGBKUP,PARM='BACKUP'
//SYSPRINT DD SYSOUT=*
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=DFRMM.MASTER.SYS2,DISP=SHR
//BACKUP DD DSN=DFRMM.MASTER.BACKUP2(+1),DISP=(,CATLG,DELETE),
// UNIT=SYSDA,SPACE=(9216,(1000,999),RLSE),
// LRECL=9216,DSORG=PS,BLKSIZE=0,RECFM=VB
```

Example 15-5 shows the JCL for restoring the SYS2 control data set on SYS1.

Example 15-5 *JCL to restore the SYS2 control data set on SYS*

```
//RESTS2 EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=DFRMM.MASTER.SYS2,DISP=SHR
//BACKUP DD DSN=DFRMM.MASTER.BACKUP2(0),DISP=SHR
```

Example 15-6 shows the JCL for executing the merge process for two control data sets.

Example 15-6 *Merge JCL*

```
//BACKS1 EXEC PGM=EDGBKUP,PARM='BACKUP'
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=DFRMM.MASTER.SYS1,DISP=SHR
//BACKUP DD DSN=DFRMM.MASTER.BACKUP1(+1),DISP=(,CATLG,DELETE),
// UNIT=SYSDA,SPACE=(6233,(1000,999),RLSE),
// DCB=(MODEL.VB9216,DSORG=PS)
//*****
//** COPY THE OWNER RECORDS FROM BOTH SOURCE CDSs TO **
//** A SEQUENTIAL DATASET TO BE USED FOR THE MERGED CDS. **
//*****
//COPY0 EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT DD SYSOUT=*
//INS1 DD DSN=DFRMM.MASTER.SYS1,DISP=SHR
//INS2 DD DSN=DFRMM.MASTER.SYS2,DISP=SHR
//OUTS1 DD DSN=&&IN01,DISP=(,PASS),UNIT=SYSDA,
// SPACE=(1024,(5000,5000),RLSE),
// LRECL=2048,RECFM=VB
//OUTS2 DD DSN=&&IN02,DISP=(,PASS),UNIT=SYSDA,
// SPACE=(1024,(5000,5000),RLSE),
// LRECL=2048,RECFM=VB
```

```

//SYSIN DD *
REPRO INFILE(INS1) OUTFILE(OUTS1) FROMKEY(0) TOKEY(0)
REPRO INFILE(INS2) OUTFILE(OUTS2) FROMKEY(0) TOKEY(0)
/*****
/** SORT THE OWNER RECORDS FROM BOTH SYSTEMS TO A TEMPORARY **
/** SEQUENTIAL DATASET TO BE PASSED TO A LATER STEP COPYALL. **
/** REMOVE DUPLICATE OWNERS FROM SOURCE S2. **
*****/
//SORTO EXEC PGM=SORT,REGION=6M
//SYSPRINT DD SYSOUT=*
//SORTIN DD DSN=&&IN01,DISP=(OLD,DELETE)
// DD DSN=&&IN02,DISP=(OLD,DELETE)
//SORTWK01 DD UNIT=SYSDA,SPACE=(CYL,(10,10))
//SORTOUT DD DISP=(NEW,PASS),UNIT=SYSDA,DCB=*.SORTIN,
// SPACE=(1024,(5000,5000),RLSE),DSN=&&SORTO
//SYSOUT DD SYSOUT=*
//SYSIN DD *
SORT FIELDS=(5,9,CH,A)
INCLUDE COND=(5,1,CH,EQ,C'0',AND,14,1,CH,EQ,X'00') BASE OWNERS ONLY
SUM FIELDS=NONE ELIMINATE DUPS
//DEFINE EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//MSTVOL DD DISP=SHR,UNIT=3380,VOL=SER=VSMN01
//SYSIN DD *
DEL DFRMM.MASTER.MERGE
SET MAXCC=0
DEF CL(NAME(DFRMM.MASTER.MERGE) FILE(MSTVOL) -
VOLUME(VSMN01) KILOBYTES(50000 9000) FSPC(20 20) -
RECORDSIZE(512 9216) KEYS(56 0) SHR(3 3) RUS) -
DATA(NAME(DFRMM.MASTER.MERGE.DATA) -
CISZ(10240) ) -
INDEX(NAME(DFRMM.MASTER.MERGE.INDEX) IMBED REPL -
CISZ(2048) )
/*****
/** TARGET SYSTEM **
/** COPY THE RECORDS FROM BOTH SOURCE CDS TO THE TARGET CDS **
*****/
//COPY EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT DD SYSOUT=*
//INS1 DD DSN=DFRMM.MASTER.SYS1,DISP=SHR
//INS2 DD DSN=DFRMM.MASTER.SYS2,DISP=SHR
//TARGET DD DSN=DFRMM.MASTER.MERGE,DISP=SHR
//OWNER DD DSN=&&SORTO,DISP=(OLD,DELETE)
//SYSIN DD *
REPRO INFILE(INS1) OUTFILE(TARGET) FROMKEY(C) TOKEY(C) COUNT(1)
REPRO INFILE(INS1) OUTFILE(TARGET) FROMKEY(D) TOKEY(F)
REPRO INFILE(INS2) OUTFILE(TARGET) FROMKEY(D) TOKEY(F)
REPRO INFILE(INS1) OUTFILE(TARGET) FROMKEY(K) TOKEY(K)
REPRO INFILE(OWNER) OUTFILE(TARGET)
REPRO INFILE(INS1) OUTFILE(TARGET) FROMKEY(P) TOKEY(P)
REPRO INFILE(INS1) OUTFILE(TARGET) FROMKEY(R) TOKEY(V)
REPRO INFILE(INS2) OUTFILE(TARGET) FROMKEY(R) TOKEY(V)
/*****
/** SOURCE CDS 2 - ON TARGET SYSTEM **
/** COPY "K" RECORDS FROM THE SOURCE CDS 2 TO THE TARGET CDS. **

```

```

/** RETURN CODE 8 or 12 INDICATES DUPLICATE RECORDS.          **
/*****
//COPYK2 EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT DD SYSOUT=*
//INS2 DD DSN=DFRMM.MASTER.SYS2,DISP=SHR
//TARGET DD DSN=DFRMM.MASTER.MERGE,DISP=SHR
//SYSIN DD *
REPRO INFILE(INS2) OUTFILE(TARGET) FROMKEY(K) TOKEY(K)
/*****
/** SOURCE CDS 2 - ON TARGET SYSTEM **
/** COPY "P" RECORDS FROM THE SOURCE CDS 2 TO THE TARGET CDS. **
/** RETURN CODE 8 or 12 INDICATES DUPLICATE RECORDS. **
/*****
//COPYP2 EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT DD SYSOUT=*
//INS2 DD DSN=DFRMM.MASTER.SYS2,DISP=SHR
//TARGET DD DSN=DFRMM.MASTER.MERGE,DISP=SHR
//SYSIN DD *
REPRO INFILE(INS2) OUTFILE(TARGET) FROMKEY(P) TOKEY(P)
/*****
/** MERGED TARGET CDS **
/** RUN EDGUTIL WITH PARM='MEND' TO UPDATE THE NEW MERGED **
/** CDS TO HAVE THE CORRECT COUNTS FOR RACK/BINS. **
/** ALSO SETS OWNER VOLUME COUNTS AND BUILDS OWNER VOLUME **
/** INFORMATION. **
/*****
//MEND EXEC PGM=EDGUTIL,PARM='MEND'
//MASTER DD DISP=SHR,DSN=DFRMM.MASTER.MERGE
//SYSPRINT DD SYSOUT=*
/*****
/** MERGED TARGET CDS **
/** RUN EDGUTIL WITH PARM='VERIFY(ALL)' ON THE NEW MERGED **
/** CDS TO VERIFY THE INTEGRITY OF THE NEW CDS. **
/*****
//VERIFY EXEC PGM=EDGUTIL,PARM='VERIFY(ALL)'
//SYSPRINT DD SYSOUT=*
//MASTER DD SUBSYS=(BLSR,'DDNAME=MASTERB')
//MASTERB DD DISP=SHR,DSN=DFRMM.MASTER.MERGE
/* Use of Batch LSR can speed the running of EDGUTIL
/* IF you have not got BLSR setup on your system you must replace
/* the MASTER DD statement with the following one, and delete
/* the MASTERB DD statement
/*MASTER DD DISP=OLD,DSN=DFRMM.MASTER.MERGE,
/* AMP=('BUFND=50,BUFNI=50')
//SYSIN DD DUMMY

```

15.5 Splitting the CDS

Use the following steps to split a control data set into two control data sets. SYS1 and SYS2 are the new systems to which control data set data is being split:

1. Refer to 15.1.3, "Considerations" on page 634, and use the information to analyze the control data set that is being split. If you identify any potential problems, make the

recommended changes before continuing. Consider how you plan to split the ranges of volumes, rack numbers, and bin numbers. You must prepare the AMS statements for use in job steps COPYV and COPYB.

2. Stop DFSMSrmm on all systems affected by the splitting of the control data set. You cannot use tapes while you split a control data set.
3. Run the merge job shown in Example 15-6 on page 644. The job includes the following steps:
 - a. **BACKS**: Backs up the control data set to be split
 - b. **COPYO**: Copies the owner records from the source control data set to a temporary file for input to step SORTO
 - c. **SORTO**: Removes the owner volume records
 - d. **DEFINE**: Deletes and then redefines the target control data sets
 - e. **COPY**: Copies those records from the source control data set that must be copied to both target control data sets
The copying of the record ranges that need to be split are performed in later steps
 - f. **COPYP2**: Copies the product records to SYS2
This assumes that all product volumes are split out to SYS2 control data set. If this is not the case, refer to “Product record” on page 658 and update the JCL based on the recommended actions.
 - g. **COPYV**: Copies the volume and rack numbers to the two target control data sets
The JCL example shows how you can specify to AMS which records are to be copied. You must update the sample AMS statements to copy the actual ranges you want to have in each target control data set.
 - h. **COPYB**: Copies the bin numbers to the two target control data sets
The JCL example shows how you can specify to AMS which records are to be copied. You must update the sample AMS statements to copy the actual ranges you want to have in each target control data set.
 - i. **MEND1**: Validates the newly created SYS1 control data set and makes any corrections to record relationships
It also builds the owner volume records, corrects the count of owned volumes, corrects the rack number and bin number counts in the control record, and deletes any unused data set records.
 - j. **MEND2**: Validates the newly created SYS2 control data set and makes any corrections to record relationships
It also builds the owner volume records, corrects the count of owned volumes, corrects the rack number and bin number counts in the control record, and deletes any unused data set records.
 - k. **VERIFY1**: Verifies that the SYS1 control data set is logically sound and ready to use
 - l. **VERIFY2**: Verifies that the SYS2 control data set is logically sound and ready to useIf the merge job is successful, you are ready to use the new control data sets.
4. Back up the newly created control data sets for SYS1 and SYS2. Run the backup job shown in Example 15-8 on page 652.
5. Update SYS1.PARMLIB member EDGRMMxx on all affected systems as show in Table 15-3.

Table 15-3 *Commands to update EDGRMMxx*

Command	Required change
OPTION	Update DSNNAME to identify the new control data set name.
OPTION	Update SYSID to identify a unique system ID. If you do not specify a value, DFSMSrmm uses the SMF system ID.
OPTION	If you specified CDSID in PARMLIB for the source control data set, ensure that CDSID still matches the value you used on the source control data set.
LOCDEF	Update to reflect the locations used in each control data set.
VLPOOL	Update to reflect the new set of pools. Update the SYSID to match the value specified on the OPTION SYSID keyword. If SYSID is not specified, ensure that the VLPOOL SYSID, if required, matches the SMF system ID of the running system.
REJECT	Update to reflect any changes in VLPOOL definitions.

6. Start the DFSMSrmm procedure on each system, using the new PARMLIB member.
7. Check that the merged information is good by using DFSMSrmm commands and the ISPF dialog to retrieve and display information.
8. Execute any commands that are identified to be run in 15.1, "When to merge or split" on page 634.
9. Run DFSMSrmm inventory management on each of the new control data sets. During this run DFSMSrmm, VRSEL and DSTORE processing identifies volumes to be moved to the correct location. If any moves are identified, use EDGRPTD to produce the picking lists and actions of any moves that you know are still required.

The inventory management run should execute all inventory management functions including backup.

Example 15-7 shows the JCL for executing the split process to create two target control data sets.

Example 15-7 Sample JCL to split a DFSMSrmm CDS

```
//BACKS      EXEC PGM=EDGBKUP,PARM=' BACKUP'
//SYSPRINT   DD   SYSOUT=*
//MASTER     DD   DSN=DFRMM.MASTER.CDS,DISP=SHR
//BACKUP      DD   DSN=DFRMM.MASTER.BACKUP(+1),DISP=(,CATLG,DELETE),
//              UNIT=SYSDA,SPACE=(6233,(1000,999),RLSE),
//              DCB=(MODEL.VB9216,DSORG=PS)
//*****
//** COPY THE OWNER RECORDS FROM SOURCE CDS TO                **
//** A SEQUENTIAL DATASET TO BE USED FOR THE NEW CDSs          **
//*****
//COPY0      EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT   DD   SYSOUT=*
//SOURCE      DD   DSN=DFRMM.MASTER.CDS,DISP=SHR
//OUT         DD   DSN=&&OUTO,DISP=(,PASS),UNIT=SYSDA,
//              SPACE=(1024,(5000,5000),RLSE),
//              LRECL=2048,RECFM=VB
//SYSIN       DD   *
              REPRO INFILE(SOURCE) OUTFILE(OUT) FROMKEY(0) TOKEY(0)
//*****
//** SORT THE OWNER RECORDS FROM BOTH SYSTEMS TO A TEMPORARY  **
```

```

/** SEQUENTIAL DATASET TO BE PASSED TO A LATER STEP COPYALL.  **
/** REMOVE DUPLICATE OWNERS FROM SOURCE S2.  **
/*****
//SORTO EXEC PGM=SORT,REGION=6M
//SYSPRINT DD SYSOUT=*
//SORTIN DD DSN=&&OUTO,DISP=(OLD,DELETE)
//SORTWK01 DD UNIT=SYSDA,SPACE=(CYL,(10,10))
//SORTOUT DD DISP=(NEW,PASS),UNIT=SYSDA,DCB=*.SORTIN,
// SPACE=(1024,(5000,5000),RLSE),DSN=&&SORTO
//SYSOUT DD SYSOUT=*
//SYSIN DD *
SORT FIELDS=(5,9,CH,A)
INCLUDE COND=(5,1,CH,EQ,C'O',AND,14,1,CH,EQ,X'00') BASE OWNERS ONLY
SUM FIELDS=NONE ELIMINATE DUPS
//DEFINE EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYS1VOL DD DISP=SHR,UNIT=3380,VOL=SER=VSMN01
//SYS2VOL DD DISP=SHR,UNIT=3380,VOL=SER=VSMN02
//SYSIN DD *
DEL DFRMM.NEW.SYS1
DEL DFRMM.NEW.SYS2
SET MAXCC=0
DEF CL(NAME(DFRMM.NEW.SYS1) FILE(SYS1VOL) -
      VOLUME(VSMN01) KILOBYTES(50000 9000) FSPC(20 20) -
      RECORDSIZE(512 9216) KEYS(56 0) SHR(3 3) RUS) -
DATA(NAME(DFRMM.NEW.SYS1.DATA) -
      CISZ(10240) ) -
INDEX(NAME(DFRMM.NEW.SYS1.INDEX) IMBED REPL -
      CISZ(2048) )
DEF CL(NAME(DFRMM.NEW.SYS2) FILE(SYS2VOL) -
      VOLUME(VSMN02) KILOBYTES(50000 9000) FSPC(20 20) -
      RECORDSIZE(512 9216) KEYS(56 0) SHR(3 3) RUS) -
DATA(NAME(DFRMM.NEW.SYS2.DATA) -
      CISZ(10240) ) -
INDEX(NAME(DFRMM.NEW.SYS2.INDEX) IMBED REPL -
      CISZ(2048) )
/*****
/** SOURCE SYSTEM  **
/** COPY THE RECORDS FROM SOURCE TO BOTH TARGET CDSs  **
/*****
//COPY EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT DD SYSOUT=*
//OUTS1 DD DSN=DFRMM.NEW.SYS1,DISP=SHR
//OUTS2 DD DSN=DFRMM.NEW.SYS2,DISP=SHR
//SOURCE DD DSN=DFRMM.MASTER.CDS,DISP=SHR
//OWNER DD DSN=&&SORTO,DISP=(OLD,DELETE)
//SYSIN DD *
REPRO OUTFILE(OUTS1) INFILE(SOURCE) FROMKEY(C) TOKEY(C) COUNT(1)
REPRO OUTFILE(OUTS2) INFILE(SOURCE) FROMKEY(C) TOKEY(C) COUNT(1)
REPRO OUTFILE(OUTS1) INFILE(SOURCE) FROMKEY(D) TOKEY(D)
REPRO OUTFILE(OUTS2) INFILE(SOURCE) FROMKEY(D) TOKEY(D)
REPRO OUTFILE(OUTS1) INFILE(SOURCE) FROMKEY(K) TOKEY(K)
REPRO OUTFILE(OUTS2) INFILE(SOURCE) FROMKEY(K) TOKEY(K)
REPRO INFILE(OWNER) OUTFILE(OUTS1)
REPRO INFILE(OWNER) OUTFILE(OUTS2)

```

```

/*****
/**          SOURCE TO A SINGLE TARGET          **
/** COPY "P" RECORDS FROM THE SOURCE CDS TO THE ONE TARGET **
/*****
//COPYP2 EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT DD SYSOUT=*
//OUTS2 DD DSN=DFRMM.NEW.SYS2,DISP=SHR
//SOURCE DD DSN=DFRMM.MASTER.CDS,DISP=SHR
//SYSIN DD *
REPRO INFILE(SOURCE) OUTFILE(OUTS2) FROMKEY(P) TOKEY(P)
/*****
/**          SOURCE TO TWO TARGETS          **
/** COPY "V" RECORDS BY VOLSER RANGE 100000-199999 TO SYS1 **
/** COPY "V" RECORDS BY VOLSER RANGE A00000-A00999 TO SYS2 **
/** COPY RACK RECORDS FOR THIS RANGE OF VOLUMES AS WELL **
/*****
//COPYV EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT DD SYSOUT=*
//OUTS1 DD DSN=DFRMM.NEW.SYS1,DISP=SHR
//OUTS2 DD DSN=DFRMM.NEW.SYS2,DISP=SHR
//SOURCE DD DSN=DFRMM.MASTER.CDS
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'E500F1F0') COUNT(100000)
/* copy volumes from V 100000 to V 199999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'C500F3F4F8F0404040F1F0') -
      TOKEY(X'C500F3F4F8F0404040F1F9F9F9F9F9')
/* COPY racks from E 3480 100000 to E 3480 199999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'C600F3F4F8F0404040F1F0') -
      TOKEY(X'C600F3F4F8F0404040F1F9F9F9F9F9')
/* COPY racks from F 3480 100000 to F 3480 199999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'E400F3F4F8F0404040F1F0') -
      TOKEY(X'E400F3F4F8F0404040F1F9F9F9F9F9')
/* COPY racks from U 3480 100000 to U 3480 199999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS2) -
      FROMKEY(X'V 00C1F0') COUNT(1000)
/* COPY from V A00000 to V A00999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS2) -
      FROMKEY(X'C500C3C1D9E3E2404040C1F0') -
      TOKEY(X'C500C3C1D9E3E2404040C1F0F0F9F9F9')
/* COPY racks from E CARTS A00000 to E CARTS A00999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS2) -
      FROMKEY(X'C600C3C1D9E3E2404040C1F0') -
      TOKEY(X'C600C3C1D9E3E2404040C1F0F0F9F9F9')
/* COPY racks from F CARTS A00000 to F CARTS A00999 */
REPRO INFILE(SOURCE) OUTFILE(OUTS2) -
      FROMKEY(X'E400C3C1D9E3E2404040C1F0') -
      TOKEY(X'E400C3C1D9E3E2404040C1F0F0F9F9F9')
/* COPY racks from U CARTS A00000 to U CARTS A00999 */
/*****
/**          SOURCE TO TWO TARGETS          **
/** COPY BIN RECORDS FOR STORE VLT1 TO SYS1 **
/** COPY BIN RECORDS FOR STORE VLT2 TO SYS2 **

```

```

/*****
//COPYB      EXEC PGM=IDCAMS,REGION=5000K
//SYSPRINT DD  SYSOUT=*
//OUTS1      DD  DSN=DFRMM.NEW.SYS1,DISP=SHR
//OUTS2      DD  DSN=DFRMM.NEW.SYS2,DISP=SHR
//SOURCE     DD  DSN=DFRMM.MASTER.CDS,DISP=SHR
//SYSIN      DD  *
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'D9E4E5D3E3F140404040C3C1D9E3E2404040F0F0F0F0F0') -
      TOKEY(X'D9E4E5D3E3F140404040C3C1D9E3E2404040F0F0F0F9F9F9')
/* COPY bins from RUVLT1   CARTS   000000 for 1000 bins */
REPRO INFILE(SOURCE) OUTFILE(OUTS1) -
      FROMKEY(X'E2E4E5D3E3F140404040C3C1D9E3E2404040F0F0F0F0F0') -
      TOKEY(X'E2E4E5D3E3F140404040C3C1D9E3E2404040F0F0F0F9F9F9')
/* COPY bins from RUVLT1   CARTS   000000 for 1000 bins */
REPRO INFILE(SOURCE) OUTFILE(OUTS2) -
      FROMKEY(X'D9E4E5D3E3F240404040F3F4F8F0404040F0F0F0F0F0') -
      TOKEY(X'D9E4E5D3E3F240404040F3F4F8F0404040F0F0F1F9F9F9')
/* COPY bins from RUVLT1   3480    000000 for 2000 bins */
REPRO INFILE(SOURCE) OUTFILE(OUTS2) -
      FROMKEY(X'E2E4E5D3E3F240404040F3F4F8F0404040F0F0F0F0F0') -
      TOKEY(X'E2E4E5D3E3F240404040F3F4F8F0404040F0F0F1F9F9F9')
/* COPY bins from RUVLT1   3480    000000 for 2000 bins */
/*****
/**          EACH TARGET CDS                                **
/** RUN EDGUTIL WITH  PARM='MEND' TO UPDATE THE NEW TARGET  **
/** CDS TO HAVE THE CORRECT COUNTS FOR RACK/BINS.          **
/** ALSO SETS OWNER VOLUME COUNTS AND BUILDS OWNER VOLUME  **
/** INFORMATION, AND DELETES UNUSED DATA SET RECORDS      **
/*****
//MEND1      EXEC PGM=EDGUTIL,PARM='MEND'
//MASTER    DD  DISP=SHR,DSN=DFRMM.NEW.SYS1
//SYSPRINT DD  SYSOUT=*
//MEND2      EXEC PGM=EDGUTIL,PARM='MEND'
//MASTER    DD  DISP=SHR,DSN=DFRMM.NEW.SYS2
//SYSPRINT DD  SYSOUT=*
/*****
/**          EACH TARGET CDS                                **
/** RUN EDGUTIL WITH  PARM='VERIFY(ALL)' ON THE NEW TARGET  **
/** CDS TO VERIFY THE INTEGRITY OF THE NEW CDS.            **
/*****
//VERIFY1    EXEC PGM=EDGUTIL,PARM='VERIFY(ALL)'
//SYSPRINT DD  SYSOUT=*
//MASTER    DD  SUBSYS=(BLSR,'DDNAME=MASTERB')
//MASTERB   DD  DISP=SHR,DSN=DFRMM.NEW.SYS1
/* Use of Batch LSR can speed the running of EDGUTIL
/* IF you have not got BLSR setup on your system you must replac
/* the MASTER DD statement with the following one, and delete
/* the MASTERB DD statement
/*MASTER    DD  DISP=OLD,DSN=DFRMM.NEW.SYS1,
/*          AMP=('BUFND=50,BUFNI=50')
//SYSIN     DD  DUMMY
//VERIFY2    EXEC PGM=EDGUTIL,PARM='VERIFY(ALL)'
//SYSPRINT DD  SYSOUT=*
//MASTER    DD  SUBSYS=(BLSR,'DDNAME=MASTERB')

```

```
//MASTERB DD DISP=SHR,DSN=DFRMM.NEW.SYS2
/* Use of Batch LSR can speed the running of EDGUTIL
/* IF you have not got BLSR setup on your system you must replace
/* the MASTER DD statement with the following one, and delete
/* the MASTERB DD statement
/*MASTER DD DISP=OLD,DSN=DFRMM.NEW.SYS2,
/* AMP=('BUFND=50,BUFNI=50')
//SYSIN DD DUMMY
```

Example 15-8 shows how you can create one backup procedure that works on both systems.

Example 15-8 *Sample JCL to back up the DFSMSrmm CDS and JOURNAL*

```
//BACKCDS PROC ID=
//BACKUP EXEC PGM=EDGBKUP,PARM='BACKUP'
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=DFRMM.MASTER.SYS&ID.,DISP=SHR
//JORUNAL DD DSN=DFRMM.JRNL.SYS&ID.,DISP=SHR
//BACKUP DD DSN=DFRMM.MASTER.BACKUP&ID.(+1),DISP=(,CATLG,DELETE),
// UNIT=SYSDA,SPACE=(9216,(1000,999),RLSE),
// LRECL=9216,DSORG=PS,BLKSIZE=0,RECFM=VB
//JRNLBKUP DD DSN=DFRMM.JRNL.BACKUP&ID.(+1),DISP=(,CATLG,DELETE),
// UNIT=SYSDA,SPACE=(9248,(1000,999),RLSE),
// LRECL=9248,DSORG=PS,BLKSIZE=0,RECFM=VB
// PEND
//BACKS1 EXEC BACKCDS,ID=1
//BACKS2 EXEC BACKCDS,ID=2
```

15.5.1 Converting to an existing CDS

When you convert information to DFSMSrmm, and want the target to be an existing CDS, you are really merging databases. We recommend that you follow the normal conversion process as described in the conversion IBM Redbooks. Follow them to the point where the control data set is built and verified, you have started DFSMSrmm, and you have executed any commands necessary to complete the conversion. At that point, you know the conversion is good, and that you have a valid and complete CDS.

You can now follow the process explained in 15.4, “Merging the CDS” on page 643, to merge two DFSMSrmm CDSs.

If you have identified duplicate records, you may be able to change the extract and conversion control statements to avoid those duplicates rather than taking the actions recommended in 15.1.3, “Considerations” on page 634.

15.6 Merging DFSMSrmm environments

This section discusses the aspects of moving a system using DFSMSrmm into a sysplex where all the existing systems share a single DFSMSrmm database:

- ▶ The necessity of merging DFSMSrmm environments, or possibility of having more than one DFSMSrmm environment on a single sysplex
- ▶ A checklist of points to consider should you decide to merge the DFSMSrmm
- ▶ A methodology for performing the merge

- A list of tools and documentation that can assist with this exercise

While some concepts in this chapter may apply to tape management systems other than DFSMSrmm, this chapter has been written specifically for DFSMSrmm by people familiar with that product. The term that we use in this chapter and in the index to describe systems sharing a single tape management system database is *RMMplex*. This is opposed to TAPEplex or TMSplex, or similar terms.

15.6.1 One or more RMMplexes

It *is* possible to have more than one RMMplex in a single sysplex. By definition, the RMMplex is the set of systems that share a single DFSMSrmm control data set (CDS).

If your target environment is a BronzePlex, the incoming system may share tape drives with systems in the target sysplex, but it does not share any DASD other than the volumes containing the CDSs. If it does not share catalogs and the DFSMSrmm database (at a minimum, and probably the RACF database and Storage Management Subsystem (SMS) environment), then the incoming system is not part of the same RMMplex as the target sysplex.

If your target environment is a GoldPlex, the incoming system will share the system volumes (and therefore, potentially, the volume containing the DFSMSrmm database) with the other systems in the target sysplex. However, because all the user catalogs and RACF and so on are *not* shared, you will once again probably not merge your RMMplexes.

Finally, if your target environment is a PlatinumPlex, the incoming system *will* share the user DASD (and RACF and SMS and so on) with the other systems in the target sysplex. In this case, you *would* normally merge the RMMplexes.

15.6.2 Considerations for merging RMMplexes

This section contains (in checklist format) a list of elements that you must consider should you decide to merge two or more RMMplexes. We say *RMMplexes* rather than *DFSMSrmm CDSs*, because there is more to consider than just merging the contents of the DFSMSrmm CDSs.

Due to the interrelationship between the items in Table 15-4, we strongly recommend that you merge the SMS environment, the hierarchical storage management (HSM) environment, the security environment (for example, RACF), the catalog environment, and the DASD environment, at the same time that you merge the DFSMSrmm environments.

Table 15-4 *Considerations for merging RMMplexes*

Considerations	Note	Type	Done
Ensure toleration service is installed if necessary	1	P	
Identify and eliminate duplicate records in the DFSMSrmm CDSs	2	P	
Review PARMLIB options	3	P	
Review DFSMSrmm Procedure	4	P	
Review DFSMSrmm CDS and Journal data set sizes, and attributes	5	P	
TCDB Tape CDS	6	P	
Review any housekeeping jobs	7	P	

Considerations	Note	Type	Done
Security considerations	8	P	
Review DFSMSrmm exits	9	P	

The Type specified in Table 15-4 on page 653 relates to the sysplex target environment. The letter B represents a BronzePlex, G represents a GoldPlex, and P represents a PlatinumPlex. The steps in the following sections correspond to those in Table 15-4 on page 653.

Step 1: Ensure the toleration service is installed if necessary

Before you can start a DFSMSrmm merge process, check the DFSMSrmm software levels. If the DFSMSrmm software levels are different, then check for compatibility PTFs, and confirm that they are installed and applied.

To obtain a list of toleration and compatibility APARs for DFSMSrmm, refer to the DFSMS program directory, the DFSMSrmm PSP bucket, and Information APAR II08878.

Step 2: Identify and eliminate duplicate records within the CDSs

Note: Numerous references in this section are made to duplicate records and how to handle them. However, the first step is to *identify* whether you have duplicate records. The easiest way is to run the jobs in Example 15-9 on page 668 and Example 15-10 on page 668. The description of the first step of the job, “Step S001ICET” on page 665, discusses how the merge job handles the various record types, and specifically, what it does with any duplicate records that it finds. Look in the data sets containing the duplicate records to see which record types you need to address.

Figure 15-7 on page 655 summarizes the process of merging multiple RMM CDSs. In this example, each LPAR has its own CDS. In our sysplex merge, we (hopefully) only have two: one from the incoming system and one from the target sysplex.

When merging CDSs, it is important to understand the contents of the CDS. For example, if you merge two CDSs, you cannot merge them if the same resource is defined in each CDS. You must analyze the contents of the CDS to ensure that there are no duplicate resources (or at least, no conflicting duplicates).

If you find that there is a large number of duplicate tape volsers, then consider whether you really want to merge your DFSMSrmm CDSs. You may have a large number of duplicate volumes and still want to merge the systems. In this case, before you perform the merge, consider using one of the many TAPECOPY utility programs available to copy the duplicate volumes to volumes with unique volsers.

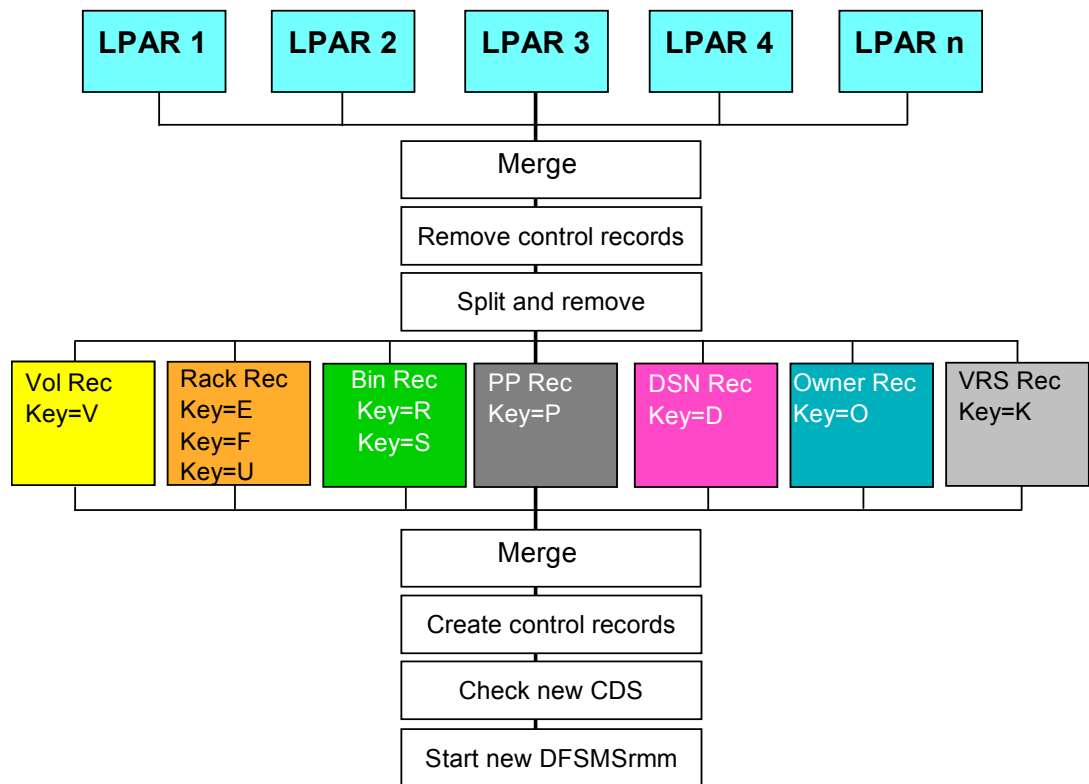


Figure 15-7 Merging process for multiple RMM CDSs

A sample merge job is shown in Example 15-10 on page 668. In addition to performing the merge, the job can also help to identify duplicate records in advance of the actual merge. The job can be continually rerun until all duplicates (or at least those that require action) are resolved. The merge job is described in more detail in “Merging the CDSs” on page 665.

The merge job uses REPRO for merging the system's DFSMSrmm CDSs into a flat file, sorts the file, discarding duplicate records in the process, and creates a new merged DFSMSrmm CDS. Duplicate records are written to sysout files for you to investigate and take action. Note that all the duplicates that are deleted are from the same system. If you plan to use the merge process to remove duplicate records, all of the records that you want to discard *must* come from the same system. For example, you cannot use the merge to discard VRS records from RMMPLEX1 and volume records from RMMPLEX2.

15.6.3 CDS record types

This section describes each resource type in the CDS and explains what you can do to handle duplicate records.

The record types contained in the DFSMSrmm CDS are listed in Table 15-5.

Table 15-5 Record types in the DFSMSrmm CDS

Key type	Record type
C	Control
CA	Action

Key type	Record type
D	Data set
E	Empty rack
F	Scratch rack
K	VRS
O	Owner
P	Product
R	Empty bin
S	In-use bin
U	In-use rack
V	Volume

Volume record

The volume record contains detailed information about a volume and its contents. It also identifies the data sets that are on the volume and volume chaining information. Some fields in the records identify the existence of other records in the CDS.

When moving volume records from one CDS to another, you must be sure to move the related records as well. These include the data set records, owner record, rack record, bin records, and product record. If you do not move them, the EDGUTIL utility may not be able to identify all the missing records, and therefore may not be able to create all the missing records.

When you merge CDSs, you should not have any duplicate volume serials. In fact, from an integrity point of view, you should not have duplicate volume serials in the same sysplex in any case, regardless of whether it causes a problem in DFSMSrmm. If you do, you must remove the duplicate volumes before starting the merge process. You should understand why there are duplicate volumes, and based on your understanding, select the best approach for dealing with them. While the merge job fixes some duplicate record problems by simply discarding one of the duplicates, you *must* ensure that there are no duplicate volume records before you start the merge. When you run the merge job, duplicate volume records are written to the file specified on the DUPSMMAIN DD statement.

If you have duplicate volumes, there can be duplicate data set records for those volumes. If you delete the volumes, the related duplicate data set records are deleted automatically as part of the process of removing the duplicate volume.

Control record

Each DFSMSrmm CDS contains a control record. The control record contains the dates and times of when the inventory management functions last ran, the counts of rack numbers and bin numbers for built-in storage locations, and the settings for some DFSMSrmm options.

When merging databases, only one control record is required. You must use the EDGUTIL program with PARM=MEND to correct the record counts after the merge job has completed, and before you start DFSMSrmm using the new CDS.

The control record also contains the CDSID value. When merging the databases, retain the control record from the system that contained the CDSID that you want to continue to use (probably the one from DFSMSrmm on the target sysplex). All systems that use the

DFSMSrmm CDSs must have a matching CDSID specified in the EDGRMMxx member of PARMLIB before being allowed to use the CDS.

As an alternative, you can bypass copying all control records. After you load the new DFSMSrmm CDS, you can create a new control record by using EDGUTIL with PARM=CREATE. After this is completed, use the same program with PARM=MEND to update the control record to correct the rack and bin counts.

Use the RMM LISTCONTROL CNTL subcommand to check the options which are in use, such as EXTENDED BIN and STACKED VOLUME. If the options in use differ between the CDSs, we recommend that you enable options to bring the CDSs in line with each other, and follow the steps to implement those options *before* starting the merge.

Action and move records

Action records contain information about the librarian actions that are outstanding. They include the release actions and any volume movements that are required. Duplicate records are deleted by the merge job, because there is a strong chance that they exist in each CDS, and the records are rebuilt by DFSMSrmm during inventory management after the merge.

Data set record

A data set record exists for each volume on which the related data set has been written. Each record is uniquely identified by the volser and physical file number of the data set. Remember that it is perfectly acceptable to have multiple tapes all containing the same data set (unlike DASD data sets where you generally would try to avoid having the same data set name on more than one volume).

When merging CDSs, you want to be sure that there are no duplicate records across the two CDSs. Because the key of the data set record includes the volser, avoiding duplicate volsers ensures that there will be no duplicate data set records.

VRS record

The VRS records contain the retention and movement policies for your data. When merging the CDSs, you may find that you have the same policies defined in each CDS for some types of data. The policies are identified by fully qualified or generic data set name and job name, volser, and volser prefix.

When you merge CDSs, check for and resolve any duplicate policies. To help you do this, use the RMM TSO SEARCHVRS subcommand to obtain a list of all policies. If you have a policy for the same data set name and job name combination in more than one CDS, check that the retention and movement defined are the same. If they are the same, you can use the policy from any CDS. If they are not the same, you must resolve the conflict before you merge the CDSs. This way, duplicates can be ignored, regardless of which system the records are taken from.

The most important reason for checking the VRS definitions is to determine which VRS policy applies to a given data set and jobname combination and what impact this can have after the merge. If you use generic VRS definitions, such as TOTHSM.**, it is possible that the VRS definition controlling a given data set and job could change even if you do not have duplicate policy names. This can potentially result in your keeping fewer generations of that data set or job than you had intended. This is similar to the discussion in the RACF chapter about data sets being protected by different RACF profiles after a merge. Check all the VRS definitions for potential clashes between VRS definitions. If you find a clash, check, and, if necessary, modify the attributes of the VRS to meet your requirements.

There is a way to check whether the VRS records in the merged CDS will provide the results you expect. You can run EDGHSKP with PARM='VRSEL,VERIFY' using the test CDS that is created by the merge job. Specifying this PARM results in a trial run on inventory management. The resulting ACTIVITY file contains a list of all changes that will be made. By using the EDGJACTP sample job and looking at the summary of changes in matching VRS, you can see if you would have this problem.

Owner record

There is one owner record for each identified user that owns a DFSMSrmm resource such as a volume or VRS. There can also be owner records for users that own no resources. Perhaps you have defined these so that the information can be used within your installation.

The owner record contains detailed information about the user and the count of volumes owned. For users who own volumes, DFSMSrmm uses multiple owner records to provide directed retrieval of volume and data set information during search commands. You only need to copy the "Base" Owner records. When there are multiple owner records, there is no need to copy the Owner records that contain just volume information. When the records are loaded into the new CDS, your merge job will use the EDGUTIL utility with the PARM=MEND option to set the correct counts of owned volumes and build the owner volume records for owned volumes. A sample is shown in Step S004MEND in the "DFSMSrmm Merge job" in Example 15-10 on page 668.

Duplicate owner records are not a problem if the duplicate owners are the same user. In this case, duplicates can be safely ignored, regardless of which system the records are taken from.

If the duplicate owners are different users, you must decide how to handle that situation. You may have to assign different user IDs or define a new owner ID and transfer the owned resources to the new owner ID. Your objective is to have no duplicate owners, or to have any duplicates be for the same user in each case.

To add a new owner, type the command shown in Figure 15-8 on the RMMplex on which the owner is being changed.

```
RMM ADDOWNER new_owner_id DEPARTMENT(MERGE)
```

Figure 15-8 RMM ADDOWNER subcommand

To delete the duplicate owner and transfer volume ownership to the new owner ID (after you add the new owner ID), use the command shown in Figure 15-9 on the same RMMplex as the ADDOWNER command.

```
RMM DELETEOWNER old_owner_id NEWOWNER(new_owner-id)
```

Figure 15-9 RMM DELETEOWNER subcommand

Product record

A product record identifies a single product and software level and the volumes associated with that product. When you merge CDSs, the existence of duplicate product records can be resolved by running EDGUTIL MEND after the merge.

As an alternative to relying on MEND to resolve any duplicates, you can resolve the duplicates before the merge by choosing one of the product names to change. First, list all the

volumes associated with the product name, using a command similar to this one you can see in Figure 15-10.

```
RMM LISTPRODUCT oldproduct
```

Figure 15-10 RMM LISTPRODUCT subcommand

Then create a new product name using the ADDPRODUCT command similar to this command as shown in Figure 15-11.

```
RMM ADDPRODUCT newproduct LEVEL(V01R01M00) NAME(.....) -  
DESCRIPTION(.....) OWNER(.....)
```

Figure 15-11 RMM ADDPRODUCT subcommand

Next change all the volumes that were associated with the old product name to the new product name using the CHANGEVOLUME command as shown in Figure 15-12.

```
RMM CHANGEVOLUME vvvvvv NUMBER(newproduct) FEATCD(A001) LEVEL(V01R03M00)
```

Figure 15-12 RMM CHANGEVOLUME subcommand

Finally, delete the old product name using the DELETEPRODUCT command as shown in Figure 15-13.

```
RMM DELETEPRODUCT oldproduct LEVEL(V01R01M00)
```

Figure 15-13 RMM DELETEPRODUCT subcommand

If you do not resolve the duplicates before the merge, there are duplicate product records during the merge. All the ones in the second CDS specified on the EXTRACT DD statement are discarded during the merge. The discarded records are written to the file specified on the DUPSMMAIN DD statement. Therefore, if you decide to leave some duplicate records in the CDS, so they can be deleted during the merge, make sure that all those records are in the same CDS. After the merge, running EDGUTIL MEND cleans up the associations between the Product and Volume records.

Rack numbers record

There is a single rack record for each shelf location you defined to DFSMSrmm. Rack records are optional when the rack number and the volume serial number have the same value. There may be one for each volume defined. There can be more if you also defined empty rack numbers for use when volumes are added to the CDS. The rack number represents the volume's external volser.

The DFSMSrmm record for each rack number can begin with one of three keys: E for empty, F for full, and U for in-use. It is possible that you can have duplicate rack numbers in the two CDSs without the merge job spotting them as duplicates. For example, rack number 200000 in one DFSMSrmm CDS can be Empty (so the key would be E200000), and rack number 200000 in the DFSMSrmm CDS can be Full (so the key would be F200000). However, the merge job ignores the key when identifying duplicate records so duplicate rack numbers *will* be identified.

When you merge CDSs, you must not have any duplicate rack numbers. Any duplicates must be deleted prior to the merge. Note that if there are duplicate records, the “U” records will be selected ahead of the “F” records, and finally the “E” records. Duplicates will be discarded, regardless of which CDS each record may have resided in prior to the merge. Any duplicates found at the time of the merge will be written to the file specified on the DUPSEFU DD statement.

Each rack number must have only one record type: either E, F, or U. You cannot have two record types for an individual rack number. If the duplicate already contains a volume, you can reassign the volume to another rack number by using the RMM CHANGEVOLUME subcommand. There are two different ways to reassign a volume to a new rack number as you can see in Figure 15-14.

```
RMM CHANGEVOLUME PP1234 POOL(A*)  
RMM CHANGEVOLUME PP1234 RACK(AP1234)
```

Figure 15-14 RMM CHANGEVOLUME subcommand

In the first command, DFSMSrmm picks an empty rack in pool A*. In the second command, you select the empty rack to be used. Remember that the rack number is the external label on the tape, so changing large numbers of rack numbers is not a trivial exercise. It helps if you can select a new rack number that is similar to the old one. For example, we changed rack number PP1234 above to rack number AP1234. Before cleaning up any duplicate rack numbers, refer to the description of the 15.1.12, “Volume record” on page 640 for an explanation of how to handle duplicate volumes.

If you currently have rack numbers for volumes when the rack number and the volume serial number have the same value, you can choose to stop using rack numbers before starting the merge. This reduces the numbers of records in the CDS. To change a volume to no longer use a rack number, you can use these commands as shown in Figure 15-13 on page 659.

```
RMM CHANGEVOLUME 100200 NORACK  
RMM DELETERACK 100200
```

Figure 15-15 RMM CHANGEVOLUME and DELETERACK subcommands

Bin numbers record

There is a single bin record for each storage shelf location you define to DFSMSrmm.

Like the rack numbers we just discussed, each bin number record can potentially have one of two different keys; in this case, R (empty) and S (in-use). Because the same bin can be represented by two different keys, the merge jobs may not correctly identify all duplicates. For example, bin number 000029 can be represented by record R000029 in one CDS, and by record S000029 in the other. However, as for the rack numbers, the sample merge job is set up to ignore the key when searching for duplicate bin records, so duplicates are identified. They are written to the file specified on the DUPSRS DD statement. Note that if there are duplicate records, the “S” records are selected and the “R” records are discarded, regardless of which CDS each record may have resided in prior to the merge.

When you merge CDSs, you must not have any duplicate bin numbers. Any duplicates must be addressed before you do the merge. If the duplicate already contains a volume, you can reassign the volume to another bin number by using the RMM CHANGEVOLUME subcommand. You can reassign a volume to a new bin number by issuing the following command as shown in Figure 15-16 on page 661.

```
RMM CHANGEVOLUME A00123 BIN(000029)
```

Figure 15-16 RMM CHANGEVOLUME subcommands to assign a bin number

You can also remove the volume from the storage location, free the bin number, and delete the unused bin number. See the commands in Figure 15-17 how to delete duplicate bin number X00022 in location VAULT1 of media name CARTS.

```
RMM CHANGEVOLUME A00123 LOCATION(HOME) CMOVE  
RMM DELETEBIN X00022 MEDIANAME(CARTS) LOCATION(VAULT1)
```

Figure 15-17 RMM CHANGEVOLUME subcommands to change the location

The next time you run inventory management, the volume is reassigned to the storage location, and another bin number is assigned.

Note: When you need to reassign bin numbers, consider how off-site movements are managed in your installation. If you depend on the volume being in a specific slot in a storage location, ensure that any volumes that you change are physically moved in the storage location.

15.6.4 Step 3: Review the DFSMSrmm PARMLIB options

To allow for a simplified backout process, we recommend that you do not change the current DFSMSrmm PARMLIB member, but rather create a new merged version. On the merge day, you can simply rename the members to pick up the merged definitions.

Review the IEFSSNxx member, and confirm that the DFSMSrmm subsystem name and definitions are the same on all systems that will be sharing the same DFSMSrmm CDS. Also review the EDGRMMxx member for differences and consolidate the members. Be aware of any changes you make that require supporting changes to RACF. You should also investigate whether all systems can share the same EDGRMMxx PARMLIB member as opposed to having a separate member for each DFSMSrmm instance.

The EDGRMMxx options that are affected by the merging of CDSs are:

► OPTIONS

The options identify the installation options for DFSMSrmm. When you are merging CDSs, consolidate the OPTIONS command operands from all CDSs you are merging to ensure that you are using the same OPTIONS command operands on all DFSMSrmm subsystems.

After you merge DFSMSrmm CDSs, the CDSID of at least one DFSMSrmm subsystem will probably change. You must update the CDSID parameter to match the new CDSID, or if you have created a new DFSMSrmm CDS control record and you have not specified a CDSID, DFSMSrmm creates the ID in the control record from the CDSID automatically.

If the DFSMSrmm CDSs are kept in synch with the catalogs on the system, and you have enabled this using the inventory management CATSYNCH parameter, you should review the CATSYSID values specified for each system. If all systems sharing the merged DFSMSrmm CDS will also have access to the same user catalogs for tape data sets (which should be the case), you can use CATSYSID(*). However, if the catalogs are not fully shared, you must produce a consolidated list of systems IDs for use with the CATSYSID and set the correct values in EDGRMMxx for each system.

Before you use the merged CDS, we recommend that you mark the CDS as not in synch with the catalogs. Use the EDGUTIL program with UPDATE and the SYSIN statements with CATSYNCH(NO) to do this. After you start DFSMSrmm using the newly merged CDS, you can re-enable catalog synchronization using inventory management as detailed in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

If you use the VRSMIN option to specify a minimum number of VRSSs, remember to update the value if you have a different number as a result of the merge.

► **LOCDEF**

These options identify your locations to DFSMSrmm. When you merge CDSs, consolidate the LOCDEF options from all CDSs that you merge to ensure that you have no duplicates, and that no locations are left out. If you have duplicates, ensure that they are identified as the same location type and with the same media names.

► **VLPOOL**

These options identify the ranges of rack numbers and pools of shelf space you have in your library. If your library is not changing, we recommend that you do not change your current VLPOOL options.

If you merge CDSs and your existing VLPOOL definitions are different for each existing CDS, merge the definitions together to cover the merged sets of volumes and rack numbers. Ensure that the operands specified on each VLPOOL are correct when the same pool prefix is defined in more than one source environment.

► **REJECT**

These options identify the ranges of volumes to be rejected for use on the system. The reject specifies the rack number prefix, or if a volume has no library shelf location, DFSMSrmm checks the volume serial number. The range of volumes that you specify on the REJECT statement should match the VLPOOL definitions that you have defined in RMM.

For example, you may currently specify that the incoming system can only use volumes beginning with A*, and the target sysplex systems can only use volumes beginning with B*. In this case, you need to update the REJECT statement at the time of the merge to indicate that all those systems can use tapes starting with either A* *or* B*.

Where you have a tape library shared by multiple systems, define the same VLPOOL definitions to all systems and use the REJECT definitions to prevent systems from using volumes that are for use on other systems. The actions you take for the REJECT definitions should match how you handle the VLPOOL definitions.

You can use the REJECT option for volumes defined to DFSMSrmm, to control the use of volumes on the system. And for undefined volumes, you can use it to control the partitioning of system managed libraries. Consider both of these aspects as you merge the REJECT definitions.

► **OPENRULE**

These options identify special processing for DFSMSrmm to perform during OPEN processing. It applies equally to both system-managed and non-system managed volumes.

In a shared CDS RMMplex, these commands allow you to control sharing of volumes between systems while still maintaining a single CDS. OPENRULE allows you to specify what cannot be used as well as what can be used.

► **PRTITION**

These options identify special processing for DFSMSrmm to perform during library entry, export/import, eject, and CUA processing, during OPEN processing, and EXPROC processing. The PRTITION commands apply to all volumes, not only system-managed

volumes. However, only system-managed volumes benefit for the functions such as library insert, CUA, and eject.

15.6.5 Step 4: Review the DFSMSrmm started task JCL

To allow for a simplified backout process, we recommend that you do not change the current DFSMSrmm procedure, but rather create a new merged version. On the merge day, you can rename the two procedures to perform the merge.

Review the DFSMSrmm procedure and investigate whether all systems can share the same DFSMSrmm procedure. This should be possible with little or no work as the only system-specific information that is contained in the DFSMSrmm procedure is the names of the PDO (EDGPDOx) data sets. PDO data sets are optional, but if you do define them, they should be unique on each system. We suggest that you include the system name (&SYSNAME) in the data set name.

The DFSMSrmm CDS and Journal names can be specified in either the DFSMSrmm JCL or in the PARMLIB member EDGRMMxx. For flexibility, we recommend that they are specified in EDGRMMxx. This allows you to easily change the CDS name by switching to a different PARMLIB member.

You should also check that the REGION size specified for the DFSMSrmm started procedure is large enough. Review the latest recommendations for this in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

Step 5: Review the CDS and journal data set sizes and attributes

To allow for a simplified backout process, we recommend that you create new versions of the DFSMSrmm CDS and Journal data sets. On the merge day, you can rename the old ones, and rename the new ones to be ready to start DFSMSrmm after the merge.

Before you create the new data sets, calculate the new combined CDS size, and update the merge job with this information so that the CDS allocated in that job is valid. At the same time, calculate the new journal data set size, and create a job to allocate the data set.

Review the sample allocation jobs for the DFSMSrmm CDS, member EDGJMFAL in SYS1.SAMPLIB and for the DFSMSrmm journal, member EDGJNLAL in SYS1.SAMPLIB to ensure that you are defining the new CDS with the latest information regarding record size, CSize, and Shareoptions. For more information about these data sets, refer to *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

Step 6: TCDB tape configuration data set

If the systems to be merged contain Volume Entries cataloged inside either a General Tape Volume Catalog (SYS1.VOLCAT.VGENERAL, for example) or a specific Tape Volume Catalog (SYS1.VOLCAT.Vx, for example), you need to list all the volume entries and create the volume entries in the target TCDB with the same attributes as in the current system. You can use a REPRO MERGECAT to copy entries from the old to the new volume catalogs if required.

Confirm that the SMS ACS routines are allocating tape data sets to the appropriate storage groups associated with the tape libraries.

Review the DEVSUPxx member in SYS1.PARMLIB if the TCDB is being merged and make sure that all systems are entering the tapes with the same category number.

If the category number changes for any systems, there is no need to make any changes for private volumes. When the volume returns to scratch, its category changes to one of the new scratch categories. However, you must make changes for volumes already in Scratch status. Look for more information in the “Processing Default Categories when using DEVSUPxx in an ATLDs” section in *DFSMS Object Access Method Planning, Installation, and Storage Administration Guide for Tape Libraries*, SC35-0427.

Obtain a list of volumes whose storage group name is *SCRTCH* using the ISMF Option 2.3 Mountable Tape. Use the ISMF ALTER command (not the line operator) to change the volume use attribute for all volumes in the list from scratch to scratch. This causes the library manager category for each volume to change to the new value established through DEVSUPxx.

Step 7: Review any housekeeping jobs

Review any housekeeping jobs and confirm that data set names are valid for the new merged environment. It should be possible to eliminate any DFSMSrmm jobs that were running on the incoming system previously, except for any report jobs that were not running on the target sysplex.

If you switched on the catalog synchronization feature, check that the CATSYSID definitions are correct and that the inventory management vital record processing is running on the correct system. Also, the inventory management expiration processing must be checked and running on multiple systems if not all user catalogs are shared (hopefully, in a PlatinumPlex *all* catalogs are shared between all systems).

Step 8: Review security considerations

Chapter 8 in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, discusses security for DFSMSrmm. If the systems to be merged are merging the RACF environment at the same time, and you have set up the PARMLIB definitions the same on all systems, there should be no additional security concerns.

If either TAPEVOL or TAPEDSN are active, make sure all EDGRMMxx definitions for TPRACF are the same. If you did not switch on TAPEVOL or TAPEDSN on all systems before you merged the DFSMSrmm CDS, but now you will use them on all systems sharing the DFSMSrmm CDS, define all needed resources before the merged DFSMSrmm is started.

Step 9: Review DFSMSrmm exits

Determine what DFSMSrmm exits are in use (for example, EDGUX100, EDGUX200, CBRUXCUA, CBRUXENT, CBRUXEJC and CBRUXVNL). Review the functions of these exits on the incoming system and the target sysplex systems. If they are not the same, see if you can create a single version of each exit so that all systems will use identical exits, ensuring consistent processing.

15.6.6 Methodology for merging RMMplexes

You can choose from various approaches to merge RMMplexes. This section discusses one approach and provides some advice about the approach which provides the smallest risk or is the easiest to implement.

An alternate approach to the one discussed within this book is to create TSO RMM ADD... subcommands for all the volumes, data sets, racks, bins, and vital record specifications being merged. This approach can be used to avoid any outage to DFSMSrmm. Use care to ensure that the appropriate security options are already in place, for example, TAPEVOL/TAPEDSN(TVTOC). Since no tools are available to do this, you need to create these yourself.

The approach discussed in this document uses REPRO for the merging system's DFSMSrmm CDSs, sorts the resulting flat file, and creates a new merged DFSMSrmm CDS. Duplicate and unnecessary records are not included in the new CDS. Duplicates are written to sysout files for your review, and to confirm that you do not require them. DFSMSrmm is then restarted on all systems using the new merged DFSMSrmm CDS.

Merging the CDSs

Follow these steps to merge the CDSs:

1. Prior to the day of the merge, you should have addressed all the issues and tasks identified in 15.6.2, "Considerations for merging RMMplexes" on page 653.
2. Stop DFSMSrmm on all systems in both RMMplexes. When you run the merge job, DFSMSrmm on all systems must be down. When you stop DFSMSrmm, do *not* use the OPT=RESET function, as it allows tapes to be processed without DFSMSrmm's knowledge. You will not be able to use any tapes from this time until the merge is complete.
3. Back up both sets of CDSs and Journals using the sample JCL in Example 15-9 on page 668. This JCL uses REPRO to back up the files. While you should not change any of these files during the merge, we take the backups to protect you from any accidental damage. Also, by using REPRO for the backup, we can use the backup files as input to the merge process.
4. If required, perform any merging of Volume Entries in the Target TCDB Tape Catalog Database.
5. Run the merge job shown in Example 15-10 on page 668, using the data sets created in Step 3.

The sample merge job uses ICETOOL. If you do not have this program available, there is an alternate method using IDCAMS discussed in this chapter. If SYNCTOOL is available instead of ICETOOL, this job may run with little or no changes to the JCL and statements.

The sample merge job can be run while RMM is running and while you are performing testing. However, when you are doing the actual merge, all RMM subsystems should be stopped before you run the merge job.

The merge job consists of a number of steps. The steps, and the data sets used by each step are explained here:

a. Step S001ICET

The first step in the job simply deletes the output files that may have been left from the last run of this job.

b. Step S001ICET

The next step in the job uses the ICETOOL utility of DFSORT. The input data sets are sequential unloads of the two DFSMSrmm CDSs (created with REPRO) and are specified on the DD EXTRACT statement. If duplicates are discovered in the EXTRACT files, the first occurrence is retained and the second and subsequent ones are discarded, with the exceptions previously noted for the rack and bin records. Therefore, the order of the DD statements on the EXTRACT is critical. The first DD should refer to the flat file from the RMMplex whose CDSID you want to retain.

Whenever duplicate records are detected, the records from that RMMplex are retained and those from the other RMMplex are discarded. For this reason, be sure that any records that you want the merge job to discard are all in the same RMMplex.

The MAINCNTL DD statement contains the ICETOOL statements that will be used to sort and select most record types (C, K, P, and V). The associated SELECT statement in the TOOLIN DD specifies that duplicates should not be written the output file for these record types (MAIN1S), but should instead be written to DUPSMAIN for you to investigate and resolve.

The EFUXCNTL DD statement contains the ICETOOL statements that will be used to sort and select record types E, F, and U. They also ensure that the output file (EFU1S) will have only one record type for each rack. If the same rack number is discovered in more than one record type, records are selected with the “U” types first, followed by the “F” types, and finally the “E” types. Duplicate records are written to DUPSEFU for you to investigate and resolve.

The RSXXCNTL DD statement contains the ICETOOL statements that will be used to sort and select record types “R” and “S” into the data set specified on the RS1S DD statement. They also make sure that you have only one record type for each bin. If the same bin number is discovered in more than one record type, the “S” record is selected, and the “R” record is written to DUPSRS for you to investigate and resolve.

The OOOOCNTL DD statement contains the ICETOOL statements that will be used to sort and select type O records into the data set specified on the OO1S DD statement. The statement ensures that you have only one record for each owner. It also discards type O records that contain volume information (refer to “Owner record” on page 658).

Finally, the DDDDCNTL DD statement contains the ICETOOL statements that will be used to sort and select all the data set records. If you have addressed any duplicate volumes, there should be no issues with copying all data set records. The key includes the volser and the physical file number. Therefore, we have not provided a DUPSD DD statement as a destination for duplicate records.

c. Step S002ICET

This step also uses the ICETOOL Utility of DFSORT. The input data sets are the temporary data set created by S001ICET that contain the (non-duplicate) CDS records. An output data set (DD MERGED) is created in the same format as the input data sets for S001ICET.

d. Step S003AMS

This step uses IDCAMS to define a new DFSMSrmm CDS, and REPROs the sequential file created in S002ICET (DD MERGED) into the new CDS. This is the CDS that is used in the new merged RMMplex.

e. Step S004MEND

This step uses the EDGUTIL utility to repair all the record counts, set correct owners, and create missing records, as described previously. Running EDGUTIL with the MEND parameter only updates the CDS referred to on the MASTER DD card, so you can safely run this utility as many times as you like prior to the actual merge, *as long as you ensure that the MASTER DD points at your test CDS.*

If any of the volumes defined in the newly merged DFSMSrmm CDS are contained within an ATL or VTS, then the TCDB must already contain correct entries for these volumes. That is, SYS1.VOLCAT.VGENERAL must already have these volumes defined before the MEND can complete successfully. If all the volumes are inside an ATL or VTS, you can use the EDGUTIL MEND(SMSTAPE) function to add/correct the volumes in the TCDB.

Note: If you specify SMSTAPE, the TCDB is updated, do not use this option until you perform the actual merge.

f. Step S005MEND

If there are many records to be fixed, there can be quite a lot of output from EDGUTIL. To gain a clear picture of which records the MEND was unable to fix (and there should be *very* few of these), run the EDGUTIL program again, this time with the VERIFY parameter. This identifies any actions that MEND was not able to take, and the ones for which you need to consider taking action. The message that you are most likely to see is "VOLUME xxxxxx NOT FOUND IN VOLUME CATALOG". It is fixed when you run MEND with the SMSTAPE option.

6. Run EDGUTIL with UPDATE to turn off CATSYNCH. Turn off catalog synchronization before the merge is complete since the catalogs need to be re-synchronized. Use the sample job in Example 15-11 on page 671.
7. Perform any renames of CDSs, Journals, PARMLIB members, started tasks and JCL.
8. Start the DFSMSrmm procedure using the new PARMLIB member.
9. Check that the merged information is valid and correct by using DFSMSrmm commands, and the ISPF dialog to retrieve and display information.
10. Run EDGHSKP with CATSYNCH if catalog synchronization is to be maintained by DFSMSrmm. We recommend doing this for the performance benefits it can have for inventory management when policies are used on catalog retention. Do this before the first housekeeping job (EDGHSKP) is run after the merge.
11. If re-synchronization between the new merged DFSMSrmm CDS and TCDB is required, run the DFSMSrmm EDGUTIL program with a parameter of 'VERIFY(VOLCAT)'. In addition, if the systems in the RMMplex are all running OS/390 2.10 or later, specify 'VERIFY(SMSTAPE)'.
12. Run DFSMSrmm inventory management on the new CDS. During this run, DFSMSrmm VRSEL and DSTORE processing identifies volumes to be moved to the correct location. If any moves are identified, use EDGRPTD to produce the picking lists and actions of any moves that you know are still required.

The inventory management run should execute all inventory management functions including backup.

Backout

If backout is required at any stage, because you created new versions of the CDS, Journal, PARMLIB member and started task JCL, you can simply rename the old data sets back to the original names. There should be no problems as a result of the additional Volume Entries you may have created in the TCDB. However, they need to be cleaned up before re-attempting the merge process.

15.6.7 Tools and documentation

This section provides information about tools and documentation that may help you complete this project.

Example 15-9 contains sample JCL that you can use to back up the DFSMSrmm CDSs and Journal data sets after the DFSMSrmm subsystems are stopped. This job does *not* reset the contents of the Journal data sets. Even though we do not make any changes to any of these data sets during the merge process, it is prudent to make a backup of them before any changes are made, simply to protect yourself from any mistakes that may arise during the merge.

You can also use the JCL to create the input to the merge job for dummy runs in order to identify any duplicate records that may exist. However, if you run this JCL while the DFSMSrmm subsystems are running, you must comment out the //MASTER and //JOURNAL DD cards. If DFSMSrmm is *not* running when you run the EDGBKUP job, uncomment the MASTER and JOURNAL DD statements, and fill in the correct names for those data sets.

Example 15-9 JCL to back up the DFSMSrmm CDSs and journals

```
//S001BKUP EXEC PGM=EDGBKUP,PARM='BACKUP(NREORG)'
//SYSPRINT DD SYSOUT=*
//*MASTER DD DISP=SHR,DSN=RMMPLEX1.RMM.CDS
//*JOURNAL DD DISP=SHR,DSN=RMMPLEX1.RMM.JOURNAL
//BACKUP DD DSN=h1q.RMMPLEX1.CDS.BACKUP,DISP=(,CATLG,DELETE),
// DCB=(DSORG=PS,RECFM=VB,LRECL=9216),
// UNIT=SYSALLDA,SPACE=(CYL,(xxx,xx),RLSE)
//JRNLBKUP DD DSN=h1q.RMMPLEX1.JRNL.BACKUP,DISP=(,CATLG,DELETE),
// DCB=(DSORG=PS,RECFM=VB,LRECL=32756,BLKSIZE=32760),
// UNIT=SYSALLDA,SPACE=(CYL,(xxx,xx),RLSE)
//S002BKUP EXEC PGM=EDGBKUP,PARM='BACKUP(NREORG)'
//SYSPRINT DD SYSOUT=*
//*MASTER DD DISP=SHR,DSN=RMMPLEX2.RMM.CDS
//*JOURNAL DD DISP=SHR,DSN=RMMPLEX2.RMM.JOURNAL
//BACKUP DD DSN=h1q.RMMPLEX2.CDS.BACKUP,DISP=(,CATLG,DELETE),
// DCB=(DSORG=PS,RECFM=VB,LRECL=9216),
// UNIT=SYSALLDA,SPACE=(CYL,(xxx,xx),RLSE)
//JRNLBKUP DD DSN=h1q.RMMPLEX2.JRNL.BACKUP,DISP=(,CATLG,DELETE),
// DCB=(DSORG=PS,RECFM=VB,LRECL=32756,BLKSIZE=32760),
// UNIT=SYSALLDA,SPACE=(CYL,(xxx,xx),RLSE)
```

Example 15-10 contains the merge job that we referred to and discussed earlier in this chapter. You can also use this job to identify duplicate records during the preparation for the merge.

Example 15-10 DFSMSrmm merge job

```
//CLEANUP EXEC PGM=IEFBR14
//DD1 DD DSN=h1q.MERGED.RMM.MASTER.SEQ,DISP=(MOD,DELETE),
// SPACE=(CYL,0),UNIT=SYSDA
//DD2 DD DSN=h1q.MERGED.RMM.MASTER,DISP=(MOD,DELETE),
// SPACE=(CYL,0),UNIT=SYSDA
/*
//S001ICET EXEC PGM=ICETOOL
//TOOLMSG DD SYSOUT=*
//DFSMSG DD SYSOUT=*
//DFSPARM DD *
VLSHRT
/*
//DUPSMAIN DD SYSOUT=*
//DUPSEFU DD SYSOUT=*
```

```

//DUPSRs DD SYSOUT=*
//DUPSO DD SYSOUT=*
//EXTRACT DD DISP=SHR,DSN=h1q.RMMPLEX1.CDS.BACKUP
// DD DISP=SHR,DSN=h1q.RMMPLEX2.CDS.BACKUP
//MAIN DD DSN=&&MAIN,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//EFU DD DSN=&&EFU,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//RS DD DSN=&&RS,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//0000 DD DSN=&&0000,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//DDDD DD DSN=&&DDDD,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//MAIN1S DD DSN=&&MAIN1S,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//EFU1S DD DSN=&&EFU1S,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//RS1S DD DSN=&&RS1S,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//0001S DD DSN=&&0001S,REFDD=*.EXTRACT,SPACE=(CYL,(1,10),RLSE),
// UNIT=3390,DISP=(,PASS)
//TOOLIN DD *
SORT FROM(EXTRACT) TO(MAIN) USING(MAIN)
SELECT FROM(MAIN) TO(MAIN1S) ON(5,44,CH) FIRST -
DISCARD(DUPSMAN)
SORT FROM(EXTRACT) TO(EFU) USING(EFUX)
SELECT FROM(EFU) TO(EFU1S) ON(15,6,CH) FIRST -
DISCARD(DUPSEFU)
SORT FROM(EXTRACT) TO(RS) USING(RSXX)
SELECT FROM(RS) TO(RS1S) ON(15,6,CH) FIRST -
DISCARD(DUPSRs)
SORT FROM(EXTRACT) TO(0000) USING(0000)
SELECT FROM(0000) TO(0001S) ON(5,44,CH) FIRST -
DISCARD(DUPSO)
SORT FROM(EXTRACT) TO(DDDD) USING(DDDD)
//MAINCNL DD *
OPTION EQUALS
SORT FIELDS=(5,44,CH,A)
INCLUDE COND=(5,2,CH,NE,C'CA',AND,5,2,CH,NE,C'CM',AND,
5,1,CH,NE,C'O',AND,
5,1,CH,NE,C'D',AND,
5,1,CH,NE,C'R',AND,5,1,CH,NE,C'S',AND,
5,1,CH,NE,C'E',AND,5,1,CH,NE,C'F',AND,5,1,CH,NE,C'U')
/*
//EFUCNL DD *
OPTION EQUALS
SORT FIELDS=(5,1,CH,D,15,6,CH,A)
INCLUDE COND=(5,1,CH,EQ,C'E',OR,5,1,CH,EQ,C'F',OR,5,1,CH,EQ,C'U')
/*
//RSXCNL DD *
OPTION EQUALS
SORT FIELDS=(5,1,CH,D,15,6,CH,A)
INCLUDE COND=(5,1,CH,EQ,C'R',OR,5,1,CH,EQ,C'S')
/*

```

```

//0000CNTL DD *
  OPTION EQUALS
  SORT FIELDS=(5,1,CH,A)
  INCLUDE COND=(5,1,CH,EQ,C'0',AND,14,1,CH,EQ,X'00')
/*
//DDDDCNTL DD *
  OPTION EQUALS
  SORT FIELDS=(5,1,CH,A)
  INCLUDE COND=(5,1,CH,EQ,C'D')
/*
//S002ICET EXEC PGM=ICETOOL
//TOOLMSG DD SYSOUT=*
//DFSMSG DD SYSOUT=*
//DFSPARM DD *
VLSHRT
/*
//EXTRACT DD DISP=SHR,DSN=&&MAIN1S
//          DD DISP=SHR,DSN=&&EFU1S
//          DD DISP=SHR,DSN=&&RS1S
//          DD DISP=SHR,DSN=&&0001S
//          DD DISP=SHR,DSN=&&DDDD
//MERGED DD DSN=h1q.Merged.RMM.Master.Seq,
//          REFDD=*.EXTRACT,SPACE=(CYL,(xxx,yy),RLSE),
//          UNIT=SYSALLDA,DISP=(,CATLG)
//TOOLIN DD *
  SORT FROM(EXTRACT) TO(MERGED) USING(MOST)
//MOSTCNTL DD *
  OPTION EQUALS
  SORT FIELDS=(5,56,CH,A)
/*
//S003AMS EXEC PGM=IDCAMS,REGION=OM
//SYSPRINT DD SYSOUT=*,OUTLIM=1000000
//RMM DD DISP=SHR,DSN=h1q.Merged.RMM.Master.Seq
//SYSIN DD *
  DEFINE CLUSTER(NAME(h1q.Merged.RMM.Master) -
    FREESPACE(20 20) -
    KEYS(56 0) -
    REUSE -
    RECSZ(512 9216) -
    SHR(3 3) -
    CYLINDERS(xxx yy) -
    VOLUMES(vvvvvv)) -
    DATA(NAME(h1q.Merged.RMM.Master.DATA) -
    CISZ(10240)) -
    INDEX(NAME(h1q.Merged.RMM.Master.INDEX) -
    CISZ(2048) -
    NOIMBED -
    NOREPLICATE)
  REPRO IFILE(RMM) ODS(h1q.Merged.RMM.Master)
/*
//*-----
//S004MEND EXEC PGM=EDGUTIL,PARM='MEND'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=h1q.Merged.RMM.Master
//SYSIN DD DUMMY

```



```
//*-----  
//S005MEND EXEC PGM=EDGUTIL,PARM='VERIFY'  
//SYSPRINT DD SYSOUT=*  
//MASTER DD DISP=SHR,DSN=h1q.Merged.RMM.Master  
//SYSIN DD DUMMY
```

Example 15-11 contains sample JCL that you can use to turn off the CATSYNCH feature after you merge the CDSs, and before you start the DFSMSrmm subsystems using the new CDS.

Example 15-11 *CATSYNCH job*

```
//EDGUTIL EXEC PGM=EDGUTIL,PARM='UPDATE'  
//SYSPRINT DD SYSOUT=*  
//MASTER DD DISP=SHR,DSN=h1q.Merged.RMM.Master  
//SYSIN DD *  
CONTROL CATSYNCH(NO)  
/*
```

The following documents contain information that will be helpful as you proceed through this project:

- ▶ *DFSMSrmm Implementation and Customization Guide*, SC26-7405
- ▶ *DFSMSrmm Reporting*, SC26-7406
- ▶ *Merging Systems into a Sysplex*, SG24-6818



Security topics

This appendix provides the information you need to implement your installation security policies in DFSMSrmm using the FACILITY class profiles, and to activate RACF TAPEVOL and TAPEDSN classes. To protect DFSMSrmm functions, you need to use an external security product, such as RACF. You can use the information provided here, and the information provided in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405, and *z/OS Security Server RACF Security Administrator's Guide*, SA22-7683, for authorizing users and ensuring security.

While the focus is on RACF, other security products are covered where possible.

RACF implementation

This section takes you through the process of using RACF to implement security for DFSMSrmm resources and functions in your installation.

You must perform the following activities before you can implement DFSMSrmm:

- ▶ Assign a RACF user ID to DFSMSrmm
- ▶ Identify DFSMSrmm to RACF
- ▶ Define DFSMSrmm resources to RACF
- ▶ Define RACF groups for DFSMSrmm users
- ▶ Define DFSMSrmm resources to RACF
- ▶ Define the STGADMIN.ADR.DUMP.CNCURRNT resource

Note: You have to define the STGADMIN.ADR.DUMP.CNCURRNT resource only when you use DFSMSdss with concurrent copy.

Assigning a RACF user ID for DFSMSrmm

For a started procedure to access any of the system resources, a user ID must be associated with that task. The RACF user ID for DFSMSrmm does not need to match the name of the procedure name you created in PROCLIB; any installation-selected RACF user ID is acceptable.

The user ID should have all the proper authorizations for accessing the system resources. We recommend defining a new user ID for each started procedure rather than using the default started task user ID (for example, STCUSER). In our system, we associated user ID *DFRMM* with the DFSMSrmm started procedure.

All data sets are created for use by the DFSMSrmm procedure, add the DFSMSrmm RACF user ID to the access list for the data sets. We used the following RACF command shown in Figure A-1 to define the DFSMSrmm user ID.

```
ADDUSER DFRMM DFLTGRP(SYS1) NAME('DFSMSrmm Userid')
```

Figure A-1 Define a DFSMSrmm userid

Note: If you do not specify the DFLTGRP parameter on the ADDUSER command, the current connect group of the user issuing the ADDUSER command is used as the default.

Identifying DFSMSrmm to RACF

Before RACF 2.1, the only way to associate a started procedure with a RACF user ID was by coding the RACF started procedures table, ICHRIN03. With RACF 2.1, assigning RACF identities to started procedures has been greatly simplified by the introduction of the RACF STARTED class. You can add or modify security definitions for new and existing started procedures by issuing the RDEFINE and RALTER commands.

RACF STARTED class

The STARTED class allows you to assign RACF identities to started procedures dynamically using the RDEFINE and RALTER commands. Resource names in the STARTED class have the format *membername.jobname*. You assign identities such as the RACF user ID and group

ID, using fields in the STDATA segment. You can define the started procedure resource, using either a generic profile name or a discrete profile name.

An RACF generic profile describes one or more data sets that have a similar name structure. An RACF discrete profile describes a specific data set on a specific volume. In our system we created generic profiles for started procedures.

Issue the RACF commands shown in Figure A-2 to assign RACF identities to the DFSMSrmm started procedure and refresh the in-storage profiles, using the SETROPTS REFRESH command. The SETROPTS GENERIC command is needed only when you define generic profiles.

```
RDEFINE STARTED (DFRMM.*) UACC(NONE) STDATA(USER(DFRMM) GROUP(SYS1))
SETROPTS RACLIST(STARTED) REFRESH
SETROPTS GENERIC(STARTED) REFRESH
```

Figure A-2 Assign RACF identities to DFSMSrmm started procedure

Note: If you plan to use the EDGLABEL, EDGXPROC, or the EDGBKUP procedure, you must define the procedures in the RACF STARTED class.

Example A-1 shows a batch job that can be used to add the DFSMSrmm user ID in the RACF started class.

Example: A-1 JCL to add DFSMSrmm user ID in RACF STARTED class

```
RACFRMM JOB ,RMM,NOTIFY=SCHLUM,
//      MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
/* *****
/* Note
/* ====
/* To execute this job you need the RACF SPECIAL attribute
/* *****
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/* CREATE AN ENTRY IN THE DYNAMIC STARTED PROCEDURE TABLE FOR DFRMM */
/* ***** */
RDEFINE STARTED DFRMM.* -
          STDATA(USER(DFRMM) -
                  GROUP(SYS1) -
                  PRIVELEGED(NO) -
                  TRACE(NO) -
                  TRUSTED(NO) )
```

ICHRIN03 started procedures table

We do not recommend using the RACF started procedures table (ICHRIN03). The preferred way to add started procedure users to the RACF database is to use the STARTED class.

Defining DFSMSrmm resources to RACF

Define RACF FACILITY class profiles to control access to DFSMSrmm functions protected by the DFSMSrmm resource. The DFSMSrmm resources you protect with RACF profiles in the

FACILITY class each have an entity name prefixed with STGADMIN.EDG. Table A-1 lists the DFSMSrmm resources.

If you do not protect DFSMSrmm resources with RACF, an equivalent security product, or you do not have a security product in place, DFSMSrmm provides control for some resources as defined in the *DFSMSrmm Implementation and Customization Guide*, SC26-7405.

Table A-1 DFSMSrmm resources and protected functions

DFSMSrmm resource	Function protected
STGADMIN.EDG.ACTIONS.action ¹	Setting of the release action.
STGADMIN.EDG.AV.status.volser ²	Adding of volumes.
STGADMIN.EDG.CD.COPYFROM.dsname ⁴	Use of CHANGEDATASET COPYFROM subcommand to copy the data set attributes from one data set dsname to another data set.
STGADMIN.EDG.CD.VX ⁴	Overriding of DFSMSrmm VRSEL processing for a data set.
STGADMIN.EDG.CMOVE.location.destination	Confirmation of moves and ejects
STGADMIN.EDG.CRLSE.action ¹	Confirmation of the release action.
STGADMIN.EDG.CV.[HOLD NOHOLD].volser ³	Setting and resetting the volume HOLD attribute
STGADMIN.EDG.CV.RM3	Use of the RMM CHANGEVOLUME RETENTIONMETHOD subcommand to update the retention method for a volume.
STGADMIN.EDG.DV.SCRATCH.volser	Deleting of scratch volumes.
STGADMIN.EDG.FORCE	Changing of information recorded by DFSMSrmm during O/C/EOV processing. Use of the FORCE parameter that can be used in the ADDDDATASET, CHANGEDATASET, CHANGEVOLUME, and DELETEDATASET subcommand to changing information recorded by DFSMSrmm during O/C/EIV processing
STGADMIN.EDG.EDGUPDT.UPDATE	Use of the EDGUPDT utility UPDATE function.
STGADMIN.EDG.HOUSEKEEP	Use of DFSMSrmm inventory management functions
STGADMIN.EDG.HOUSEKEEP. REPEXT	Use to authorize the creation of report extract files when no other inventory management function is requested
STGADMIN.EDG.IGNORE.TAPE. volser	Use of duplicate volume serial numbers and ignored volumes Recommendation: Do not assign an access level to the STGADMIN.EDG.IGNORE.TAPE.RMM.volser resource to any specific user group. Instead, wait until a tape volume that must be ignored by DFSMSrmm is identified, to define a resource granting a user or user group the needed access level. When the volume is no longer needed, delete the resource.
STGADMIN.EDG.IGNORE.TAPE.RMM.volser	Use of duplicate volume serial numbers and ignoring volumes if the volume is defined to DFSMSrmm Recommendation: Do not assign an access level to the STGADMIN.EDG.IGNORE.TAPE.RMM.volser resource to any specific user group. Instead, wait until a tape volume that must be ignored by DFSMSrmm is identified, to define a resource granting a user or user group the needed access level. When the volume is no longer needed, delete the resource.

DFSMSrmm resource	Function protected
STGADMIN.EDG.IGNORE.TAPE. NORMM.volser	Use of duplicate volume serial numbers and ignoring volumes if the volume is not defined to DFSMSrmm
STGADMIN.EDG.INIT	Setting of the INIT action.
STGADMIN.EDG.LABEL.volser	Creation of standard tape labels
STGADMIN.EDG.LIST	List and search DFSMSrmm resources.
STGADMIN.EDG.LISTCONTROL	Use of the RMM LISTCONTROL subcommand to display DFSMSrmm CDS control record information and EDGRMMxx PARMLIB settings
STGADMIN.EDG.MASTER	Access to information in the DFSMSrmm CDS
STGADMIN.EDG.MOVES.location.destination	Initiation of moves and ejects.
STGADMIN.EDG.NOLABEL.volser	Creation of tapes without labels
STGADMIN.EDG.OPERATOR	Use of the initialize and erase function
STGADMIN.EDG.OWNER. owner	Access to owned resources. DFSMSrmm checks this entity only if the command issuer is not the owner of the resource or does not have CONTROL access to STDADMIN.EDG.MASTER
STGADMIN.EDG.RELEASE	Use of the RMM DELETEVOLUME RELEASE subcommand to process any release actions specified for a volume
STGADMIN.EDG.RESET.SSI	Use of the RESET facility for removing DFSMSrmm from the system
STGADMIN.EDG.VRS	Use of the RMM LISTVRS or SEARCHVRS subcommands to obtain information about VRSs. Use of the RMM ADDVRS and DELETEVRS subcommands to define or remove vital record specifications
STGADMIN.EDG.INERS.WRONGLABEL	Processing for volume mounted with the wrong label using the EDGINERS initialization process
Notes: 1 Action can be either SCRATCH, RETURN, REPLACE, NOTIFY, ERASE, or INIT. 2 Status can be either SCRATCH, USER, MASTER, or VOLCAT. 3 If you use a generic profile, the minimum non-generic profile name checked for by DFSMSrmm is 'STGADMIN.EDG.CV.' 4 If you use a generic profile, the minimum non-generic profile name checked for by DFSMSrmm is 'STGADMIN.EDG.CD.'	

Defining RACF groups for DFSMSrmm users

To implement RACF security for DFSMSrmm resources, you should define a set of user groups to handle different kinds of resources at different access levels. Here is our suggestion for function groups:

General user

They may want to manage volumes they own and request information about resources defined to DFSMSrmm.

Storage administrator

These persons are responsible for specifying the policies to retain and move data sets and volumes.

System programmer The system programmer is responsible for the DFSMSrmm installation and implementation, and provides support for DFSMSrmm in the computing center.

Librarian The librarian creates scratch pull lists, moves the tapes between the built-in locations (LOCAL, DISTANT, REMOTE, SHELF), installation defined storage locations, and system managed libraries.

Inventory management functions
The inventory management function is responsible for the daily and weekly DFSMSrmm inventory management.

Operator The operator handles DFSMSrmm system requests including initializing and erasing tapes.

To implement RACF security for DFSMSrmm resources, we define a RACF group for each of the user groups, connect the user IDs to the appropriate function group, and permit only the RACF groups to the resources in the FACILITY class and to the DFSMSrmm data set profiles.

Table A-2 provides suggestions for authorizing different types of users. Customers should implement security authorization to DFSMSrmm resources according to their working environments.

Table A-2 Access level for each group

Resource	General User	System Administrator	System Programmer	Librarian	Inventory Management Function	Operator
STGADMIN.EDG.ACTIONS.action	*	*	*	*	*	*
STGADMIN.EDG.AV.status.volser	*	*	*	*	*	*
STGADMIN.EDG.CD.COPYFROM.dsname	*	*	*	*	*	*
STGADMIN.EDG.CD.VX	*	*	*	*	*	*
STGADMIN.EDG.CMOVE.location.destination	*	*	*	*	*	*
STGADMIN.EDG.CRLSE.action	*	*	*	*	*	*
STGADMIN.EDG.CV.HOLD.volser	*	*	*	*	*	*
STGADMIN.EDG.CV.RM	*	*	*	*	*	*
STGADMIN.EDG.DV.SCRATCH.volser	*	*	*	*	*	*
STGADMIN.EDG.EDGUPDT.UPDATE	-					
STGADMIN.EDG.FORCE	-	U	-	U	-	-
STGADMIN.EDG.HOUSEKEEP	-	R	-	R	R	-
STGADMIN.EDG.HOUSEKEEP. REPEXT	-	-	-	-	-	-
STGADMIN.EDG.IGNORE.TAPE. volser	tbd	tbd	tbd	tbd	tbd	tbd
STGADMIN.EDG.IGNORE.TAPE.RMM.volser	tbd	tbd	tbd	tbd	tbd	tbd

Resource	General User	System Administrator	System Programmer	Librarian	Inventory Management Function	Operator
STGADMIN.EDG.IGNORE.TAPE. NORMM.volser	tbd	tbd	tbd	tbd	tbd	tbd
STGADMIN.EDG.INIT	*	*	*	*	*	*
STGADMIN.EDG.LABEL.volser	-	-	-	-	-	-
STGADMIN.EDG.LIST	*	*	*	*	*	*
STGADMIN.EDG.LISTCONTROL	*	*	*	*	*	*
STGADMIN.EDG.MASTER	R	C	R	C	R	R
STGADMIN.EDG.MOVES.location.destination	*	*	*	*	*	*
STGADMIN.EDG.CV.NOHOLD.volser	*	*	*	*	*	*
STGADMIN.EDG.NOLABEL.volser	-	-	-	-	-	-
STGADMIN.EDG.OPERATOR	-	U	R	U	-	U
STGADMIN.EDG.OWNER. owner	tbd	tbd	tbd	tbd	tbd	tbd
STGADMIN.EDG.RELEASE	R	R	R	R	R	R
STGADMIN.EDG.RESET.SSI	-	-	A	-	-	-
STGADMIN.EDG.VRS	R	C	R	R	R	R
STGADMIN.EDG.INERS.WRONGLABEL	-	C	-	C	-	-
Note: A Alter access C Control access R Read access U Update access - Not used TBD to be defined * If Entity not defined access is based on STGADMIN.EDG.MASTER access						

Example A-2 shows a batch job to create the DFSMSrmm RACF groups and connect user IDs to the appropriate group. You can use this JCL as a base and modify it to fit your installation standards.

Example: A-2 JCL to create RACF groups and connect user IDs

```
//RACFGRP JOB ,RMM,NOTIFY=SCHLUM,
//      MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
//* *****
/* Note
/* ====
/* To execute this job you need the RACF SPECIAL attribute
/* *****
```

```

//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/* DEFINE RACF ACCESS GROUP FOR ADMINISTRATOR */
/* ***** */
ADDGROUP EDGADMIN -
    SUBGROUP(SYS1) OWNER(SYS1)
    CONNECT WOODY GROUP(EDGADMIN) AUTHORITY(USE)
    CONNECT KUEHN GROUP(EDGADMIN) AUTHORITY(USE)
    CONNECT GOHR GROUP(EDGADMIN) AUTHORITY(USE)
    CONNECT ETZ GROUP(EDGADMIN) AUTHORITY(USE)
    CONNECT BENGT GROUP(EDGADMIN) AUTHORITY(USE)
    CONNECT NELSON GROUP(EDGADMIN) AUTHORITY(USE)
/* ***** */
/* DEFINE RACF ACCESS GROUP FOR SYSTEM PROGRAMMER */
/* ***** */
ADDGROUP EDGSYSPG -
    SUBGROUP(SYS1) OWNER(SYS1)
    CONNECT SCHLUM GROUP(EDGSYSPG) AUTHORITY(USE)
    CONNECT SIEGEL GROUP(EDGSYSPG) AUTHORITY(USE)
/* ***** */
/* DEFINE RACF ACCESS GROUP FOR LIBRARIAN */
/* ***** */
ADDGROUP EDGLIB -
    SUBGROUP(SYS1) OWNER(SYS1)
    CONNECT JOY GROUP(EDGLIB) AUTHORITY(USE)
    CONNECT NADINE GROUP(EDGLIB) AUTHORITY(USE)
    CONNECT ZERBINI GROUP(EDGLIB) AUTHORITY(USE)
    CONNECT STRAUS GROUP(EDGLIB) AUTHORITY(USE)
    CONNECT SEYFERT GROUP(EDGLIB) AUTHORITY(USE)
/* ***** */
/* DEFINE RACF ACCESS GROUP FOR INVENTORY MANAGEMENT FUNCTION */
/* ***** */
ADDGROUP EDGJOB -
    SUBGROUP(SYS1) OWNER(SYS1)
    CONNECT WILDEN GROUP(EDGJOB) AUTHORITY(USE)
    CONNECT HENICKE GROUP(EDGJOB) AUTHORITY(USE)
    CONNECT KEIL GROUP(EDGJOB) AUTHORITY(USE)
/* ***** */
/* DEFINE RACF ACCESS GROUP FOR INVENTORY MANAGEMENT FUNCTION */
/* ***** */
ADDGROUP EDGOPER -
    SUBGROUP(SYS1) OWNER(SYS1)
    CONNECT TAUBER GROUP(EDGOPER) AUTHORITY(USE)
    CONNECT ZINK GROUP(EDGOPER) AUTHORITY(USE)
    CONNECT LOVELACE GROUP(EDGOPER) AUTHORITY(USE)

```

RACF data set profiles

To protect the DFSMSrmm data sets, we define discrete profiles for the following data sets, and then permit the appropriate RACF groups to the following RACF data set profiles:

- ▶ DFSMSrmm control data set
- ▶ DFSMSrmm journal data set
- ▶ DFSMSrmm message data set
- ▶ DFSMSrmm housekeeping data sets
- ▶ DFSMSrmm report data sets

► DFSMSRmm security report data sets

To protect all other DFSMSRmm data sets, define a generic data set profile using the high level qualifier of your DFSMSRmm data sets.

Note: Refer to the *z/OS Security Server RACF Command Language Reference*, SA22-7687, for enhanced generic naming considerations. The enhanced generic naming option applies only to data sets and allows you to use a double asterisk (**) in the DATASET class. It also changes the meaning of the single asterisk (*) at the end of a profile name.

Example A-3 on page 681 shows a batch job to define all the DFSMSRmm data set profiles and permit the appropriate RACF groups and DFSMSRmm procedure to the data set profiles.

Example: A-3 JCL to define DFSMSRmm RACF data set profiles

```
//RACFDSN JOB ,RMM,NOTIFY=SCHLUM,
//      MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
/* *****
/* Note
/* ====
/* To execute this job you need the RACF SPECIAL attribute
/* *****
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/* DEFINE RACF GROUP */
/* ***** */
ADDGROUP RMM -
        SUBGROUP(SYS1) OWNER(SYS1)
/* ***** */
/* DEFINE GLOBAL DATA SET PROFILE */
/* ***** */
ADDSD 'RMM.*' UACC(NONE)
PERMIT 'RMM.*' -
        ID(EDGADMIN) ACC(NONE)
PERMIT 'RMM.*' -
        ID(EDGLIB) ACC(NONE)
PERMIT 'RMM.*' -
        ID(EDGJOB) ACC(NONE)
PERMIT 'RMM.*' -
        ID(EDGOPER) ACC(NONE)
PERMIT 'RMM.*' -
        ID(EDGSYSPG) ACC(NONE)
/* ***** */
/* DEFINE CONTROL DATA SET DATA SET PROFILE */
/* ***** */
ADDSD 'RMM.PROD.CDS' UACC(NONE) GENERIC
PERMIT 'RMM.PROD.CDS' GENERIC-
        ID(EDGADMIN) ACC(READ)
PERMIT 'RMM.PROD.CDS' GENERIC-
        ID(EDGLIB) ACC(READ)
PERMIT 'RMM.PROD.CDS' GENERIC-
        ID(EDGSYSPG) ACC(ALTER)
/* ***** */
/* DEFINE JOURNAL DATA SET DATA SET PROFILE */
/* ***** */
```

```

ADDSD 'RMM.PROD.JRNL' UACC(NONE) GENERIC
  PERMIT 'RMM.PROD.JRNL' GENERIC-
    ID(EDGADMIN) ACC(READ)
  PERMIT 'RMM.PROD.JRNL' GENERIC-
    ID(EDGLIB) ACC(READ)
  PERMIT 'RMM.PROD.JRNL' GENERIC-
    ID(EDGSYSPG) ACC(ALTER)
/* ***** */
/*  DEFINE MESSAGE DATA SET DATA SET PROFILE */
/* ***** */
ADDSD 'RMM.PROD.MSGS' UACC(NONE) GENERIC
  PERMIT 'RMM.PROD.MSGS' GENERIC-
    ID(EDGADMIN) ACC(UPDATE)
  PERMIT 'RMM.PROD.MSGS' GENERIC-
    ID(EDGLIB) ACC(UPDATE)
  PERMIT 'RMM.PROD.MSGS' GENERIC-
    ID(EDGSYSPG) ACC(ALTER)
/* ***** */
/*  DEFINE HOUSEKEEP DATA SET DATA SET PROFILE */
/* ***** */
ADDSD 'RMM.HSKP.*' UACC(NONE)
  PERMIT 'RMM.HSKP.*' -
    ID(EDGADMIN) ACC(ALTER)
  PERMIT 'RMM.HSKP.*' -
    ID(EDGLIB) ACC(ALTER)
  PERMIT 'RMM.HSKP.*' -
    ID(EDGSYSPG) ACC(ALTER)
  PERMIT 'RMM.HSKP.*' -
    ID(EDGOPER) ACC(ALTER)
  PERMIT 'RMM.HSKP.*' -
    ID(EDGJOB) ACC(ALTER)
/* ***** */
/*  DEFINE REPORT DATA SET DATA SET PROFILE */
/* ***** */
ADDSD 'RMM.REPORT.*' UACC(NONE)
  PERMIT 'RMM.REPORT.*' -
    ID(EDGADMIN) ACC(ALTER)
  PERMIT 'RMM.REPORT.*' -
    ID(EDGLIB) ACC(ALTER)
  PERMIT 'RMM.REPORT.*' -
    ID(EDGSYSPG) ACC(ALTER)
  PERMIT 'RMM.REPORT.*' -
    ID(EDGOPER) ACC(ALTER)
  PERMIT 'RMM.REPORT.*' -
    ID(EDGJOB) ACC(ALTER)
/* ***** */
/*  DEFINE SECURITY DATA SET DATA SET PROFILE */
/* ***** */
ADDSD 'RMM.SECURITY.*' UACC(NONE)
  PERMIT 'RMM.SECURITY.*' -
    ID(EDGSYSPG) ACC(ALTER)
  PERMIT 'RMM.SECURITY.*' -
    ID(EDGJOB) ACC(ALTER)
  PERMIT 'RMM.SECURITY.*' -
    ID(AUDITOR) ACC(READ)
/* ***** */
/*  PERMIT DFRMM TO THE CONTROL DATA SET, JOURNAL DATA SET,
/*  EXTRACT DATA SET, ACTIVITY LOG AND REPORT DATA SET */
/* ***** */
  PERMIT 'RMM.MASTER.CDS' GENERIC-

```

```

        ID(DFRMM) ACC(CONTROL)
    PERMIT 'RMM.MASTER.JOURNAL' GENERIC-
        ID(DFRMM) ACC(UPDATE)
    PERMIT 'RMM.MASTER.MSGS' GENERIC-
        ID(DFRMM) ACC(UPDATE)
    PERMIT 'RMM.HSKP*' -
        ID(DFRMM) ACC(UPDATE)
/*

```

RACF resource profiles

The following DFSMSrmm resources, described in Table A-1 on page 676, require RACF profiles in the FACILITY class:

- ▶ STGADMIN.EDG.ACTIONS.action
- ▶ STGADMIN.EDG.AV.status.volser
- ▶ STGADMIN.EDG.CD.COPYFROM.dsname
- ▶ STGADMIN.EDG.CD.VX
- ▶ STGADMIN.EDG.CMOVE.location.destination
- ▶ STGADMIN.EDG.CRLSE.action
- ▶ STGADMIN.EDG.CV.HOLD.volser
- ▶ STGADMIN.EDG.CV.RM
- ▶ STGADMIN.EDG.DV.SCRATCH.volser
- ▶ STGADMIN.EDG.EDGUPDT.UPDATE
- ▶ STGADMIN.EDG.FORCE
- ▶ STGADMIN.EDG.HOUSEKEEP
- ▶ STGADMIN.EDG.HOUSEKEEP.REPEXT
- ▶ STGADMIN.EDG.IGNORE.TAPE.volser
- ▶ STGADMIN.EDG.IGNORE.TAPE.RMM.volser
- ▶ STGADMIN.EDG.IGNORE.TAPE.NORMM.volser
- ▶ STGADMIN.EDG.INIT
- ▶ STGADMIN.EDG.LABEL.volser
- ▶ STGADMIN.EDG.LIST
- ▶ STGADMIN.EDG.LISTCONTROL
- ▶ STGADMIN.EDG.MASTER
- ▶ STGADMIN.EDG.MOVES.location.destination
- ▶ STGADMIN.EDG.CV.NOHOLD.volser
- ▶ STGADMIN.EDG.NOLABEL.volser
- ▶ STGADMIN.EDG.OPERATOR
- ▶ STGADMIN.EDG.OWNER.owner
- ▶ STGADMIN.EDG.RELEASE
- ▶ STGADMIN.EDG.RESET.SSI
- ▶ STGADMIN.EDG.VRS
- ▶ STGADMIN.EDG.INERS.WRONGLABEL

Example A-4 shows a batch job to define the resource profiles in the RACF FACILITY class and permits the RACF function groups as described in Table A-2 on page 678.

Example: A-4 JCL to define DFSMSrmm resources in the FACILITY class

```

//RACFRDEF JOB ,RMM,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
/* ***** */
/* Note
/* ====
/* To execute this job you need the RACF SPECIAL attribute
/* ***** */

```

```

//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.FORCE */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.FORCE -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.FORCE -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.FORCE -
        ID(EDGSYSPG) ACC(READ) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.FORCE -
        ID(EDGLIB) ACC(READ) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.HOUSEKEEP */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.HOUSEKEEP -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.HOUSEKEEP -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.HOUSEKEEP -
        ID(EDGSYSPG) ACC(READ) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.HOUSEKEEP -
        ID(EDGLIB ) ACC(READ) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.HOUSEKEEP -
        ID(EDGJOB ) ACC(READ) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.HOUSEKEEP.RPTEXT */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.HOUSEKEEP.RPTEXT -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.HOUSEKEEP.RPTEXT -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.HOUSEKEEP.RPTEXT -
        ID(EDGSYSPG) ACC(READ) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.HOUSEKEEP -
        ID(EDGLIB ) ACC(READ) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.HOUSEKEEP.RPTEXT -
        ID(EDGJOB ) ACC(READ) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.IGNORE.TAPE.* */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.* -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.IGNORE.TAPE.* -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.IGNORE.TAPE.* -
        ID(EDGSYSPG) ACC(READ) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.IGNORE.TAPE.* -
        ID(EDGJOB ) ACC(UPDATE) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.IGNORE.TAPE.NORMM.* */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.* -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.* -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.IGNORE.TAPE.NORMM.* -

```

```

        ID(EDGSYSPG) ACC(READ) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.IGNORE.TAPE.NORMM.* -
        ID(EDGJOB ) ACC(UPDATE) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.IGNORE.TAPE.RMM.* */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.* -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.* -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.IGNORE.TAPE.RMM.* -
        ID(EDGSYSPG) ACC(READ) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.IGNORE.TAPE.RMM.* -
        ID(EDGJOB ) ACC(UPDATE) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.LABEL.* */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.LABEL.* -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.LABEL.* -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.LABEL.* -
        ID(EDGLIB) ACC(ALTER) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.LABEL.* -
        ID(EDGSYSPG) ACC(ALTER) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.LABEL.* -
        ID(EDGSYSPG) ACC(ALTER) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.LISTCONTROL */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.LISTCONTROL -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.LISTCONTROL -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.LISTCONTROL -
        ID(EDGADMIN) ACC(CONTROL) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.LISTCONTROL -
        ID(EDGSYSPG) ACC(CONTROL) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.LISTCONTROL -
        ID(EDGLIB ) ACC(CONTROL) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.LISTCONTROL -
        ID(EDGJOB) ACC(CONTROL) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.MASTER */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.MASTER -
    UACC(READ) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.MASTER -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.MASTER -
        ID(EDGADMIN) ACC(CONTROL) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.MASTER -
        ID(EDGSYSPG) ACC(CONTROL) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.MASTER -
        ID(EDGLIB) ACC(CONTROL) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.MASTER -
        ID(EDGJOB) ACC(CONTROL) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.MASTER -
        ID(EDGOPER) ACC(CONTROL) CLASS(FACILITY)
/* ***** */

```

```

/* DEFINE RESOURCE STGADMIN.EDG.NOLABEL.* */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.NOLABEL.* -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.NOLABEL.* -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.NOLABEL.* -
        ID(EDGADMIN) ACC(ALTER) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.NOLABEL.* -
        ID(EDGLIB ) ACC(ALTER) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.NOLABEL.* -
        ID(EDGOPER ) ACC(ALTER) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.OPERATOR */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.OPERATOR -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.OPERATOR -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.OPERATOR -
        ID(EDGSYSPG) ACC(UPDATE) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.OPERATOR -
        ID(EDGLIB) ACC(UPDATE) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.OPERATOR -
        ID(EDGOPER) ACC(UPDATE) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.OWNER.* */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.OWNER.* -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.OWNER.* -
        GLOBALAUDIT(ALL(UPDATE))
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.RELEASE */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.RELEASE -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.RELEASE -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.RELEASE -
        ID(DFHSM) ACC(READ) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.RESET.SSI */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.RESET.SSI -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.RESET.SSI -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.RESET.SSI -
        ID(EDGSYSPG) ACC(ALTER) CLASS(FACILITY)
    PERMIT STGADMIN.EDG.RESET.SSI -
        ID(EDGOPER) ACC(ALTER) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.VRS */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.VRS -
    UACC(NONE) OWNER(SYS1)
    RALT FACILITY STGADMIN.EDG.VRS -
        GLOBALAUDIT(ALL(UPDATE))
    PERMIT STGADMIN.EDG.VRS -

```



```

        ID(EDGADMIN) ACC(CONTROL) CLASS(FACILITY)
PERMIT STGADMIN.EDG.VRS -
        ID(EDGSYSPG) ACC(CONTROL) CLASS(FACILITY)
PERMIT STGADMIN.EDG.VRS -
        ID(EDGLIB) ACC(CONTROL) CLASS(FACILITY)
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.INERS.WRONGLABEL */
/* ***** */
RDEFINE FACILITY STGADMIN.EDG.INERS.WRONG.LABEL -
        UACC(NONE) OWNER(SYS1)
RALT FACILITY STGADMIN.EDG.INERS.WRONG.LABEL -
        GLOBALAUDIT(ALL(UPDATE))
PERMIT STGADMIN.EDG.INERS.WRONG.LABEL -
        ID(EDGOPER) ACC(UPDATE) CLASS(FACILITY)
PERMIT STGADMIN.EDG.INERS.WRONG.LABEL -
        ID(EDGSYSPG) ACC(CONTROL) CLASS(FACILITY)
PERMIT STGADMIN.EDG.INERS.WRONG.LABEL -
        ID(EDGLIB) ACC(CONTROL) CLASS(FACILITY)

```

Protecting DFSMSdss resources

You can use EDGHSKP or EDGBKUP with DFSMSdss DUMP, or the access method services (AMS) REPRO command to create a backup copy of your DFSMSrmm CDS and journal data set. To use DFSMSdss concurrent copy, you must define or update the STGADMIN.ADR.DUMP.CNCURRNT profile in the RACF FACILITY class.

Example A-5 shows a batch job to define the resource in the RACF FACILITY class.

Example: A-5 JCL to define DFSMSdss RACF FACILITY class profile

```

//RACFRDEF JOB ,RMM,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
/* ***** */
/* Note
/* ====
/* To execute this job you need the RACF SPECIAL attribute
/* ***** */
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/* DEFINE RESOURCE STGADMIN.EDG.CNCURRNT */
/* ***** */
RDEFINE FACILITY STGADMIN.ADR.DUMP.CNCURRNT -
        UACC(NONE) OWNER(SYS1 )
PERMIT STGADMIN.ADR.DUMP.CNCURRNT-
        ID(EDGADMIN) ACC(READ) CLASS(FACILITY)

```

Authorizing DFSMShsm to DFSMSrmm resources

DFSMShsm can provide enhanced management functions for the tape volumes that DFSMSrmm uses for each of its tape functions. DFSMSrmm treats DFSMShsm like any other tape user, not like a tape manager.

To run DFSMShsm with DFSMSrmm, DFSMShsm procedure names must be defined to RACF. Add the name of the DFSMShsm procedure to the RACF STARTED class or the

started procedure table ICHRIN03. Use a DFSMSHsm user ID other than the default user ID. Refer to *z/OS DFSMSHsm Storage Administrators Guide*, SH35-0421, for information about defining the DFSMSHsm and ABARS procedure names.

Before you can use DFSMSrmm with DFSMSHsm, you must authorize DFSMSHsm to the following resources:

- ▶ STGADMIN.EDG.MASTER
- ▶ STGADMIN.EDG.RELEASE
- ▶ STGADMIN.EDG.OWNER.*hsmid*

If you have multiple DFSMSHsm user IDs, for example, in a multisystem environment, and any DFSMSHsm ID can return tapes to scratch status or to the DFSMSHsm tape pool, you must authorize each DFSMSHsm user ID. Define a resource profile STGADMIN.EDG.OWNER.*hsmid* for each DFSMSHsm user ID, and give the other DFSMSHsm user IDs UPDATE access to it.

When using DFSMSrmm with DFSMSHsm, DFSMSHsm needs access to the following resources to use scratch tapes:

- ▶ **STGADMIN.EDG.MASTER:** READ
- ▶ **STGADMIN.EDG.RELEASE:** READ
- ▶ **STGADMIN.EDG.OWNER.*hsmid*:** UPDATE

DFSMSHsm needs access to the following resources to use a DFSMSHsm- managed scratch tape pool:

- ▶ **STGADMIN.EDG.MASTER:** UPDATE
- ▶ **STGADMIN.EDG.OWNER.*hsmid*:** UPDATE

Example A-6 shows a batch job to add the name of the DFSMSHsm procedure in the STARTED class, and permits the DFSMSHsm user ID to the required DFSMSrmm RACF FACILITY class profiles.

Example: A-6 JCL to define DFSMSHsm to DFSMSrmm resources

```
//RACFHSM JOB ,RMM,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
//* *****
//* Note
//* ====
//* To execute this job you need the RACF SPECIAL attribute
//* *****
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/* Create an entry in the dynamic STARTED procedure table for DFHSM */
/* ***** */
RDEFINE STARTED DFHSM.* -
          STDATA(USER(DFHSM) -
                  GROUP(SYS1) -
                  PRIVELEGED(NO) -
                  TRACE(NO) -
                  TRUSTED(NO) )
/* ***** */
/* Permit DFHSM to STGADMIN.EDG.nnnnn resources */
/* ***** */
PERMIT STGADMIN.EDG.MASTER -
      ID(DFHSM) ACC(READ) CLASS(FACILITY)
```

```

PERMIT STGADMIN.EDG.RELEASE -
      ID(DFHSM) ACC(READ) CLASS(FACILITY)
PERMIT STGADMIN.EDG.OWNER.DFHSM*
      ID(DFHSM) ACC(UPDATE) CLASS(FACILITY)

```

Authorizing ABARS to DFSMSrmm resources

To run ABARS with DFSMSrmm, the ABARS procedure names must be defined to RACF. Add the name of the ABARS procedure to the RACF STARTED class or to the started procedure table ICHRIN03. Refer to the *z/OS DFSMSHsm Storage Administrators Guide*, SH35-0421, for information about defining the DFSMSHsm and ABARS procedure names to RACF.

To use DFSMSrmm with ABARS, you must authorize ABARS to the following resources:

- ▶ STGADMIN.EDG.MASTER
- ▶ STGADMIN.EDG.RELEASE
- ▶ STGADMIN.EDG.OWNER.*abarsid*

If you have multiple ABARS user IDs, for example, in a multisystem environment, and any ABARS ID can return tapes to scratch status, you must authorize each ABARS user ID. Define a RACF resource profile STGADMIN.EDG.OWNER.*abarsid* for each ABARS user ID, and give the other ABARS user IDs UPDATE access to it. This allows one ABARS to release the tapes initially obtained from scratch by another ABARS procedure.

The ABARS user ID needs access to the following resources to use DFSMSrmm with ABARS:

- ▶ **STGADMIN.EDG.MASTER:** READ
- ▶ **STGADMIN.EDG.RELEASE:** READ
- ▶ **STGADMIN.EDG.OWNER.*abarsid*:** UPDATE

Example A-7 shows a batch job to add the name of the ABARS procedure in the STARTED class, and permit the ABARS user ID to the required DFSMSrmm RACF FACILITY class resources.

Example: A-7 JCL to define ABARS to DFSMSrmm resources

```

//RACFABAR JOB ,RMM,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
//* *****
//* Note
//* ====
//* To execute this job you need the RACF SPECIAL attribute
//* *****
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/* Create an entry in the dynamic STARTED procedure table for ABARS */
/* ***** */
RDEFINE STARTED ABARS.* -
      STDATA(USER(ABARS ) -
      GROUP(SYS1 ) -
      PRIVELEGED(NO) -
      TRACE(NO) -
      TRUSTED(NO) )
/* ***** */

```

```

/* Permit ABARS to STGADMIN.EDG.nnnnn resources */
/* ***** */
PERMIT STGADMIN.EDG.MASTER -
      ID(ABARS) ACC(READ) CLASS(FACILITY)
PERMIT STGADMIN.EDG.RELEASE -
      ID(ABARS) ACC(READ) CLASS(FACILITY)
PERMIT STGADMIN.EDG.OWNER.ADSM*
      ID(ABARS) ACC(UPDATE) CLASS(FACILITY)
PERMIT STGADMIN.EDG.RESET
      ID(ABARS) ACC(READ) CLASS(FACILITY)

```

Authorizing EDGBKUP to DFSMSrmm resources

You use the `OPTION JOURNALFULL(xx)` command in PARMLIB member EDGRMMnn to define a percentage full threshold for the journal data set. When DFSMSrmm detects that the journal has reached this threshold, the procedure specified on the `BACKUPPROC` operand is started automatically. In this IBM Redbook, we use the name `EDGBKUP` for the backup procedure name.

The `EDGBKUP` procedure requires `READ` access to the resources shown Figure A-3 to use the backup `HOUSEKEEP` function.

STGADMIN.EDG.HOUSEKEEP STGADMIN.ADR.DUMP.CNCURRNT
--

Figure A-3 Resources to protect EDGBKUP function

This procedure also requires `UPDATE` and `ALTER` access to the data set profiles shown in Figure A-4.

RMM.HSKP.*	ALTER
RMM.PROD.MSG	UPDATE

Figure A-4 Data set profiles to protect EDGBKUP copies

Example A-8 shows a batch job to add the `EDGBKUP` procedure in the `STARTED` class, and permit it to the required RACF `FACILITY` class resources.

Example: A-8 JCL to define EDGBKUP to DFSMSrmm resources

```

//RACFBKUP JOB ,RMM,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
/* ***** */
/* Note
/* ====
/* To execute this job you need the RACF SPECIAL attribute
/* ***** */
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
/* ***** */
/* Create an entry in the STARTED procedure table for EDGBKUP */
/* ***** */
RDEFINE STARTED EDGBKUP.* -
      STDATA(USER(DFRMM) -
              GROUP(SYS1) -

```

```

PRIVELEGED(NO) -
TRACE(NO) -
TRUSTED(NO) )
/* ***** */
/* Permit EDGHSKP to STGADMIN.EDG.nnnnn resources */
/* ***** */
PERMIT STGADMIN.EDG.HOUSEKEEP -
ID(EDGBKUP) ACC(READ) CLASS(FACILITY)
PERMIT 'RMM.PROD.MSGS' GENERIC -
ID(EDGBKUP) ACC(UPDATE)
PERMIT 'RMM.HSKP.*' -
ID(EDGBKUP) ACC(ALTER)
PERMIT STGADMIN.ADR.DUMP.CNCURRNT -
ID(EDGBKUP) ACC(READ) CLASS(FACILITY)

```

Authorizing EDGXPROC to DFSMSrmm resources

Use the OPTION SCRATCHPROC command in PARMLIB member EDGRMMnn to specify the name of the procedure that you want to be started automatically to replenish scratch volumes in an automated tape library dataserer (IBM 3494 or IBM 3495). In this redbook, we use the default procedure name of EDGXPROC by specifying SCRATCHPROC(EDGXPROC) in EDGRMMnn. A sample EDGXPROC procedure is shipped with DFSMSrmm.

When a manual or automatic tape library dataserer detects a low-on-scratch condition, where more scratch volumes are needed, Operation Activity Management (OAM) issues a write-to-operator message. DFSMSrmm intercepts the message, and starts the procedure you specify with SCRATCHPROC value in PARMLIB.

The EDGXPROC procedure requires READ access to the resources as shown Figure A-5 to use the expiration processing HOUSEKEEP function.

STGADMIN.EDG.HOUSEKEEP

Figure A-5 Resources to protect EDGHSKP function

This procedure also requires UPDATE access to the data set profile shown Figure A-6.

RMM.PROD.MSG

Figure A-6 Data set profile to protect the message file

Example A-9 shows a batch job to add the EDGXPROC procedure in the STARTED class and permit it to the required RACF FACILITY class resources.

Example: A-9 JCL to define EDGXPROC to DFSMSrmm resources

```

//RACFXPRC JOB ,RMM,NOTIFY=SCHLUM,
//          MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
/* ***** */
/* Note
/* ====
/* To execute this job you need the RACF SPECIAL attribute
/* ***** */
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPT DD SYSOUT=*

```

```
//SYSTSIN DD *
/* ***** */
/* Create an entry in the dynamic STARTED procedure table for ABARS */
/* ***** */
RDEFINE STARTED EDGXPROC.* -
    STDATA(USER(DFRMM ) -
        GROUP(SYS1 ) -
        PRIVELEGED(NO) -
        TRACE(NO) -
        TRUSTED(NO) )
/* ***** */
/* Permit EDGXPROC to STGADMIN.EDG.nnnnn resources */
/* ***** */
PERMIT STGADMIN.EDG.HOUSEKEEP -
    ID(EDGXPROC) ACC(READ) CLASS(FACILITY)
PERMIT 'RMM.PROD.MSGS' GENERIC-
    ID(EDGXPROC) ACC(ALTER)
```

Authorizing EDGLABEL to DFSMSrmm resources

You can run the EDGINERS utility as an operator-started procedure, so that the operator or librarian can make requests for tape labeling and erasing. The name of the procedure can be any valid alphanumeric procedure name from one to eight characters. This book uses EDGLABEL as the procedure name.

To automatically initialize or erase tapes, all volumes must be defined in the DFSMSrmm control data set, and the requested action must be set. In the automatic processing mode, EDGINERS requests that DFSMSrmm scan its control data set for the first 10 volumes waiting to be initialized or erased. If no volumes are found, the procedure ends without an error.

Note: DFSMSrmm erases volumes using the hardware security erase feature when supported.

The EDGLABEL procedure requires UPDATE access to the resource shown Figure A-7 to use the expiration processing HOUSEKEEP function.

STGADMIN.EDG.OPERATOR

Figure A-7 Resource profile to protect the tape label function

Example A-10 shows a batch job to add the EDGLABEL procedure in the STARTED class and permit it to the required RACF FACILITY class resources.

Example: A-10 JCL to define EDGLABEL to DFSMSrmm resources

```
//RACFLBL JOB ,RMM,NOTIFY=SCHLUM,
//      MSGCLASS=H,CLASS=A,MSGLEVEL=(1,1),REGION=6M
/* ***** */
/* Note
/* ====
/* To execute this job you need the RACF SPECIAL attribute
/* ***** */
//STEP01 EXEC PGM=IKJEFT01
//SYSPRINT DD SYSOUT=*
//SYSTSPRT DD SYSOUT=*
```

```
//SYSTSIN DD *
/* ***** */
/* Create an entry in the dynamic STARTED procedure table for LABEL */
/* ***** */
RDEFINE STARTED EDGLABEL.* -
    STDATA(USER(DFRMM ) -
        GROUP(SYS1 ) -
        PRIVELEGED(NO) -
        TRACE(NO) -
        TRUSTED(NO) )
/* ***** */
/* Permit EDGLABEL to STGADMIN.EDG.nnnnn resources */
/* ***** */
PERMIT STGADMIN.EDG.OPERATOR -
    ID(EDGLABEL) ACC(READ) CLASS(FACILITY)
```

CA-Top Secret implementation

The following sections include the necessary definitions for CA-Top Secret product.

Defining user groups

Example A-11 shows the definitions of the different user groups for tape management.

Example: A-11 CA-Top Secret user groups definitions

```
/* ***** */
/* DEFINE CA-Top Secret ACCESS GROUP FOR ADMINISTRATOR */
/* ***** */
TSS CRE(EDGADMIN) DEPT(XXXX) NAME('EDG ADMIN') TYPE(PROFILE)
TSS ADD(WOODY ) PROFILE(EDGADMIN)
TSS ADD(KUEHN ) PROFILE(EDGADMIN)
TSS ADD(GOGR ) PROFILE(EDGADMIN)
TSS ADD(ETZ ) PROFILE(EDGADMIN)
TSS ADD(BENGT ) PROFILE(EDGADMIN)
TSS ADD(NELSON) PROFILE(EDGADMIN)
/* ***** */
/* DEFINE CA-Top Secret ACCESS GROUP FOR SYSTEM PROGRAMMER */
/* ***** */
TSS CRE(EDGSYSPG) DEPT(XXXX) NAME('EDG SYSPG') TYPE(PROFILE)
TSS ADD(SCHLUM ) PROFILE(EDGSYSPG)
TSS ADD(SIEGEL ) PROFILE(EDGSYSPG)
/* ***** */
/* DEFINE CA-Top Secret ACCESS GROUP FOR LIBRARIAN */
/* ***** */
TSS CRE(EDGLIB) DEPT(XXXX) NAME('EDG LIBRARIAN') TYPE(PROFILE)
TSS ADD(JOY ) PROFILE(EDGLIB)
TSS ADD(NADINE ) PROFILE(EDGLIB)
TSS ADD(ZERBINI ) PROFILE(EDGLIB)
TSS ADD(STRAUS ) PROFILE(EDGLIB)
TSS ADD(SEYFERT ) PROFILE(EDGLIB)
/* ***** */
/* DEFINE CA-Top Secret GROUP FOR INVENTORY MANAGEMENT FUNCTION */
/* ***** */
TSS CRE(EDGJOB) DEPT(XXXX) NAME('EDG JOB') TYPE(PROFILE)
TSS AD(HENICKE ) PROFILE(EDGJOB)
TSS AD(KEIL ) PROFILE(EDGJOB)
```

```

TSS AD(BRIAN ) PROFILE(EDGJOB)
/* ***** */
/*  DEFINE CA-Top Secret ACCESS GROUP FOR OPERATOR */
/* ***** */
TSS CRE(EDGOPER) DEPT(XXXX) NAME('EDG OPERATOR') TYPE(PROFILE)
TSS ADD(LOVELACE) PROFILE(EDGOPER)
TSS ADD(TAUBER ) PROFILE(EDGOPER)
TSS ADD(ZINK   ) PROFILE(EDGOPER)

```

Protecting DFSMSrmm data sets

Example A-12 shows the profile definitions for the different DFSMSrmm data sets.

Example: A-12 CA-Top Secret data sets profile definition

```

/* ***** */
/*  DEFINE GLOBAL DATA SET PROFILE */
/* ***** */
TSS ADD(XXXX) DSN(RMM.)
TSS PE(EDGADMIN) DSN(RMM.*) ACCESS(NONE)
TSS PE(EDGLIB ) DSN(RMM.*) ACCESS(NONE)
TSS PE(EDGJOB  ) DSN(RMM.*) ACCESS(NONE)
TSS PE(EDGOPER ) DSN(RMM.*) ACCESS(NONE)
TSS PE(EDGSYSPG) DSN(RMM.*) ACCESS(ALL)
/* ***** */
/*  DEFINE CONTROL DATA SET DATA SET PROFILE */
/* ***** */
TSS PE(EDGADMIN) DSN(RMM.EGZB.CDS) ACCESS(UPDATE)
TSS PE(EDGLIB ) DSN(RMM.EGZB.CDS) ACCESS(UPDATE)
TSS PE(EDGJOB  ) DSN(RMM.EGZB.CDS) ACCESS(UPDATE)
TSS PE(EDGOPER ) DSN(RMM.EGZB.CDS) ACCESS(UPDATE)
TSS PE(EDGSYSPG) DSN(RMM.EGZB.CDS) ACCESS(ALL)
/* ***** */
/*  DEFINE JOURNAL DATA SET DATA SET PROFILE */
/* ***** */
TSS PE(EDGADMIN) DSN(RMM.EGZB.JRNL) ACCESS(UPDATE)
TSS PE(EDGLIB ) DSN(RMM.EGZB.JRNL) ACCESS(UPDATE)
TSS PE(EDGJOB  ) DSN(RMM.EGZB.JRNL) ACCESS(UPDATE)
TSS PE(EDGOPER ) DSN(RMM.EGZB.JRNL) ACCESS(UPDATE)
TSS PE(EDGSYSPG) DSN(RMM.EGZB.JRNL) ACCESS(ALL)
/* ***** */
/*  DEFINE MESSAGE DATA SET DATA SET PROFILE */
/* ***** */
TSS PE(EDGADMIN) DSN(RMM.HSKP.MESSAGE.DSET) ACCESS(UPDATE)
TSS PE(EDGLIB ) DSN(RMM.HSKP.MESSAGE.DSET) ACCESS(UPDATE)
TSS PE(EDGJOB  ) DSN(RMM.HSKP.MESSAGE.DSET) ACCESS(UPDATE)
TSS PE(EDGOPER ) DSN(RMM.HSKP.MESSAGE.DSET) ACCESS(UPDATE)
TSS PE(EDGSYSPG) DSN(RMM.HSKP.MESSAGE.DSET) ACCESS(ALL)
/* ***** */
/*  DEFINE HOUSEKEEP DATA SET DATA SET PROFILE */
/* ***** */
TSS PE(EDGSYSPG) DSN(RMM.HSKP.*) ACCESS(ALL)
TSS PE(EDGLIB ) DSN(RMM.HSKP.*) ACCESS(ALL)
TSS PE(EDGJOB  ) DSN(RMM.HSKP.*) ACCESS(ALL)
TSS PE(EDGOPER ) DSN(RMM.HSKP.*) ACCESS(ALL)
/* ***** */
/*  DEFINE REPORT DATA SET DATA SET PROFILE */
/* ***** */
TSS PE(EDGSYSPG) DSN(RMM.REPORT.*) ACCESS(ALL)

```



```
TSS PE(EDGSYSPG) DSN(RMM.REPORT.*) ACCESS(ALL)
TSS PE(EDGSYSPG) DSN(RMM.REPORT.*) ACCESS(ALL)
TSS PE(EDGSYSPG) DSN(RMM.REPORT.*) ACCESS(ALL)
```

Defining DFSMSrmm resources

Example A-13 shows the definitions for the DFSMSrmm resources.

Example: A-13 CA-Top Secret DFSMSrmm resource definition

```
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.HOUSEKEEP */
/* ***** */
TSS ADD(XXXX) IBMFAC(STGADMIN) IF STGADMIN not already defined
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.IGNORE.HOUSEKEEP) ACCESS(READ) ACTION(AUDIT)
TSS PE(EDGJOB ) IBMFAC(STGADMIN.EDG.IGNORE.HOUSEKEEP) ACCESS(READ) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.IGNORE.TAPE.* */
/* ***** */
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.IGNORE.TAPE.*) ACCESS(UPDATE) ACTION(AUDIT)
TSS PE(EDGJOB ) IBMFAC(STGADMIN.EDG.IGNORE.TAPE.*) ACCESS(UPDATE) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.IGNORE.TAPE.RMM.* */
/* ***** */
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.IGNORE.TAPE.RMM.*) ACCESS(UPDATE) ACTION(AUDIT)
TSS PE(EDGJOB ) IBMFAC(STGADMIN.EDG.IGNORE.TAPE.RMM*) ACCESS(UPDATE) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.IGNORE.TAPE.NORMM.* */
/* ***** */
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.IGNORE.TAPE.NORMM.*) ACCESS(UPDATE) ACTION(AUDIT)
TSS PE(EDGJOB ) IBMFAC(STGADMIN.EDG.IGNORE.TAPE.NORMM.*) ACCESS(UPDATE) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.LABEL.* */
/* ***** */
TSS PE(EDGLIB ) IBMFAC(STGADMIN.EDG.LABEL.*) ACCESS(ALL) ACTION(AUDIT)
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.LABEL.*) ACCESS(ALL) ACTION(AUDIT)
TSS PE(EDGJOB ) IBMFAC(STGADMIN.EDG.LABEL.*) ACCESS(ALL) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.LISTCONTROL */
/* ***** */
TSS PE(EDGADMIN) IBMFAC(STGADMIN.EDG.LISTCONTROL) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.LISTCONTROL) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGLIB ) IBMFAC(STGADMIN.EDG.LISTCONTROL) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGJOB ) IBMFAC(STGADMIN.EDG.LISTCONTROL) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGOPER ) IBMFAC(STGADMIN.EDG.LISTCONTROL) ACCESS(CONTROL) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.MASTER */
/* ***** */
TSS PE(EDGADMIN) IBMFAC(STGADMIN.EDG.MASTER) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.MASTER) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGLIB ) IBMFAC(STGADMIN.EDG.MASTER) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGJOB ) IBMFAC(STGADMIN.EDG.MASTER) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGOPER ) IBMFAC(STGADMIN.EDG.MASTER) ACCESS(CONTROL) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.NOLABEL.* */
/* ***** */
TSS PE(EDGADMIN) IBMFAC(STGADMIN.EDG.NOLABEL.*) ACCESS(ALL) ACTION(AUDIT)
TSS PE(EDGLIB ) IBMFAC(STGADMIN.EDG.NOLABEL.*) ACCESS(ALL) ACTION(AUDIT)
TSS PE(EDGOPER ) IBMFAC(STGADMIN.EDG.NOLABEL.*) ACCESS(ALL) ACTION(AUDIT)
```

```

/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.OPERATOR  */
/* ***** */
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.OPERATOR) ACCESS(UPDATE) ACTION(AUDIT)
TSS PE(EDGLIB ) IBMFAC(STGADMIN.EDG.OPERATOR) ACCESS(UPDATE) ACTION(AUDIT)
TSS PE(EDGOPER ) IBMFAC(STGADMIN.EDG.OPERATOR) ACCESS(UPDATE) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.OWNER.*  */
/* ***** */
TSS PE(EDGLIB ) IBMFAC(STGADMIN.EDG.OWNER.*) ACCESS(UPDATE) ACTION(AUDIT)
TSS PE(EDGADMIN) IBMFAC(STGADMIN.EDG.OWNER.*) ACCESS(UPDATE) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.RELEASE  */
/* ***** */
TSS PE(DFHSM ) IBMFAC(STGADMIN.EDG.RELEASE) ACCESS(READ) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.RESET.SSI  */
/* ***** */
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.RESET.SSI) ACCESS(ALTER) ACTION(AUDIT)
TSS PE(EDGOPER ) IBMFAC(STGADMIN.EDG.RESET.SSI) ACCESS(ALTER) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.VRS  */
/* ***** */
TSS PE(EDGADMIN) IBMFAC(STGADMIN.EDG.VRS) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.VRS) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGLIB ) IBMFAC(STGADMIN.EDG.VRS) ACCESS(CONTROL) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.INERS.WRONGLABEL  */
/* ***** */
TSS PE(EDGOPER ) IBMFAC(STGADMIN.EDG.INERS.WRONGLABEL) ACCESS(UPDATE) ACTION(AUDIT)
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.EDG.INERS.WRONGLABEL) ACCESS(CONTROL) ACTION(AUDIT)
TSS PE(EDGLIB ) IBMFAC(STGADMIN.EDG.INERS.WRONGLABEL) ACCESS(CONTROL) ACTION(AUDIT)
/* ***** */
/*  DEFINE RESOURCE STGADMIN.ADR.DUMP.CNCURRNT  */
/* ***** */
TSS PE(EDGSYSPG) IBMFAC(STGADMIN.ADR.DUMP.CNCURRNT) ACCESS(READ) ACTION(AUDIT)
TSS PE(EDGLIB ) IBMFAC(STGADMIN.ADR.DUMP.CNCURRNT) ACCESS(READ) ACTION(AUDIT)
TSS PE(EDGJOB ) IBMFAC(STGADMIN.ADR.DUMP.CNCURRNT) ACCESS(READ) ACTION(AUDIT)

```

ACF2 implementation

The following sections include the necessary definitions for the ACF2 product.

Defining data set rules

Example A-14 shows the definition rules for the DFSMSrmm data sets.

Example: A-14 ACF2 data set rules definition

```

ADDGROUP EDGADMIN = UIDSTRING1
ADDGROUP EDGSYSPG = UIDSTRING2
ADDGROUP EDGLIB   = UIDSTRING3
ADDGROUP EDGJOB   = UIDSTRING4
ADDGROUP EDGOPER  = UIDSTRING5
/* ***** */
/*  DEFINE DATA SET RULE FOR HLQ 'RMM'  */
/* ***** */
$KEY(RMM)

```

```

EGZB.CDS UID(UIDSTRING1) E(A) R(A) W(A)
EGZB.CDS UID(UIDSTRING3) E(A) R(A) W(A)
EGZB.CDS UID(UIDSTRING4) E(A) R(A) W(A)
EGZB.CDS UID(UIDSTRING5) E(A) R(A) W(A)
EGZB.CDS UID(UIDSTRING2) E(A) R(A) W(A) A(A)
EGZB.JRNL UID(UIDSTRING1) E(A) R(A) W(A)
EGZB.JRNL UID(UIDSTRING3) E(A) R(A) W(A)
EGZB.JRNL UID(UIDSTRING4) E(A) R(A) W(A)
EGZB.JRNL UID(UIDSTRING5) E(A) R(A) W(A)
EGZB.JRNL UID(UIDSTRING2) E(A) R(A) W(A) A(A)
HSPK.MESSAGE.DSET UID(UIDSTRING1) E(A) R(A) W(A)
HSPK.MESSAGE.DSET UID(UIDSTRING3) E(A) R(A) W(A)
HSPK.MESSAGE.DSET UID(UIDSTRING4) E(A) R(A) W(A)
HSPK.MESSAGE.DSET UID(UIDSTRING5) E(A) R(A) W(A)
HSPK.MESSAGE.DSET UID(UIDSTRING2) E(A) R(A) W(A) A(A)
HSPK.* UID(UIDSTRING3) E(A) R(A) W(A) A(A)
HSPK.* UID(UIDSTRING4) E(A) R(A) W(A) A(A)
HSPK.* UID(UIDSTRING5) E(A) R(A) W(A) A(A)
HSPK.* UID(UIDSTRING2) E(A) R(A) W(A) A(A)
REPORT.* UID(UIDSTRING3) E(A) R(A) W(A) A(A)
REPORT.* UID(UIDSTRING4) E(A) R(A) W(A) A(A)
REPORT.* UID(UIDSTRING5) E(A) R(A) W(A) A(A)
REPORT.* UID(UIDSTRING2) E(A) R(A) W(A) A(A)
* UID(*) UID(UIDSTRING2) E(A) R(A) W(A) A(A)
* UID(*)

```

Where:

dsnmask

Specifies the name of the data set or a mask to which this entry applies. This parameter is required. For example, WORK.MASTER represents the name of the data set PAYROLL.WORK.MASTER. (The \$KEY or the \$PREFIX control statement specifies the high-level index PAYROLL.) You must specify the dsnmask parameter before any other parameters in a rule entry.

Read(Allow|Log|Prevent)

Specifies read access and the action A, L, or P that you want eTrust CA-ACF2 to take when the environment matches. Before this access permission applies, the actual access attempt must match the environment defined by other parameters of the access rule entry.

The letter codes are defined as:

A	Allow the access
L	Permit the access but log the event
P	Prevent the access

If not specified in the rule entry, this access permission defaults to READ(P). ACF99900 messages are not issued for data set read accesses.

Write(Allow|Log|Prevent)

Specifies write access and the action (A, L, or P) that you want eTrust CA-ACF2 to take when the environment matches. This parameter works similarly to how the READ parameter does. If not specified in the rule entry, this access permission defaults to WRITE(P).

Allocate(Allow|Log|Prevent)

Specifies allocate access and the action (A, L, or P) that you want eTrust CA-ACF2 to take when the environment matches. This parameter specifies that a user has create, delete, rename, and catalog authority to a data set. If not specified in the rule entry, this access permission defaults to ALLOC(P).

Execute(Allow|Log|Prevent)

Specifies execute access and the action (A, L, or P) that you want eTrust CA-ACF2 to take when the environment matches. This parameter works similarly to how the READ parameter does. However, its access permission is the specified value or the value of the READ parameter, whichever designates the most permissive access. (For example, if READ(P) and EXEC(L) are specified, then EXEC(L) applies.) If READ access specifies A or L and EXEC access is not specified, it will default to the same access as READ. Also, if READ access is less restrictive than EXEC access (for example R(A) E(L)) then EXEC access will be set to the same access as READ.

Defining DFSMSrmm resources

Example A-15 shows the definitions for the DFSMSrmm resources.

Example: A-15 ACF2 DFSMSrmm resources definition

```
/* ***** */
/*   DEFINE RESOURCE STGADMIN.EDG.HOUSEKEEP          */
/* ***** */
$KEY(STGADMIN.EDG.HOUSEKEEP) TYPE(FAC)
UID(UIDSTRING2) SERVICE(READ) ALLOW
UID(UIDSTRING4) SERVICE(READ) ALLOW
UID(UIDSTRING5) SERVICE(READ) ALLOW
/* ***** */
/*   DEFINE ACF2 RESOURCE STGADMIN.EDG.IGNORE.TAPE.VOLSER.* */
/* ***** */
$KEY(STGADMIN.EDG.IGNORE.TAPE.VOLSER.*) TYPE(FAC)
UID(UIDSTRING2) SERVICE(READ) ALLOW
UID(UIDSTRING4) SERVICE(READ,UPDATE) LOG
/* ***** */
/*   DEFINE ACF2 RESOURCE STGADMIN.EDG.IGNORE.TAPE.RMM.VOLSER.* */
/* ***** */
$KEY(STGADMIN.EDG.IGNORE.TAPE.RMM.VOLSER.*) TYPE(FAC)
UID(UIDSTRING2) SERVICE(READ) ALLOW
UID(UIDSTRING4) SERVICE(READ,UPDATE) LOG
/* ***** */
/*   DEFINE ACF2 RESOURCE STGADMIN.EDG.IGNORE.TAPE.NORMM.VOLSER.* */
/* ***** */
$KEY(STGADMIN.EDG.IGNORE.TAPE.NORMM.VOLSER.*) TYPE(FAC)
UID(UIDSTRING2) SERVICE(READ) ALLOW
UID(UIDSTRING4) SERVICE(READ,UPDATE) LOG
/* ***** */
/*   DEFINE ACF2 RESOURCE STGADMIN.EDG.LABEL.*        */
/* ***** */
$KEY(STGADMIN.EDG.LABEL.*) TYPE(FAC)
UID(UIDSTRING1) LOG
UID(UIDSTRING2) LOG
UID(UIDSTRING5) LOG
/* ***** */
```

```

/*  DEFINE ACF2 RESOURCE STGADMIN.EDG.LIST.CONTROL
/* ***** */
$KEY(STGADMIN.EDG.LISTCONTROL) TYPE(FAC)
UID(UIDSTRING1) SERVICE(UPDATE) LOG
UID(UIDSTRING2) SERVICE(UPDATE) LOG
UID(UIDSTRING3) SERVICE(UPDATE) LOG
UID(UIDSTRING4) SERVICE(UPDATE) LOG
UID(UIDSTRING5) SERVICE(UPDATE) LOG
/* ***** */
/*  DEFINE ACF2 RESOURCE STGADMIN.EDG.MASTER
/* ***** */
$KEY(STGADMIN.EDG.MASTER) TYPE(FAC)
UID(UIDSTRING1) SERVICE(UPDATE) LOG
UID(UIDSTRING2) SERVICE(UPDATE) LOG
UID(UIDSTRING3) SERVICE(UPDATE) LOG
UID(UIDSTRING4) SERVICE(UPDATE) LOG
UID(UIDSTRING5) SERVICE(UPDATE) LOG
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.NOLABEL.*
/* ***** */
$KEY(STGADMIN.EDG.NOLABEL.*) TYPE(FAC)
UID(UIDSTRING1) LOG
UID(UIDSTRING3) LOG
UID(UIDSTRING5) LOG
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.MASTER
/* ***** */
$KEY(STGADMIN.EDG.MASTER) TYPE(FAC)
UID(UIDSTRING2) SERVICE(READ,UPDATE) LOG
UID(UIDSTRING3) SERVICE(READ,UPDATE) LOG
UID(UIDSTRING5) SERVICE(READ,UPDATE) LOG
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.OWNER.*
/* ***** */
$KEY(STGADMIN.EDG.OWNER) TYPE(FAC)
UID(UIDSTRING1) SERVICE(READ,UPDATE) LOG
UID(UIDSTRING3) SERVICE(READ,UPDATE) LOG
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.RELEASE
/* ***** */
$KEY(STGADMIN.EDG.RELEASE) TYPE(FAC)
UID(DFHSM) SERVICE(READ) ALLOW
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.RESET.SSI
/* ***** */
$KEY(STGADMIN.EDG.RESET.SSI) TYPE(FAC)
UID(UIDSTRING2) LOG
UID(UIDSTRING5) LOG
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.VRS
/* ***** */
$KEY(STGADMIN.EDG.VRS) TYPE(FAC)
UID(UIDSTRING1) SERVICE(READ,UPDATE) LOG
UID(UIDSTRING2) SERVICE(READ,UPDATE) LOG
UID(UIDSTRING3) SERVICE(READ,UPDATE) LOG
/* ***** */
/*  DEFINE RESOURCE STGADMIN.EDG.INERS.WRONGLABEL
/* ***** */
$KEY(STGADMIN.EDG.INERS.WRONGLABEL) TYPE(FAC)
UID(UIDSTRING1) LOG

```

```
UID(UIDSTRING2) LOG
UID(UIDSTRING5) SERVICE(READ,UPDATE) LOG
/* ***** */
/*   DEFINE ACF2 RESOURCE STGADMIN.ADR.DUMP.CNCURRNT   */
/* ***** */
$KEY(STGADMIN.EDG.INERS.WRONGLABEL) TYPE(FAC)
UID(UIDSTRING1) SERVICE(READ) ALLOW
UID(UIDSTRING2) SERVICE(READ) ALLOW
UID(UIDSTRING4) SERVICE(READ) ALLOW
```



The DFSMSrmm competitive matrix and migration tools

This appendix provides a comparison of DFSMSrmm and other tape management systems. It also provides a reference to additional information regarding migration tools.

Tape management systems competitive matrix

Most of the tape management systems have similar capabilities, but usually they are implemented in a different way. Table B-1 is a competitive matrix that includes most of the possible features and compares DFSMSrmm with CA-1, TLMS, and Control-T.

No	Not available
Impl Dif	Implementation differences
Limited	Limited capability
Req	Acknowledged user requirement
?	If a “?” is added to any of the above, this indicates a response we believe is correct, but could not verify. If the “?” is shown alone, this means that we do not know whether the function is provided.

A “Yes” answer in Table B-1 does not imply that all solutions are the same. There are implementation differences between the products in several areas, which may also mean that solutions are not functionally identical.

Table B-1 DFSMSrmm competitive matrix

Function	RMM	CA-1	TLMS	Control-T
Sharing control data set				
Sharing control data set across systems - without use of shared DASD	Yes	No	No	No
Protect tape data sets from accidental destruction				
Tape must be scratch for output scratch request	Yes	Yes	Yes	Yes
Data set on tape must match inventory	Yes	Yes	Yes	Yes
Control use of bypass label processing (BLP)	Yes	Yes	Yes	Yes
Prevent re-initialization of tape with unexpired data	Yes	Yes	Yes	Yes
Prevent overwrite of unexpired data	Yes	Yes	Yes	Yes
Allow controlled overwrite of existing data sets	Yes	No	No	No
Standard protection for volumes owned by external data managers to prevent overwriting their own volumes	Yes	No	No	No

Manage tape data set retention				
Provide mechanism to identify retention requirements	Yes	Yes	Yes	Yes
Allow retention by RETPD or EXPDT or policy	Yes	Yes	Yes	Yes
Allow retention by RETPD or EXPDT AND policy	Yes	No	No	Yes

Function	RMM	CA-1	TLMS	Control-T
Support retention by...				
Date (days since last ref, days since create)	Yes	Yes	Yes	Yes
Permanent	Yes	Yes	Yes	Yes
Catalog status	Yes	Yes	Yes	Yes
Cycle	Yes	Yes	Yes	Yes
Cycles by day	Yes	Yes	Yes	Yes
Non-managed	Yes	Yes	Yes	Yes
Data set name or DSN mask (option with JOBNAME)	Yes	Yes	Yes	Yes
Pseudo GDG data set name	Yes	Limited	No	No
Reevaluation of all policies on each policy run	Yes	Imp Diff	No	No
Create scratch lists via reports	Yes	Yes	Yes	Yes
Create scratch lists online	Yes	No	No	Yes
Ability to manage vaults by slot number	Yes	Yes	Yes	Yes
Ability to manage vaults without slot number	Yes	No	No	No
EXPDT = 99365 means never scratch	Yes	Yes	Yes	Yes
Supports DFSMS management class	Yes	Yes	Yes	Yes
Management class for retention only	Yes	Yes	Yes	Yes
Management class for movement as well as retention	Yes	No	No	No
Installation-wide default and maximum retention	Yes	No	No	Yes
Policies based on full or partial/generic VOLSER	Yes	Yes	No	Yes
Dynamic update of status and other details in TCDB	Yes	No	No	No
Support for special expiration dates: 98nnn, 99nnn, 90nnn	Yes	Yes	Yes	Yes
Support for user defined special expiration dates:	Yes	No	No	No
Operating environment and support				
Multiple images via shared inventory and catalogs	Yes	Yes	Yes	Yes
Multiple images via shared inventory and unshared catalogs	Yes	Yes	No	No
Support for all MVS volume serial numbers - including special and national characters	Yes	No	No	No
Enterprise capability	Yes	Yes	Yes	Yes
Interface available for STK Silo & VSM	Yes	Yes	Yes	Yes
STK Media Type Support	Yes	Yes	Yes	Yes
STK Media Type customizable	Yes	No	No	No
Management of DFSMSHsm/TSM/OAM tapes	Yes	Yes	Yes	Yes
Requires DFSMSHsm TAPEVOLUME Exit	No	Yes	Yes	Yes

Function	RMM	CA-1	TLMS	Control-T
Built-in interface from DFSMSHsm to DFSMSrmm for recycle	Yes	No	No	No
Application programming interface (API)	Yes	Yes	No	No
High-Level Language API (Java C++)	Yes	No	No	No
API available as a Web Service	Yes	No	No	No
CIM Agent provided for Standard, Open, Enterprise Storage Interface	Yes	No	No	No
Dynamic real-time file stacking	No	Yes	Yes	Yes
Supported by data set stacking products	Yes	Yes	Yes	Yes
Automatically use Fast Tape Positioning for all system tape activity	Yes	Yes	No	No
Utility to cross-check SMSTAPE environment (TCDB and LM) with tape database	Yes	No	No	No
Utility to synchronize SMSTAPE environment from tape database	Yes	Yes	Yes	No
Pull list for batch jobs in PDS	Yes (tools)	Yes	Yes	No
Ability to execute TSO commands from operator console	Yes	Yes	Yes	No
Integration with job scheduling program (like-vendor)	Yes	Yes	Yes	Yes
Ability to interface with job restart program	Yes	Yes	Yes	Yes
Integration with ACS MC for non-system managed tape	Yes	Yes	No	Yes
Use of SMS SG for scratch pooling	Yes	No	No	No
Use of SMS MC for scratch pooling	No	Yes	Yes	No
Scratch pool VOLSER range by data set name or JOBNAME	Yes	Yes	Yes	Yes
Scratch pools include multiple ranges	Yes	Yes	Yes	Yes
Scratch pool selectable at the volume level	Yes	No	No	No
Optional auxiliary messages sent to console	Yes	Yes	Yes	Yes
Job accounting information kept in inventory	Yes	Yes	Yes	Yes
Utility for displaying tape headers on console	No	Yes	No	Yes
Optional sticky labels by JCL or online request	Yes	Yes	Yes	Yes
Powerful REXX interface via commands enabling easy customization of product capabilities	Yes	No	No	No
Support UTC/GMT dates and Times for true multi-time zone Enterprise-wide support	Yes	No	No	No
Manage off-site tape vaults				

Function	RMM	CA-1	TLMS	Control-T
Update of volume location when returned volume is used	No	Yes	No	No
Support for any number of vaults	Yes	Yes	No	Yes
Unlimited number of system-managed libraries	Yes	Limited	Limited	Yes
Up to 8-character vault/location names	Yes	No	No	Yes
Management by full or partially qualified data set name	Yes	Yes	Yes	Yes
Management of GDGs by full or partially qualified DSN	Yes	Yes	Yes	Yes
Data set name qualified by job name	Yes	Yes	Yes	Yes
Matching policy name available at the data set level	Yes	No	No	No
Unrestricted movement	Yes	No	No	Yes
Unrestricted movement among system-managed libraries	Yes	No	No	No
Ability to manage multivolume/multifile volumes as individual volumes	Yes	No	No	Yes
Ability to manage multivolume/multifile volumes as a single set of volumes	Yes	Yes	Yes	Yes
Supports movement by BOXES of volumes	Yes	No	Yes	Yes
Supports a location/library name for any place or robot library	Yes	Limited	Limited	Yes
Works with StorageTek ACS	Yes	Yes	Yes	Yes
Works with StorageTek VSM	Yes	Yes	Yes	Yes
Control data set				
Special utilities required for data set management	No	Yes	Yes	No
Contains details of all resources; data sets, volumes and policies	Yes	No	No	No
Non-intrusive backup of control data set	Yes	Yes	Yes	No
Dynamic add and delete of volumes at any time	Yes	No	No	Yes
Unlimited number of volume serial ranges	Yes	No	Yes	Yes
Dynamic add and delete of volume ranges	Yes	No	No	Yes
Option to track only first data set on volume	Yes	Yes	Yes	Yes
Dynamic addition of new fields via normal processing - no reorg/reload required	Yes	No	No	No
Identifies volume type; logical, physical, stacked	Yes	No	No	No
Installation and maintenance				
Packaged with DFSMS	Yes	No	No	No

Function	RMM	CA-1	TLMS	Control-T
Packaged with OS/390 and z/OS	Yes	Yes	Yes	No
Installation integrated with DFSMS	Yes	No	No	No
Maintenance requires operating systems hooks	No	No	No	No
Ability to run in 'shadow mode' in parallel with production tape management system	Yes	No	No	Yes
Volume functions				
Supports SL, AL, NL, BLP	Yes	Yes	Yes	Yes
RACF-control for NL and BLP	Yes	Yes	Yes	No
Support for NSL	No	Yes	Yes	Yes
Initialization of new tapes	Yes	Yes	Yes	Yes
WTOR required for initialization	No	Yes	Yes	No
Prevents re-use of volumes with permanent/track errors	Yes	No	No	No
Policy-based security erase of scratch volumes using HW function	Yes	No	No	Limited
Duplicate volume support	Yes	Yes	No	Yes
Secure erase of volume driven by RACF Erase on Scratch support	Yes	No	No	No
Duplicate volume supports input and output processing	Yes	No	No	No
Reporting				
Standard reports	Yes	Yes	Yes	Yes
Batch inquiry	Yes	Yes	Yes	Yes
Extract file	Yes	No	No	Yes
Online inquiry and update of data sets and volumes	Yes	Yes	Yes	Yes
Online inquiry and update of policies	Yes	No	No	Yes
Extensive online search capabilities	Yes	No	No	Yes
User customizable Report Generator (report writer)	Yes	Yes	Yes	Yes
Security				
Non-RACF				
Password data set	No	Yes	Yes	No
Volume access list for TAPEVOL maintenance	Yes	No	No	No
Prevents specific mount of scratch tapes	Yes	No	No	Yes
Release actions according to security classification	Yes	No	No	No
Supports the SAF interface	Yes			

Function	RMM	CA-1	TLMS	Control-T
RACF				
Supports use by DFP to control data set access	Yes	Yes	Yes	Yes
Authorize different functions by FACILITYclass	Yes	No	No	Yes
Authorize different functions using custom class	No	Yes	Yes	No
Enhances TAPEVOL with TVTOC for total security without use of exits	Yes	No	No	No
BAU Administration can be devolved and restricted at the function level	Yes	No	No	No
Authorization required for use of foreign tapes	Yes	Yes	Yes	Yes
Volser and HDR1 used to recognize foreign tapes processed with BLP	Yes	No	No	No

Migration tools

Migration tools for CA-1/TMS and CA-DYNAM/TLMS, ICF-Usercatalog, Control-T (BMC Software, Inc.) and ASG-ZARA/AutoMedia are a supported aspect of the product and actively maintained to keep them current. Although a tool was created for migration from EPIC, it is not being maintained because support for EPIC was dropped after it was purchased by Computer Associates International.

For more information about the conversion process and tools, refer to the following IBM redbooks:

- *Converting to DFSMSrmm from CA-1*, SG24-6241
- *Converting to DFSMSrmm from TLMS*, SG24-6242
- *Converting to DFSMSrmm from Control-T*, SG24-6243
- *Converting to DFSMSrmm from ZARA/AutoMedia* SG24-6275
- *Converting to DFSMSrmm from ICF-usercatalog*, SG24-6244.

Glossary

A

abend. Abnormal end of task.

ACS Automatic class section

AL. American National Standards Label.

AMODE. Addressing mode.

AMS. Access method services.

ANDVRS. An RMM ADDVRS TSO subcommand operand. See Using AND.

ANSI. American National Standards Institute.

APAR. Authorized program analysis report.

APF. Authorized program facility.

API. Application Programming Interface.

ASA. American Standards Association.

assigned date. The date that the volume is assigned to the current owner. Assigned date is not meaningful for a scratch volume.

ATL. Automated tape library.

AUL. ANSI and user header or trailed label.

automated tape library. A device consisting of robotic components, cartridge storage areas, tape subsystems, and controlling hardware and software, together with the set of tape volumes that reside in the library and can be mounted on the library tape drives. See also tape library. Contrast with manual tape library.

automatic cartridge loader. An optional feature of the 3480 Magnetic Tape Subsystem that allows reloading of multiple tape cartridges. This feature is standard in the 3490 Magnetic Tape Subsystem.

automatic recording. In DFSMSrmm, the process of recording information about a volume and the data sets on the volume in the DFSMSrmm control data set at open or close time.

availability. For a storage subsystem, the degree to which a data set or object can be accessed when requested by a user.

B

B Bytes

backup. The process of creating a copy of a data set or object to be used in case of accidental loss.

basic catalog structure. The name of the catalog structure in the integrated catalog facility environment. See also integrated catalog facility catalog.

BCS. Basic catalog structure.

bin number. The specific shelf location where a volume resides in a storage location; equivalent to a rack number in the removable media library. See also shelf location.

BLP. Bypass label processing.

BTLS. Basic Tape Library Support.

built-in storage location. One of the DFSMSrmm defined storage locations: LOCAL, DISTANT, and REMOTE.

BVIR Bulk volume information retrieval

C

cache fast write. A storage control capability in which the data is written directly to cache without using non-volatile storage. Cache fast write is useful for temporary data that is readily recreated., such as the sort work files created by DFSORT. Contrast with DASD fast write.

cartridge eject. For an IBM 3494 or IBM 3495 Tape Library Data set, the act of physically removing a tape cartridge usually under robot control, by placing it in an output station. The software logically removes the cartridge by deleting or updating the tape volume record in the tape configuration database. For a manual tape library dataser, the act of logically removing a tape cartridge from the manual tape library dataser by deleting or updating the tape volume record in the tape configuration database.

cartridge entry. For either an IBM 3494 Tape Library Dataserver, IBM 3495 Tape Library Dataserver, or a IBM Model M10 3495 Tape Library Dataserver, the process of logically adding a tape cartridge to the library by creating or updating the tape volume record in the tape configuration database. The cartridge entry process includes the assignment of the cartridge to scratch or private category in the library.erm2 definition.

Cartridge System Tape. The base tape cartridge media used with 3480 or 3490 Magnetic Tape Subsystems. Contrast with Enhanced Capacity Cartridge System Tape.

CDS. Control data set.

cell. A single cartridge location within an automated tape library dataser. See also rack number.

CIM The Common Information Model
An implementation-neutral, object-oriented schema for describing network management information. The Distributed Management Task Force (DMTF) develops and maintains CIM specifications.

circular file. A type of file that appends data until full. Then, starting at the beginning of the file, subsequent incoming data overwrites the data already there.

class. A collection of RACF-defined entities (users, groups, and resources) with similar characteristics. Classes are defined in the class descriptor table (CDT), except for the USER, GROUP, and DATASET classes.

cluster The combination of a TS7700 Virtualization Engine, 3953 L05 Library Manager, and a TS3500 Tape Library to form a virtual tape subsystem.

class authority (CLAUTH). An attribute enabling a user to define RACF profiles in a class defined in the class descriptor table. A user can have class authority to zero or more classes.

CLAUTH attribute. See class authority.

conditional access list. The portion of a resource profile that specifies the users and groups that may access the resource at a specified level when a specified condition is true. For example, with program access to data sets, the condition is that the user must be executing the program specified in the access list. Contrast with standard access list.

command A control signal that initiates an action or the start of a sequence of actions

command line. On a display screen, a display line usually at the bottom of the screen in which only commands can be entered.

Composite Library ID The library sequence number reported to a host by each attached virtual device

concurrent copy. A function to increase the accessibility of data by enabling you to make a consistent backup or copy of data concurrent with the usual application program processing.

confirmation panel. A DFSMSrmm panel that lets you tell DFSMSrmm to continue or stop a delete or release action. You specify whether or not you want to confirm delete or release requests in your dialog user options.

container. A receptacle in which one or more exported logical volumes can be stored. A stacked volume containing one or more logical volumes and residing outside a virtual tape server library is considered to be the container for those volumes.

control data set. A VSAM key-sequenced data set that contains the complete inventory of your removable media library, as well as the movement and retention policies you define. In the control data set DFSMSrmm records all changes made to the inventory, such as adding or deleting volumes.

control data set ID. A one-to-eight character identifier for the DFSMSrmm control data set used to ensure that, in a multi-system, multi-complex environment, the correct management functions are performed.

convenience input. The process of adding a small number of tape cartridges to the IBM 3494 Tape Library Dataserver and IBM 3495 Tape Library Dataserver without interrupting operations, by inserting the cartridges directly into cells in a convenience input station.

convenience input/output station. A transfer station with combined tape cartridge input and output functions in the IBM 3494 Tape Library Dataservers only.

convenience input station. A transfer station, used by the operator to add tape cartridges to the IBM 3494 Tape Library Dataserver or an IBM 3495 Tape Library Dataserver, which is accessible from outside the enclosure area.

convenience output. The process of removing a small number of tape cartridges from the IBM 3494 Tape Library Dataserver or an IBM 3495 Tape Library Dataserver without interrupting operations, by removing the cartridges directly from cells in a convenience input station.

convenience output station. A transfer station, used by the operator to remove tape cartridges from the automated tape library dataserver, which is accessible from outside the enclosure area.

conversion. In DFSMSrmm, the process of moving your removable media library inventory from another media management system to DFSMSrmm. DFSMSrmm manages the inventory and policies once you have converted it.

create date. Create date for a data set is the date that the data set is written to tape. Create date can also be the date a data set was read if it was created before DFSMSrmm is in use. Create date is updated each time a data set is replaced and not extended. Create date for volumes and other resources defined to DFSMSrmm is the date the resource is defined to DFSMSrmm or the date specified on the command as the create date.

CST. Cartridge System Tape.

D

data set profile. A profile that provides RACF protection for one or more data sets. The information in the profile can include the data set profile name, profile owner, universal access authority, access list, and other data. See discrete profile and generic profile.

DASD. Direct access storage device.

DASF fast write. An extended function of some models of the IBM 3990 Storage Control in which data is written concurrently to cache and nonvolatile storage and automatically scheduled for destaging to DASD. Both copies are retained in the storage control until the data is completely written to the DASD, providing data integrity equivalent to writing directly to the DASD. Use of DASD fast write for system-managed data sets is controlled by storage class attributes to improve performance. See also dynamic cache management. Contrast with cache fast write.

DASD volume. A DASD space identified by a common label and accessed by a set of related addresses. See also volume, primary storage, migration level 1, migration level 2.

data column. A vertical arrangement of identical data items, used on list panels to display an attribute, characteristic, or value of one or more objects.

data entry panel. A panel in which the user communicates with the system by filling in one or more fields.

DCB. Data control block.

Data Facility Sort. An IBM licensed program that is a high-speed data processing utility. DFSORT provides an efficient and flexible way to handle sorting, merging, and copying operations, as well as providing versatile data manipulation at the record, field, and bit level.

device. This term is used interchangeably with unit. You mount a tape on a unit or device, such as a 3490.

DFSMS environment. An environment that helps automate and centralize the management of storage. This is achieved through a combination of hardware, software, and policies. In the DFSMS environment for MVS, the function is provided by DFSORT, RACF, and the combination of DFSMS/MVS® and MVS.

DFSMS/MVS. An IBM System/390® licensed program that provides storage, data, and device management functions. When combined with MVS/ESA™ SP™ Version 5 it composes the base MVS/ESA operating environment. DFSMS/MVS consists of DFSMSdftp, DFSMSdss, DFSMShsm, and DFSMSrmm.

DFSMSdftp. A DFSMS/MVS functional component or base element of OS/390, that provides functions for storage management, data management, program management, device management, and distributed data access.

DFSMSdss. A DFSMS/MVS functional component or base element of OS/390, used to copy, move, dump, and restore data sets and volumes.

DFSMShsm. A DFSMS/MVS functional component or base element of OS/390, used for backing up and recovering data, and managing space on volumes in the storage hierarchy.

DFSMShsm-managed volume. (1) A primary storage volume, which is defined to DFSMShsm but which does not belong to a storage group. (2) A volume in a storage group, which is using DFSMShsm automatic dump, migration, or backup services. Contrast with system-managed volume and DFSMSrmm-managed volume.

DFSMShsm-owned volume. A storage volume on which DFSMShsm stores backup versions, dump copies, or migrated data sets.

DFSMSrmm. A DFSMS/MVS functional component or base element of OS/390, that manages removable media.

DFSMSrmm control data set. See control data set.

DFSMSrmm-managed volume. A tape volume that is defined to DFSMSrmm. Contrast with system-managed volume and DFSMShsm-managed volume.

disaster recovery. A procedure for copying and storing an installation's essential business data in a secure location, and for recovering that data in the event of a catastrophic problem. Compare with vital records.

discrete profile. A resource profile that provides RACF protection for a single resource.

DISTANT. A DFSMSrmm built-in storage location ID. See built-in storage location.

Distributed Library ID The unique library sequence number possessed by each cluster in a grid configuration.

DSNB. Data set name block.

dual copy. A high availability function made possible by nonvolatile storage in some models of the IBM 3990 Storage Control. Dual copy maintains two functionally identical copies of designated DASD volumes in the logical 3990 subsystem, and automatically updates both copies every time a write operation is issued to the dual copy logical volume.

dump class. A set of characteristics that describes how volume dumps are managed by DFSMSHsm.

duplexing. The process of writing two sets of identical records in order to create a second copy of data.

dynamic cache management. A function that automatically determines which data sets will be cached based on the 3990 subsystem load, the characteristics of the data set, and the performance requirements defined by the storage administrator.

E

EBCDIC Extended binary-coded decimal interchange code. A coded character set consisting of 8-bit coded characters.

ECCST. Enhanced Capacity Cartridge System Tape.

EDM. External data manager.

EHPCT. Extended High Performance Cartridge Tape.

EiB Exbibyte; IEC Standard is 1024 pebibytes

eject. The process used to remove a volume from a system-managed library. For an automated tape library dataserer, the volume is removed from its cell location and moved to the output station. For a manual tape library dataserer, the volume is not moved, but the tape configuration database is updated to show the volume no longer resides in the manual tape library dataserer.

Enhanced Capacity Cartridge System

Tape. Cartridge system tape with increased capacity that can only be used with 3490E Magnetic Tape Subsystems. Contrast with Cartridge System Tape.

entry panel. See data entry panel.

EOV. End of volume.

EREP. Environmental Record Editing and Printing program.

expanded output. Expanded output occurs when you specify OUTPUT=FIELDS and EXPAND=YES. For those subcommands for which expanded output applies, your application program receives more variable data than for standard output.

expiration. The process by which data sets and volumes are identified as available for reuse. In DFSMSrmm, all volumes have an expiration date or retention period set for them either by vital record specification policy, by user-specified JCL when writing a data set to the volume, or by an installation default. When a volume reaches its expiration date or retention period, it becomes eligible for release.

expiration date. The date at which a file is no longer protected against automatic deletion by the system.

expiration processing. The process of inventory management that ensures expired volumes are released and carries out required release actions on those volumes.

export. The operation to remove one or more logical volumes from a virtual tape server library. First, the list of logical volumes to export must be written on an export list volume and then, the export operation itself must be initiated.

export list volume. A virtual tape server logical volume containing the list of logical volumes to export.

exported logical volume. A logical volume that has one through the export process and now resides on a tacked volume outside a virtual tape server library.

external label. A label attached to the outside of a tape cartridge that is to be stored in an IBM 3494 Tape Library Dataserver or IBM 3495 Tape Library Dataserver. The label might contain the DFSMSrmm rack number of the tape volume.

extract data set. A data set that you use to generate reports.

F

field format. Field format is where the output consists of Structured Field Introducers and variable data rather than output in line format.

filtering. The process of selecting data sets based on specified criteria. These criteria consist of fully or partially-qualified data set names or of certain data set characteristics.

FIPS. Federal Information Processing Standard.

FMID. Function modification identifier.

FRR. Functional recovery routines.

FTP File transfer protocol.

G

GB Gigabytes

GDG. Generation data group.

GDS. Generation data set.

generation data group. A collection of data sets kept in chronological order. Each data set is a generation data set.

generation number. The number of a generation within a generation data group. A zero represents the most current generation of the group, a negative integer (-1) represents an older generation and, a positive integer (+1) represents a new generation that has not yet been cataloged.

generic profile. A resource profile that can provide RACF protection for zero or more resources. The resources protected by a generic profile have similar names and identical security requirements, though with RACFVARS, a generic profile can protect resources with dissimilar names, too. For example, a generic data set profile can protect one or more data sets.

GiB Gibibyte; IEC Standard is 1024 mebibytes

giga (G). The information-industry meaning depends upon the context:

1. G = 1,073,741,824(2^30) for real and virtual storage
2. G = 1,000,000,000 for disk storage capacity (e.g., 4Gb fixed disk)
3. G = 1,000,000,000 for transmission rates.

GMT Greenwich Mean Time.

GPR. General purpose register.

grid A series of clusters connected to one another by means of TCP/IP to form a disaster recovery solution where logical volume attributes and data are replicated across the clusters to ensure the continuation of production work

GRS. Global Resource Serialization.

guaranteed space. A storage class attribute indicating the space is to be preallocated when a data set is created. If you specify explicit volume serial numbers, SMS honors them. If space to satisfy the allocation is not available on the user-specified volumes, the allocation fails.

H

hardware configuration definition. An interactive interface in MVS that enables an installation to define hardware configurations from a single point of control.

HCD see Hardware configuration definition.

HLQ. High-level qualifier.

HPCT. High Performance Cartridge Tape.

high capacity input station. A transfer station, used by the operator to add tape cartridges to the IBM 3494 Tape Library Dataserver or IBM 3495 Tape Library Dataserver, which is inside the enclosure area.

high capacity output station. A transfer station, used by the operator to remove tape cartridges from the automated tape library dataserver, which is inside the enclosure area.

home. See home location.

home location. For DFSMSrmm, the place where DFSMSrmm normally returns a volume when the volume is no longer retained by vital records processing.

I

IBM. International Business Machines.

ICETOOL. DFSORT's multipurpose data processing and reporting utility.

ICF. Integrated catalog facility.

ICF catalog. A catalog that is composed of a basic catalog structure (BCS) and its related volume tables of contents (VTOCs) and VSAM volume data sets (VVDSSs). See also basic catalog structure and VSAM volume data set.

ID. Identifier. see identifier(ID)

identifier (ID) In programming languages, a lexical unit that names a language object; for example, the names of variables, arrays, records, labels, or procedures. An identifier usually consists of a letter optionally followed by letters, digits, or other characters.

One or more characters used to identify or name a data element and possibly to indicate certain properties of that data element.

A sequence of bits or characters that identifies a program, device, or system to another program, device, or system.

import. The operation to enter previously exported logical volumes residing on a stacked volume into a virtual tape server library. First, the list of logical volumes to import must be written on an import list volume and the stacked volumes must be entered, and then, the import operation itself must be initiated.

import list volume. A virtual tape server logical volume containing the list of logical volumes to import. This list can contain individual logical volumes to import and/or it can contain a list of stacked volumes in which all logical volumes on the stacked volume are imported.

imported logical volume. An exported logical volume that has gone through the import process and can be referenced as a tape volume within a virtual tape server library. An imported logical volume originates from a stacked volume that went through the export process.

improved data recording capability. A recording mode that can increase the effective cartridge data capacity and the effective data rate when enabled and used. IDRC is always enabled on the 3490E Magnetic Tape Subsystem.

installation defined storage location. A storage location defined using the LOCDEF command in the EDGRMMxx parmlib member.

Interactive Storage Management Facility (ISMF). The interactive interface of DFSMS/MVS that allows users and storage administrators access to the storage management functions.

Interactive System Productivity Facility (ISPF). An IBM licensed program used to develop, test, and run interactive, panel-driven dialogs.

interface A shared boundary. Examples include a hardware component to link two devices or a portion of storage or registers accessed by two or more computer programs.

in transit. A volume is in transit when it must be moved from one location to another and DFSMSrmm believes that the move has started, but has not yet received confirmation that the move is complete. For a volume moving from a system-managed library, the move starts when the volume is ejected.

internal label. The internal label for standard label tapes is recorded in the VOL1 header label, magnetically recorded on the tape media.

inventory management. The regular tasks that need to be performed to maintain the control data set. See also expiration processing, storage location management processing, and vital record processing.

IPCS. Interactive Problem Control System.

IPL. Initial program load.
The process that loads the system programs from the system auxiliary storage, checks the system hardware, and prepares the system for user operations.

ISMF. Interactive Storage Management Facility.
An Interactive System Productivity Facility (ISPF) application that provides an interactive set of space management functions for users and storage administrators.

ISPF. Interactive System Productivity Facility.

ISO. International Organization of Standardization.

ITSO. International Technical Support Organization.

IVP. Installation verification procedure.

J

JCL. Job control language.
A command language that is used to identify a job to an operating system and to describe the job's requirements.

JES2. Job entry subsystem 2.

JES3. Job entry subsystem 3.

JFCB. Job file control block.

journal. A sequential data set that contains a chronological record of changes made to the DFSMSrmm control data set. You use the journal when you need to reconstruct the DFSMSrmm control data set.

K

keyword. A predefined word that is used as an identifier.

KiB Kibibyte.

kilo (K). The information-industry meaning depends upon the context:
1. K = 1024(2¹⁰) for real and virtual storage
2. K = 1000 for disk storage capacity (e.g., 4000 KB fixed disk)
3. K = 1000 for transmission ratesTerm2 definition.

KSDS. Key-sequenced data set.

L

LCS. Library Control System.

Library Control System. The Object Access Method component that controls optical and tape library operations and maintains configuration information.

Line format. Line format is where text and variable data are formatted into lines suitable for displaying at a terminal or printing on hardcopy output.

LM Library manager.

LOCAL. A DFSMSrmm built-in storage location ID. See built-in storage location.

location name. A name given to a place for removable media that DFSMSrmm manages. A location name can be the name of a system-managed library, a storage location name, or the location SHELF, identifying shelf space outside a system-managed library or storage locations.

Logical partition A subset of a single server that contains resources (processors, memory, and input/output devices). A logical partition operates as an independent system. If hardware requirements are met, multiple logical partitions can exist within a system.

logical volume. Logical volumes have a many-to-one association with physical tape media and are used indirectly by MVS applications. They reside in a Virtual Tape Server or on exported stacked volumes. Applications can access the data on these volumes only when they reside in a Virtual Tape Server which makes the data available via its tape volume cache or after the data has been copied to a physical volume through the use of special utilities.

LPAR A function that enables the creation of logical partitions. See logical partition

low-on-scratch management. The process by which DFSMSrmm replenishes scratch volumes in a system-managed library when it detects that there are not enough available scratch volumes.

LPA. Link pack area.

LSR. Local shared resources.

M

management class. A collection of management attributes, defined by the storage administrator, used to control the release of allocated but unused space: to control the retention, migration, and backup of data sets: to control the retention and backup of aggregate groups, and to control the retention, backup, and class transition of objects. If assigned by ACS routine to system-managed tape volumes, can be used to identify a DFSMSrmm vital record specification.

management value. See vital record specification management value.

manual cartridge entry processing. The process by which a volume is added to the tape configuration database when it is added to a manual tape library dataser. DFSMSrmm can initiate this process.

manual mode. An operational mode where DFSMSrmm runs without recording volume usage or validating volumes. The DFSMSrmm TSO commands, ISPF dialog, and inventory management functions are all available in manual mode.

manual tape library. A set of tape drives defined as a logical unit by the installation together with the set of system-managed volumes which can be mounted on those drives. See also tape library. Contrast with automated tape library.

master system. The MVS system where the master DFSMSrmm control data set resides.

master volume. A private volume that contains data that is available for write processing based on the DFSMSrmm EDGRMMxx parm lib MASTEROVERWRITE operand.

media format. The type of volume, recording format and techniques used to create the data on the volume.

media library. See removable media library.

media management system. A program that helps you manage removable media. DFSMSrmm is a media management system.

media name. An up to 8 character value that describes the shape or type of removable media stored in a storage location. Examples of media name are: SQUARE, ROUND, CARTRDGE, 3480.

media type. A value that specifies the volume's media type. Media type can be specified as: *, CST, ECCST, HPCT or EHPCT.

MEDIA 1. Cartridge System Tape.

MEDIA 2. Enhanced Cartridge System Tape.

MEDIA 3. High Performance Cartridge Tape.

MEDIA 4. Extended High Performance Cartridge Tape.

mega (M). The information-industry meaning depends upon the context:

1. M = 1,048,576(2²⁰) for real and virtual storage
2. M = 1,000,000 for disk storage capacity (e.g., 4000 MB fixed disk)
3. M = 1,000,000 for transmission ratesTerm2 definition.

MiB Mebibyte.

migration. The process of moving unused data to lower cost storage in order to make space for high-availability data. If you wish to use the data set, it must be recalled. See also migration level 1 and migration level 2.

migration level 1. DFSMSHsm-owned DASD volumes that contain data sets migrated from primary storage volumes. The data can be compressed. See also storage hierarchy. Contrast with primary storage and migration level 2.

migration level 2. DFSMSHsm-owned tape or DASD volumes that contain data sets migrated from primary storage volumes or from migration level 1 volumes. The data can be compressed. See also storage hierarchy. Contrast with primary storage and migration level 1.

mount A host command to load a tape cartridge into a tape drive.
A virtual action in which a logical volume is assigned to a virtual tape drive.

MVS Multiple virtual storage.

MVS image. A single occurrence of the MVS/ESA operating system that has the ability to process work.

N

name vital record specification. A vital record specification used to define additional retention and movement policy information for data sets or volumes.

NEXTVRS. An RMM ADDVRS TSO subcommand operand. See Using NEXT.

NL. No label.

non-scratch volume. A volume that is not scratch, which means it has valid or unexpired data on it. Contrast with scratch.

NSL.. Non standard label.

O

OAM. Object Access Method.

object. A named byte stream having no specific format or record orientation.

object access method (OAM). An access method that provides storage, retrieval, and storage hierarchy management for objects and provides storage and retrieval management for tape volumes contained in system-managed libraries.

OCDS. Offline control data set.

optical disk. A disk that uses laser technology for data storage and retrieval.

optical volume. Storage space on an optical disk, identified by a volume label. See also volume.

option line. See command line.

owner. In DFSMSrmm, a person or group of persons defined as a DFSMSrmm user owning volumes. An owner is defined to DFSMSrmm through an owner ID.

owner ID. In DFSMSrmm, an identifier for DFSMSrmm users who own volumes.

P

parallel running. During conversion, when you install DFSMSrmm concurrently with an existing media management system, it is called running in parallel.

parameter A variable that is given a constant value for a specified application and that may denote the application.

partitioned data set (PDS). A data set on direct access storage that is divided into partitions, called members, each of which can contain a program, part of a program, or data.

PDS. Partitioned data set.

permanent data set. A user-named data set that is normally retained for longer than the duration of a job or interactive session. Contrast with temporary data set.

PF. Program function key.

physical stacked volume. See stacked volume.

physical volume. Physical volumes have a one-to-one association with physical tape media and are used directly by MVS applications. They may reside in an automated tape library dataserwer or be kept on shelf storage either at vault sites or within the data center where they can be mounted on stand-alone tape drives.

PiB Pebibyte.

pool. A group of shelf locations in the removable media library whose rack numbers share a common prefix. The shelf locations are logically grouped so that the volumes stored there are easier to find and use.

pooling. The process of arranging shelf locations in the removable media library into logical groups.

pool ID. The identifier for a pool. You define pool IDs in parmlib member EDGRMMxx.

pool storage group. A type of storage group that contains system-managed DASD volumes. Pool storage groups allow groups of volumes to be managed as a single entity. See also storage group.

PPT. Program properties table.

primary space allocation. Amount of space requested by a user for a data set when it is created. Contrast with secondary space allocation.

primary storage. A DASD volume available to users for data allocation. The volumes in primary storage are called primary volumes. See also storage hierarchy. Contrast with migration level 1 and migration level 2.

private tape volume. A volume assigned to specific individuals or functions.

primary vital record specification. The first retention and movement policy that DFSMSrmm matches to a data set and volume used for disaster recovery and vital record purposes. See also vital record specification and secondary vital record specification.

profile. Data that describes the significant characteristics of a user, a group of users, or one or more resources. A profile contains a base segment, and optionally, a number of other segments.

protect mode. In protect mode, DFSMSrmm validates all volume requests.

pseudo-generation data group. A collection of data sets, using the same data set name pattern, to be managed like a generation data group. The ~ masking character is used in DFSMSrmm to identify the characters in the pattern that change with each generation.

PSW. Program status word.

PTF. Program temporary fix.

pull list. A list of scratch volumes to be pulled from the library for use.

PUT. Program update tape.

R

RACF. See Resource Access Control Facility.

rack number. A six-character identifier that corresponds to a specific volume's shelf location in the installation's removable media library, and is the identifier used on the external label of the volume to identify it. The rack number identifies the pool and the external volume serial number for a volume residing in an automated tape library dataserwer. The rack number identifies the pool, the external volume serial, and shelf location number for a volume not residing in an automated tape library dataserwer. The rack number is not written by the tape drive. It exists as an entry in the DFSMSrmm control data set and on the external label of the tape. See also shelf location.

rack pool. A group of shelves that contains volumes that are generally read-only.

RDS. Retention data set.

ready to scratch. This describes the condition where a volume is eligible for scratch processing while it resides in a storage location. Since no other release actions are required, the volume can be returned to scratch directly from the storage location.

Recall When a logical volume is moved from a physical volume to the cache and becomes a virtual volume.

record-only mode. The operating mode where DFSMSrmm records information about volumes as you use them, but does not validate or reject volumes.

recording format. For a tape volume, the format of the data on the tape; for example, 18 tracks, 36 tracks, 128 tracks or 256 tracks.

recovery. The process of rebuilding data after it has been damaged or destroyed, often by using a backup copy of the data or by reapplying transactions recorded in a journal.

relative start generation. Relative generation zero is the latest generation of a tape; Relative generation -1 is the previous generation of that tape. Relative generation -2 is the generation before the previous one.

REMOTE. A DFSMSrmm built-in storage location ID. See built-in storage location.

removable media. See volume.

removable media library. The volumes that are available for immediate use, and the shelves where they could reside.

Resource Access Control Facility (RACF). An IBM licensed program that provides for access control by identifying and verifying the users to the system; authorizing access to protected resources; logging the detected unauthorized attempts to enter the system; and logging the detected accesses to protected resources.

Resource Group. A collection of structured fields that describe the attributes of a resource such as a volume.

Restructured Extended Executor (REXX)

Language. A general-purpose, high-level programming language, particularly suitable for EXEC procedures or programs for personal computing.

retention date. Retention date can be the date that a data set or volume is retained by a vital record specification or the date of the inventory management run when the data set or volume is no longer retained by a vital record specification.

retention period. The time for which DFSMSrmm retains a volume or data set before considering it for release. You can retain a data set or volume as part of disaster recovery or vital records management. You set a retention period through a vital record specification that overrides a data set's expiration date.

retention type. The types of retention for which DFSMSrmm retains a volume or data set before considering it for release. The retention types for data sets are BYDAYSCYCLE, CYCLES, DAYS, EXTRADAYS, LASTREFERENCEDAYS, UNTILEXPIRED, and WHILECATALOG. The retention types for volumes are DAYS and CYCLE.

REXX. Restructured Extended Executor Language.

RMF™. Resource Measurement Facility.

RMM. Removable Media Manager.

RMM complex (RMMplex). One or more MVS images that share a common DFSMSrmm control data set.

RMODE. Residence mode.

R/W Read/write.

S

SAF. System Authorization Facility.

scratch. The status of a tape volume that is available for general use, because the data on it is incorrect or is no longer needed. You request a scratch volume when you omit the volume serial number on a request for a tape volume mount.

Scratch allocation assistance (SAA) An extension of the device allocation assistance (DAA) function for scratch mount requests. It filters the list of clusters in a grid to return to the host a smaller list of candidate clusters specifically designated as scratch mount candidates.

scratch pool. The collection of tape volumes from which requests for scratch tapes can be satisfied. Contrast with rack pool.

scratch processing. The process for returning a volume to scratch status once it is no longer in use and has no outstanding release actions pending.

scratch tape. See scratch volume.

scratch volume. A tape volume that contains expired data only. See scratch.

SDB. Structured database.

SDSF. Spool Display and Search Facility.

secondary space allocation. Amount of additional space requested by the user for a data set when primary space is full. Contrast with primary space allocation.

secondary vital record specification. The second retention and movement policy that DFSMSrmm matches to a data set and volume used for disaster recovery and vital records purposes. See also vital record specification and primary vital record specification.

Selective device access control (SDAC) A function that allows only certain logical control units or subsystem ids within a composite library to have exclusive access to one or more VOLSER ranges of volumes for host-initiated mounts, ejects, and attributes or categories changes.

SFI. Structured field introducer.

shelf. A place for storing removable media, such as tape and optical volumes, when they are not being written to or read.

shelf location. A single space on a shelf for storage of removable media. DFSMSrmm defines a shelf location in the removable media library by a rack number, and a shelf location in a storage location by a bin number. See also rack number and bin number.

shelf-management. Is the function provided to manage the placement of volumes in individual slots in a location. Shelf-management is provided for the removable media library using rack numbers. For storage locations it is optional as defined by the LOCDEF options in parmlib and uses bin numbers.

shelf-resident volume. A volume that resides in a non-system-managed tape library.

shelf space. See shelf.

SL. Standard label.

slot. See shelf location.

SMF. System Management Facility.

SMP/E. System Modification Program Extended.

SMS. Storage management subsystem.

SMSplex System-managed storage complex. A group of storage devices managed collectively by a single operating system. The system automates storage management through classification of data sets and objects.

stacked volume. Stacked volumes have a one-to-one association with physical tape media and are used in a Virtual Tape Server to store logical volumes. Stacked volumes are not used by MVS applications but by the Virtual Tape Server and its associated utilities. They may be removed from a Virtual Tape Server to allow transportation of logical volumes to a vault or to another Virtual Tape Server.

standard access list. The portion of a resource profile that specifies the users and groups that may access the resource and the level of access granted to each.

standard label. An IBM standard tape label.

Standard output. Standard output is the amount of variable data displayed, printed or put into a REXX variable in response to a subcommand. When you specify OUTPUT=LINES or EXPAND=NO with OUTPUT=FIELDS, your application program receives standard output as opposed to expanded output.

storage administrator. A person in the data processing center who is responsible for defining, implementing, and maintaining storage management policies.

storage class. A collection of storage attributes that identify performance goals and availability requirements, defined by the storage administrator, used to select a device that can meet those goals and requirements.

storage group. A collection of storage volumes and attributes, defined by the storage administrator. The collections can be a group of DASD volumes or tape volumes, or a group of DASD volumes and optical volumes treated as a single object storage hierarchy.

storage location. A location physically separate from the removable media library where volumes are stored for disaster recovery, backup, and vital records management.

(storage) location dominance. The priority used by DFSMSrmm to decide where to move a volume within the removable media library during vital record specification processing. It covers all the locations; SHELF, storage locations, and system-managed tape libraries.

storage location management processing. The process of inventory management that assigns a shelf location to volumes that have moved as a result of vital record processing. See also vital record processing.

stripe. In DFSMS/MVS, the portion of a striped data set that resides on one volume. The records in that portion are not always logically consecutive. The system distributes records among the stripes such that the volumes can be read from or written to simultaneously to gain better performance. Whether it is striped is not apparent to the application program.

striping. A software implementation of a disk array that distributes a data set across multiple volumes to improve performance.

structured field. Output from the DFSMSrmm application programming interface consisting of a Structured Field Introducer and output data.

structured field introducer. An 8-byte entity that either introduces the beginning of a group of data or introduces output data that immediately follows the introducer.

subsystem. A special MVS task that provides services and functions to other MVS users. Requests for service are made to the subsystem through a standard MVS facility known as the subsystem interface (SSI). Standard MVS subsystems are the master subsystem and the job entry subsystems JES2 and JES3.

subsystem interface (SSI). The means by which system routines request services of the master subsystem, a job entry subsystem, or other subsystems defined to the subsystem interface.

SUL. IBM standard and user header or trailer label.

SVC. Supervisor call.

system-managed storage. Storage managed by the Storage Management Subsystem. SMS attempts to deliver required services for availability, performance, and space to applications. See also DFSMS environment.

system-managed tape library. A collection of tape volumes and tape devices, defined in the tape configuration database. A system-managed tape library can be automated or manual. See also tape library.

system-managed volume. A DASD, optical, or tape volume that belongs to a storage group. Contrast with DFSMSshm-managed volume and DFSMSrmm-managed volume.

system programmer. A programmer who plans, generates, maintains, extends, and controls the use of an operating system and applications with the aim of improving overall productivity of an installation.

T

tape configuration database (TCDB). One or more volume catalogs used to maintain records of system-managed tape libraries and tape volumes.

tape librarian. The person who manages the tape library. This person is a specialized storage administrator.

tape library. A set of equipment and facilities that support an installation's tape environment. This can include tape storage racks, a set of tape drives, and a set of related tape volumes mounted on those drives. See also system-managed tape library and automated tape library.

Tape Library Control System (TLCS). IBM program offering 5785-EAW. DFSMSrmm replaces TLCS.

Tape Library Dataserver. A hardware device that maintains the tape inventory associated with a set of tape drives. An automated tape library dataserver also manages the mounting, removal, and storage of tapes. An automated tape library that supports system-managed storage of tape volumes. IBM's automated tape library dataservers include the IBM 3494 Tape Library Dataserver and the IBM 3495 Tape Library Dataserver.

tape storage group. A type of storage group that contains system-managed private tape volumes. The tape storage group definition specifies the system-managed tape libraries that can contain tape volumes. See also storage group.

tape subsystem. A magnetic tape subsystem consisting of a controller and devices, which allows for the storage of user data on tape cartridges. Examples of tape subsystems include the IBM 3490 and 3490E Magnetic Tape Subsystems.

tape volume. A tape volume is the recording space on a single tape cartridge or reel. See also volume.

TB Terabytes.

TCDB. Tape Configuration Database.

TCP/IP (Transmission Control Protocol/Internet Protocol) is the basic communication language or protocol of the Internet. It can also be used as a communications protocol in a private network (either an intranet or an extranet). TCP/IP is a two-layer program. The higher layer, Transmission Control Protocol, manages the assembling of a message or file into smaller packets that are transmitted over the Internet and received by a TCP layer that reassembles the packets into the original message. The lower layer, Internet Protocol, handles the address part of each packet so that it gets to the right destination.

temporary data set. An uncataloged data set whose name begins with & or &&, that is normally used only for the duration of a job or interactive session. Contrast with permanent data set.

tera (T). The information-industry meaning depends upon the context:

1. T = 1,099,511,627,776(240) for real and virtual storage
2. T = 1,000,000,000,000 for disk storage capacity (e.g., 4 TB of DASD storage)
3. T = 1,000,000,000,000 for transmission rates.

TiB Tebibyte.

TMC. Tape management catalog.

TRS. Tape retention system.

TSO. Time sharing option.

U

Until expired. Allows the use of vital record specification policies for managing retention in a location as long as the volume expiration date has not been reached.

use attribute. (1) The attribute assigned to a DASD volume that controls when the volume can be used to allocate new data sets; use attributes are public, private, and storage. (2) For system-managed tape volumes, use attributes are scratch and private.

user volume. A volume assigned to a user, that can contain any data and can be rewritten as many times as the user wishes until the volume expires.

using AND. A method for linking DFSMSrmm vital record specifications to create chains of vital record specifications. DFSMSrmm applies policies in chains using AND only when all the retention criteria are true.

using NEXT. A method for linking DFSMSrmm vital record specifications to create chains of vital record specifications. DFSMSrmm applies policies in chains using NEXT one vital record at a time.

utilities Utility programs

utility programs A computer program used for general support of the processes of a computer; for instance, a diagnostic program.

V

virtual export. Mark a volume as exported by using the DFSMSrmm subcommands.

virtual input/output (VIO) storage group. A type of storage group that allocates data sets to paging storage, which simulates a DASD volume. VIO storage groups do not contain any actual DASD volumes. See also storage group.

Virtual tape drives Emulated tape drives that occur when the TS7740 Server emulates the function and operation of 3490 Enhanced Capacity (3490E) tape drives.

Virtual Tape Server (VTS). This subsystem, integrated into the Magstar® 3494 Tape Library, combines the random access and high performance characteristics of DASD with outboard hierarchical storage management and virtual tape devices and tape volumes.

Virtual volume Data storage on DDMs that shows the same characteristics to a host application as a physical tape volume and contains data written or read through a virtual tape drive.

vital record group. A set of data sets with the same name that matches to the same DFSMSrmm vital record specification.

vital record processing. The process of inventory management that determines which data sets and volumes DFSMSrmm should retain and whether a volume needs to move. These volumes and data sets have been assigned a vital record specification.

vital record specification. Policies defined to manage the retention and movement of data sets and volumes used for disaster recovery and vital records purposes.

vital record specification management value. A one-to-eight character name defined by your installation and used to assign management and retention values to tape data sets. The vital record management value can be any value you chose to create a match between a vital record specification and data sets and volumes in your installation. By matching the vital record specifications to the data set or volumes, DFSMSrmm applies the retention and movement policies you define in the vital record specifications. During inventory management VRSEL processing, DFSMSrmm selects the correct, best matching vital record specification for a tape data set or volume.

vital records. A data set or volume maintained for meeting an externally-imposed retention requirement, such as a legal requirement. Compare with disaster recovery.

VMF. Volume master file.

VMS. Volume management system.

VNL. Volume not in library.

VOLSER. Volume serial number.

volume. The storage space on DASD, tape, or optical devices, which is identified by a volume label. See also DASD volume, logical volume, optical volume, stacked volume, and tape volume.

volume catalog. See tape configuration database.

volume expiration date. The date the volume should expire based on the highest expiration date of the data sets that reside on the volume.

volume serial number (VOLSER). An identification number in a volume label that is assigned when a volume is prepared for use on the system. For standard label volumes, the volume serial number is the VOL1 label of the volume. For no label volumes, the volume serial number is the name the user assigns to the volume. In DFSMSrmm, volume serial numbers do not have to match rack numbers.

VPDD. Vault pattern description data set.

VRS. Vital record specification.

VTs. Virtual Tape Server.

VSAM volume data set (VVDS). A data set that describes the characteristics of VSAM and system-managed data sets residing on a given DASD volume; part of an integrated catalog facility catalog. See also basic catalog structure and integrated catalog facility catalog.

W

warning mode. The operating mode in which DFSMSrmm validates volumes as you use them, but issues warning messages when it discovers errors instead of rejecting volumes.

WORM Write Once, Read Many

Any storage media that allows data to be written only once per storage unit and never changed for secure archival purposes. It is allowed to add new content that does not change previously written units of storage.

write-to-operator. An optional user-coded service that allows a message to be written to the system console operator informing the operator of errors and unusual system conditions that may need to be corrected.

WTO. See write-to-operator.

Write-mount count A 16-bit sister value of WWID (World Wide Identifier) that represents the total number of times an LWORM volume is mounted and modified due to a write operation. This value is seeded to zero for a newly bound LWORM volume prior to the first write from BOT (beginning of tape).

WWID World Wide Identifier

A world-unique, 12-byte, 24-hex digit value assigned to a media cartridge that identifies a volume beyond volume serial. This identifier is also referred to as a cartridge unique identifier or (CUID).

Related publications

The publications listed in this section are considered particularly suitable for a more detailed discussion of the topics covered in this redbook.

IBM Redbooks

For information about ordering these publications, see “How to get Redbooks” on page 728.

- ▶ *Converting to Removable Media Manager: A Practical Guide*, SG24-4998
- ▶ *Converting to DFSMSrmm from CA-1*, SG24-6241
- ▶ *Converting to DFSMSrmm from TLMS*, SG24-6242
- ▶ *Converting to DFSMSrmm from Control-T*, SG24-6243
- ▶ *Converting to DFSMSrmm from ICF-usercatalog*, SG24-6244
- ▶ *Converting to DFSMSrmm from ZARA/AutoMedia*, SG24-6275
- ▶ *Merging Systems into a Sysplex*, SG24-6818
- ▶ *z/OS V1R3 and V1R5 DFSMS Technical Guide*, SG24-6979
- ▶ *z/OS V1R8 DFSMS Technical Update*, SG24-7435-00
- ▶ *DFSMS V1.10 and EAV Technical Guide*, SG24-7617-00
- ▶ *z/OS V1.11 DFSMS Release Guide*, SG24-7768-00
- ▶ *DFSMSshm Fast Replication Technical Guide*, SG24-7069
- ▶ *z/OS V1.12 DFSMS Technical Update*, SG24-7895

Other publications

These publications are also relevant as further information sources:

- ▶ *z/OS MVS Initialization and Tuning Reference*, SA22-7592
- ▶ *z/OS MVS System Management Facilities (SMF)*, SA22-7630
- ▶ *z/OS MVS System Messages, Vol. 5 (EDG-GFS)*, SA22-7635
- ▶ *z/OS SecureWay Security Server RACF Security Administrator's Guide*, SA22-7683
- ▶ *z/OS TSO/E Customization*, SA22-7783
- ▶ *z/OS DFSMSrmm Guide and Reference*, SC26-7404
- ▶ *z/OS DFSMSrmm Implementation and Customization Guide*, SC26-7405
- ▶ *z/OS DFSMSrmm Reporting*, SC26-7406

Online resources

These Web sites are also relevant as further information sources:

- ▶ z/OS information wizardry
<http://www.ibm.com/servers/eserver/zseries/zos/wizards/>
- ▶ OS/390 information wizardry
<http://www.ibm.com/servers/s390/os390/wizards/>
- ▶ DFSMSrmm symbols mapping and samples
<http://www.ibm.com/storage/dfsrt/>
- ▶ Mainstar Web site
<http://www.mainstar.com>

How to get Redbooks

You can order hardcopy Redbooks, as well as view, download, or search for Redbooks at the following Web site:

ibm.com/redbooks

You can also download additional materials (code samples or diskette/CD-ROM images) from that site.

Help from IBM

IBM Support and downloads

ibm.com/support

IBM Global Services

ibm.com/services

Index

Numerics

3420 451
3480 cartridge system tape 288, 451
3490 cartridge system tape 288, 451
3494 Automated Tape Library Dataserver 284
3495 Automated Tape Library Dataserver 284
3590 High Performance Tape Subsystem 451
3592 Model E05 13

A

ABEND 392
access method services REPRO command 244
ACF2 DFSMSrmm definitions 696
ACS processing
 SMS read-only variables 452
ACS processing pooling decisions 472
ACS routines 391, 449, 467
 EDGRMMxx OPTION subparameters 448
 EDGUX100 user exit 449
 how it works 449
 MSPOOL variable 450
 PREACS processing 451
 pre-ACS processing 450
 PREACS subparameter 449
 SMSACS subparameter 449
 VRSs in MC routine 472
action records 635
allocation 30
ANDVRS operand 399, 441
API
 See application programming interface
application programming interface 35
ATL
 See automated tape library
automated tape library

B

Basic Tape Library Support 284, 451
 LIBRARY command 285
 scratch pool 284
bin number 42
BronzePlex considerations for RMMplex 653
BTLS
 See Basic Tape Library Support

C

c11cona 635
c11conb 639
c11conc 635
c11cond 635
c11cono 636
c11conp 637

c11conr 638
c11cons 636
c11conv 640
CAS (catalog address space) 334
catalog address space (CAS) 334
catalog interface 32
catalog synchronization 66, 334
 authorization required 336
 catalog search interface 334
 disabling 342
 EDGHSKP utility 346
 enabling 336
 enabling JCL 350, 361
 journal considerations 338
 LISTCONTROL subcommand 342
 message data set 337
 OPTION CATSYSID 334
 synchronize catalogs 357
CA-Top Secret definitions 693
CATSYNCH 253
CATSYSID operand 356
CBRUXCUA exit 32, 664
CBRUXEJC exit 32, 664
CBRUXENT exit 32, 475–476, 664
CBRUXVNL exit 32, 664
CDS 12
 See control data set
CDS merge 643
 LOCDEF command 641
 PARMLIB options 641
 REJECT command 641
 VLPOOL command 641
CDS split 646
 LOCDEF command 641
 PARMLIB options 641
 REJECT command 641
 VLPOOL command 641
CDSID parameter 205
CLIENT parameter 205
client/server
 catalogs 210
 client start 210
 firewall 208
 LOCALTASKS 198
 operator command 214
 PORT 198
 SERVERNAME 198
 SERVERTASKS 198
 TCP/IP 209
client/server support 17, 67
command mode query 204
commands
 DISPLAY TCPIP 200
 HOMETEST 199
 HOSTNAME 201

- MODIFY DFRMM,Q A 214
- NSLOOKUP 203
- OMVS 200
- QUERY ACTIVE 214
- TSO NETSTAT 200
- concurrent copy 244
- control data set 24
 - action record 635
 - backup 219, 244
 - backup and recovery 25
 - bin record 639
 - contents 634
 - control record 635
 - data set record 635
 - maintaining 243
 - mend 270
 - merge 634
 - overview 28
 - owner record 636
 - product record 637
 - rack record 638
 - reorganize 248
 - split 634
 - verify 219, 268
 - volume record 640
 - VRS record 636
- control record 635
- converting to an existing CDS 652
- CSDID 12

D

- data set cycle 389
- data set retention 57
- data set VRS 389
- dataclass 13
- deactivate RACF functions 500
- DEVSUP command 493
- DEVSUPxx keywords
 - TAPEAUTHDSN 490
 - TAPEAUTHF1 490
 - TAPEAUTHRC4 491
 - TAPEAUTHRC8 491
- DEVSUPxx parmlib member 489, 494
- DFSMS 1.8
 - tape data set authorization 497
- DFSMS ACS support 421
- DFSMSdfp 30
- DFSMSdss 31
- DFSMSdss concurrent copy 687
- DFSMSshsm 31
 - consistency checking 283
 - TAPESECURITY 501
- DFSMSshsm tapes 401
- TVEXTPURGE 420
- DFSMSrmm
 - error diagnostic messages 21
 - interfaces 29
 - special character support 21
 - structure 27
 - TSO/E help 22

- DFSMSrmm client 68
- DFSMSrmm reporting
 - EDGRPTD utility 222
 - EDGRRPTE exec 222
- DFSMSrmm server 68
- disable catalog synchronization 342
- DISPLAY SMS command 460
- disposition processing 259
- duplicate data set names 657
- duplicate volsers 654, 657, 660
 - identifying 656

E

- EDGBKUP utility 246
- EDGDFHSM program interface 31
- EDGHSPK 12
- EDGHSPK utility 216, 243
 - VERIFY operand 436
- EDGINERS 287
- EDGINERS utility 254
- EDGMSGEX exit 30
- EDGRMMxx 13, 503
- EDGRMMxx member 657, 661
- EDGRMMxx PARMLIB member
 - ACS processing 469
- EDGRMMxx parmlib member 13
- EDGRPTD utility 222
- EDGRRPTE 13–14
- EDGRRPTE exec 222
- EDGSPLCS utility 275
- EDGTVEXT program interface 31
- EDGUTIL 12
- EDGUTIL program 656–659
- EDGUTIL utility 208, 253, 268, 666
- EDGUX100 exit 421, 475, 664
- EDGUX100 user exit 449
- EDGUX200 exit 475, 664
- enabling catalog synchronization 336
- enhanced multilevel security 18
- Enterprise Encrypted Format 2 (EEFMT2) 13
- Enterprise Format 1 (EFMT1) 13
- Enterprise Format 2 (EFMT2) 13
- entry 50
- exits
 - CBRUXCUA 32
 - CBRUXEJC 32
 - CBRUXENT 32
 - CBRUXVNL 32, 64
 - EDGMSGEX 30
 - EDGUX100 421
 - IGDACSXT 421
 - IGXMSGEX 30
- expiration date decisions 465
- expiration processing 477
- expiration processing HOUSEKEEP function 691

F

- fast tape positioning 32
- foreign tapes 261

foreign volumes 51

G

GDG

See generation data group

generation data group 391

global confirmation 223

GoldPlex considerations for RMMplex 653

H

high-speed locate 32

home location 42, 57, 60

HOUSEKEEP function 24

HOUSEKEEP processing 212

housekeeping 216, 219, 389

HSC/MVS 286

I

ICH408I message 491, 508

ICHBLP resource 488

ICHRIN03 675

ichrin2 675

IDCAMS REPRO 642

IEBGENER utility 148

IGDACSXT exit 421

IGXMSGEX exit 30

init 50

interfaces

BTLS 32

short-on-scratch processing 32

inventory management 216, 286

activities 25

CDS backup 219

CDS verify 219

daily applications 217

expiration processing 220

EXPROC 477

housekeeping 219

monthly applications 218

open data sets 223

permanent I/O errors 223

preparation 219

report extract 221

scheduling tasks 217

storage location management 220

trial run 221

vital records processing 219

weekly applications 218

IPL considerations 131

ISPF dialog 33

J

journal

activate 349, 360

disable 339

journal data set 28

journal percentage full threshold 690

L

LASTREFERENCEDAYS 393

LCLTCTC001I 431–432, 507

library management 44, 62

library name 38

LISTCONTROL subcommand 342

loan location 36

M

management class 391

management value 390

manual tape library

master volume 50

media types 13

MEND parameter 270

merging RMMplexes 653

message data set 337

messages

CBR3006I 457

EDG0105I 209, 339, 347, 357

EDG0122I 339

EDG0204I 339

EDG0358D 210

EDG2235E 334

EDG2236I 21, 334

EDG2237E 334

EDG6101E 213

EDG6137E 212

IDC3009I 507

IDG008I 464, 467

migration considerations 472

migration tools 707

MODIFY DFRMM command 470

movement 62

movement policies 57

DELAY 398

LOCATION 398

PRIORITY 398

STORENUMBER 398

MSPOOL variable 450

MTL

See manual tape library

multi-volume chain 148

multivolume set 381

MVS JCL DSNAME 12

MVS MODIFY command 209

N

n 652

name VRS 394

NEXTVRS operand 399

non-existent volumes 266

non-IBM libraries 286

non-system managed volumes 449

non-system-managed tape libraries 451

non-system-managed tape library 42

NOSECLEVEL 13

O

OAM

See Object Access Method

Object Access Method 32, 459

OCE

See open/close/end of volume

OPEN 393

Open MVS console commands 200

open/close/end of volume 30

OpenPegasus CIMOM 14

operating DFSMSrmm

initialize and erase volumes 254

processing labels 259

restart 209

start 208

OPTION command

PREACS 449

SMSACS 449

SYSID 265

P

PARMLIB members 72

PARMLIB members, DEVSUPxx 475

PARMLIB options

MOVEBY(SET) 60

OPTION RETAINBY 381

OPTION TVEXTPURGE 420

OPTION VRSEL(NEW) 394

OPTION VRSEL(OLD) 394

REJECT 63

VLPOOL 261, 264

VRSJOBNAME 384

partition an SMS library 475

partitioning a library 476

pending release 50

percentage full threshold 690

PlatinumPlex considerations for RMMplex 653

policy management

housekeeping 389

matching data sets 391

matching volumes 394

overview 375

retention types 395, 397

VRS modification 421

VRS parameters 395

VRS types 389

pooling types 264

pools 264

PREACS processing 451

pre-ACS processing 450

benefits 475

pre-ACS support 421, 473

protect tape volumes 238

R

RACF 31

assigning a user ID 674

authorize ABARS to DFSMSrmm resources 689

authorize DFSMSShm to DFSMSrmm resources 688

authorizing user groups 678

automatic tape security support 239

considerations for RMM 664

defining ABARS procedure names 689

defining DFSMSShm procedure names 687

DFSMSShm and DFSMSrmm considerations 241

DFSMSrmm access list 27

DFSMSrmm resource profiles 683

discrete profiles 680

discrete TAPEVOL profile 240

EDGBKUP procedure 690

EDGLABEL procedure 692

EDGXPROC procedure 691

enhanced generic naming considerations 681

FACILITY class profiles 675

multisystem considerations 688–689

protecting DFSMSDss resources 687

refresh instorage profiles 242

STARTED class 674

started procedures table 675

started task user ID 674

TAPEDSN class 238

TAPEVOL class 238

TAPEVOL profiles 240–241

TPRACF operand 240

TVTOC 240

user groups 123, 677

VLPOOL command 240

RACF option TAPEDSN 489

RACF profiles 241

RACF tape security support 239

racfscx 674

rack number 37

rack pool 263

record layouts 702

Redbooks Web site 728

Contact us xvi

REJECT 13

REJECT command 63

ANYUSE 475

release functions 1

release options

EXPIRYDATEIGNORE 398

SCRATCHIMMEDIATE 398

removable media library 36

REPORT 17 13

REPORT17 14

reports

searches and lists 26

RETAINBY 381

retention limits

COUNT 397

DELETEDATE 397

retention policies 57

retention types

BYDAYSCYCLE 396

CYCLES 395

DAYS 396

EXTRADAYS 397

LASTREFERENCEDAYS 396

- UNTILEXPIRED 396
- WHILECATALOG 396
- rmcsh3a 677–678
- rmcsh3c 680
- rmcsh3f 687
- rmcsh3g 687
- rmcsh3h 689
- rmcsh3i 690
- rmcsh3j 691
- rmcsh3k 692
- rmcsh3l 242
- RMM ADDVOLUME subcommand 471, 484
- RMM ADDVRS subcommand 391
- RMM CHANGEVOLUME subcommand 471
- RMM CHANGEVOLUME subcommand, CONTAINER 484
- RMM CIM provider 14
- RMM OPTION command, TPRACF(N) 21
- RMM SEACHVOLUME subcommand 504
- RMM SEARCHVOLUME subcommand 286
- RMM TSO command
 - from the console 16, 34
- RMMplex 67
 - action records 657
 - backing out the merge 667
 - backing up CDS and journals 665
 - bin numbers record 660
 - catalog synchronization 667
 - CDS and Journal data set sizes 663
 - CDS record descriptions 654
 - CDSID value 656
 - considerations for BronzePlex 653
 - considerations for GoldPlex 653
 - considerations for PlatinumPlex 653
 - control record 656
 - data set record 657
 - description of merge job steps 665
 - DFSMSRmm exits 664
 - duplicate data set names 657
 - duplicate volsers 654, 657, 660
 - EDGRMMxx member 657, 661
 - EDGUTIL program 656–659
 - handling duplicate CDs records 655
 - HOUSEKEEP processing 212
 - LOCDEF keyword considerations 662
 - managing volumes 210
 - merge methodology 665
 - move records 657
 - OPTIONS keyword considerations 661
 - overview 196
 - owner record 658
 - PARMLIB options 661
 - processing record types in merge job 665
 - product record 658
 - RACF considerations 211
 - rack numbers record 659
 - REJECT keyword considerations 662
 - relationship to TCDB 663
 - role of DEVSUPxx member 663
 - running the EDGUTIL utility after the merge 666

- sample backup JCL 668
- sample catalog synchronization job 671
- sample merge JCL 668
- security definitions 664
- sharing the started task JCL 663
- system-managed libraries 212
- toleration service 654
- useful documentation 671
- VLPOOL keyword considerations 662
- VRS record 657
- RMMSCR procedure 32
- rmmxml.xsd 18
- rsch3d 674

S

- SAF tape security 15, 486
- SAMPLIB members
 - EDGCVRSX 445
 - EDGJACTP 436
 - EDGJBKP1 219
 - EDGJBKP2 219
 - EDGJCMOV 223
 - EDGJDHKP 221
 - EDGJEJC 222
 - EDGJEXP 221
 - EDGJLOPC 216
 - EDGJMOVE 222
 - EDGJSCRL 221
 - EDGJSRDS 445
 - EDGJVIFY 219
 - EDGJWHKP 221
 - EDGLABEL 254
 - EDGPHKPA 219
- scratch categories 475
- scratch pooling 263, 266
- scratch volume 50
- security 27
- security classification 27
- SECURITY CONFLICT status 508
- security policies 673
- SERVER parameter 206
- SETROPTS CLASSACT(TAPEVOL) 489
- SETROPTS NOCLASSACT(TAPEVOL) 489
- SETROPTS NOTAPEDSN 489
- SETROPTS TAPEDSN 489
- SETSMS command 464
- shelf location 37
- SLSUX06 exit 289
- SLUADMIN utility 287
- SLUCONDB program 287
- SLUDRRMM routine 287
- SMA ACS support &MSPARM 473
- SMS
 - &ACCT_JOB 452
 - &ACCT_STEP 453
 - &ACSENVIR 453
 - &DSORG 453
 - &DSTYPE 453
 - &EXPDT 454
 - &FILENUM 454

- &GROUP 454
- &HLQ 454
- &JOB 454
- &LABEL 454
- &LIBNAME 454
- &LLQ 455
- &MSPARM 455
- &MSPDEST 455
- &MSPOLICY 455, 474
- &MSPOOL 455, 474
- &NQUAL 455
- &PGM 455
- &SYSNAME 456
- &SYSPLEX 456
- &UNIT 456
- &USER 456
- &XMODE 456
- read-only variables 452
- SMS ACS processing VRS definitions 472
- SMS ACS routines 448
- SMS ACS support 61
 - &ACSENVIR variable 452, 473
 - &MSDEST variable 473
 - &MSPOLICY variable 473
 - &MSPOOL variable 473
 - EDGUX100 exit 449, 472–473
 - expiration date decisions 465
 - how it works 449
 - IGDACSXT exit 473
 - implementing 456
 - management class define 465
 - MSPOLICY 61
 - MSPOOL 61
 - PREACS 449
 - pre-ACS interface 474
 - read only variables 452
 - scratch pooling 449, 472
 - SMS read-only variables 452
 - SMSACS 449
 - tape library define 457
 - tape storage group define 460
- SMS ACS support tailoring ACS routines 467
- SMS environment 457
- SMS interface
- SMS policy 452
- SMS read-only variables 452
- SMSplex relationship to RMM merge 663
- software interfaces 29
- software products 261
- spltc 634
- spltcnv 652
- spltm 643
- splts 646
- SSI
 - see Subsystem Interface
- stacked volumes 480
- stacked volumes support 480
 - container 481
 - non-VTS 484
- sticky labels 259

- STK tape libraries 286
- storage location 36
 - built-in 42
 - installation-defined 42
- storage location management 44, 62, 286
- strtpx 674
- subsystem interface 28
- system-managed tape
 - library name 38
- system-managed tape libraries 63
 - partitioning 476, 478
- system-managed tape library 38

T

- tape configuration database 38
- tape configuration database (TCDB) 273
- tape data set authorization
 - implementation 493
- tape data set security 27, 485, 491
- tape data sets record processing 51
- tape drive cartridge loader 469
- tape library define panel 458
- tape security
 - ARCCMDxx parmlib member 502
 - DEVSUPnn parmlib member 494
 - DEVSUPxx parmlib member 486
 - error messages 506
 - ICHBLP resource 489
 - implementtion 493
 - nonstandard label tapes 488
 - SAF 486
 - TAPEVOL and TAPEDSN processing 487
 - using combined functions 486
- tape security support 239
- tape volume catalog 663
- TAPEAUTHRC4(ALLOW) 486
- TAPEAUTHRC8(WARN) 486
- TAPEDSN option 27, 485, 487
- TAPEDSN RACF class 238
- TAPEVOL class 27, 485, 487
- TAPEVOL class profile 488
- TAPEVOL profile
 - TVTOC 488
- TAPEVOL profiles 503
- TAPEVOL RACF class 238
- target destination 60
- TCDB
 - See tape configuration database
- TCP/IP 35, 67
- Tivoli OPC 216
 - applications 217
 - batch loader 216
 - calendars 216
 - ejecting volumes 222
 - erasing and labeling volumes 221
 - Event-Triggered Tracking 217
 - move confirmation 223
 - operation 216
 - producing reports 222
 - scratch processing 221

- scratch reporting 221
- special resources 217
- workstations 216
- Tivoli Storage Manager 31
- TPRACF operand 240
- TPRACF OPTION 502
- TPRACF(AUTOMATIC) 502
- TPRACF(CLEANUP) 502
- TPRACF(CLEANUP) option 486
- TPRACF(PREDEFINED) 502
- TSM
 - See Tivoli Storage Manager
- TSO commands 34
- TSO RMM subcommands
 - LISTCONTROL OPTION 339
- TVEXTPURGE 420

U

- user address space 28
- user volume 50
- utilities
 - EDGHSKP 25
- UXTABLE 445

V

- validation 52
- vault 36
- VERIFY(SMSTAPE) 268
- VERIFY(VOLCAT) 268
- Virtual Tape Server 479
 - export processing 481
 - import processing 482
- virtual tape solutions 479
- vital record specification
 - defining 471
 - delete 422
 - management value 390
 - primary VRS 390
 - recommendations 444
 - secondary VRS 390
 - trial run 436
- vital record specification processing 376
- vital record specifications 57
- VLPOOL 13
- VLPOOL command 264, 504
 - SYSID 265
- VLPOOL RACF 503
- VOLCAT 38
- volume
 - entry 50
 - erase 221
 - init 50
 - initialize 221
 - life cycle 51
 - logical 49
 - management 49
 - master 50
 - pending release 32, 50
 - physical 49

- scratch 50
- shelf resident 42
- stacked 49
- status 50
- user 50
- volume retention 60
- volume serial number 394
- volume VRS 394
- VRS
 - See vital record specification
 - See vital record specifications
- VRS chain 394, 399
 - ANDVRS 399
 - NEXTVRS 399
- VRS management class 59
- VRS management value 59, 392
- VRS parameters 395
- VRS processing
 - CATRETPD 377
 - MOVEBY 380
 - RETAINBY 381
 - REUSEBIN 383
 - VRSCHANGE 383
 - VRSEL 385
 - VRSJOBNAME 384
 - VRSMIN 385
- VRS subchain 399
- VRS types 389
 - data set 389
 - name 394
 - volume 394
- VRSEL(NEW) 391
- VTs
 - See Virtual Tape Server
- VTs stacked volume support 480

W

- WHILECATALOG 392
- WORM tapes 17

Z

- z/OS V1R3 enhancements 18–19
- z/OS V1R5 enhancements 18
- z/OS V1R6 enhancements 2, 4–5, 8, 12, 14, 16–17



DFSMSrmm Primer

(1.5" spine)
1.5" <-> 1.998"
789 <-> 1051 pages



DFSMSrmm Primer

(1.0" spine)
0.875" <-> 1.498"
460 <-> 788 pages



DFSMSrmm Primer

(0.5" spine)
0.475" <-> 0.873"
250 <-> 459 pages



DFSMSrmm Primer

(0.2" spine)
0.17" <-> 0.473"
90 <-> 249 pages

(0.1" spine)

0.1" <-> 0.169"
53 <-> 89 pages



Redbooks

DFSMSrmm Primer

(2.5" spine)
2.5" <-> nnn.n"

1315<-> nnnn pages



Redbooks

DFSMSrmm Primer

(2.0" spine)
2.0" <-> 2.498"

1052 <-> 1314 pages



Redbooks®

DFSMSrmm Primer

**Learn how
DFSMSrmm manages
your tape
environment**

**Discover the latest
DFSMSrmm
enhancements**

**Find DFSMSrmm
implementation
details**

DFSMSrmm from IBM is the full function tape management system available in OS/390 and z/OS. With DFSMSrmm, you can manage all types of tape media at the shelf, volume, and data set level, simplifying the tasks of your tape librarian.

Are you a new DFSMSrmm user? Then this IBM Redbooks publication introduces you to the DFSMSrmm basic concepts and functions. You learn how to manage your tape environment by implementing the DFSMSrmm management policies.

Are you already using DFSMSrmm? In that case, this book provides the most up-to-date information about the new functions and enhancements introduced with the latest release of DFSMSrmm. You will find useful information for implementing these new functions and getting more benefits from DFSMSrmm.

Do you want to test DFSMSrmm functions? If you are using another tape management system and are thinking about converting to DFSMSrmm, you may start DFSMSrmm and run it in parallel to your current system for testing purposes.

This book is intended to be a starting point for new professionals and a handbook for using the basic DFSMSrmm functions.

INTERNATIONAL TECHNICAL SUPPORT ORGANIZATION

BUILDING TECHNICAL INFORMATION BASED ON PRACTICAL EXPERIENCE

IBM Redbooks are developed by the IBM International Technical Support Organization. Experts from IBM, Customers and Partners from around the world create timely technical information based on realistic scenarios. Specific recommendations are provided to help you implement IT solutions more effectively in your environment.

For more information:
ibm.com/redbooks